

# **A Mathematical Foundation for Computer Science**

**PRELIMINARY EDITION**

David Mix Barrington

**Kendall Hunt**  
publishing company

All chapter heading quotes are taken from *Monty Python's Flying Circus: All the Words (Volumes 1 and 2)* (New York: Pantheon Books, 1989) and *Monty Python and The Holy Grail (Book)* [*Monti Python ik den Hølie Gräilen (Bøk)*] (New York, Methuen Inc., 1979).

Excursion 1.11 uses text from *Through the Looking Glass, and What Alice Found There* by Lewis Carroll (London, McMillan, 1871) and *Fox in Socks* by Dr. Seuss (New York, Random House, 1965).

Problems 2.6.2 and 2.6.3 use text from *The Number of the Beast* by Robert A. Heinlein (New York: Fawcett, 1980).

There are many references in the text to *Gödel, Escher, Bach: An Eternal Golden Braid* by Douglas R. Hofstadter (New York: Basic Books, 1979).

Cover image of Stalker Castle, Scotland, by Frank Parolek © Shutterstock, Inc.

**Kendall Hunt**  
publishing company

[www.kendallhunt.com](http://www.kendallhunt.com)  
*Send all inquiries to:*  
4050 Westmark Drive  
Dubuque, IA 52004-1840

Copyright © 2019 by Kendall Hunt Publishing Company

ISBN 978-1-7924-0564-8

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the copyright owner.

Published in the United States of America

# PRELIMINARY EDITION CONTENTS

|                                                   |         |
|---------------------------------------------------|---------|
| Chapter 1: Sets, Propositions, and Predicates     | 1-1     |
| 1.1: Sets                                         | 1-2     |
| 1.2: Strings and String Operations                | 1-11    |
| 1.3: Excursion: What is a Proof?                  | 1-20    |
| 1.4: Propositions and Boolean Operations          | 1-24    |
| 1.5: Set Operations and Propositions About Sets   | 1-34    |
| 1.6: Truth-Table Proofs                           | 1-45    |
| 1.7: Rules for Propositional Proofs               | 1-52    |
| 1.8: Propositional Proof Strategies               | 1-59    |
| 1.9: Excursion: A Murder Mystery                  | 1-65    |
| 1.10: Predicates                                  | 1-68    |
| 1.11: Excursion: Translating Predicates           | 1-75    |
| Glossary for Chapter 1                            | 1-78    |
| <br>Chapter 2: Quantifiers and Predicate Calculus | <br>2-1 |
| 2.1: Relations                                    | 2-2     |
| 2.2: Excursion: Relational Databases              | 2-8     |
| 2.3: Quantifiers                                  | 2-10    |
| 2.4: Excursion: Translating Quantifiers           | 2-17    |
| 2.5: Operations on Languages                      | 2-19    |
| 2.6: Proofs With Quantifiers                      | 2-25    |
| 2.7: Excursion: Practicing Proofs                 | 2-32    |
| 2.8: Properties of Binary Relations               | 2-34    |
| 2.9: Functions                                    | 2-41    |
| 2.10: Partial Orders                              | 2-48    |
| 2.11: Equivalence Relations                       | 2-55    |
| Glossary for Chapter 2                            | 2-62    |

|                                                        |         |
|--------------------------------------------------------|---------|
| Chapter 3: Number Theory                               | 3-1     |
| 3.1: Divisibility and Primes                           | 3-2     |
| 3.2: Excursion: Playing With Numbers                   | 3-11    |
| 3.3: Modular Arithmetic                                | 3-14    |
| 3.4: There are Infinitely Many Primes                  | 3-23    |
| 3.5: The Chinese Remainder Theorem                     | 3-28    |
| 3.6: The Fundamental Theorem of Arithmetic             | 3-35    |
| 3.7: Excursion: Expressing Predicates in Number Theory | 3-43    |
| 3.8: The Ring of Congruence Classes                    | 3-46    |
| 3.9: Finite Fields and Modular Exponentiation          | 3-52    |
| 3.10: Excursion: Certificates of Primality             | 3-58    |
| 3.11: The RSA Cryptosystem                             | 3-61    |
| Glossary for Chapter 3                                 | 3-71    |
| <br>Chapter 4: Recursion and Proof by Induction        | <br>4-1 |
| 4.1: Recursive Definition                              | 4-2     |
| 4.2: Excursion: Recursive Algorithms                   | 4-10    |
| 4.3: Proof By Induction for Naturals                   | 4-13    |
| 4.4: Variations on Induction for Naturals              | 4-20    |
| 4.5: Excursion: Fibonacci Numbers                      | 4-27    |
| 4.6: Proving the Basic Facts of Arithmetic             | 4-30    |
| 4.7: Recursive Definition for Strings                  | 4-37    |
| 4.8: Excursion: Naturals and Strings                   | 4-45    |
| 4.9: Graphs and Paths                                  | 4-47    |
| 4.10: Trees and Lisp Lists                             | 4-56    |
| 4.11: Induction for Problem Solving                    | 4-66    |
| Glossary for Chapter 4                                 | 4-75    |

|                                            |      |
|--------------------------------------------|------|
| S.1: Solutions to Exercises from Chapter 1 | S-1  |
| S.2: Solutions to Exercises from Chapter 2 | S-16 |
| S.3: Solutions to Exercises from Chapter 3 | S-32 |
| S.4: Solutions to Exercises from Chapter 4 | S-48 |

## FULL VERSION CONTENTS

|                                                            |
|------------------------------------------------------------|
| Chapter 1: Sets, Propositions and Predicates               |
| Chapter 2: Quantifiers and Predicate Calculus              |
| Chapter 3: Number Theory                                   |
| Chapter 4: Recursion and Proof By Induction                |
| Chapter 5: Regular Expressions and Other Recursive Systems |
| Chapter 6: Fundamental Counting Problems                   |
| Chapter 7: Further Topics in Combinatorics                 |
| Chapter 8: Graphs                                          |
| Chapter 9: Trees and Searching                             |
| Chapter 10: Discrete Probability                           |
| Chapter 11: Reasoning About Uncertainty                    |
| Chapter 12: Markov Processes and Classical Games           |
| Chapter 13: Information Theory                             |
| Chapter 14: Finite-State Machines                          |
| Chapter 15: A Brief Tour of Formal Language Theory         |



## **AUTHOR'S NOTE**

- This Preliminary Edition contains the first four of the entire book's fifteen chapters, which form the first half of the text for COMPSCI 250 at UMass Amherst in Fall 2019. The final version will include all eight chapters used in 250 and seven others which could be used in COMPSCI 240.
- Each chapter has eight ordinary sections and three Excursions. In COMPSCI 250 the 50-minute lectures cover one or sometimes two ordinary sections, and Excursions are used for team problem-solving sessions in the weekly discussion sections. Each ordinary section contains ten Exercises (with solutions in the back) and ten Problems (suitable for homework assignments).

## **Grateful Thanks to:**

- First and foremost, my wife Jessica and daughter Julia.
- The many teachers who helped form me as a mathematician, including David Cox, the late Jim Mauldon, and especially Mark Kidwell at Amherst College, Adrian Mathias at Cambridge University, and Mike Sipser at M.I.T.
- Colleagues at UMass who helped form me as a teacher and COMPSCI 250 as a course, including Arny Rosenberg, Neil Immerman, Hava Siegelmann, Marius Minea, and dozens of graduate and undergraduate teaching assistants.
- My students in this and other courses.
- Emma Strubell, who made most of the diagrams in Chapters 1 and 2.
- Bev Kraus and Lenell Wyman at Kendall Hunt.





# Chapter 1: Sets, Propositions, and Predicates

*"I came here for a good argument."*

*"No you didn't, you came here for an argument."*

*"Well, an argument's not the same as contradiction."*

*"It can be."*

*"No it can't. An argument is a connected series of statements intended to establish a definite proposition."*

*"No it isn't."*

*"Yes it is. It isn't just contradiction."*

*"Look, if I argue with you I must take up a contrary position."*

*"But it isn't just saying 'No, it isn't'."*

*"Yes it is."*

*"No it isn't, argument is an intellectual process... contradiction is just the automatic gainsaying of anything the other person says."*

*"No it isn't."*

Our overall goal is to become familiar with a variety of mathematical objects, and learn to both make and prove precise statements about them. In this opening chapter, we will define some of the objects, develop the necessary language and vocabulary to make these statements, and begin to see how to prove them. More specifically, we will:

- Give definitions and examples of the most basic objects of discrete mathematics: *sets*, *strings*, and *formal languages*.
- Define *propositions* (boolean variables) and consider the *propositional calculus*, a method of making, manipulating, and proving combinations of propositions.
- Define *predicates*, functions that take arguments of some type and return propositions. Predicates can be used to model more complicated English statements within the propositional calculus.

## 1.1 Sets

### 1.1.1 The Mathematical Method

The practice of mathematics has two basic parts — the *design* of mathematical objects and the *application* of these objects to model some aspect of reality. We design an object by giving a **formal definition** — an exact statement of what can be said about the object, what operations can be performed on it, and what basic facts about it can be considered true. Once we have a definition, we can ask whether various statements about the object are true, and use the techniques of logic to *prove* the answers. If we show that a statement follows logically from the definition, then it must be true of *that object*.

But is the object we’ve defined really the one we wanted? How can we tell? In *pure mathematics*, our criterion is normally one of “mathematical beauty”. A definition is good to work with if it leads to interesting proofs and interesting relationships with the body of mathematics that has already been created, especially if it allows new attacks on previously unsolved problems. In *applied mathematics* the criterion is one of “scientific truth”. A good definition is one that accurately and usefully models some aspect of reality, and the tools of science can be used to test the accuracy.

The world of computing is an aspect of reality, of course, but it is one where our freedom to create new things seems almost unbounded. The mathematics of computing, therefore, often differs from the mathematics of physical science or engineering — we are more likely to have to create new mathematics to model something new. To be able to do this, we need practice in the *method* of pure mathematics, and we have to know more about the *specific* mathematical objects that are most likely to be used to design new objects or model existing ones.

The process of designing mathematical objects is very similar to the process of *object-oriented programming* in computer science. In object-oriented programming an *object* is a collection of data and code belonging to a particular *class* or *abstract data type*. The class consists of a definition of the *instances* of the class and the *methods* or *operations* that can be performed on those instances. An *implementation* of the class or data type is a *representation* of the instances as actual data items in a computer (such as bits, bytes, or words) and pieces of code to carry out the operations. There may be many possible implementations of a given class definition, but if two different implementations are each consistent with the definitions, we can tell that they will behave identically.

Because we are looking at areas of mathematics that are designed to talk about computing, we will need throughout the book to use examples of code, which will be in Java<sup>1</sup> or a variant we will call **pseudo-Java**. But in addition, we will be using computer science concepts from the beginning in our discussions of mathematical objects. For example, all of our variables will have *types*:

**Definition:** A **type** is the range of possible values for a variable. The following “mathematical” types will be used throughout this book:

---

<sup>1</sup>For the most part we will be writing free-standing methods that would look very similar in C or C++, and not all that different in Pascal. The most distinctive features of Java will largely be irrelevant to us, but we will benefit from using a particular fixed syntax and Java is becoming the one most likely to be familiar to the readers of this book.

- **boolean**: Value is **true** or **false**.
- **integer**: Value is any whole number, positive, negative, or zero.
- **natural**: Value is zero or any positive integer.
- **real**: Value is any real number.

**Example:** We can define other types as we like. In our examples in this section, we will use the type **novelist**, consisting of all people on Earth who have ever published a novel. This is a *subtype* of the type **person**, consisting of all people who have ever lived on Earth.

In object-oriented languages like Java, it is useful to have a data type to which *everything* (every possible object) belongs. Among other advantages, this allows you to write code that operates on generic “objects” without necessarily knowing what kind of objects they are. We’ll adopt this convention in our mathematical language as well:

**Definition:** The mathematical type **thing** includes any mathematical objects we may want to define in this book. A variable of type **thing** may take on a value of any type.

**Example:** The values **true** (a **boolean**), 17 (both a **natural** and an **integer**),  $\pi$  (a **real**), and Patrick O’Brian (a **novelist**) are all things. If  $x$  were a variable of type **thing**, it could take on any of these values.

It’s important to remember, though, that our mathematical data types are *not* the same as the data types in a real programming language, because we will be ignoring most of the issues created by representing these objects in a computer. In Java, all objects are eventually made up from eight **primitive types** that can be stored in actual words in the computer’s memory: **boolean**, four kinds of integers (**byte**, **short**, **int**, and **long**), two kinds of floating-point numbers for reals (**float** and **double**), and characters (the type **char**, for letters in the 32,768-letter Unicode alphabet). But, for example, our data type **integer** is the mathematical set of integers (a sequence that goes on *forever* in either direction), while the **int** type in Java is restricted to the integers that can be stored in 32 bits — those in the range from -2,147,438,648 to 2,147,438,647.

### 1.1.2 Set Definitions

So let us now begin the process of defining mathematical objects. Our most basic objects are *sets*, because many other objects we will see later are defined in terms of sets.

**Definition:** A **set** is any collection of **things**. By convention, we will limit the extent to which sets of **things** may themselves be considered as **things**. We will allow **sets of sets** only when the sets in the set contain objects that are all from the same subtype. Thus sets of sets of **naturals** are legal while sets of sets of **things** are not<sup>2</sup>.

---

<sup>2</sup>This definition gets us out of a potential problem called the **Russell paradox**, which would come about if we were allowed to define “the set of all sets that are not members of themselves” (see Problem 1.1.2). Later in the book

If  $A$  is a set, the things in  $A$  are called **elements** of  $A$ . The notation “ $x \in A$ ” means “ $x$  is an element of  $A$ ”. We can denote a set by listing its members, separated by commas, between braces.

It is common for all the elements of  $A$  to come from the same *type*  $T$  — note that  $T$  is a collection of **things** and is thus a set. In this case we say that  $A$  is “a set of elements of type  $T$ ” or “a set of type  $T$ ”.

**Example:** The set  $A = \{2, 3, 5\}$  is a set of naturals. The number 3 is an element of  $A$ , so the statement “ $3 \in A$ ” is true, while the statement “ $4 \in A$ ” is false. The set  $B = \{\text{Jane Austen, Chinua Achebe, Patrick O'Brian}\}$  is a set of novelists, that is, a set of type **novelist**. But we can also say that  $B$  is a set of type **person**. The set  $C = \{\text{Lady Murasaki, 3.26, George Eliot, } \pi\}$  contains some real numbers and some novelists. This is a perfectly legal set, because it is a collection of things, but again we will normally restrict ourselves to sets that have elements of an easily understandable type.

In denoting a set by a list, we don’t need to write down all the elements if we can make them clear in some other way. For example  $\{A, \dots, Z\}$  is the set of all capital letters,  $\{-128, \dots, 127\}$  is the set of all integers from  $-128$  through  $127$ , and  $\{1, 3, 5, \dots\}$  is the set of all odd naturals.

**Definition:** If  $w$  is a variable of type  $T$ , and  $S$  is a statement about a thing of type  $T$ , then  $\{w : S\}$  is the set of all things of type  $T$  that make  $S$  true. This is called **set builder notation**.

**Example:** Let  $x$  be a variable of type **integer**. Then  $\{x : x < 3\}$  is the set of all integers that are less than 3;  $-2$  is an element of this set but 3 and 5 are not. Let  $n$  be a variable of type **novelist**. Then  $\{n : n \text{ wrote in English}\}$  is the set of all novelists who wrote in English. George Eliot (who wrote in English) is a member of this set and Lady Murasaki (who wrote in Japanese) is not.

We can use set builder notation to define sets even when we don’t have a way to test whether the statement is true. The set  $\{n : n \text{ will write next year's best-selling novel}\}$  is a set of novelists, but we can’t tell now which novelist is in it. We may have a good reason to have to work with such sets — consider the set  $\{x : \text{input } x \text{ will cause my program to crash}\}$ .

**Definition:** Let  $A$  and  $B$  be sets. We say that  $A$  is a **subset** of  $B$  (written “ $A \subseteq B$ ”) if every element of  $A$  is also an element of  $B$ . We say that  $A$  and  $B$  are **equal** (written “ $A = B$ ”) if both  $A \subseteq B$  and  $B \subseteq A$  are true, that is, if every element of  $A$  is an element of  $B$  and also every element of  $B$  is an element of  $A$ . If  $A$  is a subset of  $B$  but  $A$  is not equal to  $B$ , we say that  $A$  is a **proper subset** of  $B$  and write  $A \subset B$ .

**Example:** Let  $D$  be the set  $\{\text{George Eliot, Lady Murasaki}\}$ ,  $E$  be the set  $\{\text{Chinua Achebe, Lady Murasaki, Patrick O'Brian, George Eliot}\}$ , and  $F$  be the set  $\{\text{Lady Murasaki, George Eliot}\}$ . We can see that  $D \subseteq E$  because the two elements of  $D$ , George Eliot and Lady Murasaki, are each elements of  $E$ . Similarly, we can see that  $D \subseteq F$  because both these novelists are also elements of  $F$ . But the statement  $E \subseteq D$  is *false*, because not all the elements of  $E$  are also elements of  $D$  — for example, Chinua Achebe is not in  $D$ . Similarly  $E \subseteq F$  is false. But  $F \subseteq D$  is true, because

---

we’ll look at some of the consequences of this kind of paradox for countability and computability theory. For now, though, we’ll take refuge in the fact that the sets we plan to use will only rarely be sets of sets, and then only sets of sets from some fixed type.

each of the elements of  $F$ , Lady Murasaki and George Eliot, is also an element of  $D$ . Since both  $F \subseteq D$  and  $D \subseteq F$  are true, we can say that  $F = D$ , or that  $F$  and  $D$  are *the same set*.

In this case it is clear that  $D$  and  $F$  have the same elements, just listed in a different order. But determining whether two sets are equal isn't always so easy, because it may not be clear whether two *elements* are equal. For example, let  $G$  be the set {the author of *Middlemarch*, the author of *The Tale of Genji*}. This set is equal to  $D$  or  $F$ , but to know this you would need some facts about literature.

It's also permissible to define a set by listing the same element twice, either deliberately (as in the set of integers  $\{3, 7, 7, 4\}$ , which is equal to the set  $\{3, 4, 7\}$ ) or accidentally (as in the set of novelists {Lady Murasaki, the author of *Middlemarch*, George Eliot}, which is *also* equal to  $D$ ,  $F$ , or  $G$ ). By our rule for equality of sets, listing an element more than once leads to the same set as just listing it once<sup>3</sup>.

**Definition:** A set is **empty** if it has no elements. Since any two empty sets are equal (see below), we speak of **the empty set**, denoted by the symbol  $\emptyset$ .

**Example:** If  $A$  and  $B$  are each empty sets, then  $A = B$  according to our definition. This is because, as we shall see later, a statement of the form “all  $x$ 's are. . .” is deemed to be *true* if there aren't any  $x$ 's at all. So it is true that “all elements of  $A$  are in  $B$ ”, and that “all elements of  $B$  are in  $A$ ”.

We can create an empty set with set builder notation (deliberately or not) by using a statement that is false for all elements of the type. For example, if  $x$  is of type **integer** then the set  $\{x : x + x = 3\}$  is empty because no integer added to itself is equal to 3.

**Definition:** A set is **finite** if there is some natural that measures how many distinct elements it has. This natural is called the **size** of the set. The size of the set  $A$  is written “ $|A|$ ”. We can **count** a finite set of size  $n$  by assigning one of the numbers in the set  $\{1, \dots, n\}$  to each element, so that every element gets a number and no element gets more than one number. This is a way to demonstrate that the size really is  $n$ .

A set is **infinite** if it is not finite — if we attempt to count it and assign numbers up to  $n$  to some of the elements, for any natural  $n$ , there will always be some elements remaining. In general we won't talk about the size of an infinite set<sup>4</sup>.

**Example:** The set of novelists  $D$  above has size 2, and the set  $E$  has size 4. The set of novelists {George Eliot, the author of *Middlemarch*, Marian Evans} has size 1, but this is only true because Marian Evans wrote *Middlemarch* using the pen name “George Eliot”. The set of *all* novelists is finite, because only finitely many people have ever written a novel, but it would be difficult or

---

<sup>3</sup>Later in Chapter 6 we will define *multisets*, which are like sets except that we keep track of *how many times* each element occurs.

<sup>4</sup>In Chapter 7, though, we'll try to make sense of the notion of the size of infinite sets, using among other words the terms “countable” and “uncountable” in a particular way. There we will learn that the integers are a “countable” set, as it happens, although we just said that we *can't* count them because they're infinite. This is an example of a common problem with mathematical terminology — common words are made into precisely defined terms that no longer match their common meanings.

impossible to find out its size.

The empty set has size 0. If a set is finite and not empty, its size is some *positive* integer because it has at least one element.

The set  $\{2,3,5\}$  is finite and has size 3. The set of all integers is infinite, because no integer is large enough to count how many there are. Similarly, the set of all positive integers is infinite, as is the set of all even positive integers. One way to show that a set is infinite is to describe a list of elements in the set that goes on forever and never repeats an element, such as  $\{1,2,3,4,\dots\}$  or  $\{2,4,6,\dots\}$ . No one knows whether the set  $\{x: \text{both } x \text{ and } x+2 \text{ are prime numbers}\}$  is finite or not (we will define prime numbers in Chapter 3).

**Definition:** A **set identity** is a statement about sets that is always true, for any sets.

**Example:** The statement “ $A \subseteq A$ ” is true, if  $A$  is any set at all. Why? In the rest of this chapter we will develop techniques to make formal proofs of facts like this. But for now, it’s worth looking informally at why such a statement should always be true. The first step is to look at the definition to see exactly what the statement means. It tells us that “ $X \subseteq Y$ ” means “every element of  $X$  is an element of  $Y$ ”, so we know that “ $A \subseteq A$ ” means “every element of  $A$  is an element of  $A$ ”. It’s pretty hard to argue *against* this last statement, so for now we’ll accept it as “obviously true”. (Later we will want to have some sort of criterion for what is obviously true and what isn’t.) Have we accomplished anything with this “proof”? We have shown that the statement about a new concept, “subset”, translates into an obviously true statement (about something being equal to itself) when we use the definition, so we have shown that the truth of the statement follows from the very meaning of “subset”. The identity isn’t very profound, but this informal “proof” is a good example of the overall mathematical method we will use throughout the book.

### 1.1.3 Exercises

We define the following sets of naturals:  $A = \{0, 2, 4\}$ ,  $B = \{1, 3, 5, 8\}$ ,  $C = \{2\}$ ,  $D = \{0, 5, 8\}$ , and  $E = \{x : x \text{ is even}\}$ .

E1.1.1 Indicate whether each of these statements is true or false:

- (a)  $0 \in A$
- (b)  $7 \in E$
- (c)  $C \subseteq A$
- (d)  $D \subseteq B$
- (e)  $D \subseteq E$
- (f)  $|D| = 4$
- (g)  $|C| = 1$
- (h)  $D$  and  $E$  have no common element.
- (i)  $E$  is finite.

E1.1.2 Are any elements in *exactly one* of the five sets  $A$ ,  $B$ ,  $C$ ,  $D$ , and  $E$ ? (That is, in one set and not in any of the other four.) If so, describe all examples of this.

E1.1.3 What data type, **boolean**, **natural**, **integer**, **real**, or **thing**, would best represent the following quantities?

- (a) A Java **Object**
- (b) The number of characters in a text file
- (c) Whether an error has been detected
- (d) The number of pixels that point  $x$  is to the left of point  $y$
- (e) The distance from point  $x$  to point  $y$

E1.1.4 Explain why each of the following set identities is true for any sets  $A$ ,  $B$ , and  $C$ :

- (a)  $\emptyset \subseteq A$
- (b)  $A \subseteq A$
- (c) If  $A \subseteq B$  and  $B \subseteq A$ , then  $A = B$
- (d) If  $A \subseteq B$  and  $B \subseteq C$ , then  $A \subseteq C$
- (e) If  $A \neq B$  and  $A \subseteq B$ , then it is not true that  $B \subseteq A$

E1.1.5 Identify each of the following sets as finite or infinite:

- (a) The set of real numbers between 0 and 1
- (b) The set of Java **float** numbers between 0 and 1
- (c) The set of rational numbers (fractions with integer numerator and denominator) between 0 and 1
- (d) The set of text files containing exactly ten gigabytes
- (e) The set of human beings that have ever lived on the earth

E1.1.6 Here are some sets of naturals given in set builder notation. Describe each set in English:

- (a)  $\{n : n \geq 4\}$
- (b)  $\{n : n = n\}$
- (c)  $\{n : n \neq n\}$
- (d)  $\{n : n = 4\}$

E1.1.7 Here are some sets described in English. Describe each in set builder notation:

- (a) The set of all naturals
- (b) The empty set
- (c) The set containing 3, 17, and no other naturals
- (d) The set of all naturals that are equal to their own squares

E1.1.8 Consider again the five sets of naturals  $A$ ,  $B$ ,  $C$ ,  $D$ , and  $E$  given above.

- (a) How many elements are in either  $A$ , in  $B$ , or in both?
- (b) How many elements are in both  $A$  and  $B$ ?

- (c) Repeat parts (a) and (b) for each of the other nine pairs of sets:  $A$  and  $C$ ,  $A$  and  $D$ , and so forth.

- E1.1.9 Is it ever possible for a set of novelists to be “equal” (as defined in this section) to a set of naturals? If so, describe all the ways that this might happen.
- E1.1.10 Is it possible for two sets each to be *proper* subsets of the other? Either give an example or explain why there cannot be one.

#### 1.1.4 Problems

- P1.1.1 How many elements are in each of these sets of numbers? Are any of them equal to each other? Which of them, if any, are subsets of which of the others?

- (a)  $\{3, 7 - 4, 7 + 4, 11\}$
- (b)  $\{1492, 5^2 - 4^2, \text{the number of players on a soccer team}\}$
- (c)  $\{5, 3^3 - 4^2, (11)^3, \text{the number of U.S. states}, 11, 1331\}$
- (d)  $\{1^2 - 0^2, 2^2 - 1^2, 3^2 - 2^2, 4^2 - 3^2, 5^2 - 4^2, 6^2 - 5^2\}$
- (e)  $\{1 + 10, 2 + 9, 3 + 8, \dots, 9 + 2, 10 + 1\}$

- P1.1.2 (The Russell Paradox) Suppose that the data type **thing** were defined so that every set of **thing** elements was itself another **thing**. Define the set  $R$  to be  $\{x: x \text{ is a set and } x \text{ is not a member of } x\}$ .

- (a) Explain why  $\emptyset$ ,  $\{1\}$ , and  $\{1, \{1\}\}$  are elements of  $R$ .
- (b) Explain why the set of all **thing** elements, given our assumption, is *not* a member of  $R$ .
- (c) Is  $R$  a member of  $R$ ? Explain why both a “yes” and a “no” answer to this question are impossible.

- P1.1.3 Let  $A$  be the set  $\{1, 2, 3\}$ . Give explicit descriptions (lists of elements) of each of the following *sets of sets*:

- (a)  $\{B : B \subseteq A\}$
- (b)  $\{B : B \subseteq A \text{ and } |B| \text{ is even}\}$  (Remember that 0 is an even number.)
- (c)  $\{B : B \subseteq A \text{ and } 3 \notin B\}$
- (d)  $\{B : B \subseteq A \text{ and } A \subseteq B\}$
- (e)  $\{B : B \subseteq A \text{ and } B \subsetneq A\}$

- P1.1.4 Let  $C$  be the set  $\{0, 1, \dots, 15\}$ . Let  $D$  be a subset of  $C$  and define the number  $f(D)$  as follows –  $f(D)$  is the sum, for every element  $i$  of  $D$ , of  $2^i$ . For example, if  $D$  is  $\{1, 6\}$  then  $f(D) = 2^1 + 2^6 = 66$ .

- (a) What are  $f(\emptyset)$ ,  $f(\{0, 2, 5\})$ , and  $f(C)$ ?
- (b) Is there a  $D$  such that  $f(D) = 666$ ? If so, find it.
- (c) Explain why, if  $D$  and  $E$  are any two subsets of  $C$  such that  $f(D) = f(E)$ , then  $D = E$ .



- P1.1.5 (uses Java, harder) Again let  $C$  be  $\{0, 1, \dots, 15\}$ . We will use Problem 1.1.4 to define a (real) Java class `Cset` to represent sets of elements of  $C$ . Remember that methods within a class may freely call each other.
- Explain why for any set  $D$  with  $D \subseteq C$ ,  $f(D)$  may be stored as a Java `int` variable.
  - Write the header and instance variable of a class `Cset` that will store a set  $D$  by storing the number  $f(D)$ .
  - Write a method `contains` that takes one `int` argument and returns a `boolean`, so that `d.contains(i)` returns `true` if and only if the number  $i$  is in the set  $d$ . If  $i$  is not an element of  $C$  this method should just return `false`.
  - Write a method `size` that takes no arguments and returns an `int`, so that `d.size()` is the number of elements in the set  $D$ .
  - Write a method `subset` that takes one `Cset` argument and returns a `boolean`, so that `d.subset(e)` returns `true` if and only if  $D \subseteq E$ .
- P1.1.6 Define  $E$  to be the empty set and further define  $F = \{E\}$ ,  $G = \{F\}$ ,  $H = \{E, F\}$ , and  $I = \{E, F, G, H\}$ .
- Describe each of these five sets as lists using only the symbols  $\emptyset$ ,  $\{$ , and  $\}$ .
  - What is the size of each of the five sets?
  - If  $X$  and  $Y$  are each any one of the five sets, we can form the statements “ $X \in Y$ ” and “ $X \subseteq Y$ ”. Exactly which of these fifty possible statements are true?
- P1.1.7 Let  $X$  be the set of even naturals that are greater than 2. Let  $Y$  be the set of naturals that can be written as  $y + z$  where  $y$  and  $z$  are odd naturals with  $y \neq z$ .
- Explain why  $X$  is a subset of  $Y$ .
  - Explain why  $Y$  is a subset of  $X$ .
- P1.1.8 If  $S$  is any finite set of positive naturals, let  $\text{sum}(S)$  be the natural we get by adding the elements of  $S$  together.
- There is exactly one set  $S$  of positive naturals with  $\text{sum}(S) = 0$ . Are there any other naturals  $n$  for which there is exactly one set  $S$  with  $\text{sum}(S) = n$ ? If so, list them all and argue that you have them all.
  - List all the sets  $S$  with  $\text{sum}(S) = 10$ . (Hint: There are exactly nine of them.)
  - Suppose  $S \neq T$  but  $\text{sum}(S) = \text{sum}(T)$ . Is it possible that  $S \subseteq T$  or that  $T \subseteq S$ ? Why or why not?
- P1.1.9 Let  $D$  be a set of dogs,  $R$  a set of retrievers, and  $G$  a set of golden retrievers. Assume that  $G \subseteq R$  and that  $R \subseteq D$ .
- Suppose we know that all three sets are different. What does this tell you about the existence of specific dogs?
  - Suppose we know that  $D \neq G$ , but that the three sets are *not* all different. What does this tell you about the existence of specific dogs?
- P1.1.10 If  $X$  is any set, the **power set** of  $X$  is a *set of sets*, defined in set builder notation as  $\{Y : Y \subseteq X\}$ . We write the power set of  $X$  as  $P(X)$ .

- (a) What is  $P(\emptyset)$ ?
- (b) If  $X = \{a, b, c\}$  (and the three elements are all different), what is  $P(X)$ ?
- (c) If  $X$  is *any* set of three distinct elements, how many elements are in  $P(X)$ ? Why does the answer to this *not* depend on the specific elements?

## 1.2 Strings and String Operations

### 1.2.1 String Definitions

Computers communicate with users and with one another by sending messages of some kind. Until the last few decades, at least, these messages have mostly been composed of either **bits** (boolean values) or **characters** (letters in some alphabet). Today these messages might include digitized sounds, pictures, or even video images, but ultimately each kind of message is encoded somehow as a series of bits or characters. (For example, if you click on some underlined text on a Web page, your computer looks up a URL, which is a series of characters, and sends that to the relevant Web server.)

Any programming language will need some way to model a series of characters so that users and programs can work with them. In many languages the *array* mechanism is used, so that an ordered series of characters is handled just like an ordered series of numbers. Java has a `String` data type, where a `String` represents a series of zero or more characters in the 137,439-character<sup>5</sup> Unicode alphabet. Strings can be created, read, and written, but can only be changed by creating an entire new string.

We will need our own mathematical type to model strings. Since different applications will require different alphabets, we will define a separate string type for every possible alphabet:

**Definition:** Let  $A$  be any finite set. To define strings over  $A$ , we call  $A$  an **alphabet** and its elements **letters**. A **string** over  $A$  is a sequence of zero or more letters from  $A$ , written together in order. The set  $A^*$  is defined to be the set of *all possible* strings over  $A$ . The set  $\{0, 1\}^*$ , of strings over the alphabet  $\{0, 1\}$ , is called the set of **binary strings**.

**Example:** The strings 001, 1, 111111111, and 0101110 are examples of binary strings. The strings over the alphabet  $\{a, b, \dots, z\}$  include *banana*, *qwerty*, *q*, and any other word consisting only of lower case letters. The ASCII alphabet consists of 128 letters, only some of which can be printed. (The others include line feeds and tabs, for example.) Some examples of ASCII strings are “#%HSGdfgGd\$^~”, “the cat”, and “39 Pine St.”. Note that “the cat” is *one* ASCII string, not two, because the space character between the two words is part of the string. In Java, the individual letters in strings come from the Unicode alphabet, whose 32,792 letters include the entire ASCII alphabet and representations of many other modern writing systems. In Java, a string is denoted by enclosing its letters in double-quotes, as long as they are ASCII letters, so the string *abc* would be written “abc”.

**Definition:** An **empty string** is a string with no letters. We denote the empty string by the special symbol<sup>6</sup>  $\lambda$  (the Greek letter “lambda”), understanding that  $\lambda$  is not a letter in our alphabet.

**Example:** If  $A$  is any alphabet at all, the empty string  $\lambda$  is a member of  $A^*$ , because it is a string of zero letters from  $A$  and  $A^*$  is the set of all strings of *zero or more* letters from  $A$ . So  $\lambda$  is an

<sup>5</sup>The Unicode Consortium regularly updates the standard for the alphabet and how it is represented by bit sequences. As of June 2018 the most recent standard was Unicode 11.0 with 137,439 characters.

<sup>6</sup>Other popular names for the empty string are  $\epsilon$ ,  $\epsilon$ , and 1, but we will stay with  $\lambda$  in this book.

ASCII string, a Java `String` (in which case it is written as `" "`, two double-quotes enclosing no characters between them), and a binary string. If the alphabet is the empty set  $\emptyset$ , it is impossible to have a string of *one or more* letters from  $\emptyset$  because there aren't any letters. But it is possible to have a string of *zero* letters from  $\emptyset$ , so  $\lambda$  is a member of  $\emptyset^*$ . Since  $\lambda$  is the *only* string in  $\emptyset^*$ , we know that  $\emptyset^* = \{\lambda\}$ .

**Definition:** Two sequences are defined to be **equal** if they consist of exactly the same elements in the same order. Hence two strings are **equal** if they have the same number of letters, the same first letter (if they have a first letter), the same second letter, and so on.

**Example:** Suppose our alphabet is  $\{a, b, c\}$  and consider the string  $abc$ . This string is equal to  $abc$  (itself), but is not equal to  $ab$  (which has fewer letters),  $aba$  (which has a different third letter),  $abcc$  (which has more letters), or  $cab$  (which has the same letters but in a different order). In Java, the strings `"abc"` and `"abc "` are *not* equal, because the first one has three letters and the second one has a fourth letter, a space.

It's this definition that lets us talk about “*the* empty string” instead of just “*an* empty string”, because any two empty strings must be equal. If two strings are empty, then they fit our definition of equal strings because they have the same number of letters (zero) and each corresponding letter is the same. It may be confusing to speak of “all the letters being the same” when there aren't any letters, but we will have to get used to it because this is part of the mathematical meaning of “all things”. We'll see this concept in more detail in Chapter 2 when we talk about quantifiers.

**Definition:** The **length** of a string is the number of letters in it. If  $B$  is an alphabet and  $i$  is a natural number,  $B^i$  is the set of strings in  $B^*$  whose length is exactly  $i$ . The length of a string  $u$  is written  $|u|$ .

**Example:** The string  $abc$  has length 3, so  $|abc| = 3$ . The Java string `"the cat"` has length 7, since the space in the middle is one of the letters. The empty string  $\lambda$  has length 0, so  $|\lambda| = 0$ . In fact for any alphabet  $A$ ,  $\lambda$  is the only string of length 0, so the set  $A^0$  is equal to  $\{\lambda\}$ .

Let's consider the binary alphabet  $B = \{0, 1\}$ . The set  $B^*$  is infinite, but the set of strings of any particular length is finite. There is exactly one binary string of length zero ( $\lambda$ ), so the set  $B^0$  is equal to  $\{\lambda\}$ . There are two strings of length one (0 and 1), so  $B^1$  is  $\{0, 1\}$ , the same set as  $B$ . If we consider all of the possibilities carefully, we find that  $B^2$  has four strings (00, 01, 10, and 11),  $B^3$  has eight, and  $B^4$  has sixteen. What about the strings of length  $k$ , for any number  $k$ ? You may know, or you may have heard, that there are exactly  $2^k$  of them. If you believe this, you should think about *why* you believe it, and how you might explain it to someone who didn't believe it — in Chapters 4 and 6 we'll learn formal techniques to solve counting problems like this.

We can make a single infinite list containing all the strings in  $B^*$  by listing first the element of  $B^0$ , then the elements of  $B^1$ , then those of  $B^2$ , and so on:

$$B^* = \{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, \dots\}.$$

Another way to generate this same list is to take the naturals in binary representation, starting

with 1:

1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, 1100, 1101, ...

and delete the initial 1 from each string. Can you explain why every binary string will eventually occur in the resulting list?

## 1.2.2 String Operations

We have already defined some operations that take strings as their arguments. The equality operation takes two strings and returns a boolean value, and the length operation takes a string and returns a natural. In this section we define a few more such *string operations*. In Chapter 4 we will give new definitions of these operators that will be more useful in formally proving facts about them.

**Definition:** If  $v$  is a string, the **reversal** of  $v$ , written  $v^R$ , is the string obtained by writing  $v$  backward.

**Example:** The reversal of  $abc$  is  $cba$ . The reversal of 111010 is 010111. The reversal of  $\lambda$  is  $\lambda$ , because we can write  $\lambda$  backward by writing no characters at all, that is, by writing  $\lambda$ . Any one-character string, such as  $a$ , is equal to its own reversal. Some strings, such as *hannah* and *radar*, are equal to their own reversals — these strings are called *palindromes*.

**Definition:** If  $v$  and  $w$  are strings, the **concatenation** of  $v$  and  $w$ , written  $vw$ , is the string obtained by first writing  $v$  and then writing  $w$ .

**Example:** Let  $v$  be the string 01 and let  $w$  be the string 110. Then  $vw$  is the string 01110 and  $wv$  is the string 11001. (So the order matters when we concatenate two strings, even though the notation looks like multiplication, where the order doesn't matter.) Using the reversal operation, we can also calculate  $v^R = 10$ ,  $w^R = 011$ , and  $v^R w^R = 10011$ . Note that  $v^R w^R$  is equal to  $(vw)^R$ , rather than to  $(vw)^R$ , because 10011 is the reversal of 11001. By convention, we execute reversals before concatenations unless there are parentheses to indicate otherwise.

Java uses the symbol  $+$  to denote concatenation of strings so that, for example, `"abc" + "def"` would denote the same string as `"abcdef"`. A common use of this notation is in output statements, such as

```
System.out.println("The answer is " + x);
```

where  $x$  can be a variable of any type. To execute this statement, Java takes the quoted string, converts the value of  $x$  to a string<sup>7</sup>, concatenates these two strings, and writes the result to the output device. Note that the output will have a space between the word *is* and the start of the value of  $x$ , because there is such a space in the quoted string.

---

<sup>7</sup>Every Java object that might ever need to be output must have a method `toString` to do this conversion.

**Definition:** A string  $w$  is a **substring** of a string  $v$  if  $w$  consists of zero or more letters that occur consecutively in  $v$ , in the same order as they occur in  $w$ . A substring that starts at the beginning of  $v$  is a **prefix**, and a substring that finishes at the end of  $v$  is called a **suffix**.

**Example:** Let  $v$  be the string *banana*. The strings *a*, *ana*, and *anana* are both substrings and suffixes of  $v$ . The strings *b*, *ba*, *ban*, and *bana* are all both substrings and prefixes of  $v$ . The strings *nan* and *anan* are each substrings of  $v$  but neither is a prefix or a suffix.

The string *ann* is *not* a substring of  $v$ . Even though these three letters occur in  $v$  in the same order, they do not occur *consecutively*. The string *nab* is also not a substring, because these three letters do not occur in  $v$  *in that order*.

The string  $\lambda$  is also a substring of  $v$ , and is a prefix and suffix of  $v$  as well (because “all the letters” of  $\lambda$  occur together in order at the beginning and at the end of  $v$ ). The string  $v$  is a substring, prefix, and suffix of itself.

### 1.2.3 Formal Languages and Decision Problems

Computers are used to perform a variety of data-processing tasks. Given some input, which can generally be thought of as a string over some alphabet, the computer is asked to produce some output, which might or might not be a string. In many cases, the output is a **boolean** value — the computer is being asked whether the input was “all right” in some sense. For example, a credit-card company’s computer is constantly being asked whether some 16-digit number is the number of a legitimate cardholder. A login procedure must decide whether its input is the correct password for the given user.

Even when the output of a program is more complicated than just a **boolean**, the program may have to answer some yes-or-no questions about the input on the way. A spell-checker must decide whether each word in the input document is in its word list before it indicates which ones are not. A compiler must decide whether its input follows the syntax rules for its language before it either creates object code or gives its error messages.

So yes-or-no questions about the input are important<sup>8</sup>. We now define one of the most important data types in computer science, to allow us to model these questions:

**Definition:** Let  $A$  be an alphabet. A **formal language** over  $A$  is any set of strings over  $A$ , that is, any subset of the set of all strings,  $A^*$ . If  $L$  is a formal language over  $A$ , the **decision problem** for  $L$  is to input a string  $w$  over  $A$  and output a **boolean** telling whether  $w$  is a member of  $L$ . An algorithm that solves the decision problem for  $L$  is called a **decision procedure** for  $L$ .

**Example:** Let  $D$  be the alphabet  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ . The set  $D^*$  of all strings over  $D$  is a language. So is the set  $NZ$  of all nonempty strings in  $D^*$  that do not begin with 0. The strings

---

<sup>8</sup>There is even a sense in which *any* output can be thought of as the answers to a series of yes-or-no questions, if it is coded as a string of bits. We could ask “Is the first bit of the output a 1?”, “Is the second bit a 1”, and so forth until we have defined the entire output string.

in  $NZ$  are just the usual decimal representations of the positive numbers, so we can write  $NZ$  as  $\{1, 2, 3, \dots\}$ . Any set of strings over  $D$ , like  $\{046, 321334, 23, 16\}$ , is a language over  $D$ . And for any set  $X$  of positive numbers, there is a language  $\{w : w \text{ is the decimal representation of a number in } X\}$ , which we might also call  $X$ . For example, let  $E$  be the set of all strings denoting positive even numbers, so  $E = \{2, 4, 6, 8, 10, \dots\}$ . Note that  $E$  is a subset of  $NZ$ . Although we could interpret the string 002 as denoting the even number 2, it is not *the* decimal representation of an even number and so it is not in  $E$ .

**Formal language theory** is the branch of mathematics where we look at languages and study how difficult their decision problems might be. In particular, we often consider whether a decision problem can be solved *in a particular way*. Because decision problems occur so often, and because so many other data processing problems can be rephrased as decision problems, formal language theory forms the basis of *computational complexity theory*, the study of what resources are required to solve particular computational problems.

Historically, much of the development of formal language theory (as well as its name) came from the problem of exactly *specifying* a language, so that even a machine could solve its decision problem. Linguists had long been formulating rules to describe the behavior of natural languages (such as English or Chinese), but these rules are so complex and contain so many exceptions that machine translation is still considered very difficult. But the “languages” used for programming computers *could* be designed so that machines could work with them. Consider some of the decisions a compiler makes when it reads your input program. Does your string `finalValue` meet Java’s rules for an identifier? Does every `{` symbol in your program have a matching `}` symbol?

Throughout this book we’ll consider strings, languages, and decision problems as motivating examples for our mathematical techniques. At the end of the book, we’ll look explicitly at the difficulty of decision problems. In Chapter 14, for example, we’ll determine exactly which languages have decision problems that can be solved using only a fixed amount of memory. And in Chapter 15 we’ll take a brief look at the rest of computational complexity theory. Some sample questions will be: Which decision problems can be solved with stacks? Which can be solved by any method? (We’ll prove some decision problems to be unsolvable.) And finally, what can we say about how the time and memory needed to solve a decision problem grow as we consider larger and larger input strings?

## 1.2.4 Exercises

E1.2.1 Determine whether each of the following statements is true or false. Here  $u$ ,  $v$ , and  $w$  are string variables representing the strings  $cba$ ,  $c$ , and  $ab$  respectively:

- (a)  $vw^R = u$ .
- (b)  $(vw)^R = u$ .
- (c)  $|u| = 3$ .
- (d)  $|u| = |v| + |w|$ .
- (e)  $u = c^R$ .
- (f)  $w$  is a suffix of  $u$ .

- (g)  $w$  is a prefix of  $u^R$ .
- (h)  $v$  is a substring of  $u$ .
- (i)  $w$  is a substring of  $u$ .

E1.2.2 Give the lengths of each of the following ASCII strings:

- (a) `banana`
- (b) `λ`
- (c) `Enter your age:`
- (d) `R-KB8?!`

E1.2.3 Let  $u$ ,  $v$ , and  $w$  be the strings *gar*, *age*, and *dam* respectively. Give the strings denoted by the expressions  $uv$ ,  $vw$ ,  $wv$ ,  $u^R w$ ,  $(uw)^R$ , and  $uw^R v$ .

E1.2.4 List all the substrings of the string *team*. Indicate which are prefixes and which are suffixes. Repeat for the string *mama*.

E1.2.5 List all the three-letter substrings of the string *rhinoceros*. Which are prefixes or suffixes?

E1.2.6 Give an example of two strings  $u$  and  $v$  such that  $u$  is *both* a prefix *and* a suffix of  $v$ , but  $u$  is not  $\lambda$  and  $u$  is not equal to  $v$ .

E1.2.7 Let  $D$  be the alphabet  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ . Let  $NZ$  be the language of all strings in  $D^*$  that do not start with 0. Let  $E$  be the set of strings in  $NZ$  that denote even numbers, and  $H$  be the set of strings in  $NZ$  that denote multiples of 100.

- (a) If you were given a string in  $D^*$ , how would you decide whether it is in  $E$ ? Whether it is in  $H$ ?
- (b) Describe the languages  $E^R$  and  $H^R$ . Exactly which strings in  $E^R$  are also in  $NZ$ ? Which strings in  $H^R$  are also in  $NZ$ ?

E1.2.8 Let  $B$  be the alphabet  $\{0, 1\}$ . How many strings are in the language  $B^5$ ? Let  $X = \{w : w \text{ has at least four 1's}\}$ . Let  $Y = \{w : 11 \text{ is not a substring of } w\}$ . Let  $Z = \{w : 00 \text{ is not a substring of } w\}$ . List the five-letter strings that are:

- (a) in  $X$
- (b) in  $Y$
- (c) in  $Z$
- (d) both in  $X$  and in  $Y$
- (e) both in  $X$  and in  $Z$
- (f) both in  $Y$  and in  $Z$
- (g) in neither  $Y$  nor  $Z$

E1.2.9 (uses Java) The `String` class in real Java has instance methods `char charAt(int n)`, giving the character in the  $n$ 'th position of the calling string<sup>9</sup>, and `int length()`, giving the length of the calling string. Using these methods, write code for the following real Java `static`<sup>10</sup> methods:

<sup>9</sup>The indexing of positions in strings, like that of positions in arrays, starts with 0, so `w.charAt(0)` returns the first letter of the `String w`. For another example, if `name` is a `String` variable with value "Elvis Presley", then `name.charAt(6)` would return the character P.

<sup>10</sup>We might like to add these as instance methods to the `String` class, but we are not allowed to do that.



- (a) `boolean equals(String u, String v)`, returning `true` if and only if  $u$  and  $v$  are equal strings,
- (b) `boolean prefix(String u, String v)`, returning `true` if and only if  $u$  is a prefix of  $v$ , and
- (c) `boolean suffix(String u, String v)`, returning `true` if and only if  $u$  is a suffix of  $v$ .

E1.2.10 Let  $A$ ,  $B$ , and  $C$  be any three languages over the same alphabet  $\Sigma$ . Suppose that whenever we concatenate a string from  $A$  with a string from  $B$ , in either order, the result is in  $C$ . Is it necessarily true that the reversal of any string in  $C$  is also in  $C$ ? Either explain why it is, or give an example of three languages  $A$ ,  $B$ , and  $C$  for which it isn't.

## 1.2.5 Problems

P1.2.1 Let  $A$  be the set  $\{a\}$ . What is the set  $A^*$ ? If  $u$  and  $v$  are *any* two strings in  $A^*$ , explain why the statements  $u^R = u$  and  $uv = vu$  must be true.

P1.2.2 Let  $A$  be any finite alphabet and let  $u$  and  $v$  be strings over  $A$ .

- (a) Suppose that  $u$  is a prefix of  $v$ . Explain, using the definition of “prefix”, why there must be some string  $w$  such that  $uw = v$ .
- (b) Suppose that  $uw = v$  for some string  $w$ . Explain, using the definition of “prefix”, why  $u$  must be a prefix of  $v$ .
- (c) Suppose  $u$  is a suffix of  $v$ . Following part (b), what can we say about the existence of a string in what concatenation equation with  $u$  and  $v$ ?
- (d) Suppose  $u$  is a substring of  $v$ . How can we describe this relationship in terms of a concatenation equation?

P1.2.3 What should be the alphabet used to model both positive and negative decimal integers as strings? Does every string over this alphabet denote an integer?

P1.2.4 In each of these cases, determine whether the given data item should be modeled as a *set* or as a *sequence*. Justify your answer.

- (a) The choice of optional features ordered on a new car.
- (b) The roads to be used to drive from one place to another.
- (c) The sales totals for each of the years from 1992 to 1996.
- (d) The years in which the sales exceeded a million dollars.
- (e) The squares on a chessboard.

P1.2.5 Let the alphabet  $C$  be  $\{a, b, c\}$ . Let the language  $X$  be the set of all strings over  $C$  with at least two occurrences of  $b$ . Let  $Y$  be the language of all strings over  $C$  that never have two occurrences of  $c$  in a row. Let  $Z$  be the language of all strings over  $C$  in which every  $c$  is followed by an  $a$ . (Recall that any string with *no*  $c$ 's is thus in  $Z$ .)

- (a) List the three-letter strings in each of  $X$ ,  $Y$ , and  $Z$ . The easiest way to do this may be to first list all 27 strings in  $C^3$  and then see which ones meet the given conditions.

- (b) List the four-letter strings that are both in  $X$  and in  $Y$ , those that are both in  $X$  and in  $Z$ , those that are both in  $Y$  and in  $Z$ , and those that are in all three sets. How many total strings are in  $C^4$ ?
- (c) Are any of  $X$ ,  $Y$ , or  $Z$  subsets of any of the others?
- (d) Suppose  $u$  and  $v$  are two strings in  $X$ . Do we know that the strings  $u^R$ ,  $v^R$ ,  $uv$ , and  $vu$  are all in  $X$ ? Either explain why this is always true, or give an example where it is not.
- (e) Repeat the previous question for the languages  $Y$  and  $Z$ .

P1.2.6 (uses Java) What is the output of the following Java code? Here recall that the `++` operator replaces a character by the following character in the alphabet, that `w += v`, for example, is an abbreviation of `w = w + v`, and that the character `letter` will be automatically converted to a `String` before being concatenated:

```
public static void printFunnyString ( )
{
    //Constructs and prints a particular string

    char letter = 'a';
    String v = "", w = "";

    for (int i = 0; i < 5; i++)
    {
        v += letter;
        w += v;
        letter ++;
    }
    System.out.println (w);
}
```

P1.2.7 (uses Java) Using the real-Java methods from Exercise 1.2.9 above, write decision procedures in real Java for the following languages of Java strings. These methods should each take a `String` as argument and return a `boolean`.

- (a) The set of all strings whose third letter exists and is `b`.
- (b) The set of all strings whose first letter exists and is the same as its last letter.
- (c) The set of all strings that are palindromes (equal to their reversals).
- (d) The set of all strings whose letters are in alphabetical order, like `abfbgkklzy`. (Java defines the order operators `<` and `<=` on characters — assume that “alphabetical order” is defined in terms of these).

P1.2.8 (uses Java) Using the real-Java methods from Exercise 1.2.9 above, write the following real-Java static methods:

- (a) `boolean substring(String u, String v)`, which returns `true` if and only if  $u$  is a substring of  $v$ , and
- (b) `boolean isPalindrome(String w)`, which returns `true` if and only if  $w$  is equal to its own reversal.

- P1.2.9 (uses Java, harder) Two strings  $u$  and  $v$  are called **anagrams** if one can be made by rearranging the letters of the other, that is, if every letter that occurs in one also occurs in the other, exactly the same number of times. (An example is the pair of words `looped` and `poodle`.) Using the methods from Exercise 1.2.9 above, write a real-Java static method `boolean anagram(String u, String v)` that returns `true` if and only if  $u$  and  $v$  are anagrams.
- P1.2.10 Let  $\Sigma$  be an alphabet with  $k$  letters and let  $n$  be any natural. How many strings are in the language  $\Sigma^n$ ? Justify your answer as best you can, though we won't have formal tools to prove this until Chapter 4.

### 1.3 Excursion: What is a Proof?

The word “proof” has a variety of meanings in different contexts. In a criminal trial, the prosecution attempts to “prove beyond a reasonable doubt” that a defendant is guilty. A scientist reports experiments that “prove” that a particular theory is correct. Religious thinkers offer “proofs” of the existence of God, or of the immortality of the soul. And finally, mathematicians and computer scientists “prove” things in their own way.

American high schools usually teach geometry at some point, in part because it is the most accessible example of the mathematical idea of proof. There a “proof” is defined to be a sequence of statements, each of which is justified in some way, and there are explicit rules telling whether a statement is legally justified or not. Typical justifications are by appeal to a *definition* (“a triangle is a figure with three sides”), an *axiom* (“through any given point there is exactly one line parallel to a given line”), or a *previously proved result* (“opposite angles at a vertex are equal”). A finished proof could be checked for correctness by a computer program, if the rules are carefully enough defined. There has even been some success by computer programs that produce such proofs by systematically looking for sequences of statements that are valid and interesting in some way.

In practice, though, actual mathematicians virtually never make completely formal proofs. Here is an example of how things actually happen. In 1985, a graduate student at MIT<sup>11</sup> was working on his Ph.D. research problem. He wanted to show that a “branching program”<sup>12</sup> that solved the “majority problem” requires more “width” the longer the input string gets (don’t worry about what the technical terms mean). Of the perhaps 100 people in the world who had thought seriously about this problem, to his knowledge all of them (including him) thought that what he was trying to show was true. Frustrated at his lack of progress, he tried constructing some branching programs and suddenly realized that he *could* solve the majority problem with width five for inputs of any length. He checked the argument and convinced himself that it was correct, despite the fact that it led to the “wrong” result.

Now his problem was either to convince the world that he was right and everyone else was wrong, or to find the mistake in his argument before embarrassing himself too much. Since his thesis advisor was out of town, he grabbed the first two other graduate students he could find who were familiar enough with the work to understand the argument. He then showed them the proof, which in fifteen minutes they believed. Within a few weeks the proof (which was in fact correct) had been spread literally around the world by word of mouth, and those 100 or so people had all changed their minds.

This proof was *not* the sequence of statements and justifications from high-school geometry. The prover could take advantage of the fact that both he and his listeners were trained mathematicians, and leave out huge numbers of steps that they could all agree upon. The process was essentially an *interactive* one, where the prover made claims, and the listeners questioned any of those claims they didn’t accept. The prover then reinforced the challenged claims by showing how they followed from simpler facts, and so on. In principle, the prover was ready to answer all possible objections.

---

<sup>11</sup>Later to write a discrete mathematics textbook.

<sup>12</sup>Of polynomial “size”, to be precise.

A written version of the proof cannot take advantage of this interactive process<sup>13</sup>. In a mathematical paper, the author decides on a particular audience and tries to present the argument anticipating any reasonable objections from that audience. It is assumed that the reader might have to stop and solve some problems (constructing some smaller proofs herself) in the course of reading the paper. The quality of the paper is checked before publication by sending it to one or more “referees”, expert readers who decide whether they are convinced by the argument and make suggestions as to how to improve it.

The number and size of proof steps to be left out depends dramatically on the expected audience of the paper. In the case of published research, the reader is an expert mathematician who is looking for the most important new contributions and can take the easier steps for granted. In the case of an expository paper, the reader wants to be educated more broadly in the area, and needs to be helped through more of the steps. And in the case of an undergraduate exercise, the reader is an instructor who already believes that the thing in question is true, but needs to be convinced that the author understands *why* it is true. The number of steps required for this may well vary within the same course — the student learns tools at the beginning that will justify her in skipping steps later<sup>14</sup>.

To conclude this Excursion, we’ll look at two examples of proof, one in a computer science setting and one in number theory. The first is a matter of *program correctness*. Here’s an example of a Java method that is supposed to get some integers from the user until it sees a blank line:

```
public static int sumInts ( )
{
    // Reads ints from System.in, one per line, until blank line.
    // Returns sum of integers read.

    int soFar = 0;
    String thisLine;

    System.out.println
        ("Enter integers, one per line, then a blank line: ");
    while ( !(thisLine = System.in.readLine()).equals(""))
    {
        soFar += Integer.parseInt (thisLine);
    }
    return soFar;
}
```

What does it mean for this code to be “correct”? We would like to be sure that if the user does the right thing (inputs the integers as requested), then the method will return the correct value. In general we can say what we want from the code by giving **preconditions** and **postconditions**.

---

<sup>13</sup>Socrates claimed that books were useless to the practice of philosophy, because they couldn’t respond to questions.

<sup>14</sup>Solow, in *How to Read and Do Proofs*, has an extensive discussion of the difference between formal proofs and the proofs that actually occur in mathematical literature. His overall theme is that mathematical proof is a game like chess, except that the rules aren’t clearly written down. Later in this chapter we’ll start to learn the rules.

A precondition is a statement that should be true *before* the code is executed — here it is “the user inputs zero or more integers, one per line, followed by a blank line”. A postcondition is something that should be true *after* the code is executed *if* the preconditions were met. Here our postcondition is “the returned value is the sum of the given integers”. We define the code to be **partially correct** if the postconditions must always be true after execution whenever the preconditions were true before. (So partial correctness says nothing about how to handle erroneous input, for example, unless the preconditions and postconditions say how to do so.)

How could we be *sure* that this code is correct? Coding mistakes are easy to make, and even if the compiler finds no syntax errors there could be subtle errors of logic. We could test the code on sample inputs, but some errors might cause bad behavior only on rare inputs that we didn’t think to test (and in most cases we could never test *all* the relevant inputs).

What we need is a formal argument — a proof! We want to show how the statement of partial correctness *follows logically* from things that we can assume to be true. The things we can assume are the basic properties of the language and the library procedures. For example, if the user does input an integer, the properties of `System.in.readLine` and `Integer.parseInt` can tell us how that integer will go from the input device into the value of some variable in our program.

Our second example deals with a set of statements about naturals, which you are to prove using only the definitions you are given. First, the definitions:

- A **natural** is a non-negative integer like 0, 1, 2, 3, and so forth.
- The **successor** of a natural is the natural that comes after it – there always is one.
- The **predecessor** of a natural is the one before – every natural except 0 has one.
- The **addition** and **multiplication** operations work the way you already understand. (But don’t assume the same for division!)
- A natural is an **even number** if it is 2 times some natural.
- A natural is an **odd number** if it is 2 times some natural, plus one.
- Naturals obey the **Least Number Axiom** – if any natural has a particular property, there must be a *smallest* natural with that property. (We often use this to get a contradiction.)

### 1.3.1 Writing Exercise

Most Excursions in this book will contain a variety of writing exercises. Your instructor should choose one based on the time available, your level of background, and the feedback he or she most needs.

1. Explain to your instructor your personal experience with the concept of “proof”. Have you had to prove things in previous mathematics courses? Did you like doing it? Did you feel

more confident about the truth of something once you had proved it? What would you like to learn about proofs in this course?

2. Explain as carefully as possible why the code in our example above is partially correct, that is, why the postcondition will be true whenever the precondition is true. (Hint: What can you say about the relationship of `soFar` to the numbers seen so far at the start of each pass through the `while` loop? A statement that is true every time you start a particular loop is called a **loop invariant**. Can you find a useful loop invariant that helps explain why this code works?)
3. Given the number theory definitions above, prove the following six statements:
  - (a) If  $x$  has a predecessor that is even, then  $x$  is odd.
  - (b) If  $x$  is an odd natural, then  $x$  has a predecessor that is even.
  - (c) If  $x$  has a predecessor that is odd, then  $x$  is even.
  - (d) If  $x$  is an even natural, then  $x$ 's predecessor (if it has one) is odd.
  - (e) Prove that every natural is either odd or even. (Hint: By the Least Number Axiom, if any natural is *neither* odd *nor* even, there's a least such natural. Could it be 0? If not, what about its predecessor? Use the results of (a) – (d) to get a contradiction.)
  - (f) Prove that no natural is both odd and even. (Similar to (e) – get a contradiction by assuming some natural is both.)

## 1.4 Propositions and Boolean Operations

### 1.4.1 Propositions

When we use the mathematical method, we are constantly manipulating statements. We assume that some statement is true, or show that if it is true, then something else is true, or we rewrite a statement into another form that has the same meaning. In order to do this, and especially in order to talk about what we are doing, we will need to have mathematical objects that model statements.

**Definition:** A **proposition** is a statement that is either true or false.

**Example:** Equations like “ $2 + 2 = 4$ ” or “ $2 + 2 = 5$ ” are all propositions, whether they are true or false. So are statements about programs, like “ $x$  has the value 7 after line 12 is executed”, or “method `test` always returns the correct value”. We can have propositions that talk about virtually any subject, such as “Fargo is the capital of North Dakota”, “John Cleese was the tallest member of Monty Python”, or “Emily Dickinson wrote more than 100 poems”. There are statements that are not propositions — some examples would be statements with undefined terms like “ $x$  is a city in Norway” (where we can’t say it is true or false until we know what  $x$  represents) or deliberate paradoxes like “This statement is false” (think about it).

A proposition can be represented as a **bit**, the fundamental unit of information that is the basis for all digital computing. Every piece of data in a digital computer is represented as some sequence of bits, each stored physically in some location in the memory, and each of our computations ultimately reduces to some manipulation of bits by the hardware.

In software terms, we can model a proposition by a variable of the primitive type `boolean`, which has two values `true` and `false`, called the **boolean constants**<sup>15</sup>. The most common use of boolean values in software is in conditional statements such as `if`, `for`, or `while` statements. In the Java statement “`if (x < 4) y++; else y--;`”, for example, the variable `y` is to be increased by one if the proposition `x < 4` is true, and decreased by one if it is false.

To complete the specification of the class, however, we will also need to define ways of dealing with boolean data. The most important of these will be the subject of the next subsection:

**Definition:** A **boolean operation** is any function whose inputs and outputs are both of the `boolean` type. The **propositional calculus** is the entire practice of calculating with boolean data.

Unfortunately, as booleans have been used in mathematics, programming, and in hardware, each community has adopted a different notation and terminology for the boolean operations. In this book we will mostly use a system of notation common in formal logic, because it’s the system most commonly used in general mathematical argument. But every programming language with a `boolean` type has its own notation, and a still different notation (called **boolean algebra**) is

---

<sup>15</sup>This is the Java usage. Pascal has a similar `boolean` type, but C and C++ identify the boolean constants with the numbers 1 and 0. We will sometimes use 1 and 0 as alternate names for `true` and `false`. Actually, C interprets any nonzero integer as “true”, which makes some common calculations easier but can sometimes lead to confusion.



used in electrical and computer engineering. The basic concepts do not depend on the details of representation, though — we'll refer to the other terminology as appropriate below.

## 1.4.2 Boolean Operators and Compound Propositions

We frequently combine statements to make more complicated statements, both in everyday language (“I will either walk to work or take the bus”), and in mathematics (“If  $x > 3$ , then  $x^2 > 9$ ”). In programming, as well, we need to combine `boolean` values in conditions — for example, we might want to continue a loop until *either* all the input is read *or* an error occurs. In each case the value of the more complicated proposition depends on the value of the smaller propositions that make up its parts.

**Definition:** A **compound proposition** is a statement made up from one or more simpler statements, called **atomic** or **base propositions**, by the use of boolean operations.

**Example:** Let the variable  $p$  represent the proposition “the value of  $x$  is 7” and  $q$  represent the proposition “the method `test` returns a value”. Some possible compound propositions involving these base propositions are “The value of  $x$  is 7, and the method `test` returns a value” and “If the method `test` returns a value, then the value of  $x$  is 7”. We'll look at the exact meaning of these and other compound propositions below as we consider each of our specific boolean operations. But if we knew the truth values of  $p$  and  $q$ , we would be able to determine the truth values of each of these compound propositions.

**Definition:** Let  $p$  and  $q$  be propositions. The **conjunction** of  $p$  and  $q$ , written “ $p \wedge q$ ” and read as “ $p$  and  $q$ ”, is the proposition that is true if both  $p$  and  $q$  are true and false otherwise.

**Example:** With  $p$  and  $q$  as defined above,  $p \wedge q$  is the statement “The value of  $x$  is 7 and the method `test` returns a value”. To know that  $p \wedge q$  is true, we need to know both that  $x$  is 7 and that `test` returns a value. To know that it is false, however, it is enough to know either that  $x$  is not 7 or that `test` does not return a value.

Java and the rest of the C family use the symbol `&&` for the “and” operation on boolean values. Actually there is also another operator `&`, which has the same effect most of the time but differs from `&&` in a subtle way. Consider the boolean expression `(x == 7) && (test() > 0)` and imagine that for some reason `test` does *not* return a value. If  $x$  is not equal to 7, the computer will know that the compound proposition is false, and not bother to run `test`. For this reason `&&` is called a “short-circuit” operation. If the statement were `(x == 7) & (test() > 0)`, on the other hand, even if  $x$  were not equal to 7 the computer would still try to evaluate the second part of the expression, and would not succeed. Thus the two expressions could cause different behavior. In general, programmers use the `&&` operator unless there is a specific reason why the second half should be checked — if nothing else it can save the computer some work.

We defined the result of the  $\wedge$  operation by giving the value of  $p \wedge q$  for each possible sequence of values for  $p$  and  $q$ . This will be an important general technique for working with compound propositions:

| $p$ | $q$ | $p \wedge q$ |
|-----|-----|--------------|
| 0   | 0   | 0            |
| 0   | 1   | 0            |
| 1   | 0   | 0            |
| 1   | 1   | 1            |

Source: David Mix Barrington

Figure 1-1: The truth table for AND.

| $p$ | $q$ | $p \vee q$ | $p \oplus q$ |
|-----|-----|------------|--------------|
| 0   | 0   | 0          | 0            |
| 0   | 1   | 1          | 1            |
| 1   | 0   | 1          | 1            |
| 1   | 1   | 1          | 0            |

Source: David Mix Barrington

Figure 1-2: The truth tables for OR and XOR.

**Definition:** A **truth table** is a representation of the values of one or more compound propositions. Each row of the table represents one of the possible sequences of values for the base propositions involved, and each column represents a compound proposition. Each value in the table is the value of the compound proposition for its row, with the base values given for its column. We use 0 and 1 to denote the boolean values, as these are sometimes easier to distinguish than  $F$  and  $T$ .

**Example:** Figure 1.1 shows the truth table for the compound proposition  $p \wedge q$ .

The natural next compound proposition to consider is “the value of  $x$  is 7 *or* `test` returns a value”. But should we consider this to be true if *both* its base propositions are true? In English the answer often depends on the context. In mathematics we cannot afford to be imprecise, and so we define *two different* “or” operations for the two possibilities:

**Definition:** Let  $p$  and  $q$  be propositions. The **disjunction** or **inclusive or** of  $p$  and  $q$ , written “ $p \vee q$ ” and read as “ $p$  or  $q$ ”, is the proposition that is true if *either or both* of  $p$  and  $q$  are true, and false only if both are false. The **exclusive or** of  $p$  and  $q$ , written “ $p \oplus q$ ” and read as “ $p$  exclusive or  $q$ ”, is the proposition that is true if *one or the other* of  $p$  and  $q$  is true, and the other false.

**Example:** Using our examples for  $p$  and  $q$ , we can read  $p \vee q$  as “ $x$  is 7, or `test` returns a value, or both” and  $p \oplus q$  as “ $x$  is 7, or `test` returns a value, but *not* both”. If  $x = 6$  and `test` returns a value, for example, both  $p \vee q$  and  $p \oplus q$  are true. If  $p \wedge q$  is true, on the other hand, then  $p \vee q$  is true but  $p \oplus q$  is false. Figure 1-2 describes the complete behavior of both of these operators by a truth table.

Java represents the inclusive OR operation by the symbol `||`. As with AND, there is an alternate operation `|` that evaluates the second input even if the first input is true. So if both inputs are defined, `||` and `|` give the same answer. But if `p` is true and `q` causes an error, evaluating `p | q` will trigger the error but `p || q` will not. We will use `||` in this book, following the general practice. The exclusive OR operation is denoted in Java by the symbol `^`.

| $p$ | $q$ | $\neg p$ | $\neg q$ |
|-----|-----|----------|----------|
| 0   | 0   | 1        | 1        |
| 0   | 1   | 1        | 0        |
| 1   | 0   | 0        | 1        |
| 1   | 1   | 0        | 0        |

Source: David Mix Barrington

Figure 1-3: The truth table for NOT.

**Definition:** Let  $p$  be a proposition. The **negation** of  $p$ , written  $\neg p$  and read as “not  $p$ ” or “it is not the case that  $p$ ”, is the proposition that is true when  $p$  is false and false when  $p$  is true.

**Example:** Again using our given meanings for  $p$  and  $q$ ,  $\neg p$  means “the value of  $x$  is not 7” and  $\neg q$  means “`test` does not return a value”. Note that there are several equivalent ways in English to express the negation, such as “It is not true that `test` returns a value”.

Figure 1-3 is a truth table giving the values of the two compound propositions  $\neg p$  and  $\neg q$ . Note that each column has two 0's and two 1's — for example,  $\neg p$  is true for *both* the rows where  $p$  is false, because the value of  $q$  has no effect on the value of  $\neg p$ .

When we use the  $\neg$  operator along with other operators, we adopt the convention that the negation applies only to the *next* proposition in the compound proposition. Thus  $\neg p \vee q$ , for example, means “either  $p$  is false, or  $q$  is true, or both”, while  $\neg(p \vee q)$  means “it is not true that  $p$  is true or  $q$  is true, or both”.

Java denotes negation by the symbol `!`. This character also occurs in the Java symbol `!=` meaning “not equal”.

Our last two boolean operators are particularly important because they model steps taken in proofs. The first captures the notion of a “legal move” in a proof. If we have established that  $p$  is true, for example, when would we be justified in saying that  $q$  must be true?

**Definition:** Let  $p$  and  $q$  be propositions. The **implication**  $p \rightarrow q$ , read as “ $p$  implies  $q$ ” or “if  $p$ , then  $q$ ” is the proposition that is true *unless*  $p$  is true and  $q$  is false. Another way to say this is that  $p \rightarrow q$  is true if  $p$  is false or if  $q$  is true, but not otherwise.

**Example:** In our running example,  $p \rightarrow q$  is the proposition “if  $x$  is 7, then `test` returns a value”. On the other hand,  $q \rightarrow p$  is the proposition “if `test` returns a value, then  $x$  is 7”, which is a *different proposition*. Each one says that a certain situation is impossible. The first,  $p \rightarrow q$ , means that it is not possible for  $x$  to be 7 and for `test` not to return a value. This would be true if `test` always returned a value, or if it checked  $x$  and continued and returned a value only if  $x$  were 7. The second statement,  $q \rightarrow p$ , says that it is impossible that `test` returns a value and that  $x$  is *not* 7. Perhaps `test` sets  $x$  to 7 just before it returns, or `test` contains a statement `while (x != 7) y++;` that sends it into an infinite loop unless  $x$  is 7.

Implication can be rather confusing on a first encounter. First, there are many ways to express

| $p$ | $q$ | $p \rightarrow q$ | $p \leftrightarrow q$ |
|-----|-----|-------------------|-----------------------|
| 0   | 0   | 1                 | 1                     |
| 0   | 1   | 1                 | 0                     |
| 1   | 0   | 0                 | 0                     |
| 1   | 1   | 1                 | 1                     |

Source: David Mix Barrington

Figure 1-4: The truth tables for  $\rightarrow$  and  $\leftrightarrow$ .

an implication in English, such as “ $p$  implies  $q$ ”, “ $p$  only if  $q$ ”, and “if  $p$ , then  $q$ ”. Their use in English may or may not capture the exact mathematical meaning of the symbol  $\rightarrow$  (given by the truth table in Figure 1-4). For example, saying “if your shoes are muddy, it must be raining” in English carries the extra implication that the two facts have something to do with each other. Mathematically, it doesn’t matter whether the propositions are connected as long as their truth values make the implication true. Also, we don’t often think about what that English statement means if your shoes are not muddy, but mathematically it is still true. The compound proposition  $p \rightarrow q$  is true *whenever*  $p$  is false.

So mathematically we could say “If  $0 = 1$ , then I am Elvis”. The great logician Bertrand Russell was allegedly once asked to prove a similar statement and came up with the following (adapted) proof: “Assume  $0 = 1$ . Add one to both sides, getting  $1 = 2$ . Elvis and I are two people (obvious). But since  $1 = 2$ , Elvis and I are one person. QED<sup>16</sup>.” This convention may appear silly on a first encounter, but it matches actual mathematical practice. The key purpose of implications in proofs is to get true statements from other true statements. If you are starting from false premises, it is not the fault of the proof method if you reach false conclusions.

Our last boolean operator is also extremely useful in proofs because it captures the notion of two propositions being *the same* in all possible circumstances.

**Definition:** Let  $p$  and  $q$  be propositions. The **equivalence** of  $p$  and  $q$ , written  $p \leftrightarrow q$  and read as “ $p$  if and only if<sup>17</sup>  $q$ ”, is the proposition that is true if  $p$  and  $q$  *have the same truth value* (are both true or both false), and false otherwise.

**Example:** Again using our values for  $p$  and  $q$ ,  $p \leftrightarrow q$  means “ $x$  is 7 if and only if **test** returns a value”.

**Facts:** If  $p$  and  $q$  are any propositions,  $p \leftrightarrow q$  has the same truth value as  $(p \rightarrow q) \wedge (q \rightarrow p)$ , and the same truth value as  $\neg(p \oplus q)$ .

The most important thing about equivalence is that it works like equality in equations — if you know that two compound propositions are equivalent, you can replace one by the other *in any context* without changing any truth values. For example, if we know  $p \leftrightarrow q$  and  $p \rightarrow (r \vee (q \wedge p))$ ,

<sup>16</sup> “QED” stands for the Latin phrase “quod erat demonstrandum”, literally meaning “which was to be proved”, and is traditionally used to declare victory at the end of a proof. We’ll generally end proofs in this book with the ♠ symbol.

<sup>17</sup> The phrase “if and only if” is traditionally abbreviated as “iff” in mathematics. You should learn to recognize this usage, although we will avoid it in this book.

then we can replace  $p$  by  $q$  in the second compound proposition and conclude that  $q \rightarrow (r \vee (q \wedge q))$  is true.

We've now defined a large set of boolean operators. By using them repeatedly, we can construct larger compound propositions from smaller ones. Once values are chosen for each of the atomic propositions in a compound proposition, we can **evaluate** the compound proposition by repeatedly applying the rules for evaluating the results of individual operators.

**Example:** Suppose  $p$  and  $q$  are **true** and  $r$  is **false**, and we want to evaluate the compound proposition  $p \leftrightarrow (q \wedge (r \oplus \neg(q \vee p)))$ . We can see that  $q \vee p$  is **true**, so  $\neg(q \vee p)$  is **false**. Because  $r$  is also **false**,  $r \oplus \neg(q \vee p)$  is **false**. By the definition of  $\wedge$ , then,  $q \wedge (r \oplus \neg(q \vee p))$  is **false**. Finally, since  $p$  is **true**, it is not equivalent to  $q \wedge (r \oplus \neg(q \vee p))$  and the entire compound proposition is **false**.

In order to evaluate a compound proposition in this way, we have to know *in which order* to apply the operations. We'll see in Problem 1.4.1 that the meaning of  $p \vee q \wedge r$ , for example, is not clear unless we use **parentheses** to indicate which operation is to take place first. (That is, do we mean  $p \vee (q \wedge r)$  or  $(p \vee q) \wedge r$ ?) In ordinary arithmetic and algebra we have a set of **precedence rules** telling us to perform multiplication before addition and so forth. Programming languages have a more formal set of rules telling which operation happens first in all situations.

In compound expressions with boolean operators, we will insist that parentheses be used to tell us the order of operations in all cases, with only two exceptions. We give the  $\neg$  operator the highest precedence, so that it is always applied to the next thing on its right (which may be a parenthesized expression). Also, we know that the operators  $\wedge$ ,  $\vee$ ,  $\oplus$ , and  $\leftrightarrow$  are associative with each other, so we don't need parentheses in expressions like  $p \vee q \vee r$ .

What if we have a compound proposition and we *don't* know the values of the atomic propositions in it? In general the compound proposition will be true for some settings of the atomic propositions and false for others, and we are interested in exactly which settings make it true. In Section 1.6 we will learn a systematic method to do this called **the method of truth tables**.

**Definition:** A **tautology** is a compound proposition that is true for all possible values of its base propositions.

**Example:** Recall our sample propositions  $p$  and  $q$ . A simple tautology is  $p \vee \neg p$ , meaning “ $x$  is equal to 7, or  $x$  is not equal to 7”. We can be confident that this is true without knowing anything about  $x$ . A more complicated tautology is  $(\neg(p \wedge q) \wedge p) \rightarrow \neg q$ . Let's see how to translate this. We first locate the last operation to be applied, which is the implication, and get “If  $(\neg(p \wedge q) \wedge p)$ , then  $\neg q$ ”. Then we attack the two pieces, using “it is not the case that” to express the complex use of  $\neg$  in the first piece. The final translation is “If it is not the case that  $x = 7$  and **test** returns a value, and  $x = 7$ , then **test** will not return a value”. If you puzzle over this statement carefully you should be convinced that it is true, and that its truth depends not on any knowledge about  $x$  or **test**, but essentially on the definition of the word “and”.

All tautologies have this quality of obviousness to them, because if they depended on the content of their component propositions, they would be false for some possible contents and thus not be

tautologies. But tautologies can still be useful in two ways. First, any tautology gives us a potential tool to use in proofs, because it generates a true statement that we might then use to justify other statements. In particular, tautologies of the form  $R \rightarrow S$  and  $R \leftrightarrow S$ , where  $R$  and  $S$  are themselves compound propositions, can be particularly useful. If  $R \rightarrow S$  is a tautology, and we know that  $R$  is true, then  $S$  must be true. If  $R \leftrightarrow S$  is a tautology, we say that  $R$  and  $S$  are **logically equivalent** and we may replace  $R$  with  $S$  in any statement without changing its truth value. In Section 1.7 we'll give some examples of such useful tautologies.

If a compound proposition is not a tautology (always true), then it is either *sometimes* true or *never* true.

**Definition:** A **contradiction** is a compound proposition that is never true for any of the possible values of its base propositions. A compound proposition is **satisfiable** if it is true for at least one of the possible settings of the variables. Thus a compound proposition is satisfiable if and only if it is not a contradiction.

### 1.4.3 Exercises

E1.4.1 Identify the following statements as true or false:

- (a) If  $p$  is true, we know that  $p \rightarrow q$  must be true.
- (b) The statement " $p \vee q$ " is a tautology.
- (c) If  $p \wedge q$  is true, we know that  $p \vee q$  is also true.
- (d) The statement " $p \rightarrow (q \wedge (r \vee \neg p))$ " is a compound proposition.
- (e) You can never determine the truth value of a compound proposition without knowing the truth values of all of its base propositions.

E1.4.2 Which of the following sentences are propositions?

- (a) Montréal is the capital of Québec.
- (b) Tell me the capital of Québec.
- (c) The next city I will name is the capital of Québec.
- (d) I don't know what the capital of Québec is.

E1.4.3 It was claimed in this section that "This statement is false" is not a proposition. What exactly is the problem with assigning a truth value to this statement?

E1.4.4 Evaluate the following Java boolean expressions:

- (a)  $(3 \leq 4) \ \&\& \ !(5 == 6)$ .
- (b)  $!((4 < 3) \ || \ (2 != 3))$ .
- (c)  $((3 == 3) \ \wedge \ \text{false}) \ \wedge \ (2 + 2 = 5)$ .
- (d)  $((2 < 3) \ || \ !(4 < 3)) \ \&\& \ (!(2 < 3) \ \&\& \ (4 < 3))$ .

E1.4.5 Evaluate the following compound expressions, given that  $p$  is true and  $q$  is false. Show enough of your work to make your reasoning clear.

- (a)  $p \rightarrow q$ .
- (b)  $\neg q \leftrightarrow p$ .
- (c)  $\neg p \oplus \neg q$ .
- (d)  $p \vee (q \wedge \neg q)$ .
- (e)  $(p \vee q) \wedge \neg q$ .
- (f)  $(\neg p \vee q) \rightarrow (\neg p \wedge (\neg q \rightarrow p))$ .

- E1.4.6 Convert each of the compound propositions in Exercise 1.4.5 above into English, translating  $p$  as “fish live in water” and  $q$  as “trout live in trees”.
- E1.4.7 Each of the following statements from an advertisement uses the word “or”. Indicate for each one whether the inclusive or exclusive “or” is meant:
- (a) Applicants may be men or women.
  - (b) Applicants must be fluent in either English or Spanish.
  - (c) Applicants must have a bachelor’s degree or at least three years experience.
  - (d) Applicants must be U.S. citizens or non-citizens with green cards.
- E1.4.8 Letting  $p$  represent the proposition “mackerel are fish” and  $q$  represent “trout live in trees”, write each of the following English statements as a compound proposition.
- (a) If mackerel are fish, then trout live in trees.
  - (b) The statement that trout live in trees implies that mackerel are fish.
  - (c) Mackerel are fish, and trout live in trees only if trout do not live in trees.
  - (d) If either mackerel are fish or trout live in trees, but not both, then trout do not live in trees and mackerel are not fish.
- E1.4.9 Convert each of the implications in Exercise 1.4.8 above to OR statements, using the rule that  $x \rightarrow y$  is equivalent to  $\neg x \vee y$ . Write each of the new statements both as a compound proposition and as an English sentence.
- E1.4.10 Each of the statements of Exercise 1.4.8 contains exactly one implication. In each statement, *replace* the implication with an equivalence operation, and express the resulting statement in English.

#### 1.4.4 Problems

- P1.4.1 Show that the compound propositions  $p \vee (q \wedge r)$  and  $(p \vee q) \wedge r$  are *not* equivalent, by finding values of the base propositions that make them different.
- P1.4.2 Find equivalent forms for each of the following compound propositions, where “0” and “1” represent the boolean constants:  $x \wedge x$ ,  $x \wedge 0$ ,  $x \wedge 1$ ,  $x \vee x$ ,  $x \vee 0$ ,  $x \vee 1$ ,  $x \oplus x$ ,  $x \oplus 0$ ,  $x \oplus 1$ ,  $x \rightarrow x$ ,  $x \rightarrow 0$ ,  $x \rightarrow 1$ ,  $x \leftrightarrow x$ ,  $x \leftrightarrow 0$ ,  $x \leftrightarrow 1$ .
- P1.4.3 Another boolean operation is the “NAND” operation, where “ $x$  **nand**  $y$ ” is defined to be  $\neg(x \wedge y)$ . Explain why  $x$  **nand**  $x$  is equivalent to  $\neg x$ . Can you find a compound proposition using only the NAND operator that is equivalent to  $x \wedge y$ ? How about  $x \vee y$ ?

P1.4.4 (uses Java) Let  $p$  and  $q$  be Java boolean variables. What is the value of  $q$  after the following code is executed? Does it matter what the initial values of  $p$  and  $q$  are?

```
if ((p && (!q)) || (q && (!p)))
    q = false;
else q = q && p;
```

P1.4.5 Evaluate the following compound expressions, given that  $x$  and  $z$  are **true** and that  $y$  is **false**. Show enough of your work to make your reasoning clear.

- (a)  $x \wedge (y \vee z)$ .
- (b)  $(x \wedge y) \vee z$ .
- (c)  $(x \rightarrow y) \rightarrow z$ .
- (d)  $x \rightarrow (y \rightarrow z)$ .
- (e)  $(x \oplus \neg y) \leftrightarrow z$ .
- (f)  $x \oplus (\neg y \leftrightarrow z)$ .
- (g)  $((x \vee y) \leftrightarrow (z \rightarrow \neg y)) \oplus (y \wedge \neg z)$ .
- (h)  $((y \wedge \neg x) \oplus (\neg y \rightarrow z)) \leftrightarrow (z \vee \neg x) \leftrightarrow z$ .

P1.4.6 Translate each of the compound propositions in Problem 1.4.5 into English, using the following base propositions:  $x$  is “The sun will come up tomorrow”,  $y$  is “pigs have wings”, and  $z$  is “the project will go over budget”.

P1.4.7 (uses Java) Java has an operator for the  $\oplus$  operation, denoted by  $\wedge$ , but it has no direct way to express  $\rightarrow$  or  $\leftrightarrow$ . Is this a big problem?

- (a) Show that the  $\rightarrow$  and  $\leftrightarrow$  operations can be translated into some combination of the operators  $\&\&$ ,  $||$ , and  $!$ . Illustrate your translation method by writing the compound propositions  $p \rightarrow (q \leftrightarrow r)$  and  $(p \leftrightarrow r) \rightarrow (q \leftrightarrow \neg(p \rightarrow r))$  as Java boolean expressions.
- (b) Java does not define relational operators such as “less than” on booleans, but it does define the equality operator `==`. Is this operator of any help in expressing  $\rightarrow$  or  $\leftrightarrow$ ? If so, how?

P1.4.8 A set of boolean operators is called **complete** if it is possible to express an equivalent of *any* compound proposition with only the operators in the set.

- (a) Show that the set  $\{\wedge, \vee, \neg\}$  is complete by explaining how each of the operators  $\leftrightarrow$ ,  $\rightarrow$ , and  $\oplus$  may be translated into a compound proposition involving  $\wedge$ ,  $\vee$ , or  $\neg$ . Illustrate your translation procedure on the compound proposition  $p \leftrightarrow (q \rightarrow (r \oplus p))$ .
- (b) Determine whether each of the following sets of operators is complete:
  - i.  $\{\wedge, \neg\}$
  - ii.  $\{\wedge, \vee\}$
  - iii.  $\{\vee, \neg\}$
  - iv.  $\{\text{nand}\}$  (See Problem 1.4.3 above.)

P1.4.9 Choosing variables for the base propositions as needed, translate these English statements into compound propositions.



- (a) If you don't eat your meat, you can't have any pudding.
- (b) If I'm wearing the antlers, I am dictating, and if I'm not wearing the antlers, I'm not dictating.
- (c) If this penguin was from the zoo, it would have "Property of the Zoo" stamped on it, and if penguins molt, this penguin could not have "Property of the Zoo" stamped on it.
- (d) It is not true that if I am arguing, then you must have paid.

P1.4.10 Letting  $p$  denote "mackerel are fish" and  $q$  denote "trout live in trees", translate each of the following four statements into English:  $\neg p \rightarrow q$ ,  $\neg(p \rightarrow q)$ ,  $\neg p \leftrightarrow q$ , and  $\neg(p \leftrightarrow q)$ . Are any two of these four statements logically equivalent?

## 1.5 Set Operations and Propositions About Sets

### 1.5.1 Set Operations

In Section 1.1 we implicitly defined a number of *operations* on sets. We can ask whether two sets are the same (equal), or whether one is a subset of another — in computing terms these are functions that take two sets as inputs and return a **boolean** answer. Similarly, we can take a **thing** and a set of type **thing** and ask whether the thing is an element of the set — another **boolean** answer. Then there is the “size” function, which inputs a set and returns a **natural** if the set is finite.

Another important class of operations on sets are those where the result of the operation is also a set. The simplest of these is the *union* operation:

**Definition:** If  $A$  and  $B$  are two sets, the **union** of  $A$  and  $B$ , written “ $A \cup B$ ”, is the set of all elements that are in *either or both* of  $A$  and  $B$ .

**Example:** Consider the following two sets of novelists:  $A$  is the set {Jane Austen, Patrick O'Brian, Chinua Achebe} and  $B$  is the set {Jane Austen, George Eliot, Lady Murasaki}. The union of  $A$  and  $B$ ,  $A \cup B$ , is the set {Jane Austen, Patrick O'Brian, Chinua Achebe, George Eliot, Lady Murasaki}. The union of {2,3,5} and {4, 7, 12} is {2,3,4,5,7,12}. The union of {2,3,5} and {5,3} is {2,3,5}, as is the union of {2,3,5} and  $\emptyset$ , or the union of {2,3,5} and {2,3,5}. Note that if we start with two sets of the same type, the union also has this type. For example, since everything in both set  $A$  and set  $B$  above is a novelist, everything in the union must also be a novelist.

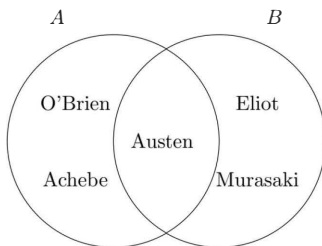
**Definition:** If  $A$  and  $B$  are two sets, the **intersection** of  $A$  and  $B$ , written “ $A \cap B$ ”, is the set of all elements that are *both* in  $A$  and in  $B$ . If  $A \cap B$  is the empty set, so that they have no elements in common, we say that  $A$  and  $B$  are **disjoint**.

**Example:** If  $A$  and  $B$  are the sets of novelists above,  $A \cap B$  is the set {Jane Austen}, because Jane Austen is the only novelist in both sets. The intersection of {2,3,5} and {4,7,12} is empty, because no natural is in both of these sets, so these two sets are disjoint. The intersection of {2,3,5} and {5,3} is {3,5}. The intersection of {2,3,5} and  $\emptyset$  is  $\emptyset$ , and the intersection of {2,3,5} and {2,3,5} is {2,3,5} itself. Note that if we start with two sets of the same type, the intersection is also of that type.

As our examples get more complicated, it's often useful to represent sets and elements by a particular kind of picture:

**Definition:** A **Venn diagram** is a pictorial representation of sets, as follows. Individual things are represented by dots on the diagram, or simply by their names. A set is represented by a closed curve that puts its elements on the inside and all other things on the outside. If the areas for two sets do not overlap, this indicates that the two sets are disjoint.

**Example:** Figure 1.5 illustrates the relationships among the sets of novelists  $A$  and  $B$ , and their elements by means of a Venn diagram. The circle labeled “ $A$ ” encloses the three elements of set  $A$ , and the other circle encloses the three elements of set  $B$ .



Source: David Mix Barrington

Figure 1-5: A Venn diagram for two sets of novelists.

Venn diagrams will be useful as we define more operations on sets:

**Definition:** Let  $A$  and  $B$  be two sets. The **difference** of  $A$  and  $B$ , written “ $A \setminus B$ ”, is the set of all elements that are in  $A$  but not in  $B$ . The **symmetric difference** of  $A$  and  $B$ , written “ $A \triangle B$ ”, is the set of all elements that are in  $A$  or in  $B$ , but *not in both*.

**Example:** With our same two sets of novelists,  $A$  and  $B$ , the set  $A \setminus B$  is {Patrick O’Brian, Chinua Achebe}, because these two novelists and no others are in  $A$  and not in  $B$ . The set  $B \setminus A$  is not the same as  $A \setminus B$  in general, and in our example it is {George Eliot, Lady Murasaki}. The symmetric difference  $A \triangle B$  is {Patrick O’Brian, Chinua Achebe, George Eliot, Lady Murasaki}. Jane Austen, who is in both  $A$  and  $B$ , is not in  $A \triangle B$ .

We can notice some set identities involving these new operations. It is never possible for a single element to be in both  $A \setminus B$  and  $B \setminus A$ , so we know that these two sets are disjoint — in symbols<sup>18</sup>,  $(A \setminus B) \cap (B \setminus A) = \emptyset$ . Furthermore, to be  $A \triangle B$  an element must either be in  $A$  but not  $B$ , or in  $B$  but not  $A$ . This means that the union of  $A \setminus B$  and  $B \setminus A$  is *the same set* as  $A \triangle B$  — we can write this set identity as

$$A \triangle B = (A \setminus B) \cup (B \setminus A).$$

**Definition:** Let  $A$  be a set of type  $T$ . The **complement** of  $A$ , written “ $\overline{A}$ ”, is the set of all things of type  $T$  that are not in  $A$  (in symbols,  $\overline{A} = T \setminus A$ ).

**Example:** In our example,  $\overline{A}$  is the set of all novelists that are not Patrick O’Brian, Chinua Achebe, or Jane Austen. We can’t write  $\overline{A}$  as a list without first having a list of all novelists, but we can say, for example, that Lady Murasaki and Fyodor Dostoevsky are members of  $\overline{A}$ . Set builder notation allows us to write  $\overline{A}$  as  $\{x : x \text{ is a novelist who is not in set } A\}$ , and in general this is the best way to specify such a set unless the whole type happens to be finite.

Note that if we change the type, we change the complement. Our set  $A$  can also be viewed as a set

<sup>18</sup>This is our first example of an expression involving more than one set operation. Just as in arithmetic, we have to use parentheses to tell which operation is to be executed first, unless we know that it doesn’t matter or we have some rule like the hierarchy of operations to tell us which to execute first. Until we know more about sets, we’d better put in enough parentheses to tell us exactly what to do.

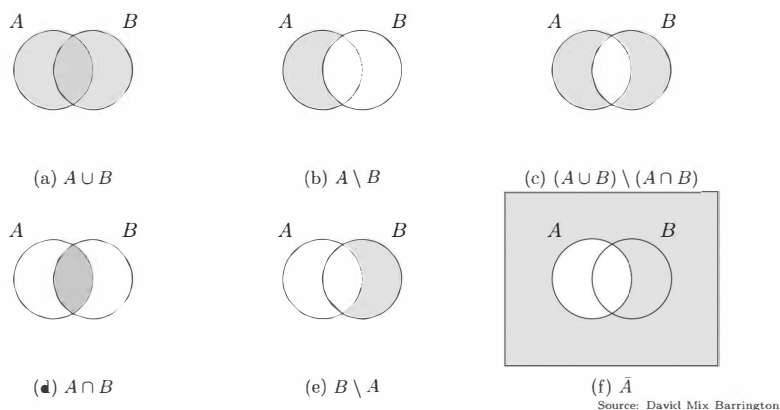


Figure 1-6: Venn diagrams for set operations.

of type *person*, and if we considered it that way then  $\bar{A}$  would also include all those human beings who are not novelists. In computer science settings we will generally know the type of a particular set, so this shouldn't be too much of a problem.

Note also that in order to take complements, we must refer to the set of all elements of the correct type. This set is important enough to be given a name:

**Definition:** Let  $T$  be a type. The **universal set**  $U$  is the set of all elements of type  $T$ .

In Venn diagrams, we sometimes represent the universal set by a box around all the other figures. Figure 1-6 illustrates all the set operations defined so far.

### 1.5.2 Propositions About Sets

Recall that in Section 1.1 we defined a *set identity* to be a statement about sets that is true for any sets. The set operations we've just defined give rise to many set identities, such as the example  $(A \setminus B) \cap (B \setminus A) = \emptyset$  that we justified above. In this subsection we'll see how to *translate* statements about sets into compound propositions, so that we will be able to *prove* set identities with the techniques to be developed later in the chapter.

We have defined a number of operations on sets, and a number of operators on boolean values. You may have noticed similarities between the two lists of operators, and this is no coincidence. There is a way to convert each use of a set operator into a use of a corresponding boolean operator, and vice versa. More precisely, these conversions operate on a particular kind of proposition — statements about things being in sets.

**Definition:** A **membership statement** is any statement that a particular thing is in a particular set, such as “ $x$  is an element of  $A$ ”, or equivalently “ $x \in A$ ”.

If we are considering  $k$  different sets, then there are  $k$  different basic membership statements we can make about a given element  $x$  –  $x$  might or might not be in each of the  $k$  sets. There are  $2^k$  different possible combinations of truth values for these  $k$  basic propositions. Thus in the next section when we use truth tables to argue about these propositions or with compound propositions involving them, we will have to consider  $2^k$  possible cases.

The set of all possible elements in the given type is thus divided into  $2^k$  parts, one part for each of the ways to assign truth values to the  $k$  membership statements. These  $2^k$  parts (some of which may be empty) correspond exactly to the  $2^k$  regions of a Venn diagram of the  $k$  sets. (See Problem 1.5.4 to learn about drawing Venn diagrams for any number of sets.) Statements about these sets and about set operations on them are implicitly statements about these  $2^k$  subsets, and thus can be interpreted as compound propositions involving the membership statements.

**Example:** Let’s look at the membership statement for the union of two sets  $A$  and  $B$ . What does it mean to say “ $x \in (A \cup B)$ ”? Well, we have defined  $A \cup B$  to be the set of things that are either in  $A$  or in  $B$  (or in both). So for a thing  $x$ , to be in  $A \cup B$  means exactly being in  $A$  or in  $B$  (or in both). The statement “ $x$  is in  $A$  or in  $B$  (or both)” is exactly what is meant by “ $(x \in A) \vee (x \in B)$ ”.

We have just given a proof! Because this fact about unions is something that we might want to look up later, we’ll record it as a “Fact”. The symbol  $\spadesuit$  at the end of the statement means that we are claiming that the statement doesn’t need any further proof (in this case, because we’ve just proved it in the Example). When we have a more complicated statement to prove, we’ll state it as a “Theorem”, follow it with a “Proof”, and put a  $\spadesuit$  at the end of the Proof.

**Fact:** Let  $A$  and  $B$  be sets. Then

$$A \cup B = \{x : (x \in A) \vee (x \in B)\}.$$

$\spadesuit$

Each of our other set operators matches up in the same way with a logical operator:

**Theorem:** Let  $A$  and  $B$  be sets. Then

$$\begin{aligned} A \cap B &= \{x : x \in A \wedge x \in B\} \\ \bar{A} &= \{x : \neg(x \in A)\} \\ A \triangle B &= \{x : x \in A \oplus x \in B\} \\ A \setminus B &= \{x : x \in A \wedge \neg(x \in B)\} \end{aligned}$$

**Proof:** In each of the four cases we can proceed just as in the Example for  $\cup$  and  $\vee$ . For example, the membership statement  $x \in (A \cap B)$  means that  $x$  is in both  $A$  and in  $B$ , by the definition of  $\cap$ .

This is the same thing as “ $(x \in A) \wedge (x \in B)$ ”. In the other three cases we just use the definitions of  $\bar{A}$ ,  $A \triangle B$ , and  $A \setminus B$  from earlier in this section. ♠

There are two special sets for which we can easily translate membership statements. The empty set  $\emptyset$  has no elements, so we can guarantee that the membership statement  $x \in \emptyset$  is false, whatever  $x$  is. Similarly, for every data type we have a universal set  $U$ , so that whenever  $x$  is a variable of that type we can guarantee that  $x \in U$  is true. We can summarize these two operations in another Fact.

**Fact:** Let  $x$  be a variable of some type  $T$ . Then:

$$\begin{aligned}\emptyset &= \{x : 0\} \\ U &= \{x : 1\}\end{aligned}$$

♠

When we say that two sets are equal, or that one is a subset of another, these statements translate as well into compound predicates made up from membership statements:

$$\begin{aligned}A = B &\iff ((x \in A) \leftrightarrow (x \in B)) \\ A \subseteq B &\iff ((x \in A) \rightarrow (x \in B))\end{aligned}$$

Also note that this last table uses two similar but different symbols,  $\leftrightarrow$  and  $\iff$ . We’ve already defined  $\leftrightarrow$  as a boolean operator, and  $\iff$  can be read as “is defined to be true exactly when”. One *is* a symbol in the propositional calculus, and the other is used to talk *about* the propositional calculus. For very good reasons, logicians have a long tradition of using different symbols for a logical system, on one hand, and the language used to talk about that logical system<sup>19</sup>.

By using the correspondences in this table, we can take any statement about sets (using these operators) and systematically convert it into a compound proposition. If we then prove that the proposition is a tautology, we can conclude that the set statement is a set identity. Here are two examples:

**Example:** We prove that  $A \setminus B$  is a subset of  $A$ . By the rule above, we need to prove the compound proposition

$$(x \in (A \setminus B)) \rightarrow (x \in A)$$

which translates (using the definition of the  $\setminus$  operator) to:

$$((x \in A) \wedge \neg(x \in B)) \rightarrow (x \in A).$$

<sup>19</sup>This distinction between “object language” and “meta-language” is another of Hofstadter’s main themes, starting from his first chapter.

Note that the two atomic propositions in this compound proposition are  $x \in A$  and  $x \in B$ . We can abbreviate these as  $a$  and  $b$  respectively and rewrite the compound proposition as  $(a \wedge \neg b) \rightarrow a$ . Is this compound proposition a tautology? In the next section we will learn a systematic method to verify this, but for now note that this implication must be true both if  $a$  is false (since then  $a \wedge \neg b$  is false) and if  $a$  is true (since  $a$  is the conclusion). So the statement  $(A \setminus B) \subseteq A$  is a set identity.

**Example:** If  $A$ ,  $B$ , and  $C$  are any three sets,  $A \setminus (B \cup C)$  is equal to the intersection of  $A \setminus B$  and  $A \setminus C$ . After all, if an element is in  $A$  but not  $B \cup C$ , it must both be in  $A$  and not in  $B$ , and in  $A$  but not in  $C$ . This means that the following set equality must always be true:

$$A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C).$$

By the definition of equality of sets above this set statement translates to the following:

$$[x \in (A \setminus (B \cup C))] \leftrightarrow [x \in ((A \setminus B) \cap (A \setminus C))].$$

Now again we convert these membership statements into compound propositions where the base propositions are membership statements about the original sets. By definition of  $\setminus$ , for example,  $x \in (A \setminus (B \cup C))$  translates to  $(x \in A) \wedge \neg(x \in (B \cup C))$ . Then  $x \in (B \cup C)$  becomes  $(x \in B) \vee (x \in C)$ . Applying the same process to the other side of the  $\leftrightarrow$ , we finally get:

$$[(x \in A) \wedge \neg((x \in B) \vee (x \in C))] \leftrightarrow [((x \in A) \wedge \neg(x \in B)) \wedge ((x \in A) \wedge \neg(x \in C))].$$

Again we make the compound proposition easier to read by replacing the base propositions  $x \in A$ ,  $x \in B$ , and  $x \in C$  by the boolean variables  $a$ ,  $b$ , and  $c$  respectively. This gives us:

$$[a \wedge \neg(b \vee c)] \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)].$$

This compound proposition is a tautology. In the following section we'll verify this systematically.

### 1.5.3 Exercises

E1.5.1 Let  $U$  be a particular set of farm animals including a set  $S$  of sheep, a set  $H$  of horses, a set  $B$  of black animals, and a set  $F$  of female animals, plus perhaps others not contained in any of these subsets. Give English descriptions of the following sets:

- (a)  $H \cap B$
- (b)  $F \cup (S \cap B)$
- (c)  $(F \cup S) \cap B$
- (d)  $\overline{F \cap H}$
- (e)  $F \setminus (S \cup H)$
- (f)  $H \triangle ((F \cup S) \setminus (B \cap \overline{F}))$

E1.5.2 Using the basic predicates in Exercise 1.5.1, give set expressions for the following sets:

- (a) The set of female animals that are not horses.
- (b) The set animals that are either female sheep or black horses.
- (c) The set of animals that are neither black nor sheep.
- (d) The set of animals that are either female black sheep, or are non-black horses, but not both.

E1.5.3 If  $A$  is the set  $\{x: x \text{ is a string with five or more letters}\}$  and  $B$  is the set  $\{y: y \text{ has two } a\text{'s in a row}\}$ , give set builder definitions of the following sets:

- (a)  $A \cup B$
- (b)  $A \cap B$
- (c)  $A \setminus B$
- (d)  $A \cup \overline{(A \cap B)}$

E1.5.4 Recall the following sets of naturals from Exercise 1.1.1:  $A = \{0, 2, 4\}$ ,  $B = \{1, 3, 5, 8\}$ ,  $C = \{2\}$ ,  $D = \{0, 5, 8\}$ ,  $E = \{x : x \text{ is even}\}$ . Describe the following sets, by a list or by set builder notation:

- (a)  $A \cup B$
- (b)  $A \cap B$
- (c)  $B \cup D$
- (d)  $B \cap D$
- (e)  $C \cup E$
- (f)  $A \cup \overline{E}$
- (g)  $B \setminus E$
- (h)  $E \setminus D$
- (i)  $\overline{C \cup B}$
- (j)  $(\overline{C}) \cup B$
- (k)  $B \triangle D$
- (l)  $A \triangle E$

E1.5.5 Draw a Venn diagram for the five sets of naturals  $A$ ,  $B$ ,  $C$ ,  $D$ , and  $E$  listed in Exercise 1.5.4 above. Of course you can't make individual dots for *all* the naturals, but use some sort of suggestive notation.

E1.5.6 Describe each of these sets of naturals in set builder notation:

- (a)  $\{0, 1, 2, \dots, 13\}$
- (b)  $\{23, 24, \dots, 134\}$
- (c)  $\{1, 11, 21, 31, \dots\}$
- (d)  $\{0, 1, 4, 9, 16, \dots\}$

E1.5.7 Is it ever possible for  $A \setminus B$  and  $B \setminus A$  to be equal? If so, under exactly what circumstances can it happen? If not, explain why not.



E1.5.8 Explain (informally, as in the Example above) why each of the following set identities is true for any sets  $A$  and  $B$ :

- (a)  $(A \cap B) \subseteq B$
- (b)  $(A \cap B) \subseteq (A \cup B)$
- (c)  $A \cap A = A$
- (d)  $A \cup A = A$
- (e)  $A \setminus A = \emptyset$
- (f)  $A \Delta A = \emptyset$
- (g)  $(A \Delta B) \cup (A \cap B) = A \cup B$

E1.5.9 Translate each of the set statements of Exercise 1.5.8 into a logical statement about elements. For example, part (a) translates to “ $(x \in A) \wedge (x \in B) \rightarrow (x \in A)$ ”.

E1.5.10 Informally explain why each of the logical statements of Exercise 1.5.9 is a tautology.

#### 1.5.4 Problems

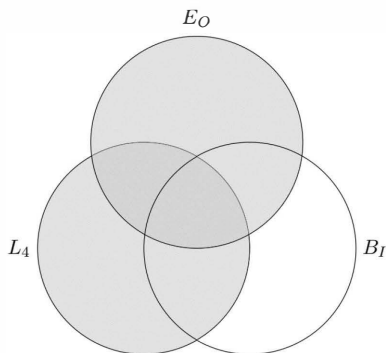
P1.5.1 Translate the following set statements into compound propositions using the base propositions  $x \in A$ ,  $x \in B$ , and  $x \in C$ :

- (a)  $((A \Delta B) \setminus C) = (A \setminus C) \Delta (B \setminus C)$ .
- (b)  $A \cup (B \cap C) \subseteq (A \cap C) \cup (B \cup A)$ .
- (c)  $((A \cap B) \cup C) \subseteq ((A \cup B) \cup C)$ .
- (d)  $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ .
- (e)  $A \Delta (B \setminus C) = (A \Delta B) \cup (A \cap C)$ .
- (f)  $\overline{(A \cup B) \cup C} = \overline{A} \cap (\overline{B} \cap \overline{C})$ .

P1.5.2 Which of the statements in Problem 1.5.1 are valid set identities? Informally justify the ones that are and give a counterexample (any example of three sets  $A$ ,  $B$ , and  $C$  making the statement false) for the others.

P1.5.3 Let  $S$  be the data type consisting of the fifty United States of America. Let  $L_4$  be the set consisting of all states whose names have exactly four letters. Let  $B_I$  be the set of states whose names begin with the letter “I”. Let  $E_O$  be the set of states whose names end with the letter “O”. Give lists to describe the following sets:

- (a)  $L_4 \cup E_O$ .
- (b)  $E_O \setminus (L_4 \cup B_I)$ .
- (c)  $(E_O \setminus L_4) \cup B_I$ .
- (d)  $B_I \cap E_O$ .
- (e)  $(B_I \cap E_O) \cup (B_I \Delta E_O)$ .
- (f)  $L_4 \cap \overline{(B_I \cup E_O)}$ .
- (g)  $B_I \cap \overline{(E_O)}$ .



Source: David Mix Barrington

Figure 1-7: A Venn diagram showing relationships among three sets, with  $L_4 \cup E_O$  shaded.

(h)  $B_I \setminus E_O$ .

P1.5.4 It is possible (see Figure 1.7) to arrange three loops to include all possibilities of elements being in some of the three sets and not others.

(a) In Figure 1.7, the three loops represent the sets  $L_4$ ,  $E_O$ , and  $B_I$  from Problem 1.5.3. The shading represents the elements in  $L_4 \cup E_O$ . Draw Venn diagrams shaded to represent the other seven sets from Problem 1.5.3.

(b) Can you draw *four* loops so that all the possible combinations of four sets are represented in a Venn diagram? How about more than four?

P1.5.5 A **Carroll diagram**<sup>20</sup> is another way to represent operations on sets. To represent two sets, draw a square cut horizontally into two parts (for  $A$  and  $\bar{A}$ ) and vertically into two parts (for  $B$  and  $\bar{B}$ ). The four little squares represent the four sets  $A \cap B$ ,  $A \cap \bar{B}$ ,  $\bar{A} \cap B$ , and  $\bar{A} \cap \bar{B}$ . For three sets, add another square in the middle, so that inside that square represents  $C$  and outside it represents  $\bar{C}$  (see Figure 1-8).

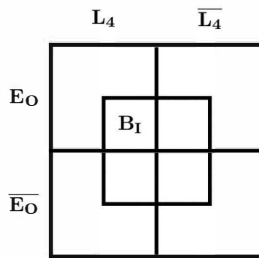
(a) Figure 1-8 shows a Carroll diagram for  $L_4 \cup E_O$  with base sets  $L_4$ ,  $E_O$ , and  $B_I$ . Draw Carroll diagrams for the other seven sets from Problem 1.5.3.

(b) How might you extend Carroll diagrams to show the relationships among four sets instead of three? How about five or more?

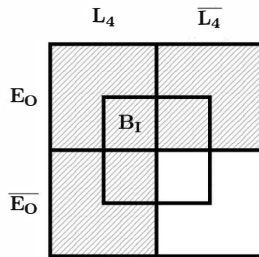
P1.5.6 Let  $\Sigma$  be the alphabet  $\{a, b, c, \dots, z\}$  and let  $U$  be the set  $\Sigma^3$  of three-letter strings with letters from  $\Sigma$ . Let  $X$  be the set of strings in  $U$  whose first letter is  $c$ . Let  $Y$  be the set of strings whose second letter is  $A$ , and let  $Z$  be the set of strings whose last letter is  $t$ . Describe each of the following sets in English, and determine the number of strings in each set.

(a)  $X \cap Y$

<sup>20</sup>Invented by logician and humorist Charles Dodgson, better known as Lewis Carroll, the author of *Alice in Wonderland*.



(a) A Carroll diagram



(b) Carroll diagram of the set  $L_4 \cup E_O$

Source: David Mix Barrington

Figure 1-8: Two Carroll diagrams, one showing the set  $L_4 \cup E_O$

- (b)  $X \cap Y \cap Z$
- (c)  $Y \cup Z$
- (d)  $X \cap (Y \cup Z)$

P1.5.7 Any subset statement may be interpreted as saying that some particular set is empty. Let  $D$  be a set of dogs, and let  $B$ ,  $R$ , and  $F$  be three subsets of  $D$  containing the black dogs, the retrievers, and the female dogs respectively. The statement  $B \subseteq R$ , for example, can be interpreted as “all the black dogs are retrievers”, or “there are no black dogs who are not retrievers”, that is,  $B \cup \overline{R} = \emptyset$ . For each of the following subset statements, identify the set that is claimed to be empty, both in English and in symbols:

- (a)  $B \cap F \subseteq R$
- (b)  $F \subseteq R \cap B$
- (c)  $B \subseteq R \cup F$

P1.5.8 (harder) If  $X$  is any set, define  $P_2(X)$  to be the set of all two-element subsets of  $X$ . For example, if  $X = \{a, b, c, d\}$ , then  $P_2(X)$  is the set  $\{\{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{c, d\}\}$ , containing six subsets.

- (a) We will say that a subset  $Y$  of  $P_2(X)$  **covers**  $X$  if every element of  $X$  is in exactly one of the subsets in  $Y$ . For  $X = \{a, b, c, d\}$ , the subset  $\{\{a, b\}, \{c, d\}\}$  covers  $X$ . Show that  $P_2(X)$  can be divided into three disjoint subsets, each of which covers  $X$ .
- (b) Now let  $X = \{a, b, c, d, e, f\}$ . Show that  $P_2(X)$  can be divided into five subsets, each of which has three elements, such that each of the subsets covers  $X$ .

P1.5.9 Following the definition in Problem 1.5.8, let  $X = \{a, b, c, d, e\}$  and let  $Y = P_2(X)$ . List the subsets in  $Y$  (there are exactly ten of them). How many of these subsets are disjoint from  $\{a, b\}$ ? How many are subsets of  $\{a, b\}$ ?

P1.5.10 Let  $X = \{1, 2, 3, \dots, 10\}$  and let  $I$  be the set of all **intervals** in  $X$ , that is, subsets of the form  $\{a, \dots, b\}$  for some naturals  $a$  and  $b$ .

- (a) How many intervals are in the set  $I$ ? (Don't forget the empty set.)
- (b) How many of the intervals are subsets of  $\{2, 3, 4, 5, 6\}$ ?
- (c) How many of the intervals are disjoint from  $\{2, 3, 4, 5, 6\}$ ?

## 1.6 Truth-Table Proofs

### 1.6.1 The Setting For Propositional Proofs

There is a single general situation where we want to use the propositional calculus. We have a number of base propositions, which we've named by a number of boolean variables, and we want to ask one or more questions of the following kind:

- Is a particular compound proposition  $P$  *always true* no matter what values the base propositions have? That is, is  $P$  a tautology?
- Does the truth of some compound proposition  $P$ , the **premise**<sup>21</sup>, force the truth of another compound proposition  $C$ , the **conclusion**? To prove this we need to show that it's impossible for  $P$  to be true and  $C$  to be false, meaning (by our definition of implication) that  $P \rightarrow C$  is a tautology.
- Are two compound propositions  $P$  and  $Q$  **equivalent**, meaning that they always have the same truth value? To prove this we need to show that  $P \leftrightarrow Q$  is a tautology.

Notice that each of these questions comes down in the end to showing that some compound proposition is a tautology. How do we go about doing this? There are two basic methods:

- Since a compound proposition is a tautology if and only if it is true *for all settings of the base propositions*, we can check all these settings in a systematic way. In this section we'll present the *method of truth tables*, which is one way to do this. This method always works, but it gets less and less efficient the more base propositions we have.
- As in high school geometry, we can construct a proof as a sequence of compound propositions, where each one is justified by some *proof rule*. In a **deductive sequence proof**, the proof rules guarantee the truth of the new statement, assuming the truth of one or more of the previous statements. In an **equational sequence proof**, the proof rules guarantee that the new statement is equivalent to one of the previous ones. In the following sections we'll present a set of rules and give some strategies for applying them to construct sequence proofs. When we can find a sequence proof, it is usually shorter than a truth table and often gives us a better idea of *why* the result is true. But we don't have a general method that is guaranteed to produce a sequence proof in all possible situations — we must apply creativity and experience to find one.

A **truth table** is a rectangular array with columns representing compound propositions and rows representing each possible choice of values for the base propositions. How many rows is this? If there is one base variable, we have two choices, **true** and **false**. (Because “ $T$ ” and “ $F$ ” tend to look alike in handwriting, and we're going to be writing these constants a lot, we'll represent them

---

<sup>21</sup>We can talk about a single premise, because if there were more than one we can make a single one by ANDING them together.

by “1” (true) and “0” (false).) If there are two variables, we have four choices: 00, 01, 10, and 11. For three variables we have eight choices — four by setting the first variable to zero (000, 001, 010, and 011) and four by setting it to one (100, 101, 110, 111). In general for  $n$  variables there are  $2^n$  choices, given by writing the numbers from 0 through  $2^n - 1$  in order *in binary notation*<sup>22</sup>.

### 1.6.2 Example of a Truth-Table Proof

We’ll demonstrate the method of truth tables on our sample compound proposition from the last section, which we derived there from a set identity:

$$[a \wedge \neg(b \vee c)] \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)].$$

Since there are three atomic variables, we set up eight rows in our truth table, for the eight choices from 000 to 111. At the top of the table we’ll write our formula, and below each occurrence of an atomic variable in the formula we’ll put the appropriate value for the row, copied from the left of the row:

| $a$ | $b$ | $c$ | $[(a \wedge \neg(b \vee c)]$ | $\leftrightarrow$ | $[(a \wedge \neg b) \wedge (a \wedge \neg c)]$ |
|-----|-----|-----|------------------------------|-------------------|------------------------------------------------|
| 0   | 0   | 0   | 0                            | 0                 | 0                                              |
| 0   | 0   | 1   | 0                            | 0                 | 0                                              |
| 0   | 1   | 0   | 0                            | 0                 | 0                                              |
| 0   | 1   | 1   | 0                            | 0                 | 0                                              |
| 1   | 0   | 0   | 1                            | 1                 | 0                                              |
| 1   | 0   | 1   | 0                            | 1                 | 0                                              |
| 1   | 1   | 0   | 1                            | 1                 | 0                                              |
| 1   | 1   | 1   | 1                            | 1                 | 1                                              |

Now we have to fill in the rest of the table. When we’re done, the final answer will appear under the operator that is evaluated last, the  $\leftrightarrow$ . (This one comes last because all the others are inside parentheses at a deeper level of nesting.) We proceed by taking the operators from the inside out and putting the result of that operation in the column directly under that operator. In this example, we can’t do the first  $\wedge$  because we don’t have its second argument yet, and we can’t do the first  $\neg$  because it requires that the first  $\vee$  be done first. So we do that  $\vee$ , filling out the column under it by taking the OR of the two booleans on either side of it in that row:

<sup>22</sup>Why should you believe this number is actually  $2^n$ , or that this trick of writing the numbers in binary always works? We’ll prove these facts in Chapter 4, but for now notice that we have a *recursive* method of listing the choices: set the first variable to zero, list all choices of the other variables, reset the first variable to one, and again list all choices of the other variables. Adding a new variable doubles the length of the list, and we can see (at least informally) that this operation takes a list of the binary numbers in order and gives back another binary list of double the size.

| $a$ | $b$ | $c$ | $[(a \wedge \neg (b \vee c)) \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)]]$ |
|-----|-----|-----|---------------------------------------------------------------------------------------------|
| 0   | 0   | 0   | 0                                                                                           |
| 0   | 0   | 1   | 0                                                                                           |
| 0   | 1   | 0   | 0                                                                                           |
| 0   | 1   | 1   | 0                                                                                           |
| 1   | 0   | 0   | 1                                                                                           |
| 1   | 0   | 1   | 1                                                                                           |
| 1   | 1   | 0   | 1                                                                                           |
| 1   | 1   | 1   | 1                                                                                           |

Now we can process the first  $\neg$ , by taking the bit *under the*  $\vee$  and reversing it. We take that bit because it represents the content of the entire phrase “ $b \vee c$ ” inside the parentheses:

| $a$ | $b$ | $c$ | $[(a \wedge \neg (b \vee c)) \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)]]$ |
|-----|-----|-----|---------------------------------------------------------------------------------------------|
| 0   | 0   | 0   | 0                                                                                           |
| 0   | 0   | 1   | 0                                                                                           |
| 0   | 1   | 0   | 0                                                                                           |
| 0   | 1   | 1   | 0                                                                                           |
| 1   | 0   | 0   | 1                                                                                           |
| 1   | 0   | 1   | 1                                                                                           |
| 1   | 1   | 0   | 1                                                                                           |
| 1   | 1   | 1   | 1                                                                                           |

Now we can process the first  $\wedge$ , because we finally have both its arguments. We go down that column, in each row writing down the AND of the value under the  $a$  and the value under the  $\neg$ :

| $a$ | $b$ | $c$ | $[(a \wedge \neg (b \vee c)) \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)]]$ |
|-----|-----|-----|---------------------------------------------------------------------------------------------|
| 0   | 0   | 0   | 0                                                                                           |
| 0   | 0   | 1   | 0                                                                                           |
| 0   | 1   | 0   | 0                                                                                           |
| 0   | 1   | 1   | 0                                                                                           |
| 1   | 0   | 0   | 1                                                                                           |
| 1   | 0   | 1   | 1                                                                                           |
| 1   | 1   | 0   | 1                                                                                           |
| 1   | 1   | 1   | 1                                                                                           |

So we go on, doing any operator as long as we have both its operands, until finally we can do the  $\leftrightarrow$ . The final table looks like this, verifying (since the  $\leftrightarrow$  column is all ones) that this formula is a tautology (and furthermore, that the two formulas on either side of the  $\leftrightarrow$  are logically equivalent):

| $a$ | $b$ | $c$ | $[(a \wedge \neg (b \vee c)) \leftrightarrow ((a \wedge \neg b) \wedge (a \wedge \neg c))]$ |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----|-----|-----|---------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0   | 0   | 0   | 0                                                                                           | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0   | 0   | 1   | 0                                                                                           | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0   | 1   | 0   | 0                                                                                           | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0   | 1   | 1   | 0                                                                                           | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 1   | 0   | 0   | 1                                                                                           | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1   | 0   | 1   | 1                                                                                           | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
| 1   | 1   | 0   | 1                                                                                           | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1   | 1   | 1   | 1                                                                                           | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |

So now we know in principle how to test whether any compound proposition is a tautology. This allows us to solve a wide variety of problems — any problem where the individual conditions can be modeled by booleans. There’s one important difficulty, however. As the number of base propositions gets larger, the number of lines in our truth table gets *really* larger. Even seven variables would give 128 lines, more than you probably want to do by hand. You could program this process on a computer, of course, but what if you have forty or fifty variables? The number  $2^{40}$  is about  $10^{12}$ , and  $10^{12}$  bits is a lot of memory to store your table<sup>23</sup>. Even if your individual operations are very fast, doing even one operation for each entry in the table is going to take a long time.

### 1.6.3 Exercises

- E1.6.1 Use a truth table to show that the compound proposition  $(p \oplus q) \leftrightarrow (\neg(p \rightarrow q) \vee \neg(q \rightarrow p))$  is a tautology.
- E1.6.2 Construct a truth table for the compound proposition  $p \leftrightarrow [(p \rightarrow q) \vee (q \wedge p)]$ . Is this compound proposition a tautology?
- E1.6.3 Construct a truth table for the compound proposition  $(p \wedge (q \vee r)) \oplus (r \oplus (p \wedge \neg q))$ . Use your result to draw a Venn diagram (see Section 1.5) for the set  $(P \cap (Q \cup R)) \triangle (R \triangle (P \cap \overline{Q}))$ .
- E1.6.4 Construct truth tables for the compound propositions  $p \vee (q \wedge r)$  and  $(p \vee q) \wedge r$  and use them to find a setting of the basic propositions for which these two compound propositions do not have the same truth value.
- E1.6.5 A truth table for two basic propositions has four lines, and thus  $2^4 = 16$  different possible columns of zeroes and ones. List these sixteen columns and find an explicit compound proposition giving rise to each.
- E1.6.6 The **if-then-else** function is a compound proposition on three variables, written  $ITE(x, y, z)$ . If  $x$  is true,  $ITE(x, y, z)$  is equal to  $y$ , and if  $x$  is false,  $ITE(x, y, z)$  is equal to  $z$ . Build a truth table for this function, showing for which of the eight setting of the variables it is true.
- E1.6.7 Here is a truth table purporting to show that  $p \rightarrow q$  and  $q \rightarrow p$  are logically equivalent. Since this is not true, there must be something wrong with the truth table. Where is the mistake?

<sup>23</sup>Since there are eight bits in a byte, you need about  $2^{40}/8$  bytes or about 137 gigabytes. This might be barely feasible, but what about  $2^{50}$  or  $2^{60}$ ?



|                     |                   |                     |
|---------------------|-------------------|---------------------|
| $(p \rightarrow q)$ | $\leftrightarrow$ | $(q \rightarrow p)$ |
| 0                   | 1                 | 0                   |
| 0                   | 1                 | 1                   |
| 1                   | 0                 | 0                   |
| 1                   | 1                 | 1                   |

- E1.6.8 We know that a compound proposition is a tautology if the truth table column for the final operation of the proposition is all ones. How would we know whether a compound proposition is a contradiction? How would we know whether it is satisfiable? (These terms are defined at the end of Section 1.3.)
- E1.6.9 Suppose I have a compound proposition with  $k$  different base propositions. A full truth table for this compound proposition would have  $2^k$  lines. How many lines would I need to evaluate if I know that a particular one of the base propositions is true? How many would I have to evaluate if I know that the implication  $p \rightarrow q$ , where  $p$  and  $q$  are base propositions, is false?
- E1.6.10 Normally, to evaluate a column of a truth table, I must look at the two columns on which it depends. But if the operator of the new column is  $\wedge$ , for example, I can fill in a 0 for the new column, without looking at the second old column, whenever the first old column has a 0. What is the similar rule if the operator of the new column is  $\vee$ ? What about  $\rightarrow$ ,  $\leftrightarrow$ , and  $\oplus$ ? In each case either give a rule to evaluate the new column based on just one of the old columns in some cases, or explain why no such rule exists.

### 1.6.4 Problems

- P1.6.1 (suitable for an Excursion) A compound proposition is said to be in **disjunctive normal form** or **DNF** if it is the OR of zero or more **clauses**, and each clause is the AND of one or more **literals**. (A literal is a base proposition or its negation.) For example,  $(\neg p \wedge q \wedge r) \vee (\neg q \wedge \neg r) \vee (r)$  is in DNF.

Any compound proposition is equivalent to one in disjunctive normal form, as we will now see.

- Explain how to construct, for any line of a truth table, a clause that is true for that line and false for all other lines.
- Illustrate this by giving a clause that is true for the last line of the truth table on four variables, where  $p$ ,  $q$ ,  $r$ , and  $s$  are all true, and for no other lines.
- Explain how any compound proposition is the OR of clauses of the type you have just constructed.
- Illustrate this process by giving a DNF compound proposition that is equivalent to  $(p \wedge (q \vee r)) \oplus (r \oplus (p \wedge \neg q))$ .

- P1.6.2 (suitable for an Excursion) A compound proposition is in **conjunctive normal form** or **CNF** if it is the AND of zero or more clauses, and each clause is the OR of one or more literals. For example,  $(\neg p \vee q \vee r) \wedge (\neg q \vee \neg r) \wedge (r)$  is in CNF. Any compound proposition is equivalent to some compound proposition in CNF. Explain why this is true and how to use a truth table to find this CNF version. (Hint: Given any line in the truth table, design a

clause that is false for that line and true for all the others.) Illustrate your method by finding a CNF compound proposition that is equivalent to  $(p \wedge (q \vee r)) \oplus (r \oplus (p \wedge \neg q))$ .

P1.6.3 In Problems 1.6.1 and 1.6.2 we defined DNF to include “the OR of zero clauses” and defined CNF to include “the AND of zero clauses”. What does this mean? What compound proposition is referred to in each case? (Hint: You need these two cases in order to be able to represent *any* compound proposition in CNF or DNF. What case would lead your conversion method to use no clauses?)

P1.6.4 A compound proposition is said to be **symmetric** if its truth value depends only on the *number* of basic propositions that are true, and not on *which ones* are true. For example,  $p \vee q \vee r \vee s$  is symmetric because it is false if none of the basic propositions are true, and true if one or more of them is true.

- Write a compound proposition that is true if and only if exactly two of the basic propositions  $p$ ,  $q$ ,  $r$ , and  $s$  are true.
- Suppose that you have a compound proposition with  $k$  basic propositions and you know that it is symmetric. Explain how you can evaluate it using a truth table with only  $k + 1$  lines rather than  $2^k$  lines. (Hint: Some of the lines in the ordinary truth table are now guaranteed to be identical.)

P1.6.5 (uses Java) Write a Java program that takes as input a number  $k$  and prints out  $2^k$  lines, labelled as for a truth table with the first line starting “0 0 ... 0” and the last line starting “1 1 ... 1”.

P1.6.6 The if-then-else operator defined in Exercise 1.6.6 can be used to express any compound proposition<sup>24</sup>. Suppose that  $f$  is a compound proposition involving an atomic variable  $p$  and some other variables. We can write  $f$  as  $ITE(p, f_1, f_0)$  where  $f_1$  is the compound proposition giving the value of  $f$  when  $p$  is true, and  $f_0$  gives the value of  $f$  when  $p$  is false. If we continue the process by expanding  $f_0$  and  $f_1$  using  $ITE$ ’s with other variables, we can get an expression for  $f$  that uses only  $ITE$  operators, with atomic variables as the first arguments and constants or  $ITE$ ’s as the second and third arguments. For example,  $p \wedge q$  can be written as  $ITE(p, q, 0)$  and then as  $ITE(p, ITE(q, 1, 0), 0)$ .

- Write an expression for  $p \oplus q$  using only the  $ITE$  operator, with atomic variables in the first argument and constants or  $ITE$ ’s in the other arguments.
- Write such an expression for  $(p \rightarrow q) \rightarrow (r \rightarrow s)$ .
- Explain how to use a truth table to construct such an expression for any compound proposition.

P1.6.7 The set  $D = \{b, c, d, l, n, p, s, w\}$  consists of the eight dogs Bingley, Cardie, Duncan, Lily, Nico, Pushkin, Scout, and Whistle. There are three subsets  $B = \{d, h, l, s\}$ ,  $F = \{c, l, p, s\}$ , and  $R = \{c, n, s, w\}$  of black dogs, female dogs, and retrievers respectively.

- Suppose  $x$  is one of the dogs in  $D$ . Indicate how you can determine which dog  $x$  is by asking three yes-or-no questions about  $x$ .
- Define six subsets of the naturals  $\{1, \dots, 64\}$ , each containing 32 numbers, such that you can determine any number  $n$  from the answers to the six membership questions for these sets.

<sup>24</sup>This construction is called **Boole’s Expansion Theorem** or **Shannon Decomposition**.

- (c) (harder) Is it possible to solve part (a) using three sets of dogs that do *not* have four elements each? What about part (b), with six sets that do not have 32 elements each? Explain your answer.

P1.6.8 Another use for truth tables is to take a set of compound propositions and determine which settings of the atomic variables, if any, make them all true. Excursion 1.9 has some more complex examples of such problems, but here are some simple ones using the three atomic variables  $p$ ,  $q$ , and  $r$ .

- (a) What is the only setting of the variables that makes  $p \rightarrow q$  and  $q \oplus r$  both false?  
 (b) What is the only setting of the variables that makes  $p \wedge (q \rightarrow r)$ ,  $r \rightarrow (p \wedge q)$ , and  $p \leftrightarrow (q \oplus r)$  all true?

P1.6.9 In an **islander problem**<sup>25</sup>, we are given a set of statements that are about one another. Each of a set of islanders makes a statement about the truth or falsity of their own or other islanders' statements. Each statement is either true or false, so we have a set of compound propositions with an atomic variable for each. Normally the problem is set up so that exactly one setting of the variables matches the variables. That is, each statement whose variable is set to true becomes true, and each one set to false becomes false. Given the following situations, which islanders, if any, are telling the truth?

- (a) There are two islanders.  $A$  says "Exactly one of us is lying".  $B$  says "If  $A$  is telling the truth, then so am I."  
 (b) There are three islanders.  $A$  says "It is not the case that both the other two are telling the truth."  $B$  says "At least one of the other two is telling the truth."  $C$  says " $A$  is telling the truth, and if  $B$  is telling the truth then so am I."

P1.6.10 In the game of **Twenty Questions**, one player thinks of an object (traditionally specified to be either "animal", "vegetable", or "mineral") and the other player tries to identify it by asking a series of questions, each of which can be answered "yes" or "no". The questioner's goal is to do so using twenty or fewer questions.

- (a) Suppose that the category is "vegetable", and the questioning player knows that the target object must be on a specific list of one million vegetable objects. Describe a strategy that will allow them to always win the game. (Hint: Give each object on the list a number.)  
 (b) (harder) Suppose that now that category is "mineral", and that there are 1.1 million objects on the mineral list. Argue that given any plan by the questioner, there must be at least one of the objects that they will fail to identify using twenty questions. (Hint: This is an example of **binary search**, and you may have seen an analysis of the limitations of binary search in a data structures course.)

---

<sup>25</sup>The classic setting for such problems is a remote island where some inhabitants always tell the truth and the others always lie. The logician Raymond Smullyan was famous for designing such puzzles to illustrate logical principles, as in his book *What is the Name of This Book?*.

## 1.7 Rules for Propositional Proofs

### 1.7.1 Equational Sequence Proofs

Any propositional tautology can be verified using the method of truth tables, but such a proof becomes impractical once the number of basic propositions grows beyond a very few. Furthermore, a truth-table proof in general gives us no insight as to *why* a particular statement is a tautology. We need another method to verify tautologies, one that offers the promise of shorter proofs and one that works with our intuition about the meaning of the propositional operators.

In chess, a beginner can spend an hour or two learning the rules that define which moves are legal, and then spend a lifetime learning how to make good moves in an actual game against an opponent. Propositional proofs are similar. In this section we present a set of proof rules that, when used correctly, yield valid propositional proofs. In the following section we'll discuss some strategies for constructing proofs.

**Example:** In Section 1.6 we used truth tables to verify the tautology  $[a \wedge \neg(b \vee c)] \leftrightarrow [(a \wedge \neg b) \wedge (a \wedge \neg c)]$ . Look at the left-hand side of this equivalence,  $a \wedge \neg(b \vee c)$ . Because an OR is false if and only if both its arguments are false, we can convert<sup>26</sup>  $\neg(b \vee c)$  into  $\neg b \wedge \neg c$ , changing the entire left-hand side of our proposition to  $a \wedge \neg b \wedge \neg c$ . This looks almost the same as the right-hand side. All we have to do is change the order of a bunch of literals that are ANDed together, and write the  $a$  twice. But in a series of ANDs, it doesn't matter what order they come in or how they are grouped by parentheses — the AND operation is **commutative** and **associative** just like addition or multiplication on numbers.

This argument is an example of our first type of propositional proof.

**Definition:** An **equational sequence proof** is a sequence of compound propositions, where each is equivalent to the one before it. A convenient way to write such a proof is the **two-column format** (usually used in high-school geometry classes) where we put the compound propositions in the left column and the justification for each step in the right column.

**Example:**

|                                              |                       |
|----------------------------------------------|-----------------------|
| $a \wedge \neg(b \vee c)$                    | Premise               |
| $a \wedge (\neg b \wedge \neg c)$            | DeMorgan's Law        |
| $(a \wedge a) \wedge (\neg b \wedge \neg c)$ | Identity Rule for AND |
| $a \wedge (a \wedge \neg b) \wedge \neg c$   | Associativity of AND  |
| $a \wedge (\neg b \wedge a) \wedge \neg c$   | Commutativity of AND  |
| $(a \wedge \neg b) \wedge (a \wedge \neg c)$ | Associativity of AND  |

You may well be willing to accept the validity of all the proof rules we used in this proof, but in general we need to know:

---

<sup>26</sup>This particular rule is called *DeMorgan's Law* and will be part of our list of rules later in this Section.

- How do we know when a rule is valid?
- Which rules should we keep at our fingertips?

The answer to the first question is easy. Any compound proposition that is a *tautology* is the basis of a valid rule. For example, we can verify with a two-line truth table that  $(a \wedge a) \leftrightarrow a$  is a tautology. This justifies our use of the Identity Rule for AND above.

But of course  $(b \wedge b) \leftrightarrow b$  is also a tautology, as is  $((p \rightarrow q) \wedge (p \rightarrow q)) \leftrightarrow (p \rightarrow q)$ . Once we have a tautology  $T$ , we can *substitute* any compound propositions for the basic propositions in  $T$  and get a new tautology. We just have to be careful of two things: we must always substitute the same compound proposition for the same basic proposition, and we must arrange parentheses so that the newly substituted compound propositions are treated as units. (See Problem 1.7.6 for examples of how you can get into trouble by violating these principles.)

Tautologies of the form  $P \leftrightarrow Q$ , where  $P$  and  $Q$  are compound propositions, give us rules that we may use in an equational sequence proof. For example, since  $(p \wedge q) \leftrightarrow (q \wedge p)$  is a tautology, we know that we can exchange the two sides of an AND operation – we say that AND is **commutative**. Similarly, since  $[p \wedge (q \wedge r)] \leftrightarrow [(p \wedge q) \wedge r]$  is a tautology, we can rearrange the parentheses in any sequence of formulas ANDed together – we say that AND is **associative**. The inclusive OR and the exclusive OR are also each commutative and associative, as we'll verify in Exercise 1.7.2 and Problem 1.7.5 below. But recall that these operations are not associative *with one another*: for example,  $p \wedge (q \oplus r)$  and  $(p \wedge q) \oplus r$  are not equivalent.

Note that what we are doing here is very similar to the process of manipulating equations in algebra. We have learned various rules that tell us, for example, that some polynomials are equal to other polynomials. In an algebraic derivation, we might start with one polynomial and apply a series of these rules to show that it is equal to another, using a series of intermediate polynomials that are all equal. As with our logical proofs, it is easy to learn what the valid rules are, but it takes practice to be able to apply the right sequence of rules to derive our equality efficiently.

### 1.7.2 Deductive Sequence Proofs

In our other type of propositional proof, we have more freedom: We don't need each statement to be equivalent to the last as long as each new statement is guaranteed to be true if all its predecessors are true. We gradually build up a set of statements that are true *if* the premises are true, and it we can add the conclusion to this set, we are done!

**Definition:** A **deductive sequence proof** is a sequence of compound propositions where each is implied by one or more of the propositions before it. That is, a new compound proposition  $Q$  is justified if for some earlier propositions  $P_1, P_2, \dots, P_k$ , the compound proposition  $(P_1 \wedge P_2 \wedge \dots \wedge P_k) \rightarrow Q$  is a tautology. We may also write deductive sequence proofs in the two-column format, with the propositions on the left and the justifications for each step on the right.

**Example:** We will show that if  $q$  and  $(p \vee q) \rightarrow (r \wedge s)$  are both true, then  $s$  must be true:

|                                          |                             |
|------------------------------------------|-----------------------------|
| 1. $q$                                   | Premise                     |
| 2. $(p \vee q) \rightarrow (r \wedge s)$ | Premise                     |
| 3. $p \vee q$                            | Right Joining, line 1       |
| 4. $r \wedge s$                          | Modus Ponens, lines 3 and 2 |
| 5. $s$                                   | Right Separation, line 4    |

Note that *any* tautology of the form  $P \rightarrow Q$  gives us a valid rule for use in deductive sequence proofs. The rule of **Modus Ponens**, from the tautology  $((P \rightarrow Q) \wedge P) \rightarrow Q$ , is the most common way to add a new statement – every implication is a tool that will give us a new true statement if its premise is true.

There is one subtle way in which we must be more careful with deductive sequence proofs. If  $P \leftrightarrow Q$  is a tautology, we may use an equational sequence step to replace  $P$  with  $Q$  *in any context at all*. For example, we may go from  $r \wedge \neg(s \oplus P)$  to  $r \wedge \neg(s \oplus Q)$  in one step. The same is *not* necessarily true in deductive sequence proofs. If  $P \rightarrow Q$  is a tautology, we may use a deductive sequence step to go from  $P$  to  $Q$ . But replacing  $P$  by  $Q$  in other contexts may or may not be valid. For example, we may replace  $r \vee (s \wedge P)$  by  $r \vee (s \wedge Q)$ , but not  $r \vee (s \oplus P)$  with  $r \vee (s \oplus Q)$ . In Problem 1.7.7 you will look at why this is.

### 1.7.3 Some Useful Tautologies

We conclude this section with a list of tautologies that are frequently useful in propositional proofs. A natural question to ask about any particular list is whether it provides all the rules needed to prove any tautology of the propositional calculus. It turns out, though we won't prove it here, that the system of rules corresponding to the following list of tautologies is *complete*, which means that it *can* prove any tautology<sup>27</sup>. There are other tautologies we might have included as well — in the Exercises you'll use these rules to verify some of these.

Let's begin with the basic algebraic properties of the operators  $\wedge$ ,  $\vee$ , and  $\oplus$ . Each operator is both commutative and associative, and has special behavior when used on an argument and a boolean constant, or an argument and itself. This gives us the following equivalences:

**Commutativity**  $(p \wedge q) \leftrightarrow (q \wedge p)$ ,  $(p \vee q) \leftrightarrow (q \vee p)$ ,  $(p \oplus q) \leftrightarrow (q \oplus p)$

**Associativity**  $(p \wedge (q \wedge r)) \leftrightarrow ((p \wedge q) \wedge r)$ ,  $(p \vee (q \vee r)) \leftrightarrow ((p \vee q) \vee r)$ ,  $(p \oplus (q \oplus r)) \leftrightarrow ((p \oplus q) \oplus r)$

**Zero Rules**  $(p \wedge 0) \leftrightarrow 0$ ,  $(p \vee 0) \leftrightarrow p$ ,  $(p \oplus 0) \leftrightarrow p$

**One Rules**  $(p \wedge 1) \leftrightarrow p$ ,  $(p \vee 1) \leftrightarrow 1$ ,  $(p \oplus 1) \leftrightarrow \neg p$

**Equal Operands**  $(p \wedge p) \leftrightarrow p$ ,  $(p \vee p) \leftrightarrow p$ ,  $(p \oplus p) \leftrightarrow 0$

<sup>27</sup>We already had a complete proof system for the propositional calculus, of course, in the method of truth tables, but this is another one that often has shorter proofs. The existence of a complete proof system is related to the property of “obviousness” that all tautologies of the propositional calculus share. The central subject of Hofstadter's book, *Gödel's Incompleteness Theorem*, is that there is *no* complete proof system for number theory. But then, as we'll see in Chapter 3, number theory can express statements that are not at all obvious.

**Distributive Laws**  $(p \wedge (q \vee r)) \leftrightarrow ((p \wedge q) \vee (p \wedge r)), (p \vee (q \wedge r)) \leftrightarrow ((p \vee q) \wedge (p \vee r)), (p \wedge (q \oplus r)) \leftrightarrow ((p \wedge q) \oplus (p \wedge r))$

Three of our boolean operators can be *defined* in terms of  $\wedge$ ,  $\vee$ , and  $\neg$ :

**Definition of Implication**  $(p \rightarrow q) \leftrightarrow (\neg p \vee q)$ .

**Definition of Equivalence**  $(p \leftrightarrow q) \leftrightarrow ((p \wedge q) \vee (\neg p \wedge \neg q))$ .

**Equivalence and Implication**  $(p \leftrightarrow q) \leftrightarrow ((p \rightarrow q) \wedge (q \rightarrow p))$ .

**Definition of Exclusive Or**  $(p \oplus q) \leftrightarrow ((p \wedge \neg q) \vee (\neg p \wedge q))$ .

Four implications follow from the basic meaning of  $\wedge$  and  $\vee$ :

**Left Separation**  $(p \wedge q) \rightarrow p$ .

**Right Separation**  $(p \wedge q) \rightarrow q$ .

**Left Joining**  $p \rightarrow (p \vee q)$ .

**Right Joining**  $p \rightarrow (q \vee p)$ .

Then there are the fundamental properties of  $\neg$ :

**Excluded Middle**  $p \vee \neg p$ .

**Double Negative**  $\neg \neg p \leftrightarrow p$ .

**DeMorgan And-To-Or**  $\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$ .

**DeMorgan Or-To-And**  $\neg(p \vee q) \leftrightarrow (\neg p \wedge \neg q)$ .

Finally, here are several tautologies involving implication, that will be useful in formulating proof strategies in the next section:

**Modus Ponens**  $((p \rightarrow q) \wedge p) \rightarrow q$

**Trivial Proof**  $q \rightarrow (p \rightarrow q)$

**Vacuous Proof**  $\neg p \rightarrow (p \rightarrow q)$

**Proof By Contradiction**  $(\neg p \rightarrow 0) \rightarrow p$

**Contrapositive Rule**  $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$

**Proof By Cases**  $((p \wedge r) \rightarrow q) \wedge ((p \wedge \neg r) \rightarrow q) \rightarrow (p \rightarrow q)$

**Hypothetical Syllogism**  $((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$

### 1.7.4 Exercises

E1.7.1 Verify, using truth tables, that the following statements, from our list of tautologies above, are actually tautologies:

- (a) Definition of Exclusive Or
- (b) Left Separation
- (c) DeMorgan And-To-Or
- (d) Proof By Cases
- (e) Proof By Contradiction
- (f) Hypothetical Syllogism

E1.7.2 We asserted above that  $\wedge$ ,  $\vee$ , and  $\oplus$  are each commutative and associative operations.

- (a) Verify the **Commutativity of AND** rule  $(p \wedge q) \leftrightarrow (q \wedge p)$  by a truth table.
- (b) Verify the **Commutativity of OR** rule  $(p \vee q) \leftrightarrow (q \vee p)$  by an equational sequence proof, using the Definition of Implication, Contrapositive, and Double Negation rules.
- (c) Verify the **Commutativity of Exclusive OR** rule  $(p \oplus q) \leftrightarrow (q \oplus p)$  by an equational sequence proof, using Commutativity of AND and OR and the Definition of Exclusive OR rule.
- (d) Verify the **Associativity of AND** rule  $((p \wedge q) \wedge r) \leftrightarrow (p \wedge (q \wedge r))$  by a truth table.
- (e) Verify the **Associativity of OR** rule  $((p \vee q) \vee r) \leftrightarrow (p \vee (q \vee r))$  rule using the DeMorgan rules and Associativity of AND.

E1.7.3 Give the tautologies produced by the following substitutions:

- (a) Substitute  $a \wedge \neg b$  for  $p$  and  $a \oplus b$  for  $q$  in the DeMorgan Or-To-And rule.
- (b) Substitute  $a \rightarrow b$  for  $p$ ,  $b \rightarrow a$  for  $q$ , and  $a \leftrightarrow b$  for  $r$  in the Hypothetical Syllogism rule.
- (c) Substitute  $a \wedge b$  for  $p$  and  $a \vee b$  for  $q$  in the Equivalence and Implication rule.

E1.7.4 Verify that each of the following compound propositions is a tautology by showing that it follows from one of our rules by substitution:

- (a)  $((a \vee \neg b) \wedge (c \rightarrow d)) \rightarrow (a \vee \neg b)$
- (b)  $(r \rightarrow \neg p) \vee \neg(r \rightarrow \neg p)$
- (c)  $((a \wedge b) \rightarrow b) \leftrightarrow (\neg b \rightarrow \neg(a \wedge b))$

E1.7.5 Prove that  $(p \leftrightarrow q) \leftrightarrow (\neg(p \oplus q))$  is a tautology by an equational sequence proof.

E1.7.6 Find an example of compound propositions  $P$  and  $Q$  such that  $P \rightarrow Q$  is a tautology but  $(r \oplus P) \rightarrow (r \oplus Q)$  is not a tautology. Remember that you can show that a compound proposition is not a tautology by providing a single line of its truth table in which it evaluates as false.

E1.7.7 Use the Definition of Implication Rule and the commutativity of OR to give an equational sequence proof of the Contrapositive Rule.

E1.7.8 Given the premises  $p \vee \neg q$  and  $\neg(q \rightarrow r)$ , give a deductive sequence proof of the statement  $p$ .



- E1.7.9 A rule sometimes called **Tertium Non Datur** (Latin for “there is no third option”) says that from the premises  $p \vee q$  and  $\neg p$ , you may derive the conclusion  $q$ . Give a deductive sequence proof of this rule.
- E1.7.10 Prove the first Distributive Law above by giving two deductive sequence proofs, showing that each half of the equivalence implies the other.

### 1.7.5 Problems

- P1.7.1 The **Modus Ponens** rule above says that if you know both  $p \rightarrow q$  and  $p$ , you may conclude  $q$ . In Lewis Carroll’s *What the Tortoise Said to Achilles*<sup>28</sup>, the character Tortoise accepts the Modus Ponens rule as valid in principle, but raises an objection to it in practice. In order to conclude  $q$  from  $p \rightarrow q$  and  $p$ , he demands that his friend Achilles write down the instance of Modus Ponens  $(p \wedge (p \rightarrow q)) \rightarrow q$ . Then to apply *that*, he demands to have  $(p \wedge (p \rightarrow q) \wedge (p \wedge (p \rightarrow q) \rightarrow q)) \rightarrow q$  written down, and so forth.
- Is Modus Ponens a valid rule? Can you explain why Achilles may use it without having to invoke it again every time he wants to justify a use of it?
- P1.7.2 Our list of tautologies shows how to define  $\oplus$ ,  $\rightarrow$ , and  $\leftrightarrow$  in terms of  $\wedge$ ,  $\vee$ , and  $\neg$ . Find and verify (by truth table or propositional proof) tautologies that define each of  $\wedge$ ,  $\vee$ ,  $\oplus$ , and  $\leftrightarrow$  in terms of  $\rightarrow$  and  $\neg$ .
- P1.7.3 The **Left Zero Rule for AND** is the tautology  $(0 \wedge p) \leftrightarrow 0$ . Verify this rule either by truth table or by equational or deductive sequence proof. Formulate correct Right Zero, Left One, and Right One rules for AND, then four analogous rules for OR. Justify all these rules. (Hint: Using commutativity and DeMorgan can save you some work.)
- P1.7.4 **Boolean algebra** is another way to present the rules governing 0 and 1 and the AND, OR, and NOT operations. If we let 0 and 1 be the only two numbers that exist, redefine  $1 + 1$  to be 1, and let all other additions and multiplications behave as for the naturals, we have a closed number system – the sum or product of any pair of numbers from the set  $\{0, 1\}$  is in this set.
- Show that if we interpret addition as OR, multiplication as AND, and the operation taking  $x$  to  $1 - x$  as NOT, this number system is the same as the boolean logic we have been studying.
  - The **AND-Over-OR Distributive Law** is the tautology  $(p \wedge (q \vee r)) \leftrightarrow ((p \wedge q) \vee (p \wedge r))$ . Verify this law in logic, and explain how it corresponds to the familiar distributive law in arithmetic.
  - Formulate and verify the analogous **OR-Over-AND Distributive Law** in logic. (Hint: Reduce it to the other distributive law with DeMorgan.) Translate this into a rule in arithmetic. Is this other distributive law valid for ordinary addition and multiplication on the naturals?
- P1.7.5 Verify the **Associativity of Exclusive OR** rule  $((p \oplus q) \oplus r) \leftrightarrow (p \oplus (q \oplus r))$  by first converting both sides to ANDs and ORs using the Definition of Exclusive OR rule, then using the distributive law and the commutativity and associativity rules.

<sup>28</sup>Reprinted in Hofstadter’s *Gödel, Escher, Bach*.

- P1.7.6 Suppose we substitute  $a \oplus b$  for  $p$  and  $a \wedge b$  for  $q$  in the contrapositive rule to get  $((a \oplus b) \rightarrow (a \wedge b)) \leftrightarrow ((\neg a \wedge b) \rightarrow (\neg a \oplus b))$ . Verify that this result is *not* a tautology. Why didn't our substitution lead to a valid tautology?
- P1.7.7 Sometimes it *is* valid to substitute  $Q$  for  $P$  in the middle of a larger compound proposition in a deductive sequence proof, when  $P \rightarrow Q$  is a tautology.
- Verify the tautologies  $(p \rightarrow q) \rightarrow ((p \wedge r) \rightarrow (q \wedge r))$  and  $(p \rightarrow q) \rightarrow ((p \vee r) \rightarrow (q \vee r))$ .
  - Suppose that  $P$  is a compound proposition that contains the variable  $x$  once and contains only AND and OR operators. Let  $Q$  be the compound proposition made from  $P$  by replacing  $x$  with another variable  $y$ . Explain informally why  $(x \rightarrow y) \rightarrow (P \rightarrow Q)$  is a tautology.
  - Give an example where  $P$  is a compound proposition containing  $x$  once,  $Q$  is a compound proposition containing  $y$  once, and  $(x \rightarrow y) \rightarrow (P \rightarrow Q)$  is not a tautology.
- P1.7.8 Define the **truecount** of a compound proposition to be the number of different settings of the basic propositions that make it true. (This is the number of 1's in its column of the truth table.) Thus if there are  $k$  basic propositions, the truecount is some number from 0 to  $2^k$ , inclusive.
- Find the truecount of  $p \rightarrow (q \rightarrow r)$ , assuming  $k = 3$ .
  - Note that the truecount will change depending on how many variables we consider. What is the truecount of  $p \rightarrow (q \rightarrow r)$  if we also consider the variable  $s$ , so that  $k = 4$ ? Explain why if we consider a variable not appearing in  $P$ , the truecount of  $P$  must be an even number.
  - What happens to the truecount of the successive statements in an equational sequence proof? Of the successive statements of a deductive sequence proof?
  - Give an example of two compound propositions  $P$  and  $Q$  on the same set of variables, such that  $P$  and  $Q$  have the same truecount but neither  $P \rightarrow Q$  nor  $Q \rightarrow P$  is a tautology.
- P1.7.9 Here we have another islander problem as in Problem 1.6.9. There are three islanders:  $A$  says "If  $B$  and  $I$  are both telling the truth, then so is  $C$ ",  $B$  says "If  $C$  is telling the truth, then  $A$  is lying", and  $C$  says "It is not the case that all three of us are telling the truth." Using propositional variables  $a$ ,  $b$ , and  $c$  to represent the truth of the statements of  $A$ ,  $B$ , and  $C$  respectively, we can represent the problem by the three premises  $a \leftrightarrow ((a \vee b) \rightarrow c)$ ,  $b \leftrightarrow (c \rightarrow \neg a)$ , and  $c \leftrightarrow \neg(a \wedge b \wedge c)$ . Determine the conclusion as a conjunction of three literals, and give a deductive sequence proof of this conclusion from the premises.
- P1.7.10 For each of the parts of Problem 1.6.8, state the conclusion as a conjunction of literals (a single clause, in the terminology of Problem 1.6.1), and give a deductive sequence proof that the premises imply the conclusion.

## 1.8 Propositional Proof Strategies

### 1.8.1 Direct Proof and the Forward-Backward Method

To come up with a propositional proof we must use valid rules in a valid way. Thus we must know a variety of rules and how to use them properly. This knowledge, however, does not tell us which rules to use and what order to use them in. Deciding this turns out to be a hard problem.

Computer programs called *theorem provers* exist that can in some cases make “intelligent” choices about how to carry out a proof in some formal system. But the general problem of finding the *shortest* proof of a given tautology, or even of finding a proof of reasonable length if one exists, is believed by most researchers to be *impossible* to solve in a reasonable amount of time<sup>29</sup>. The general methods that are known, such as the method of truth tables, take time that increases impractically quickly as the number of variables increases.

We can, however, learn some strategies to discover propositional proofs. The goal of an equational sequence or deductive sequence proof is to construct a chain of equivalences or of implications leading from some premise  $P$  to some conclusion  $C$ . (If we have more than one premise, we can for simplicity think of  $P$  as being the AND of them. If we have more than one conclusion, we can think of  $C$  as being *their* AND.)

The simplest way to prove an implication  $P \rightarrow C$  is called a **direct proof**. We **assume** that the premises are true, and begin applying deductive rules to get more statements that are **consequences** of the premises, which means that they must be true for any setting of the variables in which the premises are true. If we can get the conclusion as one of these consequences, we have completed our direct proof. Formally, we define the following:

**Fantasy Rule:** If we assume  $P$  and derive  $C$  from it by deductive rules, we have proved the implication  $P \rightarrow C$ .

Note that since the premises and all the consequences are guaranteed to be true in any setting where the premises are true, we also know this for any conjunction of these premises and consequences. Formalizing this, we have:

**Conjunction Rule:** If we have  $P$  and  $Q$  as premises or consequences, we may derive  $P \wedge Q$  as a consequence.

A chain of implications from  $P$  to  $C$  is called a **direct proof**, as contrasted with a proof goal that has been transformed as in Subsection 1.8.2. If we are very lucky, there may be a single rule that allows us to change  $P$  to  $C$ , and we can use it and be done. But in general we must look for a longer chain of rules.

In order to construct a direct proof, we must choose a deductive move to make. The **forward-backward method**<sup>30</sup> offers two suggestions as to which move we should choose:

<sup>29</sup>These researchers can't *prove* that it's impossible, though — for more on this see Chapter 15.

<sup>30</sup>This terminology is from Solow's *How to Read and Do Proofs*, which goes into proof strategies in much greater

- Use a rule to change  $P$  into some statement  $P'$ , then work on showing that  $P'$  implies  $C$ .
- Find some statement  $C'$  that can be used to get  $C$  in one step, then work on showing that  $P$  implies  $C'$ .

A good choice of  $P'$  or  $C'$  will bring you one step closer to finding the proof, but a bad choice could bring you one step further away. How can we increase the likelihood of a good choice? Perhaps we can choose  $P'$  to look more like  $C$  than  $P$  did. Perhaps  $P'$  is of a form more suitable for the use of one of our rules. Or perhaps we have tried all the other possibilities already, and changing  $P$  to  $P'$  looks like the only idea left. The right thing to do will always depend on the situation.

### 1.8.2 Transforming the Proof Goal

Another possible strategy is to *transform* the goal into another goal that is simpler or easier, but still suffices to prove the original goal. Some of our standard tautologies involving implication give us ways to do this:

**Vacuous Proof:** The rule is  $\neg p \rightarrow (p \rightarrow q)$ . If  $\neg P$  happens to be a tautology, then, we can prove *it* and use this rule to then prove  $P \rightarrow C$ .

**Trivial Proof:** Here the rule is  $q \rightarrow (p \rightarrow q)$ . Thus proving  $C$  by itself, without using  $P$ , is sufficient to prove  $P \rightarrow C$ .

The names of these two rules suggest that they are silly or useless, but that is not necessarily so. In most implications, it is possible for the premise to be true or for the conclusion to be false. In these cases trivial or vacuous proof will not work. But when we divide a problem into subproblems, as we'll soon see, one or more of the pieces may be trivially or vacuously true implications, and we need a way to dispense with these cases.

It may be convenient to transform our goal into an *equivalent* one that is more attractive to us. Our tautologies offer two ways to do this:

**Contrapositive Rule:** The rule says  $(\neg q \rightarrow \neg p) \leftrightarrow (p \rightarrow q)$ . If we like  $\neg C \rightarrow \neg P$ , for example because the premise  $\neg C$  is easier to derive things from, we can prove this and then conclude  $P \rightarrow C$ . A direct proof of  $\neg C \rightarrow \neg P$  is also called an **indirect proof** of  $P \rightarrow C$ .

**Proof By Contradiction:** Here the rule is  $(\neg p \rightarrow 0) \rightarrow p$ . If from  $\neg(P \rightarrow C)$ , or equivalently  $P \wedge \neg C$ , we can derive the always-false proposition 0, then  $P \rightarrow C$  must be a tautology.

Finally we have several ways to *subdivide* a goal into two or more subgoals, just as in programming we subdivide a complex task into simpler ones:

---

depth.

**Equivalence and Implication:** The rule  $(p \leftrightarrow q) \leftrightarrow ((p \rightarrow q) \wedge (q \rightarrow p))$  tells us that we can prove an equivalence by proving two implications, naturally dividing the goal into two subgoals.

**Proof By Cases:** The rule  $(((p \wedge r) \rightarrow q) \wedge ((p \wedge \neg r) \rightarrow q)) \rightarrow (p \rightarrow q)$  allows us to split the goal into two other subgoals, based on some third proposition.

**Hypothetical Syllogism:** The rule is  $((p \rightarrow r) \wedge (r \rightarrow q)) \rightarrow (p \rightarrow q)$ . This gives us another way to split the goal into two subgoals, by choosing some intermediate proposition.

One example of subdividing the goal comes from the film *Monty Python and the Holy Grail*. Sir Bedevere wants to prove the conclusion “She is a witch” from the premise “She weighs the same as a duck”. He chooses the syllogism method (very popular in the Middle Ages) with the intermediate proposition “She is made of wood”. Then he sets about trying to prove both “If she weighs the same as a duck, then she is made of wood” and “If she is made of wood, then she is a witch”<sup>31</sup>.

Once his listeners have accepted his arguments for these two subgoals, Sir Bedevere uses a balance scale to demonstrate his original premise and reaches his desired conclusion.

It’s important to remember that at any time we may happen to be working with premises that are *false*. For example, suppose we split the implication  $p \rightarrow q$  into the two subgoals  $(p \wedge r) \rightarrow q$  and  $(p \wedge \neg r) \rightarrow q$ , using Proof By Cases. We are guaranteed that at least one of these two premises is false in any actual setting of the variables, since  $r$  must be either true or false.

At a given point in our proof, a certain number of premises are *active*, meaning that we are assuming that they are true at this point. When we use a previously proved statement in a rule to prove a new statement, we must be sure that it follows from premises that are active at the time. We also have to make sure we don’t confuse our premises with our desired conclusions!

**Example:** From the premises  $p \vee (q \wedge r)$  and  $(p \vee r) \rightarrow s$ , we will prove  $s$ . Let’s use Proof By Cases:

---

<sup>31</sup>The division of the goal into subgoals is valid, though of course Sir Bedevere’s arguments for the subsidiary statements are rather suspect. For example he soon argues “Witches burn, and so does wood, so witches are made of wood.” The propositional calculus only yields correct conclusions if it is given correct premises!

|     |                                   |                                              |
|-----|-----------------------------------|----------------------------------------------|
| 1.  | $p$                               | Assumption for Proof By Cases                |
| 2.  | $p \vee r$                        | Left Joining, line 1                         |
| 3.  | $(p \vee r) \rightarrow s$        | Premise                                      |
| 4.  | $s$                               | Modus Ponens, lines 2 and 3                  |
| 5.  | $p \rightarrow s$                 | Fantasy Rule, lines 1-4, end of first case   |
| 6.  | $\neg p$                          | Assumption for Proof By Cases                |
| 7.  | $p \vee (q \wedge r)$             | Premise                                      |
| 8.  | $\neg p \rightarrow (q \wedge r)$ | Definition of Implication, line 7            |
| 9.  | $q \wedge r$                      | Modus Ponens, lines 6 and 8                  |
| 10. | $r$                               | Right Separation, line 9                     |
| 11. | $p \vee r$                        | Right Joining, line 10                       |
| 12. | $(p \vee r) \rightarrow s$        | Premise                                      |
| 13. | $s$                               | Modus Ponens, lines 11 and 12                |
| 14. | $\neg p \rightarrow s$            | Fantasy Rule, lines 6-13, end of second case |
| 15. | $s$                               | Proof By Cases, lines 5 and 14               |

We'll see more examples of Proof By Cases in Excursion 1.9.

### 1.8.3 Exercises

- E1.8.1 Use the Equivalence and Implication Rule to prove the **Left Zero Rule for AND**,  $(0 \wedge p) \leftrightarrow 0$ . (Hint: In one direction you can use a Vacuous Proof, and in the other Separation.)
- E1.8.2 Given the premises “At least one of witches, very small rocks, or churches float” and “Very small rocks do not float, and if witches float then so do ducks”, prove the conclusion “Either churches or ducks (or both) float”.
- E1.8.3 Suppose we are trying to prove  $(p \wedge q) \rightarrow (p \vee q)$ . For each of the following methods of subdividing this proof, give the two subgoal implications and then prove each of them.
- Proof By Cases with  $q$  as the new variable.
  - Hypothetical Syllogism with intermediate statement  $p$ .
- E1.8.4 Suppose again that we are trying to prove  $(p \wedge q) \rightarrow (p \vee q)$ . Give the transformed versions of this implication produced by the Contrapositive and Proof By Contradiction methods. Give deductive sequence proofs of each of these new implications.
- E1.8.5 In each of the following proof tasks, suggest a good *first* move in the proof according to the forward-backward method.
- From  $p \wedge \neg q$ , prove  $\neg p \rightarrow (q \vee r)$ .
  - From  $\neg(p \vee (q \vee r))$ , prove  $\neg r \vee (\neg p \wedge q)$ .
  - From  $q \wedge (\neg r \rightarrow \neg q) \wedge (r \rightarrow p)$ , prove  $p$ .
- E1.8.6 For each of the proof tasks in Exercise 1.8.5, suggest a good *last* move for the proof according to the forward-backward method.
- E1.8.7 In a famous Zen Buddhist *koan*, Ganto tells two monks “If you say a word, I will cut off your heads, and if you do not say a word, I will also cut off your heads.” Formulate this

as a compound statement and use Proof by Cases to derive “I will cut off your heads” as a consequence of it.

E1.8.8 Here are some questions about the final proof example:

- (a) What was the motivation for the step taken at line 2?
- (b) Why is the proof not complete after  $s$  has been proved on line 4?
- (c) What was the motivation for the step taken at line 8?
- (d) Why does line 12 restate the premise instead of using line 3?

E1.8.9 Use Hypothetical Syllogism to get the result of the final proof example, using “ $p \vee r$ ” as the intermediate statement.

E1.8.10 Suppose I am trying to prove  $P \rightarrow C$ , and I have succeeded in proving it with the additional assumption  $q \leftrightarrow r$ , that is, I have proved  $((P \wedge (q \leftrightarrow r)) \rightarrow C$ . What more do I need to do to complete my proof?

#### 1.8.4 Problems

P1.8.1 Carry out deductive sequence proofs of the following implications using the specified proof methods:

- (a)  $(p \wedge \neg q \wedge \neg r) \rightarrow (p \vee q)$ , by direct proof.
- (b) The same, by indirect proof (the Contrapositive Rule).
- (c) The same, by contradiction.
- (d)  $[(p \vee q) \wedge (\neg p \vee \neg q) \wedge (p \leftrightarrow q)] \rightarrow r$ , by vacuous proof.
- (e)  $(p \oplus q \oplus r) \rightarrow (p \vee (q \oplus r) \vee (r \rightarrow q))$ , by trivial proof.
- (f)  $[(p \rightarrow q) \wedge (r \rightarrow q) \wedge (p \rightarrow r)] \rightarrow \neg(p \wedge r \wedge \neg q)$ , using proof by cases with cases  $p$  and  $\neg p$ .
- (g)  $(p \wedge (q \oplus r)) \rightarrow (q \vee r)$ , by Hypothetical Syllogism with intermediate step  $r \oplus q$ .

P1.8.2 A variant of the Proof By Cases rule is as follows: Given the premises  $p \vee q$ ,  $p \rightarrow r$ , and  $q \rightarrow r$ , derive  $r$ .

- (a) Formulate the tautology that justifies this rule.
- (b) Prove this tautology using ordinary Proof By Cases, choosing your own intermediate statement and giving deductive sequence proofs for each of the two cases.

P1.8.3 Suppose that you have proved 0 from the premise  $P \wedge \neg Q$ . Show how you can use Proof By Cases and this proof to construct a direct proof of  $P \rightarrow Q$ .

P1.8.4 Here you will complete a famous proof, known to the ancient Greeks, that the number  $\sqrt{2}$  is irrational (that it cannot be expressed as  $p/q$  where  $p$  and  $q$  are naturals). Suppose that  $\sqrt{2} = p/q$  and that the fraction  $p/q$  is in lowest terms. Then by arithmetic,  $p^2 = 2q^2$ . Now use Proof By Cases, where the new proposition is “ $p$  is even”. Derive a contradiction in each case, and argue using Proof By Contradiction that  $\sqrt{2}$  is irrational. (Remember, as we will show formally in Chapter 3, that a natural is even if and only if its square is even.)

- P1.8.5 Here is an example of a Proof By Cases that has raised philosophical issues for some mathematicians, because it proves that two numbers exist without determining what they are. The statement proved is “There are two real numbers  $x$  and  $y$  such that  $x$  and  $y$  are irrational and  $x^y$  is rational”. The new proposition is “ $\sqrt{2}^{\sqrt{2}}$  is rational”. Show that the conclusion follows under the assumption that the new proposition is true, and under the assumption that it is false. (You may quote the result of Problem 1.8.4, that  $\sqrt{2}$  is irrational.)
- P1.8.6 The rule of **Modus Tollens** says that from the premises  $p \rightarrow q$  and  $\neg q$ , you may derive the conclusion  $\neg p$ .
- Prove this rule by using the Contrapositive Rule to transform the first premise.
  - Prove this rule by Contradiction, deriving a contradiction by assuming the two premises and the negation of the conclusion.
  - Prove this rule by Proof By Cases, first assuming  $p$  and then assuming  $\neg p$ .
- P1.8.7 Prove that the compound propositions  $p \wedge (q \rightarrow r)$  and  $\neg(p \rightarrow (q \wedge \neg r))$  are equivalent by using the Equivalence and Implication Rule and constructing two deductive sequence proofs.
- P1.8.8 Consider the islander problem of Problem 1.7.9. For each of the three islanders, either derive a contradiction from the assumption they are telling the truth, or derive a contradiction from the assumption that they are lying.
- P1.8.9 Consider a finite set  $S$  of atomic variables, and a set of premises  $P$ , each of the form  $x \rightarrow y$ , where  $x$  and  $y$  are each literals (members of  $S$  or their negations). If two of these premises are  $x \rightarrow y$  and  $y \rightarrow z$ , for example, we can derive the consequence  $x \rightarrow z$  by Hypothetical Syllogism. Suppose that we exhaustively check every pair of premises and consequences, and add any new consequences we can derive in this way, until we can go no further.
- Argue informally that this process cannot go on forever.
  - Suppose we derive a consequence  $x \rightarrow \neg x$ , where  $x$  is a literal. What conclusion can we draw about settings of the variables making all the premises true?
  - Suppose we derive both  $x \rightarrow \neg x$  and  $\neg x \rightarrow x$ ? What conclusion can we then draw about settings making all the premises true?
  - Suppose that the process stops without the situation in part (c) occurring. Argue that there is at least one setting of the variables making all the premises true.
- P1.8.10 In Excursion 1.3 we proved by contradiction that every natural is either even or odd. Using the Least Number Axiom, if there is any natural that is neither, there must be some natural  $x$  that is neither. Furthermore, since 0 is even, if such an  $x$  exists it must have a predecessor  $x - 1$ . Consider the atomic propositions  $xe$  for “ $x$  is even”,  $xo$  for “ $x$  is odd”,  $pe$  for “ $x - 1$  is even”, and  $po$  for “ $x - 1$  is odd”. We can derive some compound statements, involving these atomic variables, from the premises “ $x$  is neither even nor odd”, “ $x - 1$  is either even or odd, or both”, “if  $x - 1$  is even, then  $x$  is odd”, and “if  $x - 1$  is odd, then  $x$  is even”.
- Write these compound propositions symbolically and derive a contradiction from them. Explain why this proves that every natural is either even or odd.
  - With the same atomic propositions, formulate a similar proof that no natural is both even and odd.



## 1.9 Excursion: A Murder Mystery

Detectives are famous for the use of deductive reasoning. “When you have eliminated the impossible,” said Sherlock Holmes, “whatever remains, however improbable, must be the truth”. The case at first appears to offer a vast number of possibilities for what happened, but the clues eliminate some of these possibilities and may allow us to draw some conclusions.

In this Excursion we offer some examples of propositional deduction in the form of a murder mystery<sup>32</sup>. In our first example, the possible situations are modeled by four boolean variables, and we are given four clues. It happens that *exactly one* of the  $2^4 = 16$  combinations of truth values for the variables is consistent with all the clues. It would be straightforward but tedious to find out which with a truth table – our job is to use the rules of propositional logic to get the answer more quickly.

- The killer was Professor Plum ( $p$ ) or Colonel Mustard ( $\neg p$ ), but not both.
- The murder happened in the kitchen ( $k$ ) or the study ( $\neg k$ ), but not both.
- The murder weapon was a candlestick ( $c$ ), a baseball bat ( $b$ ), both ( $c \wedge b$ ), or neither ( $\neg c \wedge \neg b$ ).
- (I) If Professor Plum did it or if it was with the candlestick, then it was not with the baseball bat ( $(p \vee c) \rightarrow \neg b$ ).
- (II) If it was in the study, the baseball bat was used ( $\neg k \rightarrow b$ ).
- (III) If it was in the kitchen, then it was Colonel Mustard with the candlestick ( $k \rightarrow (\neg p \wedge c)$ ).
- (IV) If it was with the candlestick, then it was Professor Plum in the study ( $c \rightarrow (p \wedge \neg k)$ ).

Where do we start? Since we don’t yet know the conclusion of our proof, we can’t use the forward-backward method in its standard form. But we can begin with the forward part, looking at what interesting compound statements we might be able to derive from the clues.

Since the clues are in the form of implications, it would be easier if we had some premises to start with. We could start with any tautology. Here’s an idea – we have both  $k$  and  $\neg k$  on the left-hand side of rules. Maybe we can prove some conclusion in the case where  $k$  holds *and* in the case where  $\neg k$  holds, and thus prove this conclusion by cases. Let’s start by seeing what we can derive from  $k$ :

|                                              |                                          |
|----------------------------------------------|------------------------------------------|
| $k$                                          | Premise                                  |
| $\neg p \wedge c$                            | Modus Ponens, Clue III                   |
| $c$                                          | Right Separation                         |
| $p \wedge \neg k$                            | Modus Ponens, Clue IV                    |
| $(\neg p \wedge c) \wedge (p \wedge \neg k)$ | Conjunction                              |
| $(\neg p \wedge p) \wedge (c \wedge \neg k)$ | Commutativity, Associativity of $\wedge$ |
| $0 \wedge (c \wedge \neg k)$                 | Variant of Excluded Middle               |
| $0$                                          | Left Zero                                |

---

<sup>32</sup>Inspired by the board game *Clue*, a trademark of Parker Brothers games.

The assumption that it was in the kitchen leads to a contradiction. We've thus proved by contradiction that if the clues hold, the murder was in the study. We can now take this and run with it!

$\neg k$  Conclusion from above  
 $b$  Modus Ponens, Clue II

So it was the baseball bat! What more can we learn from this? We don't have  $b$  on the left-hand side of a rule, but we have  $\neg b$  on the right-hand side, which is just as good if we take the contrapositive:

$b \rightarrow \neg(p \vee c)$  Contrapositive of Rule I  
 $\neg(p \vee c)$  Modus Ponens, last two lines  
 $\neg p \wedge \neg c$  DeMorgan Or-to-And  
 $\neg k \wedge b \wedge \neg p \wedge \neg c$  Conjunction

So it was Colonel Mustard, in the study, with the baseball bat. Actually there is one other possibility that we should rule out before we make this our final conclusion. It might be that *no* setting of the four variables makes all the clues true<sup>33</sup>. We should check that our deduced setting satisfies all four clues. But it does: the premise  $p \vee c$  of Clue I is false, the conclusion  $b$  of Clue II is true, and both the premise  $k$  of Clue III and the premise  $c$  of Clue IV are false. So we have found the *unique* setting of the variables satisfying the clues.

### 1.9.1 Writing Exercises

Here are two other murder mysteries for you to solve.

- Here the meanings of the four basic propositions  $b$ ,  $c$ ,  $k$ , and  $p$  are exactly the same as in the example above, given by the first three unnumbered statements. Your new clues are:
  - Clue I: If the candlestick was used, then Professor Plum did it.
  - Clue II: If Professor Plum did it, then he did not use either the baseball bat or the candlestick.
  - Clue III: If the baseball bat was not used, then Professor Plum did it in the kitchen.
  - Clue IV: If the candlestick was not used, then the baseball bat was used in the study.
- In this case we have six boolean variables to worry about. The murderer may have been Colonel Mustard ( $m$ ), Professor Plum ( $p$ ), or Miss Scarlet ( $s$ ), or any combination of all or none of these. Again the location was either the kitchen ( $k$ ) or the study ( $\neg k$ ) but not both. The weapon may have been the bat ( $b$ ), the candlestick ( $c$ ), both, or neither. You have six clues:

Your clues are:

- Clue I: If the candlestick was used, it was in the kitchen and Colonel Mustard was involved.

---

<sup>33</sup>This would mean that in the real world, one of the clues is false – detectives do sometimes make mistakes!

- (b) Clue II: If either Professor Plum is innocent or Miss Scarlet is guilty, then either the bat or the candlestick, or both, was used.
- (c) Clue III: If it was Professor Plum with the candlestick, then Colonel Mustard is innocent.
- (d) Clue IV: If Miss Scarlet is innocent, either Professor Plum was involved or it was done in the study.
- (e) Clue V: If the baseball bat was used, then it was in the study and Professor Plum is innocent.
- (f) Clue VI: Unless<sup>34</sup> it was Miss Scarlet in the kitchen, the candlestick was used.

---

<sup>34</sup>You may want to reformulate Clue VI as an equivalent statement using the words “if” and “then” in place of “unless”.

## 1.10 Predicates

### 1.10.1 Statements That Include Variables

We have seen how propositions can be made up of smaller propositions combined with boolean operators, and how by modeling each smaller proposition by a boolean variable, we can represent the larger statement as a compound proposition. By using rules that apply to all booleans, then, we can carry out propositional proofs that relate these compound propositions to each other.

Propositional logic views any proposition as something that can be true or false. Any two false propositions, such as “*aaa* is a string of two letters” and “ $\lambda$  is a string of two letters”, are considered to be the same. For many purposes, this is an oversimplification. If we were to use these two particular propositions in an argument, we would probably be interested in what they *have in common* other than both being false. Each of them makes *the same kind of statement* about a string.

**Definition:** A **predicate** is a statement that *would become* a proposition if the values of some variables were supplied. The variables whose values are necessary to determine the predicate’s meaning are called **free variables**.

**Example:** Let us define “ $P(w)$ ” to mean “ $w$  is a string of two letters”, and write our two statements above as “ $P(aaa)$ ” and “ $P(\lambda)$ ”. This notation captures the fact that similar claims are being made about two different strings. The statement  $P(w)$  by itself is not a proposition. It is neither true nor false until a value for  $w$  has been supplied. (Here we are using “ $w$ ” as a variable of type `String` rather than as a string of one letter. Also recall that  $\lambda$  is a constant of type `String`, representing the empty string.)

In general we define a predicate in just this way:

- We give the predicate’s **signature**, indicating how many free variables there are and what order they come in, and then
- We give a **template**, a statement about the objects denoted by the free variables. Usually the template makes clear what type the free variables have.

When we are given a predicate with specific values supplied (called the **arguments**), we determine its meaning by *substituting* these values for the corresponding free variables in the template. It is important that each argument be substituted for the correct free variable according to the order in which the arguments occur and in which the free variables occur in the signature. For example, if  $L(u, v)$  is defined by the template “string  $u$  is longer than string  $v$ ”, then the proposition  $L(aaa, ba)$  must be interpreted as “string *aaa* is longer than string *ba*”, *not* “string *ba* is longer than *aaa*”. If a free variable occurs more than once in the template, the same argument must be substituted for it each time.

### 1.10.2 Predicates Viewed as Functions

In fact a predicate is nothing more nor less than a *function* whose output type is boolean. Both mathematical functions and Java methods have signatures, because they must receive the correct number of inputs, each of the correct type. Both Java methods and mathematical functions also have templates that determine how to evaluate the function value once suitable inputs are given.

In Java the signature of a function is given when the function is defined. If we want a predicate (that is, a boolean method) named `isCloseTo` whose arguments are an `int` and a `float`, then the first line of the method declaration might look like this:

```
boolean isCloseTo (int n, float x) {
```

The variables `n` and `x` are called the **formal parameters** of the method. This first line would be followed by the template. The template of a Java function is simply the code that tells the interpreter how to calculate the function value once it is given the input values (the **actual parameters**). For example, we might calculate whether the two values are within 0.01 of one another:

```
    // return whether x is within 0.01 of n
    float diff = Math.abs(n-x);
    return (diff < 0.01);
} // end of isCloseTo
```

When a call to this method occurs, the interpreter executes the template code, supplying the values of the actual parameters whenever the code calls for the formal parameters.

In the case of a mathematical function the template is some expression into which we substitute the variables. For example, if the real-valued function  $f$  is defined by the expression  $f$  to be  $x^2 + 2xy$ , we know that in evaluating  $f(3, 4)$  we substitute 3 for  $x$  and 4 for  $y$ . The template of a mathematical predicate is an expression that ultimately has a boolean value, such as " $x^2 + 2xy \geq 6$ ". If we call this predicate  $P(x, y)$ , we evaluate  $P(3, 4)$  by again substituting 3 for  $x$  and 4 for  $y$  to get " $3^2 + 2 \cdot 3 \cdot 4 \geq 6$ ". Since this inequality is a true statement,  $P(3, 4)$  is true.

The most common ways to form an algebraic expression that has a boolean value is to make an **equality** or an **inequality** statement. The  $=$  sign can be thought of as a binary operator that takes two arguments, the left-hand side and the right-hand side, and returns a boolean value that is true if and only if the two sides are equal. The operators  $<$ ,  $\leq$ ,  $>$ , and  $\geq$  behave similarly, returning true if and only if the two sides have the appropriate order relationship. By making equality or inequality statements we can form predicates out of any arithmetic operation, such as the three-place predicate " $x + y = z$ ", or even out of any function, as in the two-place predicate " $y = f(x)$ ".

It should be noted that the arithmetic, equality, and inequality operators are **overloaded** both in typical mathematical writing and in Java. The equality predicate on real numbers is different from

the equality predicate on naturals, because they have different signatures and thus take arguments of different types. In Java the `==` operator is used in both cases, and the interpreter determines from context which operation is meant. A programmer can define several methods with the same name and different signatures, again relying on the interpreter to choose the right meaning according to the context. Mathematical writing is very similar. We decide informally how much information is needed to allow the reader to determine exactly what is meant by each operation we use.

Note that we've actually used predicates already in this book, when we defined set builder notation. Translated into our new terminology, what we said was that we could define a set by naming a variable and then giving a one-place predicate with that variable free, all inside braces. So from the one-place predicate on strings, " $w$  is a string of two letters", we can form the set (or language)  $\{w : w \text{ is a string of two letters}\}$ . The set thus defined is a subset of the type of the variable, consisting of those elements of the type that make the predicate true.

Many operators give us additional examples of predicates. When we define " $x \leq y$ " to mean " $x$  is less than or equal to  $y$ ", we're defining a statement that becomes a proposition when we know the values of the free variables  $x$  and  $y$ . For each type on which  $\leq$  is defined, we have a predicate taking two free variables of that type.

### 1.10.3 Exercises

E1.10.1 Give a formal definition with a signature and template for the predicates defined by each of the following statements:

- (a) "The first real number is between the second real number and the third real number."
- (b) " $x^2 + y^2 = 1$ "
- (c) "The first three runners were all from team  $X$ "
- (d) "His batting average and on-base percentage are both better than those of the center fielder for that team."

E1.10.2 Let  $\Sigma = \{a, b\}$ . Define the following predicates on strings:  $P(w)$  means " $w$  is a string of two letters",  $Q(w)$  means " $w$  starts and ends with the same letter", and  $R(v, w)$  means " $v$  and  $w$  start with the same letter". Translate the following symbolic statements into English, and determine whether they are true or false:

- (a)  $R(a, ab) \wedge [P(ba) \oplus Q(aab)]$
- (b)  $R(a, a) \rightarrow [Q(aaa) \vee (P(b) \leftrightarrow R(b, aba))]$
- (c)  $P(a) \rightarrow (Q(aa) \wedge (R(b, bbb) \oplus P(\lambda)))$

E1.10.3 Again let  $\Sigma = \{a, b\}$ . Using the predicates defined for Exercise 1.10.2 above, translate the following English statements into symbols. Identify the free variables, if any, in each statement.

- (a) If  $ab$  has two letters, and a string  $w$  starts with the same letter as  $ab$ , then  $w$  starts and ends with the same letter.
- (b) If either  $aba$  starts and ends with the same letter, or it starts with the same letter as  $bab$ , but not both, then  $aa$  starts with the same letter as  $bb$  and has two letters.

- (c) The strings  $u$  and  $v$  start with the same letter unless either  $u$  or  $v$ , but not both, has two letters and starts and ends with the same letter.
- E1.10.4 In Java a method may have an empty argument list, meaning that it takes no arguments. Can a predicate have an empty argument list? What do you call such a predicate?
- E1.10.5 Write a pseudo-Java method that evaluates the “between” predicate on real numbers from Exercise 1.10.1 (a). Note that a real is not between any real and itself, and that  $a$  can be between  $b$  and  $c$  if either  $b < c$  or  $c < b$ .
- E1.10.6 Let  $M$  be the set of men who live in Seville and let  $b$  be one of them, the Barber of Seville. Define the predicate  $S(s, y)$ , where the type of  $x$  and  $y$  is  $M$ , to mean “ $x$  shaves  $y$ ”.
- Give a symbolic template for a predicate  $P(x)$  that means “ $b$  shaves  $x$  if and only if  $x$  does not shave himself”.
  - Argue that  $P(b)$  must be false.
- E1.10.7 Let  $D$  be a set of dogs, containing Cardie ( $c$ ), Duncan ( $d$ ), and perhaps others. Define the predicates  $B(x)$ ,  $F(x)$ , and  $R(x)$  to mean “dog  $x$  is black”, “dog  $x$  is female”, and “dog  $x$  is a retriever” respectively. Translate the following statements into symbols:
- It is not the case that if Cardie is a retriever, then Duncan is either female, a retriever, or both.
  - Exactly one of Cardie and Duncan is female.
  - Cardie is not black if and only if at least one of Cardie and Duncan is black.
- E1.10.8 Assuming that all three statements of Exercise 1.10.7 are true, determine the truth values of each of the three predicates for both Cardie and Duncan.
- E1.10.9 In Problem 1.7.9 we modeled an islander problem by using a boolean variable for each islander, indicating whether their statement was true. We could instead model the problem with a single predicate  $T(x)$ , where the type of  $x$  is “islander” and  $T(x)$  means “the statement of islander  $x$  is true”.
- Rewrite the statements of Problem 1.7.9 using only this  $T$  predicate and boolean operators.
  - Does this change affect the difficulty of the problem?
- E1.10.10 In Java, the `String` class has a method `charAt` that takes an `int` as argument and returns the character in a given position of the string. Define a predicate  $CharAt(w, i, a)$  meaning “the character in position  $i$  of string  $w$  is  $a$ ”.
- Write a predicate meaning “ $a$  is the first character of string  $w$ ”.
  - Write a predicate meaning “ $a$  is the last character of string  $w$ ”. (You may use “ $|w|$ ” to denote the length of  $w$ .)
  - Express  $CharAt(w^R, i, a)$ , where  $w^R$  is the reversal of  $w$ , in terms of the  $CharAt$  predicate acting on  $w$ .

### 1.10.4 Problems

P1.10.1 Let  $\Sigma = \{a, b\}$ . Using the same string predicates defined in Exercises 1.10.2 and 1.10.3 above, express the following sets in set builder notation (that is, determine their predicate form):

- (a)  $\{aa, bb\}$ .
- (b)  $\{a, aa, aaa, aba, aaaa, aaba, abaa, abba, aaaaa, aaaba, aabaa, aabba, \dots\}$ .
- (c)  $\{aaaa, aaab, abaa, abab, baba, babb, bbba, bbbb\}$ . (You will also need the concatenation operation here.)

P1.10.2 If  $P$  is a binary predicate and the expression  $P(P(x, y), P(y, x))$  is valid, what do you know about the signature of  $P$ ? Give three different possible templates for  $P$  and evaluate this expression in each case as a function of  $x$  and  $y$ .

P1.10.3 (uses Java) Since predicates have boolean type, they can be combined with boolean operators. Suppose  $P(x, y)$  and  $Q(x, y)$  are predicates with the same signature – we'll say  $x$  is of type `Xtype` and  $y$  is of type `Ytype`. Suppose further that we have Java methods `isP` and `isQ` to evaluate  $P$  and  $Q$  respectively. Write a Java method `isPandQ` to evaluate the predicate " $P(x, y) \wedge Q(x, y)$ ".

P1.10.4 (uses Java) Write real Java methods, with boolean return type, to evaluate each of the following predicates. Choose appropriate types for each variable.

- (a) "The length of string  $w$  is greater than  $n$ "
- (b) " $b^2 - 4ac \geq 0$ "
- (c) "The entries of this array of `int` values are in arithmetic progression, increasing by 1 each time"

P1.10.5 Java allows methods to be **recursive**, meaning that the template of the method may contain a call to the method itself, as long as the sequence of method calls eventually stops. We can define a predicate recursively in mathematical language, but we have the same problem of needing to ensure that the chain of recursive definitions finally stops. We will look at this in much greater detail in Chapter 4, but for now determine whether each of the following functions is properly defined. For each one that *is* properly defined, give a simpler non-recursive definition for it.

- (a) One binary string argument:  $A(w) \Leftrightarrow ((w = \lambda) \vee (w = 01) \vee A(\text{tail}(w)))$ , where the tail of a nonempty string is obtained by deleting the first letter and  $\text{tail}(\lambda) = \lambda$ .
- (b) Two natural arguments  $B(x, y) = x$  if  $y = 0$ , otherwise  $B(x, y) = B(x + 1, y - 1)$ .
- (c) One natural argument:  $C(n) = 0$  if  $n = 0$ , otherwise  $C(n) = C(n + 1) + C(n - 1)$ .
- (d) One natural argument:  $D(k) = 5$  if  $k = 0$ ,  $D(k) = k + D(k + 1)$  if  $k$  is odd,  $D(k) = k/2 + D(k - 2)$  if  $k$  is even and positive.
- (e) One binary string argument:  $E(u) = 0E(u^R)1$  if  $u \neq \lambda$ ,  $E(\lambda) = 011$ .

P1.10.6 We can define binary relations on the naturals for each of the five relational operators. Let  $LT(x, y)$ ,  $LE(x, y)$ ,  $E(x, y)$ ,  $GE(x, y)$ , and  $GT(x, y)$  be the predicates with templates  $x < y$ ,  $x \leq y$ ,  $x = y$ ,  $x \geq y$ , and  $x > y$  respectively.



- (a) Show how each of the five predicates can be written using only  $LE$  and boolean operators. Use your constructions to rewrite  $(LE(a, b) \oplus (E(b, c) \vee GT(c, a)) \rightarrow (LT(c, b) \wedge GE(a, c))$  in such terms.
- (b) Express each of the five predicates using only  $LT$  and boolean operators, and rewrite the same compound statement in those terms.

P1.10.7 (uses Java) If  $S$  is the set of naturals less than some given natural  $m$ , we can represent a unary predicate on  $S$  in Java with a `boolean` array of dimension  $m$ . We can then write methods that take such arrays as input. Write real-Java static methods to compute the following. Note that your methods may assume that the dimension of the input arrays is  $m$ , and are allowed to fail if it is not.

- (a) A static method `boolean equal(boolean[] a, boolean[] b)` that returns `true` if and only if the predicates represented by `a` and `b` are logically equivalent.
- (b) A static method `boolean implies(boolean[] a, boolean[] b)` that returns `true` if and only if  $b(x)$  is true whenever  $a(x)$  is true.
- (c) A static method `boolean common(boolean[] a, boolean[] b)` that returns `true` if and only if there is some natural  $x$  such that  $a(x)$  and  $b(x)$  are both true.
- (d) A static method `int leastCommon(boolean[] a, boolean[] b)` that returns the smallest natural  $x$  such that  $a(x)$  and  $b(x)$  are both true, or returns  $m$  if there is no such natural.

P1.10.8 The **game of SET**<sup>35</sup> is played with a deck of 81 cards. A card has four attributes (color, number, shape, and shading, which we will call attributes 0, 1, 2, and three respectively) each of which can have one of three values. We may thus represent a card as a string of length 4 over the alphabet  $\{0, 1, 2\}$ . A group of three cards “forms a set” if for each of the four attributes, the values of that attribute for the three cards are either all the same or all different. For example, the cards 0111, 1101, and 2101 form a set because the values for the first and third attributes are all different, and the values for the second and fourth attributes are all the same.

- (a) Define four predicates  $A_0, A_1, A_2$ , and  $A_3$ , whose first argument is a card and whose second is one of the values 0, 1, or 2.  $A_i(c, j)$  means “attribute  $i$  of card  $c$  has value  $j$ ”, so that  $A_0(0110, 0)$ ,  $A_1(0110, 1)$ ,  $A_2(0110, 1)$ , and  $A_3(0110, 0)$  are all true. Using these predicates and boolean operators, write a predicate  $SET(a, b, c)$  that states that cards  $a$ ,  $b$ , and  $c$  form a set.
- (b) (uses Java) Write a real-Java static method that takes three strings and returns a boolean telling whether the corresponding cards form a set. Do not worry about what your method does if the strings do not represent cards.

P1.10.9 This problem uses the definitions of Problem 1.10.8.

- (a) Argue that given any two cards, there is exactly one card in the deck that forms a set with them.
- (b) (uses Java) Write a real-Java static method that takes two strings and (if they both represent cards) returns a string representing a card that forms a set with the two input cards.

---

<sup>35</sup> “SET” is a registered trademark of Carmel LLC and the SET game is a product of Set Enterprises.

- P1.10.10 Suppose we have a set of sports teams in a league  $L$ , and a set of games  $G$  among those teams. Each game has a home team and a visiting team, who are two different teams in the league. We have predicates  $H(g, t)$  meaning “team  $t$  was the home team in game  $g$ ” and  $V(g, t)$  meaning “team  $t$  was the visiting team in game  $g$ ”.
- (a) Consider the predicate  $P(s, t)$  meaning “team  $s$  played a game against team  $t$ ”. Can you express this predicate in terms of the predicates  $H$  and  $V$ , along with boolean operators? Either do it or explain (informally) why you cannot.
  - (b) If I have a list of the games played, and access to the  $H$  and  $V$  predicates, how could I compute the value of  $P(s, t)$  given two teams  $s$  and  $t$ ?
  - (c) (uses Java) Assume that you have a Java class `Game` that includes instance methods `boolean home(Team t)` and `boolean visitor (Team t)`. Write a static method `boolean played(Team s, Team t, Game[] season)` that will input two teams and an array of games, and return `true` if and only if  $P(s, t)$  is true for the set of games in the array.

## 1.11 Excursion: Translating Predicates

The German philosopher Goethe famously said “Mathematicians are a kind of Frenchmen: Whatever you say to them, they translate it into their own language and forthwith it is something entirely different.” In order to practice mathematics we need to develop the skills of translating English into formal mathematical language and vice versa.

Every time we’ve introduced a piece of mathematical notation, we’ve given an English meaning for it that in some sense tells us how to translate it into English. Translating “ $\wedge$ ” as “and” is generally straightforward, but we’ve mentioned complications, like the two different concepts represented by English “or”, or the many different English ways to express “ $p \rightarrow q$ ”. When English statements are made of substatements connected by boolean operations, we can carry out a translation by giving each substatement a name and forming a compound proposition.

In Section 1.10 we have just defined *predicates* to be statements that become true or false when arguments of specified types are supplied. Predicates allow us to translate some English statements into compound propositions in a way that preserves some of the commonalities among the parts. For example, let’s formalize a line of Dr. Seuss: “Knox in box. Fox in socks.” We can define a predicate  $\text{IsIn}(x, y)$  that means “creature  $x$  is in thing  $y$ ”, and the statement becomes  $\text{IsIn}(\text{Knox}, \text{box}) \wedge \text{IsIn}(\text{Fox}, \text{socks})$ .

It’s important in the formal language that the arguments to a predicate come in the right order and have the correct type. Natural language does not always follow this rule, so that statements may be ambiguous. Consider the Abbott and Costello comedy routine *Who’s On First*, a conversation about a baseball team. We can formalize many of the statements Abbott and Costello make using a predicate  $\text{Plays}(x, y)$  meaning “Player  $x$  is playing position  $y$ ”. When Costello says “Who’s on first?”, he means to say “Tell me  $x$  such that  $\text{Plays}(x, \text{first})$  is true”. Abbott misinterprets his question as the statement  $\text{Plays}(\text{Who}, \text{first})$ , which happens to be true because “Who” is one of the players, and says “Naturally.” Costello thinks Abbott means “ $\text{Plays}(\text{Naturally}, \text{first})$ ”, and so on. (The full text of the comedy routine is easily found on the Web.)

We *cannot* formalize this in our language without resolving the ambiguity, because of the data typing. Since  $\text{Plays}$  is a predicate whose first argument is a player and whose second argument is a position, Abbott’s formal answer would reveal the key information that Who is a player, and not part of the question. Someone like Hofstadter might say that the humor here derives from confusion between the **object language** (what you use to say things) and the **meta-language** (what you use to talk about statements in the object language). Modern formal systems are very careful to distinguish these, with the result that it’s very hard to translate something like “Who’s on First” into them. (Remember the distinction we made earlier between  $\leftrightarrow$ , a symbol of the propositional calculus, and  $\iff$ , a symbol we use to say that two propositional calculus statements are equivalent.)

Sometimes the hardest part of translating from words to symbols is determining exactly what the words mean. A good practical way to proceed is often to rephrase the English and decide whether your rephrasing means the same thing. An English “if”, for example, might or might not mean logical implication in the mathematical sense – consider the difference between “If you are ready at 8:00, I will pick you up” and “If I don’t see you, have a nice vacation”.

Literal translation of symbolic language into English often produces something that is syntactically legal but sounds strange, a bit like a word-for-word translation of one natural language into another<sup>36</sup>. For example, you can usually translate  $\neg$  by “it is not the case that”, but “It is not the case that pigs fly” is more easily understood as “Pigs do not fly”.

Finally, English is just more expressive than any formal language, and there are going to be things you can't translate well. Predicate calculus talks about sets of things (which have to be known and typed) and particular relations between those things (which have to be defined). It can't ask questions, convey shades of meaning, or do any number of other things. Throughout history various philosophers have sought to formalize the “Laws of Thought” as a symbolic language, often with the intent of forcing speakers to clarify their thinking<sup>37</sup>. The computer scientists currently struggling with these issues are the “natural language processing” researchers, part of the larger research community of “artificial intelligence” or “AI”. The aim of a typical project in this area was to take a mass of text, such as all stories about corporate takeovers in a year of the *Wall Street Journal*, and construct a database that (with some margin for error) records which company is trying to do what to which other company. It's a tremendously difficult problem, even though the domain is restricted so that the range of possible events is more or less known. The attempts to stage “Turing tests”, where computers try to fool people into thinking they're human by conversing in natural language over email, have been even less successful<sup>38</sup>.

In informal mathematics we often use a hybrid of English and symbolic speech, designed to force us to say precisely what we mean. (This is the practice that so annoyed Goethe.) We'll practice this language a lot over the course of the book, but learning it is necessarily a subjective process, somewhat like learning a new natural language.

### 1.11.1 Writing Exercise

Here are some translation exercises involving predicates that will help you test where you are in dealing with the symbolic language. For the symbol to English translations, make the English as clear and readable as you can. This may involve a second pass after you've first written something to get the meaning clear.

1. In the following passage from Lewis Carroll's *Through the Looking Glass*, the White Knight makes a number of statements about a song  $S$  and various strings. Translate these statements into symbols, using the predicates  $NameOf(x, y)$ , meaning “string  $x$  is the name of thing  $y$ ”, and “ $IsCalled(x, y)$ ”, meaning “thing  $x$  is called by string  $y$ ”. Note that in our terminology both songs and strings are also thing objects.

---

<sup>36</sup>For example, the French equivalent of the English question “What is that?” is “Qu'est-ce que c'est?”, which literally means “What is that that that is?”. Mark Twain famously made an amusing direct English translation of the French translation of his short story *The Notorious Jumping Frog of Calaveras County*.

<sup>37</sup>In Orwell's *Nineteen Eighty-Four*, the evil dictatorship invents a formalized version of English that is designed to prevent certain thoughts.

<sup>38</sup>The Loebner Prize competition has been conducted annually since 1990, and no computer has ever succeeded in convincing a judge that it is human. In some early competitions, however, some judges identified several real people as computers. They apparently had an exaggerated view of the state of the art in this area.

*"The name of the song is called 'Haddocks Eyes.'"*

*"Oh, that's the name of the song, is it?" Alice said, trying to feel interested.*

*"No, you don't understand," the Knight said, looking a little vexed. "That's what the name is called. The name really is 'The Aged Aged Man.'"*

*"Then I ought to have said 'That's what the song is called'?" Alice corrected herself.*

*"No, you oughtn't": that's quite another thing! The song is called 'Ways and Means': but that's only what it's called, you know!"*

*"Well, what is the song, then?" said Alice, who was by this time completely bewildered.*

*"I was coming to that, the Knight said. The song really is 'A-sitting On A Gate': and the tune's my own invention."*

2. Translate this baseball rule into English, where the meaning of the predicates and basic propositions is as follows:  $\text{RunnerOn}(b)$  means "there is a runner on base  $b$ ",  $\text{Outs}(n)$  means "the current number of outs is  $n$ ",  $\text{InfieldFly}$  means "the batted ball is a fly that can be caught with ordinary effort by an infielder",  $\text{Fair}$  means "the batted ball is fair",  $\text{BatterOut}$  means "the batter is out" and  $\text{MayAdvance}$  means "runners may advance at their own risk".

$$\begin{aligned} &(\text{RunnerOn}(\text{first}) \wedge \text{RunnerOn}(\text{second}) \wedge \\ &(\text{Outs}(0) \vee \text{Outs}(1)) \wedge \text{InfieldFly} \wedge \text{Fair}) \rightarrow \\ &\hspace{15em} (\text{BatterOut} \wedge \text{MayAdvance}) \end{aligned}$$

3. Translate this passage by Dr. Seuss from English into symbols, inventing your own predicates:

Chicks with bricks come.  
 Chicks with blocks come.  
 Chicks with bricks and blocks and clocks come.  
 ...  
 Socks on chicks and chicks on fox.  
 Fox on clocks on bricks on blocks.  
 Bricks and blocks on Knox on box.

4. Translate the following passage, adapted from Dr. Seuss, from symbols to English. (You should be literal rather than try to produce Seuss' original text exactly.) The predicates used are  $\text{Likes}(x, t)$  (person  $x$  likes thing  $t$ ),  $\text{LikesIn}(x, t, p)$  (person  $x$  likes thing  $t$  in place  $p$ ),  $\text{LikesWith}(x, t, u)$  (person  $x$  likes thing  $t$  with creature  $u$ ), and  $\text{IsPlace}(t)$  (thing  $t$  is a place). Note that the compound proposition makes use of an **implicit type cast**: a variable of type **thing** is used where a variable of type **place** is demanded<sup>39</sup>. The constant  $\text{GEH}$ , of type **thing**, means "green eggs and ham".

$$\begin{aligned} &\neg \text{LikesIn}(\text{I}, \text{GEH}, \text{box}) \wedge \neg \text{LikesWith}(\text{I}, \text{GEH}, \text{fox}) \wedge \\ &\neg \text{LikesIn}(\text{I}, \text{GEH}, \text{house}) \wedge \neg \text{LikesWith}(\text{I}, \text{GEH}, \text{mouse}) \wedge \\ &((p = \text{here} \vee p = \text{there}) \rightarrow \neg \text{LikesIn}(\text{I}, \text{GEH}, p)) \wedge \\ &(\text{IsPlace}(y) \rightarrow \neg \text{LikesIn}(\text{I}, \text{GEH}, y)) \wedge \\ &\hspace{15em} \neg \text{Likes}(\text{I}, \text{GEH}) \end{aligned}$$

<sup>39</sup>A careful analysis will show that this predicate need not be evaluated unless its argument has the correct type.

## Index

- actual parameters 1-68
- alphabet 1-11
- anagram 1-19
- arguments of a predicate 1-67
- Associativity Rules 1-51, 1-52, 1-53
- assumptions of a proof 1-58
- atomic proposition 1-25
  
- base propositions 1-25
- binary string 1-11
- bit 1-24
- Boole's Expansion Theorem 1-49
- boolean** type 1-3
- boolean algebra 1-24
- boolean constant 1-24
- boolean operation 1-24
  
- Carroll diagram 1-42
- Commutativity Rules 1-51, 1-52, 1-53
- complement of a set 1-35
- complete set of operators 1-32
- compound proposition 1-25
- concatenation of strings 1-13
- conclusion 1-44
- conjunction 1-25
- Conjunction Rule 1-58
- conjunctive normal form (CNF) 1-48
- consequences 1-58
- contradiction 1-30
- Contrapositive Rule 1-54, 1-59
- covering a set 1-43
  
- decision problem 1-14
- decision procedure 1-14
- deductive sequence proof 1-44, 1-52
- Definition of Equivalence Rule 1-54
- Definition of Implication Rule 1-54
- DeMorgan Rules 1-54
- difference of sets 1-35
- direct proof 1-58
- disjoint sets 1-34
- disjunction 1-26
- disjunctive normal form (DNF) 1-48
- Distributive Laws 1-54
- Double Negative Rule 1-54
  
- elements 1-4
- empty set 1-5
- equal sets 1-4
- empty string 1-11
- Equal Operand Rules 1-52
- equal sequences 1-12
- equal strings 1-12
- equality statement 1-68
- equational sequence proof 1-44, 1-51
- equivalence 1-28
- Equivalence and Implication Rule 1-60
- equivalent propositions 1-44
- evaluate a compound proposition 1-29
- even number 1-22
- Excluded Middle Rule 1-54
- exclusive or 1-26
  
- Fantasy Rule 1-58
- finite set 1-5
- formal definition 1-2
- formal language 1-14
- formal language theory 1-15
- formal parameters 1-68
- forward-backward method 1-58
- free variable 1-67
  
- Hypothetical Syllogism Rule 1-54, 1-60
  
- if and only if 1-28
- if-then-else function 1-47
- implication 1-277
- implicit type cast 1-76
- inclusive or 1-26
- indirect proof 1-59
- inequality statement 1-68
- infinite set 1-5
- integer** pseudo-Java type 1-3
- intersection of sets 1-34
- interval 1-43
- irrational number 1-62
  
- Joining Rules 1-54
  
- language 1-14
- Least Number Axiom 1-22
- length of a string 1-12

- letters 1-11
- loop invariant 1-23
- membership statement 1-37
- meta-language 1-74
- method of truth tables 1-29
- Modus Ponens Rule 1-53
- Modus Tollens Rule 1-63
- natural** pseudo-Java type 1-3
- negation 1-27
- object language 1-74
- odd number 1-22
- One Rules 1-53
- overloading of operators 1-68
- parentheses 1-29
- partially correct code 1-22
- postcondition 1-21
- power set 1-9
- precedence rules 1-29
- prefix 1-14
- primitive type 1-3
- precondition 1-21
- predecessor of a natural 1-22
- predicate 1-67
- premise 1-44
- Proof By Cases Rule 1-54, 1-60
- proof by contradiction 1-54, 1-59
- proper subset 1-4
- proposition 1-24
- propositional calculus 1-24
- pseudo-Java 1-2
- QED 1-28
- real** pseudo-Java type 1-3
- recursive methods 1-71
- reversal of a string 1-13
- Russell Paradox 1-3
- satisfiable compound proposition 1-30
- Separation Rules 1-54
- set 1-3
- set builder notation 1-4
- SET game 1-72
- set identity 1-6
- Shannon Decomposition 1-49
- signature of a predicate 1-67
- size of a set 1-5
- string 1-11
- subset 1-4
- substring 1-14
- successor of a natural 1-22
- suffix 1-14
- symmetric compound proposition 1-49
- symmetric difference of sets 1-35
- tautology 1-29
- template of a predicate 1-67
- Tertium Non Datur Rule 1-56
- thing** pseudo-Java type 1-3
- trivial proof 1-54, 1-59
- truecount of a compound proposition 1-57
- truth table 1-26, 1-44
- Twenty Questions 1-50
- two-column format 1-51
- type 1-2
- union of sets 1-34
- universal set 1-36
- vacuous proof 1-54, 1-59
- Venn diagram 1-34
- Zero Rules 1-53





## Chapter 2: Quantifiers and Predicate Calculus

*“Quiet! Quiet! There are ways of telling whether she is a witch.”*  
*“There are? Tell us. What are they, wise Sir Bedevere? Do they hurt?”*  
*“Tell me... what do you do with witches?”*  
*“Burn them.”*  
*“And what do you burn, apart from witches?”*  
*“... Wood?”*  
*“So why do witches burn?”*  
*“...Because they’re made of wood?”*

A vast number of the statements we want to make in mathematics are of the form “there exists an object such that” something happens, or “for every object” of a particular type, something happens. In this chapter we look at the exact *meaning* of statements in these forms, techniques for *proving* such statements, and a number of important mathematical concepts that are *defined* using such statements. Specifically:

- The *predicates* we defined in Chapter 1 have an equivalent formulation in terms of *relations*, the sets of objects or sequences of objects making the predicate true.
- The symbol  $\exists$  for “there exists” and  $\forall$  for “for all” are called *quantifiers* – we’ll define these symbols and give several examples of their use.
- There are four basic rules for *proving* statements involving quantifiers – we’ll present these and show how they are used.
- Finally we will use quantifiers to give formal *definitions* of several concepts: some operations on formal languages, some properties of relations, and some special kinds of relations: *functions*, *partial orders*, and *equivalence relations*. This will give us more chances to practice quantifier proofs.

## 2.1 Relations

### 2.1.1 Sets of Pairs and Tuples

It is clear that the concepts of *predicate* and *set* are closely related. When we define a set using set builder notation, we give a unary predicate (a predicate with one free variable) that is true for the elements of the set and false for those elements that are of the correct *type* to be in the set, but not in it. For example, if our type is  $\{a, \dots, z\}$  we can express the set of vowels either by a list,  $\{a, e, i, o, u\}$ , or with set builder notation,  $\{ch : ch \text{ is a vowel}\}$ .

In general the set  $S$  and the predicate “ $x$  is in  $S$ ” are *almost* the same thing. They are not the same thing because they are objects of different *types*:  $S$  is a set while the predicate is a predicate. Set builder notation lets us create a set from any predicate, as we have seen. Similarly, any set  $S$  has its **membership predicate**, which is just “ $x$  is in  $S$ ”.

What if a predicate has more than one free variable? For example, with the same type of lower-case letters, let  $B(ch_1, ch_2)$  be the predicate “ $ch_1$  comes before  $ch_2$  in the alphabet”. We can no longer easily identify this predicate with a set of letters: “ $\{ch_1 : B(ch_1, ch_2)\}$ ”, for example, is not a properly defined set because we can’t tell what its members are without knowing the value of  $ch_2$ . Can we define a set that bears the same relation to this *binary* predicate as an ordinary set bears to its membership predicate?

We can, and the way to do it in this case is to make the set not a set of letters but a set of *pairs* of letters. More generally, any predicate with two free variables corresponds to a set of pairs:

**Definition:** Let  $X$  and  $Y$  be sets. If  $x$  is an element of  $X$  and  $y$  an element of  $Y$ , we define the **pair**  $\langle x, y \rangle$  to be a new object made from  $x$  and  $y$  *in that order*. The **direct product**  $X \times Y$  is the set of all such pairs. If  $P(x, y)$  is a predicate with two free variables  $x$  and  $y$  of type  $X$  and  $Y$  respectively, the **binary relation** corresponding to  $P$  is the set  $R = \{\langle x, y \rangle : P(x, y)\}$  (In English, “the set of pairs  $\langle x, y \rangle$  such that  $P(x, y)$  is true”).

In the example above, the binary relation corresponding to the predicate  $B$  is the set of pairs  $\langle ch_1, ch_2 \rangle$  such that  $ch_1$  comes before  $ch_2$  in the alphabet.

More generally still, we can define relations to correspond to predicates with more free variables.

**Definition:** Let  $Q(x_1, \dots, x_k)$  be a predicate with  $k$  free variables, where the type of each free variable  $x_i$  is  $X_i$ . Recall that a **sequence** of values  $\langle x_1, \dots, x_k \rangle$  is a single object made from the  $k$  specified objects *in the given order*. We define the **relation** corresponding to  $Q$  to be the set of sequences  $\langle x_1, \dots, x_k \rangle$  making  $Q(x_1, \dots, x_k)$  true. (Naturally, each value  $x_i$  must be from the correct type  $X_i$  to make this happen.)

**Example:** Let  $V(p, c, m)$  be the predicate defined to mean “person  $p$  likes to visit country  $c$  in month  $m$ ”. Then if  $V(\text{Jessica}, \text{Italy}, \text{March})$  is true, the sequence  $\langle \text{Jessica}, \text{Italy}, \text{March} \rangle$  is an element of the relation corresponding to  $V$ . This relation is a set of **triples**, each consisting of a person, a country, and a month in that order. (Sequence of  $k$  objects are often called  **$k$ -tuples**,

|                                                                      | $\{x, y, z\}$                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\left\{ \begin{array}{c} 1, \\ 2, \\ 3, \\ 4, \end{array} \right\}$ | $\left\{ \begin{array}{ccc} \langle 1, x \rangle & \langle 1, y \rangle & \langle 1, z \rangle \\ \langle 2, x \rangle & \langle 2, y \rangle & \langle 2, z \rangle \\ \langle 3, x \rangle & \langle 3, y \rangle & \langle 3, z \rangle \\ \langle 4, x \rangle & \langle 4, y \rangle & \langle 4, z \rangle \end{array} \right\}$ |

Source: David Mix Barrington

Figure 2-1: The direct product of the sets  $\{1, 2, 3, 4\}$  and  $\{x, y, z\}$

with 2-tuples also known as pairs and 3-tuples as triples.)

The **signature** of a predicate is the sequence of types of its arguments. The elements of a relation all come from the same data type, the set of sequences where each element has the type appropriate to its place in the sequence as given by the signature of the predicate. The entire set of such sequences is called the **direct product** of the individual data types. In our last example, if  $P$ ,  $C$ , and  $M$  are the sets of people, countries, and months respectively, the relation is a subset of the direct product  $P \times C \times M$ , the set of all person-country-month triples. Note that the direct product is itself a relation – it comes from the predicate with the correct signature that is always true:  $P \times C \times M = \{(p, c, m) : 1\}$ .

We can form a direct product from any two sets  $A$  and  $B$  by taking the set of pairs where the first element is from  $A$  and the second is from  $B$ . Figure 2-1 illustrates this in the case where  $A = \{1, 2, 3, 4\}$  and  $B = \{x, y, z\}$ . The figure, with the pairs arranged in a rectangle, should give you some idea of why this set is called a “product”. Note that if  $A$  and  $B$  are both finite sets,  $|A \times B| = |A| \cdot |B|$ .

## 2.1.2 Storing Relations

There are three common ways to store a predicate or relation in a computer:

- We can have an *array* of boolean values, storing the boolean value of the predicate for every possible choice of the predicate’s free variables.
- We can have a boolean *method* that computes each of these values on demand, given the free variables as inputs.
- We can have a *list* of the tuples in the relation, usually sorted in some way to make searches, insertions, and deletions easier.

The boolean array is only possible if the types of the free variables are finite, of course, and is only practical if these types are small enough that the array can be stored in the available memory. The code method is always possible in principle, but to be practical it requires that computing the predicate value from the free variables be practical. Finally, the feasibility and practicality of the list of tuples depends on the size of the list. Many predicates that occur in practice are *sparse*,

meaning that the number of tuples in the relation is much smaller than the number in the entire direct product of the types. (That is, the predicate is *usually* false.) In this case the list may be the most efficient way to store the relation.

Before leaving the topic of relations we should informally visit the definition of the word **function**. We normally think of a function as an object that takes one or more *inputs* from a specified type, its **domain**, and produces an output from another specified type, its **range**. We used this informal definition in one of our definitions of the word “predicate” — a predicate is a function whose range is boolean.

Formally, though, mathematicians usually define a function to be a certain kind of *relation* — a set of pairs where each pair’s first element is from the domain and second element from the range. Let  $f$  be a function from some set  $A$  to some set  $B$ . Viewed as a relation,  $f$  will be a subset of  $A \times B$ , in particular the set

$$\{\langle x, y \rangle : f(x) = y\}$$

of pairs where the first element is in the domain and the second is its corresponding function value. Often this set of tuples is called the **graph** of the function, because when we plot the graph of a function in an algebra or calculus class we include a point  $\langle x, y \rangle$  if and only if  $y = f(x)$ .

Every function gives rise to a relation in this way, but only some relations can be the graph of a function. A function must have the two properties of being *total* (every input has at least one output) and *well-defined* (every input has at most one output). We’ll give formal definitions and examples of these properties and others in Sections 2.8 and 2.9.

### 2.1.3 Exercises

E2.1.1 Let  $A$  be the set  $\{1, 2\}$ ,  $B$  be the set  $\{x, y, z\}$ , and  $C$  be the set  $\{\text{cat}, \text{dog}\}$ . List the following direct products:

- (a)  $A \times B$
- (b)  $B \times A$
- (c)  $C \times A$
- (d)  $A \times B \times C$
- (e)  $B \times A \times B$

E2.1.2 Consider the following subsets of and unary predicates on the set  $\{a, b, \dots, z\}$ . Give the subset corresponding to each predicate and a predicate corresponding to each set.

- (a)  $\{a, b, c, d\}$
- (b)  $A(ch)$  meaning “ $ch$  comes directly after a vowel in the alphabet”
- (c)  $\{c, f, i, l, o, r, u, x\}$
- (d)  $B(ch)$  meaning “ $ch$ , as a block capital letter, is the same when seen in a mirror”

E2.1.3 Consider the following binary relations and binary predicates on the sets  $C = \{b, c, d\}$  and  $V = \{a, e, i, o, u\}$ . Give the relation (the subset of  $C \times V$ ) corresponding to each predicate and a predicate corresponding to each relation:

- (a)  $\{\langle b, a \rangle, \langle b, e \rangle, \langle b, i \rangle, \langle b, o \rangle, \langle b, u \rangle\}$
- (b)  $W(ch_1, ch_2)$  meaning “ $ch_1$  followed by  $ch_2$  forms an English word”
- (c)  $\{\langle c, e \rangle, \langle c, u \rangle, \langle d, e \rangle, \langle d, u \rangle\}$
- (d)  $P(ch_1, ch_2)$  meaning “either  $ch_1 = b$  or  $ch_2 = o$ , but not both”

E2.1.4 (uses Java) Each of the following blocks of Java code takes two numbers  $i$  and  $j$ , in the range  $\{0, 1, 2, 3\}$ , and returns a `boolean` value. Each thus defines a binary predicate on this set. Describe each predicate as a boolean array.

- (a) `k = i+1; m = j-1; return (k*k == m*m);`
- (b) `return (i%2 == j%2);`
- (c) `return ((i > j) && (i+j == 6));`

E2.1.5 Let  $A = \{\langle 0, 1 \rangle, \langle 1, 2 \rangle, \langle 2, 3 \rangle\}$  and  $B = \{\langle x, x \rangle : x \in \{0, 1, 2, 3\}\}$  be binary relations on the set  $\{0, 1, 2, 3\}$ . Describe the following binary relations on  $\{0, 1, 2, 3\}$ :

- (a)  $A \cup B$
- (b)  $A \cap B$
- (c)  $A \setminus B$
- (d)  $B \cup \{\langle y, x \rangle : \langle x, y \rangle \in A\}$

E2.1.6 We can think of addition as giving us a *ternary* relation on the naturals, by defining  $\text{Plus}(i, j, k)$  to be true if and only if  $i + j = k$ . List the elements of the relation  $\text{Plus}$  where each of  $i$ ,  $j$ , and  $k$  is at most 3.

E2.1.7 In each of these examples of a set  $S$  and a unary relation  $R$ , which of our three methods of storing the relation (boolean array, method, or list) is likely to be most suitable, and why?

- (a)  $S$  is a 100 by 100 array of pixels,  $R$  is the pixels that are black in a particular black-and-white picture.
- (b)  $S$  is the set of all strings of up to 20 letters from  $\{a, \dots, z\}$ ,  $R$  is the set of strings that occur as words in the King James Bible.
- (c)  $S$  is a 100 by 100 array of pixels,  $R$  is the pixels that are inside the circle defined by the equation  $(x - 30)^2 + (y - 40)^2 = 400$ .

E2.1.8 Let  $X$  be the set  $\{1, 2, 3, 4, 5\}$  and let  $B$  be the set of triples  $\langle x, y, z \rangle$  such that both  $x < y$  and  $y < z$ . List the elements of  $X$ .

E2.1.9 Let  $k$  and  $n$  be two positive naturals. How many different possible  $k$ -ary relations are there on the set  $\{0, \dots, n - 1\}$ ? For what values of  $k$  and  $n$  is this number less than 1000?

E2.1.10 Let  $S$  be the set  $\{0, 1, 2, 3, 4, 5, 6\}$  and let the function  $f$  take each number  $x$  to  $(x + 3)\%7$ , where “ $\%$ ” represents the Java integer remainder function. List the elements of the graph of the function  $f$ . Describe the boolean array representing the graph as a binary relation.

## 2.1.4 Problems

- P2.1.1 Let  $A$  be any set. What are the direct products  $\emptyset \times A$  and  $A \times \emptyset$ ? If  $x$  is any thing, what are the direct products  $A \times \{x\}$  and  $\{x\} \times A$ ? Justify your answers.
- P2.1.2 (uses Java) Let `isInA(int x)` be a method that takes one input  $x$  in the range from 0 to `ALENGTH - 1` and returns a boolean that tells whether  $x$  is in the set  $A$ . Write a method that prints a list of the elements in  $A$ .
- P2.1.3 (uses Java) Let  $L$  be an array of objects of type `Pair`, giving the elements of a relation  $R \subseteq A \times B$ . Write a method that takes as input an element  $x$  of  $A$  and an element  $y$  of  $B$ , and returns a boolean telling whether  $\langle x, y \rangle \in R$ . Assume that `Pair` has methods `getFirst()` and `getSecond()` that return its first and second components respectively.
- P2.1.4 Let  $A$ ,  $B$ , and  $C$  be the sets defined in Exercise 2.1.1 above. Explicitly list the direct products  $(A \times B) \times C$  and  $A \times (B \times C)$ . (Note that an element of  $A \times (B \times C)$  is a pair whose first element is in  $A$  and whose second element is in  $B \times C$ .) If  $X$ ,  $Y$ , and  $Z$  are any sets, we generally do not distinguish among  $X \times (Y \times Z)$ ,  $(X \times Y) \times Z$ , and the set of triples  $X \times Y \times Z$ . Explain how we may think of an element of each of these sets as an element of one of the others, by describing which elements of each set correspond to which elements of the others.
- P2.1.5 Let  $n$  be a natural and let  $I(x)$  be a unary relation on the set  $\{0, \dots, n-1\}$ . Let  $w$  be the binary string of length  $n$  that has 1 in position  $x$  whenever  $I(x)$  is true and 0 in position  $x$  when  $I(x)$  is false. (As in Java, we consider the positions of the letters in the string to be numbered starting from 0.) What is the string corresponding to the predicate  $I(x)$  meaning “ $x$  is an even number” in the case where  $n = 5$ ? The case where  $n = 8$ ? If  $w$  is an arbitrary string and  $I(x)$  the corresponding unary predicate, describe the set corresponding to the predicate in terms of  $w$ .
- P2.1.6 Let  $P(x, y)$  be a binary predicate where the type of  $x$  is  $A$  and the type of  $y$  is  $B$ . We define the **projection**  $\pi_1(P)$  to be the unary predicate on  $A$  defined as follows:  $\pi_1(P)(x)$  is true if and only if there exists an element  $y$  of  $B$  such that  $P(x, y)$  is true. Similarly,  $\pi_2(P)$  is the unary predicate on  $B$  defined so that  $\pi_2(P)(y)$  is true if and only if there exists an element  $x$  of  $A$  such that  $P(x, y)$  is true.
- (a) Let  $A = B = \{0, 1, 2, 3\}$  and define  $P(x, y)$  to mean “ $x < y$ ”. Describe  $\pi_1(P)$  and  $\pi_2(P)$  as both predicates and relations.
  - (b) What can you say about the predicate  $P$  if you know that  $\pi_1(P)$  and  $\pi_2(P)$  are each always false? What if they are each always true?
  - (c) (uses Java) Suppose you are given a boolean method `isInP(int x, int y)` where each of  $x$  and  $y$  are restricted to the range from 0 through `LENGTH - 1`. Write boolean methods that will return the value of the predicates  $\pi_1(P)$  and  $\pi_2(P)$ , given appropriate input.
- P2.1.7 Let  $X$  be a set of at least two people. We will ask each person in  $X$  to write down the name of a person in  $X$  other than themselves. Let  $R$  be the relation  $\{\langle a, b \rangle : a\text{'s chosen name is } b\}$ . Suppose we start with a given person  $a$ , move to the person that  $a$  wrote down, then to the person that they wrote down, and so forth until we reach a given person  $b$ . Let  $S$  be the relation  $\{\langle a, b \rangle : \text{Starting from } a, \text{ we eventually reach } b \text{ by this process}\}$ .
- (a) If from  $a$ , we reach  $a$  again before finding  $b$ , explain why we will never reach  $b$ .

- (b) If  $\langle a, b \rangle$  is not in  $S$ , when can we give up on the process? Will we definitely reach  $a$ ?
- (c) (uses Java) If the elements of  $X$  are represented by the integers  $\{0, \dots, n-1\}$  for some  $n$ , and the relation  $R$  is given by a two-dimensional boolean array, write a method that will test whether a given pair is a member of  $S$ .

P2.1.8 (uses Java) Suppose that the function  $f$  has both input and output in the set  $\{0, \dots, n-1\}$  where  $n$  is some positive natural. We are given a Java method `int f(int x)` that computes  $f$  given any input in that range.

- (a) Write a method (with an empty argument list) that will return a two-dimensional boolean array representing the graph of  $f$  as a relation.
- (b) Write a method (with an empty argument list) that will return a list of the elements of the graph of  $f$  as two-dimensional array `G[] []` of `int` values. The first index should range from 0 to  $s-1$ , where  $s$  is the number of pairs in the relation. For every natural  $x$  with  $x < s$ , `G[x, 0]` and `G[x, 1]` should be the two elements of a pair in the relation, and every pair should appear in the array exactly once.

P2.1.9 Suppose we are given an  $n$  by  $n$  array of `boolean` values, and we want to know whether it represents the graph of a function from  $\{0, \dots, n-1\}$  to  $\{0, \dots, n-1\}$

- (a) Informally explain how to do this.
- (b) (uses Java) Write a static real-Java method that takes such an array as its argument and returns a `boolean` telling whether the binary relation represented by the array is a function.

P2.1.10 Let  $C$  be a set of  $n$  candidates and  $J$  be a set of  $n$  jobs. Define the relation  $Q$  to be  $\{\langle c, j \rangle \mid \text{candidate } c \text{ is qualified for job } j\}$ . Define the relation  $A$  to be  $\{\langle c, j \rangle \mid \text{candidate } c \text{ is assigned to job } j\}$ .

- (a) We say that candidate  $c$  is “possible” for job  $j$  if they are qualified for  $j$ , they are not assigned to any other job, and no one else is assigned to  $j$ . Informally describe how you would determine, given boolean arrays for the relations  $Q$  and  $A$ , whether  $c$  is possible for  $j$ .
- (b) One way to assign candidates to jobs would be to start with an empty assignment and repeatedly find a possible candidate for a job and assign them to it, until there are no possible candidates left. This **greedy algorithm** will not necessarily assign the greatest possible number of candidates to jobs. Find an example where there is a possible assignment of qualified candidates for all the jobs, but there is a sequence of assignments of possible candidates that stops, with no further assignments possible, while some jobs are still unfilled.

## 2.2 Excursion: Relational Databases

One of the most important uses of computers in the real world is to store and manipulate large amounts of data, called **databases**. The mathematical notion of a relation is the basis of one of the most important ways to maintain a database, the **relational database**. Such a database consists of a number of data types and predicates or relations defined on them, possibly of different signatures.

For example, consider a database storing information on the computer science students at a particular college. Each student has a name, a student number, and grades for some number of classes. We can represent this information in predicates such as  $\text{NumberIs}(s, n)$ , “student  $s$  has student number  $n$ ”,  $\text{Took}(n, c)$ , “the student with number  $n$  took class  $c$ ”, or  $\text{GotGrade}(s, c, g)$ , “student  $s$  took course  $c$  and got a grade of  $g$ ”.

A database program will want to manipulate the database in various ways, by inserting new information, updating or deleting old information, and answering specific queries. The difficulty of carrying out these tasks can depend on how the information is stored. We could maintain an array of boolean values with the answer to every possible question that could be asked about a predicate value. But since the size of such an array could easily be too large to work with, we more often store the predicates as relations. For each tuple in each of the relations, we store a **record** that contains the elements of the tuple. The records in each relation can then be stored, usually sorted in some way so that we can more easily search for, insert, or delete records.

In this Excursion we look at one particular operation that we often want to perform on relational databases, the **join** of two relations<sup>1</sup>. Let  $A$ ,  $B$ , and  $C$  be finite data types. Let  $R \subseteq A \times B$  and  $S \subseteq B \times C$  be two binary relations. The join of  $R$  and  $S$  is a relation  $T \subseteq A \times C$  defined as follows. The pair  $\langle a, c \rangle$  is in  $T$  if and only if there is some element  $b$  of  $B$  such that both  $\langle a, b \rangle$  is in  $R$  and  $\langle b, c \rangle$  is in  $S$ .

Consider the binary relations  $\text{NumberIs}$  and  $\text{Took}$  from our example above. What is the join of these two relations? First we must check to make sure that the join can be defined, which it can be because the type of the second argument of  $\text{NumberIs}$  and the type of the first argument of  $\text{Took}$  are both **StudentNumber**. The join will be a set of pairs of the form  $\langle s, c \rangle$  where  $s$  is a student name and  $c$  is a class. When will  $\langle s, c \rangle$  be in  $T$ ? Exactly when there is some number  $n$  such that  $\text{NumberIs}(s, n)$  and  $\text{Took}(n, c)$  are both true. This will happen when  $s$  is the name of a student who took class  $c$ .

In the exercises we’ll consider a database where there are three types  $A$ ,  $B$ , and  $C$  and relations  $R$  and  $S$  defined as above. For simplicity we will assume that the three types are each subranges of the naturals, of size  $\text{ASIZE}$ ,  $\text{BSIZE}$ , and  $\text{CSIZE}$  respectively. Thus an element of  $A$  is a Java `int` in the range from 0 through  $\text{ASIZE} - 1$ , inclusive, and similarly for  $B$  and  $C$ . We’ll consider three different ways to store  $R$  and  $S$ :

- Two boolean arrays  $R[\text{ASIZE}, \text{BSIZE}]$  and  $S[\text{BSIZE}, \text{CSIZE}]$ ,

<sup>1</sup>There are many variants of the join operation used in the database community – we offer only one illustrative example here.



- Two boolean functions `isInR(int,int)` and `isInS(int,int)`, which we will assume to throw exceptions if one of their inputs is out of range, and
- Two lists of objects, of type `RRecord` or `SRecord`, where these types have method `getA()`, `getB()`, and `getC()` as appropriate. For example, if `x` is an object of type `SRecord`, then `x.getC()` will return the second component of the pair represented by `x`, the component that is of type `C`.

The relation  $T \subseteq A \times C$  is defined to be the join of  $R$  and  $S$  as above.

### 2.2.1 Writing Exercises

1. Write a method that will construct a boolean array `T[ASIZE,CSIZE]` such that the boolean value `T[i,j]` tells whether the pair  $\langle i,j \rangle$  is in  $T$ . Your code should access the relations  $R$  and  $S$  by looking up values of the arrays `R` and `S` only.
2. Write a method `isInT(int,int)` that when given inputs  $i$  and  $j$  will return a boolean that tells whether  $\langle i,j \rangle$  is in  $T$ . You should access  $R$  and  $S$  using the methods `isInR` and `isInS` only. Your method should throw an exception if  $i$  or  $j$  is out of range, though this will probably happen through an exception being thrown by `isInR` or `isInS`.
3. Write a method `isInT` as in Exercise 2, but this time assume that  $R$  and  $S$  are given to you as lists of records of the types given above. Do not assume that these lists are sorted.
4. Write a method that produces  $T$  in the form of a list of records of the type `TRecord`, again assuming that  $R$  and  $S$  are given by similar lists. You may assume that a constructor `TRecord(int,int)` has been created that returns a `TRecord` object with two given components. Make sure that your list does not contain more than one copy of the same pair. You may assume that the lists for  $R$  and  $S$  are sorted, but if you do this you must produce a sorted list for  $T$ .

## 2.3 Quantifiers

### 2.3.1 What They Are and What They Mean

A predicate, such as “ $w$  is a string of two letters”, makes a statement about one or more objects represented by variables of particular types. The meaning of the statement depends on the value of the variables. In mathematical language, though, we’d like to make *absolute* statements that don’t depend on the particular values of variables, like “there is some string with two letters” or “every string has two letters”. These two types of statements, “there exists” and “for all”, occur so often in mathematics that special symbols called **quantifiers** have been defined to represent them, and a branch of mathematics called the **predicate calculus** has been developed to manipulate and prove statements containing quantifiers. In this section we will see how to use quantifiers and what they mean.

Remember that the variables of a predicate are called **free variables** when their meaning must be supplied to define the meaning of the predicate. The effect of a quantifier is to **bind** one of the variables, making it a **bound variable**, so that its role in the statement is determined by the quantifier. When we say “there exists a string  $w$  such that  $w$  has two letters,” for example, we don’t need to know what  $w$  is to know what the statement means. The statement is simply a proposition, one that in this case happens to be true. In the same way we could say “for every string  $w$ ,  $w$  has two letters,” and get a proposition that happens to be false.

**Definition:** A **quantified statement** is a statement built from a predicate  $P$  by one or more applications of the following procedure. We write the following in front of  $P$ :

- The **existential quantifier** symbol  $\exists$  (“there exists”) or the **universal quantifier** symbol  $\forall$  (“for all”<sup>2</sup>),
- a variable  $x$  of one of the types occurring freely in  $P$ ,
- a colon, to separate the parts of the statement, and
- $P$  itself, with  $x$  used according to its type.

**Example:** Let  $P(w)$  mean “ $w$  is a string with exactly two letters”. The quantified statement  $\exists w : P(w)$  means “there exists a string  $w$  such that  $w$  has two letters,” and  $\forall w : P(w)$  means “for every string  $w$ ,  $w$  has two letters”. We can equivalently state these two propositions in a form without explicitly mentioning  $w$ : “there is a string of two letters” and “every string has two letters”.

A peculiarity of the translation from symbolic language to English is that the colon separating the quantifier and variable from the rest of the statement has two different translations depending on context. With an existential quantifier the colon is usually read as “such that”, as in “there exists

---

<sup>2</sup>One can define quantifiers of many other kinds, and these are studied in the logic and computer science literatures. We’ll see two of these in Problems 2.3.4 and 2.3.5.

an  $x$  such that  $P(x)$  is true.” With a universal quantifier the colon is usually read as a comma, as in “for every  $x$ ,  $P(x)$  is true”. However, between two quantifiers of the same type we can also read the colon as “and”, as in “there exists an  $x$  and there exists a  $y$  such that  $P(x, y)$  is true” or “for all  $x$  and for all  $y$ ,  $P(x, y)$  is true”. We’ll have some more advice on translating quantifiers in Excursion 2.4.

Note that the name of the bound variable has no effect on the meaning of the quantified statement. As long as  $u$  or  $v$  are variables of the same type as  $w$  (here the type `String`), the statements  $\exists u : P(u)$ ,  $\exists v : P(v)$ , and  $\exists w : P(w)$  all mean exactly the same thing. However, the meaning of a bound variable is limited to its **scope**, the part of the statement that is affected by the quantifier. Consider the statements  $P(v) \wedge \exists v : P(v)$  and  $P(v) \wedge \exists w : P(w)$ . By what we just said, they should have the same meaning, and they do. But this means that the first statement is effectively using *two different* variables named  $v$ , which while syntactically correct is somewhat bad style. You may be familiar with the similar concept of scope in programming languages – when a variable is declared there is a particular area of the code in which it may be used.

We have been careful to assign types to all our variables, which is important because the meaning of quantified statements can depend crucially on the types of the variables involved. The type of the variable bound by a particular quantifier is also called the **universe of discourse** of that quantifier. For example, the statement  $\exists w : P(w)$  means precisely “there exists an object  $w$  of  $w$ ’s type such that  $P(w)$  is true”, and  $\forall w : P(w)$  means “for every object  $w$  of  $w$ ’s type,  $P(w)$  is true.” Changing the type changes the meaning: “all deer have antlers” is a very different statement from “all animals have antlers”. In programming or mathematics we are used to being careful about types<sup>3</sup>, but in examples taken from everyday language it may be less natural to do so.

## 2.3.2 Multiple Quantifiers

If we have a predicate with more than one free variable, there are many ways to form a quantified statement from it – we can apply either of the quantifiers to any of the variables. More importantly, we can apply quantifiers to the variables *in any order*, and the order we apply them may affect the meaning of the eventual statement. Let’s take the predicate “ $x \leq y$ ”, which has two free variables, and define the type of each variable to be `natural`. By applying a single quantifier to this predicate, we will get a quantified statement with one variable bound and the other still free. (This new statement is thus a predicate with one free variable.) There are four different ways to do this:

- We can bind  $x$  with an existential quantifier to get  $\exists x : x \leq y$ , which is read as “there exists a natural  $x$  such that  $x$  is less than or equal to  $y$ ”. This new statement turns out to be true for any  $y$ , because we could choose  $x$  to be 0.
- We can bind  $x$  with a universal quantifier to get  $\forall x : x \leq y$ , read as “for all  $x$ ,  $x$  is less than

<sup>3</sup>In Java and similar languages we must define the type of a variable as soon as we introduce it. In FORTRAN, an older language, variables had a default type of `integer` if they started with a letter from I through N, or `real` otherwise. This convention was adopted to mimic longstanding mathematical practice, where  $i$  and  $n$  are usually integer variables and  $x$  and  $y$  are usually real.

or equal to  $y$ ". This is false for any natural  $y$ , because there is always going to be a possible value of  $x$  that is bigger than  $y$  (such as  $y + 1$ ).

- We can bind  $y$  with an existential to get  $\exists y : x \leq y$ . This is true for any  $x$ , because we can always ensure that  $x \leq y$  is true by an appropriate choice of  $y$ , such as  $y = x + 1$ .
- We can bind  $y$  with a universal to get  $\forall y : x \leq y$ . If  $x = 0$ , this is true, because  $0 \leq y$  is true for any  $y$ . But otherwise, the possible choices for  $y$  would include the natural  $x - 1$ , for example, and this choice would make  $x \leq y$  false.

To make a *proposition* from  $x \leq y$ , we have to bind *both* variables. We can do this by taking any of the quantified statements above and binding its remaining free variable with another quantifier. Note that the new quantifier goes in front of the first one, so that the quantified statement is within its scope. The order in which we apply the quantifiers can be important. For example, the proposition

$$\forall x : \exists y : x \leq y$$

is true, because there is always a natural  $y$  greater than any given natural  $x$ . But

$$\exists y : \forall x : x \leq y$$

is false, because there is no *single* natural  $y$  that is greater than or equal to all  $x$ .

It turns out, though, that two quantifiers *of the same kind* can be exchanged without affecting the meaning of the statement. Consider  $\exists x : \exists y : x \leq y$  and  $\exists y : \exists x : x \leq y$ , for example. There is no difference between the English meanings of "there exists an  $x$  such that there exists a  $y$  such that  $x \leq y$ " and "there exists a  $y$  such that there exists an  $x$  such that  $x \leq y$ " – both are equivalent in meaning to "there exist an  $x$  and a  $y$  such that  $x \leq y$ ." Similarly both the combinations  $\forall x : \forall y :$  and  $\forall y : \forall x :$  can be read as "for every  $x$  and every  $y$ ," so interchanging two adjacent universal quantifiers also preserves the meaning.

We have to be particularly careful about the meaning of quantifiers whose universe of discourse has no elements at all. This might seem an absurd situation, but it is not as uncommon as you might think – for example a form might ask "Will all your children accompany you?" even if you have no children at all.)

Mathematics has adopted the convention that if there are no objects of the type, all  $\exists$  statements ranging over that type are false, and all  $\forall$  statements are true. An example used by Bertrand Russell was the type of "present Kings of France"<sup>4</sup>. Let  $B(x)$ , where  $x$  ranges over this type, mean " $x$  is bald". By our rule,  $\exists x : B(x)$  should be false, and  $\forall x : B(x)$  should be true. You are probably willing to agree that the proposition "There exists a present King of France who is bald" is false, but the truth of the proposition "Every present King of France is bald" is not quite as obvious. It might help to consider the following proof rules, which hold in part because of this convention:

$$\begin{aligned}\exists x : B(x) &\iff \neg \forall x : \neg B(x) \\ \forall x : B(x) &\iff \neg \exists x : \neg B(x)\end{aligned}$$

---

<sup>4</sup>France hasn't had a King since 1848, well before Russell was born.

So we can convert “Every present King of France is bald” to “There does not exist a present King of France who is not bald”, which is pretty clearly true. The best way to get a feel for the meaning of such statements is to practice using these rules.

### 2.3.3 Exercises

E2.3.1 Translate the following sentences into English. All variables are of type **natural**. Determine whether each is true or false.

- (a)  $\forall x : \forall y : (x + y) = (y + x)$
- (b)  $\forall y : \exists x : x^2 = y$
- (c)  $\forall x : \exists y : x^2 = y$
- (d)  $\exists x : \forall y : x^2 + 4 \leq 4x + y$

E2.3.2 Translate the following English statements into symbols. Use the following predicates first defined in Exercise 1.10.2:  $P(w)$  means “ $w$  is a string of two letters”,  $Q(w)$  means “ $w$  starts and ends with the same letter”, and  $R(v, w)$  means “ $v$  and  $w$  start with the same letter”. The type of all the variables should be “strings over the alphabet  $\{a, b\}$ ”.

- (a) There is a string of two letters that starts and ends with the same letter.
- (b) Every string of two letters starts and ends with the same letter.
- (c) If for every string, there is another string that starts with the same letter, then every string has two letters.
- (d) The string  $w$  starts and ends with the same letter, and every string that starts and ends with the same letter starts with the same letter as  $w$ .
- (e) If  $w$  is any string of two letters, and  $w$  starts and ends with the same letter, and  $w$  starts with the same letter as  $ab$ , then  $w$  is equal to  $aa$ .

E2.3.3 Which of the statements in Exercise 2.3.2 have free variables? For each one that does not, explain why it is true or why it is false. For each one that does, describe the set of strings or tuples of strings that make it true.

E2.3.4 For each statement in Exercise 2.3.2, write its *negation* in both symbols and English.

E2.3.5 In each of the following statements, identify which variables are free and which are bound.

- (a)  $\forall x : [(P(y) \rightarrow \exists z : (Q(x) \wedge R(y, z))) \vee R(x, y)]$ .
- (b)  $\exists x : \forall y : \exists z : (P(x) \wedge R(w, z) \wedge Q(z))$ .
- (c)  $(\forall x : R(x, y)) \rightarrow (\exists y : (R(x, y) \wedge Q(z)))$ .

E2.3.6 Here we use quantifiers to rewrite two definitions from earlier in the chapter.

- (a) In Problem 2.1.6 we defined the **projections**  $\pi_1(P)$  and  $\pi_2(P)$  of a binary relation  $P$ . Give quantified statements, using  $P$ , to define each of these new relations.
- (b) In Excursion 2.2 we defined the **join** of two binary relations. Restate this definition using a quantified statement.

- E2.3.7 In Excursion 1.2 we used the **Least Number Axiom**, which says that any nonempty set of naturals has a least element. Letting  $P(n)$  be the membership predicate for the set, formulate this axiom as a quantified statement.
- E2.3.8 In Excursion 1.2 we also used the concepts of **predecessor**, **successor**, **even number**, and **odd number** on naturals. Using addition, multiplication, and the inequality operators  $<$  and/or  $>$ , formulate quantified statements meaning “ $x$  is the predecessor of  $y$ ”, “ $x$  is the successor of  $y$ ”, “ $x$  is an even number”, and “ $x$  is an odd number”.
- E2.3.9 Find a quantified statement, using only one quantifier, that is logically equivalent to the statement “ $\exists x : \exists y : P(x) \wedge P(y)$ ”.
- E2.3.10 As in Problem 2.1.10, let  $C$  be a set of  $n$  candidates,  $J$  be a set of  $n$  jobs,  $Q(c, j)$  mean “candidate  $c$  is qualified for job  $j$ ”, and  $A(c, j)$  mean “candidate  $c$  is assigned to job  $j$ ”, Formulate a quantified statement that means “Each candidate is assigned to exactly one job, for which they are qualified.”

### 2.3.4 Problems

- P2.3.1 Using the given string predicates from Exercise 2.3.2 above, express the following relations in set builder notation (that is, determine their predicate form):
- $\{\langle aa, aa \rangle, \langle aa, ab \rangle, \langle ab, aa \rangle, \langle ab, ab \rangle, \langle ba, ba \rangle, \langle ba, bb \rangle, \langle bb, ba \rangle, \langle bb, bb \rangle\}$ .
  - $\{\langle aa, aa \rangle, \langle bb, bb \rangle\}$
  - $\{\langle ab, ba \rangle, \langle ab, bb \rangle, \langle ba, aa \rangle, \langle ba, ab \rangle\}$
- P2.3.2 Suppose that for *any* unary predicate  $P$  on a particular type  $T$ , you know that the proposition  $(\exists x : P(x)) \leftrightarrow (\forall x : P(x))$  is true. What does this tell you about  $T$ ? Justify your answer – state a property of  $T$  and explain why this proposition is always true if  $T$  has your property, and not always true if  $T$  does not have your property.
- P2.3.3 Let  $A$  and  $B$  be two types such that  $A$  is a *proper* subset of  $B$  (so that  $B$  contains all the elements of  $A$  plus at least one other element). Let  $P$  be a unary predicate on  $A$ , and let  $Q$  be a unary predicate on  $B$ . Suppose that  $\forall a : \forall b : P(a) \rightarrow Q(b)$  is true. Which of the following four statements is guaranteed to be true? For each statement, explain why it is always true or give an example where it is false. (Hint: Consider the case where  $A$  is empty.)
- $(\forall a : P(a)) \rightarrow (\forall b : Q(b))$
  - $(\forall b : Q(b)) \rightarrow (\forall a : P(a))$
  - $(\exists a : P(a)) \rightarrow (\exists b : Q(b))$
  - $(\exists b : Q(b)) \rightarrow (\exists a : P(a))$
- P2.3.4 The **unique existential quantifier**  $\exists!$  has the following interpretation:  $\exists! x : P(x)$  means “there is a unique  $x$  such that  $P(x)$  is true”, or equivalently “there is exactly one  $x$  such that  $P(x)$  is true”. Show how to write an expression with ordinary quantifiers and boolean operations (and the equals sign) that is equivalent to  $\exists! x : P(x)$ .

- P2.3.5 The **majority quantifier**  $M$  has the following interpretation, whenever there are only finitely members of the type over which it quantifies:  $Mx : P(x)$  means “ $P(x)$  is true for a majority of the possible  $x$ ”, or “ $P(x)$  is true for more  $x$  than it is false”. Consider the two sentences  $M\langle x, y \rangle : Q(x, y)$  and  $Mx : My : Q(x, y)$ . Explain the English meaning of these two sentences, where the type of  $x$  is “election districts”, the type of  $y$  is “voter number”, and  $Q(x, y)$  means “voter number  $y$  in election district  $x$  voted for the Silly Party”. Give examples from real political systems<sup>5</sup> where either of these sentences is equivalent to “the Silly Party wins the election”. Describe an example where one of the two sentences is true and the other is false.
- P2.3.6 (uses Java) Let  $X = \{x_1, \dots, x_n\}$  be a set of  $n$  objects and  $Y = \{y_1, \dots, y_n\}$  be a set of  $m$  objects. Let  $R$  be a binary relation from  $X$  to  $Y$ , represented by a two-dimensional array  $R$ , so that  $R[i, j]$  represents the truth value of the statement “ $\langle x_i, y_j \rangle \in R$ ” (or, equivalently, “ $R(x_i, y_j)$ ”).
- Write a method `boolean e (int j)` that takes one `int` argument and returns the truth value of the statement “ $\exists x : R(x, y_j)$ ”.
  - Write a method `boolean a (int j)` that takes one `int` argument and returns the truth value of the statement “ $\forall x : R(x, y_j)$ ”.
- P2.3.7 Let  $D$  be a set of dogs, with  $R$  being the subset of retrievers,  $B$  being the subset of black dogs, and  $F$  being the subset of female dogs, with membership predicates  $R(x)$ ,  $B(x)$ , and  $F(x)$  respectively. Suppose that the three statements  $\forall x : \exists y : R(x) \oplus R(y)$ ,  $\forall x : \exists y : B(x) \oplus B(y)$ , and  $\forall x : F(x) \oplus F(y)$  are all true. What can you say about the number of dogs in  $D$ ? Justify your answer.
- P2.3.8 Let  $X$  be a set and  $R(x)$  a unary predicate on  $X$ .
- Write a quantified statement that says “there are exactly two elements  $x$  in  $X$  such that  $R(x)$  is true”.
  - Write a quantified statement that says “there are at least three elements  $x$  in  $X$  with  $R(x)$  true, and at least three with  $R(x)$  false”.
- P2.3.9 Let  $D$  be a set of dogs and let  $T$  be a subset of terriers, so that the predicate  $T(x)$  means “dog  $x$  is a terrier”. Let  $F(x)$  mean “dog  $x$  is fierce” and let  $S(x, y)$  mean “dog  $x$  is smaller than dog  $y$ ”. Write quantified statements for the following, using *only* variables whose type is  $D$ :
- There exists a fierce terrier.
  - All terriers are fierce.
  - There exists a fierce dog who is smaller than all terriers.
  - There exists a terrier who is smaller than all fierce dogs, except itself.
- P2.3.10 A set of numbers is said to be **dense** if given any two distinct numbers  $x$  and  $y$ , there exists a number strictly between  $x$  and  $y$ .
- Write a quantified statement that is true if and only if the type we are quantifying over is dense.

---

<sup>5</sup>The Electoral College used to decide the U.S. president is similar to one of these, but more complicated because of the different sizes of the “election districts” (states).

- (b) Write the negation of this statement (so that it means that the set is *not* dense).
- (c) Argue that the set of rational numbers is dense.
- (d) Argue (using the statement of part (b)) that the set of naturals is not dense.



## 2.4 Excursion: Translating Quantifiers

In Excursion 1.11 we practiced translations from symbolic language to English and from English to symbolic language, in the case of statements involving fixed propositions, predicates, and boolean operations. Now that we have defined existential and universal quantifiers, we need to practice translating those as well.

In Section 2.3 we learned the basic rules for translating quantified statements into English. The prefix  $\exists x :$  becomes “there exists an  $x$  such that”, or if we prefer to specify the type of  $x$ , “there exists an object of  $x$ ’s type such that”. Similarly the prefix  $\forall x :$  becomes “for all  $x$ ,” or “for all objects of  $x$ ’s type,” with the comma included in the translated quote. The colon in the quantified statement is translated differently according to context, as “such that” after  $\exists$ , a comma after  $\forall$ , or “and” between two quantifiers of the same type.

Translating from English to symbols is sometimes not as straightforward, in part because of the difficulty of determining exactly what the words mean. A good practical way to proceed is often to rephrase the English and decide whether your rephrasing means the same thing. For example, Lewis Carroll once designed a logic puzzle including the statement “Babies are illogical”. This might mean “Some babies are illogical”, or “At least one baby is illogical”, but it’s probably best interpreted as “All babies are illogical” which gets translated into quantifiers as  $\forall x : B(x) \rightarrow I(x)$ . (Here we’ve defined a couple of predicates, “ $B(x)$ ” meaning “ $x$  is a baby” and  $I(x)$  meaning “ $x$  is illogical”.) You have to ask yourself whether “Given any creature, if it is a baby, then it is illogical” means exactly what is meant by the original English (most mathematicians, anyway, would say that it does).

Handy tricks in such rephrasing are some equivalences in the symbolic language. For example, we have the equivalence between  $\forall x : P(x)$  and  $\neg\exists x : \neg P(x)$ . So you might change “All babies are illogical” to “There is no baby who is not illogical”. There is a similar rule to turn an existential into a negated universal, by changing  $\exists x : P(x)$  to  $\neg\forall x : \neg P(x)$ . Handling negations is important when rephrasing statements using the *contrapositive rule* to reverse implications, as when we change “If  $0 = 1$ , then all trout live in trees” to “If it is not the case that all trout live in trees, then  $0 \neq 1$ ” and then to “If there exists a trout that does not live in trees, then  $0 \neq 1$ .” Negations can also move around inside an English statement and even inside words, so that “There is no baby who is not illogical” could just as easily be “There is no baby who is logical”.

### 2.4.1 Writing Exercises

Here are a few translations, which will help test where you are as far as being able to deal with the symbolic language. For the symbol to English translations, make the English as clear and readable as you can. This may involve a second pass after you’ve first written something to get the meaning clear. Unlike above, all these examples are from more-or-less real mathematical text<sup>6</sup>.

---

<sup>6</sup>This text contains a number of terms with which you may well not be familiar — the last exercise, for example, involves material from Chapter 8 of this book. But the translation should not depend on the meanings of these terms.

1. (Translate to English — all variables are of type **natural**. The symbol “.” denotes multiplication.)  $\forall a : \exists b : [(b > a) \wedge \neg[\exists c : \exists d : (c > 1) \wedge (d > 1) \wedge (c \cdot d = b)]]$
2. (Translate to symbols, using the following predicates (all variables are real numbers):  $C(a)$  means “ $a$  continually increases”,  $R(a, b)$  means “ $a$  remains less than  $b$ ”,  $L(a, b)$  means “ $a$  approaches a limit  $b$ ”.) “If  $x$  continually increases but remains less than some number  $c$ , it approaches a limit; and this limit is either  $c$  or some lesser number.” (Hint: Assign a variable to “the limit”.)
3. (Translate to symbols, using “ $|a - b| < c$ ” to represent “ $a$  is within  $c$  of  $b$ ”. If you like you may declare some variables to be of type **real** and some of type **positive real**.): “For every positive real number  $\epsilon$  there exists a positive real number  $\delta$  such that whenever a real number  $x$  is within  $\delta$  of  $x_0$ ,  $f(x)$  is within  $\epsilon$  of  $c$ .” What are the free variables in this statement? (Hint: Look carefully at the word “whenever”.)
4. (Translate to English, where all variables are of type “node”,  $EP$  means “the graph has an Euler Path”,  $E(a, b)$  means “there is an edge from  $a$  to  $b$ ”,  $P(a, b)$  means “there is a path from  $a$  to  $b$ ”, and  $O(a)$  means “ $a$  has an odd number of neighbors”):  $[\forall x : \exists y : E(x, y)] \rightarrow [EP \leftrightarrow ((\forall x : \forall y : P(x, y)) \wedge [\exists x : \exists y : \forall z : (x \neq y) \wedge (O(z) \leftrightarrow ((z = x) \vee (z = y)))]))]$

## 2.5 Operations on Languages

### 2.5.1 Language Concatenation

Quantifiers are commonly used in mathematics to *define* one concept in terms of another. Consider, for example, the join operation of Excursion 2.2 where we took two binary relations  $R$  and  $S$  and formed a new binary relation  $T$  from them. To define  $T$  in terms of  $R$  and  $S$ , we can give a quantified expression that is true of two elements  $a$  and  $c$  if and only if the pair  $\langle a, c \rangle$  is a member of  $T$ :

$$T(a, c) \leftrightarrow \exists b : (R(a, b) \wedge S(b, c)).$$

This expression refers to  $R$  and  $S$ , and tells us how to test a pair for membership in  $T$  if we are able to test pairs for membership in  $R$  or  $S$ .

In this section we will define several operations on formal languages – operations that in Chapter 5 will allow us to define the class of regular languages. Remember that a formal language over a fixed finite alphabet  $\Sigma$  is any subset of  $\Sigma^*$ , that is, any set of finite strings whose letters are all in  $\Sigma$ . Normally we consider strings over only one alphabet at a time.

Because languages are sets, any operation on sets is also an operation on languages. Thus if  $A$  and  $B$  are languages over  $\Sigma$ , then so are  $A \cup B$ ,  $A \cap B$ ,  $\bar{A}$ ,  $A \setminus B$ , and  $A \triangle B$ . As we saw in Section 1.5, the membership predicate for each of these new sets can be defined by boolean operations on the membership predicates for  $A$  and  $B$ .

Recall that the **concatenation** of two strings  $u$  and  $v$ , written  $uv$ , is the string obtained by writing  $v$  after  $u$  with nothing in between. We now define the *concatenation of two languages* in terms of concatenation of strings, using quantifiers.

**Definition:** Let  $A$  and  $B$  be two languages over  $\Sigma$ . The language  $AB$ , the **concatenation** of  $A$  and  $B$ , is defined as

$$\{w : \exists u : \exists v : (u \in A) \wedge (v \in B) \wedge (w = uv)\}.$$

Thus  $AB$  consists of those strings that can be obtained by concatenating *any* string in  $A$  with *any* string in  $B$ .

**Example:** Let  $A = \{ab, ba, bab\}$  and let  $B = \{a, ba\}$ . Let's systematically look at all the ways to choose a string from  $A$  and a string from  $B$ , so we can list all the elements of  $AB$ . (Of course we can only hope to do this because  $A$  and  $B$  are finite – if one or both were infinite we would have to determine  $AB$  in another way.) If we choose  $ab$  from  $A$ , we concatenate this with each element of  $B$  and find that  $aba$  and  $abba$  are in  $AB$ . By choosing  $ba$  from  $A$  we find that  $baa$  and  $baba$  are in  $AB$ . Then finally by choosing  $bab$  from  $A$  we find that  $baba$  and  $babba$  are in  $AB$ .

Each of the three choices from  $A$  gave rise to two strings in  $AB$ , making six total strings in  $AB$ . However two of these turned out to be the same as we constructed  $baba$  in two different ways, so  $AB$  has only five strings – it is the set  $\{aba, abba, baa, baba, babba\}$ . The size  $|AB|$  of  $AB$  can be *no greater than*  $|A||B|$ , but it can be less – how much less is a question we will explore in Exercise

### 2.5.4 and Problem 2.5.4.

Because the operation of concatenation on strings is not commutative ( $uv$  and  $vu$  are in general not the same string) we would not expect the operation of concatenation on languages to be commutative. With  $A$  and  $B$  as above, what is  $BA$ ? If we choose  $a$  from  $B$  we find that  $aab$ ,  $aba$ , and  $abab$  are in  $BA$ , and choosing  $ba$  from  $B$  we find that  $baab$ ,  $baba$ , and  $babab$  are there as well. So  $BA = \{aab, aba, abab, baab, baba, babab\}$  is not the same language as  $AB$ , and furthermore is it not even the same size, since it has six strings instead of five.

Concatenation of strings *is* associative, however, as  $(uv)w$  and  $u(vw)$  are always the same string. Therefore it is not surprising that concatenation of languages is also associative: if  $A$ ,  $B$ , and  $C$  are any languages,  $(AB)C$  and  $A(BC)$  are the same language. (Why? We'll explore this in Problem 2.5.2.) We take advantage of this fact in our notation, by writing a concatenation of three or more languages without parentheses, for example  $ABC$ .

Since we can concatenate any two languages, we can also concatenate a language with itself. Because we write concatenation in the same way as multiplication, we write repeated concatenation of a language with itself in the same way we write *powers* of numbers. Thus the language  $AA$  is written  $A^2$ ,  $AAA$  is written  $A^3$ , and so on. Clearly  $A^1$  is just  $A$  itself, but what about  $A^0$ ? We want to define this so that familiar rules like  $A^i A^j = A^{i+j}$  hold true, which means that we should have  $A^0 A^j = A^j$ , for example. Thus  $A^0$  should be the **identity element** for the concatenation operation, a language  $I$  such that  $IX = X$  for any language  $X$ .

Can we find such a language? Recall that the identity element for concatenation of strings is the empty string  $\lambda$ , because  $\lambda w$  and  $w\lambda$  are both equal to  $w$  for any string  $w$ . Thus  $\lambda$  is a likely element for our identity language  $I$ , and in fact it is the only element we need – we can take  $I = \{\lambda\}$ . Let's check that with this definition,  $IX = X$ . We need to show that for any string  $w$ ,  $w \in IX$  if and only if  $w \in X$ . Using the Equivalence and Implication Rule, this reduces to proving  $(w \in IX) \rightarrow (w \in X)$  and  $(w \in X) \rightarrow (w \in IX)$ . Each of these steps is easy using the definition of  $IX$ . If  $w \in IX$ , then  $w = uv$  for some  $u \in I$  and  $v \in X$ , but then  $u$  must be  $\lambda$  so that  $w = v$  and thus  $w \in X$ . Conversely, if  $w \in X$ , then because  $w = \lambda w$ ,  $\lambda \in I$ , and  $w \in X$ , it follows directly from the definition that  $w \in IX$ .

### 2.5.2 The Kleene Star Operation

Our final operation on languages is defined in terms of these powers. If  $A$  is any language, we define the **Kleene star** (usually just **star**) of  $A$  to be the language

$$A^* = A^0 \cup A^1 \cup A^2 \cup A^3 \cup \dots,$$

the union of *all* powers of  $A$ . Another way to write this using quantifiers, and a variable of type **natural**, is

$$(w \in A^*) \leftrightarrow \exists i : w \in A^i.$$

From this latter definition we can see that  $A^*$  is the set of strings that can be written as the concatenation of *any number* of strings from  $A$ .

We've implicitly used this notation already when we defined  $\Sigma^*$  to be the set of all strings whose letters are from the alphabet  $\Sigma$ . If we view  $\Sigma$  as a language, which means merely thinking of it as a set of one-letter strings rather than a set of letters, then the definition of the star operator tells us that  $\Sigma^*$  is the set of all strings that can be made by concatenating together any number of strings from  $\Sigma$ . Since any string can be made by concatenating the correct number of one-letter strings (possibly *zero* such strings, if the string is  $\lambda$ ), these two definitions of  $\Sigma^*$  are the same.

Similarly, if we apply the star operator to a *subset* of the alphabet (again regarded as a language of one-letter strings), we get an easily understood language. For example,  $\{a\}^*$  is the language consisting of all strings of  $a$ 's. The empty string, of course, is included in this language because it is a member (the only member) of the language  $\{a\}^0$ . When we say "strings consisting of  $a$ 's", we must remember that this includes strings with *no*  $a$ 's as long as they have no other letters either.

Some languages are easy to define by combining the star operator and concatenation of languages. For example, we can write  $\{a\}^*\{b\}^*$  to represent the concatenation of the all- $a$ 's language  $\{a\}^*$  and the all- $b$ 's language  $\{b\}^*$ . The resulting language consists of all strings that have zero or more  $a$ 's followed by zero or more  $b$ 's.

For a final example of the star operation, we return to our example language  $A = \{ab, ba, bab\}$  from above. What is  $A^*$ ? We can start to calculate:

$$\begin{aligned} A^0 &= \{\lambda\} \\ A^1 &= \{ab, ba, bab\} \\ A^2 &= \{abab, abba, abbab, baab, baba, babab, babba, babbab\} \\ A^3 &= \{ababab, ababba, ababbab, abbaab, abbaba, abbabab, abbabba, abbabbab, \\ &\quad baabab, baabba, baabbab, babaab, bababa, bababab, bababba, bababab, \\ &\quad bababbab, babbaab, babbaba, babbabab, babbabba, babbabbab\} \end{aligned}$$

We have only just started listing the elements of  $A^*$ , and since the language is infinite we can never hope to finish listing them! Even this simple definition and simple choice of  $A$  has given us a potentially interesting computational problem. Given an arbitrary string, how do we determine whether it is in  $A^*$  or not? We'll see how to do this in Chapter 14.

### 2.5.3 Exercises

E2.5.1 Let  $\Sigma = \{a, b\}$ . Describe the languages  $\{a\}\Sigma^*$ ,  $\{b\}\Sigma^*$ ,  $\Sigma^*\{a\}$ , and  $\Sigma^*\{b\}$  using set builder notation.

E2.5.2 Let  $\Sigma = \{a, b\}$  and describe the following languages, each of which is made up from the languages from Exercise 2.5.1 using set operations:

- (a)  $\{a\}\Sigma^* \cup \Sigma^*\{b\}$
- (b)  $\{a\}\Sigma^* \cap \Sigma^*\{b\}$
- (c)  $\{a\}\Sigma^* \cup \{b\}\Sigma^*$

(d)  $\Sigma^*\{a\} \cap \Sigma^*\{b\}$

E2.5.3 Let  $\Sigma = \{a, b\}$ ,  $X = \{aaa, ab\}$ ,  $Y = \{a, b, bb\}$ , and  $Z = \{a, aa, ab\}$ . List the elements of the following languages:

(a)  $XY$

(b)  $ZY$

(c)  $XYX$

(d)  $XYZ \cup ZXY$

E2.5.4 Let  $i$  and  $j$  be naturals. Show that there exist two languages  $A$  and  $B$  such that  $|A| = i$ ,  $|B| = j$ , and  $|AB| = i + j - 1$ . (Hint: start with  $i = j = 2$ .)

E2.5.5 Let  $\Sigma = \{a, b\}$  and view  $\Sigma$  as a language of one-letter strings. Describe the language  $\Sigma^3$ . If  $k$  is any natural, how many strings are in the language  $\Sigma^k$ ?

E2.5.6 Let  $A$  and  $B$  be languages over an alphabet  $\Sigma$ , with membership predicates  $A(w)$  and  $B(w)$ .

(a) Write a quantified statement, with variables of type **string** and the concatenation operation on strings, to express the statement  $AB = BA$ .

(b) Write a similar quantified statement to express the statement  $AB \neq BA$ .

E2.5.7 In general, the concatenation and Kleene star operations do not commute with one another.

(a) Give an example of two languages  $A$  and  $B$ , over the alphabet  $\Sigma = \{0, 1\}$ , such that the languages  $(AB)^*$  and  $A^*B^*$  are equal.

(b) Given an example of two languages  $A$  and  $B$  over  $\Sigma = \{0, 1\}$  such that  $(AB)^* \neq A^*B^*$ .

E2.5.8 Suppose that  $X$  is a finite language with  $n$  distinct strings, each of length exactly  $k$ . If  $t$  is any natural, how many strings are in the language  $X^t$ ? Justify your answer.

E2.5.9 Let  $\Sigma = \{a, b, \dots, z\}$  and let  $WL$  be a finite list of English words. The **Spelling Bee** game is to find as many words as possible in  $WL$  that are at least four letters long, use only letters in a particular set  $X$  of seven letters, and use one special letter in  $X$  at least once.

(a) Using Kleene star, language concatenation, and set operators, describe the set of strings that have four or more letters and only use letters from the set  $X = \{a, b, f, i, r, t, u\}$ .

(b) Describe the set of letters that meet the conditions of part (a) and also use the letter  $f$  at least once.

E2.5.10 A **pangram** in the Spelling Bee game of Exercise 2.5.9 is a string that meets the other conditions and also uses each letter in  $X$  at least once. Write an expression for the set of pangrams with the set  $X = \{a, b, f, i, r, t, u\}$ . Can you find such a pangram that is an English word?

## 2.5.4 Problems

P2.5.1 What is  $\emptyset^*$ ? What is  $\{\lambda\}^*$ ? Justify your answers. Explain why if  $A$  is any language *other than*  $\emptyset$  or  $\{\lambda\}$ ,  $A^*$  has infinitely many elements.

- P2.5.2 Let  $X$ ,  $Y$ , and  $Z$  be any three languages. Using the definition of concatenation of two languages, make quantified expressions for the membership predicates of the languages  $XY$ ,  $YZ$ ,  $(XY)Z$ , and  $X(YZ)$ . Explain how the set identity  $(XY)Z = X(YZ)$ , follows from your definitions of  $(XY)Z$  and  $X(YZ)$  by the commutativity of  $\exists$ . (This identity is called the **associativity of language concatenation**.)
- P2.5.3 Let  $A$  and  $B$  be finite languages. Suppose that if  $u$  and  $v$  are any two different strings in  $A$ , neither  $u$  nor  $v$  is a prefix of the other. (We say that  $u$  is a prefix of  $v$  if there exists a string  $w$  such that  $uw = v$ .) Explain why, under this assumption,  $|AB| = |A||B|$ .
- P2.5.4 Is the following statement true or false? “Let  $i$ ,  $j$ , and  $k$  be naturals with  $i + j - 1 \leq k$  and  $k \leq ij$ . Then there exist languages  $A$ ,  $B$  where  $|A| = i$ ,  $|B| = j$ , and  $|AB| = k$ .” Justify your answer.
- P2.5.5 (uses Java) Suppose we have boolean methods `isInA(String w)` and `isInB(String w)` to test membership in two languages  $A$  and  $B$ . Write a real-Java boolean method `isInAB(String w)` that decides whether  $w$  is in the language  $AB$ . (Hint: If we are to have  $w = uv$  with  $u \in A$  and  $v \in B$ , how many possible candidates are there for  $u$  and  $v$ ?)
- P2.5.6 Suppose that  $A$  is a language such that  $\lambda \notin A$ . Let  $w$  be a string of length  $k$ . Show that there exists a natural  $i$  such that for every natural  $j > i$ , every string in  $A^j$  is longer than  $k$ . Explain how this fact can be used to decide whether  $w$  is in  $A^*$ .
- P2.5.7 A finite language  $C$  is called a **prefix code** if there *do not* exist two strings in  $C$ , one of which is a proper prefix of the other.
- If  $\Sigma = \{a\}$ , explain exactly which languages over  $\Sigma$  are prefix codes.
  - If  $\Sigma = \{0, 1\}$ , describe all prefix codes over  $\Sigma$  that contain only strings of length at most 2. (Hint: There are exactly 26.)
  - If  $C$  is a prefix code with  $n$  strings in it, and  $k$  is a natural, how many strings are in the language  $C^k$ ? Justify your answer.
  - Explain why part (c) also provides an answer to Exercise 2.5.8.
- P2.5.8 (uses Java) In the Spelling Bee game of Exercise 2.5.9, suppose that we are given a pseudo-Java method `boolean inWL (string w)` that tells whether a given string is in the set  $WL$ .
- Write a pseudo-Java method `void spellingBee (char[ ] letters)` that will take an array of seven letters and list all the strings in  $WL$  that meet the conditions, with `letters[0]` being the special character that must be included). You may assume that no string in  $WL$  has more than 14 letters.
  - Now assume instead that  $WL$  is given to you in a file, so that you have methods `string getNext( )` for the next string in  $WL$  and `boolean eof( )` for tell whether there are any strings left in  $WL$ . Write a pseudo-Java method as in part (a) to list the strings in  $WL$  that meet the Spelling Bee conditions for a given set of letters.
  - Which of the methods in parts (a) and (b) will run faster, assuming a realistic word list? (Hint: The Oxford English Dictionary contains fewer than one million words.)
- P2.5.9 (uses Java) Write methods as in Problem 2.5.8 that produce lists of pangrams in the Spelling Bee game for a given set of letters. A pangram (see Exercise 2.5.10) is a word that meets the conditions and also uses each of the seven letters at least once.

P2.5.10 It is possible for two different finite languages  $X$  and  $Y$  to have the same Kleene star, that is, for  $X^* = Y^*$  to be true.

- (a) Prove that  $X^* = Y^*$  if and only if both  $X \subseteq Y^*$  and  $Y \subseteq X^*$ .
- (b) Use part (a) to show that  $X^* = Y^*$  if  $X = \{a, abb, bb\}$  and  $Y = \{a, bb, bba\}$ .
- (c) Prove that if  $X^* = Y^*$  and  $\lambda \notin X \cup Y$ , then the shortest string in  $X$  and the shortest string in  $Y$  have the same length.



## 2.6 Proofs With Quantifiers

### 2.6.1 The Four Proof Rules

Now that we have our two quantifiers and know what they mean, we can formulate rules for proving statements in the predicate calculus. We begin, of course, with all the rules for the propositional calculus, as the *data type* of quantified statements is still `boolean` and the propositional calculus applies to all objects of that type. So, for example, we know that

$$[(\exists x : A(x)) \wedge ((\exists x : A(x)) \rightarrow (\forall x : B(x)))] \rightarrow \forall x : B(x)$$

is a theorem of the predicate calculus — a statement that is true for any possible predicates  $A$  and  $B$ . Why? If we substitute  $p$  for “ $\exists x : A(x)$ ” and  $q$  for “ $\forall x : B(x)$ ”, it becomes “ $[p \wedge (p \rightarrow q)] \rightarrow q$ ”, which we can recognize as a tautology (the rule of Modus Ponens).

The more interesting proof rules, however, will deal with the meaning of the quantifiers. We’ve seen one already, in the rule of interchanging  $\neg\forall\neg$  with  $\exists$ , or  $\neg\exists\neg$  with  $\forall$ . But in a general forward-backward proof setting there are four basic situations that might come up, and each of them has its own special proof rule:

- A  $\exists$  quantifier in the premise, which would allow us to use the *Rule of Instantiation*,
- A  $\exists$  quantifier in the conclusion, which allows for the *Rule of Existence*,
- A  $\forall$  quantifier in the premise, which allows for the *Rule of Specification*, and finally
- A  $\forall$  quantifier in the conclusion, which allows for the *Rule of Generalization*.

Each of these situations also suggests a proof *strategy*, which tells you how you might break down your current forward-backward proof into a smaller subgoal, either by moving forward from the premise or backward from the conclusion. The strategies are useful whenever the premise or conclusion are stated in terms of quantifiers, which is quite often in mathematics. We’ll now take a more detailed look at each of the four situations, with an example of a simple proof using each<sup>7</sup>.

### 2.6.2 Examples For Each Rule

To begin, then, consider the situation where we are given a premise of the form  $\exists x : A(x)$ . The premise tells us that some object  $a$  of the correct type exists, for which  $A(a)$  is true. We don’t know anything about this object other than its type and the single proposition  $A(a)$ . What this rule will let us do is to *give a name* to this object so we can refer to it later. In English, we say “Let  $a$  be a thing such that  $A(a)$  is true,” and then use  $a$  in the rest of the proof. In symbols, we get the

---

<sup>7</sup>Each of the four situations is the subject of a chapter in Solow, where he goes through each proof strategy in considerably greater detail.

**Rule of Instantiation:** From the statement  $\exists x : A(x)$  you may derive the statement  $A(a)$ , where  $a$  is a new variable of the correct type.

For example, let's take the premise "There exists a pig with wings", or  $\exists x : (P(x) \wedge W(x))$ . Here the variable  $x$  is of type **animal**,  $P(x)$  means " $x$  is a pig", and  $W(x)$  means " $x$  has wings". The Rule of Instantiation allows us to conclude  $P(a) \wedge W(a)$ , where  $a$  is an **animal** about which we know only this single fact. Whether using this rule in this context is a good idea depends on what we're trying to prove, which brings us to the second situation.

If we are trying to prove a *conclusion* of the form  $\exists x : A(x)$ , we will want to make use of the following

**Rule of Existence:** From the statement  $A(a)$ , where  $a$  is *any* object of the correct type, you may derive the statement  $\exists x : A(x)$ .

This gives us a proof strategy of sorts to prove our conclusion. We think of some  $a$  such that  $A(a)$  is true<sup>8</sup>, prove  $A(a)$ , and use the Rule of Existence to conclude  $\exists x : A(x)$ . In our example, suppose we had "There exists a pig with wings" as our premise and wanted to prove "There exists an animal with wings", or  $\exists x : W(x)$ . Our new strategy says that we can get our conclusion if we can prove  $W(b)$ , for any  $b$  just as long as  $b$  is an **animal**<sup>9</sup>. Fortunately, we've already derived the statement  $P(a) \wedge W(a)$  from the premise, for some animal  $a$ . The propositional calculus rule of Right Separation gives us  $W(a)$ , and this is just what we need to get  $\exists x : W(x)$  by the Rule of Existence.

Now to universal quantifiers. The meaning of a premise of the form  $\forall x : A(x)$  is that  $A(a)$  is true for *any* object  $a$  of the correct type. The useful form of this fact for proofs is the following

**Rule of Specification:** If  $a$  is any object of the correct type, then from the statement  $\forall x : A(x)$  you may derive the statement  $A(a)$ .

For example, suppose that to our premise "There exists a winged pig" ( $\exists x : (P(x) \wedge W(x))$ ) we add the premise "All winged animals are birds" ( $\forall x : (W(x) \rightarrow B(x))$ ). How would we go about proving "There exists a pig that is a bird" ( $\exists x : P(x) \wedge B(x)$ )? Since we have an existential quantifier in the conclusion, our earlier strategy suggests that we find some animal  $b$  such that  $P(b) \wedge B(b)$  is true. We already have an animal  $a$  such that  $P(a) \wedge W(a)$  is true. From this we can get  $P(a)$  by separation, and if we could somehow prove  $B(a)$  we could get  $P(a) \wedge B(a)$  by joining and use  $a$  in the role of  $b$ . How can our second premise help us get  $B(a)$ ? We apply the Rule of Specification to it *with our own choice* of variable,  $a$ , getting  $W(a) \rightarrow B(a)$ . Now it's easy to finish the proof by using separation to get  $W(a)$  and Modus Ponens to get  $B(a)$ .

---

<sup>8</sup>Of course, this first step may remind you uncomfortably of the first step of comedian Steve Martin's method to become a millionaire and never pay income taxes ("First, get a million dollars..."), but at least the strategy gives us an idea of what we need.

<sup>9</sup>Why switch letters in the middle of the explanation? The basic rule is that we may use whichever letters we want as long as we don't use the same one twice in a context where it could lead to some false statement being introduced. It's usually best to be slightly paranoid and pick a different letter whenever any confusion might arise. Here,  $a$  is being used as the name of the winged pig provided by the premise, so we'll use another name for the winged animal we're about to prove to exist, even though they'll eventually prove to be the same animal.

The last situation, and the most complicated, is when the desired conclusion has a universal quantifier. What do we need to know in order to conclude  $\forall x : A(x)$ ?  $A(a)$  must be true for *any choice of  $a$  at all*. We can prove this by the following

**Rule of Generalization:** If, using only the assumption that  $a$  is of the correct type, you can prove  $A(a)$ , you may derive  $\forall x : A(x)$ .

In English, this tends to be expressed “Let  $a$  be arbitrary”, followed by a proof of  $A(a)$ , and the conclusion “Since  $a$  was arbitrary, we have proved  $\forall x : A(x)$ .”

For example, using the premise “All winged animals are birds” as before, we can prove “All winged pigs are birds”, or  $\forall x : [(P(x) \wedge W(x)) \rightarrow B(x)]$ . Let  $a$  be an arbitrary animal. We need to prove  $(P(a) \wedge W(a)) \rightarrow B(a)$ . Since this is an implication, we can use a direct proof, assuming  $P(a) \wedge W(a)$  and trying to prove  $B(a)$ . As before, we use the Rule of Specification on the premise to get  $W(a) \rightarrow B(a)$  for this particular arbitrary  $a$ , whereupon we can conclude  $B(a)$  by propositional calculus rules. Since  $a$  was arbitrary, and we proved  $(P(a) \wedge W(a)) \rightarrow B(a)$  without any assumptions, the Rule of Generalization allows us to conclude  $\forall x : [(P(x) \wedge W(x)) \rightarrow B(x)]$ .

You may have noticed that like the propositional calculus, the predicate calculus seems to be able to prove only obvious statements. If the truth of a statement has nothing to do with the *meaning* of the predicates, of course, we can’t expect to get any insight about the meaning. It’s somewhat more difficult (and beyond the scope of this book) to *prove* that the predicate calculus is complete (that is, that all true statements are provable), but this can be done.

The real importance of these proof strategies, though, is that they remain valid and useful even when other proof rules are added that *do* depend on the meaning of the predicates. We’ll see examples of this starting with the case of number theory in Chapter 3.

## 2.6.3 Exercises

E2.6.1 Indicate which quantifier proof rule to use in each situation, and outline how to use it:

- (a) The desired conclusion is “All trout live in trees”.
- (b) You have the premise “Tommy lives in trees”, and Tommy is a trout.
- (c) You have the premise “All trout live in trees”, and Tommy is a trout.
- (d) You have the premise “Some trout lives in trees”.

E2.6.2 Prove that the statements  $\forall x : \forall y : P(x, y)$  and  $\forall y : \forall x : P(x, y)$  are logically equivalent, by using the proof rules from this section to prove that each implies the other.

E2.6.3 Repeat Exercise 2.6.2 for the statements  $\exists x : \exists y : P(x, y)$  and  $\exists y : \exists x : P(x, y)$ .

E2.6.4 Use the proof rules to prove the statement  $\forall y : \exists x : P(x, y)$  from the premise  $\exists u : \forall v : P(u, v)$ . Is the converse of this implication always true?

E2.6.5 The law of vacuous proof can easily be combined with the Rule of Generalization to prove that *any proposition at all* holds for all members of an empty class. Demonstrate this by

proving both “All Kings of France are bald” ( $\forall x : K(x) \rightarrow B(x)$ ) and “All Kings of France are not bald” ( $\forall x : K(x) \rightarrow \neg B(x)$ ) from the premise “There does not exist a King of France” ( $\neg \exists x : K(x)$ , or  $\forall x : \neg K(x)$ ), using these two rules.

E2.6.6 If we know that the type  $X$  over which we are quantifying is finite and have a list of its elements, we can use this fact in proofs.

- (a) Write a quantified statement, with variables ranging over the type  $X$ , that says  $X = \{c, d\}$ .
- (b) Prove  $\forall x : P(x)$  from three premises: the statement of part (a),  $P(c)$ , and  $P(d)$ . (Hint: Use Proof By Cases as part of your Generalization.)

E2.6.7 Let  $D = \{c, d, s\}$  be the set of dogs consisting entirely of Cardie, Duncan and Scout, and let  $A = \{b, r, s\}$  be the set of activities consisting entirely of barking, retrieving, and swimming. The predicate  $L(x, y)$  means “dog  $x$  likes activity  $y$ ”. We will take as our premise the statement “Every dog likes at least two different activities.”

- (a) Write the premise as a quantified statement. Don’t forget to make the activities distinct.
- (b) Prove  $L(c, r) \vee L(c, s)$  from the premise.
- (c) Prove  $\forall x : \exists y : L(x, y) \wedge (y \neq b)$  from the premise.

E2.6.8 Define the following predicates over the type of “people”:  $WSD(x)$  means “ $x$  weighs the same as a duck”,  $MW(x)$  means “ $x$  is made of wood”, and  $IW(x)$  means “ $x$  is a witch”.

- (a) Translate the premises “all people who weigh the same as ducks are made of wood” and “all people who are made of wood are witches” into quantified statements.
- (b) Assume that person  $c$  weighs the same as a duck. Using the premises of (a), prove that person  $c$  is a witch.
- (c) Translate the conclusion “all people who weigh the same as ducks are witches” into a quantified statement, and prove it from the premises using Generalization and Specification. Note that the Hypothetical Syllogism rule cannot be used *inside* a quantifier.

E2.6.9 Let  $D$  be a set of dogs,  $A$  be a set of activities, and  $L(x, y)$  the predicate meaning “dog  $x$  likes activity  $y$ ”. Consider the premises “Every two dogs like some common activity” and “if any two dogs like the same activity, then they are the same dog”.

- (a) Translate the premises into quantified statements.
- (b) Prove a contradiction from these two premises and the third statement  $\exists x : \exists x' : x \neq x'$ , where the variables in the last statements are of type **dog**.
- (c) Is it possible for both the premises to be true? If so, how?

E2.6.10 As in Excursion 1.2, define the predicates  $E(n)$  and  $O(n)$  on naturals to mean “ $\exists k : n = 2k$ ” and “ $\exists k : n = 2k + 1$ ” respectively. Prove the statements  $\forall n : E(n) \rightarrow O(n + 1)$  and  $\forall n : O(n) \rightarrow E(n + 1)$  respectively, using quantifier proof rules. You do not need to justify standard facts about addition and multiplication.

### 2.6.4 Problems

P2.6.1 Following Lewis Carroll, take the premises “All angry dogs growl” ( $\forall x : (A(x) \wedge D(x)) \rightarrow G(x)$ ), “All happy dogs wave their tails”, “All angry cats wave their tails”, “All happy cats growl”, “All animals are either angry or happy”, and “No animal both growls and waves its tail”, and prove the conclusion that no animal is both a dog and a cat. Use predicate calculus and indicate which proof rule justifies each step. Proof by contradiction is probably simplest.

P2.6.2 In Robert Heinlein’s novel *The Number of the Beast*, the following two logic puzzles occur, in which one is to derive a conclusion from six premises. (Heinlein designed these in the spirit of Lewis Carroll.) Your task is to give formal proofs that the conclusions are valid. In the first, the type of the variables is “my ideas”, and the premises are:

- Every idea of mine, that cannot be expressed as a syllogism, is really ridiculous; ( $\forall x : \neg ES(x) \rightarrow RR(x)$ )
- None of my ideas about Bath-buns are worth writing down; ( $\forall x : B(x) \rightarrow \neg WWD(x)$ )
- No idea of mine, that fails to come true, can be expressed as a syllogism; ( $\forall x : \neg T(x) \rightarrow \neg ES(x)$ )
- I never have any really ridiculous idea, that I do not at once refer to my solicitor; ( $\forall x : RR(x) \rightarrow RS(x)$ )
- My dreams are all about Bath-buns; ( $\forall x : D(x) \rightarrow B(x)$ )
- I never refer any idea of mine to my solicitor, unless it is worth writing down. ( $\forall x : RS(x) \rightarrow WWD(x)$ )

The conclusion is “all my dreams come true”, or  $\forall x : D(x) \rightarrow T(x)$ . Prove this from the premises using the rules of propositional and predicate calculus.

P2.6.3 Heinlein’s second puzzle has the same form. Here you get to figure out what the intended conclusion is to be<sup>10</sup>, and prove it as above:

- Everything, not absolutely ugly, may be kept in a drawing room;
- Nothing, that is encrusted with salt, is ever quite dry;
- Nothing should be kept in a drawing room, unless it is free from damp;
- Time-traveling machines are always kept near the sea;
- Nothing, that is what you expect it to be, can be absolutely ugly;
- Whatever is kept near the sea gets encrusted with salt.

P2.6.4 We can now adjust our rules from Section 1.5 for translating set identities into the propositional calculus, by adding a quantifier to the translations of  $A \subseteq B$  and  $A = B$  to get  $\forall x : A(x) \rightarrow B(x)$  and  $\forall x : A(x) \leftrightarrow B(x)$ . Here “ $A(x)$ ” is an abbreviation for the predicate “ $x \in A$ ”. We can also use an existential quantifier to express the statement that a set is not empty — “ $\exists x : A(x)$ ” is equivalent to “ $A \neq \emptyset$ ”. Translate the following set statements into the predicate calculus, and prove them with the rules from this section.

<sup>10</sup>You must also translate the statements into formal predicate calculus — note for example the two different phrasings used for “is quite dry”. In the novel, the solver of the puzzle concludes (correctly) that the nearby aircar is also a time-traveling machine, but strictly speaking this is not a valid conclusion from the given premises.

- (a)  $[(A \subseteq B) \wedge ((A \cap C) \neq \emptyset)] \rightarrow [(B \cap C) \neq \emptyset]$ .
- (b)  $[(A \subseteq C) \wedge (B \subseteq C)] \rightarrow [(A \triangle B) \subseteq C]$ .
- (c)  $[(A \cup B) \neq \emptyset \wedge ((A \cap B) = \emptyset)] \rightarrow (A \triangle B) \neq \emptyset$ .
- (d)  $[(A \setminus B) \neq \emptyset \wedge ((C \setminus A) \neq \emptyset \wedge ((B \setminus C) \neq \emptyset))] \rightarrow [(C \subseteq B) \rightarrow (\overline{(B \cup C)} \neq \emptyset)]$ .

P2.6.5 If we have a single binary predicate  $P(x, y)$ , there are *eight* ways to make it into a sentence by quantifying the free variables  $x$  and  $y$ . Recall that in the Exercises we proved that  $\exists x : \exists y : P(x, y)$  and  $\exists y : \exists x : P(x, y)$  are equivalent, that  $\forall x : \forall y : P(x, y)$  and  $\forall y : \forall x : P(x, y)$  are equivalent, and that  $\exists y : \forall x : P(x, y)$  implies  $\forall x : \exists y : P(x, y)$ . It is also easy to prove that  $\exists x : \forall y : P(x, y)$  implies  $\forall y : \exists x : P(x, y)$ .

- (a) These are in fact the only implications among these eight statements that are tautologies (true for all possible relations  $P$ ). Explain why, by giving examples of relations where each other implication fails. (Hint: You may find empty types to be useful for some of your examples.)
- (b) Suppose you are given the additional assumption that at least one  $x$  and at least one  $y$  exist within their respective types. Can you prove any additional implications?

P2.6.6 Let  $I(x)$ ,  $x < y$ , and  $x \leq y$  be predicates on naturals, where we know that  $(x \leq y) \leftrightarrow [(x < y) \vee (x = y)]$ . Given the two premises

$$\forall x : \forall y : [I(x) \wedge \neg I(y)] \rightarrow (x < y)$$

and

$$\exists z : \neg I(z),$$

use the proof rules to prove the statement

$$\forall x : \exists y : (x \leq y) \wedge \neg I(y).$$

P2.6.7 Let  $D$  be a set of dogs including the particular dogs Cardie ( $c$ ) and Duncan ( $d$ ). Define the following predicates on  $D$ :  $W(x)$  means “dog  $x$  likes walks”,  $T(x)$  means “dog  $x$  is a terrier”,  $R(x)$  means “dog  $x$  is a Rottweiler”, and  $S(x, y)$  means “dog  $x$  is smaller than dog  $y$ ” or equivalently “dog  $y$  is larger than dog  $x$ ”. Using the first five of the statements below, prove the last statement using quantifier rules. (Translate the statements into symbolic ones before you start.)

- Duncan is a terrier.
- All terriers are smaller than Cardie.
- Cardie is smaller than some Rottweiler.
- If a first dog is smaller than a second dog, and the second is smaller than a third dog, then the first dog is smaller than the third.
- All dogs like walks.
- There is a Rottweiler who is larger than some terrier who likes walks.

P2.6.8 Let  $D$  be a set of dogs including the particular dog Nala ( $n$ ) and let  $C$  be a set of conditions including the particular condition “Spanish Inquisition ( $SI$ )”. Define the predicates  $T(x)$  meaning “dog  $x$  is a terrier”,  $E(x, y)$  meaning “dog  $x$  expects condition  $y$ ”, and  $F(x, y)$  meaning “dog  $x$  fears condition  $y$ ”. Translate the following statements into symbolic form, and use them to prove that Nala is not a terrier.

- No terrier fears any condition.
- Nala fears any condition that she does not expect.
- No dog expects the Spanish Inquisition.

P2.6.9 We now have some, but not all, of the tools we will need to formalize the proofs of Excursion 1.2, using the predicates  $E(n)$  and  $O(n)$  on naturals defined above in Exercise 2.6.10.

- Prove the quantified statement  $\forall n : (E(n) \leftrightarrow O(n+1)) \wedge (E(n+1) \leftrightarrow O(n))$ , using quantifier proof rules, the given definitions, and standard facts about addition and multiplication.
- Explain informally how the existence of a number that is both even and odd, or a number that is neither, would cause a contradiction given the statement of part (a), the Least Number Principle, and the fact that a natural has a predecessor if and only if it is not 0.

P2.6.10 One day, a set of five dogs met after their morning walks to compare their observations of birds. The set  $D$  of dogs consisted exactly of Arly ( $a$ ), Cardie ( $c$ ), Duncan ( $d$ ), Mia ( $m$ ), and Whistle ( $w$ ). The set  $S$  of bird species in question consisted exactly of Bluebird ( $B$ ), Crow ( $C$ ), Heron ( $H$ ), Mallard ( $M$ ), and Woodpecker ( $W$ ). The predicate  $R(x, y)$  means “dog  $x$  observed a bird of species  $y$ ”. Translate the following statements into symbolic language, and use them to prove that some dog observed a Heron.

- There was a species that was observed by all the dogs.
- Arly observed a Crow and no other species.
- Every dog other than Arly observed at least two different species.
- Each dog, other than Arly and Cardie, observed all the species that Cardie observed and at least one that Cardie did not. (Note: Different dogs may have observed different species that Cardie did not.)
- Duncan was the one and only dog to observe a Woodpecker, and Whistle was the one and only dog to observe a Bluebird.

## 2.7 Excursion: Practicing Proofs

In this Excursion we will apply our proof methods, for both the propositional and predicate calculi, to prove some statements about strings and formal languages.

First, recall the definitions of a *string* over a fixed alphabet, of the *concatenation* of two strings, and of the *concatenation product* of two languages. While we have a formal definition of the concatenation product of languages in terms of quantifiers and concatenation of strings, we lack the formal machinery to prove even some very basic facts about the latter concept. For example, we've defined the *reversal*  $w^R$  of a string  $w$  to be  $w$  written backwards. Playing around with examples should convince you of the general rules  $(w^R)^R = w$  and  $(uv)^R = v^R u^R$ , but we're not in a position to prove these facts until we have a more formal definition (which will come in Chapter 4). For now, we'll *assume* that these two statements are true.

A **palindrome** is a string  $w$  such that  $w^R = w$ . Examples are  $\lambda$ , *hannah*, and a statement attributed to Napoleon, “Able was I ere I saw Elba”, which we might represent as the string *ablewasiereisawelba*. We'll write  $Pal(w)$  to mean “ $w$  is a palindrome”. Now let's try to prove the following fact:

**Theorem:**  $\forall u : \exists v : Pal(uv)$ . That is, any string can be extended to give a palindrome.

**Proof:** It's usually good to start by trying a direct proof, trying to prove our goal statement with no assumptions. So let  $u$  be an arbitrary string, and we'll try to prove  $\exists v : Pal(uv)$  without any assumptions about  $u$  other than that it's a string (then, by the Rule of Generalization, we'll be done). How to prove  $\exists v : Pal(uv)$ ? If we can think of a particular  $v$  such that we can prove  $uv$  is a palindrome, then we'll be done by the Rule of Existence. What  $v$  would work? It presumably depends on  $u$ , and in fact  $u^R$  seems to work in examples. Can we prove  $Pal(uu^R)$  in general? Yes, given the observations above. We need to show that  $(uu^R)^R = uu^R$ . Applying the second rule above,  $(uu^R)^R = (u^R)^R u^R$  and, since  $(u^R)^R = u$  by the second rule, this is equal to  $uu^R$ . So a suitable  $v$  always exists, though note that because of the order of the quantifiers,  $v$  may depend on  $u$  rather than being the same for all  $u$ . Since we assumed nothing about  $u$ , we may use the Rule of Generalization to conclude  $\forall u : \exists v : Pal(uv)$ . ♠

Here's another example using the concatenation product of languages from Section 2.5:

**Theorem:** For any language  $L$ ,  $L\emptyset = \emptyset$ .

**Proof:** The first step is to note that this statement starts “ $\forall L$ ”, so we let  $L$  be an arbitrary language and plan to finish with the Rule of Generalization. By the definition of set equality, we want to show that any string is in one of these languages if and only if it's in the other. Unrolling the definition of  $L\emptyset$ , we find that we want to prove

$$\forall x : [x \in \{w : \exists u : \exists v : (w = uv) \wedge (u \in L) \wedge (v \in \emptyset)\}] \leftrightarrow [x \in \emptyset].$$

We can simplify the statement on the left to

$$\exists u : \exists v : (x = uv) \wedge (u \in L) \wedge (v \in \emptyset).$$

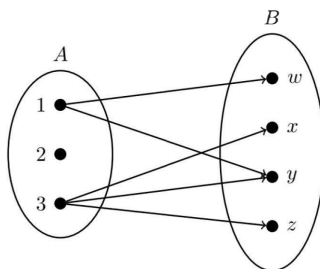


But  $v \in \emptyset$  is false for any  $v$ , and the AND of a false statement with anything else is false. The left half thus reduces to  $\exists u : \exists v : 0$  which is false for any  $x$ , so it is equivalent to  $x \in \emptyset$  and we're done. (Remember that  $\exists x : 0$  is definitely false, while  $\forall x : 0$  is true for an empty domain and false otherwise.  $\exists x : 1$  is true unless the domain is empty, while  $\forall x : 1$  is always true.) ♠

### 2.7.1 Writing Exercises:

Here are a few practice statements to prove. For this exercise, you should go slightly overboard in justifying your steps. All the small-letter variables are of type “string in  $A^*$ ” (where  $A$  is a non-empty finite set, and  $A^*$  is the set of all strings of letters in  $A$ ) and all the capital-letter variables are of type “language over  $A$ ” (or “subset of  $A^*$ ”).

1.  $\forall x : Pal(x) \leftrightarrow Pal(x^R)$ .
2.  $\forall L : \forall M : \forall N : L(M \cup N) = LM \cup LN$ .
3.  $\exists u : \forall v : \exists w : ((uv = w) \wedge (uw = v))$ .
4. Assuming  $\forall u : \exists v : (uv \in L)$  and  $\exists a : \forall y : \forall z : (yaz \neq y)$ , prove  $\neg \exists w : \forall x : [(x \in L) \rightarrow \exists y : (w = xy)]$ . Is this conclusion true without the first assumption? (The second assumption is true for strings over any nonempty alphabet.)



Source: David Mix Barrington

Figure 2-2: A relation on  $A = \{1, 2, 3\}$ ,  $B = \{w, x, y, z\}$ .

## 2.8 Properties of Binary Relations

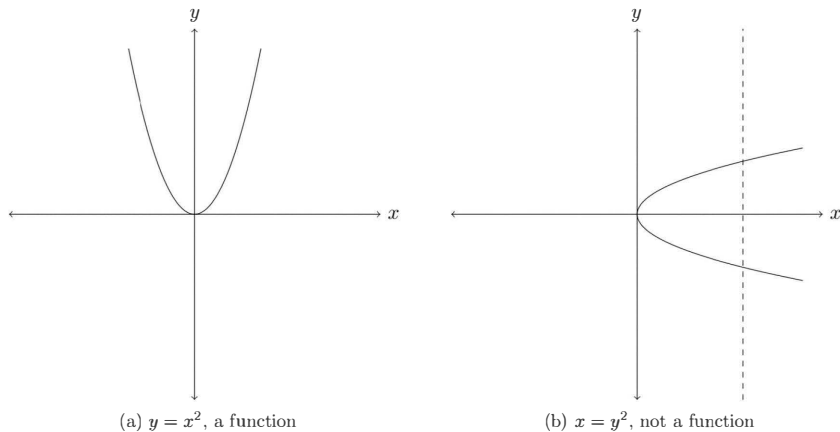
### 2.8.1 The Definition of a Function

In Section 2.1 we defined a binary relation to be a subset of the direct product  $A \times B$  for some sets  $A$  and  $B$ . For any given element  $a$  of  $A$ , there may or may not be an element  $b$  of  $B$  such that  $\langle a, b \rangle$  is in the relation, and if there is one such  $b$  there may or may not be more than one. We will sometimes call such a relation “a relation from  $A$  to  $B$ ”. Remember also that we may identify a relation with its corresponding predicate, so we may say “ $R(a, b)$ ” interchangeably with “ $\langle a, b \rangle \in R$ ”.

We can represent such a binary relation by a diagram like that in Figure 2-2. We draw a dot for each member of  $A$  on the left side of the diagram, a dot for each element of  $B$  on the right side, and an arrow from left to right for each element of the relation: If  $\langle a, b \rangle$  is in the relation we draw an arrow from  $a$  to  $b$ . Note that a given dot on the left might have no arrows, one arrow, or more than one arrow leaving it.

In Section 2.1 we noted that a **function** can be thought of as a certain kind of relation, containing the pair  $\langle x, f(x) \rangle$  for each element  $x$  of the domain. When can a relation be thought of as a function? A function must assign *exactly one* range element to each element of the domain. That is, for every element  $a$  there must be exactly one element  $b$  such that  $\langle a, b \rangle$  is in the relation. In terms of the diagram, there must be exactly one arrow out of each dot on the left. (Thus the relation of Figure 2-2 is certainly *not* a function.)

You may have been taught in a calculus class to recognize whether a particular equation “defines a function”. An equation with two variables, each ranging over the real numbers, always defines a relation from  $\mathbf{R}$  to  $\mathbf{R}$ , namely, the set of pairs of reals that satisfy the equation. The equation  $y = x^2$  defines a function from  $x$  to  $y$ , because for each value of  $x$  there is exactly one value of  $y$  making the equation true. The equation  $x = y^2$ , on the other hand, does not define a function because for every positive value of  $x$  there are *two* values of  $y$ , namely  $\sqrt{x}$  and  $-\sqrt{x}$ , that satisfy



Source: David Mix Barrington

Figure 2-3: The graphs of two equations on two real variables.

it. The relation defined by the equation  $x = y^2$  is not a function. Figure 2-3 shows the graphs of these two relations, and the vertical line intercepting the second graph twice to indicate that it is not the graph of a function.

Using quantifiers, we can give logical statements that define this property of being a function. It turns out to be convenient to regard the “exactly one” condition as the AND of two conditions.

**Definition:** We say that a binary relation is **total** if every element of the first set is associated with *at least one* element of the second:

$$\forall a : \exists b : \langle a, b \rangle \in R$$

Remember that the existential quantifier does not say anything about how many values of its variable satisfy its quantified part, as long as there is at least one.

**Definition:** A binary relation  $R$  is **well-defined** if for every first argument  $a$  there is *at most one* second argument  $b$  such that  $R(a, b)$  is true.

Expressing this property with quantifiers is not as straightforward as expressing totalness. One way to express it is to say what *does not* happen – there are never two different  $b$ ’s associated to the same  $a$ :

$$\neg \exists a : \exists b : \exists c : (\langle a, b \rangle \in R) \wedge (\langle a, c \rangle \in R) \wedge (b \neq c)$$

or, using the negation-of-quantifier rule three times:

$$\forall a : \forall b : \forall c : \neg[(\langle a, b \rangle \in R) \wedge (\langle a, c \rangle \in R) \wedge (b \neq c)]$$

or, rewriting the negated AND as an implication,

$$\forall a : \forall b : \forall c : [(\langle a, b \rangle \in R) \wedge (\langle a, c \rangle \in R)] \rightarrow (b = c).$$

The last version can be informally read in English as “if any element of  $A$  is related to two elements of  $B$ , those two elements are the same,” which is another way of saying “every element of  $A$  is related to at most one element of  $B$ ”. In terms of the diagram, the last statement says “for any dot on the left, any two arrows out of it go the the same place”.

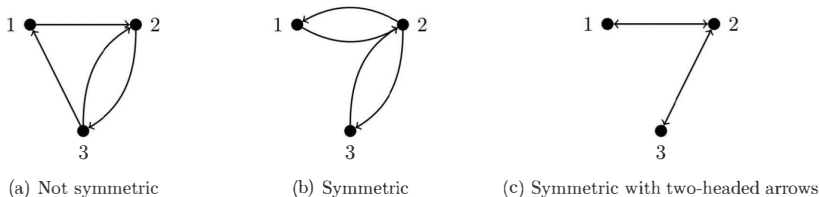
Once we have quantified statements with  $R$  as a free variable that say “ $R$  is total” and “ $R$  is well-defined”, we can take the AND of these two statements to get a single statement saying “ $R$  is a function”. In Section 2.9 we will explore functions further and use quantifiers to define some important properties of functions.

## 2.8.2 Properties of Binary Relations on a Single Set

It is possible, in fact even common, for the two sets in a binary relation to be the same set, giving us a **binary relation on a set**. (If the set is  $A$ , the relation is a subset of  $A \times A$ .) Two examples of such a relation are **the equality relation** ( $E(x, y) \iff x = y$ ) on any set, and **the order relation** on an ordered set (such as  $L(x, y) \iff x \leq y$  on the integers). We now need to look at two important types of binary relations on a set that generalize these two examples. Relations like equality are called *equivalence relations*, and relations like order (in the specific sense to be defined below) are called *partial orders*. To define these concepts, we need to look at some properties that a binary relation  $R$  on a set might or might not have.

**Definitions:** Let  $R$  be a binary relation on a set  $A$ .

- The relation  $R$  is **reflexive** if  $\forall x : R(x, x)$ , that is, if it always holds for an element and itself.
- The relation  $R$  is **antireflexive** if  $\forall x : \neg R(x, x)$ , that is, if it never holds for an element and itself. Note that this is different from being “not reflexive” – it is not hard to think of a relation that is neither.
- The relation  $R$  is **symmetric** if  $\forall x : \forall y : R(x, y) \rightarrow R(y, x)$ , that is, if you can always switch the variables in a true instance and keep it true.
- The relation  $R$  is **antisymmetric** if  $\forall x : \forall y : (R(x, y) \wedge R(y, x)) \rightarrow (x = y)$ . That is, if you switch the variables in a true instance, and the variables aren’t equal, you always get a false instance. Note that “antisymmetric” does not mean the same as “not symmetric” — it is not hard to think of a relation that is neither.



Source: David Mix Barrington

Figure 2-4: Some diagrams representing binary relations on the set  $\{1, 2, 3\}$ .

- The relation  $R$  is **transitive** if  $\forall x : \forall y : \forall z : (R(x, y) \wedge R(y, z)) \rightarrow R(x, z)$ . This means that a “chain” of two true instances involving the same variable  $y$  in this way can be “collapsed” to get another true instance.

The less-than-or-equals relation is reflexive, antisymmetric, and transitive. Any relation on a set that has these three properties is defined to be a **partial order**. Partial orders are the subject of Section 2.10 below.

The equality relation is reflexive, symmetric, and transitive. Any relation on a set that has these three properties is called an **equivalence relation**. Equivalence relations are the subject of Section 2.11 below.

Since binary relations on a single set are a kind of binary relation, we can represent them by diagrams of the kind shown in Figure 2.2, with a dot for each element of the set on the left and another dot for each element on the right. But with relations on a set we can also draw another kind of diagram, with only one dot for each element of the set. We still draw an arrow from  $a$  to  $b$  whenever  $\langle a, b \rangle$  is in the relation. Each of our new properties of relations can be described in terms of a particular property of this diagram:

- A relation is reflexive if and only if every dot in its diagram has a loop (an arrow from that dot to itself).
- A relation is antireflexive if and only if no dot in its diagram has a loop.
- A relation is symmetric if and only if every arrow in its diagram has a matching arrow in the other direction. We sometimes draw a symmetric relation using two-headed arrows as in Figure 2-4.
- A relation is anti-symmetric if and only if *no* arrow has a matching arrow in the other direction, unless this arrow is a loop.
- A relation is transitive if and only if every two-arrow path in the diagram has a corresponding arrow. (If there is an arrow from  $a$  to  $b$  and another from  $b$  to  $c$ , then there must be an arrow from  $a$  to  $c$ , and this holds even if two or more of  $a$ ,  $b$ , and  $c$  are equal.)

### 2.8.3 Exercises

- E2.8.1 Give an example of a relation, from one two-element set to another, that is total but not well-defined. Give an example (using the same sets) of a relation that is well-defined but not total. Draw diagrams for both your examples.
- E2.8.2 Consider the first quantified statement for “well-defined” given in the section. Explain why the condition “ $b \neq c$ ” is necessary for the statement to have the intended meaning. If we omitted this condition from the definition, exactly which relations would be “well-defined”?
- E2.8.3 Let  $A = \{1, 2, 3, 4\}$  and let  $B = \{w, x, y, z\}$ . For each of the following relations from  $A$  to  $B$ , draw its diagram, indicate whether it is total, and indicate whether it is well-defined. Justify your answers.
- (a)  $\{\langle 1, x \rangle, \langle 1, y \rangle, \langle 2, z \rangle, \langle 3, w \rangle, \langle 4, z \rangle\}$
  - (b)  $\{\langle 3, z \rangle, \langle 2, z \rangle, \langle 4, x \rangle, \langle 1, w \rangle\}$
  - (c)  $\{\langle 2, w \rangle, \langle 4, y \rangle\}$
  - (d)  $\{\langle 3, x \rangle, \langle 2, z \rangle, \langle 3, y \rangle\}$
  - (e)  $\emptyset$
  - (f)  $A \times B$
- E2.8.4 Give an example of a binary relation on a set  $A$  that is neither symmetric nor antisymmetric. Make your set  $A$  as small as you can.
- E2.8.5 Can a symmetric relation have loops in its diagram? Must it have loops in its diagram? Can it be reflexive? Must it be reflexive? If it can have loops, explain how the condition “every arrow in its diagram has a matching arrow in the other direction” is satisfied for the loop arrow. (Hint: How would you formally define “the matching arrow for the arrow from  $a$  to  $b$ ”, and what happens to this definition if  $a = b$ ?)
- E2.8.6 Draw diagrams (with only one dot per element) for the following binary relations on the set  $A = \{1, 2, 3, 4, 5\}$ . Indicate for each relation whether it is reflexive, symmetric, antisymmetric, or transitive.
- (a)  $\{\langle a, b \rangle : a + b = 6\}$
  - (b)  $\{\langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 2, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 2 \rangle, \langle 3, 5 \rangle, \langle 4, 3 \rangle, \langle 5, 3 \rangle\}$
  - (c)  $\{\langle a, b \rangle : (a = b) \vee (a + b = 3)\}$
  - (d)  $\{\langle a, b \rangle : a + 2b \leq 12\}$
- E2.8.7 Find a relation that is both reflexive and transitive but is neither symmetric nor antisymmetric. Use as few elements as possible in your base set.
- E2.8.8 Consider a finite set  $C$  of candidates, a finite set  $J$  of jobs, and a predicate  $Q(c, j)$  meaning “candidate  $c$  is qualified for job  $j$ ”. Do not assume that  $C$  and  $J$  are the same size.
- (a) What does it mean, in terms of candidates and jobs, for  $Q$  to be a total relation?
  - (b) What does it mean for  $Q$  to be a well-defined relation?
  - (c) What does it mean for  $Q$  to be a function?

- (d) What does it mean, in terms of  $Q$ , if it is possible to assign a *different* job to each candidate such that each candidate is qualified for their assigned job?

E2.8.9 Give an example of a binary relation on a set  $A$  that is neither reflexive nor antireflexive. Make your set  $A$  as small as you can.

E2.8.10 Consider the equation  $y^k = p(x)$ , where  $k$  is a natural and  $p(x)$  is a polynomial with real coefficients. Let the relation  $R_{k,p}$  from  $\mathbf{R}$  to  $\mathbf{R}$  be the set  $\{(x, y) : y^k = p(x)\}$ .

- (a) Show that if  $k$  is an odd number, then  $R_{k,p}$  is a function.
- (b) Let  $k$  is a positive even number. Give an example of polynomials  $p$  such that  $R_{k,p}$  is a function, such that it is not total, and such that it is not well-defined.
- (c) Show that if  $k = 0$ , then  $R_{k,p}$  cannot be a function.

### 2.8.4 Problems

P2.8.1 Let  $A = \{1, 2\}$  and  $B = \{x, y\}$ . There are exactly sixteen different possible relations from  $A$  to  $B$ . List them. How many are total? How many are well-defined? How many are functions? How many are neither well-defined nor total?

P2.8.2 Consider the following binary relations on the naturals (non-negative integers). Which ones are reflexive? Symmetric? Anti-symmetric? Transitive? Partial orders? Justify your claims.

- (a)  $A(x, y)$ , defined to be true if and only if  $y$  is even.
- (b)  $B(x, y)$ , defined to be true if and only if  $x < y$ .
- (c)  $C(x, y)$ , defined to be true if and only if  $x + 2 \geq y$ .
- (d)  $D(x, y)$ , defined to be true if and only if  $x \neq y$ .
- (e)  $E(x, y)$ , defined to be true if and only if the English *name* of  $x$  comes no later than the name of  $y$  in alphabetical order. (So, for example,  $E(8, 81)$  is true because **eight** comes before **eighty-one**, and  $E(8, 8)$  is true because **eight** comes no later than **eight**.)

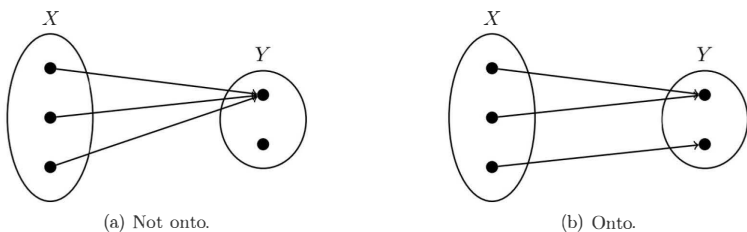
P2.8.3 Let  $A$  be a set and let  $R$  be a binary relation on  $A$  that is reflexive, symmetric, antisymmetric, and transitive. (Thus  $R$  is *both* a partial order and an equivalence relation.) Prove that  $R$  must be the equality relation on  $A$ .

P2.8.4 Let  $A$  and  $B$  be sets and let  $R \subseteq A \times B$  be a relation. The **inverse relation** of  $R$  is the subset  $R^{-1}$  of  $B \times A$  defined so that  $R^{-1} = \{(b, a) : (a, b) \in R\}$ . Prove or disprove each of the following statements:

- (a)  $R$  is total if and only if  $R^{-1}$  is total.
- (b)  $R$  is well-defined if and only if  $R^{-1}$  is well-defined.
- (c)  $R$  is a function if and only if  $R^{-1}$  is a function.
- (d) If  $R \subseteq A \times A$ ,  $R$  is reflexive if and only if  $R^{-1}$  is reflexive.
- (e) If  $R \subseteq A \times A$ ,  $R$  is symmetric if and only if  $R^{-1}$  is symmetric.
- (f) If  $R \subseteq A \times A$ ,  $R$  is antisymmetric if and only if  $R^{-1}$  is antisymmetric.
- (g) If  $R \subseteq A \times A$ ,  $R$  is transitive if and only if  $R^{-1}$  is transitive.

- P2.8.5 Let  $A = \{1, 2, 3\}$ . There are exactly 512 different possible binary relations on  $A$ .
- Describe how you could list them all, given enough time.
  - How many are reflexive? How many symmetric? How many antisymmetric?
  - (harder) How many are both reflexive and transitive? (This probably requires an exhaustive search.)
  - How many are equivalence relations? How many are partial orders?
- P2.8.6 (uses Java) We can represent a relation  $R$  from a finite set  $A$  to a finite set  $B$  by a Java array of booleans, so that `r[i][j]` tells whether the pair  $\langle i, j \rangle$  is in the relation.
- Write a static real-Java method `boolean isTotal(boolean [][] r)` that tells whether the input array represents a relation that is total.
  - Write a similar static real-Java method `isWellDefined` that tells whether  $R$  is well-defined.
  - Write a similar method to tell whether  $R$  is a function. (This can be one line of code using the methods of (a) and (b).)
- P2.8.7 (uses Java) We can represent a binary relation  $R$  on a finite set  $A$  by a Java array of booleans, so that `r[i][j]` tells whether the pair  $\langle i, j \rangle$  is in the relation. Write real-Java static methods to test whether the input array is:
- reflexive
  - antireflexive
  - symmetric
  - antisymmetric
  - transitive
- P2.8.8 A binary relation on an  $n$ -element set  $A$  has between 0 and  $n^2$  pairs.
- Show that for every natural  $i$  with  $0 \leq i \leq n^2$ , there exists a *symmetric* relation with exactly  $i$  pairs.
  - For what values of  $i$  can we have a *reflexive and symmetric* relation on  $A$  with exactly  $i$  pairs? (Hint: The answer depends on whether  $n$  is even or odd.)
  - For what values of  $i$  can we have an *antisymmetric* relation on  $A$  with exactly  $i$  pairs?
- P2.8.9 As in Problem 2.8.8, we can ask for what values of  $i$  we can have a *transitive* relation with exactly  $i$  pairs.
- Show that for  $n = 0$ ,  $n = 1$ , or  $n = 2$ , all values of  $i$  with  $0 \leq i \leq n^2$  are possible.
  - Show that if  $n \geq 3$ , we cannot have a transitive relation with exactly  $n^2 - 1$  pairs.
  - Show that with  $n = 3$  we can have a transitive relation with  $i$  pairs if  $0 \leq i \leq 7$  or  $n = 9$ .
  - (harder) For  $n > 3$ , what is the maximum number of pairs *other than*  $n^2$  that we could have in a transitive relation?
- P2.8.10 As in Exercise 2.8.8, let  $C$  be a set of candidates,  $J$  be a set of jobs, and define the relation  $Q \subseteq C \times J$  by the predicate  $Q(x, y)$  meaning “candidate  $x$  is qualified for job  $y$ ”. Find an example where  $C$  and  $J$  are finite sets of the same size,  $Q$  and  $Q^{-1}$  (as defined in Problem 2.8.4) are both total, but there is no way to assign each candidate a different job for which they are qualified.





Source: David Mix Barrington

Figure 2-5: Examples of onto and non-onto functions.

## 2.9 Functions

### 2.9.1 Surjections and Injections

In Section 2.8 we defined what it means for a relation to be a **function** — it must be both **total** and **well-defined**, so that there is exactly one output value for each input value. Here we will look at two properties that functions (or any other relations) might or might not have. These properties are the **duals**<sup>11</sup> of the properties of being total and of being well-defined, obtained from those properties by interchanging the roles of the two variables  $x$  and  $y$ .

**Definition:** Let  $R$  be a binary relation. We say that  $R$  is **onto** if every element of the range is “hit” by some input: “ $\forall y : \exists x : R(x, y)$ ”. (So we get this definition by switching  $x$  and  $y$  in the definition of “total”.) A function that is onto is called a **surjection**.

**Example:** In calculus, the relation  $R(x, y) = \{ \langle x, y \rangle : y = x^3 \}$  is onto because every real number is the cube of some number, its cube root. However,  $y = x^2$  is *not* onto as a function from the real numbers to the real numbers. (It becomes onto if we look at only the non-negative real numbers, because every one of those has a square root.) For another example on finite sets, see Figure 2-5.

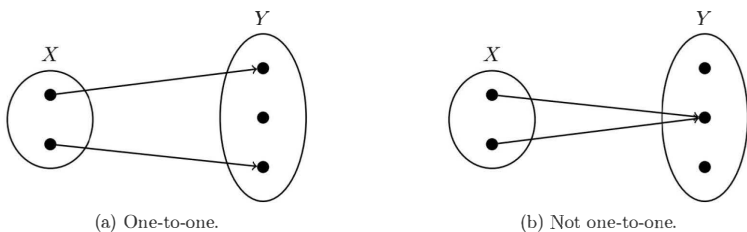
Note that proving a relation to be onto or not onto is a simple application of the predicate calculus rules. To show that it is, we let  $y$  be arbitrary and show that an  $x$  exists such that  $R(x, y)$  is true. To show that it isn’t, we find an example of a  $y$  such that for an arbitrary  $x$ ,  $R(x, y)$  is false.

**Definition:** We say that a relation  $R$  is **one-to-one** if it is never true that two different first arguments make the relation true for the same second argument:

$$\neg \exists x : \exists y : \exists z : (x \neq y) \wedge R(x, z) \wedge R(y, z).$$

(This is the dual obtained by switching the first-argument and second-argument variables in the definition of “well-defined”.) An equivalent way to write this statement is as a universally quantified

<sup>11</sup> “Dual” is one of those vague mathematical words that are best learned by seeing a lot of examples — a duality is what happens when you have two concepts that it’s meaningful to switch, like AND and OR. Dualities sometimes help you remember definitions or save time in proofs, because very often the dual of a valid statement is another valid statement.



Source: David Mix Barrington

Figure 2-6: Examples of one-to-one and non-one-to-one functions.

implication:

$$\forall x : \forall y : \forall z : ((R(x, z) \wedge R(y, z)) \rightarrow (x = y)).$$

If  $R$  is a function, this means that no two different inputs are mapped to the same output, and we call such a function an **injection**.

**Example:** The function  $y = x^3$  (by which we mean the relation  $\{(x, y) : y = x^3\}$ ) is one-to-one, but the function  $y = x^2$  (if the domain includes negative numbers) is not, because both  $x$  and  $-x$  are mapped to the same output. Figure 2-6 shows another example with finite sets.

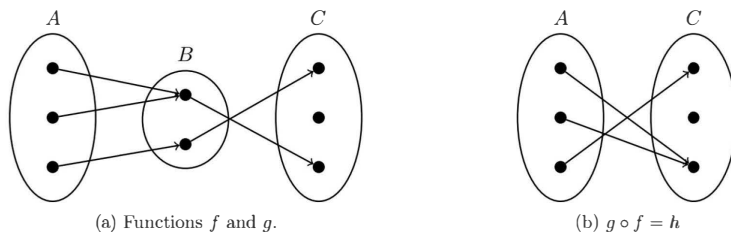
**Definition:** A function that is both one-to-one and onto is called a **bijection** or a **one-to-one correspondences** (sometimes also a **matching**). The function  $y = x^3$  is a good example of a bijection on the real numbers, as is the **identity function**  $y = x$  on any set at all.

## 2.9.2 Size of Finite Sets

Many of the Exercises and Problems we've seen so far have been of the form “*How many* objects exist that meet these conditions?”. Such problems form a division of mathematics called **combinatorics**, which will be our main topics for Chapters 6 and 7. While we won't yet start to go into systematic ways to solve such counting problems, we should note that some of the definitions we've just made are the starting point of combinatorics.

What happens, after all, when you count a finite set  $A$  in the real world? You assign numbers to the objects in  $A$ , thus creating a function whose domain is  $\{1, \dots, n\}$  for some natural  $n$  and whose range is  $A$ . But not every function with that domain and range will do to count the objects. If you assign two different numbers to the same object, your count will be wrong. And if there is one of the objects to which you don't assign any number, your count will be wrong. The first problem is exactly the failure of your function to be one-to-one, and the second is its failure to be onto. So what you need for a proper counting is a function that is both one-to-one and onto, which we've just defined as a bijection.

The following Theorem sets out the basic relationship between the size of finite sets and the existence of functions from one to another. We don't yet have the tools to prove this formally, but we'll explore



Source: David Mix Barrington

Figure 2-7: The composition of two functions.

some aspects of it in the Problems for this section.

**Size-Function Theorem:** Let  $A$  and  $B$  be finite sets with sizes  $|A|$  and  $|B|$ . Then:

- (1) There exists a bijection from  $A$  to  $B$  if and only if  $|A| = |B|$ .
- (2) There exists an injection from  $A$  to  $B$  that is not a bijection if and only if  $|A| < |B|$ .
- (3) There exists a surjection from  $A$  to  $B$  that is not a surjection if and only if  $|A| > |B|$ .

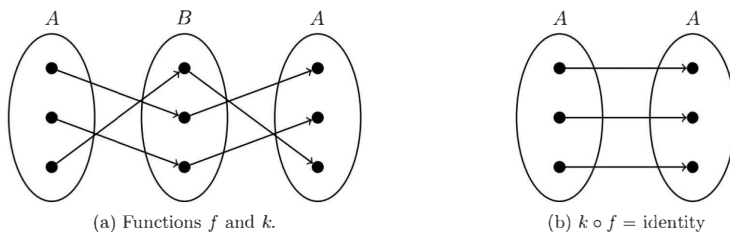
Note that we are stating this theorem only for finite sets, because in Section 1.1 we defined the size of a set only if it is finite. We'll see later (in Chapter 7) that the existence of a bijection will serve as the *definition* of “same number of elements” for infinite sets. The analogs of parts (2) and (3) of this Theorem, however, will not be true! For example, with  $A$  and  $B$  both equal to  $\mathbf{N}$ , we have both injections (such as  $f(n) = n + 1$  and surjections (such as  $f(n) = n - 1$ , with  $f(0) = 0$ ) from  $A$  to  $B$  that are not bijections.

### 2.9.3 Composition and Inverses

If  $f$  is a function from  $A$  to  $B$  and  $g$  a function from  $B$  to  $C$ , it's possible to take an element from  $a$ , apply  $f$  to it, and then apply  $g$  to the result. We can define a single function  $h$  from  $A$  to  $C$  by the rule  $h(x) = g(f(x))$ , and we define the **composition** of the two functions to be this function  $h$ . We also write this relationship<sup>12</sup> as  $h = g \circ f$ . Figure 2-7 illustrates the composition of two functions on finite sets.

If there is a function  $k$  from  $B$  to  $A$  such that  $k \circ f$  and  $f \circ k$  are each *identity functions* (whose

<sup>12</sup>You may have expected this composition, where  $f$  was performed first and then  $g$ , to be written  $f \circ g$  instead. But this notation is necessary because we are writing the function to the *left* of its argument, as we've done all along. The best way to remember this may be to note that the  $g$  and  $f$  stay in the same relative position as we go from  $g(f(x))$  to  $(g \circ f)(x)$ , and that once we know the domain and range types of  $f$  and  $g$  there is usually only one way the composition can be formed.



Source: David Mix Barrington

Figure 2-8:  $f$  and  $k$  are inverse functions.

output is always the same as their input) we say that  $k$  is the **inverse**<sup>13</sup> of  $f$ . This means that  $k$  has the effect of “undoing”  $f$  and vice versa, as doing first one and then the other has the same effect as doing nothing at all. For example,  $y = x^3$  and  $y = x^{1/3}$  are inverses of one another, as functions from the real numbers to themselves. Figure 2-8 shows another example with finite sets.

If a function has an inverse, we can show that both the functions are bijections. Consider  $f$  from  $A$  to  $B$  and  $k$  from  $B$  to  $A$ , as above. First we’ll prove that  $f$  is one-to-one. If  $x$  and  $y$  are two distinct arbitrary elements of  $A$ , for example, it can’t be true that  $f(x) = f(y)$ , because then  $k(f(x))$  and  $k(f(y))$  would be the same element, and because  $k \circ f$  is the identity function this element would have to be equal to both  $x$  and  $y$ .  $f$  must be onto as well, as any element  $z$  of  $B$  is hit by the element  $k(z)$  of  $A$  — since  $f \circ k$  is the identity function we know that  $f(k(z)) = z$ . Proving that  $k$  is a bijection requires only the same argument with the  $f$ ’s and  $k$ ’s reversed.

The connection between bijections and inverse functions is even closer than that, because every bijection *must* have an inverse. If  $f$  from  $A$  to  $B$  is a bijection, and  $y$  is an element of  $B$ , the onto and one-to-one properties together tell us that there is *exactly one* element  $x$  of  $A$  such that  $f(x) = y$ . We just define  $k(y)$  to be this element, and we have both that  $f(k(y)) = y$  and that  $k(f(x)) = x$ .

As another example, let’s prove that the composition of two injections is also an injection. Let the two original functions be  $f : A \rightarrow B$  and  $g : B \rightarrow C$ . We are given the assumptions  $\forall x : \forall y : (f(x) = f(y)) \rightarrow (x = y)$  and  $\forall x : \forall y : (g(x) = g(y)) \rightarrow (x = y)$ . We want to prove  $\forall x : \forall y : (g(f(x)) = g(f(y))) \rightarrow (x = y)$ . (Note that we’ve used the same variable names,  $x$  and  $y$ , in each of these three quantified expressions, even though the variables are of different types. In general it’s not hard to look at the unquantified part of each statement and determine what the type of each variable has to be for the statement to make sense. For example, in the first assumption and the conclusion the function  $f$  is applied to  $x$  and  $y$ , so the type of these variables must be  $A$ .)

This gives us a good chance to practice our general techniques to prove quantified statements. The statement we are trying to prove is a universal quantification, so we pick an arbitrary  $x$  and  $y$  from  $A$  (the correct data type), assume that  $g(f(x)) = g(f(y))$ , and try to prove  $x = y$ . We know from

<sup>13</sup>We can also define composition and inverses for relations other than functions, as for example by saying that  $(S \circ R)(x, z)$  is true if and only if  $\exists y : R(x, y) \wedge S(y, z)$ , and defining inverse in terms of composition as before.

the second assumption above that if  $g(w) = g(z)$  for any  $w$  and  $z$  in  $B$ , then  $w = z$ . So letting  $w = f(x)$  and  $z = f(y)$ , we can conclude  $f(x) = f(y)$ . We can then get  $x = y$  by applying the first assumption, without even renaming any variables.

In Problem 2.9.2 we'll prove that the composition of two surjections is a surjection, and thus that the composition of two bijections is a bijection.

## 2.9.4 Exercises

- E2.9.1 Let  $f(x) = x + 2$  and  $g(x) = 2x + 3$  be two functions from naturals to naturals. What are the functions  $f \circ g$  and  $g \circ f$ ? Are either of these functions injections, surjections, or bijections? Does either have an inverse?
- E2.9.2 Determine which of the following functions from naturals to naturals are injections, surjections, and bijections. If a function is a bijection, give its inverse.
- (a)  $f(x) = x^2 + 2x + 1$ .
  - (b)  $g(0) = g(1) = g(2) = 0$ , and for any  $x > 2$ ,  $g(x) = x - 3$ .
  - (c)  $h(x) = x + 1$  if  $x$  is even,  $h(x) = x - 1$  if  $x$  is odd.
  - (d)  $i(x) = x$ .
  - (e)  $j(x) = 7$ .
  - (f)  $k(x) = y$ , where  $y$  is the largest natural such that  $y^2 \leq x$ .
- E2.9.3 In each of the following examples, describe the domain (input type) and range (output type) of the two functions  $f$  and  $g$ . Determine whether either the composition  $f \circ g$  or  $g \circ f$  makes sense, and if so describe it as a function.
- (a)  $f(x)$  is the salary of employee  $x$ , and  $g(y)$  is the job title of employee  $y$ .
  - (b)  $f(x)$  is the job title of employee  $x$ , and  $g(y)$  is the salary that goes with job title  $y$ .
  - (c)  $f(x)$  is the employee who is the supervisor of employee  $x$ , and  $g(y)$  is the salary of employee  $y$ .
  - (d)  $f(x)$  is the tax paid on a salary of  $x$ , and  $g(y)$  is the salary of employee  $y$ .
- E2.9.4 Define the following functions from the set of strings over  $\{a, b\}$  to itself. If  $w$  is any string, let  $f(w) = w^R$ , let  $g(w) = wa$ , and let  $h(w) = v$  if  $w = va$  for some  $v$ , or  $h(w) = w$  if  $w$  does not end in  $a$ .
- (a) Describe the functions  $f \circ g$ ,  $f \circ h$ ,  $g \circ f$ ,  $g \circ h$ ,  $h \circ f$ ,  $h \circ g$ , and  $f \circ g \circ h$ .
  - (b) Are any of these three function surjections, injections, or bijections? Do any of them have inverses? If so, describe the inverses.
  - (c) Describe the functions  $f \circ f$ ,  $g \circ g$ , and  $h \circ h$ .
- E2.9.5 Prove that composition of functions is associative where it is defined. That is, if  $f$  is a function from  $A$  to  $B$ ,  $g$  from  $B$  to  $C$ , and  $h$  from  $C$  to  $D$ , prove that  $(h \circ g) \circ f$  and  $h \circ (g \circ f)$  are the same function from  $A$  to  $D$ . (Two functions with the same domain and range are defined to be equal if they have the same output for every input.)

- E2.9.6 Here we relate our new properties of relations to the definition of a function.
- (a) Prove that a binary relation  $R$  is onto if and only if its inverse relation  $R^{-1}$  is total.
  - (b) Prove that a binary relation  $R$  is one-to-one if and only if its inverse relation  $R^{-1}$  is well-defined.
  - (c) Explain why  $R$  is both onto and one-to-one if and only if  $R^{-1}$  is a function.
  - (d) Give an example where  $R$  is both onto and one-to-one but is not a function itself.
- E2.9.7 Let  $f$  and  $g$  be two bijections on a set  $A$ . What is the inverse of the function  $f \circ g$ , in terms of the inverse functions  $f^{-1}$  and  $g^{-1}$ ? Prove your answer.
- E2.9.8 Let  $A$  be a set of  $r$  elements and  $B$  be a set of  $n$  elements. How many possible different functions are there from  $A$  to  $B$ ? Explain your answer in the special cases of  $r = 0$ ,  $r = 1$ ,  $n = 0$ , and  $n = 1$ .
- E2.9.9 For what sets  $A$ , if any, can we be sure that any function from  $A$  to  $B$  is an injection? For what  $B$ , if any, can we be sure that any function from  $A$  to  $B$  is a surjection?
- E2.9.10 Fix a natural  $n$ . Let  $A$  be the power set of  $\{0, 1, \dots, n-1\}$  and let  $B$  be set of all binary strings of length  $n$ . Define a bijection from  $A$  to  $B$ , and its inverse from  $B$  to  $A$ .

## 2.9.5 Problems

- P2.9.1 Let  $f$  from  $A$  to  $B$  be any injection. Define  $C$  to be the set of range values hit by  $f$ , the set  $\{f(x) : x \in A\}$  or equivalently  $\{y : \exists x : y = f(x)\}$ . (This set is also often called “ $f(A)$ ”.) Let  $g$  be the function from  $A$  to  $C$  defined so that  $f(x) = g(x)$ . (Note that  $g$  is not the same function as  $f$  because it has a different range, though as a relation it consists of the same ordered pairs.) Prove that  $g$  is a bijection.
- P2.9.2 Prove that if  $f : A \rightarrow B$  and  $g : B \rightarrow C$  are both surjections, then so is their composition  $(g \circ f) : A \rightarrow C$ . Explain carefully why this, together with a result proved in the text of this section, implies that the composition of two bijections is a bijection.
- P2.9.3 Let  $f : A \rightarrow B$  and  $g : B \rightarrow C$  be functions such that  $g \circ f$  is a bijection. Prove that  $f$  must be one-to-one and that  $g$  must be onto. Give an example showing that it is possible for neither  $f$  nor  $g$  to be a bijection.
- P2.9.4 If  $f$  is a function from a set  $A$  to itself, we can compose  $f$  with itself. We call the composition of  $f$  with itself  $k$  times the  $k$ 'th **iterate** of  $f$ , and write it  $f^{(k)}$ .
- (a) If  $f(x) = x + 2$ , what is the function  $f^{(3)}$ ?
  - (b) If  $g(x) = x^2 + x + 1$ , what is the function  $g^{(3)}$ ?
  - (c) If  $i$  and  $j$  are any naturals, is it always true that  $(f^{(j)})^{(k)}$  is equal to  $f^{(jk)}$ ? Why or why not?
  - (d) How should we define  $f^{(0)}$ ? Why?
- P2.9.5 Let  $A = \{v, w\}$  and  $B = \{x, y, z\}$  be sets of characters.
- (a) List all the possible functions from  $A$  to  $B$ . Determine which are injections.

- (b) List all the possible functions from  $B$  to  $A$ . Determine which are surjections.
- (c) How many possible functions are there from  $A$  to  $A$ ? How many can be expressed as  $g \circ f$ , where  $f$  is a function from  $A$  to  $B$  and  $g$  is a function from  $B$  to  $A$ ?
- (d) How many possible functions are there from  $B$  to  $B$ ? How many can be expressed as  $f \circ g$ , where  $g$  and  $f$  are functions as above?

P2.9.6 Let  $f$  be any bijection from the set  $\{1, 2, 3\}$  to itself. Prove that the iterate  $f^{(6)}$  (as defined in Problem 2.9.4) is the identity function.

P2.9.7 Let  $A$  be a set and  $f$  a bijection from  $A$  to itself. We say that  $f$  **fixes** an element  $x$  of  $A$  if  $f(x) = x$ .

- (a) Write a quantified statement, with variables ranging over  $A$ , that says “there is exactly one element of  $A$  that  $f$  does not fix.”
- (b) Prove that if  $A$  has more than one element, the statement of part (a) leads to a contradiction. That is, if  $f$  does not fix  $x$ , and there is another element in  $A$  besides  $x$ , then there is some other element that  $f$  does not fix.

P2.9.8 Let  $A$  and  $B$  be finite sets and let  $R$  be a relation from  $A$  to  $B$  that is total but not well-defined.

- (a) Prove that there is a relation  $S$ , whose pairs are a subset of the pairs in  $R$ , that is a function from  $A$  to  $B$ .
- (b) Assuming in addition that  $R$  is one-to-one, prove that  $S$  is an injection.

P2.9.9 Here we will prove pieces of parts (2) and (3) of the Size-Function Theorem assuming that part (1) is true.

- (a) Assume that a function  $f$  from  $A$  to  $B$  exists that is an injection but not a bijection. Prove, using part (1) and the result of Problem 2.9.1, that there is a proper subset  $C$  of  $B$  such that  $|A| = |C|$ .
- (b) Assume that a function  $f$  from  $A$  to  $B$  exists that is a surjection but not a bijection. Prove, using part (1) and the results of Problems 2.9.1 and 2.9.8, that there is a proper subset  $D$  of  $A$  such that  $|D| = |B|$ . (Hint: Look at the inverse relation of  $f$ , which must be total and but not well-defined, and is both one-to-one and onto because  $f$  is a function.)

P2.9.10 In some cases the existence of an injection is enough to prove the existence of a bijection.

- (a) Prove (using the Size-Function Theorem) that if an injection  $f$  from  $A$  to  $A$  exists, then it must be a bijection.
- (b) Prove (using the Size-Function Theorem) that if  $A$  and  $B$  are two finite sets, and there exist injections  $f$  from  $A$  to  $B$  and  $g$  from  $B$  to  $A$ , then both  $f$  and  $g$  are bijections.

## 2.10 Partial Orders

### 2.10.1 Definition and Examples

In Section 2.8 we defined a **partial order** to be a binary relation, from some set  $A$  to itself, that has the following three properties:

- It is **reflexive**:  $\forall x : R(x, x)$ , that is, every element is related to itself,
- It is **antisymmetric**:  $\forall x : \forall y : (R(x, y) \wedge R(y, x)) \rightarrow (x = y)$ , that is, no two distinct elements are related both ways, and
- It is **transitive**:  $\forall x : \forall y : \forall z : (R(x, y) \wedge R(y, z)) \rightarrow R(x, z)$ , that is, if three elements of  $A$  are connected by a “chain” of two elements of the relation, the first and third elements must also be related.

The  $\leq$  and  $\geq$  relations have these three properties on the naturals, the real numbers, characters, strings (using lexicographic order), or any other **ordered set**. In fact, on all these sets  $\leq$  and  $\geq$  have an additional property as well:

- A relation is **fully comparable** if  $\forall x : \forall y : (R(x, y) \vee R(y, x))$ , that is, if any false instance can be made true by switching the variables.

This property is also sometimes called being “total”, but we will reserve that word for the property of relations that is part of being a function.

**Definition:** A **linear order**, also called a **total order**, is a partial order that is fully comparable.

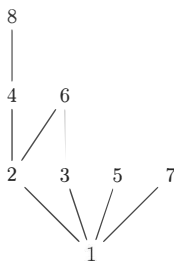
The reason for this name is that a straight line has the property that among any two distinct points, one is before the other. But there can be partial orders that are not linear orders. For example, the equality relation is a partial order – it is clearly reflexive and transitive, and it is antisymmetric because we know that if  $x \neq y$ ,  $E(x, y)$  and  $E(y, x)$  are both false.

**Example:** Another partial order that is not a linear order comes to us from number theory (the study of the naturals) and will be very important to us in Chapter 3. If  $a$  and  $b$  are naturals, then we say that  $a$  **divides**  $b$  if there exists some natural  $c$  such that  $b = a \cdot c$ . (In symbols,  $D(a, b) \iff \exists c : b = a \cdot c$ .) Equivalently, if you can divide  $b$  by  $a$  you get no remainder — in Java notation,  $b \% a == 0$ . (In Exercise 2.10.1 you are asked to prove that these two definitions are equivalent when  $a$  is positive.)

It’s easy to check systematically that this relation  $D$ , called the **division relation**, is a partial order.

- **Reflexive:** It is always true that  $D(a, a)$ , because  $1 \cdot a = a$ .





Source: David Mix Barrington

Figure 2-9: The Hasse diagram for the division relation on  $\{1, \dots, 8\}$ .

- **Antisymmetric:** Unless  $b = 0$ ,  $D(a, b)$  can only be true if  $a \leq b$ . So if both  $a$  and  $b$  are nonzero,  $D(a, b)$  and  $D(b, a)$  together force  $a \leq b$  and  $b \leq a$ , and thus  $a = b$ . If  $b = 0$ ,  $D(a, 0)$  is definitely true and  $D(0, a)$  is true only if  $a = 0$ , so the antisymmetry property holds.
- **Transitive:** Assume  $D(a, b)$  and  $D(b, c)$ , and we will prove  $D(a, c)$ . We know that  $b = a \cdot d$  for some  $d$  and that  $c = b \cdot e$  for some  $e$ . By arithmetic we have  $c = a \cdot (d \cdot e)$  and  $d \cdot e$  is the number needed to show  $D(a, c)$ .

In Problem 2.10.4 we'll define the **substring** relation on strings and show that it is also a partial order that is not total.

If a partial order is on a finite set, we can represent it pictorially by a **Hasse diagram**. This is a finite graph (a picture with finite number of dots and lines) where each element of the base set is represented by a dot, and element  $a$  is below element  $b$  in the partial order (that is,  $P(a, b)$  is true) if and only if you can go from dot  $a$  to dot  $b$  by going upward along lines (we say in this case that  $a$  is “**path-below**”  $b$ ). Another way to say this is that you draw a line from  $a$  to  $b$  if and only if  $a$  is below  $b$  and there is nothing between  $a$  and  $b$  in the partial order.

Let's have a look at the Hasse diagram for the division partial order on the set of numbers  $\{1, 2, 3, 4, 5, 6, 7, 8\}$ . We can easily list the pairs of numbers in the relation: Every number divides itself, 1 divides all of them, 2 divides the four even numbers, 3 divides 6, and 4 divides 8. Clearly 1 will go at the bottom, but what should be on the next level? 2 must be under 4, 6, and 8, 3 under 6, and 4 under 8. So 2, 4, and 8 are all on different levels, forming a vertical chain. We can put 3 on the same level as 2, with 6 above it and also above 2. Then 5 and 7 can also go on the same level as 2, with just the lines to them up from 1. The resulting picture is shown in Figure 2-9.

## 2.10.2 The Hasse Diagram Theorem

Hasse diagrams are a convenient way to represent a partial order, but is it always possible to use one to represent any possible partial order? This is a serious mathematical question. We'll answer

it by stating and proving a theorem, although because of our current lack of mathematical tools<sup>14</sup> the proof won't be entirely rigorous<sup>15</sup>. With a convincing informal argument, we can see which properties of and facts about Hasse diagrams and partial orders are important for the relationship between them.

**Hasse Diagram Theorem:** Any finite partial order is the “path-below” relation of some Hasse diagram, and the “path-below” relation of any Hasse diagram is a partial order.

**Proof:** The second statement is fairly easy to prove and is left as Problem 2.10.1: we only have to verify that the “path-below” relation of any Hasse Diagram is reflexive, anti-symmetric, and transitive.

To prove the first statement, we must show that given a finite partial order  $P$  we can always draw the diagram. (This is a useful thing to know how to do in any case.) We already know where we want to put the lines on the diagram, from the definition of a Hasse diagram: there should be a line from  $a$  to  $b$  if and only if  $P(a, b)$  is true *and* there is no distinct third element  $c$  such that  $P(a, c)$  and  $P(c, b)$ . As a quantified statement, this becomes

$$L(a, b) \Leftrightarrow P(a, b) \wedge \neg \exists c : ((c \neq a) \wedge (c \neq b) \wedge P(a, c) \wedge P(c, b)).$$

If we have a table of values of  $P(x, y)$  for all possible  $x$  and  $y$ , we can use this definition to test  $L(a, b)$ . But how do we know that we can put the dots on our diagram in such a way that we can draw all these lines? (Remember, if  $P(a, b)$  then any line from  $a$  to  $b$  must go *upward*.)

Here is a systematic procedure that we can use to lay out the elements. We first need to find a **minimal element**<sup>16</sup> of the partial order, which is an element  $m$  such that  $P(a, m)$  is true *only* when it has to be, when  $a = m$ .

**Lemma:** Any partial order on a finite nonempty set has a minimal element.

**Proof:** Informally, here is how you can always find a minimal element. Since the set has elements we may start with any element at all, say,  $a$ . If  $a$  is minimal, you are done. If not, there must be an element  $b$  such that  $P(b, a)$  and  $b \neq a$  (Why? From the *very definition* of  $a$  not being minimal.) If  $b$  is minimal, you've got your minimal element, otherwise you continue in the same way, by finding some element  $c$  such that  $P(c, b)$  and  $c \neq b$ .

How could this process possibly stop? You could find a minimal element, which is what you want. But actually *no other outcome is possible*. Since there are only a finite number of elements, you can't go on forever without hitting the same one twice. And as we'll show in Problem 2.10.2, it's simply not possible in a partial order to have a **cycle**, which is what you'd have if you did hit the

<sup>14</sup>Particularly mathematical induction from Chapter 4.

<sup>15</sup>This proof is in some sense the first piece of “real mathematics” in this book. Because we only just barely have the tools to make the argument, it turns out to be rather complicated with several lemmas relegated to the Problems. Some instructors will find their students ready to tackle this proof at this point, and others may prefer to have them skim it or skip it.

<sup>16</sup>This is not the same as a **minimum** element, which would have  $P(m, a)$  for *every* element  $a$ . Similarly, a **maximal** element has  $\forall x : P(m, x) \rightarrow (x = m)$ , and a **maximum** element has  $\forall x : P(x, m)$ .

same element twice. So a minimal element always exists<sup>17</sup>, and we've proved the Lemma. ♠

Once we have a minimal element  $a$ , we draw a dot for it at the bottom of our diagram. We'll never need to draw a line up to  $a$ , because it's minimal. But now we have to draw all the lines up *from*  $a$  to other elements — to do this we find all elements  $z$  such that  $P(a, z)$  is true and there is no  $y$  such that  $P(a, y)$ ,  $P(y, z)$ , and  $y \neq z$ . All these lines can be drawn upward because  $a$  is below everything else. But we can't draw them until we know where to put the dots for all the other elements!

Consider the set of elements remaining if we ignore  $a$ . It's still a partial order, because taking out a minimal element can't destroy any of the partial order properties. (Why? See Problem 2.10.3.) So *it* has a minimal element  $b$ , by the same reasoning. We can put the dot for  $b$  just above the dot for  $a$ , below anything else, because we'll never need to draw a line up to it except maybe from  $a$ . Once  $b$ 's dot is drawn, we take care of the upward lines from  $b$  and then consider the partial order obtained by removing  $b$  as well, continuing in the same way until all the dots are placed and we have a Hasse diagram<sup>18</sup>.

We are almost done with the proof of the theorem — we just have to verify that the construction worked as intended.

**Lemma:** The path-below relation for the constructed Hasse diagram is exactly the partial order we were given.

**Proof:** We must show that  $P(a, b)$  is true if and only if we have an upward path from  $a$  to  $b$  in the constructed diagram. This equivalence proof breaks down naturally into two implications. For the easier implication, assume that there is an upward path from  $a$  to  $b$ . We can use transitivity to prove  $P(a, b)$ , because  $P$  holds for every line in the diagram by the construction. If, for example, the path from  $a$  to  $b$  goes through  $e$ ,  $f$ , and  $g$  in turn, we know  $P(a, e)$ ,  $P(e, f)$ ,  $P(f, g)$ , and  $P(g, b)$ , and the transitive property allows us to derive first  $P(a, f)$ , then  $P(a, g)$ , and finally  $P(a, b)$ .

It remains to prove that if  $P(a, b)$  is true, then there must be a path from  $a$  up to  $b$  in the diagram. Again, we'll prove that it exists by explaining how to construct it out of lines that exist. If  $a = b$  then we have a path without doing anything. Otherwise, remember that there is a line from  $a$  to  $b$  if and only if there is no  $c$  with  $P(a, c)$  and  $P(c, b)$ . Either there is a line, then, in which case we have a path, or such a  $c$  exists. Now we apply the argument recursively to the pairs  $\langle a, c \rangle$  and  $\langle c, b \rangle$ . Either there is a  $d$  between  $a$  and  $c$ , for example, or there isn't. If there is, we recurse again, and if there isn't the construction assures us of a line from  $a$  up to  $c$ . The process must stop sometime, because there are only so many elements in the partial order and we can't (as proved in Problem 2.10.2) create a cycle. ♠

We've now finished the proof of the Hasse Diagram Theorem. ♠

---

<sup>17</sup>This argument is, of course, a disguised form of mathematical induction — you might want to take another look at it after Chapter 4.

<sup>18</sup>Technically, we have just put the elements into a linear order that is *consistent* with the given partial order, by a recursive algorithm. Common sense says that this will work, and later we will be able to prove this by mathematical induction.

### 2.10.3 Exercises

E2.10.1 Prove that the two definitions of “divides” given in the text are equivalent. That is, prove that for any two naturals  $a$  and  $b$ , with  $a > 0$ ,  $\exists c : b = a \cdot c$  if and only if  $b \% a == 0$ .

E2.10.2 Which of the following binary relations are partial orders? Which are total orders?

- (a) Baseball player  $x$  has a higher batting average than baseball player  $y$ .
- (b) Baseball player  $x$  has a batting average equal to or higher than baseball player  $y$ .
- (c) Natural  $x$  is equal to or less than half of natural  $y$  ( $(x = y) \vee (2x < y)$ ).
- (d) Over the alphabet  $\{a\}$ , string  $x$  is a prefix of string  $y$ , that is,  $\exists z : xz = y$ .
- (e) Natural  $x$  is less than twice natural  $y$  ( $x < 2y$ ).

E2.10.3 Follow the procedure from this section to make a Hasse diagram from the following partial order, successively finding and removing minimal elements:

$$\{\langle a, a \rangle, \langle a, d \rangle, \langle b, a \rangle, \langle b, b \rangle, \langle b, d \rangle, \langle b, e \rangle, \langle b, f \rangle, \langle c, a \rangle, \langle c, c \rangle, \langle c, d \rangle, \\ \langle c, e \rangle, \langle c, f \rangle, \langle d, d \rangle, \langle e, d \rangle, \langle e, e \rangle, \langle f, a \rangle, \langle f, d \rangle, \langle f, e \rangle, \langle f, f \rangle\}.$$

E2.10.4 Draw the Hasse diagram for the relation  $D(x, y)$  on the numbers from 1 through 24.

E2.10.5 Is an *infinite* partial order guaranteed to have a minimal element? Argue why or why not.

E2.10.6 Let  $R$  be a partial order on a set  $A$  and  $S$  be a partial order on another set  $B$ . Define a new relation  $T$  on the direct product  $A \times B$  as follows:  $T(\langle a, b \rangle, \langle a', b' \rangle)$  is true if and only if both  $R(a, a')$  and  $S(b, b')$  are true. Prove that  $T$  is a partial order.

E2.10.7 Let  $A$  and  $B$  be two disjoint sets,  $R$  be a partial order on  $A$ , and  $S$  be a partial order on  $B$ . We'll define two new binary relations on the union  $A \cup B$  as follows.

- (a) Let  $P$  be the union of  $R$  and  $S$ . Prove that  $P$  is a partial order.
- (b) Let  $Q$  be the union of  $R$ ,  $S$ , and  $A \times B$ . Prove that  $Q$  is a partial order.

E2.10.8 Here are two more relations for you to prove to be partial orders.

- (a) Let  $X$  be the power set of  $\{0, 1, \dots, n\}$  where  $n$  is a natural. Define the binary relation  $S$  on  $X$  so that  $S(A, B)$  is true if and only if  $A \subseteq B$ . Prove that  $S$  is a partial order.
- (b) Let  $Y$  be the set of binary strings of length  $n$ , where  $n$  is a natural. Define the binary relation  $T$  on  $Y$  so that  $T(u, v)$  is true if and only if for every  $i$  with  $0 \leq i < n$ , if the  $i$ 'th character of  $u$  is 1, then the  $i$ 'th character of  $v$  is 1. Prove that  $T$  is a partial order.
- (c) Can you describe the relationship between the partial orders in parts (a) and (b)?

E2.10.9 Recall that a **minimal** element  $m$  in a partial order  $P$  is one that satisfies the statement  $\forall x : P(x, m) \rightarrow (x = m)$ , while a **minimum** element  $m$  satisfies  $\forall x : P(m, x)$ .

- (a) Give an example of a partial order with a minimal element that is not minimum.
- (b) Prove that a minimum element must also be minimal.
- (c) We proved a Lemma that every partial order  $P$  on a finite set has a minimal element. Explain how the proof shows that for every element  $x$  of a finite partial order, there exists a minimal element  $m$  such that  $P(m, x)$  is true.

- (d) Use part (c) to argue that if a finite partial order has only one minimal element, it is also a minimum element.

E2.10.10 Let  $R$  be a binary relation on a finite set  $X$  that is reflexive and transitive. (Such a relation is called a **preorder**.) Define a new relation  $S$  such that  $S(x, y) \leftrightarrow ((x = y) \vee (R(x, y) \wedge \neg R(y, x)))$ . Prove that  $S$  is a partial order.

#### 2.10.4 Problems

P2.10.1 Explain why the “path-below” relation  $\{(x, y) : \text{There is an upward path from } x \text{ to } y\}$  of any Hasse diagram is a partial order.

P2.10.2 Prove that it is not possible to have a **cycle** in a partial order. A cycle is a set of more than one element, say  $\{a_1, a_2, \dots, a_k\}$ , each of them distinct, such that  $P(a_i, a_{i+1})$  is true for all  $i < k$  and also  $P(a_k, a_1)$ . (Hint: Argue by contradiction, using the transitive property to get a violation of the anti-symmetric property.)

P2.10.3 Prove that if we remove a minimal element from a partial order, the resulting relation is still reflexive, anti-symmetric, and transitive.

P2.10.4 In general, a string  $u$  is defined to be a **substring** of a string  $v$  if  $v = xuy$  for some strings  $x$  and  $y$ , either or both possibly empty.

- Prove that the substring relation on any set of strings is a partial order.
- Draw the Hasse diagram for the substring relation on the strings of two or fewer letters over the alphabet  $\{a, b, c\}$ .
- Draw the Hasse diagram for the same relation on the substrings of the string *abba*. Repeat for the string *cababa*.

P2.10.5 Suppose that  $n$  teams meet in a **single-elimination tournament**, where every game has a winner and a loser, the loser of each game is eliminated and never plays again, and the tournament continues until all but one team is eliminated.

- Assume that no two games end at exactly the same time. Prove that the relation  $B(x, y)$ , defined as “team  $y$  survives as least as long as team  $x$ ”, is a total order on the teams. Note that if  $w$  is the winner of the tournament,  $B(x, w)$  is true for any team  $x$ , because  $w$  was never eliminated.
- Say that team  $x$  is *shown to be no better than* team  $y$ , written  $S(x, y)$ , if either  $x = y$ ,  $y$  wins a game against  $x$ , or  $y$  wins a game against some other team  $z$  where  $x$  has been shown to be no better than  $z$ . (This is a recursive definition.) Explain why  $S$  is a partial order, and why it might not be a total order.
- In the last four rounds of the annual NCAA basketball tournament, the Sweet Sixteen teams are successively reduced to the Elite Eight, the Final Four, the set of two finalists, and the champion. Draw a Hasse diagram for the relation  $S$  arising from this tournament. (You cannot label the dots without knowing who wins the games, but this Hasse diagram always has the same shape).

- (d) A **ladder tournament** is a particular kind of single-elimination tournament where the teams are numbered from 1 through  $n$ . The first game is team  $n - 1$  against team  $n$ , the winner of that game plays team  $n - 2$ , the winner of that game plays team  $n - 3$ , and so forth until the champion is decided by the game involving team 1. What are all the possible shapes for the Hasse diagram of the  $S$  relation for such a tournament?
- (e) Is it possible for the  $B$  and  $S$  relations in a single-elimination tournament to be the same? If so, exactly how?

P2.10.6 There is only one partial order possible on the set  $\{a\}$ , because  $R(a, a)$  must be true. On the set  $\{a, b\}$ , there are three possible partial orders, as  $R(a, a)$  and  $R(b, b)$  must both be true and either zero or one of  $R(a, b)$  and  $R(b, a)$  can be true. List all the possible partial orders on the set  $\{a, b, c\}$ . (Hint: There are nineteen of them.) How many are linear orders?

P2.10.7 (uses Java) Write a real-Java static method `int minElement (boolean [ ] [ ] r)` that treats the input array as a binary relation  $R$  on  $\{0, 1, \dots, n\}$  and, if  $R$  is a partial order, returns a minimal element of it. Do not worry what happens if  $R$  is not a partial order. (Hint: Brute-force search is easier than following the proof of the Lemma in the Hasse Diagram Theorem.)

P2.10.8 Suppose that  $P$  is a partial order on some set  $X$  and that  $P$  satisfies the quantified statement  $\forall x : \exists y : \forall z : (P(x, z) \wedge (z \neq x)) \rightarrow P(y, z)$ .

- Give an example of a partial order on the set  $\{a, b, c\}$  that satisfies this statement, and of another partial order that does not.
- How would you tell, given a Hasse diagram for a partial order, whether it satisfies this statement?
- Argue informally that if  $x$  is any element of a finite partial order satisfying this statement, there is a *unique* maximal element  $m$  such that  $P(x, m)$  is true.

P2.10.9 Let  $f$  and  $g$  be two functions from  $\mathbf{R}$  to  $\mathbf{R}$ . We say that  $f$  **dominates**  $g$  if there is a number  $c$  such that for all  $x$  with  $x > c$ ,  $f(x) \geq g(x)$ . We define a relation  $D$  on functions so that  $D(f, g)$  means “ $f$  dominates  $g$ ”.

- Prove that  $D$  is a linear order on polynomials.
- Prove that  $D$  is a partial order on functions.
- Prove that  $D$  is *not* a linear order on functions. (Hint: Give an example of two functions, neither of which dominates the other.)

P2.10.10 Let  $P$  be a partial order on a finite set  $X$ . Prove that there exists a linear order  $L$  on  $X$  such that  $P \subseteq L$ . (Hint: Use the proof of the Hasse Diagram Theorem.)

## 2.11 Equivalence Relations

### 2.11.1 Definition and Examples

In Section 2.8 we defined an **equivalence relation** to be a binary relation on a set  $A$  (that is, a subset of the direct product  $A \times A$ ) that has the following three properties, expressed in quantified statements:

- **Reflexivity:**  $\forall x : R(x, x)$
- **Symmetry:**  $\forall x : \forall y : R(x, y) \rightarrow R(y, x)$
- **Transitivity:** For any elements  $x, y$ , and  $z$ , if both  $R(x, y)$  and  $R(y, z)$  are true, so is  $R(x, z)$ .

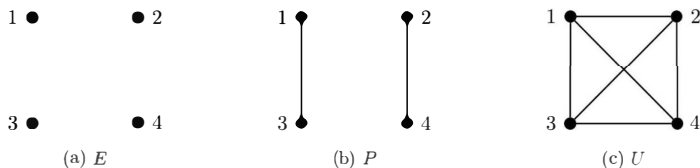
These are the three most important properties of the **equality relation**, defined so that  $E(x, y)$  is true if and only if  $x = y$ . So the equality relation is our first and most important example of an equivalence relation. Here are two more examples. Define the **parity relation**  $P(x, y)$  on naturals by the predicate “ $x$  and  $y$  are both even or both odd”. Define the **universal relation**  $U(x, y)$  on any set to always be true. We can easily check the three properties of each of these relations:

- **Reflexivity:**  $P(x, x)$  is true whether  $x$  is even or odd, and  $U(x, x)$  is certainly true.
- **Symmetry:**  $P(x, y)$  and  $P(y, x)$  are exactly the same statement, and  $U(x, y)$  and  $U(y, x)$  are both true.
- **Transitivity:** If  $P(x, y)$  and  $P(y, z)$  are both true, then  $x$  and  $z$  are both even if  $y$  is even and are both odd if  $y$  is odd. And of course  $U(x, z)$  is always true, so the desired implication is true.

It's easy to make up more examples of equivalence relations, such as “person  $a$  and person  $b$  live in the same state”. You may have noticed that all our examples of equivalence relations have something in common — in some sense they say “ $x$  and  $y$  are the same” (they are equal, they have the same remainder when divided by 2, they are in the same universe, they live in the same state). Given the English meaning of “same”, a relation of this form *will* normally be an equivalence relation, because any thing is “the same as” itself, if  $x$  is “the same as”  $y$  then  $y$  is “the same as”  $x$ , and transitivity also holds. With a definition and a little more work, we can prove the converse of this fact — that every equivalence relation can be restated in this form.

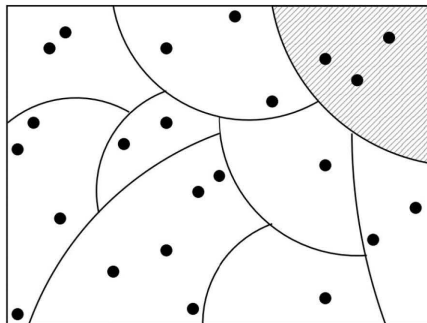
First, though, let's look at how we can describe an equivalence relation by a diagram. We'll make a dot for every element, and a line from  $x$  to  $y$  whenever  $R(x, y)$  is true. (Since the relation is symmetric, we don't need to indicate any direction on the line.) Figure 2-10 shows such diagrams for the relations  $E$ ,  $P$ , and  $U$  on the set  $\{1, 2, 3, 4\}$ .

In each case the equivalence relation divides the elements into groups, where two elements in the same group are related to each other and elements in different groups are not. There are four



Source: David Mix Barrington

Figure 2-10: Diagrams of three relations,  $E$ ,  $P$ , and  $U$ , on the set  $\{1, 2, 3, 4\}$ .



Source: David Mix Barrington

Figure 2-11: A partition of a finite set. The shaded area is one element of  $P$ .

groups with one element each for  $E$ , two groups of two elements each for  $P$ , and one group of four elements for  $U$ . We'll now show that every equivalence relation gives rise to groups of elements in this way.

## 2.11.2 The Partition Theorem

**Definition:** Let  $A$  be any set. A **partition** of  $A$  is a set  $P$ , where

- Each element of  $P$  is a subset of  $A$ ,
- The elements of  $P$  are **pairwise disjoint**, that is, if  $X$  and  $Y$  are different elements of  $P$  then  $X \cap Y = \emptyset$ , and
- The union of all the elements of  $P$  is exactly  $A$ .

Figure 2-11 shows a partition of a finite set.

The last two properties are equivalent to the statement that each element of  $A$  is contained in



*exactly one* element of  $P$  (this will be proved in Problem 2.11.1). We can now state and prove a theorem which says that equivalence relations are intimately related to partitions:

**Partition Theorem:** Let  $A$  be a set. A relation  $R \subseteq A \times A$  is an equivalence relation if and only if it is the relation “ $x$  and  $y$  are in the same set of  $P$ ” for some partition  $P$  of  $A$ .

**Proof:** The easy half of the proof is to show that given any partition  $P$  on a set  $A$ , the relation “ $x$  and  $y$  are in the same set in  $P$ ” is an equivalence relation on  $A$ . We must show that this relation (which we will abbreviate  $S(x, y)$ ) is reflexive, symmetric, and transitive. For reflexivity, let  $x$  be any element of  $A$ . Since  $x$  must be in some set in  $P$ ,  $x$  and  $x$  are in the same set in  $P$  and so  $S(x, x)$  is true. For symmetry, assume that  $S(x, y)$ , and let  $B$  be the set in  $P$  that contains  $x$  and  $y$ . Now  $y$  and  $x$  are both in  $B$ , so they are both in the same set in  $P$ , and  $S(y, x)$  is true. Finally, for transitivity, assume that  $S(x, y)$  and  $S(y, z)$  are both true. Let  $B$  again be the set containing  $x$  and  $y$ . Since  $y$  and  $z$  are in the same set in  $P$  by assumption,  $z$  is in  $B$ . Since  $x$  is also in  $B$ ,  $S(x, z)$  is true as required. We have proven that  $S$  is an equivalence relation<sup>19</sup>.

The remaining half of our problem is to take an equivalence relation  $R$  on  $A$  and construct a partition  $P$  of  $A$  such that  $R(x, y)$  will be true if and only if  $x$  and  $y$  are in the same set in  $P$ . We do this by taking  $P$  to be the set of **equivalence classes** of  $R$ . For any element  $x$  of  $A$ , we define the equivalence class of  $x$  to be the set  $\{w : R(x, w)\}$ , the set of elements that are “ $R$ -equivalent to  $x$ ”. As we take all the possible elements  $x$ , we get a collection of different sets as equivalence classes, and this collection will be exactly the partition  $P$ .

We have two things to show. First, we need to prove that  $R(x, y)$  is exactly the relation “ $x$  and  $y$  are in the same set in  $P$ ”, that is, that  $R(x, y)$  if and only if  $x$  and  $y$  are in the same set in  $P$ . Since this is an “if and only if” statement, we can break it into two parts. First assume  $R(x, y)$ . By the definition,  $y$  is in the equivalence class of  $x$ . We also know that  $x$  is in the equivalence class of  $x$ , because  $R(x, x)$  is true ( $R$  is reflexive). For the other direction of the “if and only if”, assume that  $x$  and  $y$  are in the same set in  $P$ , which must be the equivalence class of some element  $z$ . The definition tells us that  $R(z, x)$  and  $R(z, y)$  are true, from which we can conclude  $R(x, z)$  by symmetry and  $R(x, y)$ , our desired conclusion, by transitivity. Note how all three properties of an equivalence relation were used in the proof.

All that is left is to show that  $P$  is a partition. Clearly it is a set of subsets of  $A$ . The union of the sets in  $P$  must be  $A$  because every element is in some equivalence class — its own, at least. Is it possible for an element of  $A$  to be in two different equivalence classes? No, as we can show by deriving a contradiction. Suppose that  $x$  were in the equivalence class of  $y$  and also in the equivalence class of  $z$ . The definitions give us that  $R(y, x)$  and  $R(z, x)$  are true. From this we can derive  $R(y, z)$  as above, using symmetry to get  $R(x, z)$  and then using transitivity. Now we can show that the class of  $y$  and the class of  $z$  are *exactly the same set*, so that they are not two *different* sets<sup>20</sup> in  $P$ . By the definition of set equality, we must show that for any element  $w$ ,  $R(y, w) \leftrightarrow R(z, w)$ . Given  $R(y, w)$ , we use symmetry to get  $R(w, y)$ , transitivity with  $R(y, z)$  to get  $R(w, z)$ , and symmetry again to get  $R(z, w)$ . In exactly the same way we can derive  $R(y, w)$

<sup>19</sup>This is a rather boring proof, but it's worth tracing out all the details carefully at least once to make sure that we're clear on the definition. Later we can say things like “it's easy to check that  $S$  is reflexive, symmetric, and transitive” and then not do it explicitly.

<sup>20</sup>We constructed  $P$  by taking the *set* of all equivalence classes, so that if two elements produced the same equivalence class it results in only one element of  $P$ .

from  $R(z, w)$ , completing the proof that the two sets are equal. Any two sets in  $P$  which share an element are thus identical, proving that these sets are pairwise disjoint and that  $P$  is therefore a partition. ♣

Let's see some examples of the equivalence classes of various equivalence relations. For the equality relation  $E$  (defined so that  $E(x, y) \leftrightarrow x = y$ ), the classes are the **singleton sets**  $\{x\}$  for each element  $x$  of  $A$ . For the parity relation  $P$  there are two equivalence classes: the set of all odd naturals and the set of all even naturals. For the universal relation  $U$  there is only one equivalence class, consisting of all the elements of the type. Finally, for a relation such as “ $x$  and  $y$  live in the same U.S. state” we would have fifty equivalence classes, one for each state, such as  $\{x : x \text{ lives in Delaware}\}$ .

### 2.11.3 Exercises

E2.11.1 Which of the following relations are equivalence relations? For those that are, describe the equivalence classes. For those that are not, say which of the three properties fail to hold.

- (a)  $A(u, v)$ , defined as “string  $u$  and string  $v$  have the same length”
- (b)  $B(x, y)$  on integers, defined by the inequality “ $|x - y| \leq 10$ ”
- (c)  $C(x, y)$  on novelists, defined as “ $x$  and  $y$  wrote a novel in the same language”
- (d)  $D(x, y)$  on real numbers, defined as “ $\neg \exists z : z$  is an integer and is strictly between  $x$  and  $y$ ”

E2.11.2 Prove that  $R$  is symmetric if and only if  $\forall x : \forall y : R(x, y) \leftrightarrow R(y, x)$ . This new quantified statement can thus serve as an alternate definition of “symmetric”.

E2.11.3 Find the partition and draw the diagram for the following equivalence relation on the set  $\{a, b, c, d, e, f\}$ :

$$\{\langle a, a \rangle, \langle a, c \rangle, \langle a, f \rangle, \langle b, b \rangle, \langle b, e \rangle, \langle c, a \rangle, \langle c, c \rangle, \langle c, f \rangle, \langle d, d \rangle, \langle e, b \rangle, \langle e, e \rangle, \langle f, a \rangle, \langle f, c \rangle, \langle f, f \rangle\}.$$

E2.11.4 Consider the partition  $\{\{1, 4, 5\}, \{2, 6, 7\}, \{3, 8\}\}$  of the set  $\{1, 2, 3, 4, 5, 6, 7, 8\}$ . List the elements of the corresponding equivalence relation and draw a diagram of it.

E2.11.5 Let  $R$  and  $S$  be two equivalence relations on the same set  $A$ . Define a new relation  $T$  such that  $T(x, y)$  if and only if both  $R(x, y)$  and  $S(x, y)$  are true. Prove that  $T$  is an equivalence relation.

E2.11.6 Show that the following relations on strings are equivalence relations, and describe the equivalence classes of each relation on the set  $\{a, b, c\}^3$ .

- (a)  $R(u, v)$  meaning “every letter that occurs in  $u$  also occurs in  $v$ , and vice versa”.
- (b)  $A(u, v)$  meaning “ $u$  and  $v$  are anagrams” (see Problem 1.2.9).

E2.11.7 Let  $D$  be a set of dogs, containing a subset  $B$  of black dogs, a subset  $F$  of female dogs, and a subset  $R$  of retrievers. Define a relation  $E$  on dogs so that  $E(x, y)$  means “( $B(x) \leftrightarrow B(y) \wedge (F(x) \leftrightarrow F(y)) \wedge (R(x) \leftrightarrow R(y))$ )”.

- (a) Prove that  $E$  is an equivalence relation.

- (b) Describe each of the possible equivalence classes of  $E$  in English. (Note that depending on  $D$ , some of the possible classes might be empty.)

E2.11.8 Let  $\{x_1, \dots, x_n\}$  be a set of atomic variables, and let  $C$  be the set of all compound propositions using those variables. Define the relation  $E$  so that if  $c$  and  $d$  are compound propositions,  $E(c, d)$  means “ $c \leftrightarrow d$ ”.

- (a) Prove that  $E$  is an equivalence relation.  
 (b) Describe the equivalence classes of  $E$ . How many are there?

E2.11.9 Let  $X$  be a finite set and let  $P$  and  $Q$  be two partitions of  $X$ . We say that  $P$  and  $Q$  are **isomorphic** if there is a bijection  $f$  from  $X$  to  $X$  such that for any set  $S$  in  $P$ , the set  $f(S) = \{f(a) : a \in S\}$  is in  $Q$ . Let  $I$  be the binary relation on partitions of  $X$  such that  $I(P, Q)$  means “ $P$  and  $Q$  are isomorphic”. Prove that  $I$  is an equivalence relation.

E2.11.10 We can represent a rational number by a pair of integers, the **numerator** and **denominator** of a fraction, as long as the denominator is not zero. It is possible for two pairs to represent the same rational number. Define the binary relation  $E$  on pairs of integers (with the second integer nonzero) so that  $E(\langle p, q \rangle, \langle r, s \rangle)$  means that  $p/q$  and  $r/s$  are the same rational number.

- (a) Prove that  $E$  is an equivalence relation.  
 (b) Prove that there is a bijection between the rational numbers and the equivalence classes of  $E$ .  
 (c) (uses Java) Write a static method that takes four `int` arguments  $p$ ,  $q$ ,  $r$ , and  $s$  and determines whether  $p/q$  and  $r/s$  are the same rational number. Do not use the Java division operator in your code<sup>21</sup>.

## 2.11.4 Problems

P2.11.1 Let  $P$  be a set, each element of which is a subset of some fixed set  $A$ . Prove that the elements of  $P$  are pairwise disjoint and union together to give  $A$  if and only if every element of  $A$  is an element of exactly one element of  $P$ .

P2.11.2 Let  $f$  be any function from  $A$  to  $B$ . Let  $R$  be the binary relation on  $A$  defined so that  $R(x, y)$  is true if and only if  $f(x) = f(y)$ .

- (a) Prove that  $R$  is an equivalence relation.  
 (b) What do we know about the equivalence classes of  $R$  if  $f$  is one-to-one?  
 (c) What do we know about the equivalence classes of  $R$  if  $f$  is onto?  
 (d) Prove that the equivalence classes of  $R$  are in one-to-one correspondence with the range of  $f$  (that is, with the set  $f(A) = \{y : \exists x : f(x) = y\}$ ).

P2.11.3 Let  $R$  and  $S$  be two equivalence relations on the same set  $A$ . Define a new relation  $U$  such that  $U(x, y) \leftrightarrow [R(x, y) \vee S(x, y)]$ . Is  $U$  necessarily an equivalence relation? Either prove that it is or give an example where it is not.

<sup>21</sup>Because division of floating point numbers introduces rounding errors, it is possible that “`p/q == r/s`” evaluates to `false` when in fact  $p/q = r/s$  is true.

P2.11.4 There is only one possible partition of the set  $\{a\}$ . There are two possible partitions of the set  $\{a, b\}$ , corresponding to our equivalence relations  $E$  and  $U$ . There are *five* possible partitions of the set  $\{a, b, c\}$  – list them. How many partitions are there of  $\{a, b, c, d\}$ ? Of  $\{a, b, c, d, e\}$ ?

P2.11.5 Suppose we have a set of airports and a symmetric set of direct flight connections. (That is, if you can fly directly from airport  $x$  to airport  $y$  you can also fly directly from  $y$  to  $x$  – we will call this predicate  $D(x, y)$ .) The relation  $D$  may fail to be an equivalence relation because it need not be transitive. But for any such  $D$ , it turns out that there exists an equivalence relation  $F$  such that:

- $D(x, y) \rightarrow F(x, y)$
- If  $D(x, y) \rightarrow G(x, y)$  and  $G$  is an equivalence relation, then  $F(x, y) \rightarrow G(x, y)$

Find this  $F$ , describe it in terms of  $D$ , and prove both that it is an equivalence relation and that it has the specified properties.

P2.11.6 The equivalence relation of isomorphism (see Exercise 2.11.9) divides the partitions of a given set into equivalence classes.

- (a) Divide the five partitions of  $\{a, b, c\}$  into equivalence classes.
- (b) Divide the partitions of  $\{a, b, c, d\}$  into equivalence classes.
- (c) Describe the equivalence classes of partitions of  $\{a, b, c, d, e\}$  – you need not list their members.

P2.11.7 Given any partial order  $P$ , we can form its **symmetric closure**  $P^S$  by taking the union of  $P$  and  $P^{-1}$ .

- (a) Explain why  $P^S$  is reflexive and symmetric.
- (b) Given an example of a partial order  $P$  such that  $P^S$  is not an equivalence relation.
- (c) Prove that if  $P$  has the property from Problem 2.10.8, then  $P^S$  is an equivalence relation.

P2.11.8 Let  $X$  be a finite set and let  $Y$  be a collection of nonempty subsets of  $X$  that satisfy the following property: If  $S$  and  $T$  are two different sets in  $Y$ , then either  $S \subseteq T$ ,  $T \subseteq S$ , or  $S \cap T = \emptyset$ .

- (a) Give an example of such a system where  $X$  has four elements and  $Y$  has seven sets.
- (b) A set in  $Y$  is called “atomic” if it has no proper subset in  $Y$ . Must the atomic sets of  $Y$  form a partition of  $X$ ? Prove your answer.
- (c) Define a relation  $E$  on  $X$  such that  $E(a, b)$  means “for every set  $S$  in  $Y$ ,  $(a \in S) \leftrightarrow (b \in S)$ ”. Prove that  $E$  is an equivalence relation.
- (d) Prove that  $E(a, b)$  is true if and only if either  $a$  and  $b$  are in the same atomic set, or neither  $a$  nor  $b$  is in any atomic set.

P2.11.9 (uses Java) We have been representing a binary relation  $R$  on the set  $\{0, 1, \dots, n-1\}$  by a two-dimensional boolean array `r`. If  $R$  is an equivalence relation, we can also represent it by a one-dimensional array that tells which equivalence class contains which element of the base set. We will name each equivalence class by the smallest number it contains, and have a one-dimensional `int` array `c` so that `c[i]` is the name of  $i$ ’s class, that is, the smallest number  $j$  in the set such that  $R(i, j)$  is true.

- (a) Write a real-Java static method `int [ ] class (boolean [ ] [ ] r)` that returns the one-dimensional array listing the classes of each element under  $R$ .
- (b) Write a real-Java static method `boolean [ ] [ ] relation (int [ ] c)` that returns the two-dimensional array representing the equivalence relation whose classes are given by the input array.

P2.11.10 (uses Java) Suppose we have an equivalence relation  $R$  on the set  $\{0, 1, \dots, n-1\}$  represented by a one-dimensional `int` array `c`, giving the equivalence class of each element as in Problem 2.11.9. Now suppose  $R$  is to be changed to a new equivalence relation  $R'$ , by adding a new pair  $\langle i, j \rangle$  and any other pairs that are needed to preserve symmetry and transitivity.

- (a) Write a real-Java static method `void update (int [ ] c, int i, int j)` that alters `c` so that it now represents this new equivalence relation  $R'$ , including the pair  $\langle i, j \rangle$ .
- (b) Write a real-Java static method `void update (boolean [ ] [ ] r, int i, int j)` that alters `r` so that it now represents  $R'$ . You may find it convenient to use the methods of Problem 2.11.9.

## Index

- antireflexive relation 2-36
- antisymmetric relation 2-36
- associativity of language concatenation 2-23
  
- binary relation 2-2
- binding a variable 2-10
- bijection 2-42
- binary relation 2-2
- binary relation on a set 2-39
- bound variable 2-10
  
- combinatorics 2-42
- composition of functions 2-43
- concatenation of languages 2-19
- concatenation of strings 2-19
- cycle 2-50, 2-53
  
- database 2-8
- denominator 2-59
- dense set of numbers 2-15
- direct product 2-2, 2-3
- division relation 2-48
- dividing a natural 2-48
- domain 2-4
- dominating a function 2-54
- dual properties 2-41
  
- equality relation 2-36
- equivalence classes 2-57
- equivalence relation 2-37, 2-55
- even number 2-14
- existential quantifier 2-10
  
- fixing an element 2-47
- free variable 2-10
- fully comparable relation 2-45
- function 2-4, 2-34
  
- graph of a function 2-4
- greedy algorithm 2-7
  
- Hasse diagram 2-49
- Hasse Diagram Theorem 2-50
  
- identity element 2-20
- identity function on a set 2-42
- injection 2-42
- inverse of a function 2-43
- inverse relation 2-32
- isomorphic partial orders 2-59
- iterate of a function 2-46
  
- join of relations 2-8, 2-13
  
- Kleene star operation 2-20
- $k$ -tuple 2-2
  
- ladder tournament 2-54
- Least Number Axiom 2-14
- linear order 2-48
  
- majority quantifier 2-15
- membership predicate 2-2
- matching 2-42
- maximal element of a partial order 2-50
- maximum element of a partial order 2-50
- minimal element of a partial order 2-50
- minimum element of a partial order 2-50
  
- numerator 2-59
  
- odd number 2-14
- one-to-one correspondence 2-42
- one-to-one relation 2-41
- onto relation 2-341
- order relation 2-56
- ordered set 2-48
  
- pair 2-2
- pairwise disjoint sets 2-56
- palindrome 2-32
- pangram 2-22
- parity relation 2-55
- partial order 2-37, 2-48
- partition of a set 2-56
- Partition Theorem 2-57
- path-below relation 2-49
- predecessor 2-14
- predicate calculus 2-10
- prefix code 2-23
- preorder 2-53
- projection 2-6, 2-13
  
- quantified statement 2-10

quantifiers 2-10

range 2-4

record 2-8

reflexive relation 2-36

relation 2-2

relational database 2-8

Rule of Existence 2-26

Rule of Generalization 2-27

Rule of Instantiation 2-26

Rule of Specification 2-26

scope of a quantifier 2-11

sequence 2-2

signature of a predicate 2-3

single-elimination tournament 2-57

singleton set 2-58

Size-Function Theorem 2-43

star operation 2-20

substring relation 2-49, 2-53

surjection 2-41

symmetric closure 2-60

symmetric relation 2-36

total binary relation 2-35

total order 2-48

transitive relation 2-37

triple 2-2

tuple 2-2

unique existential quantifier 2-14

universal quantifier 2-10

universal relation 2-53

universe of discourse 2-11

well-defined binary relation 2-35





## Chapter 3: Number Theory

*“... I didn’t expect a kind of Spanish Inquisition.”*

*“Nobody expects the Spanish Inquisition. Our chief weapon is surprise ... surprise and fear ... fear and surprise ... our two weapons are fear and surprise ... and ruthless efficiency. Our three weapons are fear and surprise and ruthless efficiency and an almost fanatical devotion to the Pope ... Our four ... no ... amongst our weaponry are such elements as fear, surprise ... I’ll come in again.”*

The *naturals* (a term that refers in this book to the set  $\{0, 1, 2, 3, \dots\}$  of *non-negative* integers) are one of the simplest and most important systems of mathematical objects. With the logical tools we’ve developed, we can state and prove propositions about the naturals ranging from the trivial through the important and ending with the unsolvable. *Number theory* is the study of the naturals and these statements, and is a major branch of mathematics with significant applications in computer science.

- We’ll begin with the basic definitions of number theory, such as *divisibility*, *prime numbers*, and *modular arithmetic*.
- We’ll give proofs from first principles of some of the most important facts about primes and modular arithmetic, culminating with the *Fundamental Theorem of Arithmetic*, which says that every positive natural has a *unique* factorization into primes.
- Finally, in some optional sections, we’ll have a look at some of the properties of exponentiation in modular arithmetic, finishing with the mathematics behind the *RSA public-key cryptosystem*.

## 3.1 Divisibility and Primes

### 3.1.1 The Natural Numbers

Mathematics can be defined as the practice of making formal systems out of common human activities<sup>1</sup>, and *counting* is perhaps the most fundamental of these. Humans count things, which led to the invention of the **naturals**, which are the possible results of a counting process — zero, one, two, three, and all the rest of the *non-negative integers*.

Computers count things as well, such as the characters in a file, the pixels on a screen, or the pennies in a bank account. At some level every piece of data in a computer is really a sequence of booleans, of course, but it is often useful to organize these bits into things which act like naturals or integers. They aren't really naturals or integers because they are *limited in size*, typically to the size that will fit in one machine word. In most computer languages, the programmer must learn the difference between numbers of this kind and *real numbers*, stored as approximations in floating-point notation. They are separate data types, each with its own operations and properties.

**Number Theory** is the branch of mathematics that deals with naturals, and in particular with the statements about naturals that one can make with the predicate calculus. We will have formulas, *statements of number theory*, where the variables will each be of type **natural** and the atomic statements will be things like “ $a = b$ ”, “ $a > b$ ”, “ $a + b = c$ ”, and “ $a \cdot b = c$ ”. With quantifiers, we can say fairly complicated things, such as “ $\exists a : \exists b : \forall c : (a \cdot c = a) \wedge (b \cdot c = c)$ ”, a statement that two naturals exist with certain properties. (Is this statement true? If so, what are the two naturals?)

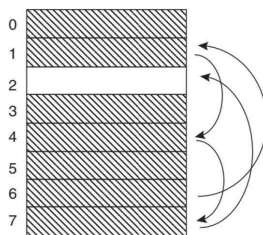
Just as in Chapter 2, we can use the predicate calculus to define properties in terms of other properties. Perhaps our most important property is the “divides” relation from Section 2.10, which is defined so that “ $a$  divides  $b$ ” if and only if  $\exists c : a \cdot c = b$ . Once this property is defined, it can be used in more number theory statements. For example, we will shortly define a *prime number* in terms of this relation. In this book we will be a bit informal about the exact rules of our language for formulas, though it's possible to fix such rules exactly and write some of the atomic formulas in terms of others<sup>2</sup>.

Why study number theory? There are practical uses for it in computer science, of course, in any situation that involves counting. You may be familiar with a *hash table*, which is a sequence of  $n$  data locations in which items are to be inserted as they arrive. You compute (using number theory, as it happens) an address for the item and see whether that address is vacant. Under one scheme called *open addressing*, if it isn't you try again at the address  $k$  locations later, where  $k$  is some fixed natural. If this brings you past the last location, you “wrap around” to the beginning. If you still get a filled location, you jump  $k$  locations again, and keep doing until there's a vacant one. One feature of this scheme is that *if  $k$  and  $n$  are chosen correctly*, if there is any open location at

---

<sup>1</sup>For a beautiful exposition of what mathematics is and what it is for, read *Mathematics: Form and Function* by Saunders Mac Lane.

<sup>2</sup>Hofstadter defines his “Typographical Number Theory” carefully so that everything is defined in terms of a symbol for zero, the equals sign, and functions for successor, addition, and multiplication. He goes to some trouble to get a particular exact number of symbols in his alphabet, for non-mathematical reasons.



©Kendall Hunt Publishing Company

Figure 3-1: A hash table. Arrows show the jumps in looking for an empty space.

all you'll eventually find it. (Figure 3-1 shows an example with  $k = 3$  and  $n = 8$ .) Determining which values of  $k$  and  $n$  have this property is a good example of a number-theoretic question.

More philosophically, studying the most fundamental mathematical system is valuable for what it tells us about how mathematics works and how the world works. There are questions in number theory that are very easy to state but that no one currently can answer<sup>3</sup>. But if you believe that the naturals exist in a philosophical sense, any statement of number theory must be true or false. Either a natural with some property exists, or it doesn't. Either all naturals have a certain property, or they don't. Just as with the propositional calculus, you'd like to find a proof system that is complete (can prove all the true statements of number theory) and consistent (doesn't prove any false statements). Although the best mathematicians in the world were seriously looking for such a thing early in this century, we now know it doesn't exist.

Gödel proved in 1931 that if you set down the rules for any mechanical proof system  $S$  that can talk about number theory, then there must be a statement of number theory that says, in effect, "I am not provable in system  $S$ ". If this statement were false, it would be provable, and  $S$  would thus be able to prove a false statement and be inconsistent. On the other hand, if it is true, then it must not be provable as it says, and  $S$  is incomplete. Showing that such a statement exists is well beyond the scope of this book — to begin with you have to define a particular proof system for number theory in far more detail than we will use here — but Hofstadter does this in *Gödel, Escher, Bach*<sup>4</sup>.

<sup>3</sup>The usual example of such a question used to be *Fermat's Last Theorem*, which is still easy to state (see the discussion in Hofstadter) but is no longer unsolved thanks to Andrew Wiles. The *Goldbach Conjecture* ("Every even number greater than two is the sum of two prime numbers") is perhaps even easier to state and remains unresolved.

<sup>4</sup>The philosophical implications of Gödel's theorem are one of Hofstadter's main topics. For example, does the fact that some mathematical questions are provably unsolvable say anything about whether true artificial intelligence can be achieved?

### 3.1.2 Primes and Primality Testing

Our goal in this chapter will be to learn some of the more interesting things that can be said within number theory, and to learn some techniques (mostly informal) to prove some of them<sup>5</sup>. We begin by giving a few of the most basic definitions. We've already seen the **division relation**  $D(a, b)$  on naturals, which in Java terms is true if and only if `b%a == 0`. Some naturals, like 60, have lots of other numbers that divide them, and some, like 59, have very few. In fact the only two naturals that divide 59 are 1 (which divides every natural) and 59 (as any natural divides itself). Naturals that have this property are called **prime numbers**<sup>6</sup>.

**Definition:** The predicate “ $P(a)$ ”, read in English as “ $a$  is **prime**”, is defined by the rule:

$$P(a) \Leftrightarrow (a > 1) \wedge (D(b, a) \rightarrow [(b = 1) \vee (b = a)]),$$

or equivalently, by expanding the definition of “divides”,

$$P(a) \Leftrightarrow (a > 1) \wedge \neg \exists b : \exists c : [(b > 1) \wedge (c > 1) \wedge (b \cdot c = a)].$$

**Definition:** A natural  $x$  (with  $x > 1$ ) is said to be a **composite number** if it is not prime. (Note that we have thus defined 1 to be neither prime nor composite). A composite number can be **factored**, or written as the product of two smaller naturals (each greater than one).

Given a natural  $a$ , how do we determine whether it is prime? Proving it to be prime would mean proving the *non-existence* of factors, which would mean ruling out all the possible naturals  $d$  between 1 and  $a$  as possible divisors of  $a$ . This **trial division algorithm** is the simplest test for primality.

We can take a shortcut that will improve the running time of trial division. If  $a = dc$  for some  $d$  and  $c$ , at least one of  $d$  or  $c$  must be at most the square root of  $a$  (Why? See Exercise 3.1.8). So if we check that all  $b$  between 1 and  $\sqrt{a}$  fail to divide  $a$ ,  $a$  must be prime.

Let's look at this test in pseudo-Java code:

```
public boolean isPrime (natural a) {
    if (a <= 1) return true;
    natural d = 2;
    while (d*d <= a) {
        if (a % d == 0) return false;
        d++;
    }
    return true;}

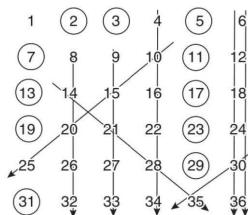
```

In fact, we could also potentially save more time by checking only those  $b$  that are prime themselves (Why?). But how would we know whether  $b$  is prime without doing the same procedure over again?

---

<sup>5</sup>The single most important *formal* proof technique, mathematical induction, will be the main subject of Chapter 4.

<sup>6</sup>The number 1 is a special case. Even though it is divisible only by itself and by 1, we are going to declare that it *isn't* prime and give our reasons later.



©Kendall Hunt Publishing Company

Figure 3-2: The Sieve of Eratosthenes.

The **Sieve of Eratosthenes** is an algorithm to construct a list of *all* the primes up to some natural  $n$ , using essentially this same method. We begin by listing all the naturals from 2 up to  $n$  (in Figure 3-2,  $n = 36$ ). (It turns out to be convenient to put them in rows of six in this way.) We then identify 2 as a prime number, and cross out all the **multiples** of 2, that is, all the even numbers. (With the rows of six, we can do this by wiping out the second, fourth, and last columns.) Then we identify 3 as prime and cross out all the multiples of 3 (the third column and the last one, though the last one is already crossed out). Then we look for the next number that isn't crossed out, 5, identify *that* as prime, and cross out all of its multiples. (These form a nice diagonal line.) We continue in the same way — in each phase we find the next number not crossed out, identify it as prime, and cross out its multiples. We can stop when we've checked all the prime numbers up through  $\sqrt{n}$ , because anything up to  $n$  that is not divisible by any of those primes must be prime itself. (In the example we can stop with 5, because the next prime is 7 and  $7^2 > 36$ .) (Would this be a practical way to tell whether some particular 20-digit number is prime?)

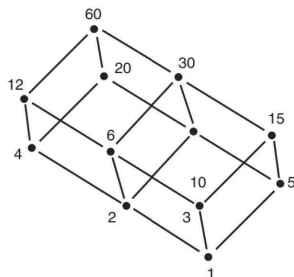
### 3.1.3 Prime Factorization

If the factors of a composite number are themselves composite, they too can be factored into yet smaller naturals. We continue in this way, stopping when we reach a prime number and factoring all the composite numbers. For example, we can factor 60 as  $4 \cdot 15$ , then factor 4 as  $2 \cdot 2$  and 15 as  $5 \cdot 3$ , so that we can write 60 as  $2 \cdot 2 \cdot 5 \cdot 3$ . Since 2, 3, and 5 are prime, we can't factor any further.

Eventually, this process should lead to a **prime factorization** of  $x$  — a sequence of prime numbers that yield  $x$  when multiplied together. We have a *recursive algorithm* for factoring, which can be informally stated: “To factor  $x$ , if  $x$  is prime stop, otherwise write  $x = y \cdot z$  with  $y > 1$  and  $z > 1$  and then factor  $y$  and  $z$ ”. This algorithm can't go into an infinite loop (Why?) and so has to stop with all the numbers being prime<sup>7</sup>.

Let's look at some recursive pseudo-Java code to print out a factorization:

<sup>7</sup>We don't yet have logical tools to prove assertions about what will “eventually” happen in such a process — this will be one of the goals of Chapter 4. For the moment, we'll rely on informal intuition about, for example, what a recursive algorithm might do.



©Kendall Hunt Publishing Company

Figure 3-3: The Hasse diagram for divisors of 60.

```
public void factor (natural x) {
// prints sequence of prime factors of x, one per line
// special cases: outputs empty sequence if x is 0 or 1
  if (x <= 1) return;
  natural d = 2;
  while (x \% d != 0) && (d < x) {
    d++;
    if (d*d >= x) d = x;}
  System.out.println(d);
  factor (x/d);}

```

Given a composite number  $x$ , there may be more than one way to factor it into two naturals. In our example above, we could have written  $60 = 3 \cdot 20$ , then  $20 = 10 \cdot 2$  and  $10 = 2 \cdot 5$ , giving us  $60 = 3 \cdot 2 \cdot 5 \cdot 2$ , a different prime factorization. One of our main results in this chapter will be the **Fundamental Theorem of Arithmetic**, which says that every natural has a *unique* prime factorization. Of course, we have to explain what we mean by “unique”, because we just gave two “different” prime factorizations of 60. But the factorizations  $2 \cdot 2 \cdot 5 \cdot 3$  and  $3 \cdot 2 \cdot 5 \cdot 2$  are just rearrangements of the same sequence of primes into a different order — each has two 2’s, a 3, and a 5 so that they have the same number of each prime. This is what the Fundamental Theorem says, that any two prime factorizations of  $x$  are “the same” in this specific way<sup>8</sup>. In the language of Chapter 2, we define an equivalence relation on sequences of primes, and say that any two factorizations of the same number are equivalent in this way.

Once we have the notion of the division relation, prime numbers almost have to arise. Think back to our Hasse diagram for the naturals that divide 60 (in Section 2-10). In the picture, if we draw it with a little care (as in Figure 3-3), — the dots arrange into a three-dimensional array where the three dimensions correspond to the three primes that divide 60: 2, 3, and 5. In fact whenever you

<sup>8</sup>Note that if we considered 1 to be a prime number, the Fundamental Theorem would be false right away, as for example  $2 \cdot 2 \cdot 3 \cdot 5 \cdot 1 \cdot 1$  would be another “prime factorization” of 60 which would not have the same number of each “prime”. It may seem a bit backward to choose a definition so that the theorem will work, but this is actually fairly common practice — we’ll see another example in Excursion 8.10.

look at the divisors of a natural and which ones divide which other ones, this pattern will occur. Every line in the Hasse diagram corresponds to multiplying by a prime.

### 3.1.4 Congruences

Along with divisibility, the other fundamental concept of number theory is that of **modular arithmetic**. Many counting systems, like the days of the week or the months of the year, are **periodic systems** because you eventually return to where you started. Suppose we want to know what month it will be 39 months from now. The property of “39” that is relevant is the *remainder* when 39 is divided by 12, which is 3. Any other number of months which had remainder 3, such as 3, 87, or 1200003, would lead to the same result as 39.

This property of having the same remainder is given a special name:

**Definition:** If two naturals  $a$  and  $b$  have the same remainder when divided by some positive natural  $r$ , we say that  $a$  and  $b$  are **congruent modulo  $r$** . Symbolically, we write this statement as “ $a \equiv b \pmod{r}$ ”. The binary relation on naturals arising from this predicate, the set  $\{(a, b) : a \equiv b \pmod{r}\}$ , is called the **congruence relation**.

We can rephrase congruence in terms of the division relation:  $a \equiv b \pmod{r}$  if and only if  $r$  divides<sup>9</sup>  $a - b$ . A third way to phrase it is by using the “integer divide” function found in many programming languages — if  $a$  and  $r$  are naturals then the remainder when  $a$  is divided by  $r$  is called “ $a \% r$ ” in Java<sup>10</sup>. So  $a \equiv b \pmod{r}$  is true if and only if  $a \% r == b \% r$  in Java<sup>11</sup>.

The simplest example of a congruence is the even and odd numbers — two naturals are congruent modulo 2 if and only if they are both even or both odd. Two naturals are congruent modulo 10 if and only if they have the same last digit. In Section 2.11 we defined *equivalence relations*, of which congruences are an important example, and we’ll look further at the congruence relation in Section 3.8. The odd numbers are an example of an *equivalence class* — the set of all things equivalent to a given thing, in this case  $\{n : n \equiv 1 \pmod{2}\}$ . In general, given naturals  $a$  and  $r$ , the set of naturals  $\{n : n \equiv a \pmod{r}\}$  is called a **congruence class**.

It is often helpful to think of a congruence class as a “number” in a new, different number system. For example, the product of an odd number and an even number is always even, no matter which particular numbers we pick. If we add a natural ending in 4 to one ending in 7, the result ends in 1. These are two examples of a general law that we’ll prove in Section 3.3 — it makes sense to add or multiply two congruence classes (with the same modulus) to get a new congruence class.

---

<sup>9</sup>There is a slight technicality here, in that  $a - b$  may not be a natural in our terminology if it is negative, but it turns out that we can speak of a natural dividing a negative integer without causing any problems.

<sup>10</sup>The Pascal notation for this operation is “ $a \bmod r$ ”, which can be confusing because it uses the word “mod” differently from the standard mathematical usage.

<sup>11</sup>That is, as long as none of the numbers involved is *negative* – see Problem 3.1.10.

### 3.1.5 Exercises

- E3.1.1 Find complete factorizations for the numbers 15, 16, 17, 132, 136, 140, 175, 179, and 195.
- E3.1.2 Divide the numbers from Exercise 3.1.1 into congruence classes modulo 6.
- E3.1.3 Is the % operator associative? Explain why it always is or give an example where it is not.
- E3.1.4 The low-order bit of the binary expansion of a natural  $x$  is given by  $x\%2$ . Find an expression, using the % and / operators, that gives the *third-lowest-order* bit of the binary expansion of  $x$ . (The third-lowest-order bit of the binary number 10000100 is 1.)
- E3.1.5 Find all the prime numbers less than 200 by using a Sieve of Eratosthenes. A useful shortcut is to put the numbers in rows of six, that is, put 1 through 6 in the first row, 7 through 12 in the second with 7 under 1, and so on. Actually if you do this you can dispense with all but the columns headed by 1 and 5, as the other columns contain no primes other than 2 and 3.
- E3.1.6 List all primes less than 1000 whose decimal digits are all the same. Justify your answer without reference to a list of primes.
- E3.1.7 Prove that if  $n$  is composite, then so is  $2^n - 1$ .
- E3.1.8 Explain carefully why if  $n$  is composite, it has a factor  $d$  with  $d > 1$  and  $d^2 \leq n$ .
- E3.1.9 Suppose we want to find all primes less than  $n$  using a Sieve of Eratosthenes. By writing the numbers in rows of six, we can eliminate four of the six columns and thus avoid having to consider two thirds of the possible numbers (after dealing with the first row or two). What fraction could we avoid by using eight columns? Ten columns? Twelve? Can you find a row size that would let us avoid *more* than two thirds of the numbers?
- E3.1.10 (uses Java) Argue carefully that the `isPrime` method of the text is correct. If it returns `false`, how do we know that there is a proper factor of  $a$ ? If it returns `true`, how do we know that there isn't?

### 3.1.6 Problems

- P3.1.1 The naturals from 114 through 126 are all composite. Find 100 consecutive naturals that are all composite. (Hint: The **factorial** of a natural  $k$ , written  $k!$ , is the product  $1 \cdot 2 \cdot \dots \cdot k$ . You can tell a lot about which numbers divide  $k!$ .)
- P3.1.2 Find the *smallest* sequence of *more than thirteen* consecutive composite numbers. (It may be a bit tiresome to find this by hand — can you write a computer program that will find it?)
- P3.1.3 The **greatest common divisor** of two naturals  $x$  and  $y$  is the largest number that divides both  $x$  and  $y$ . For example,  $\gcd(8, 12) = 4$  because 4 divides both 8 and 12 and no larger natural divides both.
- (a) What is the greatest common divisor of 60 and 339? How do you know?
- (b) What are the possible greatest common divisors of  $p$  and  $x$ , if  $p$  is a prime number? How can you tell which is the correct one, given any  $x$ ?



- (c) What is the greatest common divisor of  $2^3 \cdot 3^2 \cdot 5^4$  and  $2^2 \cdot 3^4 \cdot 5^3$ ? Can you describe a method to find the greatest common divisor of any two numbers, given their factorization into primes? (You might not be able to prove that your method is correct, of course, without results from later in this chapter).
- P3.1.4 The **least common multiple** of two naturals  $x$  and  $y$  is the *smallest* natural that both  $x$  and  $y$  divide. For example,  $lcm(8, 12) = 24$  because 8 and 12 each divide 24, and there is no smaller natural that both 8 and 12 divide.
- Find the least common multiple of 60 and 339.
  - Find the least common multiple of  $2^3 \cdot 3^2 \cdot 5^4$  and  $2^2 \cdot 3^4 \cdot 5^3$ .
  - Describe a general method to find the least common multiple of two naturals, given their factorization into primes (and assuming that the factorization exists and is unique).
- P3.1.5 Let  $x$  and  $y$  be any two positive naturals. Using the definitions in Problems 3.1.3 and 3.1.4, show that  $gcd(x, y) = \frac{xy}{lcm(x, y)}$
- by using the characterizations of  $gcd$  and  $lcm$  in terms of the prime factorization, and
  - directly by proving that for any natural  $z$ ,  $z$  is a common factor of  $x$  and  $y$  if and only if  $z$  divides  $xy$  and  $xy/z$  is a common multiple of  $x$  and  $y$ .
- P3.1.6 Two naturals are defined to be **relatively prime** if their greatest common divisor is 1. Note that this can happen even if either or both of the two naturals are composite.
- Show that if  $x$  is prime, and  $y$  is not a multiple of  $x$ , then  $x$  and  $y$  are relatively prime.
  - Give an example of two composite numbers that are relatively prime.
  - Recall the example of a hash table, where we have  $n$  locations and respond to a collision by moving up  $k$  locations. Show that if  $n$  and  $k$  are *not* relatively prime, then repeatedly skipping  $k$  locations will *not* reach all possible locations.
  - If  $n$  and  $k$  are relatively prime, will repeatedly skipping  $k$  locations reach all the possible locations? (Try to explain why or why not, although we won't have the tools to prove the answer until later in this chapter.)
- P3.1.7 A **perfect number** is a natural that is the sum of all its proper divisors. For example,  $6 = 1 + 2 + 3$  and  $28 = 1 + 2 + 4 + 7 + 14$ . Prove that if  $2^n - 1$  is prime, then  $(2^n - 1)2^{n-1}$  is a perfect number. (A prime of the form  $2^n - 1$  is called a **Mersenne prime**. Every perfect number known is of the form given here, but it is unknown whether there are any others.)
- P3.1.8 (uses Java) Using the Least Number Axiom of Excursion 1.2, prove carefully that the **factor** method given in the text correctly factors any natural. (Hint: Assume that  $n$  is the smallest number that it *doesn't* factor correctly, trace the method with input  $n$ , and derive a contradiction.)
- P3.1.9 (uses Java) Rewrite the **factor** method so that on any input, it gives the same output as before, but there is no use of recursion.
- P3.1.10 (uses Java) The designers of Java, following those of C and C++, made the **%** symbol a **remainder operator** rather than a **modular division operator**. The definition of division comes first:  $a/b$  is the integer obtained from the rational number  $a/b$  by rounding *towards zero*. For example,  $-7 / 5$  is  $-1$ . Then the remainder **a % b** is defined in order to satisfy the identity  $(a/b)*b + (a \% b) = a$ . This behavior might not necessarily be what you expect.

- (a) Using this Java definition, determine the values of  $7 \% 5$ ,  $-7 \% 5$ ,  $7 \% -5$ , and  $-7 \% -5$ .
- (b) Show that two integers  $a$  and  $b$  may be congruent modulo  $r$  even though  $a \% r$  is not equal to  $b \% r$ .
- (c) Argue that if  $a \% r == b \% r$ , then  $a$  and  $b$  are congruent modulo  $r$ , even if some of all of the numbers are negative.
- (d) Write a real-Java static method `int modClass(int a, int r)` that, if  $r$  is positive, returns a number  $b$  such that  $a$  and  $b$  are congruent modulo  $r$  and  $0 \leq b < r$ .
- (e) Argue that `modClass (a, r) == modClass (b, r)` if and only if  $a$  and  $b$  are congruent modulo  $r$ .

### 3.2 Excursion: Playing With Numbers

Working with prime numbers and divisibility gives rise to a number of computational problems: Does this natural divide that one? Is this natural prime? If not, what are its factors? Before the days of hand calculators, various computational tricks were developed to solve some of these by hand or even mentally. In this Excursion we'll learn some of these tricks, and use them to solve some problems about primality.

In ordinary base ten arithmetic, the easiest questions to answer about a natural are whether it is divisible by two or by five. You probably know that both these questions can be solved by looking at the last digit of the natural. If that digit is 0 or 5, the number is divisible by five, and not otherwise. If that digit is even (divisible by two) then so is the natural itself, and not otherwise.

The reason that this works is that any natural is congruent, modulo ten, to its last base ten digit. For example, 17256 is equal to  $10(1725) + 6$ , so ten divides the difference of 17256 and 6, and thus  $17256 \equiv 6 \pmod{10}$ . Two naturals congruent modulo ten are also congruent modulo five and modulo two, since both two and five divide ten.

This is all you can tell from the last digit, but the last *two* digits tell you more. A natural is congruent to its last two digits modulo 100 (for example, 17256 is congruent to 56), and this means that you can use the last two digits to test divisibility by any natural that divides 100, in particular divisibility by 4. Similarly you can test divisibility by 8 by looking at the last three digits, divisibility by 16 by looking at the last four digits, and so forth. Our example of 17256, in particular, is divisible by 4 (4 divides 56), 8 (8 divides 256) but not 16 (in fact  $17256 = 16(1078) + 8$ , and  $17256 = 8 \cdot 2157$ ).

The next trick involves the *sum of the digits* of a natural. In base ten, the number 9 has the convenient property that every power of 10 is congruent to 1 modulo 9 (that is,  $\forall k : 10^k \equiv 1 \pmod{9}$ ). We can see that this is true by looking at  $10^k - 1$ , which in base 10 is represented by a string of  $k$  nines and thus clearly equals 9 times a string of  $k$  ones. (For example, with  $k = 5$ ,  $99999 = 9 \cdot 11111$ .) This makes any natural congruent modulo 9 to the sum of its digits — for example:

$$17256 = 1(10)^4 + 7(10)^3 + 2(10)^2 + 5(10)^1 + 6(10)^0 \equiv 1 + 7 + 2 + 5 + 6 = 21,$$

because we can replace each of the powers of 10 by one and still have a valid congruence, using the fact that the sum of congruences is another congruence. 21 is not divisible by 9, so we know that 17256 isn't, but it is divisible<sup>12</sup> by 3. Because 3 divides 9, and  $21 \equiv 17256 \pmod{9}$ , we know  $21 \equiv 17256 \pmod{3}$  and thus that 3 divides 17256. So we can test divisibility by 3 for any natural with ease.

We have made some progress toward a prime factorization of 17256, as we now know that  $17256 = 8 \cdot 3 \cdot 719$ . But is 719 prime? It isn't divisible by 2, 3, or 5, but to be sure it is prime we'd have to check it for divisibility by all the primes  $p$  such that  $p^2 \leq 719$ , namely 7, 11, 13, 17, 19, and 23. There is a way to speed up mental calculation for these tests, which we'll see, but first let's see a couple of tricks for specific numbers.

---

<sup>12</sup>If we're really lazy we can use the trick again to get that 21 is congruent modulo 9 to  $2 + 1 = 3$ .

There are two tricks for testing divisibility by 11, one a simple extension of the trick for 9. If we divide a natural into *pairs* of digits starting from the right, and add the resulting two-digit numbers, we get a natural which is congruent to the original number modulo 99, and hence modulo 11. Why? It's the same principle, that since  $100 \equiv 1 \pmod{99}$  every power of 100 is also congruent to 1. This tells us that 719 is not divisible by 11 since it's congruent to  $7 + 19 = 26$ . Neither is 17256, for that matter<sup>13</sup>, since it's congruent to  $1 + 72 + 56 = 129$  and thus to  $1 + 29 = 30$ .

The other trick for 11 involves the fact that while *even* powers of 10 are congruent to 1 modulo 11, *odd* powers of 10 are congruent to 10 itself, and thus to  $-1$ . So any number is congruent modulo 11 to the *alternating* sum of its digits, arranged so that the last digit gets a plus sign. 719 is congruent to  $7 - 1 + 9 = 15$ , and 17256 is congruent to  $1 - 7 + 2 - 5 + 6 = -3$ . Note that these results are consistent with what we found above.

Breaking a natural into three-digit segments and adding them is a valid test for divisibility by 999, but since  $999 = 27 \cdot 37$  and we can already test for divisibility by three, this is really only useful for testing for divisibility by 37, which doesn't come up all that often. But we can also have larger versions of the alternating test. Alternating sums of two-digit segments tests for divisibility by 101, a relatively rare prime, but three digit segments work for 1001, a more useful number because it factors as  $1001 = 7 \cdot 11 \cdot 13$ . This doesn't help for 719, of course, but it means that for any natural of more than three digits, we can get a natural of three digits that is congruent to it modulo 7, 11, and 13. For example, from 17256 we'd get  $-17 + 256 = 239$ , which happens to not be divisible by any of these three numbers (we knew about 11 already).

Here is one more general trick for testing divisibility by one of the small primes for which the above tricks don't work (or still leave you with a three-digit number to test). Remember that in carrying out the division, you don't care about the quotient, only whether the remainder is zero. So you make progress if you can convert the given natural to a smaller natural such that the two numbers are either both divisible by the prime or both not divisible by it. A quick way to do this mentally is to first add or subtract a multiple of the prime to get a number ending in 0, then remove the 0. Clearly adding or subtracting the multiple gives you something congruent modulo the prime. Dividing by 10 won't give you something congruent, but as long as the prime in question isn't 2 or 5, it will not change the divisibility of the number by the prime (the reason for this will be clearer later in the chapter).

So to test 719 for divisibility by 7, for example, we add 21 to get 740 and divide by 10 to get 74, whereupon if we know our multiplication tables we are done because 7 does not divide 74. In this way we can soon verify that 719 is in fact prime. We checked 11, and to do 13 we can subtract  $3 \cdot 13 = 39$  from 719 to get 5 = 680, and note that 13 does not divide 68. For 17, we may add  $3 \cdot 17 = 51$  to 719 getting 770, and 17 does not divide 77 (since it does divide 68 and 85). Checking 19 is even easier, as subtracting 19 leaves 700, and 19 certainly doesn't divide 7. Finally for 23 we subtract  $3 \cdot 23 = 69$  from 719 leaving 650, and 23 does not divide 65. Since the next prime, 29, has a square bigger than 719, we are done and have shown that 719 is prime.

---

<sup>13</sup>By the Fundamental Theorem of Arithmetic, which we've not yet proved, it wouldn't be possible to multiply some naturals that are each *not* divisible by 11 (here 8, 3, and 719) and get a natural that is divisible by 11, because this would mean that the larger number had two different prime factorizations.

### 3.2.1 Writing Exercise

It's easy to come up with problems that can be solved using the tricks described above – here are some samples. Show your work clearly. If you use a calculator, write down the facts supporting your conclusion that your calculator gave you.

1. The smallest prime greater than 1000. (Clearly 1001 doesn't work, so try 1003 and succeeding odd numbers if that's not it.)
2. The largest prime number less than 1000 (it's not 999).
3. The smallest prime greater than 10000.
4. The largest prime less than 10000.
5. The prime factorization of 3153235554.

## 3.3 Modular Arithmetic

### 3.3.1 Arithmetic on Congruence Classes

We asserted in Section 3.1 that if we have two congruence classes with the same base, such as  $\{x : x \text{ ends in } 7\}$  and  $\{x : x \text{ ends in } 4\}$ , we can “add” and “multiply” them to get a new congruence class with the same base. In the example,  $27 \cdot 4294$  and  $3567 \cdot 204$  are each going to end in 8, and  $27 + 4924$  and  $3567 + 204$  are each going to end in 1. Is it obvious that we can take the sum or product of two classes like this? Perhaps, but we are making a particular claim in each case, that it *doesn't matter* which representative from each class we choose, as long as we only care about the congruence class of the result. For addition, we can state this claim (“that addition on congruence classes is well-defined”) in the predicate calculus as follows:

$$\forall r : \forall a : \forall b : \forall c : \forall d : [(a \equiv b \pmod{r}) \wedge (c \equiv d \pmod{r})] \rightarrow (a + c \equiv b + d \pmod{r})$$

Let's use our rules for the predicate calculus, and our knowledge of arithmetic, to prove this claim. We let  $r, a, b, c$ , and  $d$  be arbitrary and assume both  $a \equiv b \pmod{r}$  and  $c \equiv d \pmod{r}$ . Actually a more useful form for  $a \equiv b \pmod{r}$  in this case is  $D(r, a - b)$  or<sup>14</sup>  $\exists i : a = ir + b$ . Similarly the other assumption can be written  $\exists j : c = jr + d$ . Although we're trying to show that it's all right to add *congruences*, we know we can add equations over the *integers*, so adding these two equations together we get  $a + c = ir + b + jr + d = (i + j)r + (b + d)$ . Now we've just calculated the difference between  $a + c$  and  $b + d$  as  $(i + j)r$ , and  $r$  clearly divides this difference, so we have that  $a + c \equiv b + d \pmod{r}$ .

The argument for multiplying two congruences involves a bit more arithmetic but is very similar. We first let  $r, a, b, c$  and  $d$  be arbitrary as above with the same assumptions — our goal is to show that  $ac \equiv bd \pmod{r}$ . Again the assumptions give us  $\exists i : a = ir + b$  and  $\exists j : c = jr + d$ , so  $ac = (ir + b)(jr + d) = ijr^2 + ird + bjr + bd = r(ijr + id + bj) + bd$ . As before,  $r$  clearly divides the difference between the two numbers in question, so they are congruent modulo  $r$ .

It's not hard to show (see Exercise 3.3.2) that we can *subtract* two congruences with the same base. What about dividing? Let's first review exactly what dividing one natural by another really means. In the real numbers, every real number  $x$  (except zero) has a **multiplicative inverse**  $x^{-1}$ , another real number that gives 1 when multiplied<sup>15</sup> by  $x$ . This is not true over the integers, as 1 and  $-1$  have inverses but no other numbers do (within the data type of the integers). “Dividing by  $x$ ” is the same thing as “multiplying by  $x^{-1}$ ”, so the existence of inverses controls whether we are able to divide while staying within the data type<sup>16</sup>.

What about our new “numbers”, the congruence classes? If we look at the congruence classes modulo some base  $r$ , do they have inverses? An **inverse** of  $x$  now means a number  $y$  such that

<sup>14</sup>We can either allow  $i$  to be a possibly negative integer, or switch the roles of  $a$  and  $b$  if necessary so that  $a$  is larger.

<sup>15</sup>Why 1? Because 1 is the **multiplicative identity**, in that  $x \cdot 1 = 1 \cdot x = x$  for any real number  $x$ . Remember from Section 2.9 that the *inverse* of a function  $f$  is the function that gives the *identity* function when composed with  $f$ .

<sup>16</sup>Yes, we have the Java `/` and `%` operators over the integers, but they are not the same thing as division — here by “division” we mean the inverse operation of multiplication.

$xy \equiv 1 \pmod{r}$ . (Or, equivalently, that  $xy + kr = 1$  for some integer  $k$ . We can express the predicate “ $x$  has an inverse modulo  $r$ ” by the quantified statement  $\exists k : xy + kr = 1$ .) As it happens, sometimes they exist and sometimes they don’t. For example, 5 is an inverse of 11 modulo 6, because  $5 \cdot 11 \equiv 1 \pmod{6}$ . But 4 has no inverse modulo 6. Can you see why?

To explain exactly when inverses exist, we need to review some definitions from Problems 3.1.3, 3.1.4, and 3.1.6.

**Definition:** Let  $a$  and  $b$  be any two naturals. We say that  $a$  and  $b$  are **relatively prime** if they have no common factor. That is, there is no number  $c > 1$  such that both  $D(c, a)$  and  $D(c, b)$ .

**Definition:** The **greatest common divisor** of two naturals<sup>17</sup>  $a$  and  $b$ , written  $gcd(a, b)$ , is the largest natural that divides both  $a$  and  $b$ . So by definition,  $a$  and  $b$  are relatively prime if and only if their greatest common divisor is 1. The **least common multiple** of  $a$  and  $b$ , written  $lcm(a, b)$ , is the smallest number  $c$  such that  $a$  and  $b$  both divide  $c$ . We proved in Problem 3.1.5 that  $lcm(a, b) = ab/gcd(a, b)$ .

These definitions allow us to state a useful result that we’ll prove shortly:

**Inverse Theorem:** A natural  $a$  has an inverse mod  $m$  if and only if  $a$  and  $m$  are relatively prime.

### 3.3.2 The Euclidean Algorithm

First, though, if we are given two naturals, how could we determine whether they are relatively prime? This problem, like many others in this area, was solved by the ancient Greeks. The **Euclidean Algorithm** inputs two numbers and outputs their greatest common divisor. Call the input numbers  $a_0$  and  $a_1$ . Compute the quotient ( $a_0/a_1$  in Java) and the remainder ( $a_0 \% a_1$  in Java). Now set  $a_2$  to be the remainder  $a_0 \% a_1$ . Then set  $a_3$  to be  $a_1 \% a_2$ , and so on. Eventually you will get an  $a_k$  that is equal to zero. (Can you explain why?). Once this happens,  $a_{k-1}$  is our desired greatest common divisor. Here’s an example with  $a_0 = 72$  and  $a_1 = 46$ :

$$\begin{array}{rcl} 72 \% 46 & = & 26 \\ 46 \% 26 & = & 20 \\ 26 \% 20 & = & 6 \\ 20 \% 6 & = & 2 \\ 6 \% 2 & = & 0 \end{array}$$

So the sequence of  $a$ ’s is  $\langle 72, 46, 26, 20, 6, 2, 0 \rangle$  and  $a_5 = 2$  is the greatest common divisor. To see why we get the greatest common divisor, let’s rewrite the sequence also showing the successive

<sup>17</sup>The two naturals 0 and 0 are not relatively prime, and they (unlike any other pair of naturals) have no greatest common factor.

quotients – the / numbers:

$$\begin{aligned}72 &= 1 \cdot 46 + 26 \\46 &= 1 \cdot 26 + 20 \\26 &= 1 \cdot 20 + 6 \\20 &= 3 \cdot 6 + 2 \\6 &= 3 \cdot 2 + 0\end{aligned}$$

Because the last remainder is 0, we know that 2 divides 6. Because 2 divides both 2 and  $3 \cdot 6$  (the latter because it divides 6), it divides 20. Each of the numbers 26, 46, 72 must also be multiples of 2, because they are each a sum of multiples (a **linear combination**) of numbers already known to be multiples of 2. Thus 2 (or in general the output of the algorithm) divides both 46 and 72 (or in general, the inputs). In particular, if the output is not 1, the inputs *are not* relatively prime.

Conversely, suppose some natural  $x$  divided both 72 and 46. Because 26 is the difference of two multiples of  $x$ , it must itself be a multiple of  $x$ . Similarly, so must be 20, 6, and 2. But now we know that any common divisor divides 2, and 2 is a common divisor, so it must be the greatest common divisor (similarly, the output will always be the greatest common divisor of the inputs). In particular, if the output is 1, any common divisor divides 1 and the inputs *are* relatively prime. We have shown:

**Proposition:** On input  $a$  and  $b$ , the Euclidean Algorithm outputs the greatest common divisor of  $a$  and  $b$ . ♠

The same calculations tell us all we need to know about the existence of an inverse for  $a_1$  modulo  $a_0$ . If  $a_0$  and  $a_1$  have a common factor greater than 1, as in our example of 4 and 6 above, any linear combination of the inputs will also have that same common factor, and it is impossible for that linear combination to equal 1. On the other hand, suppose the output of the Euclidean algorithm is 1. We can use the arithmetic from the algorithm to show that 1 is a linear combination of the inputs, and hence that an inverse exists. First, here's the calculation for the example above, showing that the output 2 is a linear combination of the inputs. We begin by expressing the numbers 72 and 46 themselves as linear combinations of 72 and 46:

$$\begin{aligned}72 &= 1 \cdot 72 + 0 \cdot 46 \\46 &= 0 \cdot 72 + 1 \cdot 46\end{aligned}$$



Now since we know that  $26 = 72 - 46$ , we can subtract the second equation from the first to get:

$$26 = 1 \cdot 72 - 1 \cdot 46$$

And again, since  $20 = 46 - 26$  and  $6 = 26 - 20$ , subtracting each equation from the one before it gives:

$$\begin{aligned} 20 &= -1 \cdot 72 + 2 \cdot 46 \\ 6 &= 2 \cdot 72 - 3 \cdot 46 \end{aligned}$$

Now, because  $2 = 20 - 3 \cdot 6$ , we can obtain a linear combination for 2 by subtracting *three times* the last equation from the equation before it:

$$2 = -7 \cdot 72 + 11 \cdot 46$$

Thus 2 is shown to be a linear combination of 72 and 46.

Now let's try the same thing for two naturals that *are* relatively prime, 72 and 47. First we run the algorithm to show that they really are relatively prime, saving the quotient of each division for later use:

$$\begin{aligned} 72 &= 1 \cdot 47 + 25 \\ 47 &= 1 \cdot 25 + 22 \\ 25 &= 1 \cdot 22 + 3 \\ 22 &= 7 \cdot 3 + 1 \\ 3 &= 3 \cdot 1 + 0 \end{aligned}$$

Now we can calculate 1 as a linear combination of 72 and 47. Each line of the derivation above tells us how to get the next linear combination — for example, since  $22 = 7 \cdot 3 + 1$  and thus  $1 = 22 - 7 \cdot 3$ , we will subtract seven copies of the linear combination for 3 from the linear combination for 22 to get a linear combination for 1.

$$\begin{aligned}
72 &= 1 \cdot 72 + 0 \cdot 47 \\
47 &= 0 \cdot 72 + 1 \cdot 47 \\
25 &= 1 \cdot 72 - 1 \cdot 47 \\
22 &= -1 \cdot 72 + 2 \cdot 47 \\
3 &= 2 \cdot 72 - 3 \cdot 47 \\
1 &= -15 \cdot 72 + 23 \cdot 47
\end{aligned}$$

Now we can just read off the inverse of 47 modulo 72 — it is 23, because we can see that  $23 \cdot 47$  differs from 1 by a multiple of 72 and is thus congruent to 1 modulo 72. (In fact  $15 \cdot 72 = 1080$  and  $23 \cdot 47 = 1081$  — if you want to see the power of this method imagine searching by trial and error for a multiple of 72 and a multiple of 47 that differ by one.) (How would you get the inverse of 72 modulo 47 — do you need any more calculations?)

It's worth noting that unlike the tests for primality we discussed earlier in Section 3.1, this algorithm is actually practical for large inputs. To see this, consider numbers of, say, 100 digits. It can be proven<sup>18</sup> that the Euclidean algorithm on such numbers will terminate in a few hundred steps (it takes *linear time* in the size of the input) — bad for hand calculation but fast for a computer<sup>19</sup>. On the other hand, to test the primality of a 100-digit number by the methods we've described would be inconceivable. You would have to try individually every number up to the 50-digit square root, or  $10^{50}$  numbers. Making an Eratosthenes sieve would mean constructing a data structure of  $10^{100}$  bits. There are better primality testing methods, but they are beyond the scope of this book<sup>20</sup>. The problem of *factoring* a composite number (rather than just showing that it is composite) seems to be much harder — at the moment no practical method to factor an arbitrary 250-digit number is known. We'll see in Section 3.11 that the presumed difficulty of this problem is the basis of the RSA public-key cryptosystem.

### 3.3.3 Exercises

E3.3.1 Calculate the following quantities:

- (a)  $3 \cdot 4 + 5 \pmod{7}$
- (b)  $2 \cdot 10 + 3 \cdot 4 \cdot (4 + 9) \pmod{11}$
- (c)  $((13 - (4 \cdot 6)) \cdot 2) + 3 \pmod{23}$

E3.3.2 If  $r$ ,  $a$ ,  $b$ ,  $c$ , and  $d$  are any five numbers such that  $a \equiv b \pmod{r}$  and  $c \equiv d \pmod{r}$ , prove that  $a - c \equiv b - d \pmod{r}$ .

---

<sup>18</sup>We do this in Section 4.11.

<sup>19</sup>Though of course you couldn't code 100-digit numbers as `int` or even `long` variables in Java, so what might you do instead?

<sup>20</sup>In 2002 Agrawal, Kayal, and Saxena gave the first algorithm that always gets the right answer and tests an  $n$ -digit number in a time bounded by a polynomial in  $n$  (proportional to about  $n^{12}$ ). Thus the problem of primality testing is in the class **P** discussed in Section 10.11. In practice, the method generally used is much faster than this, but poses a tiny risk of falsely identifying a composite number as prime. We'll say more about this in Excursion 3.10.

E3.3.3 Showing all the steps, use the Euclidean Algorithm to find the greatest common divisors of the following pairs of naturals:

- (a) 315 and 453.
- (b) 317 and 453.
- (c) 4096 and 1729.
- (d) 377 and 610.
- (e) 1367439 and 1025677.

E3.3.4 If  $a$  has an inverse modulo  $r$ , can you necessarily conclude that  $r$  has an inverse modulo  $a$ ? Why or why not?

E3.3.5 Find the inverse of:

- (a) 144, modulo 377.
- (b) 511, modulo 512.
- (c) 37, modulo 512.
- (d) 512, modulo 37.

E3.3.6 Let  $a$ ,  $b$ , and  $c$  be any three positive naturals, with  $b \leq c$ . What happens when we run the Euclidean Algorithm with inputs  $a^b$  and  $a^c$ ? What is the greatest common divisor of these two numbers?

E3.3.7 (uses Java) Here we want a real-Java static method `int eA (int a, int b)` that returns the greatest common divisor of  $a$  and  $b$ , calculated by the Euclidean Algorithm.

- (a) Write such a method that uses a loop.
- (b) Write such a method that calls itself recursively.

E3.3.8 (uses Java) Consider the following alternate version of the Euclidean Algorithm:

```
public static natural simpleEA (natural a, natural b) {  
    if (b == 0) return a;  
    if (b > a) return simpleEA (b, a);  
    return simpleEA(a - b, b);} 
```

- (a) Trace the execution of `simpleEA` on inputs 4 and 10.
- (b) In the last line, how can we be sure that the subtraction  $a - b$  does not have a negative result?
- (c) Explain why `simpleEA` outputs the greatest common divisor of  $a$  and  $b$ . (Hint: Argue that its output is the same as the last nonzero number in the sequence from the original Euclidean Algorithm on input  $a$  and  $b$ .)

E3.3.9 Let  $f$  and  $g$  be **polynomials**, in one variable  $x$ , with real number coefficients. Define the relation  $D(f, g)$  to mean  $\exists h : fh = g$ , that is, “ $f$  divides  $g$ ”.

- (a) Prove that  $D$  is reflexive and transitive, but is not a partial order on polynomials because it is not antisymmetric.

- (b) A **monic** polynomial is one whose highest-degree coefficient is 1. Show that  $D$  is a partial order on monic polynomials.
- (c) Prove that for any nonzero polynomial  $p$ , there exists a monic polynomial  $m$  such that  $D(m, p)$  and  $D(p, m)$  are both true.

E3.3.10 Prove carefully that if  $a = qb + r$ , where all four numbers are naturals, and  $a$  and  $b$  are each integer linear combinations of two other naturals  $m$  and  $n$ , that  $r$  is also an integer linear combination of  $m$  and  $n$ .

### 3.3.4 Problems

- P3.3.1 Show that if  $x$  and  $y$  are relatively prime, *any* integer  $z$  can be written as a linear combination  $ax + by = z$ , where  $a$  and  $b$  are integers. Illustrate this by writing 343 as a linear combination of 1729 and 4096. If  $x$  and  $y$  are *not* relatively prime, which integers can be written as linear combinations of  $x$  and  $y$ ?
- P3.3.2 Let  $n$  be any natural. Define  $I_n$  to be the set of all naturals  $a$  such that  $a < n$  and  $a$  and  $n$  are relatively prime. The **Euler totient function**  $\phi(n)$  is the size of  $I_n$ .
- (a) Describe the sets  $I_{11}$  and  $I_{12}$ .
  - (b) Explain why  $\phi(n) = n - 1$  if and only if  $n$  is prime.
  - (c) Explain why if  $n$  is a power of 2, other than  $2^0 = 1$ , then  $\phi(n) = n/2$ .
  - (d) Explain why a natural  $a < n$  has an inverse modulo  $n$  if and only if it is in  $I_n$ .
  - (e) Show that if  $a$  and  $b$  are in  $I_n$ , then  $a \cdot b$  is congruent modulo  $n$  to a member of  $I_n$ .
  - (f) Give an example where  $a$  and  $b$  are in  $I_n$  but  $a + b$  is not congruent modulo  $n$  to any member of  $I_n$ .
- P3.3.3 Powers are defined in modular arithmetic just as in ordinary arithmetic:  $x^i \pmod{m}$  is simply  $x \cdot x \cdot \dots \cdot x \pmod{m}$  where there are  $i$  copies of  $x$  in the sum. For each of the values 8, 9, 10, 11, 12, and 13 for  $m$ , calculate  $2^i \pmod{m}$  for  $i$  from 0 to 20. In each case, find a general rule that will tell you  $2^i \pmod{m}$  for larger  $i$ . Justify your general rule as convincingly as you can.
- P3.3.4 We have defined the **factorial**  $n!$  of a natural  $n$  to be the product of all the naturals from 1 through  $n$ , with  $0!$  being defined as 1. Let  $p$  be an odd prime number. Prove that  $(p - 1)!$  is congruent to  $-1$  modulo  $p$ . (Hint: Pair as many numbers as you can with their multiplicative inverses.)
- P3.3.5 We have shown that two positive naturals  $a$  and  $b$  are relatively prime if and only if some *integer* linear combination of them equals 1, that is, if and only if  $\exists x : \exists y : ax + by = 1$  where  $x$  and  $y$  range over the integers. An equivalent definition, with  $u$  and  $v$  ranging over the *naturals*, is  $\exists u : \exists v : au = bv + 1$ . Prove each of these two statements from the other.
- P3.3.6 (uses Java) Write a real-Java method that inputs two `int` values  $a$  and  $b$  (which you may assume to be positive) and returns the inverse of  $a$  modulo  $b$ .

P3.3.7 In Exercise 3.3.9 we defined a division relation  $D$  on polynomials in one variable  $x$ , by saying that  $D(f(x), g(x))$  is true if and only if there exists a polynomial  $h(x)$  such that  $f(x)h(x) = g(x)$ . Given this definition, we can try to find the “greatest” common divisor of two polynomials with an analogue of the Euclidean Algorithm. Here “greatest” means “with the largest possible degree”.

- If  $p(x)$  and  $s(x)$  are any polynomials in  $x$ , with real coefficients, argue that there must exist polynomials  $q(x)$  and  $r(x)$  such that  $s(x) = p(x)q(x) + r(x)$ , and either  $r = 0$  or the degree of  $r$  is strictly less than the degree of  $p$ . (Hint: The process of finding  $q$  and  $r$  is much like long division in decimal arithmetic.)
- Prove that if  $s = pq + r$  as in part (a), and any polynomial  $t$  divides both  $p$  and  $s$ , it also divides both  $p$  and  $r$ .
- Prove that if  $s = pq + r$  as in part (a), any common divisor of  $p$  and  $s$  is also a divisor of  $r$ .
- By repeated division as in the Euclidean Algorithm, find a monic greatest common divisor of  $x^3 + 6x^2 - x - 30$  and  $x^2 - 2x - 35$ .
- Find two polynomials that have two different “greatest” common divisors of the same degree. That is, each common divisor has degree  $d$ , and no common divisor has degree greater than  $d$ .

P3.3.8 A chess knight, on one turn, moves either two squares vertically and one horizontally, or two horizontally and one vertically. If we consider a knight starting at the point  $\langle x, y \rangle$  in  $\mathbf{Z} \times \mathbf{Z}$ , it has eight possible moves, to  $\langle x + 1, y + 2 \rangle$ ,  $\langle x + 1, y - 2 \rangle$ ,  $\langle x - 1, y + 2 \rangle$ ,  $\langle x - 1, y - 2 \rangle$ ,  $\langle x + 2, y + 1 \rangle$ ,  $\langle x + 2, y - 1 \rangle$ ,  $\langle x - 2, y + 1 \rangle$ , or  $\langle x - 2, y - 1 \rangle$ .

- Prove that given any two points  $\langle x, y \rangle$  and  $\langle x', y' \rangle$  in  $\mathbf{Z} \times \mathbf{Z}$ , there is a sequence of knight moves from the first point to the second.
- Let  $a$  and  $b$  be different positive naturals. An  $(a, b)$ -knight also has eight possible moves, from  $\langle x, y \rangle$  to  $\langle x \pm a, y \pm b \rangle$  or  $\langle x \pm b, y \pm a \rangle$ . What conditions on  $a$  and  $b$  allow the  $(a, b)$ -knight to go from any point in  $\mathbf{Z} \times \mathbf{Z}$  to any other? Prove your answer.
- If  $a$  and  $b$  do not meet the conditions of part (b), exactly which points can the  $(a, b)$ -knight reach from  $\langle x, y \rangle$ ?

P3.3.9 Here is a “multiplication” operation on strings over the alphabet  $\{a, b, c, d\}$ . We concatenate the strings, but whenever we create a pair  $ab$ ,  $ba$ ,  $cd$ , or  $dc$  by concatenation, we delete it. If the deletion creates another such pair, we delete that as well, until we reach a string that has no such pair. For example,  $(acb)(adc) = ac(ba)dc = acdc = a(cd)c = ac$ . Two strings are called “multiplicative inverses” if they “multiply” to  $\lambda$ .

- Show that every string has a multiplicative inverse, and indicate how to find it.
- We say that a string is in “normal form” if it has none of the four forbidden pairs. Show that if a string is in normal form, it has exactly one inverse that is in normal form.

P3.3.10 Let  $f$  be a bijection on the set  $\{1, \dots, n\}$ . We know that  $f$  has an inverse function. A **transposition** on a set is a bijection that switches two elements  $i$  and  $j$  and fixes all other elements. That is,  $t(i) = j$ ,  $t(j) = i$ , and  $t(x) = x$  if  $x \notin \{i, j\}$ . Here we will show that every bijection has an inverse that is made up by composing transpositions.

- (a) Show that for any  $f$ , either  $f(n) = n$  or there exists a transposition  $t$  such that  $(f \circ t)(n) = n$ .
- (b) Suppose we know that any bijection  $g$  on  $\{1, \dots, n\}$  that has  $g(n) = n$  has an inverse that is made up by composing transpositions. Show that with this assumption, any bijection  $f$  on the set has this property.
- (c) Indicate how, using the fact in part (b), we can write a recursive algorithm that will take any  $f$  and find a sequence of transpositions that compose to the inverse of  $f$ .

## 3.4 There are Infinitely Many Primes

### 3.4.1 Specifying the Problem

One of the earliest-discovered and most important facts about the prime numbers is that they *go on forever* — however far you go in the sequence of naturals, you keep finding new primes. This was proved by the ancient Greeks<sup>21</sup> and is still thought of as one of the simplest and most beautiful proofs in mathematics.

If we want to use proof techniques for the predicate calculus, we have the immediate problem of *stating* this theorem in that language. We can state it in English as “There are infinitely many primes”, but as we don’t have a symbol for “infinitely many” we still have some work to do. Fortunately, another way to say the same thing is “Given any natural  $a$ , there is a prime bigger than  $a$ ,” which translates easily into

$$\forall a : \exists b : (b > a) \wedge P(b),$$

where  $P(b)$  means “ $b$  is prime”. We could, if we wanted, break down  $P(b)$  further using the definition of prime number. This language for “infinitely often” occurs fairly frequently in mathematics, as does its negation: we can say “there are only finitely many  $x$  such that  $A(x)$  is true” by negating the other statement and using the negation-of-quantifier rules to get “ $\exists a : \forall b : (b > a) \rightarrow \neg A(b)$ ”.

Following our general proof rules for quantified statements, we need to let  $a$  be an arbitrary natural and prove that there exists a prime number  $b$  such that  $b > a$ . The obvious thing to do would be to figure out ourselves a particular prime number greater than  $a$ , and then prove that it is prime. Curiously, we’re not going to do that. Because the sequence of primes is so irregular, finding the *next* prime after  $a$ , for example, turns out to be far more difficult than proving the existence of *some* prime greater than<sup>22</sup>  $a$ . Our proof won’t even tell us directly what the prime number greater than  $a$  *is* (though we could find it with a little more work).

### 3.4.2 The Proof

So let  $a$  be an arbitrary natural. What we’ll do is construct a natural  $z$  that isn’t divisible by any of the numbers from 2 through  $a$ . This number  $z$  might or might not be prime itself, but as long as it has a **prime divisor**, that prime divisor must be greater than  $a$  because none of the naturals less than  $a$  (except 1) is a divisor of  $z$ . So *some* prime greater than  $a$  exists<sup>23</sup>.

We define  $z$  to be the factorial of  $a$ , plus 1, or  $a! + 1$ . (Recall that the **factorial** of a number

<sup>21</sup>The proof appears in Euclid’s *Elements*, which isn’t only about geometry. In fact another Greek named Eudoxus appears to have proved the result first.

<sup>22</sup>After this theorem has been proved, of course, you know that a simple algorithm will find the next prime — keep taking numbers  $a + 1$ ,  $a + 2$ , and so on and test each for primality until you find one that is prime. But before proving this theorem you can’t rule out the possibility that this search could go on forever.

<sup>23</sup>We could find it by successively trying naturals starting with  $a + 1$  and seeing whether they are prime. Actually, it might be slightly faster to successively try these numbers to see whether they divide  $z$ , since the first number that divides  $z$  must be prime.

is the product of all the numbers up to and including it<sup>24</sup>, so that  $1! = 1$ ,  $2! = 1 \cdot 2 = 2$ , and  $7! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 = 5040$ .) Now it's obvious that  $a!$  is divisible by any number up to and including  $a$ , as that number appeared in the product used to make the factorial. (For example, 5 divides  $6!$  because  $6!$  can be written  $5 \cdot (1 \cdot 2 \cdot 3 \cdot 4 \cdot 6)$ .) Now consider  $z = a! + 1$ . Since  $a! \equiv 0 \pmod{b}$  for any number  $b \leq a$ , we know that  $z \equiv 1 \pmod{b}$  (because we can add congruences, as we proved in Section 3.3). As long as  $b$  isn't equal to 1, this implies that  $b$  does *not* divide  $z$ . So if we believe that  $z$  can be factored into primes (or even that it has one prime divisor) there must be some prime greater than  $a$  which divides  $z$ , and hence some prime greater than  $a$  exists.

### 3.4.3 Variations on a Theme

There are many possible alternate versions of this argument. We don't really care that no *natural* from 2 through  $a$  divides  $z$ , as long as we're sure that no *prime* in that range divides  $z$  (see the Exercises). So if we multiply together all the primes up through  $a$  and add 1, we get a number whose prime factors are also all greater than  $a$ . More generally, given *any* set of numbers, we can generate a new number not divisible by any of them, by multiplying them all together and adding 1.

This proof is often presented as a *proof by contradiction*<sup>25</sup>. We saw in Chapter 1 that we can prove a proposition  $p$  by assuming  $\neg p$  and deriving something absolutely false (a “contradiction”). Let  $p$  be the proposition “There are infinitely many primes.” If we assume  $\neg p$ , then there must be a finite list containing all the primes. But then if we multiply together all the primes on this list and add 1, we get a number which is not divisible by any prime on the list. Since every number greater than 1 has at least one prime divisor, we have contradicted the hypothesis that the list contained all possible prime numbers.

It's worth looking at what assumptions about the naturals we took for granted in this proof. We assumed that we could add and multiply numbers and always get new numbers<sup>26</sup> and that we could add congruences (as we proved in Section 3.3, using the laws of arithmetic only). The biggest assumption was that every number greater than 1 has at least one prime divisor, which we argued was true on the basis of the behavior of an algorithm (“try all possible proper divisors, if you find one the first one must be prime, if you don't the number is prime itself”). We'll see in Chapter 4 that this assumption can also be proved formally, from a suitable definition of the naturals.

Note also that this abstract proof has told us something about numbers that are too large for us to ever use in a real computation. We know now, for example, that there is a prime number that is too big to be written on a 20-megabyte hard disk, because our theorem tells us that there is a

<sup>24</sup> $0!$  is 1, because an empty product (a product of no terms) is defined to be 1.

<sup>25</sup>Some mathematicians, including a school called the *intuitionists*, are bothered by proofs by contradiction and prefer to turn them into direct proofs wherever possible. For our purposes, we may be satisfied that it is a valid proof technique in any situation, but direct arguments are often clearer (this is a matter of individual taste). Avoiding argument by contradiction (called in Latin *reductio ad absurdum*) does have the advantage that you don't accidentally use the *invalid* proof technique of *reductio ad erratum*. This consists of assuming the negation of the desired proposition and deriving as many consequences as possible, until a typographical or logical error results in your deriving too consequences that contradict each other.

<sup>26</sup>This isn't true of the `int` or `long` data types in Java — why?



prime number bigger than  $2^{160,000,000}$ . Does it even make sense to talk about such absurdly big<sup>27</sup> numbers?

### 3.4.4 Exercises

- E3.4.1 Let  $a$  and  $z$  be naturals and suppose that no prime number  $b$ , with  $2 \leq b \leq a$ , divides  $z$ . Prove that no composite number in this range divides  $z$  either.
- E3.4.2 Calculate the number  $z = a! + 1$  for each value of  $a$  from 1 through 10, and find a prime number greater than  $a$  that divides  $z$ . (Hint: It may be useful to calculate  $\sqrt{z}$  first, to see how many primes you will have to check as possible divisors.)
- E3.4.3 Let  $f(a)$  be defined to be the product of all the primes less than or equal to  $a$ , plus 1. (So  $f(6) = 2 \cdot 3 \cdot 5 + 1 = 31$ , for example.) (The product of primes itself is sometimes called the **primorial** of  $a$ .) Find the smallest value of  $a$  such that  $f(a)$  is composite.
- E3.4.4 Prove carefully that given any set  $S$  of naturals, each greater than 1, there exists a single number  $n$  such that for any number  $x$  in  $S$ ,  $n \% x == 1$ . (That is, prove
- $$\forall S : (\forall x : (x \in S) \rightarrow (x > 1)) \rightarrow \exists n : \forall x : (x \in S) \rightarrow n \% x = 1.)$$
- E3.4.5 Suppose that the arithmetic progression  $\langle a, a + b, a + 2b, \dots \rangle$  contains infinitely many primes. Prove that  $a$  and  $b$  are relatively prime. (Hint: Argue by contradiction.)
- E3.4.6 Let  $n$  be any positive natural. We say that a natural  $a$  with  $0 < a < n$  is a **perfect square**<sup>28</sup> modulo  $n$  if  $a$  is congruent to  $b^2$  modulo  $n$  for some natural  $b$ . (We don't count 0 among the perfect squares, though it is the square of 0.)
- (a) For each  $n$  from 3 through 15, determine which numbers in  $\{0, 1, \dots, n\}$  are perfect squares.
- (b) Prove that if  $a$  and  $b$  are perfect squares, so is  $ab$ .
- E3.4.7 Explain why, if  $n$  is odd, there can be no more than  $(n - 1)/2$  perfect squares modulo  $n$ .
- E3.4.8 In Problem 3.9.5 we will prove that  $p - 1$  is a perfect square modulo a prime  $p$  if and only if  $p = 2$  or  $p$  is of the form  $4k + 1$ . Verify this fact for the primes less than 20.
- E3.4.9 It is also true that if  $p$  is a prime with  $p > 3$ , then  $-3$  is a perfect square modulo  $p$  if and only if  $p$  is of the form  $6n + 1$ . Verify this fact for all such  $p$  less than 20.
- E3.4.10 Fix a natural  $n$  and let  $r$  be the number of primes that are less than  $2^n$ . We know that every positive natural  $x$  with  $x \leq n$  has a factorization into primes, so that  $x = p_1^{e_1} p_2^{e_2} \dots p_r^{e_r}$ . Thus we have a function from the set  $\{1, 2, \dots, 2^n\}$  into the set of tuples  $\langle e_1, e_2, \dots, e_r \rangle$ .
- (a) Explain why for any such  $x$ , each number  $e_i$  must be in the range from 0 to  $n$ .
- (b) Why must this function be one-to-one?

<sup>27</sup>It's possible for us to ~~name~~ certain naturals that are too big to fit on a disk, such as  $10^{10^{10^{10}}}$ . But there can only be so many naturals named by reasonably short strings, whatever naming system we adopt, because there are only so many reasonably short strings. What if we allow names like "the smallest natural that cannot be described on a 20-megabyte disk"? If this is an allowable name, it fits on the disk, and we have a logical problem known as the **Richard paradox**. Hofstadter describes how similar paradoxes form the basis of Gödel's Theorem.

<sup>28</sup>This is also called a **quadratic residue** modulo  $n$ .

### 3.4.5 Problems

- P3.4.1 Show that there are infinitely many primes that are congruent to 3 modulo 4. (Hint: Suppose there were a finite list of such primes. Construct a natural that is not divisible by any of them, but is congruent to 3 modulo 4. Could this natural be a product only of the other primes, those congruent to 1 modulo 4?)
- P3.4.2 Show that there are infinitely many primes that are congruent to 5 modulo 6.
- P3.4.3 Problems 3.4.1 and 3.4.2 should make us wonder about the rest of the primes, those congruent to 1 modulo 4 or congruent to 1 modulo 6. Actually a 19th-century theorem of Dirichlet says that any **arithmetic progression**  $a, a + b, a + 2b, a + 3b, \dots$ , with  $a$  and  $b$  relatively prime, contains infinitely many primes. The proof of this is well beyond the scope of this book, but here, with some help from later in the chapter, we can show that there are infinitely many primes congruent to 1 modulo 4:
- (a) If  $S = \{p_1, \dots, p_j\}$  is any set of  $4k + 1$  primes, let  $k = 4 \cdot (p_1^2 \cdot \dots \cdot p_j^2) + 1$ . Argue that  $k$  must have a prime factor  $p$  that is not among the  $p_i$ 's.
  - (b) Prove that  $k - 1$  is congruent to  $-1$  modulo  $p$ , and is a perfect square modulo  $p$ .
  - (c) Using the result referred to in Exercise 3.4.8, prove that there is a  $4k + 1$  prime that is not in  $S$ .
- P3.4.4 Assuming the fact claimed in Exercise 3.4.9, prove that there are infinitely many primes of the form  $6n + 1$ . (**Hint:** Given a finite set of such primes, construct a number  $k$  not divisible by any of them such that  $-3$  is a perfect square modulo any prime dividing  $k$ .)
- P3.4.5 Of the naturals less than 2, exactly half are relatively prime to 2 (0 is not, 1 is). Of the naturals less than  $2 \cdot 3 = 6$ , two are relatively prime to 6 (1 and 5) and the others are not, so the fraction that are is  $1/3$ . Of the naturals less than  $2 \cdot 3 \cdot 5 = 30$ , exactly eight (1, 7, 11, 13, 17, 19, 23, and 29) are relatively prime to 30, a  $4/15$  fraction. These fractions follow a pattern:  $1/2$  is  $2^0/(1 \cdot 2)$ ,  $1/3$  is  $2^1/(2 \cdot 3)$ , and  $4/15$  is  $2^2/(3 \cdot 5)$ . This naturally leads to a conjecture: If  $n$  is the product of the first  $k$  primes, the fraction of the naturals less than  $n$  that are relatively prime to  $n$  is  $2^{k-1}/(p_{k-1} \cdot p_k)$ , where  $p_{k-1}$  and  $p_k$  are the  $(k-1)$ 'st and  $k$ 'th primes respectively. Investigate this conjecture for larger  $k$ . Can you prove or disprove it?
- P3.4.6 A **Fermat number** is a natural of the form  $F_i = 2^{2^i} + 1$ , where  $i$  is any natural. In 1730 Goldbach used Fermat numbers to give an alternate proof that there are infinitely many primes.
- (a) List the Fermat numbers  $F_0, F_1, F_2, F_3$ , and  $F_4$ .
  - (b) Prove that for any  $n$ , the product  $F_0 \cdot F_1 \cdot \dots \cdot F_n$  is equal to  $F_{n+1} - 2$ .
  - (c) Argue that no two different Fermat numbers can share a prime factor. Since there are infinitely many Fermat numbers, there must thus be infinitely many primes.
- P3.4.7 Here is yet another proof that there are infinitely many primes, due to Filip Saidak in 2006.
- (a) Let  $n$  be any natural with  $n > 1$ . Argue that  $N_2 = n(n+1)$  must have at least two different prime factors.

- (b) Define  $N_3 = N_2(N_2 + 1)$ . Argue that  $N_3$  must have at least three different prime factors ( $N_2$ 's two, plus at least one more).
- (c) Continue the argument to show that for any number  $k$ , there must be a natural with at least  $k$  different prime factors, and hence that there must be infinitely many primes.

P3.4.8 Using the result of Exercise 3.4.10, we can get one more proof that there are infinitely many primes. Suppose that for any  $n$ , the number  $r$  of primes that are  $\leq 2^n$  was bounded by some fixed number  $c$ . Show that the function given by prime factorization cannot be one-to-one if  $n$  is sufficiently large.

P3.4.9 Let  $r(n)$  be the number of primes that are less than or equal to  $2^n$ . A natural question, once we know that  $r(n)$  is unbounded, is to estimate how fast it grows with  $n$ . The **Prime Number Theorem** says that is proportional to  $2^n/n$ , but proving that is beyond us here. What can we show given the result of Exercise 3.4.10? That is, how large must  $r(n)$  be to allow the function from  $\{1, \dots, 2^n\}$  to  $\{0, 1, \dots, n\}^r$  to be one-to-one?

P3.4.10 Here is an argument that gets a better lower bound on the function  $r(n)$  from Problem 3.4.9, though it uses an assumption that we are not yet able to prove. Consider finding all the primes less than  $2^n$  with a Sieve of Eratosthenes. We begin with  $2^n$  numbers. Removing multiples of 2 eliminates  $1/2$  of them. Removing multiples of 3 removes  $1/3$  of them. Our assumption will be that it removes  $1/3$  of *the numbers remaining* after the multiples of 2 have been removed. Then we will assume that removing multiples of 5 eliminates  $1/5$  of those remaining, and so forth. We know that once we have eliminated all multiples of primes that are at most  $\sqrt{2^n} = 2^{n/2}$ , the numbers remaining are prime.

- (a) Given our assumptions, explain why the eventual fraction of numbers remaining is more than  $(1/2)(2/3)(3/4) \dots ((2^{n/2} - 1)/2^{n/2})$ .
- (b) Explain why the result of part (a) implies that  $r(n) \geq 2^{n/2}$ .

| JULY 2019 |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|
| S         | M  | T  | W  | T  | F  | S  |
|           | 1  | 2  | 3  | 4  | 5  | 6  |
| 7         | 8  | 9  | 10 | 11 | 12 | 13 |
| 14        | 15 | 16 | 17 | 18 | 19 | 20 |
| 21        | 22 | 23 | 24 | 25 | 26 | 27 |
| 28        | 29 | 30 | 31 |    |    |    |

| AUGUST 2019 |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|
| S           | M  | T  | W  | T  | F  | S  |
|             |    |    |    | 1  | 2  | 3  |
| 4           | 5  | 6  | 7  | 8  | 9  | 10 |
| 11          | 12 | 13 | 14 | 15 | 16 | 17 |
| 18          | 19 | 20 | 21 | 22 | 23 | 24 |
| 25          | 26 | 27 | 28 | 29 | 30 | 31 |

©Kendall Hunt Publishing Company

Figure 3-4: Pill days are circled — pill-Wednesdays come every five weeks.

### 3.5 The Chinese Remainder Theorem

#### 3.5.1 Two Congruences With Different Moduli

Suppose that you have to take a pill every five days. How often will you take your pill on a Wednesday? This is an example of two *interacting periodic systems* that can be analyzed using number theory. Assign a natural to each day, perhaps by taking the days since some arbitrary starting point<sup>29</sup>, and notice that our two conditions can be described by modular arithmetic. Day number  $x$  is a Wednesday if and only if  $x \equiv c \pmod{7}$ , where  $c$  depends on the day of the week of our starting point. Similarly, day number  $x$  is a pill day if and only if  $x \equiv d \pmod{5}$ , where again  $d$  depends on the starting point. The numbers of the days that are *both* Wednesdays and pill days will be those naturals  $x$  that satisfy *both* of these congruences.

We've seen how to work with more than one congruence with the *same base*, but here we have two congruences with different bases. How do we solve such a **system of congruences**? A bit of playing around with the above example (see Figure 3-4) will show that the special days appear to occur exactly every 35 days, and this is an instance of a general phenomenon first noted in ancient China<sup>30</sup>:

**The Chinese Remainder Theorem (Simple Form):** If  $m$  and  $n$  are relatively prime, then the two congruences  $x \equiv a \pmod{m}$  and  $x \equiv b \pmod{n}$  are satisfied if and only if  $x \equiv c \pmod{mn}$ , where  $c$  is a natural depending on  $a$ ,  $b$ ,  $m$ , and  $n$ .

<sup>29</sup>Astronomers, for example, start counting with 1 January 4713 B.C., the start of the “Julian Period”.

<sup>30</sup>The problem is solved in *Master Sun's Mathematical Manual* from the 3rd century C.E. (for an example with the three moduli 3, 5, and 7), and by the fifth-century Indian mathematician-astronomer Aryabhata. The earliest known detailed general solution is by the Chinese mathematician Qin Jiushao in 1247.

We'll soon prove this simple form of the theorem and then move to the full statement of it, involving more than two congruences. But first, why do we need the part about  $m$  and  $n$  being relatively prime? If we don't have it, the conclusion might be false, as in the example of the two congruences  $x \equiv 1 \pmod{4}$  and  $x \equiv 4 \pmod{6}$  which have *no* solution (Why not?). In Problem 3.5.3 we'll look at how to solve an arbitrary system of congruences (or determine that it has no solution) by converting it into a system where the bases are relatively prime.

**Proof of the Simple Form of the CRT:** We need to show that  $x \equiv a \pmod{m}$  and  $x \equiv b \pmod{n}$  if and only if  $x \equiv c \pmod{mn}$ , which means that we need to first define  $c$  and then show both halves of a logical equivalence. Our main technical tool will be the Inverse Theorem, which tells us (since  $m$  and  $n$  are relatively prime) that there are two integers  $y$  and  $z$  such that  $ym + zn = 1$ . This implies both  $ym \equiv 1 \pmod{n}$  ( $y$  is the inverse of  $m$  modulo  $n$ ) and  $zn \equiv 1 \pmod{m}$  ( $z$  is the inverse of  $n$  modulo  $m$ ). To construct  $c$ , we'll use these congruences and our facts about multiplying and adding congruences over a single base.

To get something congruent to  $a$  modulo  $m$ , we can multiply both sides of the congruence  $zn \equiv 1 \pmod{m}$  by  $a$  to get  $azn \equiv a \pmod{m}$ . (If we like, we can think of this as multiplying by the congruence  $a \equiv a \pmod{m}$ .) Similarly, multiplying the other congruence by  $b$  gives us  $bym \equiv b \pmod{n}$ . Now we can notice that the left-hand sides of each of these congruences are congruent to 0 modulo the other base. So if we add the congruence  $bym \equiv 0 \pmod{m}$  to  $azn \equiv a \pmod{m}$ , we get  $azn + bym \equiv a \pmod{m}$ , and similarly we can get  $azn + bym \equiv b \pmod{n}$ . Setting  $c$  to be  $azn + bym$ , then, we have a solution to both congruences. Furthermore, as long as  $x \equiv c \pmod{mn}$ ,  $x$  is equal to  $c + kmn$  for some integer  $k$ , and since both  $m$  and  $n$  divide  $kmn$  we know that  $x$  will satisfy both congruences as well.

It remains to show that if  $x$  satisfies both  $x \equiv a \pmod{m}$  and  $x \equiv b \pmod{n}$ , then it satisfies  $x \equiv c \pmod{mn}$  as well. Let  $d = x - c$ . It's easy to see that  $d = x - azn - bym$  is divisible by both  $m$  and  $n$ , using the arithmetic above. We need to show that  $d$  is divisible by  $mn$ , using the fact that  $m$  and  $n$  are relatively prime. If  $d = 0$  this is trivially true — if not we may run the Euclidean Algorithm<sup>31</sup> to find the greatest common divisor of  $d$  and  $mn$ , which we'll name  $q$ . This  $q$  must be a common multiple of  $m$  and  $n$  because both these numbers divide both  $d$  and  $mn$ , and the Euclidean Algorithm will preserve this property. But by Problem 3.1.5, because  $m$  and  $n$  are relatively prime, we know that  $mn$  is their *least* common multiple, making  $mn = q$  the only choice. Since  $q$  divides  $d$ , we are done —  $x$  and  $c$  are congruent modulo  $mn$ . ♠

**Example:** Suppose that we need to solve the two congruences  $x \equiv 4 \pmod{15}$  and  $x \equiv 8 \pmod{16}$ . Since  $m = 15$  and  $n = 16$  are relatively prime, the Chinese Remainder Theorem tells us that the solution will be of the form  $x \equiv c \pmod{240}$ . In small cases, it's often easiest to find  $c$  by trial and error — in this example checking 8, 24, 40, and so on until we run into a number that is congruent to 4 modulo 15. But let's go through the general solution method. We have a formula for  $c$ , but it requires the inverses of 15 and 16 modulo each other ( $y$  and  $z$  in the expression  $azn + bym$ ). Since  $16 \equiv 1 \pmod{15}$ , we can take  $z = 1$ , and since  $15 \equiv -1 \pmod{16}$  we can take  $y = -1$  or  $y = 15$ . (If we weren't so lucky we'd have to use the Euclidean Algorithm to get the inverses, as in Section 3.3.) This gives us  $c = 4 \cdot 1 \cdot 16 + 8 \cdot (-1) \cdot 15 = 64 - 120 = -56 \equiv 184 \pmod{240}$ .

<sup>31</sup>Actually  $d$  may be negative, but if so we may run the Euclidean Algorithm on  $-d$  and  $mn$ .

### 3.5.2 The Full Version of the Theorem

If we have more than two congruences, the condition on the bases becomes a little more complicated. If any two of the bases have a common factor greater than one, it might be impossible to satisfy those two congruences together, and thus definitely impossible to satisfy the entire system. So to have a solution, we need to have the bases be **pairwise relatively prime**, which means that any two of them are relatively prime to each other.

**The Chinese Remainder Theorem (Full Version):** Let  $m_1, m_2, \dots, m_k$  be a sequence of positive naturals that are pairwise relatively prime. Any system of congruences  $x \equiv a_1 \pmod{m_1}$ ,  $x \equiv a_2 \pmod{m_2}$ ,  $\dots$ ,  $x \equiv a_k \pmod{m_k}$  is equivalent to a single congruence  $x \equiv c \pmod{M}$ , where  $M = m_1 \cdot m_2 \cdot \dots \cdot m_k$  and  $c$  is a natural that depends on the  $a_i$ 's and on the  $m_i$ 's.

**Proof:** If  $m_1, m_2, \dots, m_k$  are pairwise relatively prime, then the number  $m_1 m_2$  must be relatively prime to each of the numbers  $m_3, m_4, \dots, m_k$ . (We'll prove this as Exercise 3.5.1.) So if we apply the simple form of the Chinese Remainder Theorem to the first two congruences, getting a single congruence  $x \equiv b \pmod{m_1 m_2}$ , we are left with a system of  $k - 1$  congruences whose bases are pairwise relatively prime. Similarly, we can combine this new first congruence with the third using the simple form of the theorem, and continue in this way until there is only one congruence left<sup>32</sup>. Because we are multiplying bases each time that we combine congruences, this last congruence has the desired form. And since at each step we replaced a system of congruences by an equivalent system (one which was satisfied by exactly the same values of  $x$ ), the last congruence is equivalent to the original system.

Alternatively, we can calculate  $c$  directly and verify that it works, just as we did for the simple theorem. For each base  $m_i$ , we can calculate an inverse  $n_i$  for the natural  $M/m_i$  modulo  $m_i$ , because this number is relatively prime to  $m_i$ . Then  $a_i n_i (M/m_i)$  is congruent to  $a_i$  modulo  $m_i$ , and congruent to 0 modulo any of the other bases. If we set  $c$  to be  $a_1 n_1 (M/m_1) + a_2 n_2 (M/m_2) + \dots + a_k n_k (M/m_k)$ , then  $c$  satisfies all  $k$  of the congruences in the system. If  $x \equiv c \pmod{M}$ , then  $x - c$  is divisible by each of the bases  $m_i$ , and arguing as in the simple form of the theorem we can show that  $x - c$  must be divisible by  $M$ . ♠

To illustrate the full version of the theorem, let's return to our initial example. Suppose that along with pill days whenever  $x \equiv 3 \pmod{5}$  and Wednesdays whenever  $x \equiv 4 \pmod{7}$ , we now introduce massages every six days, whenever  $x \equiv 0 \pmod{6}$ . The full version of the theorem says that all three events will happen exactly when  $x \equiv c \pmod{210}$ , for some value of  $c$ . To calculate  $c$ , we need the numbers  $m_i$  (5, 6, and 7), the numbers  $a_i$  (3, 0, and 4), the numbers  $M/m_i$  (42, 35, and 30) and the numbers  $n_i$  (the inverse of 42 modulo 5 which is 3, the inverse of 35 modulo 6 which is 5, and the inverse of 30 modulo 7 which is 4). Then

$$\begin{aligned} c &= a_1 n_1 (M/m_1) + a_2 n_2 (M/m_2) + a_3 n_3 (M/m_3) \\ &= 3 \cdot 3 \cdot 42 + 0 \cdot 5 \cdot 35 + 4 \cdot 4 \cdot 30 \\ &= 378 + 480 = 858 \equiv 18 \pmod{210}. \end{aligned}$$

---

<sup>32</sup>This argument is a bit informal because we don't yet have formal techniques to deal with the "... in the statement of the problem — this will be remedied in Chapter 4.

We can easily check that 18 satisfies the given three congruences.

One use of the Chinese Remainder Theorem is a method to store very large naturals on a parallel computer. If you know what  $x$  is congruent to modulo several different large prime numbers (prime numbers are necessarily pairwise relatively prime), the theorem says that this specifies  $x$  modulo the product of those primes. Suppose that  $x$  does not fit on a single machine word, but that each of the remainders (modulo the different primes) does. You can put each of the remainders on a different processor and you have in a sense stored  $x$  in a distributed way. If you want to add or multiply two numbers stored in this way, it can be done in parallel, as each processor can carry out the operation modulo its prime. The only problem is that you have to combine all the remainders in order to find out what the result really is in ordinary notation. But if you have to do lots of parallelizable operations before computing the answer, it might be worthwhile to do all of them in parallel, and convert the answer to ordinary notation at the end.

### 3.5.3 Exercises

- E3.5.1 Prove that if  $m_1, m_2, \dots, m_k$  are pairwise relatively prime, then  $m_1 m_2$  is relatively prime to each of the numbers  $m_3, m_4, \dots, m_k$ .
- E3.5.2 Find a single congruence that is satisfied if and only if  $x \equiv 9 \pmod{11}$ ,  $x \equiv 6 \pmod{12}$ , and  $x \equiv 3 \pmod{13}$ .
- E3.5.3 Here are three systems of congruences where the bases are *not* pairwise relatively prime. You are to find all solutions to each system, or show that no solution exists. (Hint: What do the conditions say about whether  $x$  is even or odd?)
- (a)  $x \equiv 5 \pmod{6}$ ,  $x \equiv 7 \pmod{8}$ ,  $x \equiv 3 \pmod{10}$ .
  - (b)  $x \equiv 11 \pmod{12}$ ,  $x \equiv 9 \pmod{14}$ ,  $x \equiv 5 \pmod{16}$ .
  - (c)  $x \equiv 7 \pmod{9}$ ,  $x \equiv 4 \pmod{10}$ ,  $x \equiv 10 \pmod{12}$ .
- E3.5.4 Suppose two integers  $x$  and  $y$  satisfy the congruences  $x \equiv 4 \pmod{7}$ ,  $y \equiv 2 \pmod{7}$ ,  $x \equiv 3 \pmod{8}$ ,  $y \equiv 1 \pmod{8}$ ,  $x \equiv 7 \pmod{9}$ , and  $y \equiv 5 \pmod{9}$ . What are the residues of  $xy$  modulo 7, 8, and 9? Find a number  $z$  less than 504 such that  $xyz \equiv 1 \pmod{504}$ . (Hint: Find the residues of  $z$  modulo 7, 8, and 9 first, and you need carry out the Chinese Remainder Theorem process only once.)
- E3.5.5 We say that three naturals  $a$ ,  $b$ , and  $c$  are **relatively prime** if there does not exist a single number  $d > 1$  that divides all three. Give an example of three naturals that are relatively prime, but not pairwise relatively prime.
- E3.5.6 About a thousand soldiers are marching down a road, and their commander would like to know exactly how many there are. She orders them to line up in rows of seven, and learns that there are six left over. She then orders them to line up in rows of eight, and there are seven left over. Finally, she orders them into rows of nine, and there are three left over. How many soldiers are in the group?
- E3.5.7 Someone on the internet, calling himself Mr. Rabbit, has agreed to sell me a file of government secrets for \$100. However, Rabbit will accept payment only in one of two obscure

cryptocurrencies, Batcoins (currently worth \$51 each) and Twitcoins (currently worth \$32 each). For technical reasons, Batcoins and Twitcoins cannot be broken into fractions like Bitcoins – they must be transferred entirely or not at all. Both Rabbit and I have plenty of each kind of coin available. How can I pay Rabbit exactly \$100 by transferring integer numbers of Batcoins and/or Twitcoins from me to Rabbit and/or from Rabbit to me?

- E3.5.8 Mr. Lear, an elderly man with three daughters, is making arrangements for his retirement. His bank accounts are accessed by a four-digit code, which we may think of as a natural less than 10000. He gives each of his daughters partial information about  $x$ , so that none of them can determine  $x$  on her own. He tells Cordelia the remainder  $x \% 97$  from dividing  $x$  by 97. He tells Goneril  $x \% 115$ , and tells Regan  $x \% 119$ . Explain why any two of the daughters, by combining their information, can determine  $x$ .
- E3.5.9 Let  $p$ , and  $q$ , be two pairwise relatively prime naturals, each greater than 1. Let  $f$  be the function from  $\{0, 1, \dots, pq - 1\}$  to the set  $\{0, 1, \dots, p - 1\} \times \{0, 1, \dots, q - 1\}$  defined by  $f(x) = (x \% p, x \% q)$ . Prove that  $f$  is a bijection.
- E3.5.10 Let  $n$  and  $a$  be positive naturals. Prove that  $a$  has a multiplicative inverse modulo  $n$  if and only if for every prime  $p$  dividing  $n$ ,  $a$  has an inverse modulo  $p$ .

### 3.5.4 Problems

- P3.5.1 The **Julian calendar**<sup>33</sup> has years of 365 days unless the year number satisfies  $x \equiv 0 \pmod{4}$ , in which case the year has 366 days (a “leap year”).
- George Washington was born on 11 February 1732, a Friday, according to the Julian calendar. Explain why 11 February in year  $x$  of the Julian calendar is a Friday, if  $x \equiv 1732 \pmod{28}$ . (Note that this is not just a straightforward application of the Chinese Remainder Theorem).
  - What day of the week was 11 February 1492, according to the Julian calendar? Explain your reasoning.
  - A “perpetual calendar” is a single chart including all possible calendars for a given year. How many calendars are needed? Explain how to determine which calendar is needed for year  $x$ , if you know a congruence for  $x$  modulo 28.
- P3.5.2 The **Gregorian calendar** (the one in most general use today)<sup>34</sup> is the same as the Julian calendar except that there are 365 days in year  $x$  if  $x$  is congruent to 100, 200, or 300 modulo 400.
- In the Gregorian calendar, as students of World War II may recall, 7 December 1941 was a Sunday. We cannot, as in the case of the Julian calendar, guarantee that 7 December of

<sup>33</sup>Actually no relation to the Julian Period mentioned above — the calendar was devised by Julius Caesar and the Period was named by its inventor, Joseph Justus Scaliger, after his father, who happened to be named Julius as well. The starting date for the Period, 1 January 4713 B.C., was chosen so that three cycles, of 28, 19, and 15 years respectively, were all in their desired starting positions. How often does this happen?

<sup>34</sup>Great Britain and its colonies switched from the Julian to Gregorian calendar in 1752, when they were considerably out of step with each other — to see how this was implemented enter `cal 1752` on any Unix machine. George Washington, who was alive at the time of this change, retroactively changed his birthday to 22 February.



year  $x$  was a Sunday if  $x \equiv 1941 \pmod{28}$ , but we can guarantee it if  $x \equiv 1941 \pmod{c}$  for some value of  $c$ . Find the smallest value of  $c$  for which this is true.

- (b) Determine the day of the week on which you were born, using only the fact that 7 December 1941 was a Sunday. Show all of your reasoning.
- (c) What additional complications arise when designing a perpetual Gregorian calendar?
- (d) In what years, during the period from 1 to 1941 A.D. (or 1 to 1941 C.E.), have the Gregorian and Julian calendars agreed for the entire year?

P3.5.3 Suppose we are given a system of congruences:

$$\begin{aligned}x &\equiv a_1 \pmod{m_1} \\x &\equiv a_2 \pmod{m_2} \\&\dots \\x &\equiv a_k \pmod{m_k},\end{aligned}$$

*without* any guarantee that the  $m_i$ 's are pairwise relatively prime.

- (a) A **prime power** is a number that can be written  $p^e$  for some prime number  $p$  and some positive number  $e$ . A consequence of the Fundamental Theorem of Arithmetic (which we'll prove soon) is that any number has a unique factorization into prime powers. Show that we can convert any congruence into an equivalent system of congruences where each base is a prime power and the bases are pairwise relatively prime.
- (b) Let  $p$  be a prime number, and suppose that we have a system of congruences where each base is a power of  $p$ . Explain how to tell whether the system has a solution, and how to find it.
- (c) Using parts (a) and (b), explain how to determine whether an arbitrary system of congruences has a solution, and if so how to find all solutions.

P3.5.4 Suppose that the naturals  $m_1, \dots, m_k$  are pairwise relatively prime and that for each  $i$  from 1 through  $k$ , the natural  $x$  satisfies  $x \equiv x_i \pmod{m_i}$  and the natural  $y$  satisfies  $y \equiv y_i \pmod{m_i}$ . Explain why for each  $i$ ,  $xy$  satisfies  $xy \equiv x_i y_i \pmod{m_i}$  and  $x + y$  satisfies  $(x + y) \equiv (x_i + y_i) \pmod{m_i}$ . Now suppose that  $z_1, \dots, z_j$  are some naturals and that we have an arithmetic expression in the  $z_i$ 's (a combination of them using sums and products) whose result is guaranteed to be less than  $M$ , the product of the  $m_i$ 's. Explain how we can compute the exact result of this arithmetic expression using the Chinese Remainder Theorem only once, no matter how large  $j$  is.

P3.5.5 (uses Java) Write a real-Java static method that takes three moduli  $m_1, m_2, m_3$  and three residues  $x_1, x_2, x_3$  as input. It should check that the moduli are pairwise relatively prime, and if they are, it should output a number  $x$  that satisfies all three congruences  $x \equiv x_i \pmod{m_i}$ .

P3.5.6 In Problem 3.3.2 we defined the **Euler totient function**  $\phi(n)$ , where  $n$  is a natural, to be the number of naturals in the set  $\{0, 1, \dots, n\}$  that are relatively prime to  $n$ . The Chinese Remainder Theorem allows us to calculate  $\phi(n)$  for any  $n$  with a little work:

- (a) Prove that if  $p$  is any prime and  $e$  is any positive natural, then  $\phi(p^e) = p^e - p^{e-1} = p^{e-1}(p - 1)$ .

- (b) Prove that if  $r$  and  $s$  are any relatively prime naturals each greater than 1, then  $\phi(rs) = \phi(r)\phi(s)$ . (**Hint:** Use the bijection of Exercise 3.5.9.)
- (c) Combine (a) and (b) to get a rule to compute  $\phi(n)$  for any natural  $n$ . Illustrate your method by finding  $\phi(52500)$ .

P3.5.7 Following Exercise 3.5.9, let  $p_1, \dots, p_k$  be a pairwise relatively prime set of naturals, each greater than 1. Let  $X$  be the set  $\{0, 1, \dots, p_1 - 1\} \times \dots \times \{0, 1, \dots, p_k - 1\}$ . Define a function  $f$  from  $\{0, 1, \dots, p_1 p_2 \dots p_k - 1\}$  to  $X$  by the rule  $f(x) = \langle x \% p_1, \dots, x \% p_k \rangle$ . Prove that  $f$  is a bijection.

P3.5.8 Let  $X$  be a finite set and let  $f$  be a bijection on  $X$ . Recall that the  $n$ 'th **iterate** of  $f$ , written  $f^{(n)}$ , is the function defined so that  $f^{(n)}(x)$  is the result of applying  $f$  to  $x$  exactly  $n$  times. We define the **period** of  $f$  to be the smallest positive natural  $n$  such that  $f^{(n)}$  is the identity function.

- (a) Why must every  $f$  have a period?
- (b) Show that if  $X$  has exactly three elements, every bijection on  $X$  has period 1, 2, or 3.
- (c) How large must  $X$  be before you can have a bijection with period 6?

P3.5.9 (harder) Following Problem 3.5.8, let  $m(n)$  be the largest period of any bijection on  $X$  if  $X$  has exactly  $n$  elements.

- (a) Let  $p_1, p_2, \dots, p_k$  be pairwise relatively prime naturals with  $p_1 + \dots + p_k \leq n$ . Show that there is a bijection of period  $p_1 p_2 \dots p_k$  on  $X$ .
- (b) Let  $f$  be any bijection on  $X$ . Show that there is a set of numbers as in part (a) so that the period of  $f$  is  $p_1 \dots p_k$ .
- (c) Using this analysis, find  $m(n)$  for all  $n$  with  $n \leq 20$ .

P3.5.10 Let  $m$  and  $n$  be two relatively prime positive naturals, and consider what naturals can be expressed as linear combinations  $am + bn$  where  $a$  and  $b$  are *naturals*, not just integers.

- (a) Show that if  $m = 2$  and  $n = 3$ , any natural except 0 and 1 can be so expressed.
- (b) Determine which naturals can be expressed if  $m = 3$  and  $n = 5$ .
- (c) Argue that for any  $m$  and  $n$ , there are only a finite number of naturals that cannot be expressed in this way.

## 3.6 The Fundamental Theorem of Arithmetic

### 3.6.1 Unique Factorization Into Primes

In this section we will prove one of the most important results of number theory.

**Fundamental Theorem of Arithmetic:** Every positive natural has a *unique* factorization into a product of prime numbers.

Recall that a factorization of  $n$  is simply a list of prime numbers whose product is  $n$ . Here the word “unique” means that if we have two prime factorizations of the same natural, such as  $2 \cdot 5 \cdot 2 \cdot 3 = 3 \cdot 2 \cdot 2 \cdot 5 = 60$ , then they contain the same primes, and each prime occurs the same number of times in each factorization.

You’ve probably been told at some time by your teachers that this fact is true, and perhaps you’ve taken it on faith. Now, however, we have developed enough mathematical machinery to *prove* it, using only the definitions and simple facts about arithmetic<sup>35</sup>. For that matter, what we do here will be fairly simple, since most of the work occurred when we proved the Inverse Theorem in Section 3.3.

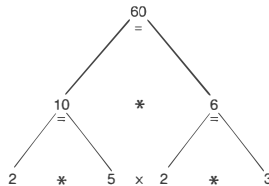
### 3.6.2 Existence of a Factorization

The first thing to prove is that *at least one* factorization exists. We’ve already argued that this is true, but let’s review the reasoning. If a natural  $x$  is prime, then it has a prime factorization containing one prime,  $x$  itself. (And 1 has a prime factorization as the product of *no* primes. We don’t worry about factoring 0.) Otherwise,  $x$  is composite and can be written  $x = a \cdot b$  where both  $a$  and  $b$  are greater than 1 and less than  $x$ . If  $a$  and  $b$  are both prime, we are done. Otherwise we write each of *them* as the product of smaller numbers, and so on until we have expressed  $x$  as a product consisting only of primes. (Figure 3-5 shows one way this process can result in a factorization of 60 into primes.)

We have in effect just described a *recursive algorithm* for producing a prime factorization (Problem 3.6.1 is to code this algorithm in Java). If we believe that this algorithm will always give us an answer, then a prime factorization must exist. The fundamental reason why the algorithm can’t go into an infinite loop is that it is acting on naturals that are always getting smaller (if it factors  $x$  as  $a \cdot b$ , both  $a$  and  $b$  must be smaller than  $x$ ) and a sequence of naturals can’t go on forever with its elements always getting smaller. But to make this reasoning completely rigorous, we’ll need the more formal tools to be developed in Chapter 4.

---

<sup>35</sup>Actually, a fully formal proof will require mathematical induction from Chapter 4. But this proof should be reasonably convincing and satisfying — the proof will tell us both that it is true and *why* it is true.



©Kendall Hunt Publishing Company

Figure 3-5: A factorization of 60.

### 3.6.3 The Uniqueness of the Factorization

The other half of the Fundamental Theorem is to show that the prime factorization is unique, which will require us to use the Inverse Theorem. How do we *know*, for example, that  $17 \cdot 19 \cdot 23 \cdot 29$  and  $3 \cdot 5^3 \cdot 7 \cdot 83$ , both odd numbers around 200000, are not equal? In this case we can multiply out the products and find that 215441 doesn't equal 217875, but in general how do we know that two *different* products of primes (not just reorderings of the same product) can't be equal to each other?

There are several ways to phrase this argument, and we're going to do this one as an *argument by contradiction*. We'll assume the negation of what we want to prove, that there are two different products of primes,  $b_1 \cdot \dots \cdot b_i$  and  $c_1 \cdot \dots \cdot c_j$ , that multiply to the same number  $a$ . For the two products to be "different", we need to assume that some prime  $p$  occurs among the  $b$ 's more times that it occurs among the  $c$ 's<sup>36</sup>.

Now we have assumed a situation which is impossible, and our job is to prove that it is impossible. The way to do this is to manipulate it until we derive another situation that we can prove to be impossible. (This can be tricky, as we have to separate what we know to be false from what we can prove to be false, to avoid constructing an invalid circular proof.) If we can show  $\neg p \rightarrow 0$ , where  $p$  is the proposition we want to prove, we will have completed a proof of  $p$  by contradiction.

How could we have told above that the products  $17 \cdot 19 \cdot 23 \cdot 29$  and  $3 \cdot 5^3 \cdot 7 \cdot 83$  are not equal, without multiplying them out? One natural answer is that 3, for example, divides the second product but not the first, so that (as in Excursion 3.2) we know that the sum of the decimal digits of the first product is not divisible by 3 and that of the second product is. This sounds convincing, but there's a problem. It's clear that 3 divides the second product, but the fact that it *doesn't* divide the first product is something we have to prove (the reason it seems obvious is that we believe that unique factorization is true).

<sup>36</sup>Of course, it could occur fewer times among the  $b$ 's. But in that case we'll rename the  $b$ 's as the  $c$ 's, and rename the  $c$ 's as the  $b$ 's, so that the  $b$ 's have more occurrences of  $p$ . There's no reason we can't do this as we started with the same assumptions about each product — that it multiplied to  $a$  and that all the terms in it were prime. This sort of argument is often stated "**Without loss of generality**, let the  $b$ 's have more occurrences of  $p$ ...".

### 3.6.4 The Atomicity Lemma

The word “atom” comes from the Greek for “indivisible”. The result we need is that primes are **atomic** – a prime can divide a product only by dividing one factor or the other:

**Atomicity Lemma, Simple Version:** Let  $p$  be a prime number, let  $a$  and  $b$  be any two naturals, and suppose that  $p$  divides  $ab$ . Then either  $p$  divides  $a$  or  $p$  divides  $b$ , or both.

**Proof:** Just to illustrate the variety of possible proof methods, let’s try an indirect proof, assuming that  $p$  divides neither  $a$  nor  $b$  and proving that it doesn’t divide  $ab$ . By the contrapositive rule, this will prove the Lemma. Since  $p$  is prime and doesn’t divide  $a$ , it is relatively prime to  $a$  (the only divisors of  $p$  are 1 and  $p$ , and  $p$  isn’t a divisor of  $a$ , so 1 is the greatest common divisor). By the Inverse Theorem, there is some number  $c$  such that  $ac \equiv 1 \pmod{p}$ . Similarly,  $p$  and  $b$  are relatively prime, and there is some number  $d$  such that  $bd \equiv 1 \pmod{p}$ . Multiplying the two congruences,  $abcd \equiv 1 \pmod{p}$ . But if  $p$  divided  $ab$ , then we would have  $ab \equiv 0 \pmod{p}$  and thus  $abcd \equiv (ab)(cd) \equiv 0(cd) \equiv 0 \pmod{p}$ . So  $p$  cannot divide  $ab$ , and we are done. ♠

As with the Chinese Remainder Theorem, the Atomicity Theorem has a natural extension to products of more than two factors, and it is this version we will need:

**Atomicity Lemma, Full Version:** Let  $p$  be a prime number and let  $c_1, \dots, c_n$  be any sequence of naturals. If  $p$  divides the product  $c_1 \cdot c_2 \cdot \dots \cdot c_n$ , then  $p$  divides at least one of the  $c_i$ ’s.

**Proof:** Since  $p$  divides  $c_1 \cdot c_2 \cdot \dots \cdot c_n = c_1 \cdot (c_2 \cdot \dots \cdot c_n)$ , by the simple form of the lemma it divides either  $c_1$  or  $c_2 \cdot \dots \cdot c_n$ . If  $p$  divides  $c_1$  we are done, and otherwise it must (by the simple form again) divide either  $c_2$  or  $c_3 \cdot \dots \cdot c_n$ . If it divides  $c_2$  we are done, if not it must either divide  $c_3$  or  $c_4 \cdot \dots \cdot c_n$ , and so on. Either it divides one of the earlier  $c_i$ ’s, or eventually we find that it must divide  $c_{n-1}$  or  $c_n$ . ♠

### 3.6.5 Finishing the Proof

The full version of the Atomicity Lemma tells us that we cannot have a prime  $p$  on one side of our equation, among the  $b$ ’s, and not on the other side among the  $c$ ’s. This is because  $p$  divides the left-hand side, which is equal to the right-hand side, and thus must divide one of the  $c_i$ ’s. But each of the  $c_i$ ’s is prime, and the prime  $p$  cannot divide another prime unless it is equal to  $p$  (since a prime’s only divisors are itself and 1).

We are almost finished — we have shown that for  $p$  to occur in only one of the products gives us a contradiction. What if it occurs in both, but occurs more times among the  $b$ ’s? The obvious thing to do is to *cancel* the  $p$ ’s among the  $c$ ’s with an equal number among the  $b$ ’s, leaving an equation with  $p$ ’s on only one side and thus a contradiction. Is such cancelling legitimate? It requires only the fact that if two naturals are equal, and we divide each by a third natural (that isn’t zero), the results are equal. This is quite true, and we’ll verify it in Exercise 3.6.3.

In abstract algebra, one considers a variety of mathematical systems in which one can multiply, and

unique factorization is one property that these systems might or might not have. For example, the integers have unique factorization as long as you don't worry about distinguishing, say,  $(-3) \cdot (-2)$  and  $3 \cdot 2$  as separate factorizations — every nonzero integer<sup>37</sup> is either a natural or  $-1$  times a natural, so it is a product of prime numbers with  $-1$  possibly multiplied in. In the real numbers, on the other hand, the concept of “prime” doesn't make sense because any non-zero real number is “divisible” by any other. We'll look at some more examples in the Problems.

### 3.6.6 Exercises

- E3.6.1 Rephrase the argument of this section as a direct proof — assume that two products of primes multiply to the same number and prove that they contain the same primes and the same number of each prime. (Hint: Show that the smallest prime in each product is the same, divide both sides of the equation by that prime, and proceed recursively on the remaining product until both sides equal 1.)
- E3.6.2 Suppose you are given two very long sequences of prime numbers. How would you test whether they represent the same factorization, that is, that one sequence is a reordering of the other?
- E3.6.3 Prove that it is not possible to divide the same number by the same number and get two different answers. That is, if  $a$  and  $b$  satisfy  $ad = bd$  and  $d > 0$ , then  $a = b$ . (**Hint:** Suppose  $a = b + c$  with  $c > 0$  and derive a contradiction. What simpler basic properties of numbers do you need?)
- E3.6.4 Prove formally and carefully, using the definition of the primality predicate  $P(x)$  in terms of the division relation  $D(x, y)$ , that  $\forall a : \forall b : (D(a, b) \wedge P(a) \wedge P(b)) \rightarrow (a = b)$ .
- E3.6.5 A positive **rational** number is defined to be a number of the form  $a/b$  where  $a$  and  $b$  are positive naturals. Explain why every positive rational number has a factorization, where each factor is either a prime number  $p$  or the reciprocal of a prime number,  $1/p$ . Is the factorization into factors of this kind unique?
- E3.6.6 Let  $r$  be any integer. If  $r$  is positive,  $\sqrt{r}$  is a real number, and if  $r$  is negative,  $\sqrt{r}$  is an imaginary number. In either case, we can consider the set  $\mathbf{Z}[\sqrt{r}]$  of numbers that can be written as  $a + b\sqrt{r}$  for integers  $a$  and  $b$ .
- Show that if  $x = a + b\sqrt{r}$  and  $y = c + d\sqrt{r}$  are two numbers in  $\mathbf{Z}[\sqrt{r}]$ , then the numbers  $x + y$  and  $xy$  are both in  $\mathbf{Z}[\sqrt{r}]$ .
  - We define the **norm**  $n(x)$  of a number  $x = a + b\sqrt{r}$  to be  $a^2 - rb^2$ . Show that for any numbers  $x$  and  $y$  in  $\mathbf{Z}[\sqrt{r}]$ ,  $n(xy) = n(x)n(y)$ .
- E3.6.7 In this problem we will work with subsets of some nonempty finite set  $X$ , and let the union operation be our “multiplication”. We will say that and say that one subset  $Y$  “divides” another subset  $Z$ , written  $D(Y, Z)$  if  $\exists W : W \cup Y = Z$ . We will define a subset  $Y$  to be “prime” if  $Y \neq \emptyset$  and if  $D(Z, Y)$  is true, then either  $Z = \emptyset$  or  $Z = Y$ .
- Show that  $D(Y, Z)$  is true if and only if  $Y \subseteq Z$ .
  - Which subsets are “prime” according to this definition?

<sup>37</sup>Zero is a special case in any system, and we don't worry about factoring it.

- (c) Explain why any subset “factors” uniquely as a product of “primes”.
  - (d) Prove a version of the Atomicity Lemma for these “primes”.
  - (e) Show that any nonempty set that is not “prime” fails to satisfy the Atomicity Lemma, even in the simple version.
- E3.6.8 Let  $p$  be a prime number and consider the set  $\{0, 1, \dots, p-1\}$  of integers modulo  $p$ . Using multiplication modulo  $p$ , we can define a division relation  $D(x, y) \leftrightarrow \exists z : xz = y$  and use this to define a notion of “prime number”. Which numbers divide which other numbers, and what are the “primes”? Does every nonzero number have a unique factorization into “primes”?
- E3.6.9 If  $t$  is any natural, we can define the **threshold- $t$  numbers** to be the set  $\{0, 1, \dots, t\}$  where the multiplication operation makes  $xy$  either the natural  $xy$ , if that is in the set, or  $t$  if it is not. (The rabbits of Robert Adams’ *Watership Down* use threshold-5 numbers.) We define the division relation according to this multiplication, so that  $D(x, y)$  means  $\exists z : xz = y$ . What are the prime numbers in this version of arithmetic? Does every nonzero number factor uniquely into primes?
- E3.6.10 (uses Java) In the game of Kenken, each square contains a one-digit number and you are sometimes told the product of the numbers in a particular group of squares. Define a **Kenken number** to be a natural that can be written as a product of one-digit numbers. Write a static real-Java method that inputs a `long` argument and returns a `boolean` telling whether the input is a Kenken number. Does your method run quickly on any possible `long` input?

### 3.6.7 Problems

- P3.6.1 (uses Java) In Section 3.1 we gave a pseudo-Java method that input a `natural` argument and printed its prime factorization.
- (a) Write a recursive program in real Java that inputs a positive `int` or `long` value and prints a sequence of prime numbers that multiply together to give the input.
  - (b) By trial and error, determine the largest prime number you can factor in ten seconds of computer time. (Primes would appear to be the worst case for this algorithm.)
- P3.6.2 Prove that any positive natural factors uniquely as a product of prime powers, where the prime powers are pairwise relatively prime (as defined in Section 3.5).
- P3.6.3 In Exercise 3.3.9 and Problem 3.3.7 we considered the set of polynomials in one variable  $x$  with real number coefficients. These include the **monic** polynomials, whose highest-degree coefficient is 1. We said that a polynomial  $f(x)$  **divides** a polynomial  $g(x)$  if and only if there is a polynomial  $h(x)$  such that  $f(x)h(x) = g(x)$ . In Problem 3.3.7 we showed that it is possible to divide a polynomial  $s(x)$  by a polynomial  $p(x)$ , finding polynomials  $q(x)$  and  $r(x)$  such that  $s(x) = p(x)q(x) + r(x)$ , and either  $r(x) = 0$  or the degree of  $r(x)$  is strictly less than the degree of  $p(x)$ .

A monic polynomial is said to be **irreducible** (that is, prime) if it cannot be written as the product of two other monic polynomials, neither of them equal to 1. So  $x^2 + 3x + 2$  is monic

but not irreducible, because it is equal to  $(x+1)(x+2)$ . On the other hand,  $x+c$  is irreducible for any  $c$ , and  $x^2+x+1$  can be shown to be irreducible<sup>38</sup>.

- (a) Let  $f$  and  $g$  be two monic polynomials that have no monic common divisor other than 1. Show that there are polynomials  $h$  and  $k$  such that  $h(x)f(x) + k(x)g(x) = 1$ . (Hint: Adapt the Euclidean Algorithm.)
- (b) Following the reasoning in this section, show that any monic polynomial factors uniquely into monic irreducible polynomials.

**P3.6.4** In this problem we will work with strings over the alphabet  $\{a, b\}$ , and let our “multiplication” be string concatenation. We say that one string  $u$  “divides” another string  $v$  if  $\exists x : \exists y : xuy = v$ , or equivalently, if  $u$  is a substring of  $v$ . As with the naturals, we redefine the word “prime” for strings so that  $P(w)$  means “ $w \neq \lambda$  and if  $D(x, w)$  is true, then either  $x = \lambda$  or  $x = w$ ”.

- (a) What strings are “prime” using this definition?
- (b) Explain why any string factors uniquely as a product (concatenation) of these “primes”.
- (c) Prove a version of the Atomicity Lemma for these “primes”.
- (d) Show that any nonempty string that is *not* one of your “primes” fails to satisfy the Atomicity Lemma, even in the simple version.

**P3.6.5** (requires exposure to complex numbers.) The **Gaussian integers** are the subset of the complex numbers that are of the form  $a + bi$  where  $a$  and  $b$  are integers and  $i$  is the square root of  $-1$ . (This is an example of the sets defined in Exercise 3.6.6, the set  $\mathbf{Z}[\sqrt{-1}]$ .) The notions of division and primality can be made to make sense in this domain as well:

- (a) In Exercise 3.6.6 we defined the **norm** of  $a + bi$  to be  $a^2 + b^2$ . The **length** of a Gaussian integer  $a + bi$  is defined to be  $\sqrt{a^2 + b^2}$ , the square root of the norm. If  $a + bi$  and  $c + di$  are two Gaussian integers, we showed that the norm of their product is the product of their norms. Show that the length of their product is the product of their lengths.
- (b) A **unit** is a Gaussian integer of length 1. What are the units?
- (c) A **prime** is a Gaussian integer whose length is greater than 1 and which cannot be written as the product of two Gaussian integers unless one is a unit. Prove that any nonzero Gaussian integer has at least one factorization as a product of primes times a unit.
- (d) Prove that  $1 + i$  is a prime and that 2 is not.
- (e) Prove that if the norm of a Gaussian integer is a prime number in the usual sense, then the Gaussian integer is a prime.

**P3.6.6** (requires exposure to complex numbers) In the complex numbers, 1 has exactly three cube roots, 1 itself,  $\omega = (-1 + \sqrt{-3})/2$ , and  $\omega^2 = (-1 - \sqrt{-3})/2$ . The **Eisenstein integers** are the set of complex numbers that can be written  $a + b\omega$  for integers  $a$  and  $b$ .

- (a) Show that the sum and product of two Eisenstein integers are each Eisenstein integers.
- (b) Show that the Eisenstein integers are a proper superset of set of the set called  $\mathbf{Z}[\sqrt{-3}]$  in Exercise 3.6.6.

---

<sup>38</sup>If it had a factor then that factor would have to be of the form  $x - c$ , and if  $x - c$  divided our polynomial then  $c$  would be a root, and this polynomial has no real roots.



- (c) We can find the **length** of an Eisenstein integer as a complex number by writing it as  $x + iy$  and computing  $x^2 + y^2$ . The **norm** of an Eisenstein integer is the square of this length. Find the norm of  $a + b\omega$  in terms of  $a$  and  $b$ .
- (d) A **unit** in the Eisenstein integers is an element of norm 1. Find all the units.
- (e) A **prime** is an Eisenstein integer that is not a unit and which cannot be written as the product of two Eisenstein integers unless one is a unit. Prove that any nonzero Eisenstein integer has at least one factorization as a product of primes times a unit.
- (f) Prove that 2 is still prime but that 3 is not. Is  $3 + \omega$  prime?

P3.6.7 (requires exposure to complex numbers) Consider the set  $R = \mathbf{Z}[\sqrt{-5}]$  as defined in Exercise 3.6.6, and the definition of units and primes as in Problems 3.6.5 and 3.6.6. What are the units in  $R$ ? Show that factorization into primes is *not* unique by giving two different factorizations of 4. (This means showing that all the alleged primes in your factorizations are actually prime in  $R$ .)

P3.6.8 Consider the set  $S = \mathbf{Z}[\sqrt{2}]$  as defined in Exercise 3.6.6, a set of real numbers.

- (a) Recall that a unit is an element with norm 1 – what are the units in  $S$ ?
- (b) Prove that 7 is not prime in  $S$ .
- (c) (harder) Prove that 5 *is* prime in  $S$ .

P3.6.9 Because there are infinitely many primes, we can assign each one a number:  $p_0 = 2$ ,  $p_1 = 3$ ,  $p_2 = 5$ , and so forth. A finite **multiset** of naturals is like an ordinary finite set, except that an element can be included more than once and we care how many times it occurs. Two multisets are defined to be equal if they contain the same number of each natural. So  $\{2, 4, 4, 5\}$ , for example, is equal to  $\{4, 2, 5, 4\}$  but not to  $\{4, 2, 2, 5\}$ . We define a function  $f$  so that given any finite multiset  $S$  of naturals,  $f(S)$  is the product of a prime for each element of  $S$ . For example,  $f(\{2, 4, 4, 5\})$  is  $p_2 p_4 p_4 p_5 = 5 \times 11 \times 11 \times 13 = 7865$ .

- (a) Prove that  $f$  is a bijection from the set of all finite multisets of naturals to the set of positive naturals.
- (b) The **union of two multisets** is taken by including all the elements of each, retaining duplicates. For example, if  $S = \{1, 2, 2, 5\}$  and  $T = \{0, 1, 1, 4\}$ ,  $S \cup T = \{0, 1, 1, 1, 2, 2, 4, 5\}$ . How is  $f(S \cup T)$  related to  $f(S)$  and  $f(T)$ ?
- (c)  $S$  is defined to be a **submultiset** of  $T$  if there is some multiset  $U$  such that  $S \cup U = T$ . If  $S \subset T$ , what can we say about  $f(S)$  and  $f(T)$ ?
- (d) The **intersection of two multisets** consists of the elements that occur in both, with each element occurring the same number of times as it does in the one where it occurs fewer times. For example, if  $S = \{0, 1, 1, 2\}$  and  $T = \{0, 0, 1, 3\}$ ,  $S \cap T = \{0, 1\}$ . How is  $f(S \cap T)$  related to  $f(S)$  and  $f(T)$ ?

P3.6.10 (requires exposure to complex numbers) One form of the **Fundamental Theorem of Algebra**, which we aren't ready to prove here, says that any nonconstant polynomial in one variable over the complex numbers has at least one root.

- (a) Assuming this theorem, prove that a polynomial over the complex numbers is irreducible (as defined in Problem 3.6.3) if and only if it is linear. (Thus every polynomial over the complex numbers factors into a product of linears.)

- (b) Assuming this theorem, prove that any polynomial over the real numbers factors as a product of linears and quadratics, and thus every irreducible polynomial over the reals is linear or quadratic. (**Hint:** First view the polynomial as being over the complex numbers and factor it as in part (a). Then prove that if  $a + bi$  is a root of a polynomial with real coefficients, so is  $a - bi$ . Then show that  $(x - (a + bi))(x - (a - bi))$  is a quadratic polynomial with real coefficients which (unless  $b = 0$ ) is irreducible over the reals.)

### 3.7 Excursion: Expressing Predicates in Number Theory

So far all our concepts of number theory have been expressible as formulas in the predicate calculus, using only a few basic predicates: equality, addition, multiplication, and names for particular numbers. In *Gödel, Escher, Bach* Hofstadter defines a formal system he calls “Typographical Number Theory”, which is just our predicate calculus but also includes a symbol for “successor”, for reasons that might become clearer<sup>39</sup> in our Chapter 4. He shows, as we have, that other concepts like “less than”, “odd number”, “perfect square”, “divides”, and “prime” can easily be expressed without new symbols:

$$\begin{aligned}x < y &\leftrightarrow \exists z : x + z + 1 = y \\ \text{Odd}(x) &\leftrightarrow \exists z : (2 \cdot z) + 1 = x \\ \text{Square}(x) &\leftrightarrow \exists z : z \cdot z = x \\ D(x, y) &\leftrightarrow \exists z : x \cdot z = y \\ P(x) &\leftrightarrow (x > 1) \wedge \neg \exists y : \exists z : (y + 2)(z + 2) = x \\ P(x) &\leftrightarrow (x > 1) \wedge \neg \exists y : D(y, x) \wedge (1 < y) \wedge (y < x)\end{aligned}$$

Note two things in the case of primality — first, there is usually more than one way to express the same concept, and second, it’s possible to shorten the formula by various “tricks”. For example, in the first expression for  $P(x)$  above we implicitly use the fact that there exists a natural  $y$  with  $y \geq 2$  and  $Q(y)$  (here  $Q$  represents any predicate) if and only if there exists a natural  $z$  such that  $Q(z + 2)$ .

In this Excursion we’ll practice expressing concepts as quantified statements in number theory, concluding (in the Writing Exercise) with the solution to a “hard problem” in Hofstadter’s book. He says, “Strangely, this one takes great cleverness to render in our notation. I would caution you to try it only if you are willing to spend hours and hours on it — and if you know quite a bit of number theory!” We’ll see whether we can do better, with copious hints.

A key advantage we have in expressing properties is that we do know *some* number theory, and we can use tricks that work for reasons that don’t show up in the formula. For example, it’s easy to express “ $y$  is the smallest prime number that divides  $x$ ” as:

$$D(y, x) \wedge P(y) \wedge \forall z : (D(z, x) \wedge P(z)) \rightarrow (y \leq z).$$

This formula expresses the given predicate *whether or not* there actually *is* always such a smallest prime number that divides  $x$ . We happen to know that such a prime number always exists, but that wasn’t necessary for us to express the predicate. The statement that the smallest prime divisor always exists is just another statement of number theory, one that happens to be true.

---

<sup>39</sup>We could replace his “ $Sx$ ” with “ $x + 1$ ”, but he wants to formally define addition and multiplication in terms of successor, as we will do in Section 4.6. Hofstadter also uses the successor function to get all his number names from zero, so that 7, for example, is written “SSSSSSS0”.

Another trick allows you to express the predicate “ $x$  is a power of 2”, meaning that 2 is the only prime occurring in the factorization of  $x$ . Because we now know the Fundamental Theorem of Arithmetic, we know that  $x$  is a power of 2 if and only if there *isn’t* any other prime dividing  $x$ . (Equivalently, we could say that there is no odd number dividing  $x$ .) This is easy to express as

$$\neg \exists y : P(y) \wedge D(y, x) \wedge \neg(y = 2).$$

Hofstadter’s “hard problem” is to express the predicate “ $x$  is a power of 10”. We can take the “power of 2” formula above and adapt it to say “ $x$  is a power of  $p$ ” for any *prime number*  $p$ , but powers of 10 present us with a problem. A power of 10 has only 2’s and 5’s in its prime factorization, which we can express easily as

$$\forall y : [D(y, x) \wedge P(y)] \rightarrow [(y = 2) \vee (y = 5)],$$

but this formula holds for many naturals that are not powers of 10, such as 64, 400, or 125. To be a power of 10, a natural must have *the same number* of 2’s as 5’s in its factorization, which can’t be expressed in any obvious way.

In the Writing Exercise we’ll work through a way of solving this problem by coding *sequences of naturals* as single naturals, allowing us to say things like “there exists a sequence of naturals such that the first one is 1, each one is 10 times the one before, and the last one is  $x$ ”. It turns out that given this trick, *virtually any discrete computational process* can be formalized, in principle, as a formula of number theory. We’ll return to this topic in Chapter 15.

### 3.7.1 Writing Exercise

We want to code a sequence of  $k$  naturals, each bounded by a single natural  $b$ , in such a way that we can express the predicate  $Codes(z, k, b, i, a)$  which means “ $z$  codes a sequence of  $k$  naturals, each less than  $b$ , and the  $i$ ’th natural in the sequence is  $a$ ”.

1. Show that given this  $Codes$  predicate, we can express “ $x$  is a power of 10” in our version of the predicate calculus.
2. We saw in Section 3.5 that the Chinese Remainder Theorem gives us a way to go between several congruences involving small naturals, on the one hand, and one congruence involving large naturals on the other. If we have  $k$  naturals, each less than  $b$ , we can represent each natural by a congruence modulo  $b$ , or modulo any natural larger than  $b$ . The problem is that we need  $k$  different bases that are pairwise relatively prime, all larger than  $b$ , and defined by a single formula. A possibility is to take the naturals  $r + 1, 2r + 1, \dots, kr + 1$  for some number  $r$ . Write a formula, with free variables  $r$  and  $k$ , which says that these  $k$  naturals are pairwise relatively prime (call it  $Bases(r, k)$ ).
3. Prove  $\forall k : \forall b : \exists r : (r > b) \wedge Bases(r, k)$ . (Hint: We actually want  $r$  to have lots of divisors, so look at factorials.)
4. Write a formula expressing “ $r$  is the least natural that is greater than  $b$  and satisfies  $Bases(r, k)$ ”. Call this “ $BestBase(r, b, k)$ ”.

5. Write the formula  $Codes(z, k, b, i, a)$  as specified above. You may use congruence notation.

| + | 0 | 1 | 2 | 3 | 4 | 5 | × | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 | 4 | 5 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 2 | 3 | 4 | 5 | 0 | 1 | 0 | 1 | 2 | 3 | 4 | 5 |
| 2 | 2 | 3 | 4 | 5 | 0 | 1 | 2 | 0 | 2 | 4 | 0 | 2 | 4 |
| 3 | 3 | 4 | 5 | 0 | 1 | 2 | 3 | 0 | 3 | 0 | 3 | 0 | 3 |
| 4 | 4 | 5 | 0 | 1 | 2 | 3 | 4 | 0 | 4 | 2 | 0 | 4 | 2 |
| 5 | 5 | 0 | 1 | 2 | 3 | 4 | 5 | 0 | 5 | 4 | 3 | 2 | 1 |

©Kendall Hunt Publishing Company

Figure 3-6: Addition and multiplication tables modulo 6.

## 3.8 The Ring of Congruence Classes

### 3.8.1 New Objects From Old

A principal tool of mathematics is the creation of new mathematical objects from old ones<sup>40</sup>. Equivalence relations give us a very general method to do this. We’ve seen that for every equivalence relation, there is a partition of the base set into equivalence classes. If we choose, we can view these classes as objects in their own right, and see what can be done with them.

The equivalence relation of **congruence** modulo  $r$  is defined so that  $C(x, y)$  is true if and only if  $x \equiv y \pmod{r}$ . It is easy to check that congruence modulo  $r$  is an equivalence relation. (That is, as long as we keep  $r$  fixed — knowing a congruence modulo  $r$  doesn’t necessarily tell us anything about congruences with other bases.)

The equivalence classes of the congruence relation modulo  $r$  are called **congruence classes**, and there are exactly  $r$  of them. For each of the  $r$  naturals that are less than  $a$ , we have a class: the set  $\{i : i \equiv a \pmod{r}\}$ . If  $r = 2$ , for example, the two classes are the even numbers  $\{0, 2, 4, \dots\}$  and the odd numbers  $\{1, 3, 5, \dots\}$ . For  $r = 9$ , the nine classes are  $\{0, 9, 18, \dots\}$ ,  $\{1, 10, 19, \dots\}$ ,  $\{2, 11, 20, \dots\}$ , and so forth.

We saw back in Section 3.3 that the congruence class of a sum or product of integers depends only on the congruence class of the addends or factors. This means, in effect, that congruence classes on the integers can be added, subtracted, multiplied, and sometimes divided. For any positive natural  $r$ , the congruence classes modulo  $r$  form a number system, which we call  $\mathbf{Z}_r$ , “the integers modulo  $r$ ”. For convenience, we usually name each element of  $\mathbf{Z}_r$  after its smallest representative — for example, the element  $\{2, 11, 20, \dots\}$  of  $\mathbf{Z}_9$  is usually called “2”. We can represent the addition and multiplication operations of  $\mathbf{Z}_r$  by tables, as Figure 3-6 does for the case of  $\mathbf{Z}_6$ .

In general this phenomenon of passing from objects to equivalence classes is called “modding out” or “taking a quotient”. We’ll see another important example in Chapter 14 when we deal with

<sup>40</sup>Why might one want to create new objects? As we said in Section 1.1, a pure mathematician would be interested in their beauty, or their usefulness in solving interesting problems. An applied mathematician would be interested in the possibility that they might model some aspect of reality.

finite-state machines. There the equivalence relation on strings will be “input strings  $x$  and  $y$  cause the machine to do the same thing”. Other examples abound in linear algebra (such as quotient spaces) and abstract algebra (such as quotient groups).

### 3.8.2 The Axioms For a Ring

If  $r$  is any positive natural, the set  $\mathbf{Z}_r$  of integers modulo  $r$  forms an algebraic system called a **ring**<sup>41</sup>. We can add or multiply any two numbers in  $\mathbf{Z}_r$ , and these operations satisfy specific properties that are familiar from various number systems:

- Addition is **commutative** ( $x + y = y + x$ ) and **associative** ( $x + (y + z) = (x + y) + z$ ).
- There is an **additive identity element** 0, such that  $x + 0 = 0 + x = x$  and  $0x = x0 = 0$  for any  $x$ .
- Every element  $x$  has an **additive inverse** “ $-x$ ”, such that  $x + (-x) = 0$ .
- Multiplication is **commutative** and **associative**.
- The **distributive law**,  $x(y + z) = xy + xz$ , holds.
- There is a **multiplicative identity element** 1 such that  $0 \neq 1$  and  $1x = x1 = x$  for any  $x$ .

Note that the naturals themselves do not form a ring, because they don’t have additive inverses. (They form a simpler system called a *semiring*, which we’ll look at in Chapter 4.) But in  $\mathbf{Z}_r$  you can always get from  $a$  to 0 by adding the equivalence class of  $r - a$  (for example,  $3 + 2 \equiv 0 \pmod{5}$ ), so every element does have an additive inverse.

In the case where the modulus is a prime number  $p$ , the integers modulo  $p$  form a number system called a *finite field* which we’ll study in the next section. Because the rings  $\mathbf{Z}_r$  and finite fields are *discrete* like digital logic systems (as opposed to *continuous* like the real or complex numbers), they tend to come up often in modeling computer systems.

### 3.8.3 Rings and the Chinese Remainder Theorem

If  $r$  is composite, we can use the Chinese Remainder Theorem to discover more about the structure of  $\mathbf{Z}_r$ . Remember that any natural can be factored uniquely as a product of *prime powers*, and that these prime powers are pairwise relatively prime (for example,  $60 = 4 \times 3 \times 5$ ). A congruence modulo  $r$ , then, is equivalent via the Chinese Remainder Theorem to a system of congruences, one for each of the prime powers in this factorization.

For example, suppose we know that  $x \equiv 23 \pmod{60}$  and  $y \equiv 44 \pmod{60}$ . The first congruence is equivalent to  $x \equiv 3 \pmod{4}$ ,  $x \equiv 2 \pmod{3}$ , and  $x \equiv 3 \pmod{5}$ . (We find the numbers 3, 2, and

<sup>41</sup>Formally, we are giving the axioms for a **commutative ring with identity** rather than a general ring.

3 by taking 23 modulo 4, 3, and 5 respectively.) Similarly, the second congruence can be converted to  $y \equiv 0 \pmod{4}$ ,  $y \equiv 2 \pmod{3}$ , and  $y \equiv 4 \pmod{5}$ .

We've seen that we can use these systems of congruences to calculate  $x + y$  and  $xy$  modulo 60. For example, by adding the pairs of congruences with the same base we find that  $x + y \equiv 3 \pmod{4}$ ,  $x + y \equiv 1 \pmod{3}$ , and  $x + y \equiv 2 \pmod{5}$ . Using the proof of the Chinese Remainder Theorem, we can convert this system to the single congruence  $x + y \equiv 7 \pmod{60}$ . Similarly, we can compute  $xy \equiv 3 \cdot 0 \equiv 0 \pmod{4}$ ,  $xy \equiv 2 \cdot 2 \equiv 1 \pmod{3}$ , and  $xy \equiv 3 \cdot 4 \equiv 2 \pmod{5}$ , and convert the resulting system to  $xy \equiv 52 \pmod{60}$ .

With respect to addition and multiplication,  $\mathbf{Z}_{60}$  behaves just like the **direct product**  $\mathbf{Z}_4 \times \mathbf{Z}_3 \times \mathbf{Z}_5$ , where we perform an operation in the direct product by performing the three operations in the individual rings in parallel. For any  $r$ , the ring  $\mathbf{Z}_r$  is equivalent<sup>42</sup> to such a direct product of rings  $\mathbf{Z}_{p^e}$ , for the prime powers  $p^e$  in the prime-power factorization of  $r$ . This fact will be useful to us in the remainder of the chapter.

### 3.8.4 Classifying Abelian Groups

Before leaving this topic we should look at one more type of algebraic structure. A **group** is a set with an operation that is associative, has an identity element, and has inverses for every element. An **abelian** group, named after the 19th-century Norwegian mathematician Niels Henrik Abel, is a group in which the operation is also commutative<sup>43</sup>. Thus a ring, as we have defined it, is also an abelian group if we consider only the addition operation.

A natural mathematical question is to **classify** the structures that obey a particular set of axioms. Two structures are thought of as “the same” if they are **isomorphic**. An **isomorphism** from one structure to another is a bijection that also respects the relevant algebraic operations. For example, a bijection  $f$  from one group  $G$  to another group  $H$  is an isomorphism if and only if it respects the rule  $f(xy) = f(x)f(y)$  for all elements  $x$  and  $y$  of  $G$ . Note that the left-hand side of this equation includes a multiplication in  $G$ , while the right-hand side includes a multiplication in  $H$ .

The Chinese Remainder Theorem essentially tells us how to classify finite abelian groups. Since our examples of abelian groups so far are additive structures of rings, we'll write the operation of an arbitrary finite abelian group as addition and call the additive identity “0”. Consider any nonzero element  $a$ . If we look at the sequence of elements  $0, a, a + a, a + a + a, \dots$ , some element must eventually be repeated because there are only finitely many elements. If  $i$  copies of  $a$  have a sum equal to  $j$  copies of  $a$ , with  $i < j$ , then  $j - i$  copies of  $a$  must add to 0. Thus every nonzero element  $a$  (of a finite abelian group) has an **additive order**  $o(a)$ , the least natural  $q$  such that  $q$  copies of  $a$  add to 0.

How large could the order be? Every element in the sequence is distinct until 0 appears for the second time, so we must have at least  $o(a)$  distinct elements in the group. Thus  $o(a)$  can be no larger than the size of the group. Could it be equal? Yes, it is equal if the group is  $\mathbf{Z}_m$ , and we

<sup>42</sup>The proper algebraic word is **isomorphic**, as we will see below. See Section 8.7 for another example.

<sup>43</sup>Q: What's purple and commutes? A: An abelian grape!



take  $a = 1$ . This group is called **cyclic**, and any two cyclic groups of the same order are isomorphic (see Exercise 3.8.6).

**Theorem:** In any finite abelian<sup>44</sup> group  $G$ , the order of any element divides the number of elements in  $G$ .

**Proof:** Since the order of 0 is 1, and 1 divides any natural, our conclusion is true for 0. Let  $a$  be any nonzero element and consider the set  $H = \{0, a, aa, \dots, (o(a)-1)a\}$ . Define an equivalence relation on the elements of  $G$ , so that  $R(x, y)$  means that there is some natural  $k$  such that  $x + ka = y$ . Like any equivalence relation,  $R$  divides its set into classes, one of which is  $H$ . In fact each of the other classes also has exactly  $o(a)$  elements, since the class of  $x$  is exactly  $\{x, x+a, x+2a, \dots, x+(o(a)-1)a\}$ . Since each element of  $G$  is in exactly one class, the number of elements in  $G$  must be the number of classes times  $o(a)$ , and thus is a multiple of  $o(a)$ . ♠

So, for example, every abelian group with a prime number of elements must be cyclic. What about a group of order  $p^2$ , where  $p$  is a prime? The only possible orders of an element are 0,  $p$ , and  $p^2$ , since these are the only naturals that divide  $p^2$ . If there is any element of order  $p^2$ , the group is cyclic and is isomorphic to  $\mathbf{Z}_{p^2}$ . If not, you'll show below in Problem 3.8.7 that the group must be isomorphic to  $\mathbf{Z}_p \times \mathbf{Z}_p$ .

We can form a great variety of abelian groups by taking direct products of cyclic groups. What the Chinese Remainder Theorem tells us is that some of these products are isomorphic to others. We know that the rings  $\mathbf{Z}_m \times \mathbf{Z}_n$  and  $\mathbf{Z}_{mn}$  are isomorphic if  $m$  and  $n$  are relatively prime, and this means that the additive structures of these rings must also be isomorphic.

In fact, although we won't be able to prove it here, such direct products are the *only* finite abelian groups:

**Theorem:** Any finite abelian group is isomorphic to a direct product of cyclic groups. ♠

### 3.8.5 Exercises

- E3.8.1 Verify that for any positive number  $r$ , the relation of congruence modulo  $r$  is an equivalence relation.
- E3.8.2 Prove that there are exactly  $r$  congruence classes modulo  $r$ , by showing that every natural is congruent to some  $a < r$ , and that if  $a$  and  $b$  are both less than  $r$  they are not congruent unless they are equal.
- E3.8.3 Determine whether each of the following is an equivalence relation. If it is, describe its equivalence classes. If it is not, indicate which of the properties do not hold for it. In each case the base set is the set of positive integers.
- (a)  $R_1(x, y)$  if and only if  $x$  and  $y$  are relatively prime.
  - (b)  $R_2(x, y)$  if and only if there is a number  $z$  such that both  $x$  and  $y$  divide  $z$ .

<sup>44</sup>This result is true for any finite group, though we won't worry about that here.

- (c)  $R_3(x, y)$  if and only if there is a number  $z$  such that both  $x$  and  $y$  are powers of  $z$ .
  - (d)  $R_4(x, y)$  if and only if there is a number  $z$  such that  $z > 1$  and  $z$  divides both  $x$  and  $y$ .
  - (e)  $R_5(x, y)$  if and only if there is a positive number  $z$  such that both  $xz = y$  and  $yz = x$ .
- E3.8.4 Let  $SPD$  (for “same prime divisors”) be the relation on the set of positive naturals defined so that  $SPD(x, y)$  if and only if for all prime numbers  $p$ ,  $p$  divides  $x$  if and only if  $p$  divides  $y$ .
- (a) Prove that  $SPD$  is an equivalence relation.
  - (b) List all numbers  $x$  such that  $x < 100$  and  $SPD(x, 12)$ .
  - (c) Describe the equivalence classes of  $SPD$ .
- E3.8.5 Define  $\mathbf{Z}_r[x]$  to be the set of polynomials in  $x$  whose coefficients are in  $\mathbf{Z}_r$ . Verify that  $\mathbf{Z}_r[x]$  satisfies all the axioms for a ring, using the usual definitions of addition and multiplication of polynomials.
- E3.8.6 Two problems about isomorphisms of abelian groups:
- (a) Prove that any two cyclic abelian groups with the same (finite) number of elements are isomorphic.
  - (b) Prove that there are two abelian groups with nine elements that are not isomorphic to one another.
- E3.8.7 A ring is said to have **zero divisors** if there are two nonzero elements (not necessarily distinct) that multiply to 0. For which naturals  $m$  does  $\mathbf{Z}_m$  have zero divisors?
- E3.8.8 Why couldn't a ring have two different additive identities, or two different multiplicative identities?
- E3.8.9 Let  $A$  be an abelian group, with the operation written as addition and the identity called 0. Pick any element other than 0 and call it 1. Define a multiplication operation on  $A$  so that for any element  $x$ ,  $0x = x0 = 0$  and  $1x = x1 = x$ , and so that  $xy = 0$  if neither  $x$  and  $y$  are equal to 0 nor 1. Prove that the resulting structure may fail to be distributive, but satisfies all the other ring axioms.
- E3.8.10 Consider the set of polynomials over  $\mathbf{Z}_2$  where we consider  $x^2$  to be equal to  $x$ . There are four elements in this set: 0, 1,  $x$ , and  $x + 1$ . Make addition and multiplication tables for this structure and verify that it is a ring. Is it isomorphic to either of the rings  $\mathbf{Z}_4$  or  $\mathbf{Z}_2 \times \mathbf{Z}_2$ ?

### 3.8.6 Problems

- P3.8.1 Let  $r$  be a natural. Define the binary relation  $T_r$  on the naturals so that  $T_r(x, y)$  is true if  $x = y$  or both  $x \geq r$  and  $y \geq r$ .  $T_r(x, y)$  is read “ $x$  is equivalent to  $y$  threshold  $r$ .” Describe all the equivalence classes of  $T_5$ . Explain why we can add and multiply the classes of  $T_r$  for any  $r$ , and construct addition and multiplication tables for the classes of  $T_5$ .
- P3.8.2 Let  $f$  be a polynomial with coefficients in  $\mathbf{Z}_r$ , that is, a member of the ring  $\mathbf{Z}_r[x]$  defined in Exercise 3.8.5 above. Recall that if  $g$  is any polynomial, we can divide  $g$  by  $f$  to get polynomials  $q$  and  $r$  such that  $g = qf + r$  and the degree of  $r$  is less than the degree of

$f$ . Define two polynomials to be congruent modulo  $f$  if they differ by a multiple of  $f$ , and consider the congruence classes of this relation. Show that each class has exactly one element whose degree is less than that of  $f$ . Show that these classes may be added and multiplied, just as for the classes in  $\mathbf{Z}_r$  (Hint: You must show that the classes of  $x + y$  and  $xy$  depend only on the class of  $x$  and  $y$  – this will be similar to the proof for ordinary congruence in Section 3.3).

- P3.8.3 Following Problem 3.8.2, let  $r = 2$  and let  $f = x^3 + x + 1$ . List the congruence classes of  $\mathbf{Z}_2[x]$  modulo  $f$ . Construct addition and multiplication tables for the ring of these congruence classes. Is this ring isomorphic to  $\mathbf{Z}_m$ , where  $m$  is its size?
- P3.8.4 Let  $B = \{0, 1\}$  and define addition and multiplication on  $B$  as for boolean algebra, so that  $1 + 1 = 1$  and all other sums and products are as given by the ring axioms. Construct addition and multiplication tables for this set. Is this a ring? Why or why not?
- P3.8.5 Let  $S$  be a non-empty set and let  $P(S)$  be the power set of  $S$  (the set of all subsets of  $S$ ). Define the “sum” of two sets  $X$  and  $Y$  to be  $X \triangle Y$ , the symmetric difference, and define the “product” of  $X$  and  $Y$  to be  $X \cap Y$ . Prove that  $P(S)$  forms a ring under these two operations (you must decide what the identity elements are).
- P3.8.6 Just as we did for abelian groups, we can classify all possible rings with certain finite numbers of elements.
- (a) Let  $p$  be any prime. Prove that every ring with exactly  $p$  elements is isomorphic to  $\mathbf{Z}_p$ .
  - (b) Find all possible rings with exactly four elements. (Hint: We know that there are two possible additive structures for a four-element set, those of  $\mathbf{Z}_4$  and  $\mathbf{Z}_2 \times \mathbf{Z}_2$ . In the latter case, the result of multiplication by 0 or 1 is forced by the axioms. If we call the other two elements  $x$  and  $x + 1$ , what are the possible values of  $x \times x$ ?)
- P3.8.7 Let  $p$  be a prime and let  $G$  be any group with  $p^2$  elements that is not cyclic. Prove that  $G$  is isomorphic to  $\mathbf{Z}_p \times \mathbf{Z}_p$ .
- P3.8.8 A natural  $n$  is called **squarefree** if there is no natural  $k$  such that  $k > 1$  and  $k^2$  divides  $n$ . Prove that any finite abelian group with a squarefree number of elements is cyclic.
- P3.8.9 Given the classification theorem for finite abelian groups, we can be more specific about the component cyclic groups of the factorization.
- (a) Prove that any finite abelian group is a direct product of cyclic groups, each of which has prime power size (but possibly with the same prime occurring more than once).
  - (b) Prove that any finite abelian group is isomorphic to a direct product  $\mathbf{Z}_{d_1} \times \mathbf{Z}_{d_2} \times \dots \times \mathbf{Z}_{d_k}$ , where each number  $d_i$  divides the number  $d_{i+1}$ .
- P3.8.10 Let  $m$  and  $n$  be two relatively prime naturals. Let  $G$  be a finite abelian group with  $mn$  elements that has an element  $a$  of order  $m$  and an element  $b$  of order  $n$ . Prove that  $G$  is isomorphic to  $\mathbf{Z}_m \times \mathbf{Z}_n$ .

## 3.9 Finite Fields and Modular Exponentiation

### 3.9.1 The Definition of a Field

In the last section we defined a ring to be a set of numbers with addition and multiplication operations that obey a particular set of properties. A **field** is a ring that has one additional property as well:

- Every element  $x$ , except for 0, has a **multiplicative inverse** “ $1/x$ ”, such that  $x \cdot (1/x) = 1$ .

In a field, then, you can divide  $x$  by  $y$  (as long as  $y$  isn't 0) by multiplying  $x$  by  $1/y$ . You can't do this over the integers, as usually  $1/x$  isn't an integer, but the rational numbers (fractions of integers) and real numbers are both fields. Our concern in this section is **finite fields** and some of their properties.

Is  $\mathbf{Z}_r$ , the ring of integers modulo  $r$ , a field? It depends on whether we can divide by any nonzero element, and we know from the Inverse Theorem exactly when we can divide. A natural has an inverse modulo  $r$  if and only if it is relatively prime to  $r$ . Let's focus in, then, on the set of numbers in  $\mathbf{Z}_r$  that have inverses, which we'll call<sup>45</sup>  $\mathbf{Z}_r^*$ . If  $r$  is prime, then every nonzero element of  $\mathbf{Z}_r$  is relatively prime to  $r$ ,  $\mathbf{Z}_r^*$  consists of all  $r - 1$  nonzero elements, and  $\mathbf{Z}_r$  is a field<sup>46</sup>. If  $r$  is composite, on the other hand, some of the nonzero elements of  $\mathbf{Z}_r$  are not in  $\mathbf{Z}_r^*$ , and thus  $\mathbf{Z}_r$  is not a field. The size of  $\mathbf{Z}_r^*$  is called  $\phi(r)$ , the **Euler totient function** — we've just shown that  $\phi(r) = r - 1$  if  $r$  is prime.

### 3.9.2 Modular Exponentiation

We're now going to look at the last basic arithmetic operation, that of **exponentiation**, in a ring  $\mathbf{Z}_r$ . Given any naturals  $a$  and  $b$ , it makes sense to talk about  $a^b$  modulo  $r$ , the product of  $b$  copies of  $a$ , taken<sup>47</sup> in  $\mathbf{Z}_r$ . This operation will turn out to be useful in testing or certifying primality in Excursion 3.10, and in implementing the RSA cryptosystem in Section 3.11.

As a bit of an aside, how can we best calculate  $a^b$  modulo  $r$ ? There are two decidedly wrong ways to do it. One is to first calculate the *natural*  $a^b$ , and then divide it by  $r$ . This could be bad if  $b$  is big, as  $a^b$  might be too big a number to fit in a word of memory. (If  $a$  and  $b$  each fit into 64 bits, for example, how big might  $a^b$  be?) We can avoid this problem by dividing by  $r$  *after every operation*, so that the numbers we multiply are always no bigger than  $r$ . The other bad idea is to calculate  $a^b$  by multiplying by  $a$   $b$  times, which would be horribly time-consuming if  $b$  were really

<sup>45</sup>Also called the **multiplicative group** of numbers modulo  $r$ . As we've said, a **group** is a set with an operation that is associative, has an identity, and has inverses. In Exercise 3.9.1 you'll check that  $\mathbf{Z}_r^*$  is a group.

<sup>46</sup>This isn't the only possible way to get a finite field — see the Problems.

<sup>47</sup>Note right away that we can't think of  $b$  as a number in  $\mathbf{Z}_r$  as we do this, as it will turn out that in general  $b \equiv c \pmod{r}$  is no guarantee that  $a^b \equiv a^c \pmod{r}$ . This is in sharp contrast to the situation for the other operations. We can still think of  $a$  as being in  $\mathbf{Z}_r$ , however.

big. (If  $b = 2^{64}$ , just for example, you'd be doing over  $10^{19}$  multiplications). Here the trick that saves us is **repeated squaring**, where we calculate  $a^{64}$ , for example, by taking  $a$ , squaring it to get  $a^2$ , squaring *that* to get  $a^4$  with only one more multiplication, then successively<sup>48</sup> getting  $a^8$ ,  $a^{16}$ ,  $a^{32}$ , and  $a^{64}$ .

As we look at the powers of  $a$  in  $\mathbf{Z}_r$ , where  $a \in \mathbf{Z}_r^*$ , the sequence  $\langle a^0, a^1, a^2, a^3, \dots \rangle$  must eventually repeat itself, because there are only so many possible elements of  $\mathbf{Z}_r$  that could ever occur in it. Once you know that  $a^s = a^t$ , you can multiply both sides of this equation by  $(1/a)^s$  and get that  $1 = a^{s-t}$ . The sequence of powers therefore must reach 1, and we define the **order** of  $a$  to be the smallest positive number  $u$  such that  $a^u = 1$  (more formally,  $a^u \equiv 1 \pmod{r}$ ). The sequence of powers is thus periodic with a period equal to the order of  $a$  — for example with  $a = 2$  and  $r = 9$  we get  $\langle 1, 2, 4, 8, 7, 5, 1, 2, 4, 8, 7, 5, 1, \dots \rangle$ . This brings us to an important fact:

**Theorem:**<sup>49</sup> For any number  $r$  and any  $a \in \mathbf{Z}_r^*$ , the order of  $a$  in  $\mathbf{Z}_r^*$  divides  $\phi(r)$ . In particular,  $a^{\phi(r)} \equiv 1 \pmod{r}$ .

**Proof:** Let  $u$  be the order of  $a$  and let  $b$  be an arbitrary element of  $\mathbf{Z}_r^*$ . Consider the *set* of elements obtained by starting with  $b$  and repeatedly multiplying by  $a$ , that is, the set  $\{b, ba, ba^2, ba^3, \dots\}$ . We actually know that this set contains exactly  $u$  elements, because if  $t \geq u$  the element  $ba^t$  has already occurred in the set as  $ba^{t-u}$  (because  $a^u \equiv 1 \pmod{r}$ ) and the elements  $b, ba, \dots, ba^{u-1}$  are distinct (otherwise if  $ba^s = ba^t$ , multiply both sides by  $1(ba^s)$  to get  $a^{t-s} = 1$  with  $t-s < u$ ). Now consider these sets for all possible  $b$ . They form a partition of  $\mathbf{Z}_r^*$ , because (as we can easily check) the relation  $R(b, c)$  defined by the predicate  $\exists s : c = ba^s$  is an equivalence relation. For  $\mathbf{Z}_r^*$ , with exactly  $\phi(r)$  elements, to be partitioned into sets with  $u$  elements each, it is clear that  $u$  must divide  $\phi(r)$ . ♠

A special case of this theorem is one of the more famous results in number theory:

**Fermat's Little Theorem:** If  $p$  is prime and  $a$  is relatively prime to  $p$ , then  $a^{p-1} \equiv 1 \pmod{p}$ . ♠

### 3.9.3 The Multiplicative Group of $\mathbf{Z}_p$ is Cyclic

We are now ready to determine the structure of  $\mathbf{Z}_p^*$  under multiplication. We've just shown that the order of every element *divides*  $p-1$ , but it takes a bit more work to show that there are some elements of order *exactly*  $p-1$ .

To do this, we need to look at *polynomials* whose coefficients are in  $\mathbf{Z}_p$ . Many of the familiar facts of algebra over the real numbers remain true over any field. For example, if  $f(x)$  is a polynomial, and evaluating  $f(a)$  for some particular field element  $a$  gives 0, then the polynomial  $x-a$  divides the polynomial  $f(x)$ . One way to see this is simply to divide the polynomial  $f(x)$  by  $x-a$ . The remainder must have degree less than that of  $x-a$  and so must itself be an element of  $\mathbf{Z}_p$ . But if

<sup>48</sup>It's a bit different if  $b$  isn't a power of two, but the same trick can be used to get  $a^b$  in about  $2k$  multiplications at worst, where  $k$  is the number of bits in the binary expansion of  $b$ . Problem 3.9.3 asks you to devise this algorithm and later in Section 8.3 we will look at how many multiplications it uses.

<sup>49</sup>Students of abstract algebra may recognize this theorem as a special case of "in any finite group, the order of any element divides the order of the group". The proof we give here essentially works in the more general setting.

$f(x) = g(x)(x - a) + b$ , then substituting  $a$  for  $x$  we get  $f(a) = g(a)(a - a) + b$ , and  $b$  must be 0 because  $f(a) = 0$ .

Because the polynomials have unique factorization, we also get the familiar fact that a polynomial of degree  $d$  can have at most  $d$  different **roots** (field elements  $a$  such that  $f(a) = 0$ ). This is because the polynomials  $x - a$  for different values of  $a$  are irreducible (“prime”, given the definition of “division” for polynomials) and so their product divides  $f(a)$ . If there were more than  $d$  roots, these polynomials would multiply together to get something with degree greater than  $d$ , which couldn’t divide a polynomial of degree  $d$ .

**Lemma:** If  $p$  is prime, and  $k$  divides  $p - 1$ , there are *at most*  $\phi(k)$  elements of  $\mathbf{Z}_p^*$  whose order is exactly  $k$ .

**Proof:** One possibility is that there are no such elements, in which case the Lemma is certainly true<sup>50</sup>. Otherwise, if there exist any elements of order exactly  $k$ , pick one and call it  $a$ . Now we can find  $k$  distinct elements satisfying the polynomial  $x^k - 1 = 0$ , namely  $1, a, a^2, \dots, a^{k-1}$ . Since this polynomial has at most  $k$  roots, there aren’t any other elements satisfying  $x^k - 1$ , so we have only these  $k$  possibilities to search for elements of order exactly  $k$ . When does  $a_i$  have order exactly  $k$ ? If and only if  $i$  is relatively prime to  $k$ , as  $(a^i)^j = a^{ij}$  is equal to 1 if and only if  $k$  divides  $ij$ , and thus  $a^i$  has order  $k$  if and only if  $i$  and  $k$  have no common multiple less than  $ik$ . There are exactly  $\phi(k)$  numbers less than  $k$  that are relatively prime to  $k$ , by the definition of the function  $\phi$ . ♠

**Theorem:** If  $p$  is prime, there is an element  $g$  of  $\mathbf{Z}_p^*$  that **generates**<sup>51</sup>  $\mathbf{Z}_p^*$ , meaning that every element of  $\mathbf{Z}_p^*$  is equal to  $g^i$  for some  $i$ .

**Proof:** We just saw that for every  $k$  dividing  $p - 1$ , there are at most  $\phi(k)$  elements of order exactly  $k$ . In fact there have to be *exactly* this many elements of each order, because otherwise there aren’t enough elements to go around! Consider the  $p - 1$  numbers in  $\mathbf{Z}_{p-1}$ . Each number  $i$  has an (additive)<sup>52</sup> order, the least positive number  $j$  such that  $i \cdot j = 0$ . For each number  $i$ , define  $n_i$  to be the number of elements of  $p - 1$  whose additive order is exactly  $i$ . The only nonzero values of  $n_i$  are when  $i$  divides  $p - 1$ , because every element of  $\mathbf{Z}_{p-1}$  satisfies  $i \cdot (p - 1) = 0$ . If  $i$  *does* divide  $p - 1$ , we can find exactly which elements have additive order exactly  $i$ . Letting  $j = (p - 1)/i$ , we can see that  $j$  is such an element, as is  $mj$  where  $m$  is any number relatively prime to  $i$ . These are all the possibilities, as if  $m$  is not relatively prime to  $i$  then  $mj$  has a smaller order, and if  $j$  does not divide a number  $x$  then  $ij$  cannot divide  $ix$ .

So the number  $n_i$ , if  $i$  divides  $p - 1$ , is equal to  $\phi(i)$ . This means that if we add up the numbers  $\phi(i)$  over all  $i$  dividing  $p - 1$  (including the case  $i = p - 1$ ), we get exactly  $p - 1$ . The orders of the  $p - 1$  numbers in  $\mathbf{Z}_p^*$  must then split up exactly the same way, because once we take away the at most  $\phi(i)$  numbers for each  $i$  dividing  $p - 1$  and less than  $p - 1$ , this analysis shows that there are at least  $\phi(p - 1)$  numbers left, and  $p - 1$  is the only possible order left for each of them. In particular, since  $\phi(p - 1)$  is positive, we know that there is at least one such element of order  $p - 1$ , and it is the desired generator. (Note that if  $g$  has order  $p - 1$ , the  $p - 1$  numbers  $1, g, g^2, \dots$ ,

<sup>50</sup>Though we’ll prove in a moment that this case never actually happens.

<sup>51</sup>Another way to say this is to say that  $\mathbf{Z}_p^*$  is a cyclic group.

<sup>52</sup>We are switching gears slightly, talking now about order as the period of the system we get by successively *adding*  $i$ , as opposed to multiplying as we did above. The reason for this is that the *multiplicative* behavior of  $\mathbf{Z}_p^*$  turns out to be identical (“isomorphic”) to the additive behavior of  $\mathbf{Z}_{p-1}$ .

$g^{p^2-2}$  are all distinct, so these must be all the numbers in  $\mathbf{Z}_p^*$ .



**Example:** Consider the set  $\mathbf{Z}_{13}^*$ . We can find the order of each element by finding all of its powers in order until we get 1. The element 1 has order 1, and with 2 we get the sequence 1, 2, 4, 8, 3, 6, 12, 11, 9, 5, 10, 7, 1, showing that 2 is a generator. Now we can stop, actually, and read off the remaining answers from the powers of 2. The other generators are the numbers  $2^m$  for  $m$  relatively prime to 12, namely  $2^5 = 6$ ,  $2^7 = 11$ , and  $2^{11} = 7$ . There are  $\phi(6) = 2$  elements of order 6 ( $2^2 = 4$  and  $2^{10} = 10$ ),  $\phi(4) = 2$  elements of order 4 ( $2^3 = 8$  and  $2^9 = 5$ ),  $\phi(3) = 2$  elements of order 3 ( $2^4 = 3$  and  $2^8 = 9$ ), and  $\phi(2) = 1$  element of order 2 ( $2^6 = 12$ ).

What does  $\mathbf{Z}_r^*$  look like if  $r$  is composite? The Chinese Remainder Theorem tells us that the ring  $\mathbf{Z}_r$  can be thought of as a direct product of rings  $\mathbf{Z}_{p^e}$  where the numbers  $p^e$  are the prime-power factors of  $r$ . It's not hard to see that  $\mathbf{Z}_r^*$  is also a direct product, the direct product of the multiplicative groups  $\mathbf{Z}_{p^e}^*$  for the numbers  $p^e$ . Why? An element  $w$  of  $\mathbf{Z}_r$  is in  $\mathbf{Z}_r^*$  if and only if it has no factor in common with  $r$  other than 1. If we think of  $w$  as a sequence of naturals, one for each prime power in the base (as we thought of 23 in  $\mathbf{Z}_{60}$  as the triple  $\langle 3, 2, 3 \rangle$ ), we can see that  $w \in \mathbf{Z}_r^*$  if and only if each of these individual numbers is relatively prime to its base. How do we justify this claim? One way is to use the Inverse Theorem in reverse. If the number is relatively prime to its base, it has an inverse modulo that base, and the sequence of inverses is an inverse for the original number. For example, since  $3 \cdot 3 \equiv 1 \pmod{4}$ ,  $2 \cdot 2 \equiv 1 \pmod{3}$ , and  $3 \cdot 2 \equiv 1 \pmod{5}$ ,  $\langle 3, 2, 3 \rangle$  has an inverse  $\langle 3, 2, 2 \rangle$  in  $\mathbf{Z}_4 \times \mathbf{Z}_3 \times \mathbf{Z}_5$ , because these two triples multiply to give  $\langle 1, 1, 1 \rangle$ . A number with an inverse is relatively prime to its base. Which numbers  $w \in \mathbf{Z}_{60}$  are prime to 60? If we view  $w$  as a triple  $\langle x, y, z \rangle$ , we need for  $x$  to be prime to 4 (thus 1 or 3),  $y$  to be prime to 3 (1 or 2), and  $z$  to be prime to 5 (1, 2, 3, or 4). There are 16 ways to choose such a triple<sup>53</sup>, yielding the fact that  $\mathbf{Z}_{60}^*$  is the 16-element set  $\{1, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 49, 53, 59\}$ . (In the Problems, you'll use this analysis to calculate  $\phi(r)$  for arbitrary numbers  $r$ .)

### 3.9.4 Exercises

- E3.9.1 Let  $r$  be any number and let  $x$  and  $y$  be any two elements of  $\mathbf{Z}_r^*$ . Prove that  $xy \in \mathbf{Z}_r^*$ . Complete the argument that  $\mathbf{Z}_r^*$  is a group by checking that the multiplication operation is associative, has an identity, and has inverses.
- E3.9.2 Let  $r$  be a natural and  $a$  be a member of  $\mathbf{Z}_r^*$ . Define the relation  $R(b, c)$ , for  $b$  and  $c$  members of  $\mathbf{Z}_r^*$ , by the predicate  $\exists s : b = ca^s$ . Prove that  $R$  is an equivalence relation on  $\mathbf{Z}_r^*$ .
- E3.9.3 Consider the congruence relation on  $\mathbf{Z}_3[x]$  modulo the polynomial  $x^2 + 1$ , and define  $C$  to be the set of congruence classes. (You should get nine of them.) Construct addition and multiplication tables for  $C$  under polynomial addition and multiplication modulo  $x^2 + 1$ . Verify that  $C$  is a field. Is it isomorphic to  $\mathbf{Z}_9$  as a ring?
- E3.9.4 Find all the elements of  $\mathbf{Z}_{63}^*$  and describe its structure as a direct product.
- E3.9.5 Find all the generators of  $\mathbf{Z}_{17}^*$  and  $\mathbf{Z}_{19}^*$ .
- E3.9.6 Show that a ring with zero divisors cannot be a field.

<sup>53</sup>Counting them by a technique to be formally presented in Section 6.1.

- E3.9.7 The **characteristic** of a finite ring is the least positive natural  $t$  such that the sum of  $t$  copies of the additive identity 1 is equal to the multiplicative identity 0.
- (a) Prove that if the characteristic of a ring is composite, the ring is not a field.
  - (b) Prove that the additive order of any element of a ring divides the characteristic.
  - (c) Argue that in any finite field, every nonzero element except 1 has the same prime additive order.
- E3.9.8 Let  $F$  be a finite field with  $n$  nonzero elements. Prove that there are exactly  $\phi(n)$  elements of  $F$  that generate  $F^*$  under multiplication.
- E3.9.9 (requires exposure to complex numbers) Is it possible to have a subset of the complex numbers that forms a finite field under the standard addition and multiplication operations?
- E3.9.10 We showed in Exercise 3.9.7 that any nonzero element of a given finite field has the same prime additive order. **Cauchy's Theorem** in algebra (which we won't prove here) says that if any prime  $q$  divides the the number of elements, there must exist an element of order  $p$ . Using these two facts, prove that if a finite field has characteristic  $p$ , it must have exactly  $p^e$  elements where  $e$  is some positive natural.

### 3.9.5 Problems

- P3.9.1 (uses Java) In Problem 3.5.6 we showed that if  $p^e$  is a prime power (that is,  $p$  is a prime number and  $e$  is a positive natural), then  $\phi(p^e) = (p - 1)p^{e-1}$ .
- (a) Using the analysis in this section, explain how to calculate  $\phi(r)$  for any  $r$ , given its prime factorization.
  - (b) Use your method to calculate  $\phi(300)$ ,  $\phi(320)$ ,  $\phi(323)$ ,  $\phi(329)$ ,  $\phi(343)$ ,  $\phi(350)$ ,  $\phi(353)$ , and  $\phi(360)$ .
  - (c) Write a real-Java static method `int phi(int n)` that returns  $\phi(n)$  given any positive natural  $n$  as input.
- P3.9.2 Let  $p$  be a prime number, and define  $\mathbf{Z}_p[x]$  to be the set of all *polynomials* in one variable  $x$  with coefficients in  $\mathbf{Z}_p$ . (Such a polynomial is a sum of zero or more **monomials**, each of the form " $ax^i$ " for some element  $a$  of  $\mathbf{Z}_p$  and some number  $i$ .) We saw in Section 3.6 that we could perform the Euclidean Algorithm on polynomials and prove a form of the Inverse Theorem. We defined a polynomial to be **monic** if its highest degree coefficient is 1, and a monic polynomial to be **irreducible** if it is not the product of two monic polynomials, neither of them equal to 1. If  $f(x)$  is any polynomial, we can define a congruence relation modulo  $f(x)$ , and the equivalence classes of this relation form an algebraic system. Prove that if  $f(x)$  is irreducible, then this system is a field. (The system is finite (Why?) and so it is a finite field. In fact, any finite field is isomorphic to a field of this form, though to prove this would be beyond the scope of this book.)
- P3.9.3 (uses Java) Write a Java program that inputs positive numbers  $a$ ,  $b$ , and  $c$  and computes  $a^b$  modulo  $c$ . Use repeated squaring, so that you loop once for each bit in the binary expansion of  $b$ , rather than, say, looping  $b$  times. Also make sure that your program has integer overflow only when necessary.



- P3.9.4 Compute  $a^{560}$  modulo 561 for several values of  $a$ . Is 561 prime? Can you prove anything about  $a^{560}$  modulo 561 for all values of  $a$ ?
- P3.9.5 Let  $p$  be an odd prime number. We asserted in Problem 3.4.3 that  $-1$  is a perfect square modulo  $p$  (the square of some number, calculated in  $\mathbf{Z}_p$ , also called a quadratic residue) if and only if  $p \equiv 1 \pmod{4}$ . Prove this claim, using the Theorem from this section about generators for the multiplicative subgroup. Can you devise a test that inputs any number in  $\mathbf{Z}_p$  and decides whether it is a perfect square?
- P3.9.6 Consider the congruence relation on  $\mathbf{Z}_3[x]$  modulo the polynomial  $x^2 + x + 1$  and construct addition and multiplication tables for the set  $D$  of nine congruence classes as in Exercise 3.9.3 above. Prove that  $D$  is a ring but *not* a field. Look at the other seven monic polynomials of degree 2 over  $\mathbf{Z}_3$  and determine which of them produce sets of congruence classes that are fields. Can you formulate a rule for determining which do and which do not? Can you prove this rule correct?
- P3.9.7 Show that a finite ring  $R$  (a commutative ring with identity) that is *not* a field must have zero divisors. (**Hint:** Let  $a$  be any nonzero element that has no inverse. What can you say about the set  $\{ab : b \in R\}$ ?)
- P3.9.8 We defined **monic** and **irreducible** polynomials (in one variable  $x$ ) above in Problem 3.9.2. Here we consider such polynomials over  $\mathbf{Z}_2$ .
- Show that there are exactly  $2^d$  different monic polynomials of degree  $d$  over  $\mathbf{Z}_2$ .
  - Show that exactly one of the quadratic monic polynomials is irreducible.
  - If  $f$  is any polynomial of degree at least 2, consider the set of four polynomials  $\{f + ax + b : a, b \in \mathbf{Z}_2\}$ . Show that at most one of these four polynomials is irreducible. (**Hint:** Consider the value of the polynomial with  $x = 0$  and  $x = 1$ . If either value is zero, the polynomial has a root.)
  - Part (c) shows that there are at most four irreducible monic polynomials of degree 4 over  $\mathbf{Z}_2$ . Are there exactly four?
- P3.9.9 Following the reasoning in Problem 3.9.8, find the exact number of monic irreducible polynomials of degree  $d$  over  $\mathbf{Z}_3$ , for  $d = 2$ ,  $d = 3$ , and  $d = 4$ .
- P3.9.10 Let  $a$  be a generator of a finite field  $F$  with characteristic  $p$  and  $p^e$  elements (following Exercise 3.9.7).
- Prove that an element  $b$  is a generator if and only if  $b^{p^e} = b$  and  $b^j \neq b$  for all  $j$  with  $1 < j < p^e$ .
  - Prove that  $b = a^p$  satisfies  $b^{p^e} = b$ .
  - Finish the proof that  $b$  is a generator. (**Hint:** Since  $p$  and  $p^e - 1$  are relatively prime, we know that  $p$  has an inverse modulo  $p^e - 1$ . And since  $c^{p^e-1} = 1$  for all  $c$  in  $F$ , the value of  $b^j$  depends only on  $j$ 's remainder modulo  $p^e - 1$ .)

### 3.10 Excursion: Certificates of Primality

I claim that the number 1756205519 is composite. Why should you believe me? In Section 3.1 we discussed a method to test whether an arbitrary natural is prime — dividing it by all numbers (or just all primes) up to its square root. With a computer, you could do this for 1756205519, but it would take far too long by hand. With a number of 100 rather than 10 digits, even a computer wouldn't help you very much.

If I tell you, though, that 1756205519 is the product of 50173 and 35003, you don't have to take this on faith, because you can multiply these two numbers by hand and check the answer against 1756205519. If they match (and they should), you would now have *proof* that 1756205519 is composite.

It's not easy to take a large composite number and find two factors for it (in fact, in the next section we'll see how the security of the RSA cryptosystem depends on this being difficult). But if you're given the factors, they provide convincing evidence of compositeness that is easy to check. We call such evidence a **certificate**. It may be very hard to *find* a certificate, and so the existence of certificates for a property doesn't automatically make the property easy to test.

What sort of a certificate could convince you quickly that some 10-digit number is *prime*? We have an algorithm to test the primality of the number, but if the number is large enough we won't have enough time for this algorithm to finish<sup>54</sup>. In this Excursion, we'll see a trick for giving certificates (due to Pratt) that uses the number theory we developed in Section 3.9.

We've just shown that if a number  $n$  is prime, then  $a^{n-1} \equiv 1 \pmod{n}$  for any number  $a$  relatively prime to  $n$  (for example, for any number in  $\mathbf{Z}_n^*$ ). This is something we can check (with a computer, anyway) because we can use repeated squaring to limit the number of multiplications to the number of bits in the binary expansion of  $n$ , and we don't have to keep around any numbers larger than  $n$ . For small  $n$ , we can even work by hand.

As an example, let's calculate  $2^{100}$  modulo 101.  $2^1 = 2$ ,  $2^2 = 4$ ,  $2^4 = 16$ ,  $2^5 = 32$ ,  $2^{10} \equiv 32 \cdot 32 \equiv 1024 \equiv 14 \pmod{101}$ ,  $2^{20} \equiv 14 \cdot 14 \equiv 196 \equiv 95 \pmod{101}$ ,  $2^{25} \equiv 95 \cdot 32 \equiv 3040 \equiv 10 \pmod{101}$ ,  $2^{50} \equiv 10 \cdot 10 \equiv 100 \pmod{101}$ , and  $2^{100} \equiv (-1) \cdot (-1) \equiv 1 \pmod{101}$ .

If  $a^{n-1}$  modulo  $n$  ever fails to equal 1 for any  $a$ , we have convincing evidence that  $n$  is composite. But what do we really *know* if we try a bunch of  $a$ 's and keep getting 1? For most<sup>55</sup> composite numbers, for most choices of  $a$ , you're not going to get 1. But some choices of  $a$  (like  $a = 1$ , for example) will always get you 1, and there are some unusual composite numbers<sup>56</sup> such as 561 for which you *always* get 1 for any  $a$  in  $\mathbf{Z}_n^*$ . This means that just getting 1 a lot is no proof that the number is prime.

There is a way around this problem — it turns out (for reasons we won't prove here) that for any

<sup>54</sup>The new primality test of Agrawal *et al.* takes time proportional to about  $n^{12}$  to test an  $n$ -digit number.

<sup>55</sup>It's rather tricky to come up with a formal meaning for "most" in this context, but it does make sense.

<sup>56</sup> $561 = 3 \cdot 11 \cdot 17$ , so by the Chinese Remainder Theorem,  $\mathbf{Z}_{561}^*$  is isomorphic to a direct product  $\mathbf{Z}_3^* \times \mathbf{Z}_{11}^* \times \mathbf{Z}_{17}^*$ . The numbers  $\phi(3)$ ,  $\phi(11)$ , and  $\phi(17)$  all just happen to divide 560, so raising any  $a \in \mathbf{Z}_{561}^*$  to the 560'th power will get you 1 in all three components, which is 1. Numbers with this property are called **Carmichael numbers**.

composite number  $n$ , at least *half* the possible  $a$ 's have a property that gives away the compositeness of  $n$ . A few, of course, aren't even in  $\mathbf{Z}_n^*$  and can't have  $a^{n-1} \equiv 1 \pmod{n}$ . But for the rest, consider calculating  $a^{n-1}$  modulo  $n$  by repeated squaring, by writing  $n-1 = 2^j i$  for some odd number  $i$ , getting  $a^i$  modulo  $n$ , and squaring it  $j$  times to get  $a^{n-1}$ . If you don't end up with 1, you've proved compositeness. The fact we won't prove is that for at least half of those  $a$  such that  $a^{n-1} \equiv 1 \pmod{n}$ , you first get to 1 during the repeated squaring by squaring a number other than  $-1$ . And of course if this happens,  $n$  must be composite, because you have found a third root for the equation  $x^2 - 1 = 0$  in  $\mathbf{Z}_n$ , and thus  $\mathbf{Z}_n$  isn't a field.

This doesn't help us to *prove* that  $n$  is prime, but it allows us to pile up very good practical evidence. If for 100 different values of  $a$ , this compositeness test *fails* to show that  $n$  is composite, there are two possibilities. Either  $n$  is really prime, or all 100 values happened to be among the set for which the test fails. If the values were chosen randomly, and we know that the test works at least half the time, the latter is very unlikely<sup>57</sup> and you have a lot of confidence that  $n$  is prime. This is how large numbers are tested for primality in practice. There are primality tests that don't have this small residue of doubt in their conclusions, but so far the best ones known still take a time that is a large polynomial in  $n$  to test  $n$ -digit numbers.

We can, however, construct **certificates for primality** using the number theory from this chapter. Remember that if  $n$  is prime, there is a **generator** of  $\mathbf{Z}_n^*$ , a number  $g$  such that every element of  $\mathbf{Z}_n^*$  can be written  $g^i$  for some  $i$  (equivalently, an element of order  $n-1$ ). No composite number  $n$  could possibly have an element with order  $n-1$ , because if  $n$  is composite there aren't  $n-1$  different elements in  $\mathbf{Z}_n^*$ . So if you believe that  $g$  has order  $n-1$ , you should be convinced that  $n$  is prime.

How can you be sure that  $g$  has the claimed order? Let's look at the example above, because it happens that 2 is a generator for  $\mathbf{Z}_{101}^*$ . The first requirement is that  $2^{100} \equiv 1 \pmod{101}$ , which we checked. This tells us that the order of 2 *divides* 100. If it were less than 100, it would have to be missing at least one of the prime divisors of 100, and hence would have to divide either 20 or 50. But along the way to computing  $2^{100}$  modulo 101, we found that neither  $2^{20}$  nor  $2^{50}$  is equal to 1 modulo 101. So 2 has order 100, and thus we know that 101 is prime.

To certify  $n$  as prime, then, we need the following:

- A number  $g$  such that  $g^{n-1} \equiv 1 \pmod{n}$ .
- A factorization of  $n-1$  into primes  $p_1, \dots, p_k$ .
- A check that none of  $g^{(n-1)/p_1}, g^{(n-1)/p_2}, \dots, g^{(n-1)/p_k}$  is congruent to 1 modulo  $n$ .
- Proof that the numbers  $p_1, \dots, p_k$  are really all prime.

The last evidence is necessary to keep us from being fooled by a fraudulent certificate which tried to slip an incorrect factorization of  $n-1$  past us. We can check that the alleged factors multiply to

<sup>57</sup>At least as unlikely as 100 consecutive flips of a fair coin all being heads.

$n - 1$ , but we need them to be prime if the certificate is to be valid. Of course, this only reduces the original problem into another instance of the same problem, raising the possibility that proving the original  $n$  will require proving more numbers prime, which themselves could require more numbers, and so on. We have shown that certificates exist, but we have a bit more work to do to show that they are *short*.

The largest prime factor of  $n - 1$  is at worst  $(n - 1)/2$ , which limits the *depth* of our recursion to the number of bits in the binary expansion of  $n$ , which we'll call  $t$  (you can only halve a number about that many times and still have anything left). The number of new primes that can show up on each level of the recursion is also limited to  $t$ , meaning that we can certify a  $t$ -bit prime by recursively certifying at most  $t^2$  other primes. This keeps the length of the certificate, and hence the number of steps to check a certificate, at a reasonable number even for 100-digit or 1000-digit primes. *Finding* the certificate for a really large number might be far more time-consuming. Generators are fairly common, as it happens, so you might hope to get one by repeated guessing, but you also need to *factor* some composite numbers of almost the same length, and there is no efficient way known to do this.

### 3.10.1 Writing Exercise

Here is an example of a problem that can be solved reasonably quickly by hand, illustrating the method clearly. If the students have access to more computing power during the exercise, it is easy to construct examples with larger numbers.

Give a complete certificate proving that 103 is a prime number. You must find a generator of  $\mathbf{Z}_{103}^*$ , factor 102, prove that your generator really is a generator, and recursively certify the prime factors of 102. (Don't worry about certifying that 2 is prime.)

|            |   |   |    |    |    |     |     |
|------------|---|---|----|----|----|-----|-----|
| plaintext  | G | A | L  | L  | I  | A   | ... |
| number     | 6 | 0 | 10 | 10 | 8  | 0   | ... |
| add three  | 9 | 3 | 13 | 13 | 11 | 3   | ... |
| ciphertext | K | O | O  | M  | D  | ... |     |

©Kendall Hunt Publishing Company

Figure 3-7: Encrypting with the Caesar cipher.

## 3.11 The RSA Cryptosystem

### 3.11.1 An Introduction to Cryptography

We'll conclude this chapter by presenting an application of number theory to **cryptography**, the transmission of secret messages. First, a bit of background. Our goal in setting up a **cryptosystem** is to take a message, called a **plaintext**, and convert (**encrypt**) it into another form, called a **ciphertext**, such that

- The intended recipient will be able to convert (**decrypt**) it back to the plaintext, and
- Anyone else intercepting the ciphertext will find it difficult or impossible to decrypt.

There are any number of ways to do this (invisible ink, hiding the message inside something, and so forth) but we are interested in the mathematical methods known as **ciphers** where we can describe the encryption and decryption as functions from strings to strings. One of the earliest ciphers is the **Caesar cipher** (used by Julius Caesar in the first century B.C.), and is easy to describe using number theory.

Consider the 23 letters of the Latin alphabet<sup>58</sup> and map them in the obvious way to the numbers in  $\mathbf{Z}_{23}$ . So A is represented by 0, B by 1, and so on until Z is represented by 22. This is a sort of cipher in itself, but it's not likely to fool anybody. Caesar then, in our terminology, encrypted his message (a sequence of numbers in  $\mathbf{Z}_{23}$ ) by adding 3 to each number modulo 23, and converting the number back to a letter. So the plaintext GALLIA EST OMNIS DIUISA IN PARTES TRES becomes the ciphertext KDOOMD HXY RPQMX GMZMXD MQ SDUYHX YUHX (see Figure 3-7). The intended recipient could decrypt by converting to numbers, subtracting 3 from each, and converting back to letters<sup>59</sup>.

This is called a **single-letter substitution cipher**, of a particularly simple form called a **rotation**. Another example with English text is the **rot13** cipher<sup>60</sup>, which adds 13 to each letter, viewed as

<sup>58</sup>Latin had no W, and didn't distinguish between the letters I and J, or between U and V. K, Y, and Z were very rare, being needed only for loan-words from other languages such as Greek.

<sup>59</sup>Of course, it's simpler to just count back three letters, which is how they would have thought of it, but it will be useful for us to keep  $\mathbf{Z}_{23}$  in mind.

<sup>60</sup>The Usenet newsgroup **rec.humor.funny** (still in existence at [www.netfunny.com](http://www.netfunny.com)) has a policy of displaying particularly offensive jokes encrypted using **rot13**, so that the reader has to do something (typically enter a single keystroke) to decrypt it. That way the reader can't complain about being forced to read an offensive joke.

a number in  $Z_{26}$ . Conveniently, the encryption and decryption algorithms for **rot13** are the same (Why?).

Single-letter substitution ciphers are convenient to use but not terribly **secure**, in that they can be easy for an enemy to **break** (decrypt without prior knowledge of the key). If you intercept a message that you know to be the ciphertext from a rotation, you could even try *all* the possible rotations, and trust to the fact that only one of them would look like a proper plaintext (for example, you don't need to know much about Latin to see that the ciphertext above isn't in Latin). A general single-letter substitution cipher can be more difficult, but is subject to certain kinds of analysis.

If you intercepted one of Caesar's letters, and you believed it to be written in a single-letter substitution cipher, a good first step might be to count the number of occurrences of each letter in the ciphertext — the **single-letter frequencies**. You would likely find that H was the most common letter, because E is the most common letter in Latin plaintext, as it is in English<sup>61</sup>. By matching various combinations of common ciphertext letters and common plaintext letters, you would likely be able to get partial words, which would give clues as to other letters, and so forth.

Such “cryptograms” are a common recreational puzzle in English, though there are two significant differences between the puzzle situation and real-world cryptography. In the former, the designer of the puzzle will frequently choose a short message with very different letter frequencies (e.g., no E's at all) to confound potential frequency analysis. But the puzzles usually retain spaces between words and even punctuation, important clues that in a real secret message would usually be omitted. (It's not hard for an English speaker to read “WHENI NTHEC OURSE OFHUM ANEVE NTSIT. . .”, for example, but it makes it very easy for the cryptanalyst if they see a one-letter word and know it has to be A or I.)

A natural idea to make frequency analysis more difficult is to *change keys* according to some fixed pattern. A simple version of this is the **keyword polyalphabetic cipher**. Let's return to Caesar's message. He and the intended recipient need to agree on a short keyword, say, “SPQR”<sup>62</sup>. Caesar writes this keyword repeatedly under his plaintext, like this:

GALLIA EST OMNIS DIUISA IN PARTES TRES  
SPQRSP QRS PQRSP QRSPQR SP QRSPQR SPQR

and then “adds” the two letters in each column, using the addition in  $Z_{23}$  and the representation of letters as numbers (for example, in the first column G (6) plus S (17) is A (0) modulo 23). The ciphertext is the string of sums:

APCDCP ULN EDFCI TBOZKR CD GRLKUL NHUL

---

<sup>61</sup>Book I of Caesar's memoir *De Bello Gallico*, for example, contains 5912 E's (12.2%), 5429 I/J's (11.2%), and 4581 U/V's (9.5%). It has one K, one Y, and no Z's.

<sup>62</sup>An abbreviation for the Latin meaning “the Senate and people of Rome”, the formal name of the Roman state.

The recipient writes the keyword repeatedly under the ciphertext, and *subtracts* the second letter in each column from the first to get the plaintext (for example A (0) minus S (17) is G (6)):

```
APDCDP ULN EDFCI TBOZKR CD GRLKUL NHUL
SPQRSP QRS PQRSP QRSPQR SP QRSPQR SPQR
-----
GALLIA EST OMNIS DIUIA IN PARTES TRES
```

Note that in general, different copies of the same letter in the plaintext will be encrypted into different letters in the ciphertext (for example, the four I's turned into two C's, a B, and a Z). This will tend to make the ciphertext letters more evenly distributed, which makes analysis harder, but there is still hope, especially if the cryptanalyst can guess the length of the keyword. The longer the keyword, the harder the analysis. How about using an entire long text as the keyword, so it need never repeat?

```
GALLIA EST OMNIS DIUIA IN PARTES TRES
ARMAUI RUM QUECA NOTROI AE QUIPRI MUSA
-----
GRYLEI XOG FHRLS QYPBHI IR GUBKXC GNYS
```

This is a lot better, but a determined analysis on enough ciphertext might still be able to break it, especially knowing that both the plaintext and the key were Latin text (for example, you would expect a lot of letters in each to be E's, I's, and U's, so the sums of those letters should come up a lot). The ideal thing would be to have a key that was *completely random*. Simply generate a sequence of random letters (perhaps by throwing a 23-sided die) as long as the desired message, and use *that* as the key.

The resulting cryptosystem is *unbreakable in principle*. Because any possible string of the right length could be generated by the random process, and no key string is any more likely than any other, no *ciphertext* is any more likely than any other. Any plaintext could produce any possible ciphertext, so that knowing the ciphertext gives you no help in determining the plaintext<sup>63</sup>. Of course, if you use the same random key for two different plaintexts, this is no longer the case. (For example, what happens if you subtract one ciphertext from the other?) For this reason, this cryptosystem is called a **one-time pad**.

You may have noticed a significant logistical problem with the one-time pad. If Caesar wants to use it to send a message from Rome to Queen Cleopatra in Egypt, say, he has to somehow see that he and Cleopatra have the same random key. But sending Cleopatra the key is *exactly* as difficult as sending her a message in the first place. For this reason, the one-time pad is only practical when you know in advance that you want someone to send you a message later, and you can give them the key before they go.

---

<sup>63</sup>This is an example of an argument in **information theory**, a field that is the subject of Chapter 13 of this book. You can get an excellent introduction to information theory by reading *The Mathematical Theory of Communication* by Shannon and Weaver, the 1948 book that first set out the theory.

You can give up some security and use a key that isn't quite random. For example, in the example above the "key" was in effect "use the *Aeneid*, starting at line 1 of book 1" which transmits a long key as long as the recipient can find a copy of the *Aeneid*<sup>64</sup>.

A better approach, at a higher level of technology of course, was used by the Germans in World War II<sup>65</sup>. The Enigma machine was a device where typewriter keys were connected to electric lights through wires that ran through a collection of wheels. Push a key for a plaintext letter, and the light for the ciphertext letter lights up. After each keystroke, the wheels move so a *different* alphabet substitution is used for the next letter. The sender and receiver have to agree on the initial setup of the wheels for each day's transmissions. In effect, the interaction of the wheels made for a sort of keyword cipher<sup>66</sup> with a very long key. What the Germans didn't know, however, was that the British had obtained<sup>67</sup> a copy of the machine and were able to read the messages if they could guess or deduce the initial setup. A team of experts<sup>68</sup> used a combination of brute-force search (inventing machines which were among the precursors of electronic computers) and exploitation of German procedural errors to get many of these daily initial setups, revealing a great deal of militarily important information to the Allies.

### 3.11.2 Public-Key Cryptography

With the advent of computers, more complicated ciphers became practical, as did more complicated techniques of analysis. Typically, they still work by having a relatively short secret piece of information, the **key**, known to the sender and receiver but kept secret from potential eavesdroppers. Ideally, the cipher will be:

- sufficiently hard to break in the absence of the key, and
- have so many different possible keys that trying all possible keys is impractical (remember that in general only the correct key will yield intelligible output).

Getting the key to the receiver, of course, is still a major problem. A novel solution was developed in the 1980's, however, called **public-key cryptography**<sup>69</sup>. Here's how it works. Suppose Caesar wants to send a secret message  $w$  to Cleopatra, and has a reasonably powerful computer available. He looks up Cleopatra's **public key**  $k$  in some kind of public directory, and computes his ciphertext  $c$  as some function of  $w$  and  $k$ , called  $e(w, k)$ . Cleopatra has her own **private key**  $s$  that she has kept secret, and applies a decoding function  $d$  to  $c$  and  $s$ , getting back  $d(c, s) = w$ . The important properties of the system are:

<sup>64</sup>Which might have been difficult for Julius Caesar, as the *Aeneid* was written well after his death. . .

<sup>65</sup>An excellent book on the history of cryptography, especially in World War II, is *The Codebreakers* by David Kahn.

<sup>66</sup>Although the individual ciphers for each letter were not rotations, as they were in our example.

<sup>67</sup>From some Polish researchers who had, amazingly, reverse-engineered it based on documents stolen from the Germans by the French.

<sup>68</sup>Including Alan Turing, about whom we'll hear more in Chapter 15.

<sup>69</sup>The concept of public-key cryptography was first invented by Diffie and Hellman. We'll describe the particular system in widest use today, called "RSA" after its inventors: Rivest, Shamir, and Adleman.



- The functions  $e$  and  $d$  are easy to compute.
- It is easy to generate pairs of keys  $k$  and  $s$  such that for any message  $w$ ,  $d(e(w, k), s) = w$ .
- It is difficult for someone who knows  $e(w, k)$  and  $k$ , but not  $s$ , to determine  $w$ .

The **RSA cryptosystem** is an implementation of public-key cryptography using number theory. Here is how Cleopatra would get her public and private keys. She first generates two large, random prime numbers  $p$  and  $q$ , by generating random numbers of the desired size and testing them for primality<sup>70</sup> using the methods discussed in Excursion 3.10. What's the right size? She is going to keep  $p$  and  $q$  secret and publish  $n = pq$ , so the two primes must be so large that an enemy doesn't have time to run any factoring algorithm<sup>71</sup> on  $n$  — 200 digits each for  $p$  and  $q$  is currently considered secure<sup>72</sup>.

Remember that by the Chinese Remainder Theorem, the multiplicative group  $\mathbf{Z}_n^*$  is a direct product of  $\mathbf{Z}_p^*$  and  $\mathbf{Z}_q^*$ , with  $\phi(n) = (p-1)(q-1)$  elements. We're going to use the fact that any element  $a$  in  $\mathbf{Z}_n^*$  satisfies the congruence  $a^{\phi(n)} \equiv 1 \pmod{n}$ . This means that the value of  $a^b$  modulo  $n$  depends only on the value of  $b$  modulo  $\phi(n)$ . Cleopatra knows the values of  $p$  and  $q$  and can compute  $\phi(n)$  easily, while an enemy has no obvious way to do this.

Cleopatra's encoding key consists of  $n$  and  $e$ , where  $e$  is any<sup>73</sup> number that is relatively prime to  $\phi(n)$ . Using the Euclidean Algorithm as in the proof of the Inverse Theorem, she finds an inverse for  $e$  modulo  $\phi(n)$ , and stores it secretly as  $d$ .

Caesar's instructions are to convert his message into numbers in  $\mathbf{Z}_n$  by any method known to both (for example, "GALLIAESTOMNIS..." could just become the decimal "06001010080004...", using the representation of letters by numbers above and just concatenating the decimal representations). Each message  $w$ , then, is a single element of  $\mathbf{Z}_n$ . The encrypting algorithm is to convert  $w$  to  $c = w^e$ , with the computation being done in  $\mathbf{Z}_n$  by repeated squaring. The decrypting algorithm, which requires Cleopatra's secret information, is to convert  $c$  to  $c^d$ , also in  $\mathbf{Z}_n$ . By the choice of  $d$  and  $e$ , Cleopatra winds up with  $w^{ed}$ , which is equal to  $w$  in  $\mathbf{Z}_n$  because  $ed \equiv 1 \pmod{\phi(n)}$ .

An enemy could break the cipher by factoring  $n$  and thus computing  $\phi(n)$ . It is conceivable that there is another way to break the cipher without factoring  $n$ , which could be implemented quickly. (At least one other proposed public-key cryptosystem, based on a different hard problem, has failed in this way.) No one knows how such an alternate attack on RSA might work, and there is some hope that if one exists it might first be discovered by a mathematician who would publish it, rather than a cryptanalyst who would keep it secret and exploit it. The same holds for factoring — an

<sup>70</sup>There is a very tiny chance that these primality tests will certify a composite number as prime, but this can be made as small as the chance of an enemy just guessing the correct secret key and so needn't be worried about.

<sup>71</sup>This means that she has to know how much computing power and what factoring algorithms her enemy might have. She might want to check the latest results of the public factoring contests, to see what the state of the art is. But an enemy who had a secret, dramatically better factoring algorithm might be able to read her messages.

<sup>72</sup>About one in every 500 200-digit naturals is prime, so it shouldn't take too long to find two of them by guessing and testing random naturals of that length.

<sup>73</sup>The number of such  $e$  less than  $\phi(n)$  is  $\phi(\phi(n))$ , of course, and depends on the prime factorization of  $\phi(n)$ , about which we know very little. We'll look at this number in the Exercises, but note that should Cleopatra have any great trouble finding such an  $e$  by trying random numbers, she could always go find a different  $p$  and  $q$ . Of course testing that  $e$  and  $\phi(n)$  are relatively prime is easy for her by the Euclidean Algorithm since she knows  $\phi(n)$ .

advantage of RSA is that its security depends on mathematical facts and conjectures that are open to public discussion.

RSA serves as a component of a cryptosystem called “PGP” (for “Pretty Good Privacy”), that has become widely used on the Internet. Because the computations of RSA take unacceptably long for routine communications, the PGP system uses RSA to send a short key which is then used in an ordinary cipher. The cipher is only “pretty good” because the numbers involved are small enough that a determined computer attack could break the cipher by known methods. But computer privacy activists argue that if a significant fraction of routine email is encrypted with even a pretty good cipher, anyone wanting to monitor communications on the Internet would be stymied by the need to determine *which* of millions of encrypted messages were worth decrypting. Thus they put a “PGP public key” (represented as a few lines of random-appearing text characters) at the end of all their messages<sup>74</sup>.

### 3.11.3 Exercises

E3.11.1 Decrypt the following Latin messages from the original Caesar cipher, as described above:

- (a) FRKMYR, HUKR XZP.
- (b) H SOZUMEZX XQXP.
- (c) HY YZ, EUZYH?
- (d) ZHQM, ZMGM, ZMFM.
- (e) URPDQM MYH GRPZP.

E3.11.2 Show that the function that takes each element  $x$  of  $\mathbf{Z}_m$  to the element  $ax + b$  is a bijection if  $a$  is relatively prime to  $m$  and  $b$  is any number. What is the inverse of this function? What happens if  $a$  and  $m$  have a common factor?

E3.11.3 (uses Java) Write a method `rotate(String w, int k)` that outputs an encrypted version of `w` using a single-letter substitution cipher as follows. Small letters in `w` should be converted to other small letters obtained by adding `k` to their numerical value (`a = 0, z = 25`) modulo 26. Capital letters should be converted to capital letters using the same system. Other characters should be unchanged. Then write a method `unrotate (String w, int n)` so that the `rotate` and `unrotate` functions for a fixed `n` are inverses. (Hint: The easiest way to write `unrotate` is to have it make a single call to `rotate`.)

E3.11.4 The ASCII encodings for the capital letters A through Z are the numbers 65 through 90 (in decimal notation). Encode the message “ALOHA” as a sequence of five numbers using ASCII, and then encode this sequence in RSA using  $n = 143$  and  $e = 113$ .

E3.11.5 Using  $n = 143$  and  $e = 113$  as in Exercise 3.11.4, decrypt the sequence  $\langle 65, 98, 65, 66, 65, 77, 65 \rangle$  into a string of ASCII letters. (You should use your ability to factor  $n$  to find the “private” decryption key.) Repeat for the sequence

$$\langle 66, 118, 15, 50, 18, 45, 112, 71, 128, 49, 114 \rangle.$$

Would a long message in this cipher be difficult to decrypt without knowledge of  $n$  and  $e$ ?

<sup>74</sup>For more on the cryptography policy debate in the USA at the turn of the millenium, see *Privacy on the Line* by Diffie and Landau.

|   |   |   |   |   |
|---|---|---|---|---|
| R | H | I | N | O |
| C | E | S | A | B |
| D | F | G | K | L |
| M | P | Q | T | U |
| V | W | X | Y | Z |

©Kendall Hunt Publishing Company

Figure 3-8: A Polybius square with keyword RHINOCEROS.

E3.11.6 Suppose that I and my friend have established RSA public keys and I would like to send a message to them. I don't mind if eavesdroppers read the message, but I want to be sure that my friend knows that I, and no one else, sent the message. (This is referred to as my message being **digitally signed**). Assuming that RSA is secure, how can I do this?

E3.11.7 A **Polybius square** is an arrangement of 25 letters<sup>75</sup> into a  $5 \times 5$  grid, which matches each letter to a pair  $(i, j)$  with  $1 \leq i, j \leq 5$ . A common arrangement of the letters is to choose a **keyword**, list the letters of the keyword first (omitting repeated letters), then follow with the rest of the letters in alphabetical order. Figure 3.8 shows a Polybius square with keyword "RHINOCEROS".

- One simple way to use a Polybius square in a cipher is to replace each plaintext letter with the pair of numbers giving its grid position, so that POLYBIUS would be encoded as 4215355425134523. What is the plaintext that gives the ciphertext 222342131514243322?
- Can you think of a setting in which transmitting numbers in  $\{1, 2, 3, 4, 5\}$  would be easier than transmitting letters from the whole alphabet?
- We can also use the square to form a keyword polyalphabetic cipher, for example by translating both the plaintext and keytext into strings from  $\{0, 1, 2, 3, 4\}$ , then adding each pair of matching symbols modulo 5, with no carries. With the square from Figure 2.8 (now with rows and columns indexed 0 to 4) and the keyword GRID, what is the plaintext producing the ciphertext 0204000310314003230400?

E3.11.8 The **rail fence** cipher is a simple example of a **transposition cipher**, where the letters of the ciphertext are just the letters of the plain text in another order. If  $r$  is a natural, the rail fence cipher with  $r$  rails assigns the first letter of the plaintext to rail 1, the second to rail 2, and so on, until letter  $r$  is assigned to rail  $r$ . Then letter  $r + 1$  goes to rail  $r - 1$ , letter  $r + 2$  to rail  $r - 2$ , and so on until letter  $2r - 1$  goes to rail 1. The process then repeats, with letter  $2r$  going to rail 2, and eventually letter  $3r - 2$  goes to rail  $r$ , letter  $4r - 3$  to rail 1, and so forth. The letter assigned to each rail form a string in the order they occur in the plaintext, and the ciphertext is the concatenation of the  $r$  rail strings.

- The string NESAN TOYXT TPNIQ IIBDP CHSIH USOOE ESIN is English text encoded with the rail fence cipher. How many rails were used, and what is the plaintext? (Ignore the spaces in the ciphertext, which are there for readability only. As usual, you have to figure out where the spaces go in the plaintext.)

<sup>75</sup>Polybius himself had only to fit the 24 Greek letters into his  $5 \times 5$  array, but with our alphabet we will again treat I and J as the same letter.

(b) In the same way, decode the ciphertext **IAOEN HGONT EEIEA OBTNH LITER UDHRL VDHBI**.

- E3.11.9 Let  $p$  be a prime number and let  $g$  be a generator of  $\mathbf{Z}_p^*$ . If  $A$  is a nonzero element of  $\mathbf{Z}_p^*$ , there is exactly one number  $a$  with  $0 \leq a < p - 1$  such that  $A \equiv g^a \pmod{p}$ . We call  $a$  the **discrete logarithm** of  $A$  with respect to  $p$  and  $g$ . If  $a$  and  $b$  are the discrete logarithms of  $A$  and  $B$  respectively, prove that  $A^b = B^a$ .
- E3.11.10 (uses Java) Write a static real-Java method `long discreteLog (long p, long g, long a)` that returns the discrete logarithm of  $a$  with respect to  $g$  (as defined in Exercise 3.11.9), assuming that  $p$  is prime and that  $g$  is a generator. For how large a  $p$  does your method return an answer within ten seconds?

### 3.11.4 Problems

- P3.11.1 To pick a key for RSA, one needs to find a number relatively prime to  $\phi(n)$ . Is it possible for  $\phi(\phi(n))$  to be so small that it would be time-consuming to find such a number?
- (a) Explain how to compute the ratio  $\phi(x)/x$  for an arbitrary number  $x$ , given its prime factorization.
  - (b) Determine this ratio for  $x = 6469693230$ . Explain why this number has the lowest such ratio for any ten-digit number.
  - (c) It is a fact, the proof of which is beyond the scope of this book, that the sum of all prime numbers less than  $x$  is fairly close to  $e^x$  (where “ $e$ ” is the number  $2.71828\dots$ , familiar from calculus). Assuming this fact, estimate the smallest possible value for  $\phi(x)/x$  for any number  $x$  with at most 400 decimal digits.
  - (d) By the same fact (called “The Prime Number Theorem”), the fraction of 400-digit numbers that are prime is about the natural logarithm (logarithm to the base  $e$  of  $x$ , written  $\ln(x)$ ) of  $10^{400}$ . Assuming this, how does the difficulty of finding a 400-digit prime compare with finding a number relatively prime to a given 400-digit number?
- P3.11.2 Why is having a plaintext of 0 or 1 in RSA is a bad idea? Are there other plaintext numbers to avoid? Is this likely to be a problem with a 400-digit  $n$ ?
- P3.11.3 (uses Java) Write a class `KeywordCipher` with encryption and decryption methods for a keyword cipher. The keyword should be a class variable and should be a string of small letters. The encryption method should use the keyword to pick the second argument of the `rotate` method from Exercise 3.11.3, and apply this method to the given plaintext letter.
- P3.11.4 (uses Java) Write a real-Java method that inputs a file of text and determines the **frequency table** of the letters in it, treating capital and small letters the same. Test your method on some English text of more than a page and report the answers.
- P3.11.5 (uses Java) A **digram** is a string of two letters. Write a method that counts the number of occurrences of each digram in its input text. Test your method on a text of more than a page and report the ten most common digrams.
- P3.11.6 Suppose you know that  $n = pq$  and  $r = \phi(n) = (p-1)(q-1)$  where  $p$  and  $q$  are primes. Show how to find  $p$  and  $q$ . (**Hint:** It is sufficient to find both  $p+q$  and  $p-q$ . The first is easy. To get the second once you have the first, look at the difference between  $(p+q)^2$  and  $(p-q)^2$ .)

P3.11.7 The **Playfair cipher** is a substitution cipher that acts on two-letter pairs rather than on single letters, which protects against attacks based on letter frequency. It is based in part on the Polybius square of Exercise 3.11.7. Consider the positions of the square to be pairs  $\langle i, j \rangle$ , where  $0 \leq i, j \leq 4$ .

We begin by taking the text (with no spaces or punctuation, and I and J identified) and adding an X between every pair of double letters (if XX occurs in the plaintext<sup>76</sup>, insert a Q to make XQX). We then divide the text into pairs (adding an X at the end if needed) and encrypt each pair of letters as another pair of letters as follows:

- If the two letters are in the same row, such as  $\langle i, j \rangle$  and  $\langle i, k \rangle$ , the ciphertext letters are  $\langle i, j + 1 \rangle$  and  $\langle i, k + 1 \rangle$ , where the addition is taken modulo 5.
- If the two letters are in the same column, such as  $\langle i, j \rangle$  and  $\langle k, j \rangle$ , the ciphertext letters are  $\langle i + 1, j \rangle$  and  $\langle k + 1, j \rangle$ , again adding modulo 5.
- Otherwise, if the letters are  $\langle i, j \rangle$  and  $\langle k, l \rangle$ , the ciphertext letters are  $\langle i, l \rangle$  and  $\langle k, j \rangle$  (so that the first letter is moved to a letter in the same *row*).

With the Polybius square from Figure 3.8, the pair EA becomes SB, WH becomes HE, and CL becomes BD.

- Using the given square, decrypt the text NPSCIZ ZIGYAF SWWFHN AERHIV MQQY.
- Which steps of the transformation from original plaintext to ciphertext are bijections?
- How does the existence of the English word DOXOLOGY complicate the decryption?

P3.11.8 (uses Java) Write a real-Java class to implement the Playfair cipher of Problem 3.11.7. You should have a method `String encrypt (String plaintext, String key)` and method `String decrypt (String ciphertext, String key)`.

P3.11.9 Another form of public-key cryptography, originally proposed by Diffie, Hellman, and Merkle, is called **key exchange**. Here the two parties (traditionally named Alice and Bob) exchange messages that allow them to jointly choose a common key in such a way that an eavesdropper seeing all their messages cannot easily learn the key.

- Alice and Bob begin by choosing a prime number  $p$  and a generator  $g$  of  $\mathbf{Z}_p^*$ . How can they be sure that  $p$  is prime and that  $g$  is a generator?
- Alice chooses a random number  $a$ , in the range from 0 to  $p - 2$ , and sends  $A = g^a$  (taken modulo  $p$ ) to Bob. Bob chooses a random number  $b$  in that range and sends  $B = g^b$  to Alice. The secret key is  $k = g^{ab}$ , with the exponentiation again being taken modulo  $p$ . Show that each party can now calculate  $k$  with no additional messages being sent.
- Suppose that an eavesdropper had an algorithm that would produce  $k$  given only the public information  $p, g, A$ , and  $B$ . Show that this algorithm would solve the general discrete logarithm problem of Exercise 3.11.9. (There is no known algorithm for this problem that scales well to numbers with hundreds of digits.)

P3.11.10 The **columnar transposition** cipher is somewhat similar to the rail-fence cipher of Exercise 3.11.8, but is harder to break because it uses a keyword. We choose a keyword of length  $r$  with no repeated letters, then break the plaintext into rows of length  $r$ , thus forming  $r$  columns.

---

<sup>76</sup>Important if the plaintext concerns the Exxon/Mobil corporation and/or baseball legend Jimmy Foxx.

|               |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|
| Keyword:      | L | E | O | P | A | R | D |
| Letter order: | 4 | 3 | 5 | 6 | 1 | 7 | 2 |
|               | E | M | M | A | W | O | O |
|               | D | H | O | U | S | E | H |
|               | A | N | D | S | O | M | E |
|               | C | L | E | V | E | R | A |
|               | N | D | R | I | C | H | X |

©Kendall Hunt Publishing Company

Figure 3-9: Encrypting with columnar transposition.

The ciphertext is just the concatenation of the columns (read top to bottom), in the order given by the alphabetical order of the keyword letters. For example, with keyword LEOPARD the plaintext EMMAW OODHO USEHA NDSOM ECLEV ERAND RICHX becomes the ciphertext WSOEC OHEAX MHNLD EDACN MODER AVSVI OEMRH, as shown in Figure 3-9.

- (a) When this cipher was used, as late as World War I, the usual method was to encrypt the message *twice* with different keywords. Here is a ciphertext made by first using columnar transposition with keyword LEOPARD, then a second columnar transposition with keyword PANTHER. Decrypt this text:

TEYEI WIHBP TUSOE ETDXD RHPFO ENFFN IOHSE SMEIC EMOHG NCNAO  
ASIIM OTASE UBEDN TTEOL SNPOE ITLSS STAB

- (b) Prove that such a **double transposition cipher**, where each key has length  $r$ , has the same effect as a single columnar transposition with a different key of length  $r^2$ .

## Index

- abelian group 3-8
- additive identity element 3-47
- additive inverse 3-38
- additive order 3-48
- arithmetic progression 3-26
- associative operation 3-47
- atomic 3-37
- Atomicity Lemma 3-37
- axioms for a ring 3-47
  
- breaking a cipher 3-61
  
- Caesar cipher 3-61
- Carmichael number 3-58
- Cauchy's Theorem 3-56
- certificate 3-58
- certificate of primality 3-59
- characteristic of a ring 3-56
- Chinese Remainder Theorem  
    (Full Version) 3-30  
    (Simple Version) 3-28
- cipher 3-61
- ciphertext 3-61
- classifying structures 3-48
- columnar transposition cipher 3-70
- commutative operation 3-47
- commutative ring with identity 3-47
- composite number 3-4
- congruence class 3-7, 3-46
- congruence modulo a natural 3-7, 3-46
- congruence relation 3-7
- cryptography 3-61
- cryptosystem 3-61
- cyclic group 3-49, 3-54
  
- decrypt 3-61
- digital signature 3-67
- discrete logarithm 3-68
- digram 3-69
- direct product of rings 3-48
- distributive law 3-47
- division of polynomials 3-39
- division relation 3-4
- double transposition cipher 3-70
  
- Eisenstein integers 3-40
  
- encrypt 3-61
- Euclidean Algorithm 3-15
- Euler totient function 3-20, 3-33, 3-52
- exponentiation 3-52
  
- factorial of a natural 3-8, 3-20, 3-23
- Fermat's Little Theorem 3-53
- Fermat Number 3-26
- field 3-52
- finite field 3-52
- frequency table 3-69
- Fundamental Theorem of Algebra 3-41
- Fundamental Theorem of Arithmetic 3-4, 3-29
  
- Gaussian integers 3-40
- generating a group 3-54
- generator of a group 3-59
- greatest common divisor 3-8, 3-15
- Gregorian calendar 3-32
- group 3-48, 3-52
  
- information theory 3-63
- integers modulo a natural 3-37
- intersection of multisets 3-41
- inverse of one natural modulo another 3-14
- Inverse Theorem 3-15
- irreducible polynomial 3-39, 3-56
- isomorphic structures 3-48
- isomorphism 3-48
- iterate of a function 3-34
  
- Julian calendar 3-32
  
- Kenken number 3-39
- key exchange 3-69
- key for a cipher 3-64
- keyword polyalphabetic cipher 3-62
  
- least common multiple 3-8, 3-15
- length of a Gaussian integer 3-40
- linear combination 3-16
  
- Mersenne prime 3-9
- modular arithmetic 3-6
- modular division operator 3-9
- monic polynomial 3-20, 3-39, 3-56
- monomial 3-56

- multiple of a natural 3-5
- multiplicative group 3-52
- multiplicative identity element 3-14, 3-47
- multiplicative inverse 3-14, 3-47, 3-52
- multiset 3-41
  
- natural numbers 3-2
- norm in  $\mathbf{Z}[\sqrt{r}]$  3-38
- number theory 3-2
  
- one-time pad 3-63
- order of an element 3-53
  
- pairwise relatively prime naturals 3-30
- perfect number 3-9
- perfect square modulo a natural 3-25
- period of a function 3-34
- periodic system 3-6
- plaintext 3-61
- Playfair cipher 3-69
- Polybius square 3-67
- polynomial 3-19
- prime divisor 3-23
- prime factorization 3-5
- prime Gaussian integer 3-40
- prime number 3-4
- Prime Number Theorem 3-27
- prime power 3-33
- primorial of a natural 3-25
- private key 3-65
- public key 3-65
- public-key cryptography 3-65
  
- quadratic residue 3-25
  
- rail fence cipher 3-67
- rational number 3-38
- remainder operator 3-9
- repeated squaring 3-53
- relatively prime naturals 3-9, 3-15, 3-31
- Richard paradox 3-25
- ring 3-47
- ring axioms 3-47
- roots of a polynomial 3-54
- rotation 3-61
- RSA cryptosystem 3-65
  
- secure cipher 3-61
- Sieve of Eratosthenes 3-5
  
- single-letter frequencies 3-62
- single-letter substitution cipher 3-61
- squarefree natural 3-51
- submultiset 3-41
- system of congruences 3-28
  
- threshold- $t$  numbers 3-39
- transposition 3-21
- transposition cipher 3-63
- trial division algorithm 3-4
  
- union of multisets 3-41
- unit (in the Gaussian integers) 3-40
  
- without loss of generality 3-36
  
- zero divisors 3-50



## Chapter 4: Recursion and Proof by Induction

*“First shalt thou take out the holy pin. Then shalt thou count to three: no more, no less. Three shall be the number thou shalt count, and the number of the counting shall be three. Four shalt thou not count, neither shalt thou count two, except that thou then proceedst to three. Five is right out! When the number three, being the third number, be reached, lobbest thou thy Holy Hand Grenade of Antioch into the midst of thy foe, who, being naughty in my sight, shall snuff it.”*

In the last chapter we worked informally with the naturals (non-negative integers) in three ways — we *defined* properties of and operations on the naturals, we *computed* on them with various algorithms, and we *proved* interesting statements (like the Fundamental Theorem of Arithmetic) from simpler, more obviously true statements (like the results of arithmetic calculations). In this chapter we get more formal, introducing a powerful general technique for each of these three activities and looking at the relationships among them:

- *Recursive Definition* is the process of defining a set of objects in terms of itself. We'll see recursive definitions of the naturals themselves, of several operations on the naturals, and then of several additional data types.
- *Recursive Algorithms* are those that make calls to themselves. Doing this always brings the risk of an endless chain of calls to the same procedure, but we will see how we can define recursive algorithms, dealing with recursively defined data types, in such a way as to be sure this won't happen.
- *Mathematical Induction* is a powerful technique for constructing formal proofs about recursively defined data types or recursive algorithms. We'll spend most of the chapter formulating inductive proofs in a variety of settings, starting with the naturals and moving on to other data types.

## 4.1 Recursive Definition

### 4.1.1 The Peano Axioms

What is the point in giving a formal definition of “the naturals”, when we already *know* what “the non-negative integers” are<sup>1</sup> and know quite a bit else about them? One reason for such a definition comes from our process of proof<sup>2</sup> — we prove interesting statements by using simpler statements, and those from simpler statements still, but we have to stop someplace. At some point we have to have a set of statements, called **axioms**, that we accept as being obviously true. Why should you accept the axioms? Because we *define* the naturals, for example, to be the set of objects for which those axioms are true. One person’s conception of the naturals may differ from another’s, but if both conceptions satisfy the axioms then both people can accept proofs based on those axioms. If they don’t, at least we’ve discovered that these two people are talking about two different concepts, and they can agree to disagree.

Our method of defining the naturals will be to state a few simple axioms that give the relationship among a small number of **undefined terms**. The fewer and simpler the axioms, the more confident we can be that we’ve gotten at the essence of the concept<sup>3</sup>. The *recursive* aspect of the definition is that the undefined terms are defined in terms of one another, raising the possibility of a circular definition. If something is defined in terms of itself, of course, it isn’t really defined at all, but we’ll see that the *interrelationship* of these terms gives us a meaningful definition.

The situation is quite similar to that in programming, where we might have an **abstract data type**, such as a stack, and two different ways to implement it, such as contiguous lists and linked lists. We’d like to write programs using stacks that will work whichever implementation we use, so we write down the essential features of stacks by specifying exactly which **operations** must be possible. (Typical operations for stacks are pushing, popping, and testing for emptiness.) Just as the mathematician wants there to be as few and as simple axioms as possible to be as close as possible to the essential features of a mathematical system, the programmer wants there to be as few and as simple operations as possible to be as close as possible to the essential features of the data type.

The following set of five axioms was formulated by Giuseppe Peano about 1900, in order to define the naturals. Note that only three undefined terms are used, which are in italics: *natural*, *zero*, and *successor*. All the rest of our vocabulary for speaking about naturals will be defined later, by using these three terms. To consider *natural* as an abstract data type, we can view the other two terms as operations that are defined on the type, with the following Java-syntax method declarations:

```
static boolean zero (natural x)
{ // Returns true if and only if x is zero.
```

---

<sup>1</sup>Do we really? Our direct experience is limited to naturals small enough to be represented somehow in our minds or in a computer. What about naturals that are very very large?

<sup>2</sup>Of course, *provability* and *truth* are not the same thing — Gödel’s Theorem says that any proof system for the naturals will fail to prove some true statement about the naturals.

<sup>3</sup>As long as we accept the axioms as true, at least.

```
static natural successor (natural x)
{ // Returns the successor of x
```

## Peano's Axioms for the Naturals

1. *Zero* is a *natural*.
2. Every *natural* has exactly one *successor*, which is a different *natural*.
3. *Zero* is not the *successor* of any *natural*.
4. No two *naturals* have the same *successor*.
5. If you start with *zero*, and keep taking *successors*, you eventually reach all the *naturals*.

Before going on, notice an important consequence of the fourth and fifth axioms. Since every natural is reachable by successor operations from zero, every natural except zero must itself be the successor of at least one other natural. By the fourth axiom, it can only be the successor of one other natural, which we'll call its **predecessor**. We're going to augment our `natural` abstract data type right away to add a third operation:

```
static natural pred (natural x)
{ // Returns the predecessor of a nonzero natural.
  // Throws an exception if called with x equal to zero.
```

Along with this operation come two more axioms:

- `pred(successor(x)) == x`, and
- If `zero(x) == false`, then `successor(pred(x)) == x`.

The fifth axiom is clearly more complicated than the first four, and is more difficult to put into mathematical language. Here are some alternate formulations — can you see why each is equivalent to the original?

- Version 1: There aren't any naturals other than those forced to exist by the first four axioms.
- Version 2: If you keep taking predecessors of a natural, you will eventually reach zero.
- Version 3: If  $S$  is a set of naturals,  $0 \in S$ , and  $\text{successor}(x) \in S$  whenever  $x \in S$ , then  $S$  is the set of all naturals.
- Version 4: (**The Principle of Mathematical Induction**) If  $P$  is a unary predicate on naturals,  $P(0)$  is true, and  $\forall x : P(x) \rightarrow P(\text{successor}(x))$  is true, then  $\forall x : P(x)$  is true.

- Version 5: (The **Well-Ordering Principle**) Any non-empty set of naturals contains a least element<sup>4</sup>.

The Principle of Mathematical Induction is the version we'll use most often, because it forms the basis for the important proof technique of mathematical induction, which we'll begin using in the next section. Why is it equivalent to the other versions? The easiest one to see equivalent is the third. If we know that the third version is true, we can prove the fourth by letting  $S$  be the set of naturals such that  $P(x)$  is true. The hypothesis of Version 4 says exactly that this  $S$  satisfies the hypothesis of Version 3, so the conclusion of Version 3 follows, and this is exactly the conclusion of Version 4. To show that Version 4 implies Version 3, we just let our property  $P(x)$  be " $x \in S$ " and work the same way.

#### 4.1.2 Recursion and the Fifth Axiom

Recursive definitions are intimately connected with **recursive algorithms**, or algorithms that call themselves. You may already know from programming that recursive algorithms are dangerous — there is always the possibility that the algorithm will call itself, and the new incarnation of the algorithm will call itself again, and that will call itself again, and the process will never stop (at least until you kill it somehow). With care, you can keep this from happening, so that the successive calls have different arguments until eventually one call terminates without calling itself, returning control to the incarnation that called it, which then terminates, and so on back until the original program terminates.

The clearest way to see that a recursion will terminate is if the argument of the call keeps getting smaller, so that it must eventually reach a value where the program will terminate without further recursion. We can restate the fifth Peano axiom in a form that guarantees that a recursive algorithm will terminate if these conditions are met:

- Suppose that an algorithm takes one argument of type **natural**, that it terminates when called with argument 0, and that when called with any nonzero argument<sup>5</sup>  $x$  it terminates, except possibly for a call to itself with argument  $pred(x)$ . Then on any input, the algorithm eventually terminates.

Now that we've defined the naturals, we can go about defining operations on them, starting with addition and multiplication. (At that point we'll be ready to talk formally about all the number theory we've done.) How do we define what  $x + y$  is? Well, from our reasoning above we know, since  $y$  is a natural, that it's either zero or the successor of another number  $pred(y)$ . So here is a definition by cases:

- If  $y = 0$ , then define  $x + y$  to be  $x$ .

<sup>4</sup>It's less obvious that this version is equivalent to the others, but we'll prove in Problem 4.1.4 that it is.

<sup>5</sup>We'll write " $x > 0$ " for "`zero(x) == false`", although we haven't officially defined the symbol " $>$ " yet. We'll do this in Problem 4.1.5.

- If  $y > 0$ , then define  $x + y$  to be the successor of  $x + \text{pred}(y)$ .

Both clauses of these definitions match what we know about how addition works, which is good, but on the other hand we've defined addition in terms of addition, which looks a bit fishy. We can rewrite the definition as the following recursive algorithm:

```
static natural plus (natural x, natural y)
{
  // Returns x+y.
  if (zero(y))
    return x;
  else return successor (plus (x, pred(y)));
}
```

The recursive-algorithm form of the fifth Peano axiom tells us that this procedure will terminate. Intuitively, this is because the second argument keeps getting smaller with each call until it reaches zero, in which case there's no more recursion.

Note that it's not obvious from the definition how to prove even obvious facts like  $\forall x : 0 + x = x$ . None of our techniques from the predicate calculus will help us here, until we have some knowledge *about the naturals* and *about addition* that imply this fact (and commutativity of addition in general, and all the other familiar properties of addition and multiplication). We'll need the method of mathematical induction to prove these<sup>6</sup>.

The definition of multiplication is quite similar to that of addition:

- For any  $x$ ,  $x \cdot 0 = 0$ , and
- If  $y > 0$ ,  $x \cdot y = (x \cdot \text{pred}(y)) + x$ .

Now multiplication is defined in terms of both itself and addition, but we know from above that the addition algorithm always terminates. This results in the following recursive Java-syntax method:

```
static natural times (natural x, natural y)
{
  // Returns x times y.
  if (zero(y))
    return 0;
  else return plus (times (x, pred(y)), x);
}
```

Again the recursion will continue until the second argument becomes zero. As long as we know that **plus** is defined for all inputs that are naturals, so is **times**. We know that, but to prove it

---

<sup>6</sup>Can you start by proving  $\forall x : 0 + x = x$ ? We'll prove the rest of the standard properties in Section 4.6.

formally we need some form of the fifth Peano axiom — that by repeatedly taking predecessors of a natural we will eventually get to zero.

There are number systems that satisfy the first four Peano axioms but not the fifth<sup>7</sup>. Such systems aren't just curiosities — their existence tells us something about the axioms. If it were possible to *prove* the fifth axiom starting from the first four, *any* system satisfying the first four would have to satisfy the fifth as well, unless it was an inconsistent system in which you could prove anything. So we can conclude that no such proof of the fifth axiom exists<sup>8</sup>.

Once you have these definitions, all sorts of properties like the commutativity and associativity of addition and multiplication can be *proved* from the definitions, by mathematical induction<sup>9</sup>. This raises the question, of course, of whether you're now allowed to use a fact like " $2 + 2 = 4$ " without proving it! It's worthwhile to prove something like that once, to see how it works, but after this section you should assume the well-known facts of arithmetic to be true (unless proving them is somehow the point of an Exercise or Problem).

### 4.1.3 Other Recursive Systems

Recursive definition is not restricted to the naturals. We can and will define a variety of mathematical systems in this way. We give some base objects, a method for constructing new objects from old ones, and an assertion that the only objects of the type are the ones obtained from the bases by using the rules. Once we have such a definition, we can define operators on the new type as we did for the naturals above, and prove statements about all the objects of the type by a general form of mathematical induction.

For example, just as all naturals are derived from zero by taking successors, all strings are derived from the empty string by appending letters. We have "Peano axioms" for the set of strings on a given alphabet, which correspond to the axioms for naturals:

- $\lambda$  is a string.
- If  $w$  is a string, and  $a$  is a letter, there is a unique string  $wa$ .
- If  $va = wb$  for some strings  $v$  and  $w$  and some letters  $a$  and  $b$ , then  $v = w$  and  $a = b$ .
- Any string except  $\lambda$  can be written as  $wa$  for a unique string  $w$  and letter  $a$ .
- Every string is derived from  $\lambda$  by appending letters as above.

---

<sup>7</sup>For example, in Problem 4.1.1 we'll consider the ordinary naturals together with a new number  $\omega$  and numbers  $\omega + i$  for every positive or negative integer  $i$ . You'll still have to define addition and multiplication on these "numbers", but once you've done so you can go ahead and start doing number theory.

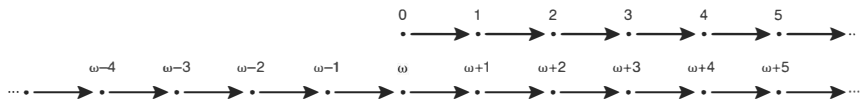
<sup>8</sup>Along the same lines, logicians also work with number systems called **non-standard models of arithmetic** that only satisfy weaker forms of the fifth axiom. The fifth axiom is somewhat vague — when you use it in proofs it really says only that the induction rule works for predicates  $P(x)$  that you can write down, so that there could be extra numbers out there such that for some reason, you can't express the fact that they're extra. Of course, such systems are rather beyond the scope of this book.

<sup>9</sup>Again, we'll do this in Section 4.6. Hofstadter does it in his Chapter VIII, more formally than we will.

Later in this chapter we'll define some operations on strings by defining them first on  $\lambda$  and then on  $wa$  (for arbitrary strings  $w$  and letters  $a$ ) in terms of their value on  $w$ . First, though, we want to study the proof method of mathematical induction in more detail, in the special case of proving statements for all naturals.

#### 4.1.4 Exercises

- E4.1.1 Prove from the Peano axioms that `successor(successor(successor(0)))`, usually called “3”, is a natural.
- E4.1.2 Prove from the definition of addition that  $2 + 2 = 4$ , where “2” denotes the output of “`successor(successor(0))`” and “4” denotes that of
- $$\text{successor}(\text{successor}(\text{successor}(\text{successor}(0)))).$$
- E4.1.3 (uses Java) Write a pseudo-Java method `boolean isThree (natural x)` that returns `true` if and only if  $x$  is equal to 3. You should use the given operations for the `natural` data type. Make sure that your method cannot ever call `pred` on a zero argument.
- E4.1.4 Write the expression  $(2 + (2 \cdot (3 + 1))) \cdot (4 + 0)$  in terms of the methods `plus` and `times` in this section. You may use the ordinary names for the numbers.
- E4.1.5 Explain informally why the statement  $\forall x : [x \neq 0 \rightarrow \exists y : x = \text{successor}(y)]$  follows from the fourth and fifth Peano axioms.
- E4.1.6 We've seen two other number systems that are in some ways like the naturals, but have only finitely many “numbers”. Which of the Peano axioms are true for each of these systems?
- (a) The numbers modulo  $m$  (for any  $m$  with  $m > 1$ ), where the numbers are  $\{0, 1, \dots, m-1\}$  and the successor operation adds 1 modulo  $m$ .
  - (b) The “threshold- $t$ ” numbers, defined in Exercise 3.6.9, have numbers  $\{0, 1, \dots, t\}$  and a successor operation that is the same as the usual one, except that the successor of  $t$  is  $t$ .
- E4.1.7 Suppose we make Peano axioms for the set  $\mathbf{Z}$  of all integers, by saying that 0 is an integer and that every integer  $x$  has both a unique successor and a unique predecessor, each different from  $x$ . Our “fifth axiom” could then say that every number is reachable from 0 by taking predecessors or successors. Clearly  $\mathbf{Z}$  obeys these axioms. Is it the only number system that does?
- E4.1.8 (uses Java) Write a static pseudo-Java method `boolean equals (natural x, natural y)` that returns `true` if and only if  $x$  and  $y$  are the same natural. Of course your method should not use the `==` operator, and should return the correct answer given any two `natural` inputs.
- E4.1.9 (uses Java) Give a recursive definition of the exponentiation operation, so that `power (x, y)` returns  $x^y$  for any naturals  $x$  and  $y$ . Write a recursive static pseudo-Java method implementing this definition. You may use the methods defined in the section.
- E4.1.10 (uses Java) Give a recursive definition for the evenness property of naturals, without using addition (except `successor`) or multiplication. Write a static recursive pseudo-Java method `boolean even (natural x)` that returns `true` if and only if  $x$  is even, and uses only the `zero` and `pred` methods defined in this section.



©Kendall Hunt Publishing Company

Figure 4-1: A strange number system. Arrows point to successors.

#### 4.1.5 Problems

- P4.1.1 Consider a number system that contains all the ordinary non-negative integers, a new element  $\omega$ , and an element  $\omega + i$  for every integer  $i$  (positive, negative, or zero), as illustrated in Figure 4-1. Show that this system satisfies the first four Peano axioms. Why doesn't it satisfy the fifth?
- P4.1.2 Can you define addition and multiplication for the number system of Problem 4.1.1 in a way that makes sense?
- P4.1.3 Prove that Versions 2 and 3 of the fifth Peano axiom are logically equivalent.
- P4.1.4 Prove that the Well-Ordering Principle (Version 5 of the fifth Peano axiom) is equivalent to one of the other versions of the fifth Peano axiom (you may choose which).
- P4.1.5 (uses Java) Give a recursive definition of the “less than” operator on numbers. (You may refer to equality of numbers in your definition.) Write a static pseudo-Java method “`boolean isLessThan (natural x, natural y)`” that returns `true` if and only if  $x < y$  and uses only our given methods. (**Hint:** Follow the example of the functions `plus` and `times` in the text.)
- P4.1.6 (uses Java) Give a recursive definition of and a recursive static method for the **natural subtraction** function, with pseudo-Java header

`natural minus (natural x, natural y).`

On input  $x$  and  $y$  this function returns  $x - y$  if this is a natural (i.e., if  $x \geq y$ ) and 0 otherwise.

- P4.1.7 Following Exercise 4.1.7, create a set of axioms that exactly define the set  $\mathbf{Z}$  of all integers.
- P4.1.8 (uses Java) As in Problems 4.1.5 and 4.1.6, write static pseudo-Java methods `natural quotient (natural x, natural y)` and `natural remainder (natural x, natural y)` that return  $x/y$  and  $x\%y$  respectively, as long as  $x$  is a natural and  $y$  is a positive natural. You may use the other methods defined in this section and its Problems.
- P4.1.9 (uses Java) Let's define a **stack** as follows<sup>10</sup>:

- The empty stack is a stack.
- If  $S$  is a stack and  $x$  is a `thing`, `S.push(x)` is a stack.
- The stacks `S.push(x)` and `T.push(y)` are equal if and only if  $S$  and  $T$  are equal and  $x$  and  $y$  are equal.

<sup>10</sup>In real Java the `Stack` class is parametrized, using generics, but here we will define a pseudo-Java stack whose elements are from the class `thing`.



- Every stack is derived from the empty stack by pushing things as above.

Here are two problems using this definition:

- Explain why we can define a **pop** operation that returns a **thing** when called from any nonempty stack.
- Assume now that we have a pseudo-Java **Stack** class with instance methods **boolean empty( )**, **void pop (thing x)**, and **thing pop( )**. Write an instance method **boolean equals(Stack T)** that returns **true** if and only if the stack *T* is equal to the calling stack. Make sure that your method has no **side effect**, that is, make sure that you leave both *T* and the calling stack just as you found them.

P4.1.10 (uses Java) A **queue** is a data structure where we may **enqueue** elements on one end and **dequeue** them from the other.

- Give a recursive definition of a **queue** of **thing** elements on the model of the definition of a stack in Problem 4.1.9.
- Give a recursive definition of the dequeuing operation. That is, define the result of the method call **Q.dequeue( )** in terms of your recursive definition of queues in part (a).
- Write a recursive instance method **boolean equals(Queue Q)** for a pseudo-Java **Queue** class that returns **true** if and only if the calling queue and *Q* are equal by your definition in part (a). You may use the instance methods **boolean empty( )**, **void enqueue (thing x)**, and **thing dequeue( )**. Your method should have no side effects, meaning that both queues should be the same before and after your method is run.

## 4.2 Excursion: Recursive Algorithms

In this Excursion we have some examples to familiarize or refamiliarize you with the notion of recursive algorithms, followed by an Exercise where you will argue that a particular recursive algorithm does the right thing.

To begin, here is an example of a real Java method to go in a class called `Stack`<sup>11</sup>. This method pops all the elements from the calling `Stack` object. It uses two other `Stack` methods – `pop` removes the top element from the stack and `isEmpty` tests the stack for emptiness:

```
void clear()
{
    // Pops and discards all elements from calling Stack
    if (!isEmpty())
    {
        pop();
        clear();
    }
}
```

So as long as the calling `Stack` isn't empty, this procedure will pop off the top element, call another version of itself to clear what's left, and stop with the stack empty. Once a version is called with an empty stack, it does nothing and finishes (it's a "no-op"). The version that called it is then done, so it finishes, and control passes through all the remaining stacked versions until the original one is reached and finishes, with the stack now successfully cleared.

There is of course a normal iterative version of this same procedure that performs the same pops in the same order – its key statement is

```
while (!isEmpty()) pop();
```

In fact, any recursive algorithm that only calls itself once and does so at the end, called a **tail recursion**, can easily be converted to an iterative program with a loop.

Recursion doesn't allow us to do anything we couldn't do already without it, but it often gives a simpler way of writing down an algorithm. (You'll see many more examples in an algorithms course.)

In many programming languages, recursive programs are less efficient than the equivalent iterative programs because the compiler doesn't convert the recursive code to machine code in the most efficient possible way. Other languages, like the Lisp family, support recursion very well. In general, the smarter your compiler, the greater the incentive to use simple, readable, verifiable recursive algorithms in place of iterative ones that might be slightly faster.

In Section 4.1 we saw some pseudo-Java examples of recursive code, implementing the recursive definitions of the `plus` and `times` methods on `natural` primitives. The result of `plus(x, y)`, for example, was defined to be `x` if `y` was zero, and to be `successor (plus (x, pred(y)))` otherwise. This is a tail recursion much like the `clear` method above. If we call the method to add 3 to `x`, this

---

<sup>11</sup>As usual, we will assume that the rest of this `Stack` class has already been written elsewhere.

method makes a call to another version of `plus` that adds 2 to  $x$ . That in turn calls another version that adds 1, which calls another version that adds 0. The last version returns  $x$ , the next-to-last then returns  $x + 1$ , the second version returns  $x + 2$ , and finally the original version returns  $x + 3$ .

How do we know that a recursive algorithm does what it should? It must obey the following three rules:

1. There must be a **base case** in which the algorithm does not make a recursive call. It must have the correct behavior in this base case.
2. If every recursive call has the correct behavior (*e.g.*, it returns the correct value), *then* the original call has the correct behavior.
3. The recursion must be **grounded**, which means that there cannot be any infinite sequence of recursive calls. That is, any sequence of recursive calls must eventually end in a base case.

These rules allow us to separate the groundedness of the recursion from its correctness. If we can show that the algorithm follows Rules 1 and 2, then it will have the correct behavior whenever it finishes, since the base case returns correct answers and each succeeding case returns correct answers because its recursive calls give it correct answers. Rule 3 takes care of the only other way it could go wrong, by entering an infinite sequence of recursive calls and never returning an answer at all.

Let's apply these rules to the `clear` method above. The base case is when the stack is already empty. The method obeys Rule 1 because if the stack is empty, it returns and does nothing, and this behavior is correct because the method terminates with an empty stack. It also clearly obeys Rule 2, because if the stack is not empty, the `pop` call will succeed and it will make a recursive call to `clear`, which by the assumption of Rule 2 empties the stack.

Why does it obey Rule 3? Here we need an assumption about stacks, in particular that any stack contains some finite number of elements. Because of the `pop` call, the recursive call to `clear` operates on a stack with *fewer* elements than the stack that was the subject of the original call. Further calls will be on stacks with fewer and fewer elements, until we eventually reach an empty stack and we are in the base case.

This use of the word “eventually” is of course imprecise, drawing on our intuition about what the word “finite” means. In Section 4.1 we saw the Peano Axioms, which formalize this intuition – one form of the fifth Peano Axiom says exactly that a recursive algorithm of this kind will eventually terminate. In the remainder of this chapter we will consider a wide variety of examples of proof by induction. Many of these can be viewed as arguments for the correctness of a recursive algorithm, like those in this excursion.

Finally, we turn to the example algorithm to be analyzed. It is claimed that: “Given a positive number as input, this algorithm will output a sequence of primes that multiply together to equal the input.” If you believe that statement (which essentially just says that this algorithm is correct)

then you must believe the “existence half” of the Fundamental Theorem of Arithmetic<sup>12</sup>.

```
void factor (natural x)
{
  // Prints sequence of prime factors of x to System.out, one per line
  // Special cases: outputs empty sequence if x is 0 or 1
  if (x <= 1) return;
  natural d = 2;
  while ((x % d != 0) && (d < x))
    d++;
  System.out.println(d);
  factor (x/d);}
}
```

**Writing Exercise:** Give a clear and convincing argument that this algorithm is correct. One way to do this is to answer the following five questions:

- Describe what happens when `factor(30)` is called.
- What is the base case for this algorithm? Why is the answer in the base case correct?
- Why does `factor` only output prime numbers? (Hint: Suppose that  $x \% d == 0$  and that  $d$  has a divisor  $e$  with  $e > 1$  and  $e < d$ . What is  $x \% e$ ?)
- Why do the numbers output by `factor(x)` multiply together to give  $x$ ?
- Why does the method obey Rule 3, that is, why must it terminate given any `natural` as its input? (Hint: Why could we guarantee that the Euclidean Algorithm always terminates?)

---

<sup>12</sup>This is similar to the way we proved the Inverse Theorem in Section 3.3, by giving an algorithm that provided an inverse whenever the theorem said that one exists.

## 4.3 Proof By Induction for Naturals

### 4.3.1 What It Is and How It Works

We now come to our first detailed look at **mathematical induction**. Mathematical induction is a general technique for proving statements about all elements of a data type, and can be used whenever that data type has a recursive definition. We're going to begin with **ordinary induction**, the simplest kind, which allows us to prove statements about all *naturals*. Later in this chapter and the next we'll learn methods for several other data types with the same kind of definition.

Formally, mathematical induction is just another proof rule like all our propositional and predicate calculus rules, because it says that if you have proved certain statements you are allowed to conclude a certain other statement. Our two goals in this section are to learn when and how to use this rule, and to convince ourselves that it is *valid* (that things proved with it are actually true). First off, let's state the proof rule:

- Let  $P(x)$  be any predicate with one free variable of type **natural**.
- If you prove both  $P(0)$  and  $\forall x : P(x) \rightarrow P(x + 1)$ ,
- Then you may conclude  $\forall x : P(x)$ .

Let's try a simple example. Define  $P(x)$  to be the statement “the sum of the first  $x$  odd numbers is  $x^2$ .” (A bit of experimentation, like  $1 = 1^2$ ,  $1 + 3 = 2^2$ ,  $1 + 3 + 5 = 3^2$ , suggests that this *might* be a general rule.) If we remember various high-school rules about summing arithmetic progressions, we know how to verify this fact, but let's use our new technique to give a formal proof.

First, we are told to prove  $P(0)$ , which says “the sum of the first zero odd numbers is zero”. True enough, once we remember that all vacuous sums are zero just as all vacuous products are one. Next we are given a  $\forall$  statement to prove, so we let  $x$  be an arbitrary natural and set out to prove  $P(x) \rightarrow P(x + 1)$ . To do this by a direct proof we must assume  $P(x)$ , that “the sum of the first  $x$  odd numbers is  $x^2$ ”, and prove  $P(x + 1)$ , that “the sum of the first  $x + 1$  odd numbers is  $(x + 1)^2$ ”. How can we do this? The key point is to notice that the second sum is *equal* to the first sum plus one more term, the  $(x + 1)$ 'st<sup>13</sup> odd number,  $2x + 1$ .

So the sum we are interested in is equal to the first sum plus  $2x + 1$ . We **apply the inductive hypothesis** by using  $P(x)$  to say that the first sum is equal to  $x^2$ . Then it follows that the second sum is  $x^2 + 2x + 1 = (x + 1)^2$ , which is just what we wanted to prove it to be.

This example illustrates several common features of inductive proofs:

---

<sup>13</sup>How do we know that the  $(x + 1)$ 'st odd number is  $2x + 1$ ? The first odd number is 1, the second is 3, and the third is 5. It appears from these three examples that the  $i$ 'th odd number is  $2i - 1$ , from which we could conclude that the  $(x + 1)$ 'st odd number is in fact  $2(x + 1) - 1 = 2x + 1$ . To be sure that this rule always holds, of course, we would need to do yet another mathematical induction. Unfortunately, we have a technical problem in that the method described above would require us to talk about “the 0'th odd number” to prove the base case. We'll deal with this technicality in the next section.

- We first have to prove a **base case**  $P(0)$ , which is often something totally obvious, as it was here. It's important that we substitute  $x = 0$  into  $P(x)$  carefully to get the correct statement  $P(0)$ .
- Then we do the **inductive step**, by proving the quantified statement  $\forall x : [P(x) \rightarrow P(x+1)]$ . Following the general rule for  $\forall$ 's, we let  $x$  be an arbitrary natural, assume that  $P(x)$  is true, and try to derive  $P(x+1)$ .  $P(x)$  is called the **inductive hypothesis**, and  $P(x+1)$  is called the **inductive goal**.
- The best thing we usually have going for us is that  $P(x)$  and  $P(x+1)$  are similar statements. In this example, the two sums differed only in that the sum in  $P(x+1)$  had one extra term. Once we knew what that term was, the inductive hypothesis told us the rest of the sum and we could evaluate the whole thing.
- Once we have proved both the base case and the inductive case, we may carefully state our conclusion, which is " $\forall x : P(x)$ ".

One mental barrier that comes up in learning induction is that  $P(x)$  is a *statement*, not a *term*. Many students have a hard time avoiding a phrasing like the following for the third bullet above: " $P(x+1)$  is equal to  $P(x)$  plus the extra term..." This is clearly a type error, and is bound to get you into trouble when you have to think of  $P(x)$  as a predicate later. It may help to give a name to one of the terms in  $P(x)$  to avoid this problem and make the statements easier to talk about. (In the example above, define  $S(x)$  to be "the sum of the first  $x$  odd numbers" and rewrite  $P(x)$  as " $S(x) = x^2$ ".)

### 4.3.2 Examples of Proof By Induction

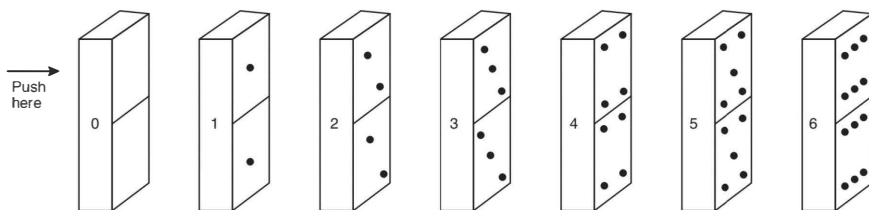
Let's try some more examples. How many binary strings are there of length  $n$ ? We've seen that the answer is  $2^n$ , so let's let  $P(n)$  be the statement "There are exactly  $2^n$  binary strings of length  $n$ ." As usual,  $P(0)$  is pretty easy to prove: "There are exactly  $2^0$  binary strings of length 0." We should know by now that  $2^0 = 1$ , and that there is exactly one string of length zero, the empty string.

Now we assume "There are exactly  $2^n$  strings of length  $n$ " and try to prove "There are exactly  $2^{n+1}$  strings of length  $n+1$ ". This means that we need some method to count the strings of length  $n+1$ , preferably by relating them somehow to the strings of length  $n$ , the subject of the inductive hypothesis. Well, each string of length  $n+1$  is obtained by appending a letter (0 or 1) to a string of length  $n$ . If there is no double counting involved (and why isn't there?) this tells us that there are exactly two strings of length  $n+1$  for each string of length  $n$ . We are *assuming* that there are exactly  $2^n$  strings of length  $n$ , so this tells us that the number of strings of length  $n+1$  is exactly twice as many, or  $2 \cdot 2^n = 2^{n+1}$ . We have completed the inductive step and thus completed the proof.

An almost identical proof tells us that an  $n$ -element set has exactly  $2^n$  subsets<sup>14</sup>. If we take  $P(n)$

---

<sup>14</sup>In fact, it's easy to see that you can match up the subsets with binary strings one for one, so there have to be the same number of each, but let's go through the whole inductive proof again for practice.



©Kendall Hunt Publishing Company

Figure 4-2: An infinite sequence of dominoes.

to be “an  $n$ -element set has exactly  $2^n$  subsets”, then  $P(0)$  is again pretty obvious (the empty set has exactly one subset, itself). Again we let  $n$  be an arbitrary natural, assume  $P(n)$ , and try to prove  $P(n + 1)$ .  $P(n + 1)$  talks about all possible subsets of an  $(n + 1)$ -element set  $S$ , which we have to relate somehow to all the subsets of some  $n$ -element set.

Give a name,  $x$ , to one element of the set  $S$  and let  $T$  be the remainder of  $S$ , so that  $S = T \cup \{x\}$ . The inductive hypothesis applies to any set with  $n$  elements, so we may assume that  $T$  has exactly  $2^n$  subsets. Now given any subset  $U$  of  $T$ , we can form two subsets of  $S$ ,  $U$  itself and  $U \cup \{x\}$ . Again there is no double counting (why?), and we obtain every subset of  $S$  in this way. So there are exactly  $2^{n+1}$  subsets of  $S$ , because we have exactly two for each of the  $2^n$  subsets of  $T$ . We have completed the inductive step and thus completed the proof.

### 4.3.3 The Validity of Induction Proofs

Now that we’ve had a bit of practice carrying out inductive proofs, let’s take a closer look at why we should believe that they are valid. Formally, the answer to this is simple — the fifth Peano axiom says that anything that you correctly prove by induction is true, and the fifth Peano axiom is part of the definition of the system of naturals. If you don’t believe in the axiom, your conception of the naturals must differ from ours (and Peano’s) somehow, so that we’re talking about two different number systems.

Informally, people usually need some kind of image or metaphor to convince themselves that this works. One popular one is to think of the integers as an endless sequence of dominoes (Figure 4-2), the first one labeled 0, the second 1, and so forth. If you push over domino 0, and you believe that every domino  $i$  is going to knock over domino  $i + 1$  when it falls, then you should believe that they’re all going to fall eventually.

Another way to think of an induction proof is as *instructions* to construct an ordinary proof. Suppose we have proofs of  $P(0)$  and  $\forall x : [P(x) \rightarrow P(x + 1)]$ , and we want to prove  $P(17)$ . By substituting specific numbers into the second proof, we can generate proofs of the implications  $P(0) \rightarrow P(1)$ ,  $P(1) \rightarrow P(2)$ , and so forth all the way up to  $P(16) \rightarrow P(17)$ . Then we can derive

$P(17)$  from  $P(0)$  using seventeen applications of the modus ponens rule<sup>15</sup>.

Many people have a lot of trouble accepting the validity of mathematical induction because it appears to use circular reasoning. You want to prove some statement  $P(x)$ , but then in the middle of the proof you suddenly assume that  $P(x)$  is true! Actually this is not circular. The original goal is to prove, for arbitrary  $x$ , that  $P(x)$  is true *without any assumptions*. In the inductive step, however, you're trying to prove the *implication*  $P(x) \rightarrow P(x+1)$  for arbitrary  $x$ .

Another counterintuitive aspect of induction is that you start out trying to prove  $\forall x : P(x)$  and you're told instead to prove  $\forall x : [P(x) \rightarrow P(x+1)]$ , which is a *more complicated* statement of the same type. The point is, of course, that because  $P(x)$  is likely to have something to do with  $P(x+1)$ , proving the implication could be a lot easier than just proving  $P(x)$  directly.

Actually, there are situations when the best way to make the proof more feasible is to *add conditions* to  $P(x)$ , that is, to change " $P(x)$ " to " $P(x) \wedge Q(x)$ " for some statement  $Q(x)$ . You would think that this would make the proof harder<sup>16</sup>, but remember that  $Q(x)$  now appears on *both sides* of the implication. As you try to prove  $[P(x) \wedge Q(x)] \rightarrow [P(x+1) \wedge Q(x+1)]$ , you can use  $Q(x)$  as a premise, which may help in proving  $P(x+1)$ . You also have to prove  $Q(x+1)$  now, but this may not be too much of a problem. In the next section we'll look at the technique of **strong induction**, which is an example of this principle.

#### 4.3.4 Exercises

- E4.3.1 Prove by induction that for all naturals  $n$ , the sum of the first  $n$  positive naturals is  $n(n+1)/2$ .
- E4.3.2 Prove by induction that for all naturals  $n$ , the sum of the first  $n$  positive perfect squares is  $n(n+1)(2n+1)/6$ .
- E4.3.3 Prove that if  $A$  is an alphabet of  $k$  letters, and  $n$  is any natural, there are exactly  $k^n$  strings of  $n$  letters over  $A$ . (Hint: Let  $k$  be an arbitrary natural and then prove the statement by induction on  $n$ .)
- E4.3.4 Prove by induction that 3 divides  $n^3 - n$ . (This is also easy to prove directly by arithmetic modulo 3, but do it by induction.)
- E4.3.5 Prove by induction for all naturals  $n$  that the size of the set of naturals  $\{k : k < n\}$  is exactly  $n$ .
- E4.3.6 Following the reasoning in Excursion 1.2, and using the definitions there, prove by induction that every natural is either even or odd, but not both.
- E4.3.7 Let  $m$  be a fixed positive natural. Prove, by induction on all naturals  $n$ , that for every  $n$  there exist naturals  $q$  and  $r$  such that  $n = qm + r$  and  $r < m$ .

---

<sup>15</sup>One student was "enlightened" about the truth of this when a professor told him to construct a computer program that would take the induction proof and a number as input, and construct this chain of implications for the desired number. He suddenly "saw" that since these "straight-line proofs" existed for every natural  $i$ , every  $P(i)$  must actually be true. Of course, this is only an intuition, because to formally *prove* that the straight-line proofs exist for *any* natural, we would need some form of mathematical induction!

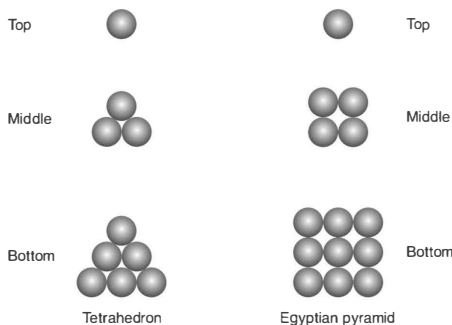
<sup>16</sup>This process has been likened to trying to pull yourself up by your own bootstraps.



- E4.3.8 Prove by induction on all naturals  $n$  that any set of  $n$  elements has exactly  $n(n-1)/2$  subsets of size exactly 2. (**Hint:** You will need to use different arguments for the inductive case based on the value of  $n$ . To prove  $P(1)$  and  $P(2)$  you can argue directly, without using the inductive hypothesis. For larger  $n$ , imitate the reasoning in our proof above that the set has  $2^n$  subsets in all.)
- E4.3.9 For any natural  $n$ , define  $S_n$  to be the sum, for  $i$  from 1 to  $n$ , of  $(-1)^i$ . So  $S_0 = 0$  because the sum of an empty set of numbers is 0,  $S_1 = (-1)^1$ ,  $S_2 = (-1)^1 + (-1)^2 = -1 + 1 = 0$ , and so forth. Find a rule giving  $S_n$  in terms of  $n$ , and prove your rule correct by induction.
- E4.3.10 Suppose I begin at time 0 with  $g$  grams of radioactive material. This material has **half-life**  $T$ , meaning that the amount of material remaining at time  $t + T$  is exactly half the amount remaining at time  $t$ , for any  $t$ . Prove that for any natural  $n$ , the amount remaining at time  $nT$  is exactly  $g2^{-n}$  grams.

### 4.3.5 Problems

- P4.3.1 Determine a formula for the sum of the first  $n$  positive perfect cubes and prove it correct by induction on all naturals  $n$ .
- P4.3.2 Let the finite sequence  $a_0, a_1, \dots, a_n$  be defined by the rule  $a_i = b + i \cdot c$ . Prove by induction on  $n$  that the sum of the terms in the sequence is  $(n+1)(a_0 + a_n)/2$ . (**Hint:** In the base case,  $n = 0$  and so  $a_0$  is equal to  $a_n$ . For the induction case, note that the sum for  $n+1$  is equal to the sum for  $n$  plus the one new term  $a_{n+1}$ .)
- P4.3.3 Let the finite sequence  $a_0, a_1, \dots, a_n$  be defined by the rule  $a_i = a_0 \cdot r^i$ , where  $r$  is some number not equal to 1. Prove by induction that the sum of the terms in the sequence is  $\frac{a_0 - a_{n+1}}{1-r}$  (where  $a_{n+1}$  is defined by the same rule even though it is not part of the sequence).
- P4.3.4 Prove by induction on all naturals  $n$  that the polynomial  $x - y$  divides the polynomial  $x^n - y^n$ . That is, prove that there exists a polynomial  $f_n(x, y)$  such that  $x^n - y^n = (x - y)f_n(x, y)$ . Give a general formula for  $f_n(x, y)$  and prove by induction that it is correct. (**Hint:** Prove (assuming that  $n \geq 1$ ) that if  $f_n$  satisfies its equation, and  $f_{n+1} = xf_n + y^n$ , then  $f_{n+1}$  satisfies its equation.)
- P4.3.5 Let  $\Sigma = \{a, b, c\}$  and let  $L_n$  for each natural  $n$  be the language consisting of exactly those strings with two  $a$ 's, two  $b$ 's, and  $n$   $c$ 's. Discover a formula for the size of  $L_n$  in terms of  $n$ , and prove your formula correct by induction. (**Hint:** You may find it useful to first solve this problem for various similar languages: strings with  $n$   $c$ 's and respectively no  $a$ 's or  $b$ 's, one  $a$  and no  $b$ 's, one of each, and two  $a$ 's and one  $b$ .)
- P4.3.6 Define  $S(n)$  to be the sum, for all  $i$  from 1 through  $n$ , of  $\frac{1}{i(i+1)}$ . Prove by induction on all naturals  $n$  (including 0) that  $S(n) = 1 - \frac{1}{n+1}$ .
- P4.3.7 Determine a formula for the number of size-3 subsets of an  $n$ -element set, for any  $n$ . Prove your formula correct by induction, using the result of Exercise 4.3.8 in your inductive case when  $n \geq 3$ .
- P4.3.8 This problem deals with two different methods of stacking cannonballs. To make a **tetrahedron** of height  $n$ , we arrange  $n(n+1)/2$  cannonballs on the ground in a triangle of side  $n$  (we



©Kendall Hunt Publishing Company

Figure 4-3: Two stacks of cannonballs, each of height 3

know the number of balls in the triangle from Exercise 4.3.1), and then place a tetrahedron of height  $n - 1$  centered on top of this triangle. To make an **Egyptian pyramid** of height  $n$ , we place  $n^2$  cannonballs on the ground in a square, then place an Egyptian pyramid of height  $n - 1$  centered on top of the square. The two stacks of height 3 are pictured in Figure 4-3.

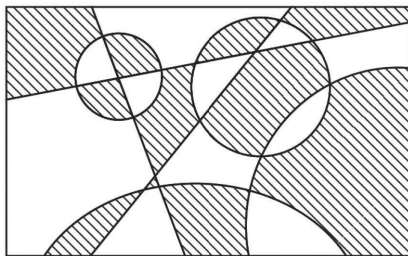
- Let  $T(n)$  be the number of cannonballs in a tetrahedron of height  $n$ , so that  $T(0) = 0$  and, for any  $n$  with  $n > 0$ ,  $T(n) = T(n - 1) + n(n + 1)/2$ . Prove by induction on all naturals  $n$  that  $T(n) = n(n + 1)(n + 2)/6$ .
- Let  $EP(n)$  be the number of cannonballs in an Egyptian pyramid of height  $n$ , so that  $EP(0) = 0$  and, for any  $n$  with  $n > 0$ ,  $EP(n) = EP(n - 1) + n^2$ . Find a polynomial formula for  $EP(n)$  and prove it correct by induction for all naturals  $n$ .

P4.3.9 Suppose I have a bank balance that holds an integer number of dollars. Every day the bank first doubles my balance (whether it is positive or negative) and then subtracts \$3 from it.

- Prove by induction on all naturals  $n$  that if my balance is 0 on Day 0, it is  $-3(2^n - 1)$  on day  $n$ .
- Prove that if the balance is strictly less than \$3 on Day 0, it will remain less than \$3 forever.
- Prove that if the balance is strictly greater than \$3 on Day 0, it will remain greater than \$3 forever.
- State a formula for the balance on Day  $n$ , given that it is  $X$  on Day 0. Prove your formula correct for all naturals  $n$  by induction.

P4.3.10 Here are three more geometric problems, suitable for an Excursion:

- Let  $L$  be a line segment. Prove that for any natural  $n$ , placing  $n$  distinct points on  $L$  divides it into exactly  $n + 1$  pieces.
- Let  $R$  be a rectangular region in the plane. Show that for any natural  $n$ , if we draw  $n$  distinct straight lines on  $R$ , dividing it into subregions, we can **two-color** the regions.



©Kendall Hunt Publishing Company

Figure 4-4: A rectangular region divided by lines and circles, then two-colored.

This means that we can color each region either red or blue, so that if two regions share a border of more than a point, they are different colors. (Note that each straight line must continue from one boundary of the region to another.)

- (c) Suppose that in part (b), we are also allowed to divide  $R$  with circles, or parts of circles interrupted only by the boundaries of  $R$ , as in Figure 4-4. Prove that for any natural  $n$ , the division of  $R$  by any number of lines and circles totaling to  $n$  can be two-colored.

## 4.4 Variations on Induction for Naturals

### 4.4.1 Not Starting at Zero

In one sense there is only one Principle of Mathematical Induction for the naturals, and only one way to use it to prove statements about them. But as we'll see in this section, there are ways to apply the original Principle in common situations when applying it directly doesn't seem to work. One could call these "new proof methods" or just variations on the original method, depending on one's personal taste.

In a footnote in Section 4.3, for example, we were interested in proving the statement "the  $i$ 'th odd number is  $2i - 1$ " from the Peano axioms defining the naturals. Induction seems like a natural way to proceed. We define  $P(i)$  to be the predicate in question, and can merrily go ahead and prove  $P(1)$  (the first odd number is 1). The induction step is also simple — since each odd number is two greater than the one before it, and the inductive hypothesis says that the  $i$ 'th odd number is  $2i - 1$ , the  $(i + 1)$ 'st odd number is two greater, or  $2i + 1$ , which is  $2(i + 1) - 1$  as desired.

The problem is that this inductive proof *doesn't* follow the framework set out by the fifth Peano axiom. Why? Because the naturals start with 0, not 1, and we didn't prove  $P(0)$ .  $P(0)$  doesn't even make sense, because there isn't a 0'th odd number. The proof, on the other hand, seems just as convincing as an ordinary induction proof, and the fact proven is certainly true. What we're going to have to do is expand our view of mathematical induction to encompass this proof and some others.

Here's another example. Let  $P(n)$  be the statement " $n! > 2^n$ " — that taking a factorial<sup>17</sup> of a number gives you a bigger number than raising two to that number. This is a good general rule, but it happens not to be true for  $n \leq 3$ . However, we can check  $P(4)$  by arithmetic ( $24 > 16$ ) and it is easy to show that for  $n \geq 4$ ,  $P(n) \rightarrow P(n + 1)$ . (We must show  $(n + 1)! > 2^{n+1}$ , which is true because  $(n + 1)! = (n + 1)n! > (n + 1)2^n \geq 2 \cdot 2^n = 2^{n+1}$ .) (Actually, the only fact about  $n$  we used in this argument was that  $n + 1 \geq 2$ , in the third step. So the *implication* " $P(n) \rightarrow P(n + 1)$ " is also true for  $n = 1$ ,  $n = 2$ , and  $n = 3$ , even though the individual statements  $P(1)$ ,  $P(2)$ , and  $P(3)$  are false.) We are reasonably convinced that the statement  $P(n)$  is true for all numbers greater than or equal to 4, but again we need to alter our formal statement of the induction law to do that.

We seem to have a revised induction law that says: "If  $P(k)$  is true, and  $P(i) \rightarrow P(i + 1)$  is true for all  $i \geq k$ , then  $P(i)$  is true for all  $i \geq k$ ." Here are three separate proofs that this law is valid:

- Define a new predicate  $Q(i)$  that says just " $P(i + k)$ ". Then  $Q(0)$  is the same statement as  $P(k)$ , which is given as true.  $Q(i) \rightarrow Q(i + 1)$  translates as  $P(i + k) \rightarrow P(i + k + 1)$ , which we know is true by plugging in  $i + k$  (which is at least as big as  $k$ ) into the induction rule  $P(i) \rightarrow P(i + 1)$ . So  $Q(i)$  is proven true for all  $i$  by ordinary mathematical induction! If  $j$  is any natural with  $j \geq k$ , then  $j - k$  is a natural and  $P(j)$  is the same statement as  $Q(j - k)$ , which must be true. So  $P(j)$  holds for all such  $j$ , as desired.
- Define  $R(i)$  to be the predicate " $(i \geq k) \rightarrow P(i)$ ". Then  $\forall i : R(i)$  is just what we want

---

<sup>17</sup>We defined the factorial function in Section 3.4 — if  $n$  is a natural then  $n!$  is the product  $1 \cdot 2 \cdot \dots \cdot n$ .

to prove. To prove *this* by ordinary induction, we first check  $R(0)$ , which is true (assuming  $k > 0$ ) because the implication is vacuously satisfied. Then we have to prove  $R(i) \rightarrow R(i+1)$ , which we can break into cases. If  $i+1 < k$ , then  $R(i+1)$  is true because it translates to  $[(i+1) \geq k] \rightarrow P(i+1)$  and this implication is also vacuously satisfied. If  $i+1 = k$ , then the implication is true because  $P(k)$  is true. And if  $i \geq k$ , the implication  $R(i) \rightarrow R(i+1)$  reduces to  $P(i) \rightarrow P(i+1)$  because the antecedents  $i \geq k$  and  $(i+1) \geq k$  are both true, and this latter implication is given to us. So the inductive step is proven, and we have proven  $\forall i : R(i)$  by ordinary induction.

- We can think of the data type `naturalAtLeastK` as being given by an recursive definition similar to the Peano axioms, with  $k$  replacing 0 in the base case. Something that is true for  $k$ , and true for  $i+1$  whenever it is true for any `naturalAtLeastK`  $i$ , must be true for all `naturalAtLeastK`'s. We will see that *any* recursive definition, with a final clause of the form “the only elements of the type are those given by these rules”, leads to an inductive proof method like this.

#### 4.4.2 Induction on the Odds or the Evens

Here's another example that deals with a different subset of the integers. Let  $P(n)$  be the statement “4 divides  $n^2 - 1$ ”, which is true for all odd numbers  $n$  (but which, as it happens, is false for all even numbers). A natural way to try to prove this is mathematical induction, with two modifications: start with  $P(1)$ , and in the inductive step show that  $P(n)$  implies  $P(n+2)$  rather than  $P(n+1)$ .  $P(1)$  is clearly true because 4 divides  $1^2 - 1 = 0$ . If we assume  $P(n)$ , that 4 divides  $n^2 - 1$ , we can do this new kind of inductive step by proving  $P(n+2)$ , that 4 divides  $(n+2)^2 - 1$ . Naturally we do this by finding the relationship between  $n^2 - 1$  and  $(n+2)^2 - 1$ :  $(n+2)^2 - 1 = (n^2 + 4n + 4) - 1 = (n^2 - 1) + 4(n+1)$ , so given that 4 divides one number it divides the other.

Just as in the example above of induction with a different starting point, we can justify this new induction method in several ways:

- If for every natural  $k$  we define the statement  $Q(k)$  to be  $P(2k+1)$ , and  $Q(k)$  holds for all naturals  $k$ , then  $P(n)$  holds for all odd  $n$ . But to prove  $\forall k : Q(k)$  by ordinary induction, we just prove  $Q(0) = P(1)$  and then  $\forall k : Q(k) \rightarrow Q(k+1)$ , which follows from  $\forall n : P(n) \rightarrow P(n+2)$ .
- If we let  $R(n)$  be “if  $n$  is odd, then  $P(n)$  is true”, can we then prove  $\forall n : R(n)$  by ordinary induction?  $R(0)$  is vacuously true, and we can prove  $R(1)$  by verifying  $P(1)$ , but in general we have a problem with  $R(n) \rightarrow R(n+1)$ . If  $n$  is even,  $R(n)$  is vacuously true, and the inductive hypothesis says nothing about  $P(n)$  or  $P(n-1)$  that we could use to prove  $P(n+1)$ . We'll get around this below by the technique of **strong induction**.
- We may inductively define the odd numbers by the Peano-like axioms “1 is an odd number”, “if  $n$  is an odd number, so is  $n+2$ ”, and so forth. Then this “odd-number induction” is justified from the new fifth axiom in the same way that ordinary induction is justified by the original Peano axioms.

### 4.4.3 Strong Induction

Our final extended version of mathematical induction is called **strong induction**. We saw above that in proving a statement for all odd numbers, ordinary induction gave us the *wrong inductive hypothesis*.  $P(n)$  was of no use in proving  $P(n+1)$  — we needed  $P(n-1)$  instead.

Here's the trick. Let  $P(n)$ , as usual, be a statement about naturals that we want to prove for all  $n$ . Define  $Q(n)$  to be the statement “ $P(i)$  is true for all  $i$  such that  $i \leq n$ ”. If we can prove  $Q(n)$  for all  $n$ , that certainly suffices (though it looks a bit strange to set ourselves the task of proving a stronger statement). So we set about proving  $\forall n : Q(n)$  by ordinary induction. The base case  $Q(0)$  is the same as  $P(0)$ . For the inductive step, we assume  $Q(n)$  and try to prove  $Q(n+1)$ . But if we can prove  $P(n+1)$ , that and  $Q(n)$  together give us  $Q(n+1)$  and we are done. So we have the rule:

- If you prove  $P(0)$ , and
- If you prove  $Q(n) \rightarrow P(n+1)$ , where  $Q(n)$  is the statement  $\forall i : (i \leq n) \rightarrow P(i)$ , then
- You may conclude  $\forall n : P(n)$ .

Note the something-for-nothing character of this rule! We have to do the same base step, but in the inductive step we have the same goal but the *stronger* inductive hypothesis  $Q(n)$  to work with. This is an example of the “boot-strapping” phenomenon mentioned in the last section — making your inductive hypothesis stronger may make the proof easier.

The way strong induction comes up in practice is that in the middle of a proof, you discover that to prove  $P(n+1)$ , what you really need instead of  $P(n)$  is some other  $P(i)$ . As long as  $i \leq n$ , you can just say that you're now using strong rather than ordinary induction and bring in  $P(i)$  as an assumption! The reason this is mathematically valid is that you *could* go and recreate this argument, using ordinary induction on this changed inductive hypothesis. As one example, we can finish our second justification of our proof of  $P(n)$  for all odd  $n$  above. We can now prove  $\forall n : R(n)$  by strong induction on  $n$ , using our hypothesis  $\forall n : P(n) \rightarrow P(n+2)$ . Assume  $R(i)$  for all  $i \leq n$  and try to prove  $R(n+1)$ . To be exact, we have to work by cases. If  $n = 0$ , we have to prove  $R(1)$  directly by verifying  $P(1)$ . If  $n$  is odd,  $n+1$  is even and thus  $R(n+1)$  is vacuously true. If  $n$  is even and  $n > 0$ , though, we need to prove  $P(n+1)$ . But our inductive hypothesis includes  $R(n-1)$ , which implies  $P(n-1)$  because  $n-1$  is also odd, and we can use our hypothesis substituting in  $n-1$  to get  $P(n-1) \rightarrow P(n+1)$  and thus  $P(n+1)$ .

Here's another example. Define the **size** of a natural to be the number of bits needed to represent it in binary. We can find this recursively, as 0 or 1 require one bit and in general,  $n$  requires one more bit than  $n/2$ :

```
natural size (natural n)
{ // Returns number of bits in binary representation of n
  if (n <= 1) return 1;
  return (1 + size(n/2)); }
```

Now suppose we want to prove that  $2^{\text{size}(n)}$  is always greater than  $n$ . We can start by verifying that  $2^{\text{size}(0)} = 2^1 = 2 > 0$  and  $2^{\text{size}(1)} = 2 > 1$ . But for the inductive case, knowing the size of  $n$  doesn't tell us anything about the size of  $n + 1$ , unless  $(n+1)/2$  happens to equal  $n$ . We'd like to be able to assume that  $\text{size}((n+1)/2)$  gives the right value. That's just what strong induction lets us do. The inductive hypothesis becomes " $2^{\text{size}(i)} > i$  for all  $i \leq n$ ", and since  $(n+1)/2$  is at most  $n$  for  $n > 1$ , we can compute (letting  $k$  equal  $(n+1)/2$ ):

$$2^{\text{size}(n+1)} = 2^{1+\text{size}(k)} = 2 \cdot 2^{\text{size}(k)} \geq 2 \cdot (k+1) > n+1.$$

There's a subtlety in the above sequence of inequalities. The natural thing to do with  $2^{\text{size}(k)}$  would be to observe that (by the inductive hypothesis) it is greater than  $k$ . But it's not necessarily true that  $2k \geq n+1$ , and in fact this is false whenever  $n+1$  is even. However, if an integer is greater than  $k$ , it is also greater than or equal to  $k+1$ , and  $2(k+1)$  is definitely greater than  $n+1$ .

#### 4.4.4 Exercises

- E4.4.1 In an algorithms course you will be expected to believe that "for sufficiently large" naturals  $n$ ,  $n^2/10$  is greater than  $47n$ . Find some particular number  $k$  such that this statement is true for all  $n \geq k$ , and prove that it is. (The best choice of  $k$  would be the smallest possible, of course.)
- E4.4.2 Repeat Exercise 4.4.1 for the statement " $2^n > 137n^3$ ".
- E4.4.3 Prove that if  $n \geq 4$ , it is possible to make up exactly  $n$  dollars with a combination of \$2 and \$5 bills. (**Hint:** Almost any of the ideas in this section can be used successfully. Strong induction is the easiest, but you could also prove it separately for odd and even  $n$ . You can also use ordinary induction with the starting point  $n = 4$ .)
- E4.4.4 Prove that if  $n$  is an odd number, then 8 divides  $n^2 + 7$ .
- E4.4.5 Prove by induction that the  $i$ 'th odd number is  $2i - 1$ , for all  $i > 0$ .
- E4.4.6 In the game of **1-2-3 Nim**, two players alternate moves in which they may take one, two, or three stones from a pile of  $n$  stones. The object of the game is to take the last stone. Prove, by strong induction on  $n$ , that the second player has a winning strategy<sup>18</sup> in the  $n$ -stone game if and only if  $n$  is divisible by 4. (We assume that if there are no stones, the second player wins because the first player cannot move. Of course if there are one, two, or three stones, the first player wins on the first move.)
- E4.4.7 Recall from Chapter 1 that in a deductive sequence proof, every statement is either an axiom (guaranteed to be true) or follows from one or more earlier statements. Prove, by strong induction on all positive naturals  $n$ , that the  $n$ 'th statement of a deductive sequence proof must be true.
- E4.4.8 Prove, by induction on all naturals  $n$ , that if an  $n$ -letter string over the alphabet  $\{a, b\}$  contains both an  $a$  and a  $b$ , then it contains either  $ab$  or  $ba$  (or both) as a substring. (**Hint:**

<sup>18</sup>In Section 9.10 we will show that in any of a large category of games, including this one, one player or the other has a strategy that leads to a win given any possible sequence of moves by their opponent.

The base cases for  $n \leq 2$  are easy. Assuming the statement  $P(n)$  for an arbitrary  $n$ , prove  $P(n+1)$  by cases, based on the last letter of the string.)

- E4.4.9 Let  $*$  be a binary operation on a set  $X$ , so that for any elements  $a$  and  $b$  of  $X$ , there is a unique element of  $X$  defined as  $a * b$ . Let  $n$  be any positive natural and let  $a_1, \dots, a_n$  be any sequence of  $n$  elements of  $X$ . Without any other assumptions on  $*$ , prove that if we apply parentheses in any way to the product  $a_1 * a_2 \dots * a_n$  to make it out of binary  $*$  operations, the result is in  $X$ .
- E4.4.10 In Section 3.5 we proved the full form of the Chinese Remainder Theorem from the Simple form, using what we can now recognize as a proof by induction. Write the full form as a statement  $P(k)$ , where  $k$  is the number of pairwise relatively prime moduli. Prove by induction on all positive naturals  $k$  that  $P(k)$  is true, assuming the simple form  $P(2)$ .

#### 4.4.5 Problems

- P4.4.1 Consider a variant of Exercise 4.4.3, for \$4 and \$11 bills (made, we might suppose, by a particularly inept counterfeiter). What is the minimum number  $k$  such that you can make up  $\$n$  for all  $n \geq k$ ? Prove that you can do so.
- P4.4.2 Give a rigorous proof, using strong induction, that every positive natural has at least one factorization into prime numbers.
- P4.4.3 Consider the following variant of the “recursive algorithm” form of the fifth Peano axiom:
- If an algorithm has one argument of type **natural**, it terminates when called with argument 0, and when called with argument  $x > 0$  it terminates except possibly for a call to itself with argument  $y$ , with  $y < x$ , then it eventually terminates for all input.

Prove that this rule is valid, using strong induction.

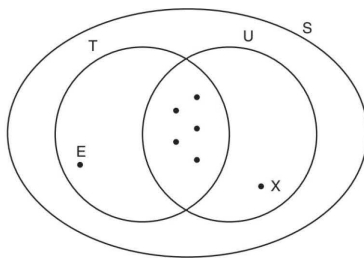
- P4.4.4 Prove the statement  $\forall x : [x > 0 \rightarrow \exists y : x = \text{successor}(y)]$ , which we used in the last section as part of our definition of the **natural** data type. Use induction on  $x$  starting with  $x = 1$ .
- P4.4.5 Find the flaw in the following alleged proof that you are the late Elvis Presley<sup>19</sup>.

*By mathematical induction, we will prove the following statement  $P(n)$  for all naturals  $n$ : “In any set  $S$  of  $n$  people, one of whom is an Elvis, all are Elvises.” (The conclusion that you are Elvis will then follow by taking  $S$  to be any set containing both you and the original Elvis.)*

*The base case  $P(0)$  is vacuously true because there is no such set of 0 people. The second base case  $P(1)$  is obviously true, because a set of one person containing an Elvis contains only Elvises. For the inductive step we need to show that for any  $n > 0$ ,  $P(n)$  implies  $P(n+1)$ . So assume that any set of  $n$  people containing an Elvis consists entirely of Elvises. Let  $S$  be an arbitrary set of  $n+1$  people, including an Elvis whom we’ll call  $E$ . Let  $x$  be an element of  $S$  other than  $E$ . Now look at*

<sup>19</sup>This is a standard example of a flawed induction proof (often phrased as a proof that all horses have the same color). It is really helpful for some people and totally useless for others. You need to first see why it *appears* to be a valid induction proof of an obviously false statement, and then find the *specific* flaw in the reasoning.





©Kendall Hunt Publishing Company

Figure 4-5: A Venn diagram for part of the Elvis proof.

$T = S \setminus \{x\}$ . It's a set of  $n$  people containing an Elvis (because it contains  $E$ ), so by the inductive hypothesis it consists entirely of Elvises. Now let  $U = S \setminus \{E\}$ .  $U$  has  $n$  elements, and it contains an Elvis because everyone in  $T$  (everyone in  $S$  except for  $x$ ) is an Elvis (see Figure 4-5). So using the inductive hypothesis again,  $U$  is all Elvises, so in particular  $x$  is an Elvis, and  $S$  is all Elvises as desired. The inductive step is done, so  $P(n)$  holds for all naturals  $n$ , and you are Elvis.

P4.4.6 I am starting a new plan for the length of my daily dog walks. On Day 0 we walk 3 miles, on Day 1 we walk 2 miles, and for all  $n > 0$  the length of our walk on Day  $n + 1$  is the average of the lengths of the walks on Days  $n - 1$  and  $n$ .

- Prove by strong induction for all naturals  $n$  that on Day  $n$ , we walk  $(7 + 2(-1/2)^n)/3$  miles. (**Hint:** Use base cases for  $n = 0$  and  $n = 1$ .)
- Give a formula for the total distance that we walk on days 0 through  $n$ , and prove your formula correct by strong induction.

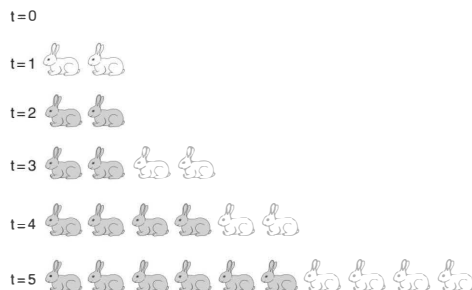
P4.4.7 A polygon is called **convex** if every line segment from one vertex to another lies entirely within the polygon. To **triangulate** a polygon, we take some of these line segments, which don't cross one another, and use them to divide the polygon into triangles. Prove, by strong induction for all naturals  $n$  with  $n \geq 3$ , that every convex polygon with  $n$  sides has a triangulation, and that every triangulation contains exactly  $n - 2$  triangles. (**Hint:** When you divide an  $n$ -gon with a single line segment, you create an  $i$ -gon and a  $j$ -gon for some naturals  $i$  and  $j$ . What does your strong inductive hypothesis tell you about triangulations of these polygons?)

P4.4.8 Pig Floyd is weighed at the beginning of every month. In Month 0 he weighs 400 kilograms, in Month 1 he weighs 350 kilograms, and in later months his weight  $W(n + 1)$  is equal to  $\sqrt{2}W(n) - W(n - 1) + 700 - 350\sqrt{2}$ .

- Calculate  $W(n)$  for all naturals  $n$  with  $n \leq 10$ . Write your answers in the form  $a + b\sqrt{2}$  where  $a$  and  $b$  are integers.
- Prove by strong induction on all naturals  $n$  that  $W(n)$  can be written in the form  $a + b\sqrt{2}$ , where  $a$  and  $b$  are integers.

- (c) Determine  $W(84)$ , Floyd's weight after seven years. You will find it easiest to discover a pattern in the numbers  $W(n)$ , and prove that this pattern holds for all  $n$  by strong induction.

- P4.4.9 Let  $*$  be a binary operation on a set  $X$  that is associative, meaning that for any three elements  $a$ ,  $b$ , and  $c$ , we have  $a * (b * c) = (a * b) * c$ . (We do not assume that  $*$  is commutative.) Let  $n$  be any positive natural and let  $a_1, a_2, \dots, a_n$  any sequence of  $n$  elements of  $X$ , not necessarily distinct. Prove that however we parenthesize the sequence " $a_1 * a_2 * \dots * a_n$ ", we get the same result. (**Hint:** Use strong induction on  $n$ . The cases of  $n = 1$  and  $n = 2$  are trivial, and  $n = 3$  is given by our assumption. Show that any parenthesization of  $a_1 * \dots * a_{n+1}$  is equal to some parenthesization of  $a_1 * \dots * a_n$  starred with  $a_{n+1}$ , then apply the inductive hypothesis.)
- P4.4.10 Let  $*$  be a binary operation on a set  $X$  that is commutative, meaning that  $a * b = b * a$  for any elements  $a$  and  $b$  of  $X$ , and associative, meaning that  $a * (b * c) = (a * b) * c$  for any elements  $a$ ,  $b$ , and  $c$  of  $X$ . (So we know from Problem 4.4.9 that we can write the product of any sequence of elements without parentheses.) Let  $n$  be any natural with  $n \geq 2$ , and let  $a_1, a_2, \dots, a_n$  be any sequence of  $n$  elements of  $X$ , not necessarily distinct. Let  $b_1, b_2, \dots, b_n$  be a sequence consisting of the same elements in another order. Prove that  $a_1 * a_2 * \dots * a_n = b_1 * b_2 * \dots * b_n$ . (**Hint:** Use strong induction on  $n$ .)



©Kendall Hunt Publishing Company

Figure 4-6: Fibonacci's rabbits. Shaded rabbits are breeding pairs.

## 4.5 Excursion: Fibonacci Numbers

In this Excursion we study the **Fibonacci numbers**, first described by Leonardo of Pisa in the 1200's. His original motivating problem involved population growth in rabbits. At time step one you begin with one pair of newborn rabbits. At every future time step, you have all the rabbits from the previous step (apparently they're immortal) plus possibly some newly born ones. The rule for births is that each pair of rabbits *except those born on the last step* produces a new pair. Conveniently, these are always one male and one female and the rabbits have no objections to mating with their close relatives. Figure 4-6 illustrates the first few stages of the process.

The number of pairs of rabbits at each time step  $n$  is called  $F(n)$  or “the  $n$ 'th Fibonacci number”, and is formally defined by the following recursive rules:

- $F(0) = 0$ .
- $F(1) = 1$ .
- For any  $n \geq 2$ ,  $F(n) = F(n-1) + F(n-2)$ .

It's immediate by strong induction on  $n$  that “ $F(n)$  is defined for any  $n$ ” (**Proof:** The base cases  $n = 0$  and  $n = 1$  are given to us by the definition, and if we know that  $F(n-2)$  and  $F(n-1)$  are defined then the third rule defines  $F(n)$  for us.) We can calculate  $F(2) = 1$  (this value is sometimes given as part of the definition),  $F(3) = 2$ ,  $F(4) = 3$ ,  $F(5) = 5$ ,  $F(6) = 8$ , and so forth.

The Fibonacci numbers are lots of fun to play with because it seems that if you do almost anything to the sequence, you get the same numbers back in some form. For example, the difference between  $F(n)$  and  $F(n+1)$  is just  $F(n-1)$ , from the third rule. If we let  $S(n)$  be the sum of  $F(i)$  as  $i$  goes from 1 to  $n$ , we get  $S(0) = 0$ ,  $S(1) = 1$ ,  $S(2) = 2$ ,  $S(3) = 4$ ,  $S(4) = 7$ ,  $S(5) = 12$ ,  $S(6) = 20$ , and so forth. Looking at the sequence, you might notice that  $S(n) = F(n+2) - 1$ , so that the summation

of the Fibonacci numbers gives more or less the Fibonacci numbers<sup>20</sup>.

As another curiosity, look at the squares of the Fibonacci numbers: 0, 1, 1, 4, 9, 25, 64, ... Nothing too obvious, but if we look at  $F(n)^2 - F(n-2)^2$ , starting from  $n = 2$ , we get 1, 3, 8, 21, 55, ... We can recognize all these as individual Fibonacci numbers, and in fact this sequence seems to contain every second Fibonacci number. We've spotted the identity

$$F(n)^2 - F(n-2)^2 = F(2n-2).$$

Is this always true, or just coincidence? With any such identity, the sensible way to proceed is to use induction. We verify the base cases and continue by assuming that the identity is true for  $n-1$  and proving it for  $n$ , usually using the key definition  $F(n) = F(n-1) + F(n-2)$ .

The inductive step in this particular case is a bit tough, though. The natural way to begin is to expand out  $F(n)^2 = F(n-1)^2 + F(n-2)^2 + 2F(n-1)F(n-2)$ . The first two terms relate to our inductive hypothesis, but the third gives us trouble. If we look at  $F(n)F(n-1)$ , we get a nice sequence (from  $n = 1$ ) 0, 1, 2, 6, 15, 40, 104, ... (Look at the differences of this sequence.) In fact this new sequence appears to *also* satisfy an identity just like the one for the squares:

$$F(2n-1) = F(n+1)F(n) - F(n-1)F(n-2)$$

.

The easiest way to prove this identity and the one above is to do them simultaneously. We assume *both of them* for the  $n-1$  and  $n-2$  cases, and prove them *both* for the  $n$  case. This is one of the two choices for a Writing Exercise below.

One more observation about the Fibonacci numbers is their relationship to the **Golden Ratio**. You may have heard of this ratio from its role in art<sup>21</sup> — there is a unique number  $\phi$  such that the ratio of one to  $\phi$  is the same as that of  $1+\phi$  to one (see Figure 4-7). By algebra, this  $\phi = (\sqrt{5}-1)/2$  or about 0.61, so the ratio is about 1.61 to one.

Once you get started with the Fibonacci numbers, the ratio of one to the next seems to approach this golden ratio fairly quickly. In Chapter 7 we'll see a general mathematical theory of how to solve recurrences and derive the equation

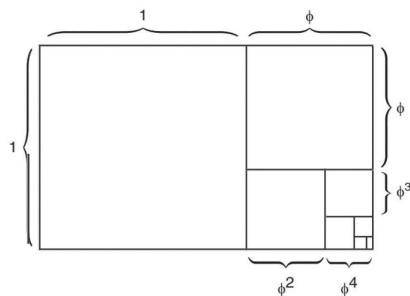
$$F(n) = \frac{1}{\sqrt{5}}[(1+\phi)^n - (-\phi)^n].$$

As  $\phi < 1$ , as  $n$  increases the  $(-\phi)^n$  terms gets smaller and smaller and the approximation  $F(n) = (1+\phi)^n/\sqrt{5}$  gets increasingly close.

Though we don't yet know how to derive this equation, we can prove it correct by induction on  $n$ , using the definitions of  $F(n)$  and  $\phi$  together with a fair bit of arithmetic.

<sup>20</sup>In Excursion 7.6 we'll look at analogies between sequences like the Fibonacci numbers and the functions occurring in calculus. When we define the operations appropriately, the Fibonacci numbers will turn out to be more or less their own "derivative" and "integral". Do other sequences besides the Fibonacci numbers relate to their own differences and sums in the same way?

<sup>21</sup>It is often claimed, for example, that the length and width of the Parthenon are in this ratio. This is apparently not true but many other things about this ratio are — see Mario Livio's book *The Golden Ratio: The Story of Phi, the World's Most Astonishing Number*.



©Kendall Hunt Publishing Company

Figure 4-7: The Golden Ratio.

**Writing Exercises:** For each statement, write a careful inductive proof that it is true for all naturals  $n$ , after an appropriate starting point.

- Both the formula

$$F(2n - 2) = F(n)^2 - F(n - 2)^2$$

and the formula

$$F(2n - 1) = F(n + 1)F(n) - F(n - 1)F(n - 2)$$

hold for  $n$ .

- Defining the number  $\phi$  to be  $\frac{\sqrt{5}-1}{2}$ ,

$$F(n) = \frac{1}{\sqrt{5}}[(1 + \phi)^n - (-\phi)^n].$$

## 4.6 Proving the Basic Facts of Arithmetic

### 4.6.1 The Semiring of the Naturals

There are a number of properties of arithmetic on the naturals that we tend to take for granted as we compute. Some of them, such as that  $x + 0 = x$  or  $x \cdot 0 = 0$ , were included in our formal definitions of the operations, but others such as  $x \cdot y = y \cdot x$  were not. The reason for this, as it turns out, is that we made the axioms and definitions *as short as possible*, leaving out any statements that could be *derived* from those already there.

Now that we have the technique of mathematical induction, we can carry out these derivations. Why bother to prove things that we already accept as true? For one thing, the existence of these proofs justifies the particular form of our definitions and gives us a small set of fundamental properties of the numbers from which all these other facts follow. For another, this task gives us some good practice in carrying out induction proofs, using a variety of proof strategies<sup>22</sup>.

In abstract algebra, the following properties of a system are called the **semiring axioms** and any system satisfying them is called a **semiring**<sup>23</sup>:

1. There are two binary operations  $+$  and  $\cdot$ , defined for all pairs of elements.
2. These operations are each **commutative**, so that  $\forall x : \forall y : (x + y) = (y + x)$  and  $\forall x : \forall y : (x \cdot y) = (y \cdot x)$ .
3. They are both **associative**, so that  $\forall x : \forall y : \forall z : (x + y) + z = x + (y + z)$  and  $\forall x : \forall y : \forall z : (x \cdot y) \cdot z = x \cdot (y \cdot z)$ .
4. There is an **additive identity**  $0$  such that  $x + 0 = 0 + x = x$ , and a **multiplicative identity**  $1$  such that  $1 \cdot x = x \cdot 1 = x$ . Also  $0 \cdot x = x \cdot 0 = 0$ .
5. Multiplication **distributes** over addition, so that  $\forall x : \forall y : \forall z : x \cdot (y + z) = x \cdot y + x \cdot z$ .

One of the biggest technical problems in constructing proofs of these properties is our tendency to assume that they are true and obvious *while we're trying to prove them*, which would be invalid circular reasoning. In particular, our standard notation for arithmetic sometimes assumes that addition and multiplication are associative, as we write  $x + y + z$  without specifying which operation is to be carried out first. For this section, you should think of arithmetic statements as being abbreviations for calls upon our formally defined functions, so that instead of  $x + y + z$  we need to say  $x + (y + z)$ , representing `plus(x, plus(y, z))`, or  $(x + y) + z$ , representing `plus(plus(x, y), z)`.

<sup>22</sup>The value of such a task is something for an individual instructor to assess, of course. Note that Hofstadter does much of the same work in his Chapter VIII, but our derivations here are considerably shorter because of the more informal proof style we have developed.

<sup>23</sup>It's called a "semiring" because these are only some of the properties of a full-fledged **ring** such as the integers. We gave the axioms for a ring in Section 3.8 — along with the semiring axioms a ring must have an **additive inverse** for every element, so that it satisfies the property  $\forall x : \exists y : x + y = 0$ . Actually, if you're keeping score, this is the definition of a **commutative semiring**, as most authors do not require the multiplication operation in a ring or semiring to be commutative. We'll encounter a number of other semirings later in the book.

We can't use the fact that these two calls return the same answer until we've proved it from the definitions.

We can go at these proofs in either of two ways. Just as a large programming problem is going to involve various subproblems, a large proof is going to involve various subproofs. We can get at this by either a **top-down** method, where we start out to do the whole thing and identify a good subproblem as something we need to finish, or a **bottom-up** method, where we decide in advance what a good subproblem might be. We'll use a mixture of the two to get experience of both<sup>24</sup>.

## 4.6.2 Properties of Addition

Remember that addition is defined by the rules  $x + 0 = x$  and  $x + S(y) = S(x + y)$ , using  $S(x)$  to represent the successor of  $x$ . (We don't want to use the notation " $x + 1$ " in this context because we want to distinguish between addition and the successor operation.) We want to show that the ring properties for addition follow from this definition of addition.

Let's begin bottom-up, by looking for one of our desired properties that ought to be easy to prove.  $\forall x : x + 0 = x$  is actually given to us, but what about  $\forall x : 0 + x = x$ ? It's a statement about all naturals  $x$ , so let's try induction on  $x$ . For the base case, we must show  $0 + 0 = 0$ , which follows from the  $\forall x : 0 + x = x$  rule by specifying  $x = 0$ . For the inductive step, we let  $x$  be arbitrary, assume  $0 + x = x$ , and try to prove  $0 + S(x) = S(x)$ . Expanding  $0 + S(x)$  by the definition of addition, we get  $S(0 + x)$ , and applying our inductive hypothesis inside the parentheses, we get  $S(x)$  as desired. So we've completed the inductive step and proved  $\forall x : 0 + x = x$ .

Now for a harder one, the commutativity of addition. Let's try to work top-down, and see where we get stuck. Write the desired property as  $\forall x : \forall y : (x + y) = (y + x)$ . We have a choice of induction or the Rule of Generalization, and we're going to take a particular choice: let  $x$  be arbitrary and use induction on  $y$ . (This is the easiest way it might work out, given that we don't have any immediate way to prove the inner statement  $\forall y : (x + y) = (y + x)$  without induction. Using induction only once for the innermost statement turns out to be the right idea in all of the examples in this section — the other variables will be able to "go along for the ride" as we vary the innermost one. If they couldn't, we would have to consider inducting on more than one variable.)

So we're trying to prove  $\forall y : (x + y) = (y + x)$ , with  $x$  arbitrary, by induction on  $y$ . The base case with  $y = 0$  turns out to be just the warmup exercise above! (We knew  $x + 0 = x$ , and we showed  $0 + x = x$ , so  $x + 0 = 0 + x$ .) How about the inductive step? We assume that  $x + y = y + x$  and try to prove that  $x + S(y) = S(y) + x$ . Well,  $x + S(y)$  is equal to  $S(x + y)$  by the definition of addition, and then equal to  $S(y + x)$  by the inductive hypothesis. The definition of addition again gets us to  $y + S(x)$ , rather than the  $S(y) + x$  we're looking for. Here is a subproblem that we can isolate and attack with another induction:

**Lemma:**  $\forall x : \forall y : S(y) + x = y + S(x)$ .

---

<sup>24</sup>Hofstadter is again worth reading on this point — he makes a nice analogy between finishing a subprogram and returning to a theme in a piece of music.

**Proof:** Actually we'd rather induct on  $x$  than  $y$ , because our definition tells us what to do with successor terms on the right of the addition, not the left. So, using the commutativity of universal quantifiers from Chapter 2, rewrite the whole thing as  $\forall y : \forall x : S(y) + x = y + S(x)$ , let  $y$  be arbitrary, and use induction on  $x$ . The base case is  $S(y) + 0 = y + S(0)$ . By the definition,  $y + S(0)$  is  $S(y + 0) = S(y)$ , which is also equal to  $S(y) + 0$ . For the inductive case, we assume  $S(y) + x = y + S(x)$  and try to prove  $S(y) + S(x) = y + S(S(x))$ . There are several ways we could go from here, but let's try working on the left-hand side. Applying the definition we get  $S(S(y) + x)$ , which is  $S(y + S(x))$  by applying the inductive hypothesis inside the parentheses. But then this is  $y + S(S(x))$ , as desired, by applying the definition again. ♠

Applying this lemma finishes the inductive step of the main proof, so we have proved  $\forall x : \forall y : (x + y) = (y + x)$ . Let's move on to the other main property of addition:

**Proposition:**  $\forall x : \forall y : \forall z : x + (y + z) = (x + y) + z$ .

**Proof:** Let  $x$  and  $y$  be arbitrary and use induction on  $z$ . For the base case, both  $x + (y + 0)$  and  $(x + y) + 0$  evaluate to  $x + y$  by using the definition. For the inductive step, we assume  $x + (y + z) = (x + y) + z$  and try to prove  $x + (y + S(z)) = (x + y) + S(z)$ . Again, the only move immediately available is to use the definition of addition. Working on the left-hand side, we get that  $x + (y + S(z))$  is equal to  $x + S(y + z)$  which is then  $S(x + (y + z))$  by using the same definition again. (Note that we have to be careful not to *assume* associativity during this argument!) By the inductive hypothesis, this is  $S((x + y) + z)$ . Using the definition in the other direction, we get  $(x + y) + S(z)$ , as desired. ♠

#### 4.6.3 Properties of Multiplication

The two rules defining multiplication are  $x \cdot 0 = 0$  and  $x \cdot S(y) = x \cdot y + x$ . In our proofs of the ring properties for multiplication, we use these rules and all the facts we have proven about addition. We begin with commutativity:

**Proposition:**  $\forall x : \forall y : x \cdot y = y \cdot x$ .

**Proof:** Again we'll let  $x$  be arbitrary and use induction on  $y$ . Just as for addition, the base case and the key move of the inductive step require separate inductions. We'll work bottom-up this time, proving these two lemmas and then using them to finish the main proof:

**Lemma:**  $\forall x : x \cdot 0 = 0 \cdot x$ .

**Proof:** Let  $x$  be arbitrary. We are given that  $x \cdot 0 = 0$ , and we prove  $0 \cdot x = 0$  for any  $x$  by induction on  $x$ . The base case is  $0 \cdot 0 = 0$ , which is given by the definition. For the induction, assume  $0 \cdot x = 0$ , and try to prove  $0 \cdot S(x) = 0$ . But  $0 \cdot S(x)$  is equal to  $(0 \cdot x) + 0$  by the definition, which is  $0 + 0$  by the inductive hypothesis, which is  $0$  by the definition of addition. ♠

**Lemma:**  $\forall x : \forall y : S(x) \cdot y = (x \cdot y) + y$ .



**Proof:** Let  $x$  be arbitrary and use induction on  $y$ . For the base case, both  $S(x) \cdot 0$  and  $(x \cdot 0) + 0$  evaluate to 0 using the definitions. For the inductive case, let  $y$  be arbitrary and assume that  $S(x) \cdot y = (x \cdot y) + y$ . We must show that  $S(x) \cdot S(y)$  is equal to  $(x \cdot S(y)) + S(y)$ . Working from the left-hand side,  $S(x) \cdot S(y)$  is equal to  $(S(x) \cdot y) + S(x)$  by the definition of multiplication. By the inductive hypothesis, this is  $((x \cdot y) + y) + S(x)$ . By associativity of addition we get  $(x \cdot y) + (y + S(x))$ , which by commutativity of addition is  $(x \cdot y) + (S(x) + y)$ , which by a lemma above is  $(x \cdot y) + (x + S(y))$ . This, by associativity of addition, is  $((x \cdot y) + x) + S(y)$ , which is  $(x \cdot S(y)) + S(y)$ , as desired, by the definition of multiplication. ♠

**Proof of Proposition:** (continued) The base case is exactly the statement of the first Lemma. For the inductive case, we assume  $x \cdot y = y \cdot x$  and want to show  $x \cdot S(y) = S(y) \cdot x$ . The left-hand side is equal to  $(x \cdot y) + x$  by the definition of multiplication, and the right-hand side is  $(y \cdot x) + x$  by the second Lemma, reversing the roles of  $x$  and  $y$ . This is equal to the desired  $(x \cdot y) + x$  by the inductive hypothesis. ♠

We're left with the associativity of multiplication and the distributive law. Making a guess as to which to do first, we try:

**Proposition:**  $\forall x : \forall y : \forall z : x \cdot (y \cdot z) = (x \cdot y) \cdot z$ .

**Proof:** Let  $x$  and  $y$  be arbitrary and use induction on  $z$ . For the base case we must prove  $x \cdot (y \cdot 0) = (x \cdot y) \cdot 0$ , and both sides evaluate to zero by the definition of multiplication. For the inductive step, we assume  $x \cdot (y \cdot z) = (x \cdot y) \cdot z$  and try to prove  $x \cdot (y \cdot S(z)) = (x \cdot y) \cdot S(z)$ . The left-hand side is equal to  $x \cdot ((y \cdot z) + y)$ . It looks like we may have made the wrong choice about which property to do first! If you succeed in proving the distributive law in Problem 4.6.2 below, we can go from this expression to  $(x \cdot (y \cdot z)) + (x \cdot y)$ , use the inductive hypothesis to get  $((x \cdot y) \cdot z) + (x \cdot y)$ , and finally use the definition of multiplication to get the desired  $(x \cdot y) \cdot S(z)$ . So remembering that we need that one more lemma, we're done. ♠

#### 4.6.4 Exercises

E4.6.1 Prove  $\forall x : x \cdot 1 = x$ , where 1 is defined to be  $S(0)$ . (**Hint:** Don't use induction.)

E4.6.2 Prove  $\forall x : 1 \cdot x = x$ , without using the commutativity of multiplication.

E4.6.3 The **subtraction operator** on naturals is defined recursively as well: For any natural  $x$ ,  $x - 0$  is  $x$  and  $x - S(y)$  is the predecessor of  $x - y$  unless  $x - y = 0$ , in which case  $x - S(y)$  is also 0. Prove by induction, using this definition, that  $(x + y) - y = x$  for any naturals  $x$  and  $y$ , and that  $(x - y) + y = x$  for any naturals  $x$  and  $y$  with  $x \geq y$ . (**Hint:** Use induction on  $y$ , letting  $x$  be arbitrary inside the inductive step to prove  $\forall x : (x + y) - y = x$  as  $P(y)$ .)

E4.6.4 Prove by induction, using the definition of natural subtraction in Exercise 4.6.3 above, that for any naturals  $x$ ,  $y$ , and  $z$ , we have  $x - (y + z) = (x - y) - z$ . (Remember that  $a - b$  is defined to be 0 if  $a < b$ .) (**Hint:** Let  $x$  and  $y$  be arbitrary and use induction on  $z$ .)

E4.6.5 Prove by induction, using the definition of natural subtraction in Exercise 4.6.3 above, that  $x \cdot (y - z) = x \cdot y - x \cdot z$  for any naturals  $x$ ,  $y$ , and  $z$ . (**Hint:** First prove  $\forall x : \forall w : (w > 0) \rightarrow x \cdot \text{pred}(w) = x \cdot w - x$ .)

- E4.6.6 Verify that the following finite structures are semirings, using the usual definitions of addition and multiplication:
- (a) The set  $\mathbf{Z}_m$  of integers modulo  $m$ , for any natural  $m$  with  $m > 1$ .
  - (b) The set  $T_k$  of “threshold numbers”, obtained by taking the naturals and considering all numbers  $k$  or greater to be the same number  $k$ . (With  $k = 5$  this is “rabbit arithmetic”).
- E4.6.7 Here is an unusual number system that is useful in optimization problems, as we will see later in Excursion 8.4. Consider the set of non-negative real numbers, together with an additional number  $\infty$  that is greater than any real number. If  $x$  and  $y$  are two such numbers, we define their “sum” to be the minimum of the two, and their “product” to be  $x + y$  under ordinary addition. Verify that this **min-plus semiring** obeys the semiring axioms.
- E4.6.8 Consider the set  $\{0, 1\}$  where the “sum” of two numbers is defined to be their boolean OR and their “product” is defined to be their boolean AND. Verify that this system obeys the semiring axioms.
- E4.6.9 Let  $S$  be any semiring and define  $S[x]$  to be the set of all polynomials in  $x$  with coefficients from  $S$ . Explain how the sum and product of two arbitrary polynomials is defined. Verify that with these operations,  $S[x]$  is a semiring.
- E4.6.10 Suppose that the binary relation  $A(x, y)$  on naturals satisfies the rules that  $A(0, 0)$  is true and for any naturals  $x$  and  $y$ ,  $A(x, y) \oplus A(Sx, y)$  and  $A(x, y) \oplus A(x, Sy)$  are both true. Prove that  $A$  is a reflexive relation.

## 4.6.5 Problems

- P4.6.1 The predicate “ $x \leq y$ ” on naturals  $x$  and  $y$  can be defined by the formula “ $\exists z : x + z = y$ ”.
- (a) Give an inductive definition of this predicate, by induction on both  $x$  and  $y$ . There are several possible ways to do this, but it turns out to be more useful to deal with the cases  $x = 0$  and  $y = 0$  separately, then define “ $Sx \leq Sy$ ” in terms of “ $x \leq y$ ”.
  - (b) Prove from your inductive definition that the relation  $\leq$  is a total order (see Section 2.10 for definitions). You may find it convenient to prove that your definition matches the  $\exists z : x + z = y$  definition and then work with that.
  - (c) Prove from your inductive definition that if  $x \leq y$ , then for any natural  $z$  both  $x + z \leq y + z$  and  $xz \leq yz$  are true. (**Hint:** Use induction on  $z$ . Some of the total order properties may be useful.)
- P4.6.2 (suitable for an Excursion) Prove

$$\forall x : \forall y : \forall z : x \cdot (y + z) = (x \cdot y) + (x \cdot z).$$

Which other properties from the section did you use? (We hope you didn’t use associativity of multiplication, since we needed the distributive law to prove that above.) Make a diagram of which properties were used in this section to prove which other properties, to make sure there’s no circular reasoning involved.

- P4.6.3 Define the following predicate  $C(a, b, m)$  for naturals  $a$  and  $b$  and positive naturals  $m$ . If  $a < m$ ,  $C(a, b, m)$  is true if and only if  $a = b$ . If  $b \geq m$ , then  $C(a, b, m)$  is false. If  $b < m$  and  $a \geq m$ , then  $C(a, b, m)$  is true if and only if  $C(a - m, b, m)$  is true. Prove that if  $b < m$ , then for any natural  $a$ ,  $C(a, b, m)$  is true if and only if  $\exists r : a = rm + b$  where  $r$  is a natural.
- P4.6.4 Define the predicate  $RP(a, b)$  for positive naturals  $a$  and  $b$  as follows.  $RP(a, b)$  is defined to be true if and only if one of the following is true:  $a = 1$ ,  $b = 1$ ,  $a > b$  and  $RP(a - b, b)$ , or  $a < b$  and  $RP(a, b - a)$ .
- Prove that  $RP(a, b)$  is true if and only if the Euclidean Algorithm from Section 3.3 returns 1 on inputs  $a$  and  $b$ . (Note that the definition implies that if  $a = b$  and  $a > 1$ , then  $RP(a, b)$  is false because none of the given conditions are true.)
  - Prove that  $RP(a, b)$  is true if and only if  $\neg \exists c : D(c, a) \wedge D(c, b) \wedge (c > 1)$  where  $D$  is the division predicate.
- P4.6.5 Define the exponentiation operator on naturals recursively so that  $x^0 = 1$  and  $x^{S(y)} = x^y \cdot x$ . Prove by induction, using this definition, that for any naturals  $x$ ,  $y$ , and  $z$ ,  $x^{y+z} = x^y \cdot x^z$  and  $x^{yz} = (x^y)^z$ .
- P4.6.6 Consider a set of boolean variables  $\{x_1, \dots, x_n\}$  and the  $2^n$  possible assignments of truth values to these variables. If  $f$  and  $g$  are two such assignments, define  $f + g$  to be the assignment  $h$  where  $h(x_i) = f(x_i) \vee g(x_i)$ , and define  $fg$  similarly in terms of  $\wedge$ . Prove that these two operators obey the semiring axioms.
- P4.6.7 (uses Java) In this problem we recursively define two more binary operators on naturals. Each operation is defined only if the second argument is nonzero. We define  $R(0, y)$  to be 0, and define  $R(S(x), y)$  to be  $S(R(x, y))$  unless  $S(R(x, y)) = y$ , in which case  $R(S(x), y) = 0$ . We define  $Q(0, x)$  to be 0, and define  $Q(S(x), y)$  to be  $Q(x, y)$  unless  $R(S(x), y) = 0$ , in which case  $Q(S(x), y) = S(Q(x, y))$ .
- Write recursive static pseudo-Java methods `natural r(natural x, natural y)` and `natural q(natural x, natural y)` to compute these two operations.
  - Compute the values  $Q(5, 2)$  and  $R(5, 2)$ , either using your method or working directly with the definitions.
- P4.6.8 Using the definition of the operators  $R$  and  $Q$  in Problem 4.6.7, prove the following facts by induction for any fixed positive natural  $y$ .
- For any natural  $x$ ,  $y(Q(x, y)) + R(x, y) = x$ .
  - For any natural  $z$ ,  $Q(z y, y) = z$  and  $R(z, y) = 0$ .
  - If  $x$  is any fixed positive natural, then for natural  $z$ ,  $Q(Q(z x y, x), y) = Q(z x y, x)$ .
- P4.6.9 Consider a semiring with exactly two elements. The axioms require one element to be 0 and the other to be 1, and define the values  $x + y$  and  $xy$  for *most* pairs of elements  $x$  and  $y$ . What are the choices available to us to remain consistent with the axioms? How many different semirings are there with two elements?
- P4.6.10 Following Problem 4.6.8, can you characterize the possible semirings with exactly three elements? (**Hints:** We can call the three elements 0, 1, and  $x$ . The multiplication is mostly

defined by the identity property of 1 and the absorbing property of 0, so that the only choice is the value of  $xx$ . There are exactly nine ways to define a commutative, associative addition operation on  $\{0, 1, x\}$  such that 0 is an identity. The remaining problem is to determine which combinations of suitable addition and multiplication operations satisfy the distributive law.)

## 4.7 Recursive Definition for Strings

### 4.7.1 Axioms for the Strings

We've now seen several examples of mathematical induction to prove statements about the naturals, which are defined recursively by the Peano axioms. In Section 4.4 we also saw that particular subsets of the naturals (such as the numbers greater than 3, or the odd numbers), subsets that have their own recursive definitions, admit their own induction schemes.

In fact *any* recursive definition of a data type gives us a way to form inductive proofs of statements about all elements of that type. Recall our other example of a recursively defined data type, the strings over a particular alphabet  $\Sigma$ :

- $\lambda$  is a string.
- If  $w$  is a string, and  $a \in \Sigma$ , then  $wa$  is a string.
- No string is formed by the above rule in two different ways: i.e, if  $wa = vb$ , then  $w = v$  and  $a = b$ .
- If  $u$  is any string, then if and only if  $u \neq \lambda$ ,  $u$  is equal to  $wa$  for a unique string  $w$  and letter  $a$ .
- The only strings are those obtained from  $\lambda$  using the second rule.

This is equivalent to the definition of strings we gave back in Chapter 1, but the earlier definition assumed that you knew what a sequence was and this doesn't. Now, just as for the naturals, let's formally define some operations on this new data type<sup>25</sup>.

To start with, the axioms tell us that certain operations exist, and we will need to use these in coding our new ones (just as we used `successor` and `pred` in defining the operations on the naturals). We'll have a boolean method `isEmpty` that will return true if and only its `string` argument is  $\lambda$ . Given a string  $w$  and a letter  $a$ , we know that  $wa$  is uniquely defined, by a function we'll call `append(w,a)`. The third and fourth rules tell us that given a string  $x \neq \lambda$ ,  $x$  is equal to  $wa$  for a unique  $w$  and  $a$ , which we'll call<sup>26</sup> `allButLast(x)` and `last(x)`. The functions `allButLast` and `last` throw exceptions if they are called with input  $\lambda$ .

<sup>25</sup>Again, this will involve formal proofs of some facts that are pretty obviously true. But we will see how all we know about strings follows from the simple definitions, and get some practice with induction proofs outside the usual setting of numbers. There will be several proofs in this format in Chapter 14.

<sup>26</sup>This notation is borrowed from the programming language POP-11 and will be convenient when we deal with finite-state machines in Chapter 14. A note for the Lisp-literate: If we chose to represent strings in Lisp as lists of atoms that were unbalanced to the left (Lisp usually does it the other way), the basic Lisp operations `car`, `cdr`, and `cons` would correspond exactly to `allButLast`, `last`, and `append`.

### 4.7.2 Defining the String Operations

We're now ready to define operations on strings. First, though, note that we are *not* working with the `String` class of Java, but with the mathematical `string` class we defined in Chapter 1<sup>27</sup>, for our pseudo-Java language. Our mathematical `string` objects will behave like Java primitives rather than objects, and we will define the operators to be static methods rather than instance methods<sup>28</sup>. The operations we define will be similar to the Java ones, but not identical. (For example, we can imagine arbitrarily long strings while the length of a `String` must be an `int`.)

That said, we define the **length** of a string  $w$ , written “ $|w|$ ” or “`length(w)`”, to be the number of letters in it. Formally, we can say that  $|\lambda| = 0$  and that for any  $w$  and  $a$ ,  $|wa| = |w| + 1$ . This definition immediately gives us a recursive algorithm:

```
natural length (string w)
{ // Returns number of letters in w
  if (isEmpty(w))
    return 0;
  else return (1 + length(allButLast (w)));}
```

In the same way, we can define the **concatenation operator**, which corresponds to the  $+$  operation on Java `String` objects. Here we define  $wx$  by recursion on the definition of  $x$ . We let  $w\lambda = w$ , and for  $x = ya$ , we let  $wx = (wy)a$ . In code:

```
string cat (string w, string x)
{ // Returns string made from w followed by x
  if (isEmpty(x))
    return w;
  else return append(cat (w, allButLast (x)), last (x));}
```

Note that when we write this pseudo-Java code, we have to resolve some ambiguities in the mathematical notation. When we write “ $wx = (wy)a$ ”, for example, we’re using the same notation to denote appending and concatenation, and if we left off the parentheses we’d be *assuming* that concatenation is associative, something we haven’t yet proved. (It is *true* that concatenation is associative, of course, but when we write the code we have to decide exactly which order of the operations is meant.)

**Reversing** a string is another example. Informally,  $w^R$  is  $w$  written backward. Formally,  $\lambda^R = \lambda$  and if  $w = xa$ ,  $w^R = ax^R$ . (Note that we need to use the concatenation operator in order to define reversal, because  $ax^R$  is a concatenation rather than an appending.) In pseudo-Java<sup>29</sup> code:

<sup>27</sup>Though note that the actual Java methods in Excursion 4.2 used the Java `String` class.

<sup>28</sup>In particular, we will test strings for equality with `==`, whereas with Java `Strings` it is possible for `u == v` to be false while `u.equals(v)` is true. Exercise 4.7.2 has you define the `==` operator from the other basic methods.

<sup>29</sup>We are assuming an implicit type cast from characters to strings when we give the character `last(w)` as an argument to `cat`.

```

string rev (string w)
{
  // Returns w written backward
  if (isEmpty(w))
    return emptystring;
  else return cat (last (w), rev (allButLast (w)))
}

```

### 4.7.3 Proof By Induction for Strings

In each case the operation is defined for all strings because the recursion is guaranteed to terminate, which in turn is because each recursive call is on a smaller string until eventually the relevant argument is the empty string. Mathematical induction<sup>30</sup> will then allow us to prove properties of these operators.

Specifically, if a set of strings contains  $\lambda$  and is closed under the operations of appending letters, it must consist of all the strings. So if  $P(w)$  is any statement with one free variable ranging over strings, we can use the following **Induction Rule For Strings**:

- Prove  $P(\lambda)$
- Prove  $\forall w : P(w) \rightarrow [\forall a : P(wa)]$ . Here the variable  $a$  ranges over letters. For *binary* strings, with alphabet  $\{0, 1\}$ , this has the special equivalent form  $\forall w : P(w) \rightarrow (P(w0) \wedge P(w1))$ .
- Conclude  $\forall w : P(w)$ .

Our definitions of the length, concatenation, and reversal functions have the property that for each letter  $a$ ,  $f(wa)$  is defined in terms of  $f(w)$  and  $a$ . This means that an inductive hypothesis telling us about  $f(w)$  will often be useful in proving things about  $f(wa)$ . We'll now see a number of examples of such proofs.

**Proposition:** For any strings  $u$  and  $v$ ,  $|uv| = |u| + |v|$ . (For example, in Figure 4-6  $|u|$  and  $|v|$  are each equal to 3 and  $|uv|$  is equal to 6. The figure shows an example of this rule in action.)

**Proof:** Let  $u$  be an arbitrary string and use induction on  $v$ . In the base case,  $v = \lambda$ ,  $|uv| = |u\lambda| = |u|$  (by definition of concatenation) and  $|u| + |v| = |u| + 0 = |u|$  by definition of length. For the inductive case, we may assume  $v = wa$  and, by the inductive hypothesis,  $|uw| = |u| + |w|$ . We must show  $|u(wa)| = |u| + |wa|$  (being careful not to assume results that might be implicit in our notation). By definition of concatenation,  $u(wa) = (uw)a$ , so  $|u(wa)| = |(uw)a| = |uw| + 1$  by definition of length, and  $= |u| + |w| + 1$  by the inductive hypothesis. Meanwhile,  $|u| + |wa|$  is also  $|u| + |w| + 1$  by the definition of length. ♠

**Proposition:** For any string  $w$ ,  $|w^R| = |w|$ .

<sup>30</sup>Induction on a recursive definition, when done on something other than naturals, is often called **structural induction**. All these techniques can still be justified from the single Principle of Mathematical Induction, of course, so whether to call this a "new proof technique" is a matter of taste.

|                         |                                       |
|-------------------------|---------------------------------------|
| $u = \text{"aab"}$      | $u^R = \text{"baa"}$                  |
| $v = \text{"bba"}$      | $v^R = \text{"abb"}$                  |
| $uv = \text{"aabbbba"}$ | $(uv)^R = \text{"abbbbaa"} = v^R u^R$ |

©Kendall Hunt Publishing Company

Figure 4-8: The reversal of the concatenation of two strings.

**Proof:** For the base case,  $|\lambda^R| = |\lambda| = 0$ . For the inductive step, we let  $w = va$  and assume  $|v^R| = |v|$ . By the definition of reversal,  $|w^R| = |(va)^R| = |a(v^R)|$ . This is  $|a| + |v^R|$  by the previous result, and this is equal to  $|a| + |v|$  by the inductive hypothesis. On the other hand,  $|w| = |va| = |v| + 1$  by the definition of length, and addition of naturals is commutative, so we have proved that  $|w^R| = |w|$ . Since we have completed the inductive step, we have completed the proof. ♠

**Proposition:** For any three strings  $x$ ,  $y$ , and  $z$ ,  $(xy)z = x(yz)$ .

**Proof:** We let  $x$  and  $y$  be arbitrary and use induction on  $z$ . If  $z = \lambda$ , both  $(xy)\lambda$  and  $x(y\lambda)$  are equal to  $xy$  by the definition of concatenation. For the inductive step, let  $z = wa$  and assume  $(xy)w = x(yw)$ . By successive application of the definition of concatenation, and one use of the inductive hypothesis, we get

$$(xy)z = (xy)(wa) = [(xy)w]a = [x(yw)]a = x[(yw)a] = x[y(wa)] = x(yz). \spadesuit$$

**Proposition:** For any strings  $u$  and  $v$ ,  $(uv)^R = v^R u^R$ . (See Figure 4-8 for an example.)

**Proof:** Again we let  $u$  be arbitrary and use induction on all strings  $v$ . For the base case,  $(u\lambda)^R$  and  $\lambda^R u^R$  are both equal to  $u^R$ . For the inductive case, we let  $v = wa$  and assume  $(uw)^R = w^R u^R$ . We have to determine what  $(uv)^R$  is, by determining how it relates to  $(uw)^R$ . Well,  $(uv)^R$  is  $(u(wa))^R$  (since  $v = wa$ ), which is equal to  $((uw)a)^R$  by the definition of concatenation. This in turn is equal to  $a(uw)^R$  by the definition of reversal, and is then  $a(w^R u^R)$  by the inductive hypothesis. If we can rewrite this as  $(aw^R)u^R$ , we are done because  $v^R = aw^R$  by the definition of concatenation. But we just proved the associativity of concatenation above. ♠

Another interpretation of the law of induction for strings is that a recursive program, that recurses on a single argument of type `string`, is guaranteed to terminate if (a) it doesn't call itself on input  $\lambda$ , and (b) it calls itself on input  $x$  only with argument `allButLast(x)`. There is a related form of "strong induction for strings", that would allow the program to call itself with any argument that is a prefix of  $x$ .

Note that we can also recursively define a *language*, like the balanced parenthesis language of Problems 4.7.6 and 4.7.7 below. As long as we have a rule that strings are in the language *only if* they can be produced by particular other rules, we have a similar inductive technique to prove that



all strings in the language have a particular property. We will see much more of this in Chapter 5 when we look at languages defined by regular expressions.

#### 4.7.4 Exercises

- E4.7.1 Prove from the string axioms that *aba* is a string.
- E4.7.2 (uses Java) Write a recursive (static) pseudo-Java method `boolean isEqual (string x, string y)` that returns `true` if and only if the strings (not Java Strings) *x* and *y* are the same string. Use only equality of letters and the predefined static methods from this section. Recall that these include a static boolean method `isEmpty (string w)` that determines whether *w* is the empty string – use this rather than using `==` on string values.
- E4.7.3 If *w* is a string in  $\{0,1\}^*$ , the **one's complement** of *w*, *oc(w)*, is the unique string, of the same length as *w*, that has a zero wherever *w* has a one and vice versa. So, for example, *oc(101) = 010*. Give a recursive definition of *oc(w)*, like the definitions in this section.
- E4.7.4 (uses Java) Write a recursive static pseudo-Java method `string oc (string w)` that returns the one's complement of a binary string, as defined in Exercise 4.7.3.
- E4.7.5 (uses Java) Write a static real-Java method to reverse a `String`. Do this first using a loop and the `charAt` method in the `String` class. Then write another, recursive version that uses only the concatenation operator `+` and the `substring` method.
- E4.7.6 If *u* and *v* are strings, we have defined *u* to be a **suffix** of *v* if there exists a string *w* such that *wu = v*. Write a recursive definition of this property like the ones in this section. (**Hint:** When is *u* a suffix of the empty string? If you know about suffixes of *v*, how do you decide about suffixes of *va*?)
- E4.7.7 (uses Java) Using the `isEmpty` and `allButLast` methods, write a recursive pseudo-Java static method `boolean isSuffix (string u, string v)` that returns `true` if and only if *u* is a suffix of *v* as defined in Exercise 4.7.6.
- E4.7.8 (uses Java) Often when you enter a password, what is displayed is not the password itself but a string of stars of the same length as the string you have entered. Given any string *w*, let *stars(w)* be this string of stars. Give a recursive definition of this *stars* function, and a recursive pseudo-Java static method computing it using the basic methods defined in this section.
- E4.7.9 (uses Java) If *u* is a string and *a* is a letter, give a recursive definition for the relation *contains(u, a)*, which is true if and only if *a* occurs at least once in *u*. Write a recursive pseudo-Java static method `boolean contains(string u, char a)` that decides this relation.
- E4.7.10 (uses Java) A string is defined to have a **double letter** if it contains a substring of the form *aa* where *a* is any letter in the alphabet. Write a recursive static pseudo-Java method `boolean hasDouble(string w)` that returns `true` if and only if *w* has a double letter. Use the basic methods given in the section.

## 4.7.5 Problems

P4.7.1 Prove by induction on strings that for any string  $w$ ,  $(w^R)^R = w$ .

P4.7.2 Prove by induction on strings that for any binary string  $w$ ,  $(oc(w))^R = oc(w^R)$ . (See Exercise 4.7.3 for the definition of one's complement.)

P4.7.3 The function `first` is defined to take one `string` argument and return the first letter of the string if there is one. (So `first(w)` has the same output as `w.charAt(0)`.) The pseudo-Java function `allButFirst` takes one `string` argument and returns the substring consisting of everything but the first letter. Both `first` and `allButFirst` should throw exceptions if called with  $\lambda$  as their argument.

- (a) Write recursive definitions for these two functions in terms of the `append` function.
- (b) (uses Java) Write pseudo-Java recursive static methods to calculate these two functions, using any or all of the primitives `isEmpty`, `append`, `last`, and `allbutLast`. Your method should be closely based on the recursive definition.

P4.7.4 (uses Java) Recall that in the `String` class in real Java, there are two functions both named `substring`. If  $i$  and  $j$  are naturals, `w.substring(i)` returns the substring of  $w$  obtained by deleting the first  $i$  characters. The two-argument function `w.substring(i,j)` returns the substring consisting of the characters with position numbers  $k$  such that  $i \leq k$  and  $k < j$ .

- (a) Define two pseudo-Java static methods named `substring` to operate on our `string` primitive data type. The first method should take a string  $w$  and a natural  $i$  and return `w.substring(i)` as defined above. It should throw an exception if  $i$  is negative or if  $i$  is greater than the length of  $w$ . The second should take a string  $w$  and two naturals  $i$  and  $j$  and return `w.substring(i,j)`. It should throw an exception if  $i$  is negative, if  $i > j$ , or if either  $i$  or  $j$  is larger than the length of  $w$ . Give recursive definitions of these two functions in terms of the basic operations on strings and naturals. Each method should throw an exception if one or both of the natural arguments is equal to or greater than the length of the string.
- (b) Prove by induction, using your definitions, that `cat (substring(w,0,i), substring(w,i)) == w` for all strings  $w$  and all naturals  $i$  such that  $i$  is less than or equal to the length of  $w$ .
- (c) Prove by induction similarly that `cat (substring (w,i,j), substring (w,j,k)) == substring (w,i,k)` for all strings  $w$  and all naturals  $i,j,k$  such that  $i \leq j \leq k$  and  $k$  is less than or equal to the length of  $w$ .

P4.7.5 (uses Java) Give a recursive definition, in terms of our given basic operations for pseudo-Java strings and naturals, of the following `charAt` function. Since strings are a primitive type in pseudo-Java, we must redefine `charAt` to take two arguments – if  $w$  is a string and  $i$  a natural, we define `charAt(w, i)` to be the character of  $w$  in position  $i$ , if any, where the first position is numbered 0. The function is undefined if there is no such character. (**Hint:** Your definition should have two cases, one for  $w = \lambda$  and one for  $w = va$ .)

Write a pseudo-Java recursive static method to calculate this `charAt` function, using your definition. Throw an exception if the function value is undefined.

P4.7.6 (uses Java) We can define the **balanced parenthesis language** using recursion. This is the set of sequences of left and right parentheses that are balanced, in that every left paren has a matching right paren and the pairs are nested properly. We'll use "*L*" and "*R*" instead of "(" and ")" for readability.

We define the language Paren by the following four rules<sup>31</sup>:

- (a)  $\lambda$  is in Paren.
- (b) If  $u$  is in Paren, then so is  $LuR$ .
- (c) If  $u$  and  $v$  are in Paren, then so is  $uv$ .
- (d) No other strings are in Paren.

Write a real-Java static method `isBalanced` that takes a `String` argument and returns a `boolean` telling whether the input string is in Paren. A non-recursive method is simpler.

P4.7.7 (hard) Another way to characterize the Paren language (defined in Problem 4.7.6 above) is by the following two properties: (1) the number of *L*'s and *R*'s in the string is equal, and (2) in any prefix of the string, the number of *L*'s is at least as great as the number of *R*'s. Prove, by induction on the definition of Paren, that every string in Paren has these two properties.

P4.7.8 (uses Java) Suppose we have a set of "good" strings, defined by a pseudo-Java method `boolean isGood(string w)` that decides whether a given string is good. We would like to know whether a given input string has any *substring* that is good. (We'll assume that the empty string is not good.)

- (a) Prove that a string  $w$  has a good substring if and only if either (1) it is itself good or (2) it can be broken into two substrings `substring(w, 0, i)` and `substring(w, i)` (using the syntax from Problem 4.7.4 above) such that one of these has a good substring.
- (b) Use this definition to write a recursive pseudo-Java method `boolean hasGoodSubstring(string w)` that returns `true` if and only if the input string has a good substring. Of course your method will call `isGood`.
- (c) Write another method that has the same output as that of part (b), but uses a loop instead of recursion.
- (d) Of the methods in parts (b) and (c), which do you think will run faster in general?

P4.7.9 (uses Java) Here is a recursive pseudo-Java method which purports to count the good substrings in a given input string, in the context of Problem 4.7.8. Is it correct? If so, argue why, and if not, write a psuedo-Java method (not necessarily recursive) that is correct.

```
public static int countGood(string w) {
    int c = 0;
    for (i = 0; i < length(w), i++)
        if (isGood(w)) c++;
    c += (countGood(substring(w, 0, i) +
        substring(w, i));}
```

---

<sup>31</sup>Real programming languages have formal definitions like this, called **grammars** ("Backus-Naur form" is a common format for language definition that is more or less the same thing.) We'll revisit grammars in Chapter 15. There are general techniques to take such a language definition and create a recursive algorithm to parse a string. There are even "compiler-compilers" that will take a language definition and generate a whole compiler!

P4.7.10 (uses Java) Here is a recursive pseudo-Java method:

```
public static boolean eis (string u, string v) {
    if (isEmpty(u)) {
        if (isEmpty(v)) return true;
        if (last(v) == ' ') return eis(u, allButLast(v));
        return false;}
    if (isEmpty(v)) {
        if (last(u) == ' ') return eis(allButLast(u), v);
        return false;}
    if (last(u) == ' ') return eis(allButLast(u), v);
    if (last (v) == ' ') return eis(u, allButLast(v));
    if (last(u) == last(v)) return eis(allButLast(u), allButLast(v));
    return false;}
```

- (a) Explain what property of the strings  $u$  and  $v$  is decided by this method. Justify your answer.
- (b) For each of the calls to `last` and `allButLast` in the code, explain why the argument of the call is a nonempty string.
- (c) Prove carefully, by induction on strings, that the method returns the result you claimed in part (a).

## 4.8 Excursion: Naturals and Strings

Induction is closely related to recursion in several ways. In proving a recursive algorithm correct, it's natural to use induction on the argument as in our examples earlier. Here our predicate  $P(n)$ , in the case where the argument to the algorithm is a `natural`, might be "the algorithm terminates with the correct output on input  $n$ ". If the algorithm calls itself only with an argument that is the predecessor of the original argument, you can complete the inductive step by assuming the correctness of the algorithm for input  $n$  and verifying it for input  $n + 1$ . The base step is generally explicit in the algorithm. This method is particularly useful for proving that a recursive algorithm explicitly follows a recursive definition.

If the recursive algorithm calls itself with arguments smaller than the original one, though not necessarily just the predecessor of the original argument, we can use strong induction (as in Section 4.4). For the inductive step, we would assume the correctness of the algorithm for all inputs  $i \leq n$ , and then prove correctness for input  $n + 1$ .

In this Excursion we are going to look at two fundamental functions, one that converts binary strings to naturals, and the other that converts naturals to binary strings. (Are these two functions actually inverses of one another?) We'll begin with the recursive definitions of how a string represents a number and vice versa.

- The string  $\lambda$  represents the natural 0.
- If  $w$  represents  $n$ , then  $w0$  represents  $2n$  and  $w1$  represents  $2n + 1$ .
- The natural 0 is represented by the string 0.
- The natural 1 is represented by the string 1.
- If  $n > 1$ , we divide  $n$  by two, let  $w$  represent the quotient (Java `n/2`), let  $a$  represent the remainder (Java `n%2`), and represent  $n$  by  $wa$ .

A few examples (see Figure 4-9) should convince you that these definitions correspond to the usual representation of naturals as binary strings. For one example, the representation of 7 is that of 3 followed by a one, that of 3 is that of 1 followed by a one, that of 1 is a one by the base case, giving us 111, the correct binary for 7.

So now we try to code these up as pseudo-Java methods, given our standard procedures for both naturals and strings (Again, recall that we are using our mathematical `string` primitive type rather than the Java `String` class):

|                                   |                                             |
|-----------------------------------|---------------------------------------------|
| <code>rep(7)</code>               | <code>value("111")</code>                   |
| <code>= rep(3) · "1"</code>       | <code>= 2 × value("11") + 1</code>          |
| <code>= rep(1) · "1" · "1"</code> | <code>= 2 × (2 × value("1") + 1) + 1</code> |
| <code>= "1" · "1" · "1"</code>    | <code>= 2 × (2 × 1 + 1) + 1</code>          |
| <code>= "111"</code>              | <code>= 7</code>                            |

©Kendall Hunt Publishing Company

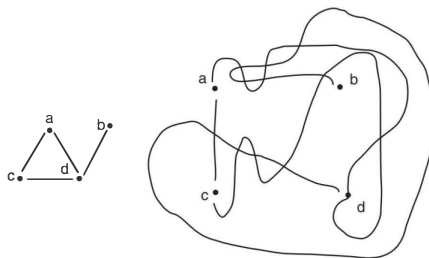
Figure 4-9: The functions from naturals to strings and vice versa.

```
static natural value (string w)
{
  // Returns natural number value of the given binary string.
  if (isEmpty(w)) return 0;
  string abl = allButLast(w);
  if (last(w) == '0')
    return 2 * value (abl);
  else return (2 * value (abl)) + 1;
}

static string rep (natural n)
{
  // Returns canonical binary string representing the given natural.
  if (n == 0) return "0";
  if (n == 1) return "1";
  string w = rep (n/2);
  if (n%2 == 0)
    return append (w, '0');
  else return append (w, '1');
}
```

**Writing Exercise:** Give a clear and convincing argument (using induction) that these algorithms are correct. Specifically:

1. Show by induction for all binary strings  $w$  that `value(w)` terminates and outputs the correct natural according to the definitions.
2. Show by (strong) induction for all naturals  $n$  that `rep(n)` terminates and outputs the correct string according to the definitions. You will need two separate base cases for  $n = 0$  and  $n = 1$ .



©Kendall Hunt Publishing Company

Figure 4-10: An undirected graph, drawn in two different ways.

## 4.9 Graphs and Paths

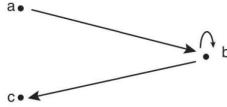
### 4.9.1 Types of Graphs

Our next examples of recursive definitions will take us into the realm of **graph theory**. We met diagrams of dots, connected by lines or arrows, in Chapter 2 as a pictorial representation of binary relations. You've probably run into several other similar diagrams to model other situations in computer science. What we're going to do now is to formally define some mathematical objects that can be represented by such diagrams, in such a way that we'll be able to prove facts about them. This will be only a brief introduction — we'll return to graph theory in Chapters 8 and 9.

- An **undirected graph** (Figure 4-10) is a set of points, called **nodes** or vertices<sup>32</sup>, and a set of lines, called **edges**. Each edge has two **endpoints**, which are two distinct nodes. No two edges have the same pair of endpoints. Furthermore, the only aspect we care about in an undirected graph is which pairs of nodes have endpoints — the binary **edge predicate**  $E(x, y)$  on nodes, meaning “there is an edge between node  $x$  and node  $y$ ”. If two graphs have the same edge predicate, we consider them to be equal although they might be drawn to look very different.
- A **directed graph** (see Figure 4-11) is a set of nodes together with a set of **directed edges** or **arcs**. Each arc is an arrow *from* one node *to* another<sup>33</sup>. No two arcs may have both the same start node and the same end node. The directed graph may also be represented by its edge predicate —  $E(x, y)$  meaning “there is an arc from node  $x$  to node  $y$ ”, and two directed graphs with the same edge predicate are considered to be equal. We can think of an undirected graph as a directed graph if we like, where each edge between  $x$  and  $y$  is viewed as two arcs, one from  $x$  to  $y$  and one from  $y$  to  $x$ .
- We'll also eventually see both directed and undirected **multigraphs**, which are like graphs except that more than one edge or arc might have the same endpoints (see Figure 4-12).

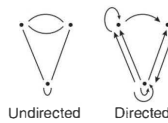
<sup>32</sup>The singular of “vertices” is “**vertex**”.

<sup>33</sup>Actually we also allow an arc from a node to itself, in which case it is also called a **loop**.



©Kendall Hunt Publishing Company

Figure 4-11: A directed graph.



©Kendall Hunt Publishing Company

Figure 4-12: Undirected and directed multigraphs.

- Also later, we'll see graphs where the nodes and/or the edges are **labeled** — associated with some other data item. Labeled graphs are a useful data structure to model all sorts of situations. For example, a labeled directed graph might have nodes representing airports, arcs for possible flights from one airport to another, and labels on the arcs for the departure time, price, or length of the flight (see Figure 4-13).

### 4.9.2 When are Two Graphs the Same?

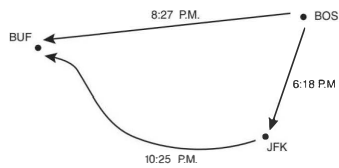
Figure 4-14 shows two different directed graphs, one with vertex set  $\{a, b, c\}$  and the other with vertex set  $\{x, y, z\}$ . Clearly these graphs are not **equal** or **identical**, because to be identical two graphs must have the same vertex set and the same edge predicate. However, there is a sense in which these two graphs are “the same graph”, and we will now make this notion precise.

In Chapter 3 we spoke of algebraic structures, such as rings, being **isomorphic**. The definition of “isomorphic” is specific to a particular class of structures<sup>34</sup>, such as rings or undirected graphs. In general an **isomorphism** from one structure to another is a bijection of the base elements of the structures, which satisfies additional rules that preserve the essential properties of the structure.

A set is a collection of elements with no other structure, and so an isomorphism of sets is just a bijection. We say that two sets are isomorphic if there exists a bijection between them, which as

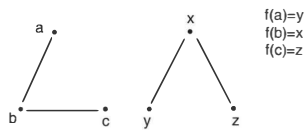
<sup>34</sup>A branch of mathematics called **category theory** starts with a formal definition of these “classes of structures”, and studies the properties that are common to all of them.





©Kendall Hunt Publishing Company

Figure 4-13: A directed graph, labeled with departure times.



©Kendall Hunt Publishing Company

Figure 4-14: Two isomorphic directed graphs.

we have seen occurs exactly when the two sets have the same size if they are finite<sup>35</sup>.

We defined a ring to be a set with addition and multiplication operations that obeyed various laws. An isomorphism  $f$  from a ring  $R$  to another ring  $S$  is a bijection of the elements that **preserves** the two operations, so that we always have  $f(x + y) = f(x) + f(y)$  and  $f(xy) = f(x)f(y)$ . The definitions of isomorphism for other algebraic structures are similar.

With each type of graph we have defined, an isomorphism from one graph  $G$  to another  $H$  is a bijection  $f$  from the vertices of  $G$  to the vertices of  $H$  that preserves the edge predicate. That is, for any two vertices  $x$  and  $y$  we must have  $E_G(x, y) \leftrightarrow E_H(f(x), f(y))$ . In our example in Figure 4-14, we can find an isomorphism  $f$  with  $f(a) = y$ ,  $f(b) = x$ , and  $f(c) = z$ . We can check that for every choice of two vertices, the isomorphism condition holds.

What does it mean for two graphs to be isomorphic? Suppose we redraw  $H$  with new vertex labels, so that every vertex  $x$  is now labeled with  $f^{-1}(x)$ . (The function  $f$  must have an inverse because it is a bijection.) We now have a graph with the same vertex set as  $G$ , and the same edge predicate as  $G$ , so this graph is identical to  $G$ ! Thus another way to say “ $G$  and  $H$  are isomorphic” is to say “the vertices of  $H$  can be relabeled to make a graph identical to  $G$ ”.

In Exercise 4.9.6 you'll prove that isomorphism is an equivalence relation, and it follows that graphs are divided into equivalence classes. This raises the natural problem of **classifying** the graphs with certain properties. Clearly all the graphs in a class have the same number of vertices, because otherwise we could not have a bijection of vertices, much less an isomorphism. In Exercise 4.9.7 you'll show that the number of edges in an undirected graph is a property of an equivalence class. For undirected graphs with three nodes this property is enough to determine the class of a graph, but for four or more nodes it is not.

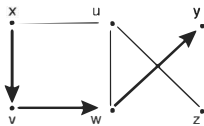
If two graphs *are* isomorphic, there is a simple proof of this fact, by giving an example of an isomorphism. But if they *are not* isomorphic, this may be harder to show. Of course in principle we could check each of the possible bijections of the vertices and see that none of them are isomorphisms, but this gets impractical very quickly as the number of vertices increases. Problem 4.9.7 defines a property of undirected graphs that is preserved by isomorphism, so that if two graphs differ with respect to this property they are not isomorphic. In general, we prove non-isomorphism by assuming that an isomorphism exists and then deriving consequences of that assumption, until we eventually reach a contradiction.

### 4.9.3 The Path Predicate

These definitions are in general *not* recursive, though we could come up with various different recursive definitions of these concepts if we wanted<sup>36</sup>. However, once we have a directed graph, there is an important relation that *is* clearly built up in one particular way and thus has a clear

<sup>35</sup>We'll look briefly at bijections among infinite sets in Chapter 7.

<sup>36</sup>Each such definition would correspond to a way of building up a graph, such as one vertex at a time with all its edges, or one edge at a time starting with just vertices. But since there isn't a single obvious way to build up a graph, there isn't a single obvious recursive definition. We won't go further into this now, because we're saving most of the graph theory in the book for Chapter 8.



©Kendall Hunt Publishing Company

Figure 4-15: A path from  $x$  to  $y$  in an undirected graph.

recursive definition. If we view an arc from  $x$  to  $y$  as saying “we can go from  $x$  to  $y$ ”, then it’s natural to wonder what trips we might take by using sequences of edges. Thus we define a **path** to be a sequence of zero or more arcs in which the destination node of each arc is the source node of the following arc (see Figure 4-15). This is an informal definition similar to our first definition of strings as sequences of letters, but we can easily turn it into a formal recursive definition.

Actually we need to define two things, the **path relation**  $P(x, y)$ , which corresponds to the **path predicate** “there is a path from  $x$  to  $y$ ”, and the paths themselves, which we’ll denote by Greek letters. Also, we’ll give the name “ $(x, y)$ ” to the arc from node  $x$  to node  $y$ , if it exists.

- For any  $x$ ,  $P(x, x)$  and  $\lambda$  is a path from  $x$  to  $x$ . (There is always a path, of length zero, from any vertex to itself, *whether or not* there is a loop on that vertex. )
- $P(x, y) \wedge E(y, z) \rightarrow P(x, z)$ . Specifically, if  $\alpha$  is a path from  $x$  to  $y$  and  $(y, z)$  is an edge, there is a path  $\beta$  from  $x$  to  $z$  consisting of  $\alpha$  followed by  $(y, z)$ . (We can make a path by taking an existing path and adding any edge that starts where the path ends.)
- All paths can be constructed in this way.

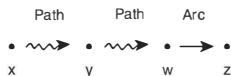
This allows us to prove statements about paths in a graph. Let’s begin with an important and obvious fact, that the path relation is transitive — you can make a path by following first one path, then another that finished where the first one starts.

**Transitivity Theorem:** If there is a path from  $x$  to  $y$ , and a path from  $y$  to  $z$ , then there is a path from  $x$  to  $z$ .

**Proof:** We’ll use induction on the second path, using the recursive definition<sup>37</sup>. Let  $x$  and  $y$  be arbitrary vertices such that there is a path  $\alpha$  from  $x$  to  $y$ . For the base case, suppose that the path from  $y$  to  $z$  has no edges. Then  $y = z$  and  $\alpha$  is also the desired path from  $x$  to  $z$ . For the induction, suppose that there is a path  $\beta$  from  $y$  to some  $w$ , an edge  $(w, z)$ , and that the inductive hypothesis applies to  $\beta$  (see Figure 4-16).

Then that inductive hypothesis tells us that there is a path  $\gamma$  from  $x$  to  $w$ , and the inductive part of the definition tells us that  $\gamma$ , followed by  $(w, z)$ , is a path from  $x$  to  $z$ . Because of the last clause

<sup>37</sup>We’ve had enough practice with recursive definitions by now to figure out the proof method straight from the definition.



©Kendall Hunt Publishing Company

Figure 4-16: A diagram for the inductive case of the proof.

of the definition, the base step and induction step of this proof covers all possible paths from  $y$  to  $z$ . ♠

The definition of paths is “bottom-up” rather than “top-down”. It allows us to show that a path exists if it does, but it doesn’t give us any immediate way to *decide* the path relation for some particular vertices  $x$  and  $y$ . This would mean determining whether  $P(x, y)$  is true or false for given input nodes  $x$  and  $y$ . This important computational problem is also called **finding a transitive closure**, because  $P$  is the smallest relation that includes  $E$  and is transitive<sup>38</sup>.

We can use the notion of paths to define various properties of both undirected and directed graphs. (We define paths in an undirected graph by viewing it as a directed graph, as described above.) For example, an undirected graph is said to be **connected** if  $\forall x : \forall y : P(x, y)$ ; that is, if there is a path between any two nodes. A directed graph, on the other hand, is said to be **strongly connected** if it has this property.

A **cycle** is a “non-trivial” path from a node to itself. Here “non-trivial” refers to paths that are not always guaranteed to exist, and the meaning of this depends on the context. Of course we don’t want to count the zero-length path from any node to itself. In a directed graph, that’s the only restriction, so that any path of one or more edges from a node to itself is called a **directed cycle**. In an undirected graph, any edge forms a directed cycle, because you can go from one endpoint over the edge to the other, and then back again. So there we define an **undirected cycle** to be a path of three or more edges from a node to itself that never reuses an edge.

An undirected graph with no undirected cycles is called a **forest**. The reason for this (which we’ll have to take on faith for the moment) is that such a graph can be divided into **trees**, which are connected forests. This is only one of a number of related concepts called **trees** – we will see another kind of tree in the next section and explore trees in much more detail in Chapter 9.

#### 4.9.4 Exercises

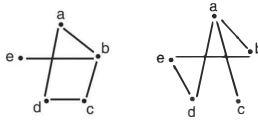
- E4.9.1 Draw directed graphs representing the equality, order and universal relations on the set  $\{1, 2, 3, 4, 5\}$ .
- E4.9.2 Any binary relation on a single set can be thought of as the edge relation for a directed graph. But only some relations could be the edge relation of an undirected graph — which ones? (See the following Exercise 4.9.3 for reminders about useful terminology.)

<sup>38</sup>In Chapter 8 we’ll present two different algorithms to compute transitive closures.

- E4.9.3 We defined several properties of binary relations in Section 2.8: reflexive, anti-reflexive, symmetric, anti-symmetric, and transitive. Describe the directed graphs of relations that have each of these properties. What does the graph of an equivalence relation look like? A partial order? How does the latter compare to the Hasse diagram from Section 2.10?
- E4.9.4 Prove that any non-empty path has a first edge. That is, if  $\alpha$  is a path from  $x$  to  $y$  and  $\alpha \neq \lambda$ , then there exists an edge  $(x, w)$  and a path  $\beta$  from  $w$  to  $y$  such that  $\alpha$  is  $(x, w)$  followed by  $\beta$ . (**Hint:** Use induction on  $\alpha$ .)
- E4.9.5 Explain why the path predicate  $P(x, y)$  on vertices is an equivalence relation on undirected graphs, but not in general on directed graphs. Prove that the relation  $P(x, y) \wedge P(y, x)$  is always an equivalence relation on any directed graph. Is the same true of  $P(x, y) \vee P(y, x)$ ? Prove your answer.
- E4.9.6 Show that isomorphism of directed graphs is an equivalence relation. Is it an equivalence relation for undirected graphs?
- E4.9.7 One important property of an undirected graph is its number of edges.
- Prove that if two undirected graphs are isomorphic, then they have the same number of edges.
  - Prove that if two undirected graphs each have three vertices and each have the same number of edges, then they are isomorphic.
  - Find two undirected graphs, each with four vertices and with the same number of edges, that are not isomorphic. Prove that there is no isomorphism between your graphs.
- E4.9.8 Let  $G$  and  $H$  be two isomorphic undirected graphs.
- Prove that if  $G$  is connected, then so is  $H$ .
  - Prove that if  $G$  is a forest, then so is  $H$ .
  - Prove that if  $G$  is a tree, then so is  $H$ .
- E4.9.9 Consider all possible directed graphs with two vertices. If we call the vertices  $a$  and  $b$ , there are exactly  $2^4 = 16$  such graphs, because there are four possible arcs and we choose whether each one is present. How many equivalence classes do these 16 graphs form under isomorphism?
- E4.9.10 Let  $G$  be a directed graph with  $n$  vertices, where  $n > 1$ . Prove that if  $G$  has a path with  $n$  edges, then it must contain a directed cycle.

#### 4.9.5 Problems

- P4.9.1 Prove formally that if  $\alpha$  is a path from  $x$  to  $y$  in an *undirected* graph, then there is a path from  $y$  to  $x$ . (**Hint:** Use induction on paths, of course, and use the Transitivity Theorem from this section.)
- P4.9.2 Prove that any directed cycle in the graph of a partial order must only involve one node. (**Hint:** If the cycle were to contain two distinct nodes  $x$  and  $y$ , what does transitivity tell you about arcs between  $x$  and  $y$ ?)



©Kendall Hunt Publishing Company

Figure 4-17: Two five-vertex graphs with the same degree sequence.

- P4.9.3 Give three different-looking (*i.e.*, not isomorphic) examples of a forest with five nodes and three edges. What do they have in common?
- P4.9.4 In Section 2.10 we proved that every partial order is the “path-below” relation of a graph called a Hasse diagram. How does the Hasse diagram relate to the graph of the partial order itself? Present the proof of the Hasse Diagram Theorem using mathematical induction.
- P4.9.5 In Exercise 4.9.5 you were asked to prove that if  $P(x, y)$  is the path predicate of any directed graph, then the predicate  $P(x, y) \wedge P(y, x)$  is an equivalence relation. The equivalence classes of this relation are called **strongly connected components**. Prove that a graph has no strongly connected components with more than one element if and only if it has no directed cycle with more than one node. Prove that if the graph has no such strongly connected component or cycle with more than one element, then its path relation is a partial order.
- P4.9.6 (uses Java) Implement a (real) Java `Path` class for directed graphs as follows. Assume that `Arc` and `Vertex` classes have already been defined, such that these objects represent directed edges and vertices in some directed graph. The `Arc` class has `source` and `destination` methods, each of which return a `Vertex`. Your `Path` class should support the following instance methods:

```
Vertex source() // first vertex in path
Vertex destination() // last vertex in path
boolean isTrivial() // true whenever path has no edges
int length() // number of edges in the path
Edge last() // last edge in path, throws exception if path is trivial
Path append(Arc a) // returns new path with a appended to calling path
// throws exception if a cannot be appended
```

- P4.9.7 In an undirected graph, the **degree** of a node is the number of edges that involve it. The **degree sequence** of an undirected graph with  $n$  vertices is a sequence of  $n$  naturals that gives the degrees of each vertex, sorted in descending order. Figure 4-17 shows two undirected graphs, each of which has degree sequence  $\langle 3, 2, 2, 2, 1 \rangle$  because each has one vertex of degree 3, three of degree 2, and one of degree 1.

- Are these two graphs isomorphic? Prove your answer.
- Prove that if two graphs are isomorphic, then they have the same degree sequence.
- Is it true that if two graphs have the same degree sequence, then they are isomorphic? Prove your answer.

P4.9.8 The **length** of a path in a directed graph is the number of edges in it.

- (a) Give a recursive definition of length, based on the recursive definition of paths in this section.
- (b) Let  $\alpha$  be a path from  $x$  to  $y$ ,  $\beta$  be a path from  $y$  to  $z$ , and  $\gamma$  be the path from  $x$  to  $z$  guaranteed by the Transitivity Theorem of this section. Prove that the length of  $\gamma$  is the length of  $\alpha$  plus the length of  $\beta$ . (Let  $\alpha$  be an arbitrary path and use induction on all paths  $\beta$ , as in the proof of the Transitivity Theorem.)

P4.9.9 Consider a directed graph where each edge is labeled by a natural. We define the **length** of a path in such a graph to be the sum of the edge weights over all edges in the path.

- (a) Give a recursive definition of this notion of the length of a path, using the recursive definition of paths from this section.
- (b) If  $\alpha$ ,  $\beta$ , and  $\gamma$  are three paths as in Problem 4.9.8 (b) but in such a directed graph, prove that the length of  $\gamma$  in this new sense is the sum of the lengths of  $\alpha$  and  $\beta$ .

P4.9.10 Repeat Exercise 4.9.9 for directed graphs with three vertices  $a$ ,  $b$ , and  $c$  but *without loops*. There are  $2^6 = 64$  possible graphs, but the number of isomorphism classes is much smaller.

## 4.10 Trees and Lisp Lists

### 4.10.1 Rooted Directed Trees

Tree structures of various kinds abound in computer science. They are the main topic of our Chapter 9 — here we will look at one recursive definition of a kind of tree, the **rooted directed tree**, as an example of recursive definition and inductive reasoning. We'll also see two key applications of rooted directed trees:

- By restricting the definition slightly we will get *rooted directed binary trees*, which form the fundamental data structure in the Lisp family of programming languages, and
- By adding labels to the nodes of the trees we will be able to model *arithmetic expressions* — we will study these along with three ways to represent them as strings.

You may recall that at the end of the previous section we mentioned “trees” as a kind of undirected graph — specifically, undirected graphs with no cycles. We'll see in Section 9.1 how these “trees” are related to rooted directed trees as defined here (Problem 4.10.4 gives a hint toward this).

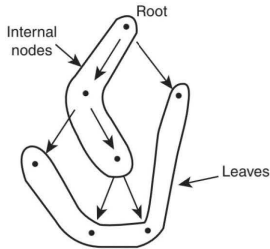
We begin, then, with a recursive definition of a rooted directed tree. It is a kind of directed graph as defined in Section 4.9, consisting of *nodes* and *arcs*. Every rooted directed tree has a **root**, which is one of its vertices.

- Any one-node directed graph (with no arcs) is a rooted directed tree. Its root is its only node.
- If  $S_1, S_2, \dots, S_k$  are  $k$  different trees with roots  $r_1, r_2, \dots, r_k$  respectively (node  $r_i$  is the root of tree  $S_i$ ), then the following directed graph  $T$  is a rooted directed tree:  $T$ 's nodes are the nodes of the  $S_i$ 's, plus one new node  $x$  which is  $T$ 's root, and  $T$ 's arcs are all the arcs of all the  $S_i$ 's, plus  $k$  new arcs — one from  $x$  to each of the nodes  $r_1, \dots, r_k$ .
- The only rooted directed trees are those that can be made by these two operations.

Figure 4-18 shows an example of a rooted directed tree and also illustrates some additional vocabulary. If  $x$  is any node of any directed tree, we say that the **in-degree** of  $x$  is the number of arcs coming into  $x$  and its **out-degree** is the number of arcs coming out of it. We divide the nodes of a rooted directed tree into **internal nodes**, which each have an out-degree of one or more, and **leaves**, which each have an out-degree of zero. In the tree in this figure, every internal node has an out-degree of exactly two — we call such a tree a **rooted directed binary tree**.

We use the language of genealogy to express relationships among the nodes of a given rooted directed tree. If there is an arc from node  $x$  to node  $y$ , we say that  $x$  is  $y$ 's **parent** and that  $y$  is  $x$ 's **child**. Two different children of the same parent are called **siblings**. If there is a path from node  $x$  to node  $y$ , we say that  $x$  is  $y$ 's **ancestor** and that  $y$  is  $x$ 's **descendent**. Modern practice





©Kendall Hunt Publishing Company

Figure 4-18: A rooted directed binary tree.

is to avoid gendered expressions as much as possible, but the terminology can be extended almost arbitrarily — for example, we could call a sibling of  $x$ 's parent either  $x$ 's “uncle” or its “aunt”.

It is easy to note properties that hold for all rooted directed trees, for example:

- The root is an ancestor of any node in the tree.
- If  $x$  is any node, there is exactly one path from the root to  $x$ .
- The root has in-degree zero, and all other nodes have in-degree one.

As usual, it is the last clause of the inductive definition that gives us a way to prove statements about all trees by induction. If  $P(T)$  is a statement that has one free variable  $T$  of type `RootedDirectedTree`, we can prove  $\forall T : P(T)$  by first proving  $P(N)$ , where  $N$  is an arbitrary one-node tree as in the first clause, and then proving  $[P(S_1) \wedge P(S_2) \wedge \dots \wedge P(S_k)] \rightarrow P(U)$ , where  $U$  is the tree made from the  $S_i$ 's using the second clause. Let's prove the three statements above, noting that the first statement follows immediately from the second.

**Lemma:** If  $T$  is any rooted directed tree with root node  $r$ , and  $x$  is any node of  $T$ , then there is exactly one path from  $r$  to  $x$ .

**Proof:** If  $T$  has only one node, then the node  $x$  must be the root and there is exactly one trivial path from the root to  $x$ . So assume that  $T$  is made from a new root  $r$  and  $k$  rooted directed trees  $S_1, \dots, S_k$  using the second clause, and that each tree  $S_i$  has a root  $r_i$ . Let  $x$  be any node of  $T$ . If  $x$  is  $T$ 's root, there is exactly one trivial path from  $x$  to itself. Otherwise assume that  $x$  is a node of tree  $S_i$ . By the inductive hypothesis, there is exactly one path from  $r_i$  to  $x$ . There is an arc from  $r$  to  $r_i$ , which combines with the path from  $r_i$  to  $x$  to produce exactly one path from  $r$  to  $x$ . No other path can go from  $r$  to  $x$ , because any path from  $r$  must either be trivial or take an edge to some  $r_j$  — if it goes to  $r_j$  with  $i \neq j$ , it can never reach  $x$  because it can never leave  $S_j$ . (None of  $S_j$ 's own arcs leaves  $S_j$ , and none of the new arcs do either.) ♠

**Lemma:** If  $T$  is any rooted directed tree, then  $T$ 's root has in-degree zero and each other node of  $T$  has in-degree one.

**Proof:** If  $T$  is a single-node tree with no arcs, then clearly the root has an in-degree of zero and there are no other nodes. So assume that  $T$  is made from a root  $r$  and  $k$  trees  $S_1, \dots, S_k$  according to the second clause, and that the inductive hypothesis applies to each of the  $S_i$ 's. Let  $x$  be any node in  $T$ . If  $x$  is the root, it has in-degree zero because neither the arcs of any of the  $S_i$ 's nor any of the new arcs go into  $x$ . If  $x$  is one of the  $r_i$ 's, a root of one of the  $S_i$ 's, then it has in-degree one because exactly one of the new arcs, but none of the arcs of any of the  $S_i$ 's, go into it. And if  $x$  is a non-root node of one of the  $S_i$ 's, it had in-degree one in  $S_i$  and keeps in-degree one in  $T$  because none of the new arcs go into it. We have shown that for arbitrary  $x$ ,  $x$  has in-degree zero if it is the root and in-degree one otherwise. ♠

By slightly tweaking this definition, we can get various related versions of rooted directed trees. For example, in Exercise 4.10.1 you'll give a definition for rooted directed trees whose internal nodes have out-degree of one *or* two — these will be the basis of our arithmetic expressions later in the section. There and otherwise, it's often important to have an order on the children of a particular node, as we normally would in any data structure application. In Section 6.10, we'll meet “Catalan trees”: rooted directed trees where internal nodes have out-degree one or two, but where an “only child” of a parent is distinguished as being either the “left child” or “right child”.

## 4.10.2 Lisp Lists

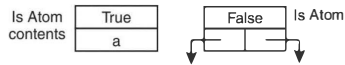
Our particular definition of rooted directed *binary* happens to correspond to the definition of a **list structure** in the Lisp family of programming languages:

- An **atom** is a list structure.
- If  $x$  and  $y$  are list structures, so is `cons(x y)`.
- The only list structures are those defined by the first two clauses.

There are two “inverse operations” to `cons`, called `car` and `cdr`. If  $z$  is constructed as `cons(x y)`, then `car(z)` is defined to be  $x$  and `cdr(z)` is defined to be  $y$ . The `car` or `cdr` of an atom is not defined.

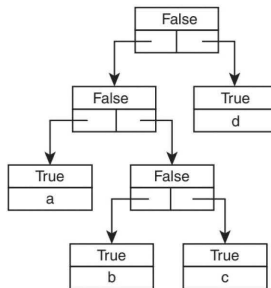
Since this book doesn't assume familiarity with any language except Java, in order to look at algorithms on list structures we'll have to imagine a class `LispLS` defined as follows using pointers (see Figures 4-19 and 4-20):

```
public class LispLS {
    boolean isAtom;           // true if this list structure is a single atom
    thing contents;           // value of the atom if isAtom is true
    LispLS left, right;       // substructures from car and cdr respectively
}
```



©Kendall Hunt Publishing Company

Figure 4-19: The two types of nodes in the LispLS data type.



©Kendall Hunt Publishing Company

Figure 4-20: A LispLS data structure.

In Problem 4.10.1 you'll write pseudo-Java methods for the basic Lisp functions on this class. Except for those basic procedures, just about anything you'd want to do to a list will involve recursion. Here's a simple example of a method to output the atoms of a list in order, assuming the basic functions are available:

```
void printAtoms()
{
    // Write list of atoms in calling list to System.out
    if (isAtom)
        System.out.println(contents);
    else {
        left.printAtoms();
        right.printAtoms();
    }
}
```

### 4.10.3 Arithmetic Expressions

Our second example of a tree-shaped structure is the **arithmetic expression**. Since most of the standard arithmetic operators take exactly two arguments, we can represent an expression by a labeled directed graph, where each operator is the label of a node and there are arcs from that node to the nodes representing the two arguments. Because there are also unary operators (such as the minus sign), however, we can't just use rooted directed binary trees – we have to allow internal nodes with one child as well. We'll call the resulting notion **expression trees**.

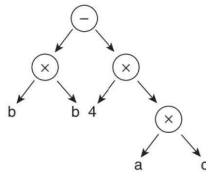
Here is a recursive definition of an arithmetic expression:

- A constant is an arithmetic expression. We can view a constant as a labeled node.
- A unary operator, acting on one arithmetic expression, is an arithmetic expression. We can view this as a root node, labeled by the unary operator, with a single arc out of it to the root of the expression being acted on.
- A binary operator, acting on two arithmetic expressions, forms an arithmetic expression. We view this as a root node, labeled by the binary operator, with two arcs out of it, one to each of the other roots.
- Nothing else is an arithmetic expression.

Figure 4-21 shows the expression tree corresponding to the arithmetic expression " $b^2 - 4ac$ ".

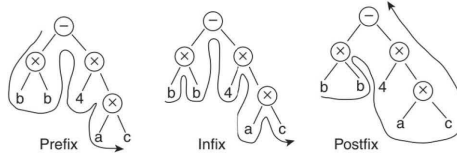
The **value** of an arithmetic expression is also defined recursively. The value of a single-node expression is the constant value of its node. The value of any other expression, with an operator at its root, is the result of applying the operator to the values of the subexpressions for the root's children.

When we record an arithmetic expression as a string, we have three choices of where to put the operators:



©Kendall Hunt Publishing Company

Figure 4-21: The expression tree for  $b^2 - 4ac$ .



©Kendall Hunt Publishing Company

Figure 4-22: The three traversal routes in an expression tree.

- Before the arguments (**prefix** or “**Polish**” notation<sup>39</sup>), as in “ $-*bb*4*ac$ ”. Lisp uses this notation, in the form “ $(- (* b b) (* 4 (* a c)))$ ”.
- Between the arguments (the usual or **infix** notation), as in “ $(b*b)-(4*a*c)$ ”. Note that we need to supply parentheses to indicate the actual tree structure of the operations, where in the other two cases this can be determined from the string alone. This is the syntax used by Texas Instruments calculators.
- After the arguments (**postfix** or “**reverse Polish**”), as in “ $bb*4ac*-$ ”. This is the syntax used by Hewlett-Packard calculators, with the “enter” key used to separate two adjacent arguments.

These three notations correspond to three ways to **traverse** an expression tree — to visit the nodes in a prescribed order and perform some operation (such as printing the node’s label) at each. (Figure 4-22 illustrates the paths of the three traversals in the tree.)

<sup>39</sup>The name of this notation is a tribute to its inventor, the Polish logician Jan Lukasiewicz (1878-1956).

We can easily come up with generic recursive procedures to carry out each of these traversals<sup>40</sup>. When given an input expression, each one must process the root node and, if there are any children of the root node, process the subtrees for each child. The only difference between the three methods is in the order in which they do this:

```
void preOrderTraversal
{
  // Apply doRoot to each node in preorder
  doRoot();
  if (!isAtom) {
    car().preOrderTraversal();
    cdr().preOrderTraversal();
  }
}

void inOrderTraversal()
{
  // Apply doRoot to each node in inorder
  if (isAtom)
    doRoot();
  else {
    car().inOrderTraversal();
    doRoot();
    cdr().inOrderTraversal();
  }
}

void postOrderTraversal
{
  // Apply doRoot to each node in postorder
  if (!isAtom) {
    car().postOrderTraversal();
    cdr().postOrderTraversal();
  }
  doRoot();
}
```

So, for example, a procedure to convert infix to reverse Polish notation, or vice versa, might proceed by reading the infix string into a tree structure and then outputting it by a postorder traversal. The code is simple, and statements about what it does are easy to prove by induction.

#### 4.10.4 Exercises

- E4.10.1 Give an inductive definition for the set of rooted directed trees that have *no more than* two children for every internal node.
- E4.10.2 (uses Java) Write a method `numAtoms()` for the `LispList` class so that if  $x$  is any `LispList`, `x.numAtoms()` returns the total number of atoms in  $x$ .
- E4.10.3 Convert the following arithmetic expressions to the specified notation (all constants are denoted by single letters or digits):
- (a) From postfix to infix: `4p*p*r*xx*yy*+-`.

<sup>40</sup>We can use the class definition from above, except that every node now has `contents` and we'll have a generic method `doRoot()` that will input a `LispList` and carry out the appropriate operation on its root node.

- (b) From postfix to prefix: `ss*cc**+`.
- (c) From infix to postfix: `(a+b)*((a*a)-(a*b)+(b*b))`.
- (d) From infix to prefix: `(a*a*a)+(3*a*a*b)+(3*a*b*b)+(b*b*b)`.
- (e) From prefix to postfix: `**ab**ab+ab`.
- (f) From prefix to infix: `--1x-*xx+*x*xx*x*x*xx`.

E4.10.4 Draw a tree to represent each of the arithmetic expressions in Exercise 4.10.3.

E4.10.5 Explain why the number of arcs in a directed graph is exactly equal to the sum of the in-degrees of the nodes. How many arcs are there in a directed tree with  $n$  nodes?

E4.10.6 The **depth** of a rooted directed tree is the greatest length of any path within it.

- (a) Prove that every path in any rooted directed tree is finite (that is, has a length that is a natural). (**Hint:** Such a path either does or doesn't involve the root node, giving you two cases.)
- (b) Give a recursive definition of the depth of a rooted binary tree.

E4.10.7 (uses Java) Write a pseudo-Java instance method `boolean contains (thing target)` for the `LispLS` class that returns `true` if and only if the calling `LispLS` object contains an atom whose value is equal to `target`. Use a method `boolean equals (thing x, thing y)` to test equality of value.

E4.10.8 Assuming that each variable has a value of 2, find the value of each of the six arithmetic expressions in Exercise 4.10.3.

E4.10.9 For each possible depth  $d$  from 0 through 5, find the arithmetic expression with the largest possible value that has depth  $d$  and all constant values equal to 1.

E4.10.10 In each of the following four cases, determine whether the value of such an arithmetic expression must be even, must be odd, or could be either. Prove your answer in each case, either with an induction or with a pair of examples.

- (a) Constants are odd naturals, all operators are `+`
- (b) Constants are even naturals, all operators are `+`
- (c) Constants are odd naturals, all operators are `*`
- (d) Constants are even naturals, all operators are `*`

## 4.10.5 Problems

P4.10.1 (uses Java) Write pseudo-Java code for the three functions `cons`, `car`, and `cdr` defined above, to be included in the `LispLS` class. The method `cons` should be static, taking two `LispLS` arguments. The other two methods should be instance methods taking no arguments. If called from an atom, `car` and `cdr` should throw a `NullPointerException`.

P4.10.2 (uses Java) Lisp commonly uses a list structure to encode a **list**, which is a sequence of items. There is a special value called `nil`, which represents an empty list, and by convention the `car` and the `cdr` of `nil` are both `nil`. A list with a single element `a` is represented by the

list structure `cons (a nil)`, and in general a list with first element `a` and remainder `x` is represented by `cons (a x)`. Write pseudo-Java methods, using the `LispLS` class declaration above and the `car`, `cdr`, and `cons` methods from Problem 4.10.1, to carry out the following operations on strings (which are here thought of as lists of letters):

- (a) Return the last letter (return `nil` if the input is `nil`).
- (b) Return the list representing the string `allButLast(x)`.
- (c) Concatenate two strings.
- (d) Reverse a string.

P4.10.3 Prove the following facts by (structural) induction for all arithmetic expressions, using the definition in this section:

- (a) The first character of the infix representation of an expression is never a binary operator.
- (b) The first character of the prefix representation of an expression is an operator, unless the expression consists of a single constant.
- (c) The first character of the postfix representation of an expression is a constant.

P4.10.4 Let  $T$  be any undirected tree (any connected undirected node with no cycles, as defined in Section 4.9), and let  $v$  be any node in  $T$ . Define  $N$  to be the set of neighbors of  $v$  (the set  $\{u : E(u, v)\}$ ). Let  $G$  be the undirected graph obtained from  $T$  by deleting all the edges involving  $v$ . Prove that if  $w$  is any node in  $G$  other than  $v$ ,  $w$  has a path to exactly one vertex in  $N$ . (**Hint:** First show that one such path exists, then show that the existence of paths to two or more nodes in  $N$  would contradict the assumptions on  $T$ .)

P4.10.5 Prove that if  $T$  is any rooted directed binary tree (where every internal node has out-degree exactly two), then the number of leaves in  $T$  is one greater than the number of internal nodes. (**Hint:** Use induction on the definition of such trees.)

P4.10.6 A **full binary tree** is a rooted binary tree where every internal node has exactly two children and every path from the root to a leaf has the same length.

- (a) Give a recursive definition of full binary trees.
- (b) Determine both the number of leaves and the total number of nodes in a full binary tree of depth  $n$ . Prove your answers using your inductive definition of full binary trees.

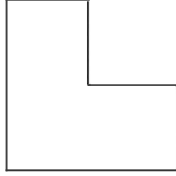
P4.10.7 (uses Java) Suppose we are given a `LispLS` object that represents a list of numbers as in Problem 4.10.2. Write a pseudo-Java static method that will take such a list as input and return a number that is the sum of all the numbers in the list. (If given `nil`, it should return 0.)

P4.10.8 Let  $G$  be a directed graph. A **spanning tree** of  $G$  is a rooted directed tree whose nodes are exactly the nodes of  $G$  and all of whose arcs are also arcs of  $G$ . Prove that if  $G$  is any strongly connected directed graph, and  $x$  is any node of  $G$ , then there exists a spanning tree of  $G$  whose root is  $x$ . (**Hint:** Prove this for all strongly connected directed graphs  $G$  by induction on the number of nodes in  $G$ .)

P4.10.9 Prove that in any arithmetic expression, where the constants are represented by single letters, the prefix and postfix representations of the expression are anagrams of one another. (That is, they are strings of the same length that have the same number of each possible character.) (**Hint:** Use induction on arithmetic expressions.)



P4.10.10 Consider an arithmetic expression  $E$ , as on Problem 4.10.9, where the constants are represented by single letters. Let  $Pre$  and  $Post$  be the prefix and postfix strings, respectively, for  $E$ . Show that  $Pre^R$  is the valid postfix representation of some arithmetic expression  $F$ , and that  $Post^R$  is the valid prefix representation of that same expression  $F$ .



©Kendall Hunt Publishing Company

Figure 4-23: An L-shaped tile.

## 4.11 Induction For Problem Solving

### 4.11.1 L-Shaped Tiles

We conclude this chapter by looking at some additional mathematical uses of induction. Mathematical induction is often presented as a technique for proving integer identities and nothing else. We've tried to show in the past few sections how it applies to fundamental facts about other recursively defined structures. Here we'll see how you can prove nontrivial things in a variety of settings.

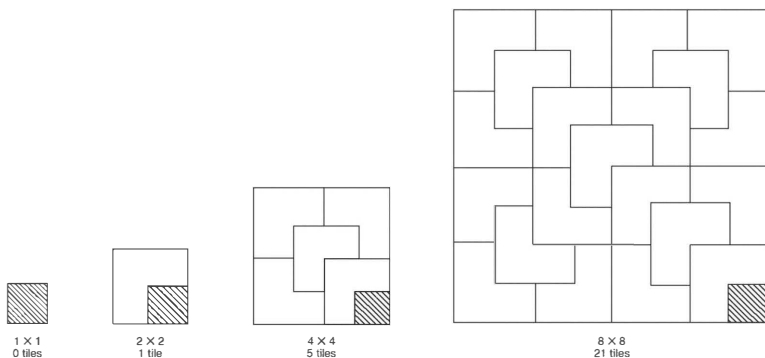
Consider the problem (due originally to Golomb) of tiling an  $8 \times 8$  chessboard with 3-square L-shaped pieces (see Figure 4-23).

Covering the board completely is *impossible*, because 3 doesn't divide 64, but suppose we leave off one of the corner squares. Here is a proof that you can do it, that works by proving a stronger result:

**Theorem:** Given any number  $n$ , it is possible to place L-shaped tiles to cover a  $2^n \times 2^n$  chessboard with any one square missing.

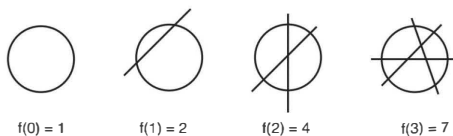
**Proof:** The base case of  $n = 0$  is easy because  $2^0 = 1$  and we can tile a  $1 \times 1$  board, with one square missing, using no tiles. For the inductive case, assume that we can do it for any  $2^n \times 2^n$  board with any one square missing, and consider a  $2^{n+1} \times 2^{n+1}$  board, also with any one square missing. Divide the board into four  $2^n \times 2^n$  boards in the obvious way. One of these four subboards has a missing square. Place a single L-shaped piece in the middle of the big board, so as to cover one square of each of the other three subboards. Now each of the four subboards is missing a square. But the remainder of each board can be tiled with L-shaped pieces, according to the inductive hypothesis. ♠

Notice how this inductive proof of the statement  $P(n)$  also provides a recursive algorithm for actually constructing such a tiling, and for that matter recursively defines a particular tiling (see Figure 4-24).



©Kendall Hunt Publishing Company

Figure 4-24: Tilings of  $2^n \times 2^n$  chessboards with one square missing.



©Kendall Hunt Publishing Company

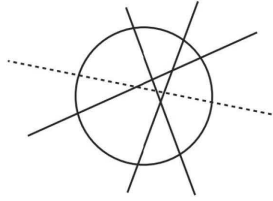
Figure 4-25: Cutting pizzas.

#### 4.11.2 Cutting Pizzas

For the next problem (originally solved by Steiner in 1826), consider dividing a round pizza into as many pieces as possible by making  $n$  cuts with a straight pizza cutter. Let  $f(n)$  be the maximum possible number of pieces. Obviously  $f(0) = 1$ ,  $f(1) = 2$ , and  $f(2) = 4$ , so you might think that  $f(n) = 2^n$ , but a bit of playing around should informally convince you that  $f(3) = 7$  (see Figure 4-25). What about  $f(4)$ ?

If we think of this as an induction problem, it's natural to take a configuration with  $n$  lines and think of putting in the  $(n + 1)$ 'st line<sup>41</sup>. This line will increase the number of pieces by dividing

<sup>41</sup>Note, by the way, that we should *not* necessarily assume that the first  $n$  lines give an optimal number of pieces, because it's not clear that the best way to maximize the pieces for  $n + 1$  is to first maximize them for  $n$ . An algorithm that always makes the choice that gives the best immediate result is called a **greedy** algorithm. Sometimes the best



©Kendall Hunt Publishing Company

Figure 4-26: A new line through the  $n = 3$  pizza.

certain old pieces into two new ones each<sup>42</sup>. So how many old pieces can the new line hit? The new line moves from one old piece to another exactly when it crosses an old line. Since it can only cross each of the  $n$  old lines once, the best case is when it crosses all  $n$  old lines and thus visits  $n$  old pieces (see Figure 4-26).

This tells us that  $f(n+1)$  is at most  $f(n) + n + 1$ , and in fact it gives us an algorithm for achieving that bound (take an optimal  $n$ -line configuration and draw a new line crossing all the old lines) so we know  $f(n+1) = f(n) + n + 1$ . The sequence continues  $f(4) = 11$ ,  $f(5) = 16$ , and in general  $f(n) = (n^2 + n + 2)/2$  (Exercise 4.11.2 is to prove this.)

A tougher problem (given as Problem 4.11.2 below, an optional Excursion) is to generalize this example to three dimensions, so that we are cutting a block of cheese rather than a pizza. The sequence here starts out “1, 2, 4, 8, 15, . . .” and turns out to be closely related to the two-dimensional version.

### 4.11.3 The Speed of the Euclidean Algorithm

Here is a final problem from number theory. You may recall that we asserted that the Euclidean Algorithm of Section 3.3 runs in time proportional to the number of digits in the input numbers<sup>43</sup>. Let’s prove a version of this statement that doesn’t involve logarithms:

**Theorem:** If both input numbers are at most  $2^n$ , the Euclidean Algorithm terminates in at most  $2n + 1$  divisions.

**Proof:** For the base case, let  $n = 0$  and note that the first division, if both numbers are one or zero, will give remainder zero. For the induction, suppose we start with  $a$  and  $b$  and calculate  $c = a \% b$  and  $d = b \% c$ . I claim that if  $a$  and  $b$  are each at most  $2^{n+1}$ , then  $c$  and  $d$  are each at most  $2^n$ . This claim suffices because on  $a$  and  $b$  the algorithm will do two divisions, plus at most

algorithm is a greedy one and sometimes it isn’t.

<sup>42</sup>Every piece of pizza we create is **convex**, meaning that the line between any two points on the piece stays on the piece. Can you prove *this* fact by induction?

<sup>43</sup>This is called a **logarithmic** or  $O(\log n)$  running time, as we will see in Chapter 7 and will be discussed more thoroughly in an algorithms course.

$2n + 1$  more once we have  $c$  and  $d$ , for a total of at most  $2n + 3 = 2(n + 1) + 1$ .

To prove the claim we will use the contrapositive method. Assume that  $c > 2^n$ . We know that  $b > 2^n$  because  $c$ , which is  $a \% b$ , must be less than  $b$ . But then since  $a > b$ ,  $a/b$  is at least 1, and  $a$  is at least  $b + c$  and thus greater than  $2^{n+1}$ . ♠

The worst case for the Euclidean algorithm actually occurs when  $a$  and  $b$  are consecutive *Fibonacci numbers*, for example 21 and 13. (Try this example, if you haven't already!). From the behavior of Fibonacci numbers, one can show that the number of divisions is at most  $\log_{1.41\dots} a$ , an improvement over the  $\log_{1.41\dots} a$  shown here.

#### 4.11.4 Exercises

- E4.11.1 Show that a  $2 \times n$  rectangle can be covered exactly with L-shaped tiles if and only if 3 divides  $n$ .
- E4.11.2 Complete the argument in the section by using induction to prove that  $f(n)$ , the maximum number of pieces that can be made from a convex pizza with  $n$  cuts, is exactly  $(n^2 + n + 2)/2$ .
- E4.11.3 The upper bound of Exercise 4.11.2 was for convex pizzas. Give an example showing that this bound can be exceeded if the original pizza is not convex. Can you prove *any* upper bound in the non-convex case?
- E4.11.4 A set of  $n$  lines on the plane is said to be in **general position** if no two lines are parallel and no three lines intersect in a single point. Prove that  $n$  lines in general position divide the plane into exactly  $f(n)$  regions, where  $f(n) = (n^2 + n + 2)/2$  is the solution to the pizza problem.
- E4.11.5 Prove by induction on the Fibonacci numbers that for any natural  $n$  except  $n = 1$ ,  $F(n + 2) \% F(n + 1) = F(n)$ . Determine exactly how many divisions the Euclidean algorithm takes if the original numbers are  $F(n + 1)$  and  $F(n)$ , and prove your answer by induction.
- E4.11.6 In how many different ways can we tile a  $2 \times n$  rectangle with  $1 \times 2$  rectangles?
- E4.11.7 Consider a  $2 \times n$  **grid graph**, an undirected graph where the nodes are arranged in a  $2 \times n$  rectangular array and there is an edge between any pair of node that are a unit distance apart. A **perfect matching** in an undirected graph is a subset of the edges such that each node in the graph is an endpoint of exactly one of the edges. Prove that the number of perfect matchings in a  $2 \times n$  grid graph is exactly equal to the answer to Exercise 4.11.6.
- E4.11.8 A **T tetromino** is a set of four squares consisting of a single square with exactly three of its four neighbors.
- (a) Prove that if  $n$  is divisible by 4, then a  $4 \times n$  rectangle can be tiled with T tetrominos.
  - (b) Prove that if  $n$  is odd, then a  $4 \times n$  rectangle *cannot* be tiled with three tetrominos. (**Hint:** Think of the squares of the rectangle being colored black and white as in a checkerboard.)
- E4.11.9 Prove that if  $i$  and  $k$  are any naturals, the Fibonacci numbers  $F(i)$  and  $F(6k+i)$  are congruent modulo 4.

E4.11.10 For what pairs of naturals  $i$  and  $j$  does the natural  $2^i + 1$  divide  $2^j + 1$ ? Prove your answer.

#### 4.11.5 Problems

P4.11.1 Show that a  $3 \times n$  rectangle can be covered exactly with L-shaped tiles if and only if  $n$  is even. (**Hint:** For the negative result, use induction on all odd numbers and an indirect proof in the inductive step.)

P4.11.2 (suitable for an Excursion) The “cheese problem” is a generalization of the “pizza problem”. Instead of a two-dimensional pizza, we have a three-dimensional convex block of cheese that is to be cut into the maximum possible number of pieces by  $n$  straight planar cuts. Find the maximum possible number of pieces,  $g(n)$ . (**Hint:** Clearly  $g(0) = 1$ ,  $g(1) = 2$ ,  $g(2) = 4$ , and  $g(3) = 8$ . But in making a fourth cut, we can’t cut all eight pieces, but only seven. Why? Because the first three cuts can only divide the *plane* of the fourth cut into seven pieces by our solution to the pizza problem. Generalizing this observation, you’ll get a recursive definition of  $g(n)$  in terms of the answer to the pizza problem,  $f(n)$ . Then it’s a matter of finding the solution to this equation, which we haven’t studied how to do systematically but which you might be able to manage. The answer is of the form  $an^3 + bn^2 + cn + d$ , but you’ll have to find the correct real numbers  $a, b, c$ , and  $d$  and show that they’re correct.)

P4.11.3 Prove the claim at the end of the section about the Euclidean Algorithm and Fibonacci numbers. Specifically, prove that if positive naturals  $a$  and  $b$  are each at most  $F(n)$ , then the Euclidean Algorithm performs at most  $n - 2$  divisions. (You may assume that  $n > 2$ .)

P4.11.4 Suppose we want to lay out a full undirected binary tree on an integrated circuit chip, with the nodes at the intersections of a rectangular grid and the edges along lines of the grid. The **H-tree** is a recursive method of doing this. Define the H-tree  $H_i$  by induction as follows:

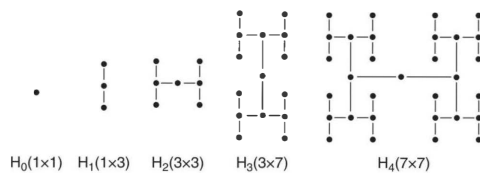
- The tree  $H_0$  has a single node and no edges.
- For any number  $k$ ,  $H_{2k+1}$  is made by taking a new root node and connecting it to the roots of two copies of  $H_{2k}$ , each with roots a distance  $2^k$  away from the new root, one copy directly above and the other directly below.
- For any positive number  $k$ ,  $H_{2k}$  is made by taking a new root node and connecting it to the roots of two copies of  $H_{2k-1}$ , each with roots a distance  $2^{k-1}$  away from the new root, one copy directly to the left and the other directly to the right.

Figure 4-27 shows the first few H-trees, through  $H_4$ .

- Draw  $H_5$  and  $H_6$ .
- How many nodes are in  $H_i$ ? How large a grid is needed to hold the layout? (For example,  $H_4$  fits on a  $7 \times 7$  grid.) As  $n$  increases, approximately what percentage of the nodes on the grid become nodes of the H-tree  $H_n$ ?
- How much total wire is used in  $H_n$ ? How far are the leaves from the root node?

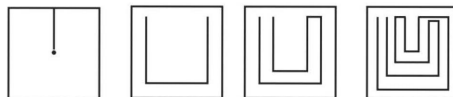
P4.11.5 Consider the following recursively defined sequence of paths in the unit square (Figure 4-28).

Path  $P_0$  goes from the middle of the top edge to the center of the square. Each succeeding path will be laid exactly through the center of the regions not touched by the previous path.



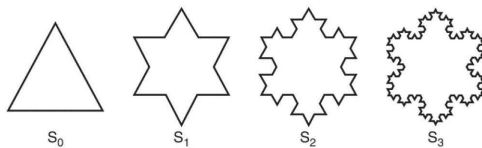
©Kendall Hunt Publishing Company

Figure 4-27: Some H-trees.



©Kendall Hunt Publishing Company

Figure 4-28: A recursively defined sequence of paths.



©Kendall Hunt Publishing Company

Figure 4-29: The first four Koch Snowflake polygons.

For example, path  $P_1$  will start in the center of the top left quarter of the square, move down to the middle of the lower left quarter, move right to the middle of the lower right quarter, and finally move up to stop at the center of the upper right quarter.  $P_2$  starts near the upper left corner, a distance  $1/8$  from each edge, and travels along seven line segments as shown until it stops a distance  $1/4$  to the right of where it started.

- How long is the path  $P_i$  for general  $i$ ?
- What is the maximum distance from any point in the square to its nearest neighbor on  $P_k$ ? Prove by induction that  $P_k$  passes through the center of every subsquare in a division of the square into  $2^{-k}$  by  $2^{-k}$  subsquares.

P4.11.6 The **Koch snowflake** is obtained by recursively defining the following family of polygons:

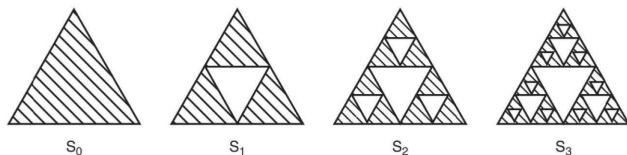
- $S_0$  is an equilateral triangle.
- $S_{n+1}$  is defined from  $S_n$  by dividing each side of the polygon into three equal parts and replacing the middle one by the other two sides of an equilateral triangle, pointing away from the center of the figure.

Figure 4-29 shows the polygons  $S_0$ ,  $S_1$ ,  $S_2$ , and  $S_3$ .

- Let  $E_n$  be the number of sides of the polygon  $S_n$ . Derive a recursive definition for  $E_n$  and a formula for  $E_n$  in terms of  $n$ . Prove your formula correct by induction.
- Let  $Q_n$  be the number of  $60^\circ$  angles in  $S_n$ , and let  $R_n$  be the number of  $240^\circ$  angles. Prove by induction that  $Q_n = 4^n + 2$  and  $R_n = 2(4^n) - 2$ . You may use without proof the fact that the total number of angles in a polygon equals its number of sides.
- Let  $A_n$  be the area of  $S_n$ . Prove by induction that  $A_n = A_0(1 + (3/5)(1 - (4/9)^n))$ .
- Let  $P_n$  be the path length (or perimeter) of the figure  $S_n$ . Prove the following statement:  $\forall m : \exists n : P_n > m$ , where the variables  $m$  and  $n$  range over the naturals. Note that the only thing you know about  $P_0$  is that it is a positive real number.

P4.11.7 In four-dimensional Euclidean space, a **hyperplane** is the three-dimensional space that is the solution set of a linear equation, such as  $a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4 = b$ . Any hyperplane divides four-space into two pieces. A set of  $k$  hyperplanes is said to be in general position (as in Exercise 4.11.4) if every pair of them intersect in a two-dimensional space, every set of three intersect in a line, and every set of four intersect in a point. Find and prove a formula for the number  $R_k$  of regions into which 4-space is divided by a set of  $k$  hyperplanes in general position. (**Hint:** Hard as this may be to visualize, the regions can be counted by the same reasoning used for the pizza and cheese numbers in this section.)





©Kendall Hunt Publishing Company

Figure 4-30: The first four approximations to the Sierpinski gadget.

P4.11.8 The **Sierpinski gadget** is defined by a sequence of two-dimensional figures as follows:

- $S_0$  is an equilateral triangle.
- Each subsequent  $S_i$  is a union of  $3^i$  equilateral triangles.
- We make  $S_{i+1}$  from  $S_i$  by taking each triangle in  $S_i$ , connecting the midpoints of its three sides to make a smaller triangle, and deleting this smaller triangle from the figure.

Figure 4-30 shows the first four figures  $S_0$ ,  $S_1$ ,  $S_2$ , and  $S_3$ .

- Prove by induction that there are exactly  $3^i$  triangles in  $S_i$ .
- Give a formula for the total area of  $S_i$  and prove this formula correct by induction.
- The Sierpinski gadget  $S$  itself is the set of points that are contained in the figure  $S_i$  for *every* natural  $i$ . Prove that the area of  $S$  is 0. Can you prove that  $S$  is non-empty? Can you prove that  $S$  contains an infinite number of points<sup>44</sup>?

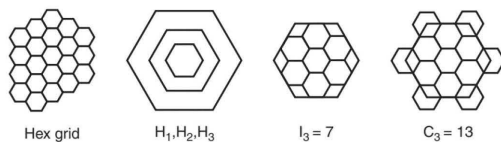
P4.11.9 Following Exercise 4.11.7, we can consider the number  $f(n)$  of perfect matchings in a  $3 \times 2n$  grid graph, which is the same as the number of ways to tile a  $3 \times 2n$  rectangle with  $1 \times 2$  dominoes.

- Prove that  $f(0) = 1$ ,  $f(1) = 3$ , and that for positive  $n$ ,  $f(n) = 3f(n-1) + 2f(n-2) + \dots + 2f(0)$ .
- Prove (probably using the formula in (a)) that for  $n > 1$ ,  $f(n) = 4f(n-1) - f(n-2)$ .
- Prove by induction (probably using the formula in (b)) that for all naturals  $n$ ,  $f(n) = ((1 + 1/\sqrt{3})(2 + \sqrt{3})^n + (1 - 1/\sqrt{3})(2 - \sqrt{3})^n)/2$ .
- Using any of these formulas, find  $f(n)$  for all  $n$  with  $n \leq 5$ .

P4.11.10 A **hex grid** is a natural tiling of two-dimensional Euclidean space by regular hexagons, all the same size. It is familiar to users of various board games, and often used to tile bathroom floors. Define the figure  $H_n$ , for any natural  $n$ , to be a regular hexagon with side  $n$ , placed on a hex grid of hexagons of side 1. We'll put the center of  $H_n$  in the center of one of the unit hexagons in the grid. The area of  $H_n$  is exactly  $n^2$  times the area of a unit hexagon.

Define the number  $I_n$ , for any natural  $n$ , to be the number of unit hexagons *entirely* contained within  $H_n$  when it is placed this way, and define  $C_n$  to be the number of unit hexagons that are *entirely or partially* contained within  $H_n$ . Of course  $I_1 = C_1 = 1$  because  $H_1$  is exactly

<sup>44</sup>In fact, in Chapter 7 we will show that it has an "uncountably infinite" number of points.



©Kendall Hunt Publishing Company

Figure 4-31: Inscribed and circumscribed hexagons.

a unit hexagon. Since  $H_2$  includes one entire hexagon and half each of six others, we have  $I_2 = 1$  and  $C_2 = 7$ .  $H_3$  contains seven entire hexagons and  $1/3$  each of six others, so  $I_3 = 7$  and  $C_3 = 13$ . In general we can see that  $I_n < n^2 < C_n$ .

Find formulas for the functions  $I_n$  and  $C_n$  in terms of  $n$ , and prove these formulas correct by induction. (**Hint:** Your formulas should have separate clauses depending on the class of  $n$  modulo 3, and your proof may use an inductive step of the form  $P(n) \rightarrow P(n+3)$ .)

## Index

1-2-3 Nim 4-23

abstract data type 4-2

additive identity 4-30

additive inverse 4-30

`allButLast` operation 4-37

ancestor node 4-56

`append` operation 4-37

applying the inductive hypothesis 4-13

arc 4-47

arithmetic expression 4-60

associative operation 4-30

atom (in Lisp) 4-58

axiom 4-2

axioms for a semiring 4-30

axioms for strings 4-37

balanced parenthesis language 4-43

base case of a recursive algorithm 4-11

base case of an induction 4-14

bottom-up method 4-31

`car` operation 4-58

category theory 4-48

`cdr` operation 4-58

classifying graphs 4-50

cheese problem 4-70

children of a node 4-56

commutative operation 4-30

commutative semiring 4-30

concatenation of strings 4-38

connected (undirected) graph 4-52

`cons` operation 4-58

convex polygon 4-25, 4-68

cycle in a graph 4-52

degree of a node 4-54

degree sequence 4-54

depth of a tree 4-63

`dequeue` operation 4-9

descendent node 4-56

directed cycle 4-57

directed edge 4-47

directed graph 4-47

distributive law for semirings 4-30

double letter 4-41

edge 4-47

edge predicate 4-47

Egyptian pyramid 4-18

Elvis proof 4-24

endpoints of an edge 4-47

`enqueue` operation 4-9

equal graphs 4-48

expression tree 4-60

Fibonacci numbers 4-27

finding a transitive closure 4-52

forest 4-52

full binary tree 4-64

general position of lines 4-69

Golden Ratio 4-28

graph theory 4-47

grammars for languages 4-43

greedy algorithm 4-67

grid graph 4-69

grounded recursion 4-11

H-tree 4-70

half-life 4-17

hex grid 4-73

hyperplane 4-72

identical graphs 4-48

in-degree of a node 4-56

induction on the odds or evens 4-21

Induction Rule for Strings 4-39

induction starting from a positive number 4-20

inductive goal 4-14

inductive hypothesis 4-14

inductive step of an induction 4-14

infix notation 4-61

internal node 4-56

isomorphic graphs 4-48

isomorphism of graphs 4-48

Koch snowflake 4-72

L-shaped tiles 4-66

labeled graph 4-48

`last` operation 4-37

leaf in a tree 4-56

length of a path 4-55

- length of a string 4-38
- list (in Lisp) 4-63
- list structure (in Lisp) 4-58
- logarithmic function 4-68
- loop 4-47
  
- mathematical induction 4-13
- min-plus semiring 4-34
- multigraph 4-47
- multiplicative identity 4-30
  
- natural subtraction operator 4-8, 4-33
- node 4-47
- non-standard model of arithmetic 4-6
  
- one's complement of a string 4-41
- operations of a data type 4-2
- ordinary induction 4-13
- out-degree of a node 4-56
  
- parent node 4-56
- path in a graph 4-50
- path predicate 4-51
- path relation 4-51
- Peano Axioms 4-3
- perfect matching 4-69
- pizza problem 4-67
- Polish notation 4-61
- pop operation 4-9
- postfix notation 4-61
- predecessor of a natural 4-3
- prefix notation 4-61
- preserving an operation 4-50
- Principle of Mathematical Induction 4-3
- push operation 4-8
  
- queue 4-9
  
- recursive algorithm 4-4
- reversal of a string 4-38
- reverse Polish notation 4-61
- ring 4-30
- root of a tree 4-56
- rooted directed binary tree 4-56
- rooted directed tree 4-56
  
- semiring 4-30
- semiring axioms 4-30
- sibling node 4-56
- side effect 4-9
  
- Sierpinski gadget 4-73
- size of a natural 4-22
- spanning tree 4-64
- stack 4-8
- string axioms 4-37
- string induction 4-37
- strong induction 4-16, 4-22
- strongly connected component 4-54
- strongly connected directed graph 4-52
- structural induction 4-39
- substring operator 4-42
- subtraction for naturals 4-8, 4-33
- successor of a natural 4-2
- suffix of a string 4-41
  
- T tetromino 4-69
- tail recursion 4-10
- tetrahedron 4-17
- top-down method 4-31
- transitive closure 4-52
- Transitivity Theorem 4-51
- traversing a rooted directed tree 4-61
- tree (as an undirected graph) 4-53
- triangulating a polygon 4-25
- two-coloring a map 4-18
  
- undefined term 4-2
- undirected cycle 4-52
- undirected graph 4-47
  
- value of an arithmetic expression 4-60
- vertex 4-47
  
- Well-Ordering Principle 4-4
  
- zero 4-2

# Solutions to Exercises in Chapters 1-4

## S.1 Exercises From Chapter 1

- Exercise 1.1.1 (a) true, it's listed  
(b) false, 7 is not even  
(c) true, every  $C$  element is in  $A$   
(d) false, 0 is not in  $D$   
(e) false, 5 is not even so 5 is not in  $E$   
(f) false,  $D$  has three elements  
(g) true,  $C$  has one element  
(h) false, 0 and 8 are common to  $D$  and  $E$   
(i) false, there are infinitely many even naturals
- Exercise 1.1.2 The elements 1 and 3 are each in  $B$  and in none of the others. The element 6 and all even naturals greater than 8 are each in  $E$  and in none of the others.
- Exercise 1.1.3 (a) **thing**  
(b) **natural**  
(c) **boolean**  
(d) **integer** (might be positive or negative)  
(e) **real**
- Exercise 1.1.4 (a) Every element of the empty set is in  $A$ , whatever  $A$  is, because there are no such elements.  
(b) Every element of  $A$  is in  $A$ .  
(c) If every element of  $A$  is in  $B$  and every element of  $B$  is in  $A$ , the two sets are the same because no element is in one but not the other.  
(d) If every element of  $A$  is in  $B$ , and every element of  $B$  is in  $C$ , then any element of  $A$  is in  $C$  because we are told that it is in  $B$  and every element of  $B$  is in  $C$ .  
(e) If each set is a subset of the other, they are equal. So if they are not equal, one of the two subset statements must be false and if it isn't  $A \subseteq B$  then it must be the other.
- Exercise 1.1.5 (a) infinite  
(b) finite, each is specified by a machine word  
(c) infinite  
(d) finite  
(e) finite, assuming that there was some time before which there were no humans (both Genesis and science say that there was)
- Exercise 1.1.6 (a) The naturals that are 4 or greater

- (b) The set of all naturals
- (c) The empty set
- (d) The set containing 4 and nothing else

- Exercise 1.1.7
- (a)  $\{n : n = n\}$
  - (b)  $\{n : n \neq n\}$
  - (c)  $\{n : n = 3 \text{ or } n = 17\}$
  - (d)  $\{n : n = n^2\} = \{0, 1\}$

- Exercise 1.1.8
- (a)  $A$  and  $B$ : 7 is either, none in both
  - (b)  $A$  and  $C$ : 3 in either, 1 in both
  - (c)  $A$  and  $D$ : 5 in either, 1 in both
  - (d)  $A$  and  $E$ : infinitely many in either, 3 in both
  - (e)  $B$  and  $C$ : 5 in either, none in both
  - (f)  $B$  and  $D$ : 5 in either, 2 in both
  - (g)  $B$  and  $E$ : infinitely many in either, 1 in both
  - (h)  $C$  and  $D$ : 4 in either, none in both
  - (i)  $C$  and  $E$ : infinitely many in either, 1 in both
  - (j)  $D$  and  $E$ : infinitely many in either, 2 in both

Exercise 1.1.9 It is possible if both sets are empty, because then every novelist in the empty set is in the empty set of naturals, or vice versa. But if there is any novelist in the first set or any natural in the second, that element is not in the other set and the sets are not equal.

Exercise 1.1.10 It is not possible. If  $A$  is a proper subset of  $B$ , there must be some element  $x$  that is in  $B$  but not in  $A$ . This means that  $B$  cannot be a subset of  $A$  at all, much less a proper subset.

- Exercise 1.2.1
- (a) true,  $c$  followed by  $ba$  is  $cba$
  - (b) false,  $(vw)^R = bac$
  - (c) true
  - (d) true,  $3 = 1 + 2$
  - (e) false,  $v = c^R$
  - (f) false,  $ab$  is not a suffix of  $cba$
  - (g) true,  $ab$  is a prefix of  $abc$
  - (h) true
  - (i) false, letters of  $w$  occur in  $u$  but in the wrong order

Exercise 1.2.2 The lengths are 6, 0, 15, and 7.

Exercise 1.2.3 The strings are *garage*, *agedam*, *damage*, *ragdam*, *madrag*, and *gargarmadage*.

Exercise 1.2.4 Prefixes of *team*:  $\lambda$ , *t*, *te*, *tea*, and *team*. Suffixes of *team*:  $\lambda$ , *m*, *am*, *eam*, and *team*. Other substrings of *team*: *e*, *ea*, and *a*. Prefixes of *mama*:  $\lambda$ , *m*, *ma*, *mam*, and *mama*. Suffixes of *mama*:  $\lambda$ , *a*, *ma*, *ama*, and *mama*. Other substrings of *mama*: *am*, which is neither a prefix nor a suffix.

Exercise 1.2.5 Prefix *rhi*, suffix *ros*, others *hin*, *ino*, *noc*, *oce*, *cer*, and *ero*.

Exercise 1.2.6 The simplest example is  $u = a$ ,  $v = aa$ .

Exercise 1.2.7 (a) It is in  $E$  if and only if it is in  $NZ$  and it ends with 0, 2, 4, 6, or 8. It is in  $H$  if and only if it is in  $NZ$  and it ends with 00.  
 (b)  $E^R$  is the set of strings in  $D^*$  that do not end in 0 and that start with 0, 2, 4, 6, or 8. The strings in  $E^R$  that are also in  $NZ$  are exactly those that do not start with 0.  $H^R$  is the set of strings that start with 00 and do not end in 0. None of these strings are in  $NZ$ .

Exercise 1.2.8 (a) 01111, 10111, 11011, 11101, 11110, and 11111  
 (b) 00000, 00001, 00010, 00100, 00101, 01000, 01001, 01010, 10000, 10001, 10010, 10100, and 10101.  
 (c) 01010, 01011, 01101, 01110, 01111, 10101, 10110, 10111, 11010, 11011, 10111, 11110, and 11111.  
 (d) none  
 (e) same as X  
 (f) 00011, 00110, 00111, 01100, 10011, 11000, 11001, and 11100.

Exercise 1.2.9 (a) 

```
boolean equals(String u, String v) {
    if (u.length() != v.length()) return false;
    for (int i = 0; i < u.length(); i++)
        if (u.charAt(i) != v.charAt(i)) return false;
    return true;}
```

  
 (b) 

```
boolean prefix(String u, String v) {
    if (u.length() > v.length()) return false;
    for (int i = 0; i < u.length(); i++)
        if (u.charAt(i) != v.charAt(i)) return false;
    return true;}
```

  
 (c) 

```
boolean suffix(String u, String v) {
    int offset = v.length() - u.length();
    if (offset < 0) return false;
    for (int i = 0; i < u.length(); i++)
        if (u.charAt(i) != v.charAt(offset + i)) return false;
    return true;}
```

Exercise 1.2.10 This is not necessarily true. The simplest counterexample is to have  $A$  and  $B$  empty, so that all possible concatenations are in  $C$  because there are no possible concatenations, and let  $C = \{ab\}$  so that  $C$  has a string whose reversal is not in  $C$ .

- Exercise 1.4.1 (a) false, would fail if  $q$  is false  
 (b) false, it fails if  $p$  and  $q$  are both false  
 (c) true, inclusive OR is true if both components are true  
 (d) true, it is properly formed from atomic propositions by boolean operations  
 (e) false, for example a tautology is true whatever the values of its components
- Exercise 1.4.2 (a) yes, a false proposition  
 (b) not a proposition, can't be true or false  
 (c) yes, a proposition that depends on a future event  
 (d) yes, a proposition depending on the speaker's state of mind
- Exercise 1.4.3 If "This statement is false" were a true proposition, it would have to be false. If it were a false proposition, then "That statement is not false" would be a true proposition, and since a proposition must be true or false this would force "That statement is true" to be true. Either assumption makes the original statement both true and false, which is impossible. So the original statement is not a proposition at all.
- Exercise 1.4.4 (a)  $\text{true} \ \&\& \ \text{true}$ , which is true  
 (b)  $\text{!(false} \ || \ \text{true)}$ , which is  $\text{!true}$ , which is false  
 (c)  $(\text{true} \ \wedge \ \text{false}) \ \wedge \ \text{false}$ , which is  $\text{true} \ \wedge \ \text{false}$ , which is true  
 (d)  $(\text{true} \ || \ \text{!false}) \ \&\& \ (\text{!true} \ \&\& \ \text{false})$ , which is  $\text{true} \ \&\& \ \text{false}$ , which is false
- Exercise 1.4.5 (a) false,  $1 \rightarrow 0 = 0$   
 (b) true,  $1 \leftrightarrow 1 = 1$   
 (c) true,  $\neg 1 \oplus \neg 0 = 0 \oplus 1 = 1$   
 (d) true,  $1 \vee (0 \wedge \neg 0) = 1 \vee 0 = 1$   
 (e) true,  $(1 \vee 0) \wedge \neg 0 = 1 \wedge 1 = 1$   
 (f) true, since first part of  $\rightarrow$  is  $\neg 1 \vee 0 = 0 \vee 0 = 0$
- Exercise 1.4.6 (a) If fish live in water, then trout live in trees.  
 (b) Trout do not live in trees if and only if fish live in water.  
 (c) Either fish do not live in water, or trout do not live in trees, but not both.  
 (d) Either fish live in water, or both trout live in trees and trout do not live in trees, or both.  
 (e) Either fish live in water or trout live in trees, or both, and trout do not live in trees.  
 (f) If either fish do not live in water or trout live in trees, or both, then fish do not live in water and if trout do not live in trees, then fish live in water.
- Exercise 1.4.7 (a) exclusive, they won't be both  
 (b) inclusive, they'd be happy with both  
 (c) inclusive, they'd be happy with both  
 (d) exclusive, they can't be both



Exercise 1.4.8 (a)  $p \rightarrow q$

(b)  $q \rightarrow p$

(c)  $p \wedge (q \rightarrow \neg q)$

(d)  $(p \oplus q) \rightarrow (\neg q \wedge \neg p)$

Exercise 1.4.9 (a)  $\neg p \vee q$ , Either mackerel are not fish or trout live in trees, or both.

(b)  $\neg q \vee p$ , Either trout do not live in trees or mackerel are fish, or both.

(c)  $p \wedge (\neg q \vee \neg q)$ , Mackerel are fish and either trout do not live in trees or trout do not live in trees, or both.

(d)  $\neg(p \oplus q) \vee (\neg q \wedge \neg p)$ , Either it is not the case that either mackerel are fish or trout live in trees, but not both, or both trout do not live in trees and mackerel are not fish, or both.

Exercise 1.4.10 (a) Mackerel are fish if and only if trout live in trees.

(b) Trout live in trees if and only if mackerel are fish.

(c) Mackerel are fish, and trout live in trees if and only if trout do not live in trees.

(d) Mackerel are fish or trout live in trees, or both, if and only if trout do not live in trees.

Exercise 1.5.1 (a) The set of black horses

(b) The set of animals that are either female or are black sheep, or both

(c) The set of black animals that are either female or sheep, or both

(d) The set of animals that are not female horses

(e) The set of female animals that are neither sheep nor horses

(f) The set of animals that are either (a) horses or (b) female or sheep, but not both black and not female, but not both (a) and (b).

Exercise 1.5.2 (a)  $F \setminus H$

(b)  $(F \cap S) \cup (B \cap H)$

(c)  $\overline{B \cup S}$

(d)  $(F \cap B \cap S) \Delta (\overline{B} \cap H)$

Exercise 1.5.3 (a)  $\{x : x \text{ either has five or more letters or has two } a\text{'s in a row, or both}\}$

(b)  $\{x : x \text{ both has five or more letters and has two } a\text{'s in a row}\}$

(c)  $\{x : x \text{ has five or more letters but does not have two } a\text{'s in a row}\}$

(d)  $\{x : x \text{ either has five or more letters or does not both have five or more letters and have two } a\text{'s in a row}\}$

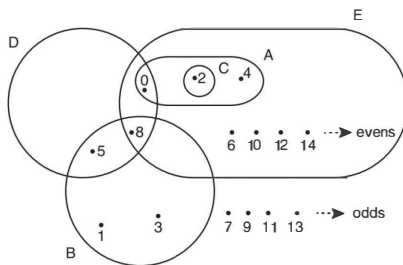
Exercise 1.5.4 (a)  $\{0, 1, 2, 3, 4, 5, 8\}$

(b)  $\emptyset$

(c)  $\{0, 1, 3, 5, 8\}$

(d)  $\{5, 8\}$

(e)  $\{x : x \text{ is even}\}$ , same as  $E$



©Kendall Hunt Publishing Company

Figure S-1: The Venn Diagram for Exercise 1.5.5

- (f)  $\{0, 1, 2, 3, 4, 5, 7, 9, 11, \dots\}$  or  $\{x : x \leq 4 \vee x \text{ is odd}\}$
- (g)  $\{1, 3, 5\}$
- (h)  $\{2, 4, 6, 10, 12, 14, \dots\}$  or  $\{x : x \text{ is even but not } 0 \text{ or } 8\}$
- (i)  $\{0, 4, 6, 7, 9, 10, 11, \dots\}$  or  $\{x : (x = 0) \vee (x = 4) \vee (x = 6) \vee (x = 7) \vee (x \geq 9)\}$
- (j)  $\{0, 1, 3, 4, 5, \dots\}$  or  $\{x : x \neq 2\}$ , same as  $\overline{C}$
- (k)  $\{0, 1, 3\}$
- (l)  $\{6, 8, 10, \dots\}$  or  $\{x : x \geq 6 \text{ and } x \text{ is even}\}$

Exercise 1.5.5 See Figure S-1 for the diagram.

- Exercise 1.5.6
- (a)  $\{x : 0 \leq x \leq 13\}$
  - (b)  $\{x : 23 \leq x \leq 134\}$
  - (c)  $\{x : x = 10y + 1 \text{ for some natural } y\}$
  - (d)  $\{x : x = y^2 \text{ for some natural } y\}$

Exercise 1.5.7 No element can be in both  $A \setminus B$  and  $B \setminus A$ , so the only way that  $A \setminus B$  can equal  $B \setminus A$  is if these two sets are both empty. This is the case if and only if  $A$  and  $B$  are the same set, that is, if  $A = B$ .

- Exercise 1.5.8
- (a) If an element is in both  $A$  and  $B$ , it must be in  $B$ .
  - (b) If an element is in both  $A$  and  $B$ , clearly it is in either  $A$  or  $B$ , or both.
  - (c) If an element is in both  $A$  and  $A$ , then it is in  $A$ , and vice versa.
  - (d) If an element is in either  $A$  or  $A$ , then it is in  $A$ , and vice versa.
  - (e) Since an element cannot be in  $A$  but not in  $A$ , the set of such elements is empty.
  - (f) Since no element can be in  $A$  or in  $A$  but not in both, the set of such elements is empty.
  - (g) To be in the left-hand set, an element must either be in  $A$  or  $B$  but not both, or be in both  $A$  and  $B$ . Clearly this is the same as being in either  $A$  or  $B$ , or both, which is what it means to be in the right-hand set.

- Exercise 1.5.9 (a)  $((x \in A) \wedge (x \in B)) \rightarrow (x \in B)$   
 (b)  $((x \in A) \wedge (x \in B)) \rightarrow ((x \in A) \vee (x \in B))$   
 (c)  $((x \in A) \wedge (x \in A)) \leftrightarrow (x \in A)$   
 (d)  $((x \in A) \vee (x \in A)) \leftrightarrow (x \in A)$   
 (e)  $((x \in A) \wedge \neg(x \in A)) \leftrightarrow 0$   
 (f)  $((x \in A) \oplus (x \in A)) \leftrightarrow 0$   
 (g)  $((x \in A) \oplus (x \in B)) \vee ((x \in A) \wedge (x \in B)) \leftrightarrow ((x \in A) \vee (x \in B))$

- Exercise 1.5.10 (a) The  $\wedge$  on the left forces both parts to be true.  
 (b) Since the  $\wedge$  on the left forces both parts to be true, at least one is true, and thus the  $\vee$  on the right is true.  
 (c) Taking the  $\wedge$  of a statement with itself yields an equivalent statement.  
 (d) Taking the  $\vee$  of a statement with itself yields an equivalent statement.  
 (e) The left-hand side can only be true if the statement  $x \in A$  is true but not true, which is impossible.  
 (f) Taking the  $\oplus$  of a statement with itself yields a false statement, since one part cannot be true without the other being true.  
 (g) If the  $\oplus$  is true, exactly one of the two parts is true, and if the  $\wedge$  is true, both parts are true. Either way, at least one is true and so the  $\vee$  is true.

Exercise 1.6.1

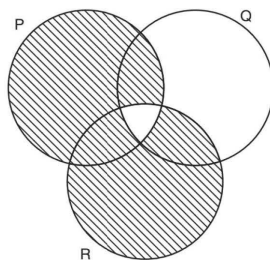
| $p$ | $q$ | $(p \oplus q)$ | $\leftrightarrow$ | $(\neg(p \rightarrow q) \vee (\neg(q \rightarrow p)))$ |
|-----|-----|----------------|-------------------|--------------------------------------------------------|
| 0   | 0   | 0              | 0                 | 0                                                      |
| 0   | 1   | 1              | 0                 | 1                                                      |
| 1   | 0   | 1              | 0                 | 1                                                      |
| 1   | 1   | 0              | 0                 | 0                                                      |

Exercise 1.6.2 It is not a tautology, because the second column of this truth table (representing the  $\leftrightarrow$ ) is not all ones.

| $p$ | $q$ | $p \leftrightarrow [(p \rightarrow q) \vee (q \wedge p)]$ |
|-----|-----|-----------------------------------------------------------|
| 0   | 0   | 0                                                         |
| 0   | 1   | 0                                                         |
| 1   | 0   | 0                                                         |
| 1   | 1   | 1                                                         |

Exercise 1.6.3 The Venn diagram is in Figure S-2.

| $p$ | $q$ | $r$ | $(p \wedge (q \vee r)) \oplus (r \oplus (p \wedge \neg q))$ |
|-----|-----|-----|-------------------------------------------------------------|
| 0   | 0   | 0   | 0                                                           |
| 0   | 0   | 1   | 1                                                           |
| 0   | 1   | 0   | 0                                                           |
| 0   | 1   | 1   | 1                                                           |
| 1   | 0   | 0   | 0                                                           |
| 1   | 0   | 1   | 1                                                           |
| 1   | 1   | 0   | 1                                                           |
| 1   | 1   | 1   | 0                                                           |



©Kendall Hunt Publishing Company

Figure S-2: The Venn Diagram for Exercise 1.6.3

Exercise 1.6.4 The truth tables show that the two compound propositions have different truth values in two of the eight possible situations, those corresponding to  $p \wedge \neg q \wedge \neg r$  and to  $p \wedge q \wedge \neg r$ .

| $p$ | $q$ | $r$ | $p \vee (q \wedge r)$ | $(p \vee q) \wedge r$ |
|-----|-----|-----|-----------------------|-----------------------|
| 0   | 0   | 0   | 0                     | 0                     |
| 0   | 0   | 1   | 0                     | 0                     |
| 0   | 1   | 0   | 0                     | 1                     |
| 0   | 1   | 1   | 1                     | 1                     |
| 1   | 0   | 0   | 1                     | 0                     |
| 1   | 0   | 1   | 1                     | 1                     |
| 1   | 1   | 0   | 1                     | 0                     |
| 1   | 1   | 1   | 1                     | 1                     |

Exercise 1.6.5 The table below has the sixteen possible columns labeled 0 through 15, with the explicit compound proposition at the bottom of each column.

|     |     |   |              |                   |     |                   |     |              |            |
|-----|-----|---|--------------|-------------------|-----|-------------------|-----|--------------|------------|
| $p$ | $q$ | 0 | 1            | 2                 | 3   | 4                 | 5   | 6            | 7          |
| 0   | 0   | 0 | 0            | 0                 | 0   | 0                 | 0   | 0            | 0          |
| 0   | 1   | 0 | 0            | 0                 | 0   | 1                 | 1   | 1            | 1          |
| 1   | 0   | 0 | 0            | 1                 | 1   | 0                 | 0   | 1            | 1          |
| 1   | 1   | 0 | 1            | 0                 | 1   | 0                 | 1   | 0            | 1          |
|     |     | 0 | $p \wedge q$ | $p \wedge \neg q$ | $p$ | $\neg p \wedge q$ | $q$ | $p \oplus q$ | $p \vee q$ |

|     |     |                        |                       |          |                 |          |                 |                      |    |
|-----|-----|------------------------|-----------------------|----------|-----------------|----------|-----------------|----------------------|----|
| $p$ | $q$ | 8                      | 9                     | 10       | 11              | 12       | 13              | 14                   | 15 |
| 0   | 0   | 1                      | 1                     | 1        | 1               | 1        | 1               | 1                    | 1  |
| 0   | 1   | 0                      | 0                     | 0        | 0               | 1        | 1               | 1                    | 1  |
| 1   | 0   | 0                      | 0                     | 1        | 1               | 0        | 0               | 1                    | 1  |
| 1   | 1   | 0                      | 1                     | 0        | 1               | 0        | 1               | 0                    | 1  |
|     |     | $\neg p \wedge \neg q$ | $p \leftrightarrow q$ | $\neg q$ | $p \vee \neg q$ | $\neg p$ | $\neg p \vee q$ | $\neg p \vee \neg q$ | 1  |

Exercise 1.6.6

| $x$ | $y$ | $z$ | Value | $ITE(x, y, z)$ |
|-----|-----|-----|-------|----------------|
| 0   | 0   | 0   | $z$   | 0              |
| 0   | 0   | 1   | $z$   | 1              |
| 0   | 1   | 0   | $z$   | 0              |
| 0   | 1   | 1   | $z$   | 1              |
| 1   | 0   | 0   | $y$   | 0              |
| 1   | 0   | 1   | $y$   | 0              |
| 1   | 1   | 0   | $y$   | 1              |
| 1   | 1   | 1   | $y$   | 1              |

Exercise 1.6.7 The columns for the atomic variables are not consistent. The column for the first  $p$  agrees with that for the second  $q$ , and vice versa. Every column with the same atomic variable at the top should be the same.

Exercise 1.6.8 The compound proposition is a contradiction if and only if the column for the final operation is all zeros. It is satisfiable if and only if there is one or more ones in that column. (That is, it is satisfiable if and only if it is not a contradiction.)

Exercise 1.6.9 If you know that  $x_1$  is true, for example, you may ignore the  $2^{k-1}$  lines that have  $x_1$  half, leaving only  $2^{k-1}$  lines to consider, half the original number. If you know that  $p \rightarrow q$  is false, then you know both that  $p$  is true and that  $q$  is false. Only a quarter of the lines of the table,  $2^{k-2}$  of them, have these two properties.

Exercise 1.6.10 If the first old column (for  $P$ , say) has a 1, you may fill in a 1 for  $P \vee Q$  without looking at  $Q$ . If  $P$  is 0, you may fill in 1 for  $P \rightarrow Q$  without looking at  $Q$ . But in the other two cases, you cannot be sure of the result of  $P \leftrightarrow Q$  or  $P \oplus Q$  without looking at both  $P$  and  $Q$ .

Exercise 1.7.1 (a)

| $p$ | $q$ | $(p \oplus q)$ | $\leftrightarrow$ | $((p \wedge \neg q) \vee (\neg p \wedge q))$ |
|-----|-----|----------------|-------------------|----------------------------------------------|
| 0   | 0   | 0              | 1                 | 0                                            |
| 0   | 1   | 1              | 1                 | 0                                            |
| 1   | 0   | 1              | 1                 | 0                                            |
| 1   | 1   | 0              | 1                 | 0                                            |

(b)

| $p$ | $q$ | $(p \wedge q) \rightarrow p$ |
|-----|-----|------------------------------|
| 0   | 0   | 1                            |
| 0   | 1   | 1                            |
| 1   | 0   | 1                            |
| 1   | 1   | 1                            |

(c)

| $p$ | $q$ | $\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$ |
|-----|-----|---------------------------------------------------------|
| 0   | 0   | 1                                                       |
| 0   | 1   | 1                                                       |
| 1   | 0   | 1                                                       |
| 1   | 1   | 0                                                       |

(d)

| $p$ | $q$ | $r$ | $((p \wedge r) \rightarrow q) \wedge ((p \wedge \neg r)) \rightarrow q \rightarrow (p \rightarrow q)$ |   |   |   |   |   |   |   |   |   |   |   |
|-----|-----|-----|-------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|
| 0   | 0   | 0   | 0                                                                                                     | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 0   | 0   | 1   | 0                                                                                                     | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 0   | 1   | 0   | 0                                                                                                     | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 0   | 1   | 1   | 0                                                                                                     | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 1   | 0   | 0   | 1                                                                                                     | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1   | 0   | 1   | 1                                                                                                     | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |
| 1   | 1   | 0   | 1                                                                                                     | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 1   | 1   | 1   | 1                                                                                                     | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 |

(e)

| $p$ | $(\neg p \rightarrow 0) \rightarrow p$ |
|-----|----------------------------------------|
| 0   | 1                                      |
| 1   | 0                                      |

(f)

| $p$ | $q$ | $r$ | $((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$ |   |   |   |   |   |   |   |   |   |   |   |
|-----|-----|-----|------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|
| 0   | 0   | 0   | 0                                                                            | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0   | 0   | 1   | 0                                                                            | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 0   | 1   | 0   | 0                                                                            | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0   | 1   | 1   | 0                                                                            | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1   | 0   | 0   | 1                                                                            | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 1   | 0   | 1   | 1                                                                            | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1   | 1   | 0   | 1                                                                            | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 1   | 1   | 1   | 1                                                                            | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

Exercise 1.7.2 (a)

| $p$ | $q$ | $(p \wedge q) \leftrightarrow (q \wedge p)$ |   |   |   |   |   |   |   |
|-----|-----|---------------------------------------------|---|---|---|---|---|---|---|
| 0   | 0   | 0                                           | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0   | 1   | 0                                           | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1   | 0   | 1                                           | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 1   | 1   | 1                                           | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

(b)

|                                  |                           |
|----------------------------------|---------------------------|
| $p \vee q$                       | Premise                   |
| $\neg \neg p \vee q$             | Double Negation           |
| $\neg p \rightarrow q$           | Definition of Implication |
| $\neg q \rightarrow \neg \neg p$ | Contrapositive            |
| $\neg q \rightarrow p$           | Double Negation           |
| $q \vee p$                       | Definition of Implication |

(c)

|                                              |                              |
|----------------------------------------------|------------------------------|
| $p \oplus q$                                 | Premise                      |
| $((p \wedge \neg q) \vee (\neg p \wedge q))$ | Definition of Exclusive Or   |
| $((\neg q \wedge p) \vee (q \wedge \neg p))$ | Commutativity of AND (twice) |
| $((q \wedge \neg p) \vee (\neg q \wedge p))$ | Commutativity of OR          |
| $q \oplus p$                                 | Definition of Exclusive Or   |

(d)

| $p$ | $q$ | $r$ | $((p \wedge q) \wedge r) \leftrightarrow (p \wedge (q \wedge r))$ |
|-----|-----|-----|-------------------------------------------------------------------|
| 0   | 0   | 0   | 0                                                                 |
| 0   | 0   | 1   | 0                                                                 |
| 0   | 1   | 0   | 0                                                                 |
| 0   | 1   | 1   | 0                                                                 |
| 1   | 0   | 0   | 1                                                                 |
| 1   | 0   | 1   | 1                                                                 |
| 1   | 1   | 0   | 1                                                                 |
| 1   | 1   | 1   | 1                                                                 |

(e)

|                                              |                      |
|----------------------------------------------|----------------------|
| $(p \vee q) \vee r$                          | Premise              |
| $\neg[\neg(p \vee q) \wedge \neg r]$         | Demorgan Or-To-And   |
| $\neg[(\neg p \wedge \neg q) \wedge \neg r]$ | DeMorgan Or-To-And   |
| $\neg[\neg p \wedge (\neg q \wedge \neg r)]$ | Associativity of AND |
| $\neg[\neg p \wedge \neg(q \vee r)]$         | DeMorgan And-to-Or   |
| $p \vee (q \vee r)$                          | DeMorgan And-To-Or   |

- Exercise 1.7.3 (a)  $\neg((a \wedge \neg b) \vee (a \oplus b)) \leftrightarrow (\neg(a \wedge \neg b) \wedge \neg(a \oplus b))$   
 (b)  $((a \rightarrow b) \rightarrow (b \rightarrow a)) \wedge ((b \rightarrow a) \rightarrow (a \leftrightarrow b)) \rightarrow ((a \rightarrow b) \rightarrow (a \leftrightarrow b))$   
 (c)  $((a \wedge b) \leftrightarrow (a \vee b)) \leftrightarrow (((a \wedge b) \rightarrow (a \vee b)) \wedge ((a \vee b) \rightarrow (a \wedge b)))$

- Exercise 1.7.4 (a) Left Separation,  $a \vee \neg b$  for  $p$ ,  $c \rightarrow d$  for  $q$   
 (b) Excluded Middle,  $r \rightarrow \neg p$  for  $p$   
 (c) Contrapositive,  $a \wedge b$  for  $p$ ,  $b$  for  $q$

Exercise 1.7.5

|                                                        |                             |
|--------------------------------------------------------|-----------------------------|
| $p \leftrightarrow q$                                  | Premise                     |
| $(p \rightarrow q) \wedge (q \rightarrow p)$           | Equivalence and Implication |
| $(\neg p \vee q) \wedge (\neg q \vee p)$               | Definition of Implication   |
| $\neg[(\neg(\neg p \vee q) \vee \neg(\neg q \vee p))]$ | DeMorgan And-To-Or          |
| $\neg[(p \wedge \neg q) \vee (q \wedge \neg p)]$       | DeMorgan Or-To-And          |
| $\neg(p \oplus q)$                                     | Definition of Exclusive OR  |

Exercise 1.7.6 Simplest is  $P = 0$  and  $Q = 1$ . The compound proposition  $(r \oplus 0) \rightarrow (r \oplus 1)$  simplifies to  $r \rightarrow \neg r$ , which is not a tautology because it is false when  $r$  is true.

Exercise 1.7.7

|                             |                           |
|-----------------------------|---------------------------|
| $p \rightarrow q$           | Premise                   |
| $\neg p \vee q$             | Definition of Implication |
| $q \vee \neg p$             | Commutativity of OR       |
| $\neg(\neg q) \vee \neg p$  | Double Negation           |
| $\neg q \rightarrow \neg p$ | Definition of Implication |

Exercise 1.7.8

1.  $\neg(q \rightarrow r)$  Second Premise
2.  $\neg(\neg q \vee r)$  Definition of Implication
3.  $q \wedge \neg r$  DeMorgan
4.  $q$  Left Separation
5.  $p \vee \neg q$  First Premise
6.  $\neg q \vee p$  Commutativity of OR
7.  $q \rightarrow p$  Definition of Implication
8.  $p$  Modus Ponens, lines 4 and 7

Exercise 1.7.9

1.  $\neg p$  Second Premise
2.  $p \vee q$  First Premise
3.  $\neg p \rightarrow q$  Definition of Implication
4.  $q$  Modus Ponens, lines 1 and 3

Exercise 1.7.10

1.  $p \wedge (q \vee r)$  Premise for first proof
2.  $p$  Left Separation, line 1
3.  $q$  Assumption for Case 1 of first proof
4.  $p \wedge q$  Conjunction, lines 2 and 3
5.  $(p \wedge q) \vee (p \wedge r)$  Left Joining, conclusion of Case 1
6.  $\neg q$  Assumption for Case 2 of first proof
7.  $q \vee r$  Right Separation, line 1
8.  $r$  Tertium Non Datur (see Exercise 1.7.8), lines 6 and 7
9.  $p \wedge r$  Conjunction, lines 2 and 8
10.  $(p \wedge q) \vee (p \wedge r)$  Right Joining, conclusion of Case 2
11.  $(p \wedge (q \vee r)) \rightarrow ((p \wedge q) \vee (p \wedge r))$  Proof by Cases, end of first proof
12.  $(p \wedge q) \vee (p \wedge r)$  Premise for second proof
13.  $p \wedge q$  Assumption for Case 1 of second proof
14.  $p$  Left Separation
15.  $q$  Right Separation, line 13
16.  $q \vee r$  Left Joining
17.  $p \wedge (q \vee r)$  Conjunction, lines 14 and 16, conclusion of Case 1
18.  $\neg(p \wedge q)$  Assumption for Case 2 of second proof
19.  $p \wedge r$  Tertium Non Datur, lines 12 and 18
20.  $r$  Right Separation
21.  $q \vee r$  Right Joining
22.  $p \wedge (q \vee r)$  Conjunction, lines 14 and 21, conclusion of Case 2
23.  $((p \wedge q) \vee (p \wedge r)) \rightarrow (p \wedge (q \vee r))$  Proof by Cases, end of second proof
24.  $((p \wedge q) \vee (p \wedge r)) \leftrightarrow (p \wedge (q \vee r))$  Equivalence and Implication

Exercise 1.8.1

- 0  $\wedge p$  Premise
- 0 Left Separation
- (0  $\wedge p$ )  $\rightarrow$  0 Direct Proof
- $\neg 0$  Definition of 0
- 0  $\rightarrow$  (0  $\wedge p$ ) Vacuous Proof
- (0  $\wedge p$ )  $\leftrightarrow$  0 Equivalence and Implication

Exercise 1.8.2 Define propositional variables  $w$  ("witches float"),  $r$  ("very small rocks float"),  $c$



(“churches float”), and  $d$  (“ducks float”).

- |     |                                            |                                                           |
|-----|--------------------------------------------|-----------------------------------------------------------|
| 1.  | $w \vee r \vee c$                          | First Premise                                             |
| 2.  | $\neg r \wedge (w \rightarrow d)$          | Second Premise                                            |
| 3.  | $c$                                        | Assumption for Proof By Cases                             |
| 4.  | $c \vee d$                                 | Right Joining                                             |
| 5.  | $\neg c$                                   | Assumption for other half of Proof By Cases               |
| 6.  | $\neg r$                                   | Left Separation (from second premise)                     |
| 7.  | $(r \vee c) \vee w$                        | Commutativity and Associativity of OR, from first premise |
| 8.  | $\neg(r \vee c) \rightarrow w$             | Definition of Implication                                 |
| 9.  | $\neg c \wedge \neg r$                     | Conjunction from lines 5 and 6                            |
| 10. | $\neg(r \vee c)$                           | DeMorgan Or-To-And                                        |
| 11. | $w$                                        | Modus Ponens (lines 10, 8)                                |
| 12. | $w \rightarrow d$                          | Right Separation, second premise                          |
| 13. | $d$                                        | Modus Ponens (lines 11, 12)                               |
| 14. | $c \vee d$                                 | Left Joining                                              |
| 15. | $(\text{premises}) \rightarrow (c \vee d)$ | Proof By Cases                                            |

Exercise 1.8.3 (a) Subgoals are  $(p \wedge q \wedge q) \rightarrow (p \vee q)$  and  $(p \wedge q \wedge \neg q) \rightarrow (p \vee q)$ . For the first proof, derive  $q$  by Right Separation and then  $p \vee q$  by Left Joining. For the second, use associativity of AND and Excluded Middle to get  $p \wedge 0$ , which is 0 by Right Zero, and derive  $p \vee q$  from 0 by Vacuous Proof.

(b) Subgoals are  $(p \wedge q) \rightarrow p$  and  $p \rightarrow (p \vee q)$ . But each of these subgoals can be proved by a single rule, Left Separation and Right Joining respectively.

Exercise 1.8.4 Contrapositive gives a premise of  $\neg(p \vee q)$  and a conclusion of  $\neg(p \wedge q)$ . The proof goes through  $\neg p \wedge \neg q$  (DeMorgan),  $\neg p$  (Left Separation), and  $\neg p \vee \neg q$  (Right Joining), with the last step DeMorgan again.

Contradiction gives a premise of  $(p \wedge q) \wedge \neg(p \vee q)$  and a conclusion of 0. The premise can be used to derive  $p \wedge q \wedge (\neg p \wedge \neg q)$  (by DeMorgan), and then commutativity and associativity of AND can be used to get  $p \wedge \neg p$  ANDed with other things, which is 0 by Excluded Middle and the Zero rules.

Exercise 1.8.5 (a) Get  $p$  by Left Separation.

(b) Get  $\neg p \wedge \neg(q \vee r)$  by DeMorgan Or-to-And.

(c) Change the second ANDed component to  $q \rightarrow r$  by Contrapositive.

Exercise 1.8.6 (a) Use the Vacuous Proof rule to get the desired implication from  $p$ .

(b) Convert  $r \rightarrow (\neg p \wedge q)$  to the desired OR statement.

(c) Use Modus Ponens on  $r \rightarrow p$  and  $r$  to get  $p$ .

Exercise 1.8.7 Letting  $s$  mean “you say a word” and  $c$  mean “I will cut off your heads”, our premise is  $(s \rightarrow c) \wedge (\neg s \rightarrow c)$ . We can easily derive  $c$  from this premise:

|    |                                                                   |                                                   |
|----|-------------------------------------------------------------------|---------------------------------------------------|
| 1. | $(s \rightarrow c) \wedge (\neg s \rightarrow c)$                 | Premise                                           |
| 2. | $s$                                                               | Assumption for Case 1                             |
| 3. | $s \rightarrow c$                                                 | Left Separation, line 1                           |
| 4. | $c$                                                               | Modus Ponens, lines 2 and 3, conclusion of Case 1 |
| 5. | $\neg s$                                                          | Assumption for Case 2                             |
| 6. | $\neg s \rightarrow c$                                            | Right Separation, line 1                          |
| 7. | $c$                                                               | Modus Ponens, lines 5 and 6, conclusion of Case 2 |
| 8. | $((s \rightarrow c) \wedge (\neg s \rightarrow c)) \rightarrow c$ | Proof By Cases, end of proof                      |

#### Exercise 1.8.8

- Since we have  $p \vee r$  as the front of an implication in the premise, we would like to form it so we can use Modus Ponens with that implication.
- We have only done one of the two cases – our proof of  $s$  used an assumption but we eventually need to prove that  $s$  holds with or without that assumption.
- We need to put the statement of line 7 into a form where we can use Modus Ponens with line 6. If we instead used Tertium Non Datur from Exercise 1.7.8, we could use line 7 directly without this transformation.
- In line 3, we were operating under the assumption of the first case, and now we are not. We cannot guarantee that something true in one case can be taken to use in a different case.

#### Exercise 1.8.9

|     |                                                                           |                                                      |
|-----|---------------------------------------------------------------------------|------------------------------------------------------|
| 1.  | $p \vee (q \wedge r)$                                                     | Assumption for first half                            |
| 2.  | $p$                                                                       | Assumption for Case 1 of first half                  |
| 3.  | $p \vee r$                                                                | Left Joining, conclusion of Case 1                   |
| 4.  | $\neg p$                                                                  | Assumption for Case 2 of first half                  |
| 5.  | $q \wedge r$                                                              | Tertium Non Datur (Exercise 1.7.8), lines 1 and 4    |
| 6.  | $r$                                                                       | Right Separation                                     |
| 7.  | $p \vee r$                                                                | Right Joining, conclusion of Case 2                  |
| 8.  | $(p \vee (q \wedge r)) \rightarrow (p \vee r)$                            | Proof By Cases, end of first half                    |
| 9.  | $p \vee r$                                                                | Assumption for second half                           |
| 10. | $(p \vee r) \rightarrow s$                                                | Second premise                                       |
| 11. | $s$                                                                       | Modus Ponens, lines 9 and 10                         |
| 12. | $(p \vee r) \rightarrow s$                                                | Conclusion of second half                            |
| 13. | $((p \vee (q \wedge r)) \wedge ((p \vee r) \rightarrow s)) \rightarrow s$ | Hypothetical Syllogism, lines 8 and 12, end of proof |

Exercise 1.8.10 We need to show that  $P$  still implies  $C$  in the other case, where  $q \leftrightarrow$  is false. So we need a new proof starting from  $P \wedge \neg(q \leftrightarrow r)$  or  $P \wedge (q \oplus r)$ .

- Exercise 1.10.1
- Signature “real  $x$ , real  $y$ , real  $z$ ”, template “ $(y < x < z) \vee (z < x < y)$ ”.
  - Signature “real  $x$ , real  $y$ ”, template “ $x^2 + y^2 = 1$ ”.
  - Signature “team  $X$ ”, template “the first three runners were from  $X$ ”.
  - Signature “player  $p$ , team  $t$ ”, template “the batting average and OBP of  $p$  are both higher than those of the center fielder of  $t$ ”.

- Exercise 1.10.2
- “The strings  $a$  and  $ab$  start with the same letter, and either  $ba$  has exactly two letters, or  $aab$  starts and ends with the same letter, but not both.” This is TRUE ( $1 \wedge (1 \oplus 0) = 1$ ).

- (b) “If  $a$  and  $a$  start with the same letter, then either  $aaa$  starts and ends with the same letter or  $b$  has exactly two letters if and only if  $b$  and  $aba$  start with the same letter.” This is TRUE ( $1 \rightarrow (1 \vee (1 \leftrightarrow 1)) = 1$ ).
- (c) “If  $a$  has exactly two letters, then  $aa$  starts and ends with the same letter and either  $b$  and  $bbb$  start with the same letter, or  $\lambda$  has exactly two letters, but not both.” This is also TRUE ( $0 \rightarrow (1 \wedge (1 \oplus 0)) = 1$ ).

- Exercise 1.10.3 (a)  $(P(ab) \wedge R(w, ab)) \rightarrow Q(w)$ ,  $w$  is the only free variable.  
 (b)  $[Q(aba) \oplus R(aba, bab)] \rightarrow [R(aa, bb) \wedge P(aa)]$ , there are no free variables.  
 (c)  $\neg R(u, v) \rightarrow [(P(u) \wedge Q(u)) \oplus (P(v) \wedge Q(v))]$ ,  $u$  and  $v$  are the free variables.
- Exercise 1.10.4 Such a predicate is called a “proposition”, since it is true or false already without any variable values needing to be supplied. There is no reason why a predicate shouldn’t have an empty argument list, though it would normally be treated as a proposition (a boolean constant).

Exercise 1.10.5

```
public boolean between (real x, real y, real z)
{ // Returns true if x is between y and z.
  if (y < z) return (y < x) && (x < z);
  if (z < y) return (z < x) && (x < y);
  return false; }
```

- Exercise 1.10.6 (a)  $S(b, x) \leftrightarrow \neg S(x, x)$   
 (b) If we substitute  $b$  for  $x$  in the statement of part (a), we get  $S(b, b) \leftrightarrow \neg S(b, b)$ , which is a contradiction.

- Exercise 1.10.7 (a)  $\neg(R(c) \rightarrow (F(d) \vee R(d)))$   
 (b)  $F(c) \oplus F(d)$   
 (c)  $B(c) \leftrightarrow (B(c) \vee B(d))$

Exercise 1.10.8 Statement (a) tells us that  $R(c)$  is true and  $F(d) \vee R(d)$  is false, so  $\neg F(d)$  and  $\neg R(d)$  are both true. Statement (b) that tells us that  $F(c)$  is true, since  $F(d)$  is false. Finally, we can examine statement (c) by cases. If Cardie is not black, then at least one of them is black and Duncan must be black. If Cardie is black, then neither is black, which is a contradiction. So  $B(c)$  is false and  $B(d)$  is true.

- Exercise 1.10.9 (a) The three statements are  $T(a) \leftrightarrow ((T(a) \vee T(b)) \rightarrow T(c))$ ,  $T(b) \leftrightarrow (T(c) \rightarrow \neg T(a))$ , and  $T(c) \leftrightarrow \neg(T(a) \wedge T(b) \wedge T(c))$ .  
 (b) All we have done is to rename the three boolean variables, so nothing changes.

- Exercise 1.10.10 (a)  $CharAt(w, 0, a)$   
 (b)  $CharAt(w, |w| - 1, a)$   
 (c)  $CharAt(w^R, i, a) \leftrightarrow CharAt(w, |w| - i - 1, a)$

## S.2 Exercises From Chapter 2

- Exercise 2.1.1 (a)  $\{\langle 1, x \rangle, \langle 1, y \rangle, \langle 1, z \rangle, \langle 2, x \rangle, \langle 2, y \rangle, \langle 2, z \rangle\}$   
 (b)  $\{\langle x, 1 \rangle, \langle x, 2 \rangle, \langle y, 1 \rangle, \langle y, 2 \rangle, \langle z, 1 \rangle, \langle z, 2 \rangle\}$   
 (c)  $\{\langle \text{cat}, 1 \rangle, \langle \text{cat}, 2 \rangle, \langle \text{dog}, 1 \rangle, \langle \text{dog}, 2 \rangle\}$   
 (d)  $\{\langle 1, x, \text{cat} \rangle, \langle 1, x, \text{dog} \rangle, \langle 1, y, \text{cat} \rangle, \langle 1, y, \text{dog} \rangle, \langle 1, z, \text{cat} \rangle, \langle 1, z, \text{dog} \rangle, \langle 2, x, \text{cat} \rangle, \langle 2, x, \text{dog} \rangle, \langle 2, y, \text{cat} \rangle, \langle 2, y, \text{dog} \rangle, \langle 2, z, \text{cat} \rangle, \langle 2, z, \text{dog} \rangle\}$   
 (e)  $\{\langle x, 1, x \rangle, \langle x, 1, y \rangle, \langle x, 1, z \rangle, \langle x, 2, x \rangle, \langle x, 2, y \rangle, \langle x, 2, z \rangle, \langle y, 1, x \rangle, \langle y, 1, y \rangle, \langle y, 1, z \rangle, \langle y, 2, x \rangle, \langle y, 2, y \rangle, \langle y, 2, z \rangle, \langle z, 1, x \rangle, \langle z, 1, y \rangle, \langle z, 1, z \rangle, \langle z, 2, x \rangle, \langle z, 2, y \rangle, \langle z, 2, z \rangle\}$
- Exercise 2.1.2 (a)  $Z(ch)$  meaning “ $ch$  comes before  $e$  in the alphabet”  
 (b)  $\{b, f, j, p, v\}$ , with  $z$  added if you consider  $y$  a vowel  
 (c)  $Y(ch)$  meaning “ $ch$  is the  $n$ ’th letter in the alphabet and  $n$  is divisible by 3”  
 (d)  $\{a, h, i, m, o, t, u, v, w, x, y\}$
- Exercise 2.1.3 (a)  $C(ch_1, ch_2)$  meaning “ $ch_1 = b$  and  $ch_2 \in V$ ”  
 (b)  $\{\langle b, e \rangle, \langle d, o \rangle\}$   
 (c)  $D(ch_1, ch_2)$  meaning “ $ch_1 = c$  or  $ch_1 = d$ , and  $ch_2 = e$  or  $ch_2 = u$ ”  
 (d)  $\{\langle b, a \rangle, \langle b, e \rangle, \langle b, i \rangle, \langle b, u \rangle, \langle c, o \rangle, \langle d, o \rangle\}$
- Exercise 2.1.4 (a)  $[[\text{true}, \text{false}, \text{true}, \text{false}], [\text{false}, \text{false}, \text{false}, \text{true}], [\text{false}, \text{false}, \text{false}, \text{false}]]$   
 (b)  $[[\text{true}, \text{false}, \text{true}, \text{false}], [\text{false}, \text{true}, \text{false}, \text{true}], [\text{true}, \text{false}, \text{true}, \text{false}], [\text{false}, \text{true}, \text{false}, \text{true}]]$   
 (c)  $[[\text{false}, \text{false}, \text{false}, \text{false}], [\text{false}, \text{false}, \text{false}, \text{false}], [\text{false}, \text{false}, \text{false}, \text{false}], [\text{false}, \text{false}, \text{false}, \text{false}]]$
- Exercise 2.1.5 (a)  $\{\langle x, y \rangle : (y = x) \vee (y = x + 1)\}$   
 (b)  $\emptyset$   
 (c)  $A$   
 (d)  $\{\langle x, y \rangle : (y = x) \vee (x = y + 1)\}$
- Exercise 2.1.6  $\{\langle 0, 0, 0 \rangle, \langle 0, 1, 1 \rangle, \langle 0, 2, 2 \rangle, \langle 0, 3, 3 \rangle, \langle 1, 0, 1 \rangle, \langle 1, 1, 2 \rangle, \langle 1, 2, 3 \rangle, \langle 2, 0, 2 \rangle, \langle 2, 1, 3 \rangle, \langle 3, 0, 3 \rangle\}$
- Exercise 2.1.7 (a) A 100 by 100 boolean array will do the job, and no other representation is likely to be better unless the picture has a simple description.  
 (b) A list will have one entry for each of the 14,654 words that occur in the Bible. This will require much less storage than a boolean array, which would have an entry for each of the  $26^{20}$  or about  $2(10^{28})$  elements of  $S$ . And there is no clear way to determine membership in  $R$  that would be easier or faster than looking for the word in a list.  
 (c) Here a method can easily calculate whether  $\langle x, y \rangle$  satisfies the rule.
- Exercise 2.1.8  $\langle 1, 2, 3 \rangle, \langle 1, 2, 4 \rangle, \langle 1, 2, 5 \rangle, \langle 1, 3, 4 \rangle, \langle 1, 3, 5 \rangle, \langle 1, 4, 5 \rangle, \langle 2, 3, 4 \rangle, \langle 2, 3, 5 \rangle, \langle 2, 4, 5 \rangle, \langle 3, 4, 5 \rangle$ .

Exercise 2.1.9 To specify a  $k$ -ary relation on an  $n$ -element set, we need to say for each of the  $n^k$  possible  $k$ -tuples whether it is in the relation or not. These  $n^k$  binary choices may be made in  $2^{n^k}$  possible ways. For  $2^{n^k}$  to be less than 1000,  $n^k$  itself must be less than 10. So we could have  $k = 1$  and  $n \leq 9$ ,  $k = 2$  and  $n \leq 3$ , or  $k = 3$  and  $n = 2$ .

Exercise 2.1.10 There are seven pairs in the graph:  $\langle 0, 3 \rangle$ ,  $\langle 1, 4 \rangle$ ,  $\langle 2, 5 \rangle$ ,  $\langle 3, 6 \rangle$ ,  $\langle 4, 0 \rangle$ ,  $\langle 5, 1 \rangle$ , and  $\langle 6, 2 \rangle$ . The  $7 \times 7$  boolean array would be mostly zeros, with 1 entries only in the seven locations corresponding to the pairs in the relation.

- Exercise 2.3.1 (a) For every two naturals  $x$  and  $y$ , the sum of  $x$  and  $y$  equals the sum of  $y$  and  $x$ . TRUE  
 (b) Every natural is the square of some natural. FALSE  
 (c) Every natural has a natural that is its square. TRUE  
 (d) There is a natural  $x$  such that for any natural  $y$ ,  $x^2 + 4$  is less than or equal to  $4x + y$ . TRUE (If we take  $x = 2$ , then for any natural  $y$  it is true that  $x^2 + 4 = 8$  is less than or equal to  $4x + y = 8 + y$ . This is the only value of  $x$  that works.)

- Exercise 2.3.2 (a)  $\exists w : P(w) \wedge Q(w)$   
 (b)  $\forall w : P(w) \rightarrow Q(w)$   
 (c)  $[\forall w : \exists x : (w \neq x) \wedge R(w, x)] \rightarrow [\forall y : P(y)]$   
 (d)  $Q(w) \wedge [\forall x : Q(x) \rightarrow R(w, x)]$   
 (e)  $[P(w) \wedge Q(w) \wedge R(w, ab)] \rightarrow (w = aa)$

- Exercise 2.3.3 (a) a true sentence  
 (b) a false sentence,  $ab$  is a counterexample  
 (c) a false sentence, the first square-bracketed statement is true and the second is false  
 (d)  $w$  is the only free variable, but the statement is false for all  $w$   
 (e)  $w$  is the only free variable, but the statement is true for all  $w$

- Exercise 2.3.4 (a)  $\forall w : \neg P(w) \vee \neg Q(w)$  Given any string, either it does not have exactly two letters or it does not start and end with the same letter, or both.  
 (b)  $\exists w : P(w) \wedge \neg Q(w)$  There exists a string of two letters that does not start and end with the same letter.  
 (c)  $[\forall w : \exists x : (w \neq x) \wedge R(w, x)] \wedge [\exists y : \neg P(y)]$  For every string, there is another string that starts with the same letter, and there exists a string that does not have exactly two letters.  
 (d)  $\neg Q(w) \vee [\exists x : Q(x) \wedge \neg R(w, x)]$  Either the string  $w$  does not start and end with the same letter, or there exists a string that starts and ends with the same letter that does not start with the same letter as  $w$ .  
 (e)  $P(w) \wedge Q(w) \wedge R(w, ab) \wedge (w \neq aa)$  The string  $w$  has exactly two letters, starts and ends with the same letter, and starts with the same letter as does  $ab$ , but  $w$  is not  $aa$ .

- Exercise 2.3.5 (a)  $y$  is free,  $x$  and  $z$  are bound

- (b)  $w$  is free,  $x$ ,  $y$ , and  $z$  are bound  
 (c)  $y$  is free in the first expression and bound in the second,  $x$  is bound in the first expression and free in the second, and  $z$  is free

Exercise 2.3.6 (a)  $\pi_1(P)(x)$  is true if and only if  $\exists y : P(x, y)$ .  $\pi_2(P)(y)$  is true if and only if  $\exists x : P(x, y)$ .

(b) If  $T$  is the join of  $R$  and  $S$ , then  $T(a, c)$  is true if and only if  $\exists b : R(a, b) \wedge S(b, c)$ .

Exercise 2.3.7  $(\exists x : P(n)) \rightarrow (\exists x : \forall y : P(x) \wedge (P(y) \rightarrow (x \leq y)))$

- Exercise 2.3.8
- Predecessor:  $(x < y) \wedge \neg \exists z : (x < z) \wedge (z < y)$
  - Successor:  $(y < x) \wedge \neg \exists z : (y < z) \wedge (z < x)$
  - Even number:  $\exists y : x = 2y$
  - Odd number:  $\exists y : x = 2y + 1$

Exercise 2.3.9 This statement is equivalent to  $\exists x : P(x)$ . If one element exists satisfying  $P$ , then two exist because they are allowed to be the same. And certainly if two exist, then one exists.

Exercise 2.3.10  $\forall c : \exists j : Q(c, j) \wedge A(c, j) \wedge \forall j' : A(c, j') \rightarrow (j = j')$

Exercise 2.5.1 The language  $\{a\}\Sigma^*$  is the set of all strings that begin with  $a$ , and  $\{b\}\Sigma^*$  is the set of all strings that begin with  $b$ . Similarly  $\Sigma^*\{a\}$  is the set of all strings that end in  $a$  and  $\Sigma^*\{b\}$  is the set of all strings that end in  $b$ .

- Exercise 2.5.2
- (a) The set of strings that begin with  $a$  or end with  $b$   
 (b) The set of strings that both begin with  $a$  and end with  $b$   
 (c) The set of strings that begin with  $a$  or begin with  $b$ , that is, all strings except  $\lambda$   
 (d) The set of strings that both end in  $a$  and end in  $b$ , that is, the empty set

- Exercise 2.5.3
- (a)  $\{aaaa, aaab, aaabb, aba, abb, abbb\}$   
 (b)  $\{aa, ab, abb, aaa, aab, aabb, aba, abbb\}$   
 (c)  $\{aaaaaaaa, aaaaaab, aaabaaa, aaabab, aaabbaaa, aaabbab, abaaaa, abaab, abbaaa, abbab, abbbab, abbbab\}$   
 (d)  $\{aaaaaa, aaaaaa, aaaaab, aaaba, aaabaa, aaabab, aaabba, aaabbaa, aaabbab, abaa, abaaaa, abaab, abba, abbaa, abbab, abbba, abbbba, abbbab, aaaab, aaaabb, aaba, aabb, aabb, aaaaabb, aaabb, aaabbb, abaaaa, abaaab, abaaabb, ababa, ababb, ababbb\}$

Exercise 2.5.4 For  $i = j = 2$  we may let both  $A$  and  $B$  be the language  $\{\lambda, a\}$ , so that  $AB = \{\lambda, a, aa\}$  which has size  $2 + 2 - 1 = 3$ . In general if we let  $A$  be  $\{\lambda, a, \dots, a^{i-1}\}$  and  $B$  be  $\{\lambda, a, \dots, a^{j-1}\}$  then  $AB = \{\lambda, a, \dots, a^{i+j-2}\}$  and thus  $|AB| = i + j - 1$ .

Exercise 2.5.5 The language  $\Sigma^3$  is the set of all strings with exactly three letters. The language  $\Sigma^k$  is the set of all strings with exactly  $k$  letters, and there are exactly  $2^k$  such strings.

- Exercise 2.5.6
- (a)  $\forall w : [\exists u : \exists v : A(u) \wedge B(v) \wedge (w = uv)] \leftrightarrow [\exists u : \exists v : A(v) \wedge B(u) \wedge (w = uv)]$   
 (b)  $\exists w : [\exists u : \exists v : A(u) \wedge B(v) \wedge (w = uv)] \oplus [\exists u : \exists v : A(v) \wedge B(u) \wedge (w = uv)]$

- Exercise 2.5.7 (a) There are several families of examples. If  $A = B$  and  $\lambda \in A$ , then any string in  $AB$  is also in both  $A$  and  $B$ . So we can make any string in  $(AB)^*$  by making it in  $A^*$  and then appending  $\lambda$  which is in  $B^*$ . Any string in  $A^*B^*$  is also in  $(AB)^*(AB)^*$  which is contained in  $(AB)^*$ .
- (b) If  $A = B = \{0\}$ , then  $(AB)^*$  is all strings consisting of an even number of 0's, while  $A^*B^*$  is all strings consisting only of 0's.
- Exercise 2.5.8 There must be exactly  $n^t$  strings in  $X^t$ . There are  $n$  ways to form a string by concatenating  $t$  strings from  $X$ , and each leads to a different string because there is only one way to divide a string of length  $tk$  into  $t$  strings of length  $k$ .
- Exercise 2.5.9 (a) We can write  $X$  as  $\{a\} \cup \{b\} \cup \{f\} \cup \{i\} \cup \{r\} \cup \{t\} \cup \{u\}$  and then write the given language as  $XX^4XX^4X^4$  or  $X^4X^4$ .
- (b)  $X^4X^4 \cap X^4\{f\}X^4$
- Exercise 2.5.10 Requiring each letter is just an extension of requiring  $f$  in Exercise 2.5.9 (b). We can write the language as  $X^4X^4 \cap X^4\{a\}X^4 \cap X^4\{b\}X^4 \cap X^4\{f\}X^4 \cap X^4\{i\}X^4 \cap X^4\{r\}X^4 \cap X^4\{t\}X^4 \cap X^4\{u\}X^4$ . An English word in this set of pangrams is "fruitbat". We could actually leave out the "four or more letters" condition because we can't contain each of the seven letters without following it.
- Exercise 2.6.1 (a) Let Tommy be an arbitrary trout, prove that "Tommy lives in trees", and use the Rule of Generalization to get the desired conclusion.
- (b) Use the Rule of Existence to conclude "There exists a trout who lives in trees".
- (c) Use the Rule of Specification to conclude "Tommy lives in trees".
- (d) Use the Rule of Instantiation to say "Let Tommy be a trout who lives in trees".
- Exercise 2.6.2 We first assume that  $\forall x : \forall y : P(x, y)$  is true and set out to prove  $\forall y : \forall x : P(x, y)$ . Let  $b$  be an arbitrary object of  $y$ 's type. Let  $a$  be an arbitrary object of  $x$ 's type. By Specification from the premise to  $a$ , we have that  $\forall y : P(a, y)$ . By Specification from this to  $b$ , we have that  $P(a, b)$  is true. Since  $a$  was arbitrary, by Generalization we conclude  $\forall x : P(x, b)$ . Since  $b$  was arbitrary, by Generalization we conclude  $\forall y : \forall x : P(x, y)$ , as desired. The proof in the other direction is identical, simply exchanging  $x$  with  $y$  and  $a$  with  $b$ .
- Exercise 2.6.3 We first assume that  $\exists x : \exists y : P(x, y)$  and set out to prove  $\exists y : \exists x : P(x, y)$ . (As in Exercise 2.6.2, the other direction of the proof will be identical, switching the two variables.) By Instantiation, let  $a$  be an object of  $x$ 's type such that  $\exists y : P(a, y)$ . By Instantiation, let  $b$  be an object of  $y$ 's type such that  $P(a, b)$ . By Existence, we may conclude that  $\exists x : P(x, b)$ . By Existence again, we conclude  $\exists y : \exists x : P(x, y)$ , as desired.
- Exercise 2.6.4 Assume that  $\exists u : \forall v : P(u, v)$ . Let  $b$  be an arbitrary object of the type of  $y$ . By Instantiation from the premise, let  $a$  be an object of the type of  $u$  such that  $\forall v : P(a, v)$ . By Specification from this to our  $b$ , we have that  $P(a, b)$ . By Existence, we have that  $\exists x : P(x, b)$ . Since  $b$  was arbitrary, by Generalization we conclude  $\forall y : \exists x : P(x, y)$ , as desired. The converse is not true, as shown by the examples in Subsection 2.3.2 (see also Problem 2.6.5).

Exercise 2.6.5 We first prove  $\forall x : K(x) \rightarrow B(x)$ . Let  $a$  be an arbitrary person. From Specification on the premise  $\forall x : \neg K(x)$ , we have that  $\neg K(a)$ . By Vacuous Proof, then, we have that  $K(a) \rightarrow B(a)$ . Since  $a$  was arbitrary, by Generalization we conclude  $\forall x : K(x) \rightarrow B(x)$ , as desired.

We now prove  $\forall x : K(x) \rightarrow \neg B(x)$ . We let  $a$  be arbitrary and derive  $\neg K(a)$  as above. Now Vacuous Proof gives us  $K(a) \rightarrow \neg B(a)$ , and since  $a$  was arbitrary we have  $\forall x : K(x) \rightarrow \neg B(x)$  as desired.

Exercise 2.6.6 (a)  $[\forall x : (x = c) \vee (x = d)] \wedge [\exists y : y = c] \wedge [\exists z : z = d]$ . We need all three conditions, to say both that  $c$  and  $d$  are in the type and that no other elements are in it.

- (b)
  - Let  $x$  be an arbitrary element of  $X$ .
  - By Specification from part (a), and Left Separation,  $(x = c) \vee (x = d)$ .
  - Case 1:  $x = c$ , and from the hypothesis  $P(c)$  we have  $P(x)$ .
  - Case 2:  $x \neq c$ , so  $x = d$  by Tertium Non Datur, and by the hypothesis  $P(d)$  we have  $P(x)$ .
  - By Proof By Cases, we have  $P(x)$ .
  - Since  $x$  was arbitrary, we have  $\forall x : P(x)$  by Generalization.

Exercise 2.6.7 (a)  $\forall x : \exists y : \exists z : L(x, y) \wedge L(x, z) \wedge (y \neq z)$ .

- (b)
  - By Specification from the premise, we have  $\exists y : \exists z : L(c, y) \wedge L(c, z) \wedge (y \neq z)$ .
  - By Instantiation, choose activities  $y$  and  $z$  such that  $L(c, y) \wedge L(c, z) \wedge (y \neq z)$ .
  - Case 1:  $y = b$ . Then  $z \neq b$ , and by an argument similar to Exercise 2.6.6,  $(z = r) \vee (z = s)$ .
  - In this case,  $L(c, z)$  implies  $L(c, r) \vee L(c, s)$  and we have our conclusion for case 1.
  - Case 2:  $y \neq b$ . By the same reasoning,  $(y = r) \vee (y = s)$ , and since  $L(c, y)$  we have  $L(c, r) \vee L(c, s)$ .
  - By Proof By Cases, we have our conclusion.
- (c)
  - Let  $x$  be an arbitrary dog.
  - By Specification from part (a),  $\exists y : \exists z : L(x, y) \wedge L(x, z) \wedge (y \neq z)$ .
  - By Instantiation, choose activities  $y$  and  $z$  such that  $L(x, y) \wedge L(x, z) \wedge (y \neq z)$ .
  - Case 1:  $y = b$ . Then since  $y \neq z$ , we have  $z \neq b$ , and since  $L(x, z)$  is true by Existence we have  $\exists w : L(x, w) \wedge (w \neq b)$  which is our conclusion with a renamed bound variable.
  - Case 2:  $y \neq b$ . Since  $L(x, y)$ , by Existence we have  $\exists y : L(x, y) \wedge (y \neq b)$ .
  - By Proof By Cases we have  $\exists y : L(x, y) \wedge (y \neq b)$ .
  - Since  $x$  was arbitrary we have  $\forall x : \exists y : L(x, y) \wedge (y \neq b)$  by Generalization.

Exercise 2.6.8 (a)  $\forall x : WSD(x) \rightarrow MW(x)$  and  $\forall x : MW(x) \rightarrow W(x)$ .

- (b) By Specification, we have  $WSD(c) \rightarrow MW(c)$  and  $MW(c) \rightarrow W(c)$ . From the given premise  $WSD(c)$ , we use Modus Ponens on the first statement to get  $MW(c)$ , then Modus Ponens on the second statement to get  $W(c)$ .



- (c) The statement is  $\forall x : WSD(x) \rightarrow W(x)$ . To prove it, we let  $x$  be an arbitrary person and assume  $WSD(x)$ . We then specify the two premises to  $x$  to get  $WSD(x) \rightarrow MW(x)$  and  $MW(x) \rightarrow W(x)$ . Using Modus Ponens twice as in part (b), we get  $W(x)$  and we have proved  $WSD(x) \rightarrow W(x)$ . (We could also combine the two specified premises by Hypothetical Syllogism to get  $WSD(x) \rightarrow W(x)$ .) Since  $x$  was arbitrary, by Generalization we have proved the desired statement.

Exercise 2.6.9 (a)  $\forall x : \forall y : \exists a : L(x, a) \wedge L(y, a)$  and  $\forall x : \forall y : [\exists a : L(x, a) \wedge L(y, a)] \rightarrow (x = y)$ .

- (b)
- By Instantiation, choose two dogs  $x$  and  $x'$  such that  $x \neq x'$ .
  - By Specification on the first premise, we have  $\exists a : L(x, a) \wedge L(x', a)$ .
  - By Specification on the second premise, we have  $[\exists a : L(x, a) \wedge L(x', a)] \rightarrow (x = x')$ .
  - By Modus Ponens, we have  $x = x'$ . This contradicts the earlier derived statement  $x \neq x'$ .
- (c) The two statements are both true if the size of  $D$  is zero or one, as then the conclusion  $x = x'$  is guaranteed to be true whatever activities the dog, if it exists, likes.

Exercise 2.6.10 (a) To prove  $\forall n : E(n) \rightarrow O(n + 1)$ :

- Let  $n$  be an arbitrary natural.
- Assume  $E(n)$ , which by the definition means  $\exists k : n = 2k$ .
- Instantiate  $k$  so that  $n = 2k$ .
- By arithmetic,  $n + 1 = 2k + 1$ .
- By Existence, with  $k$  in the role of  $k$ , we have  $\exists k : n + 1 = 2k + 1$ .
- By the definition, we have  $O(n + 1)$ .
- We have completed a Direct Proof of  $E(n) \rightarrow O(n + 1)$ .
- Since  $n$  is arbitrary, by Generalization we have proved the desired statement.

(b) To prove  $\forall n : O(n) \rightarrow E(n + 1)$ :

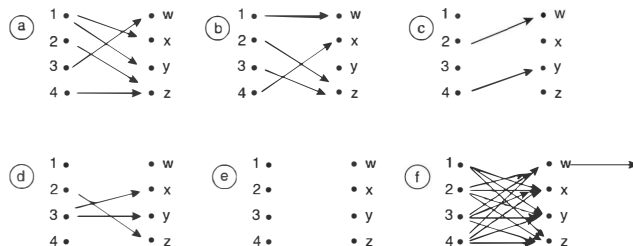
- Let  $n$  be an arbitrary natural.
- Assume  $O(n)$ , which by the definition means  $\exists k : n = 2k + 1$ .
- Instantiate  $k$  so that  $n = 2k + 1$ .
- By arithmetic,  $n + 1 = 2k + 2 = 2(k + 1)$ .
- By Existence, with  $k + 1$  in the role of  $k$ , we have  $\exists k : n + 1 = 2k$ .
- By the definition, we have  $E(n + 1)$ .
- We have completed a Direct Proof of  $O(n) \rightarrow E(n + 1)$ .
- Since  $n$  was arbitrary, by Generalization we have proved the desired statement.

Exercise 2.8.1 Let  $A = \{1, 2\}$  and  $B = \{3, 4\}$ . Define  $R$  to be the relation  $\{\langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 2, 3 \rangle\}$ . This is total but not well-defined (since 1 is mapped to two different elements). Let  $S$  be the relation  $\{\langle 1, 3 \rangle\}$ . This is well-defined but not total (since 2 is not mapped anywhere). The diagrams are in Figure S-3.



©Kendall Hunt Publishing Company

Figure S-3: Diagrams of Two Relations for Exercise 2.8.1



©Kendall Hunt Publishing Company

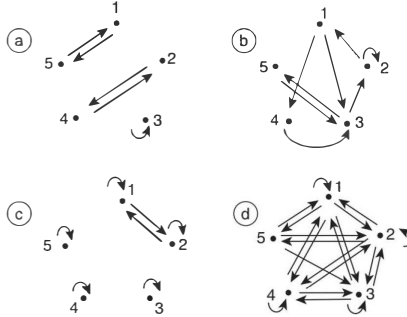
Figure S-4: Diagrams of the Relations in Exercise 2.8.3

Exercise 2.8.2 The statement was  $\neg \exists a : \exists b : \exists c : (\langle a, b \rangle \in R) \wedge (\langle a, c \rangle \in R \wedge (b \neq c))$ . If we omit the “ $b \neq c$ ” from this statement, then *no* relation is well-defined unless it is the empty relation. If any  $a$  and  $b$  exist such that  $\langle a, b \rangle \in R$ , then if we take  $b = c$  the statement is falsified.

- Exercise 2.8.3
- (a) total, but not well-defined since 1 maps to two different elements
  - (b) both total and well-defined, since each element of  $A$  is mapped to exactly one element of  $B$
  - (c) well-defined but not total as 1 and 3 are not mapped
  - (d) not total as 1 and 4 are not mapped, not well-defined as 3 is mapped to two different elements of  $B$
  - (e) well-defined but not total, no elements of  $A$  are mapped
  - (f) total but not well-defined, all elements of  $A$  are mapped to more than one element of  $B$

Diagrams are in Figure S-4.

Exercise 2.8.4 All binary relations on an empty or one-element set are *both* symmetric and antisymmetric. A binary relation a two-element set  $\{a, b\}$  is either one or the other, depending on whether the truth values of  $R(a, b)$  and  $R(b, a)$  are the same or different. But on a three-element set  $\{a, b, c\}$  we can have  $R(a, b)$  and  $R(b, a)$  both true, making  $R$  not antisymmetric, and have  $R(a, c)$  true and  $R(c, a)$  false, making  $R$  not symmetric.



©Kendall Hunt Publishing Company

Figure S-5: Diagrams of Relations for Exercise 2.8.6

Exercise 2.8.5 Since the symmetry condition cannot be violated by a loop, a symmetric relation may have loops on any or all of its vertices, or on none. Thus it could be either reflexive (all loops) or antireflexive (no loops), or neither. Since the matching arrow for an arrow from  $a$  to  $b$  goes from  $b$  to  $a$ , the matching arrow for an arrow from  $a$  to  $a$  also goes from  $a$  to  $a$ . Thus a loop is its own matching arrow, and always has a matching arrow if it exists itself.

Exercise 2.8.6 Diagrams are in Figure S-5.

- (a) symmetric, no other properties
- (b) no properties
- (c) reflexive, symmetric, transitive, not antisymmetric
- (d) not reflexive (no loop at 5), not symmetric (5 to 3, not 3 to 5), not antisymmetric (1 to 2 and 2 to 1), not transitive (5 to 3 and 3 to 4 but not 5 to 4)

Exercise 2.8.7 As we observed in Exercise 2.8.4, if we have zero, one, or two elements in our set, every reflexive relation is also transitive and we can check that each one is either symmetric, antisymmetric, or both. So we need at least three elements, and we can complete the example from Exercise 2.8.4 to solve this problem as well. We make  $R(a, b)$ ,  $R(b, a)$ ,  $R(a, c)$ , and  $R(b, c)$  true and make  $R(c, a)$  and  $R(c, b)$  false. This fails symmetry with  $a$  and  $b$  or with  $a$  and  $c$ , and fails antisymmetry with  $a$  and  $b$ . But we can check that it is transitive: in order to have  $R(x, y)$  and  $R(y, z)$  both true, we either have  $z = c$  (which makes  $R(x, z)$  true whatever  $x$  is) or have all three variables come from  $a$  and  $b$  (which also makes  $R(x, z)$  true).

- Exercise 2.8.8
- (a) Each candidate is qualified for at least one job.
  - (b) No candidate is qualified for more than one job.
  - (c) Each candidate is qualified for exactly one job. (But note that two different candidates might be qualified for the same job.)

- (d)  $Q$  must have a *subset* that is a function and also has the property that no two candidates are mapped to the same job. (This will be called being “one-to-one” in the next section.)

Exercise 2.8.9 If  $A$  is empty, then the only possibly  $R$  is also empty and is *both* reflexive and antireflexive. If  $A$  has exactly one element  $a$ , then it is reflexive if  $R(a, a)$  is true and antireflexive if it is false. But if  $A$  has two distinct elements  $a$  and  $b$ , we can make it neither by setting  $R(a, a)$  true and  $R(b, b)$  false.

- Exercise 2.8.10 (a) If  $k$  is odd, every real number has a unique  $k$ 'th root, so for any given  $x$ ,  $p(x)$  is defined and the unique possible value of  $y$  is the  $k$ 'th root of  $p(x)$ .  
 (b) With  $k = 2$  (or actually for any other positive even  $k$  as well), there are two possible values of  $y$  if  $p(x)$  is positive, one if  $p(x) = 0$ , and none if it is negative. So if  $p(x) = 0$ ,  $R_{k,p}$  is a function (the zero function. If  $p(x) = 1$ , then  $R_{k,p}$  is total but not well-defined. If  $p(x) = -1$ ,  $R_{k,p}$  is well-defined but not total.  
 (c) With  $k = 0$ , we have to have  $y^0 = p(x)$ , which is only possible if  $p(x) = 1$ . So if for any  $x$  we have  $p(x) \neq 1$ , then  $R_{0,p}$  is not total. But if  $p(x)$  is the constant function 1, then *any*  $y$  satisfies  $y^0 = p(x)$ , so  $R_{0,p}$  is not well-defined.

Exercise 2.9.1 Since  $f(g(x)) = (2x + 3) + 2 = 2x + 5$ ,  $f \circ g$  is the function taking  $x$  to  $2x + 5$ . This is an injection but not a surjection or bijection, and has no inverse. Since  $g(f(x)) = 2(x + 2) + 3 = 2x + 7$ ,  $g \circ f$  is the function taking  $x$  to  $2x + 7$ . This is also an injection but not a surjection or bijection, and has no inverse.

- Exercise 2.9.2 (a) injection, not surjection  
 (b) surjection, not injection  
 (c) bijection, inverse is itself  
 (d) bijection, inverse is itself  
 (e) neither injection nor surjection  
 (f) surjection, not injection

- Exercise 2.9.3 (a)  $f$  is employees to reals,  $g$  is employees to titles, no composition  
 (b)  $f$  is employees to titles,  $g$  is titles to reals,  $(g \circ f)(x)$  is the salary of employee  $x$   
 (c)  $f$  is employees to employees,  $g$  is employees to reals,  $(g \circ f)(x)$  is the salary of employee  $x$ 's supervisor  
 (d)  $f$  is reals to reals,  $g$  is employees to reals,  $(f \circ g)(x)$  is the tax paid by employee  $x$

- Exercise 2.9.4 (a)  $(f \circ g)(w) = aw^R$ ;  $(f \circ h)(w) = v^R$  if  $w = va$ ,  $w^R$  otherwise;  $(g \circ f)(w) = w^R a$ ;  $(g \circ h)(w) = w$  if  $w = va$ ,  $wa$  otherwise;  $(h \circ g)(w) = w$ ;  $(f \circ g \circ h)(w) = w^R$  if  $w = va$ ,  $aw^R$  otherwise.  
 (b)  $f$  is a bijection and is its own inverse,  $g$  is an injection but not a surjection and has no inverse,  $h$  is a surjection but not an injection and has no inverse.  
 (c)  $(f \circ c)$  is the identity function;  $(g \circ g)(w) = waa$ ;  $(h \circ h)(w) = v$  if  $w = vaa$ ,  $vb$  if  $w = vba$ ,  $\lambda$  if  $w = a$ ,  $w$  otherwise.

Exercise 2.9.5 We must prove that for all elements  $a$  of  $A$ ,  $((h \circ g) \circ f)(a) = (h \circ (g \circ f))(a)$ . Let  $a$  be an arbitrary element of  $A$ . Let  $b = f(a)$ ,  $c = g(b)$ , and  $d = h(c)$  – we will show that both function outputs are  $d$ . The first is defined to be  $(h \circ g)(f(a))$ , which is  $(h \circ g)(b)$ , which by definition is  $h(g(b)) = h(c) = d$ . The other function output is defined to be  $h((g \circ f)(a))$ , which is  $h(g(f(a))) = h(g(b)) = h(c) = d$ . Since  $a$  was arbitrary, the outputs of the two functions are the same for any  $a$  in  $A$  and the two functions are equal.

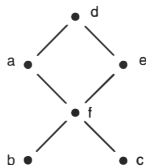
- Exercise 2.9.6 (a) We take the definition of “ $R$  is onto”,  $\forall b : \exists a : R(a, b)$ , and rewrite it in terms of  $R^{-1}$ . This gives us  $\forall b : \exists a : R^{-1}(b, a)$ , which is exactly the definition of “ $R^{-1}$  is total”. The two statements are clearly equivalent.
- (b) The definition of “ $R$  is one-to-one” is  $\forall a : \forall b : \neg b' : (R(a, b) \wedge R(a, b')) \rightarrow (b = b')$ . When we rewrite this in terms of  $R^{-1}$  we get  $\forall a : \forall b : \neg b' : (R^{-1}(b, a) \wedge R^{-1}(b', a)) \rightarrow (b = b')$ , which is exactly the definition of “ $R^{-1}$  is well-defined”. The two statements are clearly equivalent.
- (c) This follows immediately from (a) and (b).  $R$  is both onto and one-to-one if and only if  $R^{-1}$  is both total and well-defined, which is the definition of  $R^{-1}$  being a function.
- (d) Let  $A = \{a\}$ ,  $B = \{b, b'\}$ , and  $R$  be the relation from  $A$  to  $B$  given by the pairs  $\langle a, b \rangle$  and  $\langle a, b' \rangle$ . Then  $R$  is not a function because it is not well-defined, but  $R^{-1}$  is the function taking both  $b$  and  $b'$  to  $a$ .

Exercise 2.9.7 The inverse is the composition  $g^{-1} \circ f^{-1}$ . To prove this we must show that both  $g^{-1} \circ f^{-1} \circ f \circ g$  and  $f \circ g \circ g^{-1} \circ f^{-1}$  are the identity function on  $A$ . If we apply the first function to an element  $a$ , we get  $g^{-1}(f^{-1}(f(g(a))))$ . This is  $g^{-1}(g(a))$  because  $f \circ f^{-1}$  is the identity function, and this in turn is  $a$  because  $g^{-1} \circ g$  is the identity function. A similar argument applies to the second function.

Exercise 2.9.8 There are  $n^r$  possible functions, because we must make  $r$  choices, one to choose  $f(a)$  for each element of  $A$ , and each choice is thus from  $n$  possibilities. If  $r = 0$  then  $n^r = 1$ , and there is indeed one function that has no pairs at all. If  $r = 1$  then  $n^r = n$ , and we have  $n$  functions because we can map the single element of  $A$  to any of the  $n$  elements of  $B$ . If  $n = 0$  then  $n^r = 0$ , unless  $r$  is also 0, and this is correct because we cannot have a function from a non-empty set to an empty set – it would require us to map some element of  $A$  to an element of  $B$ , and the latter does not exist. If  $n = 1$  then  $n^r = 1$ , and there is exactly one function, the one that maps every element of  $A$  to the single element of  $B$ .

Exercise 2.9.9 If  $A$  has zero elements or one element, any function is an injection because we cannot violate the one-to-one condition without two different elements of  $A$ . Similarly if  $B$  has zero elements or one element, and  $A$  is non-empty, any function is a surjection because it meets the onto condition by hitting the single element if it exists. If  $A$  is empty and  $B$  is not, the unique function from  $A$  to  $B$  is not a surjection. But if both are empty, the unique function is both an injection and a surjection.

Exercise 2.9.10 Given a set  $S$ , we choose a string  $w$  such that `w.charAt(i)` is true if and only if  $i \in S$ . Given a string  $w$ , we let our set  $S$  be  $\{i : w.charAt(i)\}$ . If two sets are different, there is some element  $i$  in one and not the other, and the corresponding strings differ in the  $i$ 'th place. If two strings are different, there is a place  $i$  in which they differ, and



©Kendall Hunt Publishing Company

Figure S-6: The Hasse Diagram for Exercise 2.10.3

element  $i$  is then in one set and not the other. So both functions are injections, and are inverses of one another, so they are bijections.

**Exercise 2.10.1** Let  $a$  and  $b$  be arbitrary naturals, with  $a > 0$ . First assume that  $\exists c : b = ac$ . The natural  $b\%a$  is defined to be that natural  $r$  such that  $b = qa + r$  for some  $q$  and  $r < a$ . Since  $b = ac$ , we can take  $q = c$  and then  $r = 0$ , so  $b\%a = 0$ .

Now assume that  $b\%a \neq 0$ . By the definition of  $\%$ , there exists some natural  $q$  such that  $b = aq + 0$ . So  $b = aq$ , and by Existence  $\exists c : b = ac$ .

- Exercise 2.10.2**
- (a) not a partial order, not reflexive
  - (b) not a partial order, not antisymmetric if two different players have the same average
  - (c) is a partial order, transitive because if  $2x < y$  and  $2y < z$  then  $2x < z$ , and the cases involving equality are easy to check – not a total order because neither  $\langle 2, 3 \rangle$  or  $\langle 3, 2 \rangle$  are in it
  - (d) a total order, over  $\{a\}$  given any two different strings, the shorter is a prefix of the longer
  - (e) not a partial order, not reflexive

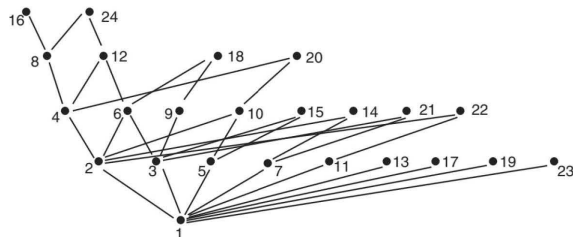
**Exercise 2.10.3** The diagram is shown in Figure S-6. The elements  $b$  and  $c$  are minimal. The element  $f$  is minimal in the partial order that remains. After that,  $a$  and  $e$  are minimal, leaving only element  $d$ .

**Exercise 2.10.4** The diagram is shown in Figure S-7.

**Exercise 2.10.5** An infinite partial order may fail to have a minimal element. For example, the negative integers form a partial order under  $\leq$  but there is no minimal element because every element  $-n$  has a smaller element  $-n - 1$ . The process of taking smaller and smaller elements need never stop because there is no bound on the number of elements.

**Exercise 2.10.6** We must check the three properties:

- Reflexive: For any pair  $\langle a, b \rangle$ ,  $T(\langle a, b \rangle, \langle a, b \rangle)$  is true because both  $R(a, a)$  and  $S(b, b)$  are true, by reflexivity of  $R$  and  $S$ .
- Antisymmetric: Suppose that  $T(\langle a, b \rangle, \langle a', b' \rangle)$  and  $T(\langle a', b' \rangle, \langle a, b \rangle)$  are both true. Then  $R(a, a')$ ,  $S(b, b')$ ,  $R(a', a)$ , and  $S(b', b)$  are all true. By antisymmetry of  $R$  and  $S$ , we must have  $a = a'$  and  $b = b'$ , which together imply  $\langle a, b \rangle = \langle a', b' \rangle$ .



©Kendall Hunt Publishing Company

Figure S-7: The Hasse Diagram for Exercise 2.10.4

- Transitive: If we have  $T(\langle a, b \rangle, \langle a', b' \rangle)$  and  $T(\langle a', b' \rangle, \langle a'', b'' \rangle)$ , we can infer  $R(a, a')$ ,  $R(a', a'')$ ,  $S(b, b')$ , and  $S(b', b'')$ . From these, by transitivity of  $R$  and  $S$ , we can infer  $R(a, a'')$  and  $S(b, b'')$ , which together imply  $T(\langle a, b \rangle, \langle a'', b'' \rangle)$ .

Exercise 2.10.7 We'll check each property for both  $P$  and  $Q$ :

- Reflexive: Let  $x$  be an arbitrary element of  $A \cup B$ . If  $x \in A$ , then  $R(x, x)$  is true, which implies both  $P(x, x)$  and  $Q(x, x)$ . If  $x \in B$ , then  $S(x, x)$  is true, which also implies both  $P(x, x)$  and  $Q(x, x)$ .
- Antisymmetric: First assume that both  $P(x, y)$  and  $P(y, x)$  are true. The elements  $x$  and  $y$  must either both be in  $A$ , in which case  $R(x, y)$  and  $R(y, x)$  together imply  $x = y$ , or both be in  $B$ , in which case  $S(x, y)$  and  $S(y, x)$  also imply  $x = y$ . If  $Q(x, y)$  and  $Q(y, x)$  are both true, then  $x$  and  $y$  must again both be in  $A$  or both be in  $B$ , because if they are in different sets only one of the two pairs can be in  $A \times B$  and neither can be in  $R$  or  $S$ . So the argument for  $P$  applies and  $x = y$ .
- Transitive: First assume that  $P(x, y)$  and  $P(x, z)$  are both true. The elements  $x$  and  $y$  must either be both in  $A$  or both in  $B$ , and similarly for  $y$  and  $z$ . So either all three are in  $A$ , in which case  $R(x, y)$  and  $R(y, z)$  imply  $R(x, z)$ , or all three are in  $B$  and the result follows from the transitivity of  $S$ .

Now suppose that  $Q(x, y)$  and  $Q(y, z)$  are both true. If all three are in  $A$  or all three are in  $B$ , the argument for  $P$  above goes through unchanged. If  $x \in A$  and  $y \in B$ , then we must have  $z \in B$  and  $Q(x, z)$  is true. Similarly, if  $y \in A$  and  $z \in B$ , we must have  $x \in A$  and  $Q(x, z)$  is again true.

Exercise 2.10.8 (a) Reflexive:  $A \subseteq A$  is always true. Antisymmetric: We know from set theory that  $A \subseteq B$  and  $B \subseteq A$  together imply  $A = B$ . Transitive: We know that  $A \subseteq B$  and  $B \subseteq C$  together imply  $A \subseteq C$ .

(b) We check the three properties:

- Reflexive:  $T(u, u)$  is true because if the  $i$ 'th character of  $u$  is 1, certainly the  $i$ 'th character of  $u$  is 1.
- Antisymmetric: If  $T(u, v)$  and  $u \neq v$ , there must be a character of  $v$  that is 1 when the matching character of  $u$  is 0. This prevents  $T(v, u)$  because it

violates the condition that if a character of  $v$  is 1, the matching character of  $u$  is 1.

- Transitive: Suppose  $T(u, v)$  and  $T(v, w)$  are both true and consider any position  $i$  of  $u$  that has a 1. The corresponding position of  $v$  must have a 1 since  $T(u, v)$  is true, and then the corresponding position of  $w$  must have a 1 since  $T(v, w)$  is true. Since  $i$  was arbitrary, we have proved  $T(u, w)$ .
- (c) Every set has a corresponding string and vice versa, as we proved in Exercise 2.9.10. The bijection  $f$  between the two preserves the ordering, in that  $S(A, B) \leftrightarrow T(f(A), f(B))$ . We will later refer to this as the two partial orders being isomorphic.

Exercise 2.10.9 (a) Let  $A = \{a, b, c\}$  and let  $R$  contain the pairs for reflexivity along with  $\langle a, c \rangle$  and  $\langle b, c \rangle$ . Then both  $a$  and  $b$  are minimal elements but neither is minimum.

- (b) Let  $x$  be a minimum element. For it to be minimal, we must be sure that no element  $y$  other than  $x$  satisfies  $P(y, x)$ . But since  $P(x, y)$  must be true, this follows from antisymmetry.
- (c) Our proof started with an arbitrary element  $x$  and followed a path downwards in the Hasse diagram. We argued that this path must reach a minimal element  $m$  since it cannot go on forever in a finite partial order. By transitivity, then, we must have  $P(m, x)$ .
- (d) If we let  $x$  be an arbitrary element, part (c) shows that  $P(m, x)$  is true, where  $m$  is the unique minimal element. Since  $x$  was arbitrary, we have proved  $\forall x : P(m, x)$  and thus that  $m$  is a minimum element.

Exercise 2.10.10 Again we check each property:

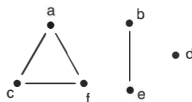
- Reflexive: If  $x = y$ , the definition immediately implies  $S(x, y)$ .
- Antisymmetric: If  $S(x, y)$  and  $S(y, x)$  are both true, and  $x \neq y$ , the definition would imply that both  $R(x, y)$  and  $R(y, x)$  are both true and false, which is impossible.
- Transitive: Suppose that  $S(x, y)$  and  $S(y, z)$  are both true. If either  $x = y$  or  $y = z$  are true, it is immediate that  $S(x, z)$  is true. Otherwise we know that  $R(x, y)$  and  $R(y, z)$  are true and that  $R(y, x)$  and  $R(z, y)$  are both false. By transitivity of  $R$ , we know that  $R(x, z)$  is true. If  $x = z$ ,  $R(x, y)$  and  $R(y, z)$  violate the antisymmetry of  $R$ . So  $x \neq z$ , and again by antisymmetry of  $R$  we know  $\neg R(z, x)$ , and we have proved  $S(x, z)$ .

Exercise 2.11.1 (a) is an e.r., classes are “strings of length  $i$ ” for each natural  $i$

- (b) not an e.r., not transitive, for example  $B(0, 10)$  and  $B(10, 20)$  are true but  $B(0, 20)$  is false
- (c) not an e.r., not transitive, for example Austen and Nabokov each wrote one in English, and Nabokov and Dostoevsky each wrote one in Russian, but Austen and Dostoevsky did not write one in the same language
- (d) not an e.r., not transitive,  $D(1, 2)$  and  $D(2, 3)$  are true but not  $D(1, 3)$

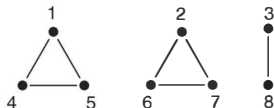
Exercise 2.11.2 First we assume  $\forall x : \forall y : R(x, y) \rightarrow R(y, x)$ , and set out to prove  $\forall x : \forall y : R(x, y) \leftrightarrow R(y, x)$ . Let  $a$  and  $b$  be arbitrary elements, and we will prove  $R(a, b) \leftrightarrow R(b, a)$ . By





©Kendall Hunt Publishing Company

Figure S-8: Diagram of the Equivalence Relation in Exercise 2.11.3



©Kendall Hunt Publishing Company

Figure S-9: Diagram of the Equivalence Relation in Exercise 2.11.4

specifying  $x$  to  $a$  and  $y$  to  $b$  in the premise, we get  $R(a, b) \rightarrow R(b, a)$ . By instead specifying  $x$  to  $b$  and  $y$  to  $a$ , we get  $R(b, a) \rightarrow R(a, b)$ , and we have the two halves of  $R(a, b) \leftrightarrow R(b, a)$ .

Conversely, if we assume  $\forall x : \forall y : R(x, y) \leftrightarrow R(y, x)$  it is easy to prove the original definition. Let  $a$  and  $b$  be arbitrary and assume  $R(a, b)$ . By specification from the premise,  $R(a, b) \leftrightarrow R(b, a)$ , and so  $R(b, a)$  is true and we have proved  $R(a, b) \rightarrow R(b, a)$ . Since  $a$  and  $b$  were arbitrary we are done by Generalization.

Exercise 2.11.3 The partition is  $\{\{a, c, f\}, \{b, e\}, \{d\}\}$ , the diagram is in Figure S-8.

Exercise 2.11.4 The equivalence relation is  $\{\langle 1, 1 \rangle, \langle 1, 4 \rangle, \langle 1, 5 \rangle, \langle 2, 2 \rangle, \langle 2, 6 \rangle, \langle 2, 7 \rangle, \langle 3, 3 \rangle, \langle 3, 8 \rangle, \langle 4, 1 \rangle, \langle 4, 4 \rangle, \langle 4, 5 \rangle, \langle 5, 1 \rangle, \langle 5, 4 \rangle, \langle 5, 5 \rangle, \langle 6, 2 \rangle, \langle 6, 6 \rangle, \langle 6, 7 \rangle, \langle 7, 2 \rangle, \langle 7, 6 \rangle, \langle 7, 7 \rangle, \langle 8, 3 \rangle, \langle 8, 8 \rangle\}$ . The diagram is shown in Figure S-9.

Exercise 2.11.5  $T$  is reflexive: Let  $x$  be arbitrary. Since  $R$  and  $S$  are both reflexive,  $R(x, x)$  and  $S(x, x)$  are both true and thus  $T(x, x)$  is true. Since  $x$  was arbitrary we have proved  $\forall x : T(x, x)$  and  $T$  is reflexive.

$T$  is symmetric: Let  $x$  and  $y$  be arbitrary and assume  $T(x, y)$ . By definition of  $T$ ,  $R(x, y)$  and  $S(x, y)$  are both true. Since  $R$  and  $S$  are symmetric,  $R(y, x)$  and  $S(y, x)$  are both true. Thus  $T(y, x)$  is true and we have proved  $T(x, y) \rightarrow T(y, x)$ . Since  $x$  and  $y$  were arbitrary we have proved  $\forall x : \forall y : T(x, y) \rightarrow T(y, x)$  and  $T$  is symmetric.

$T$  is transitive: Let  $x$ ,  $y$ , and  $z$  be arbitrary and assume  $T(x, y)$  and  $T(y, z)$ . By definition of  $T$ ,  $R(x, y)$ ,  $S(x, y)$ ,  $R(y, z)$ , and  $S(y, z)$  are all true. Since  $R$  and  $S$  are each transitive,  $R(x, z)$  and  $S(x, z)$  are true, and thus  $T(x, z)$  is true. Since  $x$ ,  $y$ , and  $z$  were arbitrary, we have proved  $\forall x : \forall y : \forall z : (T(x, y) \wedge T(y, z)) \rightarrow T(x, z)$  and  $T$  is transitive.

Exercise 2.11.6 (a) Reflexive: Every letter in  $u$  occurs in  $u$ , and vice versa. Symmetric: Switching  $u$  and  $v$  in the definition changes nothing because of the “and vice versa”. Transi-

tive: If  $R(u, v)$  and  $R(v, w)$  are both true, any letter occurring in  $u$  must occur in  $v$  because of  $R(u, v)$ , and then must occur in  $w$  because of  $R(v, w)$ . Similarly any letter in  $w$  must occur in  $v$  and therefore in  $u$ . The seven equivalence classes are  $\{aaa\}$ ,  $\{bbb\}$ ,  $\{ccc\}$ ,  $\{aab, aba, abb, baa, bab, bba\}$ ,  $\{aac, aca, acc, caa, cac, cca\}$ ,  $\{bbc, bcb, bcc, cbb, cbc, ccb\}$ , and  $\{abc, abc, bac, bca, cab, cba\}$ .

- (b) Reflexive: Certainly  $u$  and  $u$  have the same number of each letter. Symmetric: The definition does not change if we switch  $u$  and  $v$ . Transitive: If  $u$  and  $v$  have the same number of each letter, and  $v$  and  $w$  do as well, then for any letter the number occurring in  $u$  equals the number in  $v$  and thus also the number in  $w$ . There are now ten equivalence classes:  $\{aaa\}$ ,  $\{bbb\}$ ,  $\{ccc\}$ ,  $\{aab, aba, baa\}$ ,  $\{abb, bab, bba\}$ ,  $\{aac, aca, caa\}$ ,  $\{acc, cac, cca\}$ ,  $\{bbc, bcb, cbb\}$ ,  $\{bcc, cbc, ccb\}$ , and  $\{abc, acb, bac, bca, cab, cba\}$ .

Exercise 2.11.7 (a) Reflexive: If we substitute  $x$  for  $y$  all three equivalences are clearly true. Symmetric: If we switch the roles of  $x$  and  $y$  we get an equivalent statement, by the symmetry of the  $\leftrightarrow$  operator. Transitive: If  $E(x, y)$  is true, then  $B(x)$ ,  $F(x)$ , and  $R(x)$  are equivalent to  $B(y)$ ,  $F(y)$ , and  $R(y)$  respectively. If  $E(y, z)$  is true, the three values of  $B(z)$ ,  $F(z)$ , and  $R(z)$  must match those for  $y$  and thus those for  $x$ , and  $E(x, z)$  is true.

- (b) We have the set of black female retrievers, the set of black female non-retrievers, the set of black male retrievers, the set of black male non-retrievers, the set of non-black female retrievers, the set of non-black female non-retrievers, the set of non-black male retrievers, and the set of non-black male non-retrievers. (If there are any non-binary dogs, replace “male” in this list with “non-female”.) The set of all dogs is partitioned into these eight subsets.

Exercise 2.11.8 (a) We know that the  $\leftrightarrow$  operator is reflexive, symmetric, and transitive on boolean values. So for any particular setting of the variables, these three properties hold. Each of the three properties for compound propositions declares an equality of compound propositions, which means an equality of boolean values for each setting. And these equalities all follow from the properties of  $\leftrightarrow$  on booleans.

- (b) Two compound propositions are equivalent under  $E$  if and only if their truth table columns are the same. Since there are  $2^n$  boolean entries in the truth table column, there are  $2^{2^n}$  possible columns, and thus that many equivalence classes. (Note that “ $2^{2^n}$ ” means  $2^{(2^n)}$  and not  $(2^2)^n$ . Exponentiation is not associative.)

Exercise 2.11.9 We must check the three properties:

- Reflexive: The identity function has the desired property.
- Symmetric: If  $I(P, Q)$  is true, let  $f$  be a bijection such that for any  $S$  in  $P$ ,  $f(S)$  is in  $Q$ . We claim that if  $T$  is any set in  $Q$ , then  $f^{-1}(T)$  is in  $P$ . (We know that  $f^{-1}$  exists and is a bijection because  $f$  is a bijection.) The given condition on  $f$  tells us that if  $a$  and  $b$  are any two elements of  $X$ , then  $a$  and  $b$  are in the same class of  $P$  if and only if  $f(a)$  and  $f(b)$  are in the same class of  $Q$ . This is the same as saying that two elements  $c$  and  $d$  are in the same class of  $Q$  if and only if  $f^{-1}(c)$  and  $f^{-1}(d)$  are in the same class of  $P$ , which proves the claim.
- Transitive: If  $I(P, Q)$  by some function  $f$ , and  $I(Q, R)$  by some function  $g$ , then we claim that  $I(P, R)$  is true via the function  $g \circ f$ . This is because if  $a$  and  $b$  are

any two elements of  $X$ , they are in the same class of  $P$  if and only if  $f(a)$  and  $f(b)$  are in the same class of  $Q$ . It follows that this is true if and only if  $g(f(a))$  and  $g(f(b))$  are in the same class of  $R$ , proving the claim.

- Exercise 2.11.10 (a) Reflexive: Clearly  $p/q$  and  $p/q$  are the same number. Symmetric: If  $p/q$  and  $r/s$  are the same number, then clearly so are  $r/s$  and  $p/q$ . Transitive: If  $p/q$  is the same number as  $r/s$ , and  $r/s$  is the same number as  $t/u$ , then  $p/q$  is the same number as  $t/u$  by the transitivity of equality for numbers.
- (b) Every rational number  $x$  is equal to  $p/q$  for some pair of integers  $p$  and  $q$ , by definition. We will define our function  $f$  to take  $x$  to the equivalence class of the pair  $\langle p, q \rangle$ . Clearly this is a total relation, but we must check that it is well-defined, in that if  $x$  can be represented as either  $p/q$  or  $r/s$ ,  $f(x)$  is still the same equivalence class. But this is clear from the definition of  $E$ . We must also show that  $f$  is both onto and one-to-one. It is onto because every class contains at least one pair of integers  $\langle p, q \rangle$ , and this pair defines a rational number  $x = p/q$  such that  $f(x)$  is that class. And it is one-to-one because two different rational numbers must be represented by pairs that are not equivalent (by the definition of  $E$ ) and so map by  $f$  to different classes.

### S.3 Exercises From Chapter 3

Exercise 3.1.1  $15 = 3 \cdot 5$ ,  $16 = 2 \cdot 2 \cdot 2 \cdot 2$ ,  $17 = 17$ ,  $132 = 2 \cdot 2 \cdot 3 \cdot 11$ ,  $136 = 2 \cdot 2 \cdot 2 \cdot 17$ ,  $140 = 2 \cdot 2 \cdot 5 \cdot 7$ ,  
 $175 = 5 \cdot 5 \cdot 7$ ,  $179 = 179$ ,  $195 = 3 \cdot 5 \cdot 13$

Exercise 3.1.2 Remainder 0: 132. Remainder 1: 175. Remainder 2: 140. Remainder 3: 15 and 195.  
 Remainder 4: 16 and 136. Remainder 5: 17 and 179.

Exercise 3.1.3 No, for example  $(4\%3)\%2 == 1\%2 == 1$  but  $4\%(3\%2) == 4\%1 == 0$ .

Exercise 3.1.4  $x\%8$  in the set  $\{4, 5, 6, 7\}$ , or  $(x\%8)/4 == 1$

Exercise 3.1.5 After noting that 2 and 3 are prime, we list only the numbers that are congruent to 1 or to 5 modulo 6, since all the others are eliminated as multiples of 2 or 3. Boldface numbers are identified as primes, and the others have the identified larger factors in parentheses. It is enough to eliminate multiples of 5, 7, 11, and 13 since the next prime, 17, has a square greater than 200.

|           |            |            |             |
|-----------|------------|------------|-------------|
| <b>1</b>  | <b>5</b>   | <b>103</b> | <b>107</b>  |
| <b>7</b>  | <b>11</b>  | <b>109</b> | <b>113</b>  |
| <b>13</b> | <b>17</b>  | 115(5)     | 119(7)      |
| <b>19</b> | <b>23</b>  | 121(11)    | 125(5)      |
| 25(5)     | <b>29</b>  | <b>127</b> | <b>131</b>  |
| <b>31</b> | 35(5, 7)   | 133(7)     | <b>137</b>  |
| <b>37</b> | <b>41</b>  | <b>139</b> | 143(11, 13) |
| <b>43</b> | <b>47</b>  | 145(5)     | <b>149</b>  |
| 49(7)     | <b>53</b>  | <b>151</b> | 155(5)      |
| 55(5, 11) | <b>59</b>  | <b>157</b> | 161(7)      |
| <b>61</b> | 65(5, 13)  | <b>163</b> | <b>167</b>  |
| <b>67</b> | <b>71</b>  | 169(13)    | <b>173</b>  |
| <b>73</b> | 77(7, 11)  | 175(5, 7)  | <b>179</b>  |
| <b>79</b> | <b>83</b>  | <b>181</b> | 185(5)      |
| 85(5)     | <b>89</b>  | 187(11)    | <b>191</b>  |
| 91(7, 13) | 95(5)      | <b>193</b> | <b>197</b>  |
| <b>97</b> | <b>101</b> | <b>199</b> |             |

Exercise 3.1.6 The only such numbers are 2, 3, 5, 7, and 11. Any two-digit number whose digits are the same is divisible by 11. Any such three-digit number is divisible by 111, and 111 is not itself prime because it factors as  $3 \cdot 37$ .

Exercise 3.1.7 If  $n = ab$  where  $a$  and  $b$  are each greater than 1, then  $2 * n - 1 = (2^a - 1)(1 + 2^b + 2^{2b} + \dots + 2^{b(a-1)})$ , and  $2^a - 1$  is both greater than 1 and less than  $2^n - 1$ .

Exercise 3.1.8 Since  $n$  is composite,  $n = ab$  where  $1 < a, b < n$ . If  $a^2 \leq n$  then we can let  $d = a$ . If  $a^2 > n$ , we must have  $b^2 < n$  because  $a^2 b^2 = n^2$ . So in that case we can let  $d = b$ .

Exercise 3.1.9 With eight columns, four contain primes (beyond the first row) and we avoid half the numbers. With ten, four contain primes and we avoid 60% of them. With twelve, four columns contain primes and we avoid two-thirds again. But with thirty columns, only columns 1, 7, 11, 13, 17, 19, 23, and 29 contain more than one prime, so we avoid 11/15 of the numbers, more than two-thirds.

Exercise 3.1.10 If `isPrime` returns `false`, the method has found a number  $d$  that divides  $a$ . This number must be greater than 1 because  $d$  was initialized to 2 and changed only by being incremented. But  $d$  cannot be equal to  $a$ , since if it were  $d^2$  would be greater than  $a$ , and we would fall out of the `while` loop before testing whether  $d$  divides  $a$ . There are two ways to return `true`. Clearly the first is correct because 1 is defined to not be prime. To reach the second, we must increase  $d$  from 2 until  $d^2 > a$ , and not find a divisor of  $a$  in that range. But as we argued in Exercise 3.1.8, if  $a$  is composite it must have a divisor in that range.

- Exercise 3.3.1 (a)  $3 \cdot 4 + 5 \equiv 12 + 5 \equiv 5 + 5 \equiv 10 \equiv 3 \pmod{7}$   
 (b)  $2 \cdot 10 + 3 \cdot 4 \cdot (4 + 9) \equiv 20 + 12 \cdot 13 \equiv 9 + 1 \cdot 2 \equiv 11 \equiv 0 \pmod{11}$   
 (c)  $(13 - 4 \cdot 6) \cdot 2 + 3 \equiv (13 - 24) \cdot 2 + 3 \equiv -22 + 3 \equiv -19 \equiv 4 \pmod{23}$

Exercise 3.3.2 If  $a = b + ir$  and  $c = d + jr$ , where  $i$  and  $j$  are (possibly negative) integers, then  $a - c = (b - d) + (i - j)r$  and by the definition,  $a - c \equiv b - d \pmod{r}$ .

Exercise 3.3.3 (a)

$$\begin{aligned} 453 &= 1 \cdot 315 + 138 \\ 315 &= 2 \cdot 138 + 39 \\ 138 &= 3 \cdot 39 + 21 \\ 39 &= 1 \cdot 21 + 18 \\ 21 &= 1 \cdot 18 + 3 \\ 18 &= 6 \cdot 3 + 0 \end{aligned}$$

The greatest common divisor is 3.

(b)

$$\begin{aligned} 453 &= 1 \cdot 317 + 136 \\ 317 &= 2 \cdot 136 + 45 \\ 136 &= 3 \cdot 45 + 1 \\ 45 &= 45 \cdot 1 + 0 \end{aligned}$$

The greatest common divisor is 1.

(c)

$$\begin{aligned} 4096 &= 2 \cdot 1729 + 638 \\ 1729 &= 2 \cdot 638 + 453 \\ 638 &= 1 \cdot 453 + 185 \\ 453 &= 2 \cdot 185 + 83 \\ 185 &= 2 \cdot 83 + 19 \\ 83 &= 4 \cdot 19 + 7 \\ 19 &= 2 \cdot 7 + 5 \end{aligned}$$

$$\begin{aligned}
7 &= 1 \cdot 5 + 2 \\
5 &= 2 \cdot 2 + 1 \\
2 &= 2 \cdot 1 + 0
\end{aligned}$$

The greatest common divisor is 1.

(d)

$$\begin{aligned}
610 &= 1 \cdot 377 + 233 \\
377 &= 1 \cdot 233 + 144 \\
233 &= 1 \cdot 144 + 89 \\
144 &= 1 \cdot 89 + 55 \\
89 &= 1 \cdot 55 + 34 \\
55 &= 1 \cdot 34 + 21 \\
34 &= 1 \cdot 21 + 13 \\
21 &= 1 \cdot 13 + 8 \\
13 &= 1 \cdot 8 + 5 \\
8 &= 1 \cdot 5 + 3 \\
5 &= 1 \cdot 3 + 2 \\
3 &= 1 \cdot 2 + 1 \\
2 &= 2 \cdot 1 + 0
\end{aligned}$$

The greatest common divisor is 1.

(e)

$$\begin{aligned}
1367439 &= 1 \cdot 1025677 + 341762 \\
1025677 &= 3 \cdot 341762 + 391 \\
341762 &= 874 \cdot 391 + 28 \\
391 &= 13 \cdot 28 + 27 \\
28 &= 1 \cdot 27 + 1 \\
27 &= 27 \cdot 1 + 0
\end{aligned}$$

The greatest common divisor is 1.

Exercise 3.3.4 If  $r$  has an inverse modulo  $a$ , then there exists integers  $x$  and  $y$  such that  $rx + ay = 1$ . But this condition also tells us that  $a$  has an inverse modulo  $r$ , namely  $y$ .

Exercise 3.3.5 (a) The Euclidean algorithm begins with  $377 = 2 \cdot 144 + 89$  and then proceeds identically to Exercise 3.3.3 (d) above. Computing linear combinations of 377 and 144:

$$89 = 1 \cdot 377 - 2 \cdot 144$$

|    |                                 |
|----|---------------------------------|
| 55 | $-1 \cdot 377 + 3 \cdot 144$    |
| 34 | $2 \cdot 377 - 5 \cdot 144$     |
| 21 | $-3 \cdot 377 + 8 \cdot 144$    |
| 13 | $5 \cdot 377 - 13 \cdot 144$    |
| 8  | $-8 \cdot 377 + 21 \cdot 144$   |
| 5  | $13 \cdot 377 - 34 \cdot 144$   |
| 3  | $-21 \cdot 377 + 55 \cdot 144$  |
| 2  | $34 \cdot 377 - 89 \cdot 144$   |
|    | $-55 \cdot 377 + 144 \cdot 144$ |

144 is its own inverse modulo 377.

(b) Since  $511 \equiv -1 \pmod{512}$ , and  $(-1)^2 \equiv 1$ , 511 is its own inverse modulo 512.

(c) Euclidean Algorithm:

|     |                    |
|-----|--------------------|
| 512 | $13 \cdot 37 + 31$ |
| 37  | $1 \cdot 31 + 6$   |
| 31  | $5 \cdot 6 + 1$    |

Calculating linear combinations of 512 and 37:

|    |                              |
|----|------------------------------|
| 37 | $0 \cdot 512 + 1 \cdot 37$   |
| 31 | $1 \cdot 512 - 13 \cdot 37$  |
| 6  | $-1 \cdot 512 + 14 \cdot 37$ |
|    | $6 \cdot 512 - 83 \cdot 37$  |

The inverse of 37 modulo 512 is  $-83$  or 429.

(d) From the equation in (c), the inverse of 512 modulo 37 is 6.

Exercise 3.3.6 We begin by dividing  $a^c$  by  $a^b$ , and it goes in evenly with quotient  $a^{c-b}$  and remainder 0. So the algorithm terminates after one step, and the last nonzero number in the sequence is  $a^b$ , which is the greatest common divisor.

Exercise 3.3.7 (a) A method with a loop:

```
public int eA (int a, int b) { // assumes inputs non-negative
    if (a < b) {int temp = a; a = b; b = temp;}
    while (b > 0) {
        int c = a % b; a = b; b = c;
    }
    return a;
}
```

(b) A recursive method:

```

public int eA (int a, int b) { // assumes inputs non-negative
    if (a < b) return eA(b, a);
    if (b == 0) return a;
    return eA(b, a % b);}

```

- Exercise 3.3.8 (a) We begin with  $a = 4$  and  $b = 10$ . The first call is to  $a = 10$  and  $b = 4$ . The second is to  $a = 6$  and  $b = 4$ , the third is to  $a = 2$  and  $b = 4$ , the fourth to  $a = 4$  and  $b = 2$ , the fifth to  $a = 2$  and  $b = 2$ , the sixth to  $a = 0$  and  $b = 2$ , and the seventh to  $a = 2$  and  $b = 0$ . This last call terminates with output 2.
- (b) We know that  $b \leq a$  at this point in the code because we have just processed a line that does a recursive call if  $b > a$ .
- (c) When we call `simpleEA(a, b)` with  $a \geq b$ , the recursive calls strip copies of  $b$  from the first argument until it is less than  $b$ . At this point the first argument is equal to  $a \% b$  in terms of the original  $a$  and  $b$ . The following call has arguments  $b$  and  $a \% b$ , which are exactly the same as the arguments of the second call of the ordinary EA on input  $a$  and  $b$ . Thus each succeeding call of the ordinary EA is matched by a call to `simpleEA` with the same arguments. Since the original will eventually have a call to  $g$  and 0 that will return  $g$ , the new method must eventually make the same call and return the same answer.

- Exercise 3.3.9 (a) Reflexive: If we take  $h = 1$ , then  $fh = f$ , so  $D(f, f)$  is true. Transitive: If  $D(f, f')$  and  $D(f', f'')$  are both true, we know that there exist polynomials  $h$  and  $h'$  such that  $fh = f'$  and  $f'h' = f''$ . By substitution,  $fhh' = f''$  and so  $D(f, f'')$  is true. Not Antisymmetric: Let  $f = x$  and  $g = 2x$ .  $D(f, g)$  is true because we can take  $h = 2$ , and  $D(g, f)$  is true because we can take  $h = 1/2$ , but  $f \neq g$ .
- (b) After part (a), we only have to show antisymmetry. If  $D(f, g)$  and  $D(g, f)$  are both true, we know that there exist  $h$  and  $h'$  such that  $fh = g$  and  $gh' = f$ . This means that  $fhh' = f$ , and this is only possible if  $hh' = 1$ , since if it had any nonconstant term the degrees of the two sides of the equation would not match. So  $f = g$ .
- (c) Let  $c$  be the coefficient of the highest-order term of  $p$ , which must be nonzero because  $p$  is nonzero. Then let  $m = p/c$ :  $D(p, m)$  is true because we can take  $h = 1/c$ , and  $D(m, p)$  is true because we can take  $h = c$ .

Exercise 3.3.10 We are given that  $a = cm + dn$  and  $b = em + fn$ , where  $c, d, e$ , and  $f$  are integers. Since  $r = a - qb$ , we can calculate  $r = (cm + dn) - q(em + fn) = (c - qe)m + (d - qf)n$ , and we have expressed  $r$  as an integer linear combination of  $m$  and  $n$ .

Exercise 3.4.1 Suppose that  $b = c \cdot d$  where  $c$  is prime,  $2 \leq b \leq a$ , and  $b$  divides  $z$ . Then  $c$  divides  $z$  as well, since  $c \cdot (d \cdot (z/b)) = z$ .

Exercise 3.4.2

$$\begin{aligned}
 1! + 1 &= 2 \\
 2! + 1 &= 3 \\
 3! + 1 &= 7
 \end{aligned}$$



$$\begin{aligned}
4! + 1 &= 25 = 5 \cdot 5 \\
5! + 1 &= 121 = 11 \cdot 11 \\
6! + 1 &= 721 = 7 \cdot 103 \\
7! + 1 &= 5041 = 71 \cdot 71 \\
8! + 1 &= 40321 = 61 \cdot 661 \\
9! + 1 &= 362881 = 19 \cdot 71 \cdot 269 \\
10! + 1 &= 3628801 = 11 \cdot 329891
\end{aligned}$$

Exercise 3.4.3  $f(7) = 211$  which is prime.  $f(11) = 2311$  which is prime. But  $f(13) = 30031 = 59 \cdot 509$  and is thus composite.

Exercise 3.4.4 Let  $S$  be arbitrary, assume  $\forall x : (x \in S) \rightarrow (x > 1)$ , and let  $n$  be one plus the product, for all  $i \in S$ , of  $i$ . Then let  $x$  be an arbitrary element of  $S$ . Since  $n - 1$  is  $x$  times the product of the other elements of  $S$ ,  $x$  divides  $n - 1$  and  $n\%x = 1$ . Since  $x$  was arbitrary, this holds for all elements of  $S$ . Since  $S$  was arbitrary, we are done.

Exercise 3.4.5 If  $a$  and  $b$  are not relatively prime, then they have some common factor  $c$  with  $c > 1$ . Since  $c$  divides  $a$  and  $b$ , it divides any linear combination of  $a$  and  $b$  including all elements of the arithmetic progression. The only prime number that could be divisible by  $c$  is  $c$  itself, if it happens to be prime. So the arithmetic progression contains at most one prime, not infinitely many.

Exercise 3.4.6 (a) For any  $n$ , 1 is a perfect square. For 3 and 4, there are no others. For 5, there is also 4. For 6, there is also 4. For 7, there are 2 and 4. For 8, there is also 4. For 9, there are 4 and 7. For 10, there are 4, 6, and 9. For 11, there are 3, 4, 5, and 9. For 12, there are 4 and 9. For 13, there are 3, 4, 9, 10, and 12. For 14 there are 2, 4, 7, 8, 9, and 11. For 15, there are 4, 6, 9, and 10.

(b) If  $a = c^2$  and  $b = d^2$ , with both equations taken modulo  $n$ , then  $ab = (cd)^2$ .

Exercise 3.4.7 If  $n$  is odd, we can pair each nonzero number  $a$  with  $-a$ , and note that  $a$  and  $-a$  have the same square modulo  $n$ . Since each perfect square can be made by squaring two different numbers, and there are only  $n - 1$  nonzero numbers available, there can be at most  $(n - 1)/2$  perfect squares.

Exercise 3.4.8 For  $p = 2$ ,  $p - 1 = 1^2$ . The  $4k + 1$  primes are 5, where  $p - 1 = 2^2$ , 13, where  $p - 1 = 5^2$ , and 17, where  $p - 1 = 4^2$ . The other primes are 3, 7, 11, and 19. For the first three, we listed the perfect squares in the solution to Exercise 3.4.6 and  $p - 1$  was not included. For  $p = 19$ , we can check the squares of the numbers from 1 through 9 are verify that none are 18.

Exercise 3.4.9 The  $6n + 1$  primes greater than 3 are 7 (where  $-3 = 2^2$ ), 13 (where  $-3 = 6^2$ ), and 19 (where  $-3 = 4^2$ ). The other primes are 5 (where 2 is not a perfect square), 11 (where 8 is not a perfect square), and 17, where 14 is not a perfect square. We can verify this last claim by squaring all the numbers from 1 through 8, modulo 17, and never getting 14.

- Exercise 3.4.10 (a) If we take a prime number, which must be at least 2, and raise it to a power greater than  $n$ , we will get a number larger than  $2^n$ . If we then multiply by another positive number, it can only get bigger still.
- (b) This is just the uniqueness statement of the Fundamental Theorem of Arithmetic, which we will prove in Section 3.6.
- Exercise 3.5.1 Let  $i$  be arbitrary with  $2 < i \leq k$  and suppose that some number  $c$  divides both  $m_1 m_2$  and  $m_i$ . Without loss of generality, let  $c$  be prime (by replacing the original  $c$  with one of its prime factors if necessary). Since the prime factors of  $m_1 m_2$  are those primes that divide either  $m_1$  or  $m_2$ ,  $c$  must divide either  $m_1$  or  $m_2$  or both. (Formally, this step requires the Atomicity Lemma to be proved in Section 3.6.) If  $c$  divides  $m_1$ , then  $m_1$  is not relatively prime to  $m_i$ , and similarly for  $m_2$ .
- Exercise 3.5.2 This will be a congruence mod  $M = 11 \cdot 12 \cdot 13 = 1716$ . The three numbers  $M/m_i$  are 156, 143, and 132, and reducing each of these modulo  $m_i$  we get  $156 \equiv 2 \pmod{11}$ ,  $143 \equiv -1 \pmod{12}$ , and  $132 \equiv 2 \pmod{13}$ . Thus the three numbers  $n_i$  are the inverses of each  $M/m_i$  modulo  $m_i$ , or 6,  $-1$ , and 7. This gives us a  $c$  of  $9 \cdot 6 \cdot 156 + 6 \cdot (-1) \cdot 143 + 3 \cdot 7 \cdot 132 = 10338$ . So our congruence is  $x \equiv 10338 \equiv 42 \pmod{1716}$ .
- Exercise 3.5.3 (a) If there is a solution  $x$ , it must be odd, so let  $x = 2y + 1$  and notice that  $y$  must then satisfy  $y \equiv 2 \pmod{3}$ ,  $y \equiv 3 \pmod{4}$ , and  $y \equiv 1 \pmod{5}$ . These three bases are pairwise relatively prime, so there is a unique solution for  $y$  modulo  $3 \cdot 4 \cdot 5 = 60$ . We can find the solution most easily by trying  $y$ 's with  $y \equiv 1 \pmod{5}$  until we find a solution to the other two congruences: 1 and 6 fail but 11 works. So we know the three original congruences are solved if and only if  $y \equiv 11 \pmod{60}$ , which is true if and only if  $x \equiv 23 \pmod{120}$ .
- (b) Again letting  $x = 2y + 1$ , we get  $y \equiv 5 \pmod{6}$  from the first congruence and  $y \equiv 2 \pmod{8}$  from the third. The first requires  $y$  to be odd and the second requires  $y$  to be even, so there can be no common solution.
- (c) The least common multiple of the three bases is 180, so if there is any solution it will be a congruence modulo 180. The second and third congruences force  $x$  to be even, and the first and third force  $x \equiv 1 \pmod{3}$ , so we know that  $x \equiv 4 \pmod{6}$  for any solution. Of the numbers with last digit 4 up to 180, only 4, 34, 64, 94, 124, and 154 satisfy this modulo 6 condition. Of these, only 34 and 124 satisfy the modulo 9 condition (as we can discover from the sum-of-digits test from Excursion 3.2). Then 34 satisfies the modulo 12 condition and 124 does not, so we have found that the single congruence  $x \equiv 34 \pmod{180}$  exactly describes the solutions.
- Exercise 3.5.4 Representing numbers by their sequence of three residues, we have that  $x = \langle 4, 3, 7 \rangle$  and  $y = \langle 2, 1, 5 \rangle$ , so that  $xy = \langle 8, 3, 35 \rangle$  or  $\langle 1, 3, 8 \rangle$ . As it happens, this sequence of residues is its own inverse, since it squares to  $\langle 1, 1, 1 \rangle$ . It remains then, to find the single congruence modulo  $M = 7 \cdot 8 \cdot 9 = 504$  that characterizes the residue sequence  $\langle 1, 3, 8 \rangle$ . The residues of  $M/m_i$  are  $72\%7 = 2$ ,  $63\%8 = 7$ , and  $56\%9 = 2$ , so  $n_1 = 4$ ,  $n_2 = 7$ , and  $n_3 = 5$ . Thus  $c = 1 \cdot 4 \cdot 72 + 3 \cdot 7 \cdot 63 + 8 \cdot 5 \cdot 56 = 3851$ . So the solution is  $x \equiv 3851 \equiv 323 \pmod{504}$ .
- Exercise 3.5.5 The triples  $\langle 2, 3, 4 \rangle$  and  $\langle 1, 2, 4 \rangle$  are both “relatively prime” by this definition without being pairwise relatively prime.

- Exercise 3.5.6 If  $x$  is the unknown number of soldiers, we have found that  $x \equiv 6 \pmod{7}$ ,  $x \equiv 7 \pmod{8}$ , and  $x \equiv 3 \pmod{9}$ . By the CRT, this implies that  $x \equiv c \pmod{504}$  for some  $c$  that we can calculate, using some of the work from the solution to Exercise 3.5.4 above. We have that  $c$  is congruent to  $6 \cdot 4 \cdot 72 + 7 \cdot 7 \cdot 63 + 3 \cdot 5 \cdot 56 = 1728 + 3087 + 840 = 5655$ , and  $5655 \equiv 111 \pmod{504}$ . So the actual number of soldiers must be 111, 615, 1119, 1624, or larger, and the information that there are “about a thousand” tells us that 1119 is the answer.
- Exercise 3.5.7 Using the Extended Euclidean Algorithm, we compute a series of linear combinations of 51 and 32 ending in  $1 = -5 \cdot 51 + 8 \cdot 32$ . This means I could transfer \$100 by giving Rabbit 800 Twitcoins and receiving 500 Batcoins in return. There are other solutions transferring fewer coins: I could give Rabbit  $800 - 51t$  Twitcoins and get back  $500 - 32t$  Batcoins, for any integer  $t$ . Taking  $t$  to be 16, we get a solution where Rabbit gives me 16 Twitcoins worth \$512 and I give him 12 Batcoins worth \$612.
- Exercise 3.5.8 Cordelia and Goneril could apply the Simple Form of the CRT to their information and determine  $x \pmod{97 \cdot 115}$ , which is good enough since  $97 \cdot 115 > 10000$ . Similarly, Cordelia and Regan could determine  $x \pmod{97 \cdot 119}$ , and Goneril and Regan could determine  $x \pmod{115 \cdot 119}$ , and either of these remainders is good enough to determine the exact value of  $x$ . For this to work, of course, the three moduli must be pairwise relatively prime, which they are: 97 is prime,  $115 = 5 \cdot 23$ , and  $119 = 7 \cdot 17$ .
- Exercise 3.5.9 The function is one-to-one because the CRT tells us that there is only one number in that range that has given remainders mod  $p$  and mod  $q$ . It is onto because we know that the pair of congruences does have one solution in that range, so for any pair there must be an  $x$  mapping to it.
- Exercise 3.5.10 If  $n = px$  for some  $x$ , and  $b$  is the inverse of  $a$  modulo  $n$ , then  $ab \equiv 1 \pmod{n}$  which means that  $ab \equiv 1 \pmod{p}$  as well, and  $b$  is an inverse of  $a$  modulo  $p$ . For the other direction, we assume that  $a$  has an inverse modulo every prime dividing  $n$ . We can infer that  $a$  also has an inverse modulo every prime power dividing  $n$ , since by the Inverse Theorem a number has an inverse modulo  $p^e$  if and only if it is relatively prime to  $p^e$ , which is true if and only if it is relatively prime to  $p$ , which is true if and only if it has an inverse modulo  $p$ . We then write  $n$  as a product of prime powers that are pairwise relatively prime, and use the CRT to get a number  $b$  that is congruent, modulo each prime power  $p^e$ , to the inverse of  $a$  modulo  $p^e$ . Then  $ab$  is congruent to 1 modulo each of the prime powers, which by the CRT means that it is congruent to 1 modulo  $n$ .
- Exercise 3.6.1 Assume that  $p_1 p_2 \dots p_r = q_1 q_2 \dots q_s$ , where all the  $p$ 's and  $q$ 's are prime and the  $p$ 's are in order. We will prove that  $r = s$  and that the  $q$ 's can be rearranged to be identical to the sequence of  $p$ 's. First consider  $p_1$ . By atomicity, it must divide one of the  $q$ 's, and since both it and that  $q$  are prime, it must equal that  $q$ . Rearrange the  $q$ 's so that  $q_1 = p_1$ . Then consider  $p_2$ , which must divide  $q_2 q_3 \dots q_s$  since  $p_2 \dots p_r = q_2 \dots q_s$  by cancellation from the original equation. Again using atomicity and primality, there must be a  $q$  that equals  $p_2$ , and we may rearrange to make this  $q_2$ . We continue this process to get  $q_3, \dots, q_r$  equal to  $p_3, \dots, p_r$  respectively. There cannot be any more  $q$ 's because by cancellation, the product of any remaining  $q$ 's is 1.

Exercise 3.6.2 If we sort both lists of primes using any of the algorithms from a data structures class, the two lists will become identical if and only if they originally had the same number of each prime. Note that we must use a sorting algorithm that allows for two or more elements in the list to be equal.

Exercise 3.6.3 We are given that  $ad = bd$  and that  $d > 0$ . Assume that  $a$  and  $b$  are different – without loss of generality assume  $a = b + c$  with  $c > 0$ . Then by the distributive law,  $ad + cd = bd$ , and we know that  $cd > 0$  because  $c$  and  $d$  are both positive. This means that  $ad \neq bd$ , contradicting the hypothesis. So if  $ad = bd$ ,  $a \neq b$  is impossible. We have used only the distributive law and the fact that the product of two positive numbers is positive.

Exercise 3.6.4 Let  $a$  and  $b$  be arbitrary. Assume  $D(a, b) \wedge P(a) \wedge P(b)$ . By the definition of primality, we know that  $a > 1$ ,  $\forall c : D(c, a) \rightarrow ((c = 1) \vee (c = a))$ ,  $b > 1$ , and  $\forall c : D(c, b) \rightarrow ((c = 1) \vee (c = b))$ . Specifying the last statement to  $a$ , we get  $D(a, b) \rightarrow ((a = 1) \vee (a = b))$ . Since  $D(a, b)$  is true, we have  $(a = 1) \vee (a = b)$  by modus ponens. Since  $a = 1$  is ruled out,  $a = b$  must be true.

Exercise 3.6.5 Suppose  $x$  is a positive rational number, so that  $x = a/b$  where  $a$  and  $b$  are positive naturals. By the Fundamental Theorem of Arithmetic, we know that  $a = p_1 \dots p_r$  and  $b = q_1 \dots q_s$  where the  $p$ 's and  $q$ 's are prime. Thus  $x$  is equal to  $p_1 p_2 \dots p_r (1/q_1)(1/q_2) \dots (1/q_s)$ , and thus has at least one factorization into primes and inverse primes. The factorization is not unique, as for example  $2(1/2)$  and  $3(1/3)$  are both factorizations of 1. However, any positive rational number has a unique representation in lowest terms, as  $a/b$  where  $a$  and  $b$  are relatively prime. In this case the unique factorization of  $a$  and  $b$  into primes gives a distinctive factorization of  $x$  into primes and inverse primes – it is the only such factorization that does not contain both a prime and its inverse.

Exercise 3.6.6 (a) If  $x = a + b\sqrt{r}$  and  $y = c + d\sqrt{r}$ ,  $x + y = (a + c) + (b + d)\sqrt{r}$  and  $xy = (ac + rbd) + (ad + bc)\sqrt{r}$  by simple calculation. By the closure of the integers under addition and multiplication (including multiplication by  $r$ ), both numbers have the required form.

(b) From part (a) above,  $n(xy) = (ac + rbd)^2 - r(ad + bc)^2 = a^2c^2 + 2rabcd + r^2b^2d^2 - ra^2d^2 - 2rabcd - rb^2c^2$ . Cancelling the two  $2rabcd$  terms, this equals  $n(x)n(y) = (a^2 - rb^2)(c^2 - rd^2)$ .

Exercise 3.6.7 (a) If  $Y \subseteq Z$ , we can show  $D(Y, Z)$  by taking  $W = Z$ . If  $Y \subseteq Z$  is false, there is an element that is in  $Y$  but not in  $Z$ . No matter what  $W$  we union with  $Y$ , the result will still contain that element and cannot equal  $Z$ , so  $D(Y, Z)$  is false.

(b) A set is prime if and only if it is a **singleton set**, with exactly one element. Clearly any subset of a singleton set  $Y$  is either  $Y$  or  $\emptyset$ , so  $X$  is prime. Empty sets are not prime by the definition, and a set with more than one element is not prime because it has a nonempty proper subset  $Y$ , which satisfies  $D(Z, Y)$ ,  $Z \neq \emptyset$ , and  $Z \neq Y$ .

(c) If we make a separate singleton set for each element of  $Y$ , then each of these sets is prime and their union is  $Y$ . If a union of singleton sets is  $Y$ , we must have a singleton set for each element of  $Y$ , so the union must be the one that we gave.

- (d) “If  $Y$  is prime and  $D(Y, Z \cup Z')$ , then either  $D(Y, Z)$  or  $D(Y, Z')$  (or both)”.  
 Proof: Since  $Y$  is prime it equals  $\{a\}$  for some element  $a$ . Since  $Y \subseteq Z \cup Z'$ ,  $a$  must be a member of  $Z \cup Z'$ . So either  $a \in Z$  which implies  $D(Y, Z)$ , or  $a \in Z'$  which implies  $D(Y, Z')$ .
- (e) If  $Y$  is non-empty and not prime, it contains at least two distinct elements  $a$  and  $b$ . If we let  $Z = \{a\}$  and  $Z' = Y \setminus \{a\}$ , then both are nonempty (since  $b \in Z'$ ),  $D(Y, Z \cup Z')$  is true, but  $D(Y, Z)$  and  $D(Y, Z')$  are both false.

Exercise 3.6.8 Since every nonzero number  $a$  has an inverse  $a^{-1}$ , every nonzero  $a$  divides every nonzero  $b$  because  $a(a^{-1}b) = b$  and we can take  $z$  to be  $a^{-1}b$ . Every nonzero number other than 1 thus has a divisor that is neither 1 nor itself, so it is composite and there are no prime numbers at all (since we would still define 0 and 1 to not be prime). Thus 1 is the only nonzero number that has a factorization into primes, and this factorization (the empty one) is unique. So if  $p = 2$ , every nonzero number (namely, 1) has a unique factorization into primes, but this is not true for any other prime.

Exercise 3.8.9 An ordinary prime number that is less than  $t$  is still prime, and an ordinary composite number that is less than  $t$  is still composite. The number  $t$  has a proper factor is is thus composite, except in the special case of  $t = 2$ , because  $2 \times t - 1 = t$ . Numbers less than  $t$  still factor uniquely, but as long as  $t > 1$ ,  $t$  has multiple factorizations including “ $t$ ”, “ $t \times t$ ”, “ $t \times t \times t$ ”, and so forth. So unique factorization holds only for  $t = 0$  and  $t = 1$ , where there are no primes at all.

Exercise 3.8.10

```
public boolean kenkenNumber (long n) {
    while (n % 2 == 0) n /= 2;
    while (n % 3 == 0) n /= 3;
    while (n % 5 == 0) n /= 5;
    while (n % 7 == 0) n /= 7;
    return (n == 1);}
```

Any natural is a Kenken number if and only if its prime factorization includes only the one-digit primes. When we have removed all the one-digit primes from the factorization, we are left with 1 if and only if the original  $n$  was a Kenken number. This code will run quickly on any `long` argument, since it can have at most 63 prime factors and thus there will be at most 63 divisions.

Exercise 3.8.1 The predicate  $C(x, y)$  means that  $x \equiv y \pmod{r}$ .  $C$  is reflexive:  $C(x, x)$  is true because  $x \equiv x \pmod{r}$  ( $r$  divides  $x - x = 0$ ).  $C$  is symmetric: If  $C(x, y)$  is true, then  $y \equiv x \pmod{r}$  as well (because  $r$  divides  $y - x$  if it divides  $x - y$ ) and thus  $C(y, x)$ .  $C$  is transitive: If  $C(x, y)$  and  $C(y, z)$ , then  $r$  divides both  $x - y$  and  $x - z$ . So it also divides  $(x - y) + (y - z) = x - z$ , and thus  $C(x, z)$  is true.

Exercise 3.8.2 Both these facts follow from the result that for any natural  $x$  and any positive natural  $r$ , there exist naturals  $q$  and  $a$  such that  $x = qr + a$  and  $a < r$ . (This is true because repeated subtraction of  $r$  from  $x$  will eventually reach  $a$ , after  $q$  subtractions.) Clearly from this result,  $x$  is congruent to  $a$  which is less than  $r$ . If  $a$  and  $b$  are both less than  $r$ , and  $a \neq b$ , then the integer  $a - b$  is not equal to 0 and is too small in absolute value to equal  $r$  or  $-r$ , so  $r$  cannot divide  $a - b$  and thus  $a$  and  $b$  are not congruent modulo  $r$ .

Exercise 3.8.3 (a)  $R_1$  is not reflexive, is symmetric, is not transitive ( $R(2, 3)$  and  $R(3, 4)$  are true but not  $R(2, 4)$ ).

(b)  $R_2$  is always true, and thus is an equivalence relation. (We could let  $z = xy$ .)

(c)  $R_3$  is clearly reflexive and symmetric, and with a little more work we can see that it is transitive. Suppose that  $x = a^i$ ,  $y = a^j$ ,  $y = b^k$ , and  $z = b^\ell$ . Let  $m$  be the least common multiple of  $j$  and  $k$ . Let  $p$  be any prime number that divides  $y$ . By unique factorization,  $p$  must also divide both  $a$  and  $b$ . Thus the power of  $p$  that divides  $y$  must be divisible by both  $j$  and  $k$ , and thus by  $m$ . Since this holds for all prime divisors of  $y$ , there exists a number  $c$  such that  $c^m = y$ . But then since  $a$  and  $b$  are both powers of  $c$ , so are  $x$  and  $z$  and  $R_3(x, z)$  is true.

(d)  $R_4$  is reflexive and symmetric but not transitive. For example,  $R_4(2, 6)$  and  $R_4(6, 3)$  are both true but  $R_4(2, 3)$  is false.

(e)  $R_5(x, y)$  is only true if  $x = y$ , so it is the identity relation which is an equivalence relation.

Exercise 3.8.4 (a)  $SPD$  is reflexive because for any prime, clearly  $D(p, x) \leftrightarrow D(p, x)$ . It is symmetric because if  $D(p, x) \leftrightarrow D(p, y)$  for any prime,  $D(p, y) \leftrightarrow D(p, x)$  for any prime as well. For transitivity, assume  $\forall p : D(p, x) \leftrightarrow D(p, y)$  and  $\forall q : D(q, y) \leftrightarrow D(q, z)$ , where the variables  $p$  and  $q$  range only over primes. Let  $r$  be an arbitrary prime. Then  $D(r, x) \leftrightarrow D(r, y)$  and  $D(r, y) \leftrightarrow D(r, z)$  by specification, and  $D(r, x) \leftrightarrow D(r, z)$  follows. Since  $r$  was arbitrary, we have proved  $\forall r : D(r, x) \leftrightarrow D(r, z)$  and thus  $SPD(x, z)$ .

(b) The numbers that are powers of 2 (other than 1) times powers of 3 (other than 1): 6, 12, 18, 24, 36, 48, 54, 72.

(c) For any set of primes  $\{p_1, \dots, p_k\}$ , we have an equivalence class consisting of all numbers of the form  $p_1^{i_1} p_2^{i_2} \dots p_k^{i_k}$ , where each of the  $i$ 's is a positive natural.

Exercise 3.8.5 We know that addition and multiplication in  $\mathbf{Z}_r$  are both commutative and associative, and the rules for adding and multiplying polynomials make this still true in  $\mathbf{Z}_r[x]$ , along with the distributive law. The additive identity is 0 and satisfies  $0p = p0 = 0$  for any polynomial. The multiplicative identity is 1. Finally, we have additive inverses because we have a number  $-1$  that we can multiply by any polynomial  $p$  to get a  $q$  such that  $p + q = 0$ .

Exercise 3.8.6 (a) Let  $G$  and  $H$  be the groups, let  $g$  be a generator of  $G$ , and let  $h$  be a generator of  $H$ . Our isomorphism  $f$  will take the identity of  $G$  to the identity of  $H$ , and take  $g$  to  $h$ . Because it obeys the rule  $f(xy) = f(x)f(y)$ , it must take  $g^i$  to  $h^i$ , for every natural  $i$ . And this completely defines the function, because every element of  $G$  is equal to  $g^i$  for some  $i$  (including the identity, which is  $g^0$ ). The function is onto, because every element of  $H$  is equal to  $h^i$  for some  $i$ . We should also make sure that no function value is multiply defined – if some element is equal to both  $g^i$  and  $g^j$ , then  $g^{i-j}$  must be the identity, and  $i - j$  is a multiple of the order  $n$  of the group.

(b) Let one group be  $\mathbf{Z}_9$  and the other be  $\mathbf{Z}_3 \times \mathbf{Z}_3$ . Both are clearly abelian groups with nine elements, but in the second group  $x + x + x$  is the identity for every element  $x$ , where this is not true in the first group.

Exercise 3.8.7 The ring  $\mathbf{Z}_m$  has zero divisors if and only if  $m$  is composite, because two nonzero numbers can multiply to  $m$  if and only if that is the case.

Exercise 3.8.8 If  $x$  and  $y$  were both additive identities,  $x + y$  would have to be equal both to  $x$  and to  $y$ . Similarly, if they were both multiplicative identities,  $xy$  would have to be equal both to  $x$  and to  $y$ . Either is only possible if  $x = y$ .

Exercise 3.8.9 The ring rules for addition alone are satisfied because  $A$  is an abelian group. The multiplication is clearly commutative from the definition, and 1 is a multiplicative identity. The multiplication is associative because we can consider  $(xy)z$  and  $x(yz)$  depending on how many of the elements  $x$ ,  $y$ , and  $z$  are 1. If there are none, both products are 0. If there is one, both products equal the product of the other two elements. If there are two, both products are equal to the third element, and if all three are 1 then so are both products.

If we apply this construction to  $A = \mathbf{Z}_3$ , we get a multiplication that is equal to the ordinary multiplication in  $\mathbf{Z}_3$  except that now  $2 \times 2$  is 0 rather than 1. So  $2 \times (1 + 1)$  is 0, while  $(2 \times 1) + (2 \times 1)$  is 1, and the distributive law fails.

Exercise 3.8.10

| +   | 0   | 1   | x   | x+1 | *   | 0 | 1   | x | x+1 |
|-----|-----|-----|-----|-----|-----|---|-----|---|-----|
| 0   | 0   | 1   | x   | x+1 | 0   | 0 | 0   | 0 | 0   |
| 1   | 1   | 0   | x+1 | x   | 1   | 0 | 1   | x | x+1 |
| x   | x   | x+1 | 0   | 1   | x   | 0 | x   | x | 0   |
| x+1 | x+1 | x   | 1   | 0   | x+1 | 0 | x+1 | 0 | x+1 |

We can see from the tables that both operations are commutative, and that 0 and 1 are the two identities. The associative and distributive properties are harder to verify by brute force, but both hold because they hold in  $\mathbf{Z}_2[x]$ , and the only change we have made is to map every term  $x^i$  to just  $x$ . The two sides of the laws in question will map to two polynomials in  $\mathbf{Z}_2[x]$  that are equal, and they will not become unequal when we substitute  $x$  for each  $x^i$ . The ring we have constructed is not isomorphic to  $\mathbf{Z}_4$  because it obeys the rule  $y + y = 0$  for each element  $y$ . To get an isomorphism with  $\mathbf{Z}_2 \times \mathbf{Z}_2$ , we must map 0 to 0 and 1 to  $\langle 1, 1 \rangle$ , as the latter is the multiplicative identity of  $\mathbf{Z}_2 \times \mathbf{Z}_2$ . If we map  $x$  to either  $\langle 0, 1 \rangle$  or  $\langle 1, 0 \rangle$ , then we must map  $x + 1$  to the other to make the addition work. Then each of these two elements multiplies with 1 or itself to get itself, and with 0 or the other one to get 0, and thus the isomorphism works.

Exercise 3.9.1 We know that  $\gcd(x, r) = \gcd(y, r) = 1$ , and we must show that  $\gcd(xy, r) = 1$ . If any number greater than divided both  $xy$  and  $r$ , then some prime  $p$  would do so, and then  $p$  would divide either  $x$  or  $y$  by atomicity, contradicting one of the two assumptions. Multiplication in  $\mathbf{Z}_r^*$  is associative because multiplication in  $\mathbf{Z}_r$  is. The identity is 1 (which is clearly in  $\mathbf{Z}_r$ ). If  $a \in \mathbf{Z}_r^*$ , the inverse theorem tells us that  $a$  has an inverse modulo  $r$ , a number  $b$  such that  $ab \equiv 1 \pmod{r}$ .

Exercise 3.9.2  $R$  is reflexive because  $b = ba^0$ . It is symmetric because if  $b = ca^i$ , then  $c = ba^{r-1-i}$  (since  $a^{r-1} = 1$ ). It is transitive because if  $b = ca^i$  and  $c = da^j$ , then  $b = da^{i+j}$ .

Exercise 3.9.3 The addition and multiplication tables are below. Any polynomial of degree two or more is congruent modulo  $x^2 + 1$  to a polynomial of degree one or less, since we can

find a multiple of  $x^2 + 1$  that agrees with our target on all terms of degree two or more. The commutative, associative, distributive, and identity properties of the two operations follow from the similar properties of  $\mathbf{Z}_3$ . The additive inverse of the class of  $p$  is just the class of  $-p$ . From the multiplication table we can see explicitly that every nonzero element has a multiplicative inverse.  $\mathbf{Z}_3[x]$  has an Inverse Theorem like that of the naturals, so we get an inverse for every polynomial that is relatively prime to our modulus. When the modulus is **irreducible** as it is here, every nonzero congruence class contains polynomials relatively prime to the modulus.

The set  $C$  is not isomorphic to  $\mathbf{Z}_9$ , even under addition alone. Every element  $p$  of  $C$  satisfies  $p + p + p = 0$ , but in  $\mathbf{Z}_9$  only 0, 3, and 6 satisfy this property.

| +      | 0      | 1      | 2      | $x$    | $x+1$  | $x+2$  | $2x$   | $2x+1$ | $2x+2$ |
|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| 0      | 0      | 1      | 2      | $x$    | $x+1$  | $x+2$  | $2x$   | $2x+1$ | $2x+2$ |
| 1      | 1      | 2      | 0      | $x+1$  | $x+2$  | $x$    | $2x+1$ | $2x+2$ | $2x$   |
| 2      | 2      | 0      | 1      | $x+2$  | $x$    | $x+1$  | $2x+2$ | $2x$   | $2x+1$ |
| $x$    | $x$    | $x+1$  | $x+2$  | $2x$   | $2x+1$ | $2x+2$ | 0      | 1      | 2      |
| $x+1$  | $x+1$  | $x+2$  | $x$    | $2x+1$ | $2x+2$ | $2x$   | 1      | 2      | 0      |
| $x+2$  | $x+2$  | $x$    | $x+1$  | $2x+2$ | $2x$   | $2x+1$ | 2      | 0      | 1      |
| $2x$   | $2x$   | $2x+1$ | $2x+2$ | 0      | 1      | 2      | $x$    | $x+1$  | $x+2$  |
| $2x+1$ | $2x+1$ | $2x+2$ | $2x$   | 1      | 2      | 0      | $x+1$  | $x+2$  | $x$    |
| $2x+2$ | $2x+2$ | $2x$   | $2x+1$ | 2      | 0      | 1      | $x+2$  | $x$    | $x+1$  |

| $\times$ | 0 | 1      | 2      | $x$    | $x+1$  | $x+2$  | $2x$   | $2x+1$ | $2x+2$ |
|----------|---|--------|--------|--------|--------|--------|--------|--------|--------|
| 0        | 0 | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |
| 1        | 0 | 1      | 2      | $x$    | $x+1$  | $x+2$  | $2x$   | $2x+1$ | $2x+2$ |
| 2        | 0 | 2      | 1      | $2x$   | $2x+2$ | $2x+1$ | $x$    | $x+2$  | $x+1$  |
| $x$      | 0 | $x$    | $2x$   | 2      | $x+2$  | $2x+2$ | 1      | $x+1$  | $2x+1$ |
| $x+1$    | 0 | $x+1$  | $2x+2$ | $x+2$  | $2x$   | 1      | $2x+1$ | 2      | $x$    |
| $x+2$    | 0 | $x+2$  | $2x+1$ | $2x+2$ | 1      | $x$    | $x=1$  | $2x$   | 2      |
| $2x$     | 0 | $2x$   | $x$    | 1      | $2x+1$ | $x+1$  | 2      | $2x+2$ | $x+2$  |
| $2x+1$   | 0 | $2x+1$ | $x+2$  | $x+1$  | 2      | $2x$   | $2x+2$ | $x$    | 1      |
| $2x+2$   | 0 | $2x+2$ | $x+1$  | $2x+1$ | $x$    | 2      | $x=2$  | 1      | $2x$   |

Exercise 3.9.4 The multiplicative group  $\mathbf{Z}_{63}^*$  is a direct product of  $\mathbf{Z}_7^*$  and  $\mathbf{Z}_9^*$ , so the elements each have mod-7 residues in  $\{1, 2, 3, 4, 5, 6\}$  and mod-9 residues in  $\{1, 2, 4, 5, 7, 8\}$ . There are thus 36 elements, which are 1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20, 22, 23, 25, 26, 29, 31, 32, 34, 37, 38, 40, 41, 43, 44, 46, 47, 50, 52, 53, 55, 58, 59, 61, and 62.

Exercise 3.9.5 The powers of 2 modulo 17 are 2, 4, 8, 16, 15, 13, 9, 1, so  $2^8 = 1$  and 2 is not a generator. But any square root of 2 will be a generator, and trial and error tells us that  $6^2 = 2$  and so  $6^{16} = 1$  and 16 is the first  $i$  with  $6^i = 1$ .

The powers of 2 modulo 19 are 2, 4, 8, 16, 13, 7, 14, 9, 18, 17, 15, 11, 3, 6, 12, 5, 10, 1 so we see explicitly that 2 is a generator.

Exercise 3.9.6 If  $a$  and  $b$  are nonzero elements with  $ab = 0$  in a ring, then  $a$  and  $b$  cannot both have multiplicative inverses, since then we would have  $1 = (aa^{-1})(bb^{-1}) = (ab)(a^{-1}b^{-1}) = 0$ .



Exercise 3.9.7 (a) If the characteristic  $m$  is equal to  $ab$  where both  $a > 1$  and  $b > 1$ , then the numbers  $x = a1$  ( $a$  copies of 1 added together) and  $y = b1$  are both nonzero, but  $xy = (ab)1 = 0$ . So the ring has zero divisors and cannot be a field by Exercise 3.9.6.

(b) Let  $x$  be any element and  $t$  be its additive order, so that  $tx = 0$  but  $t'x \neq 0$  for all  $t' < t$ . Let  $m$  be the characteristic of the ring. We know that  $m1 = 0$ , and thus by distributivity  $mx = (m1)x = 0$ . Clearly then  $t \leq m$ . If  $t$  did not divide  $m$ , we would have  $m = qt + r$  with  $0 < r < t$ . Then  $mx$  would equal  $(qt + r)x = (qt)x + rx = 0 + rx = rx \neq 0$ , but we know that  $mx = 0$ .

(c) By part (a), the characteristic of the field must be some prime  $p$ , and by part (b) every nonzero element must then have additive order either 1 or  $p$ , and 0 is the only element with additive order 1.

Exercise 3.9.8 We know that  $F^*$  is a cyclic group, meaning that there is at least one element  $g$  such that the elements of  $F^*$  can be listed as  $1 = g^0, g, g^2, \dots, g^{n-1}$ . So we need to know the number of different values of  $i$  such that  $g^i$  is also a generator. This is true if and only if  $i$  is relatively prime to  $n$ . If  $i$  and  $n$  are relatively prime, then by the Inverse Theorem there is a natural  $j$  such that  $ij \equiv 1 \pmod{n}$  and thus  $g^{ij} = g$ . This means that any element  $g^k$  of  $F^*$  can also be written  $g^{i(jk)}$  and is a power of  $g^i$ , so we know that  $g^i$  is a generator. On the other hand, if  $i$  and  $n$  have a common divisor  $d$  with  $d > 1$ ,  $g^i$  does not generate because its order is less than  $n$  — in particular,  $(g^i)^{n/d} = 1$  because it equals  $(g^n)^{i/d}$ .

Exercise 3.9.9 It is not possible. If  $S$  were any such set of complex numbers, it must contain both an additive identity  $a$  and a multiplicative identity  $m$ . We must have  $a = 0$  in order to have  $a + m = m$ . Furthermore, we must have  $m = 1$  in order to have  $m^2 = m$  and  $m \nmid a$ . But then by closure under addition,  $S$  must also contain the elements  $1 + 1, 1 + 1 + 1, 1 + 1 + 1 + 1, \dots$ , and cannot be finite.

Exercise 3.9.10 If the number of elements is not a power of  $p$ , then some other prime  $q$  divides it, and Cauchy's Theorem says that an element of order  $q$  exists, contradicting Exercise 3.9.7.

Exercise 3.11.1 (a) COGITO, ERGO SUM (Descartes, "I think, therefore I am")  
 (b) E PLURIBUS UNUM (Great Seal of the U.S.A., "Out of many, one")  
 (c) ET TU, BRUTE? (Shakespeare's *Julius Caesar*, "And you, Brutus?")  
 (d) VENI, VIDI, VICI (Julius Caesar, "I came, I saw, I conquered")  
 (e) ROMANI ITE DOMUM (Monty Python's *Life of Brian*, "Romans, go home")

Exercise 3.11.2 If  $a$  is relatively prime to  $m$ , we know an integer  $c$  exists such that  $ac \equiv 1 \pmod{m}$  by the Inverse Theorem. If we let  $f(x) = ax + b$  and  $g(x) = c(x - b)$ , then  $f(g(x)) = g(f(x)) = x$ . If  $a$  and  $m$  have a common factor  $r > 1$ , then all values of  $f$  are congruent to  $b$  modulo  $r$  and  $f$  cannot be onto as it misses the other elements.

Exercise 3.11.3

```
public String rotate (String w, int k)
{
    // rotates each letter of w by k, leaves non-letters alone
    string out = "";
    for (int i=0; i < w.length(); i++) {
```

```

char ch = w.charAt(i);
char outch;
if (('a' <= ch) && (ch <= 'z'))
    outch = 'a' + ((ch - 'a' + k)%26);
else if (('A' <= ch) && (ch <= 'Z'))
    outch = 'A' + ((ch - 'A' + k)%26);
else outch = ch;
out += outch;}
return out;}

```

Exercise 3.11.4 Plaintext {65, 76, 79, 72, 65}, ciphertext {65, 98, 118, 128, 65}

Exercise 3.11.5 Here  $M = 143$ ,  $m_1 = 11$ ,  $m_2 = 13$ ,  $\phi(M) = (11-1)(13-1) = 120$ . To find an inverse of 113 modulo 120 we use the Euclidean Algorithm:  $120 = 1 \cdot 113 + 7$ ,  $113 = 16 \cdot 7 + 1$ ,  $7 = 1 \cdot 120 - 1 \cdot 113$ ,  $1 = -16 \cdot 20 + 17 \cdot 113$ . So we decipher by raising the ciphertext to the 17<sup>th</sup> power modulo 143, getting the messages ALABAMA and BOGUSCIPHER. Since each encoded piece is only a single letter, this is in effect a single-letter substitution cipher and could be attacked with frequency analysis, recognition of digrams and trigrams, and so forth.

Exercise 3.11.6 If I have already published my own public keys  $n$  and  $e$ , and have computed my private decoding number  $d$ , I can digitally sign my message  $w$  by publishing both  $w$  and  $s = w^d$ . Anyone can verify that  $s^e = w$ . But to falsely digitally sign a message  $w$  as coming from me, an enemy would need to compute  $w^d$  from  $w$ , which is roughly the same task as decrypting an RSA message from me.

Exercise 3.11.7 (a) ESPIONAGE

- (b) A human hand can show a number from one through five by the number of fingers, if the other party can see the hand. Small numbers would be easier to transmit using flashing lights or semaphore flags. An interesting way to disguise the message would be to create a sequence of digits, where each digit has the same remainder modulo 5 as the ciphertext, and the modulo 2 remainder is chosen randomly.
- (c) MONTYPYTHON For example, the first M (30) comes from subtracting the keyword letter G (22) from the ciphertext 02.

Exercise 3.11.8 (a) We can see that  $r = 2$  and  $r = 3$  do not give sensible plaintext, but with  $r = 4$  the 34-letter ciphertext breaks into four rails with text NESANT, OYXTPNIQII, BDPCHSIHUSO, and OEEESIN respectively. The plaintext is then NOBOD YEXPE CTSTH ESPAN ISHIN QUISI TION (Monty Python).

- (b)  $r = 2$ , plaintext INAHO LEINT HTGRO UNDTN ERELI VEDAH OBBIT (J.R.R. Tolkien).

Exercise 3.11.9 Both  $(g^a)^b$  and  $(g^b)^a$  are equal to  $g^{ab}$ .

```

Exercise 3.11.10    long discreteLog (long p, long g, long a) {
                    long power = 0;
                    long x = 1;
                    while (x != a) {x = (x*g)%p; power++;}
                    return power;}

```

On my 3.2 GHz iMac in 2019, with  $p = 193877777$  (found in a Wikipedia list of particular primes) and  $g = 3$  (found to be a generator by trial and error), the method took from 0 to 10 seconds depending on the output. (The method checks all the possible discrete logs until it finds the right one, so the running time grows linearly with the discrete log itself.)

## S.4 Exercises From Chapter 4

Exercise 4.1.1 0 is a natural by the first axiom. Then by three successive uses of the second axiom we show that  $\text{successor}(0)$ ,  $\text{successor}(\text{successor}(0))$ , and the desired

$$\text{successor}(\text{successor}(\text{successor}(0)))$$

are all naturals.

Exercise 4.1.2 Simplifying notation, we want to show that  $\text{plus}(SS0, SS0) = SSSS0$ . This follows from two uses of the definition:  $\text{plus}(SS0, S(S0)) = \text{plus}(SSS0, S(0)) = \text{plus}(SSSS0, 0) = SSSS0$ .

Exercise 4.1.3

```
public static boolean isThree (natural x)
{ // returns whether x is 3
  if (zero(x)) return false;
  if (zero(pred(x))) return false;
  if (zero(pred(pred(x)))) return false;
  if (zero(pred(pred(pred(x))))) return true;
  return false; }
```

Exercise 4.1.4  $\text{times}(\text{plus}(2, \text{times}(2, \text{plus}(3, 1))), \text{plus}(4, 0))$

Exercise 4.1.5 Let  $x$  be arbitrary. Assume  $x \neq 0$ . By the fifth axiom,  $x$  must be reachable from 0 by successor operations, which would be impossible unless  $x$  were the successor of at least one other natural. But by the fourth axiom, it cannot be the successor of more than one natural, so therefore it is the successor of exactly one natural. This natural is defined to be  $x$ 's predecessor.

Exercise 4.1.6 (a) Only Axiom 3 is false, because 0 is the successor of  $n - 1$ . (b) Axiom 2 is false because  $t$  is its own successor, and Axiom 4 is false because  $t$  and  $t + 1$  have the same successor. The others are true.

Exercise 4.1.7 With the axioms proposed, there is no reason why the predecessor of the successor of 0 must be 0 itself. We could have a number system where every number is obtained from 0 by a sequence of successor and predecessor operations, and every such sequence leads to a different number. If we add the additional axioms  $\forall x : s(p(x)) = x$  and  $\forall x : p(s(x)) = x$ , then the only system satisfying the axioms is  $\mathbf{Z}$ .

Exercise 4.1.8

```
public static boolean equals (natural x, natural y) {
  if (zero(x)) return zero(y);
  if (zero(y)) return false;
  return equals (pred(x), pred(y)); }
```

Exercise 4.1.9 We define  $x^0$  to be 1 and define  $x^{Sy}$  to be  $(x^y)x$ .

```
public static natural power (natural x, natural y) {
  if (zero(y)) return 1;
  return times(power(x, pred(y)), x); }
```

Exercise 4.1.10 “0 is even, and  $Sx$  is even if and only  $x$  is not even.”

```
public static boolean even(natural x) {  
    if zero(x) return true;  
    return !even(pred(x)) []  
}
```

Exercise 4.3.1  $P(n)$  is the statement “the sum of the first  $n$  positive naturals is  $n(n+1)/2$ ”.  $P(0)$  is true because the sum of the first 0 naturals is 0, and  $0(0+1)/2 = 0$ . For the inductive step, assume that  $P(n)$  is true. Then the sum of the first  $n+1$  positive naturals is  $n(n+1)/2$  plus the  $n+1$ ’st natural, which is  $n+1$ . We can calculate that  $n(n+1)/2 + (n+1)$  is equal to  $(n+1)(n/2 + 1) = (n+1)(n+2)/2$ , and thus that the statement  $P(n+1)$ , which says “the sum of the first  $n+1$  positive naturals is  $(n+1)(n+1+1)/2$ ”, is true. We have completed the inductive step and thus the proof.

Exercise 4.3.2  $P(n)$  says that the sum of the first  $n$  positive perfect squares is  $n(n+1)(2n+1)/6$ .  $P(0)$  is true because the sum is 0 and  $0(0+1)(2\cdot 0+1)/6 = 0$ . Assume that  $P(n)$  is true. The sum of the first  $n+1$  positive perfect squares is thus equal to  $n(n+1)(2n+1)/6 + (n+1)^2$ . This equals  $(n+1)/6$  times  $n(2n+1) + 6(n+1) = 2n^2 + 7n + 6 = (n+2)(2n+3)$ . Therefore the sum of the first  $n+1$  positive perfect squares is  $(n+1)(n+1+1)(2(n+1)+1)/6$ , and  $P(n+1)$  is true. We have completed the inductive step and thus the proof.

Exercise 4.3.3 Let  $k$  be an arbitrary natural and let  $A$  be an arbitrary alphabet of size  $k$ .  $P(0)$  says that there is exactly  $k^0 = 1$  string of length 0 over  $A$ , which is true because  $\lambda$  is the only such string. Assume that  $P(n)$  is true, so that there are exactly  $k^n$  strings of length  $n$ . A string of length  $n+1$  is made by appending one of the  $k$  letters to one of the  $k^n$  strings of length  $n$ , giving  $k \cdot k^n = k^{n+1}$  ways to make such a string. By the third Peano axiom for strings in Section 4.1, each of these ways gives a different string and there are thus exactly  $k^{n+1}$  strings of length  $n+1$  and  $P(n+1)$  is true. We have completed the inductive step and thus the proof.

Exercise 4.3.4  $P(n)$  says that 3 divides  $n^3 - n$ .  $P(0)$  is true because 3 divides  $0^3 - 0 = 0$ . Assume that  $P(n)$  is true, so that 3 divides  $n^3 - n$ . We want to show that 3 also divides  $(n+1)^3 - (n+1)$ . We calculate that this latter number is  $n^3 + 3n^2 + 3n + 1 - n - 1$ , which is  $n^3 - n$  plus  $3n^2 + 3n = 3(n^2 + n)$ . Since 3 divides this difference, it also divides  $(n+1)^3 - (n+1)$  and  $P(n+1)$  is true. We have completed the inductive step and thus the proof.

Exercise 4.3.5  $P(n)$  says that the size of  $\{k : k < n\}$  is  $n$ .  $P(0)$  says that the size of  $\{k : k < 0\}$  is 0, and this is true because no naturals are less than 0. So assume that  $P(n)$  is true. The set  $\{k : k < n+1\}$  consists of  $\{k : k < n\}$  plus the single element  $n$ , so if  $\{k : k < n\}$  is  $n$  then the size of  $\{k : k < n+1\}$  is  $n+1$ . So  $P(n+1)$  is true, and we have completed the inductive step and thus the proof.

Exercise 4.3.6 We let  $P(x)$  be the statement “ $x$  is odd or even, but not both”. The base case  $P(0)$  is true because the definitions say that 0 is even but not odd. The inductive hypothesis is  $P(x)$ , and the inductive goal is  $P(x+1)$ . Assume  $P(x)$ . For the first case, assume that  $x$  is even but not odd. Then by the facts proved in Excursion 1.2,  $x+1$  is odd

but not even and so  $P(x+1)$  is true. For the other case,  $x$  is odd but not even, and the facts from Excursion 1.2 show that  $x+1$  is even but not odd, so again  $P(x+1)$  is true.

Exercise 4.3.7 For the base case,  $n = 0$  and we can take  $q = 0$  and  $r = 0$ . For the inductive hypothesis, we assume that  $n = qm + r$ . Our inductive goal is to prove that there exist  $q'$  and  $r'$  such that  $n+1 = q'm + r'$ . If  $r < m-1$ , we can take  $q' = q$  and  $r' = r+1$ . If  $r = m-1$ , we can take  $q' = q+1$  and  $r' = 0$ . In either case,  $n+1 = q'm + r'$  is true, so we have completed the proof.

Exercise 4.3.8 For the base case,  $P(0)$  is true because there are no such subsets, and  $0(0-1)/2 = 0$ . For the inductive case, we assume that there are  $n(n-1)/2$  subsets of size 2 for any  $n$ -element set, and our inductive goal is to prove that there are  $(n+1)n/2$  such subsets for any  $n+1$  element set. If  $n+1 = 1$  we can see (without the inductive hypothesis) that there are no such subsets, and  $1(0)/2 = 0$ . If  $n+1 = 2$ , we see (without the inductive hypothesis) that there is one such subset, and  $2(1)/2 = 1$ . For the other cases, we consider an arbitrary set  $S$  of  $n+1$  elements. We name one element  $x$  and let  $T$  be the set  $S \setminus \{x\}$ . We observe that a size-2 subset of  $S$  is either a size-2 subset of  $T$ , or contains  $x$  and exactly one element of  $T$ . By the inductive hypothesis, there are  $n(n-1)/2$  size-2 subsets of  $T$ . Since there are  $n$  elements in  $T$ , there are exactly  $n$  subsets of the second kind. So the total number of size-2 subsets of  $S$  is  $n(n-1)/2 + n = (n+1)n/2$ , which proves our inductive goal.

Exercise 4.3.9 The rule is that  $S_n = 0$  if  $n$  is even, and  $S_n = -1$  if  $n$  is odd. For the base case, we have already observed that  $S_0 = 0$ , and 0 is even. The inductive case breaks down into two cases. If  $n$  is even, the inductive hypothesis says that  $S_n = 0$ , and we know that  $S_{n+1} = S_n + (-1)^{n-1} = 0 - 1 = -1$ , which is correct because  $n+1$  is odd. If  $n$  is odd, the inductive hypothesis says that  $S_n = -1$ , and we know that  $S_{n+1} = S_n + (-1)^{n-1} = -1 + 1 = 0$ , which is correct because  $n+1$  is even. In either case,  $S_{n+1}$  satisfies the rule and we have proved the inductive goal.

Exercise 4.3.10 For the base case of  $n = 0$ , we are told that the amount at time  $0T = 0$  is  $g$ , and  $g = g2^{-0}$ . The inductive hypothesis is that there are  $g2^{-n}$  grams at time  $nT$ . The inductive goal is to prove that there are  $g2^{-(n+1)}$  grams at time  $(n+1)T$ . Setting  $t$  to be  $nT$ , we are told that the amount at time  $t+T$  is half the amount at time  $t$ , which means that the amount at time  $nT+T$  is exactly  $(1/2)g2^{-n} = g2^{-(n+1)}$ . This proves the inductive goal and completes the proof.

Exercise 4.4.1 The numbers  $n^2/10$  and  $47n$  are equal if  $n = 470$ . So we will begin with  $n = 471$  as our starting point.  $P(471)$  is true because  $(471)^2/10 > 47 \cdot 471$ . For an arbitrary  $n \geq 471$ , assume that  $n^2/10 > 47n$ . Then  $(n+1)^2/10 = n^2/10 + n/5 + 1/10$ . This is greater than  $n^2/10 + 47$  because  $n/5 > 47$ , and this is greater than  $47n + 47 = 47(n+1)$  by the inductive hypothesis. So  $P(n+1)$  is true, and we have completed the inductive step and thus the proof.

Exercise 4.4.2 When is  $2^n = 137n^3$ ? I see no obvious way to solve this exactly, but we may try  $n = 10$  (too small, because  $2^{10} = 1024 < 137000$ ), and then  $n = 20$  (barely too small, because  $1048576 < 1096000$ ) before noticing that  $n = 21$  is the first natural making  $P(n)$  true ( $2097152 > 1268757$ ). So it remains to assume  $P(n)$ , for an arbitrary  $n \geq 21$ , and prove  $P(n+1)$ . We know that  $2^{n+1} = 2^n + 2^n$ , while  $137(n+1)^3 =$

$137n^3 + 411n^2 + 411n + 137$ . The right-hand side of this latter equation is clearly less than  $137n^3 + 137n^3$ , so by the inductive hypothesis it is less than  $2^n = 2^n$ . We have proved  $P(n+1)$ , completing the inductive step and thus the proof.

Exercise 4.4.3 By strong induction: For  $n < 4$  there is nothing to prove, for  $n = 4$  we can use two 2's, for  $n = 5$  we can use one 5. For  $n \geq 5$ , the strong inductive hypothesis says that we can make  $n - 1$ , so we make  $n + 1$  by adding one 2 to  $n - 1$ .

By odd-only and even-only induction: For evens with  $n \geq 4$ , the base case of  $n = 4$  is done with two 2's. If we can make  $n$ , then we can make  $n + 2$  by adding another 2, completing the induction for odds. For evens with  $n \geq 4$ , the base is  $n = 5$  which we make with one 5. The inductive case goes from  $n$  to  $n + 2$ , again by adding one 2.

By ordinary induction: For  $n < 4$  there is nothing to prove, and for  $n = 4$  we use two 2's. Assume we can make  $n$ , with  $n \geq 4$ . If we used a 5 to make  $n$ , replace it with three 2's to make  $n + 1$ . If there is no 5 in the sum for  $n$ , since  $n \geq 4$  there must be at least two 2's and we can thus make  $n + 1$  by replacing two 2's with a 5. Either way, under the assumption that we could make  $n$ , we showed that we could make  $n + 1$ , completing the ordinary induction.

Exercise 4.4.4 The base case  $P(1)$  is true because 8 divides  $1^2 + 7 = 8$ . For the inductive case, assume that 8 divides  $n^2 + 7$  and we will show that 8 divides  $(n + 2)^2 + 7$ . We calculate that  $(n + 2)^2 + 7 = n^2 + 4n + 4 + 7 = (n^2 + 7) + 4(n + 1)$ . But since  $n$  is odd,  $n + 1$  is even and  $4(n + 1)$  is divisible by 8. Since  $(n + 2)^2 + 7$  is the sum of two numbers that are each divisible by 8, it is itself divisible by 8. We have completed the induction and proved the statement for all odd numbers.

Exercise 4.4.5 The base case is to prove that the first odd number is 1, and this is true by definition. The inductive case is to show that if the  $n$ 'th odd number is  $2n - 1$ , then the  $n + 1$ 'st odd number is  $2(n + 1) - 1 = 2n + 1$ . This is true because by definition the  $n + 1$ 'st odd number is two greater than the  $n$ 'th odd number.

Exercise 4.4.6 Let  $P(n)$  be the statement the second player wins the  $n$ -stone game if and only if  $n$  is divisible by 4.  $P(0)$  is true because the second player wins the game with no stones, since the first player cannot move. Now assume that  $P(m)$  is true for all  $m$  with  $m \leq n$ , and consider the game with  $n + 1$  stones. If  $n + 1$  is not divisible by 4, the first player can leave a number of stones that is divisible by 4 by taking one, two, or three stones. In the game that follows (in which the second player moves first), we know that the "second player" of that game has a winning strategy. This is the first player of the original game, so  $P(n + 1)$  is true in this case. (The number of stones is not divisible by 4, and the second player does not win.) The remaining case is when  $n + 1$  is divisible by 4, and we must prove that the second player has a winning strategy. The first player's move will leave a number that is not divisible by 4, which means that the second player can leave a number that is divisible by 4, and we know by the strong inductive hypothesis that the second player has a winning strategy in this game. Thus  $P(n + 1)$  holds in this case as well.

Exercise 4.4.7 For the base case, with  $n = 1$ , the first statement must be an axiom, and axioms are true. Assume that the  $m$ 'th statement is true for all  $m$  with  $m \leq n$ . Then the  $n + 1$ 'st statement is either an axiom, which is true, or follows from one or more earlier

statements. In this second case, the strong inductive hypothesis tells us that all the earlier statements are true, so the  $n + 1$ 'st statement must be true as well.

**Exercise 4.4.8** The base cases of  $n = 0$  and  $n = 1$  are both true vacuously because a string of that length cannot contain both an  $a$  and a  $b$ . If  $n = 2$ , the string must be either  $ab$  or  $ba$ , so it contains either  $ab$  or  $ba$ . Now let  $n$  be an arbitrary natural, with  $n \geq 2$ , and assume  $P(n)$ . Consider an arbitrary string  $w$  of length  $n + 1$  that contains both an  $a$  and a  $b$ . Write  $w$  as either  $xa$  or  $xb$ , where  $x$  is a string of length  $n$ . If  $x$  contains both an  $a$  and a  $b$ , the inductive hypothesis  $P(n)$  tells us it contains  $ab$  or  $ba$  as a substring, so  $w$  does as well. If  $x$  does not contain both letters, and  $w = xa$ , then  $x$  must be the string  $b^n$ , and  $w$  contains a substring  $ba$  at the end. Similarly, if  $x$  does not contain both letters and  $w = xb$ , then  $x = a^n$  and  $w$  has an  $ab$  substring at the end. We have proved  $P(n + 1)$  by cases.

**Exercise 4.4.9** We use strong induction on all positive naturals  $n$ , letting  $P(n)$  be the statement that any product of any  $n$  elements is in  $X$ . The base case  $P(1)$  is true because the product is the element  $a_1$ , which is assumed to be in  $X$ . Assume that  $P(m)$  is true for all  $m$  with  $m \leq n$ , and consider any product of  $n + 1$  elements. This product must be made at the end by multiplying the product of the first  $k$  elements times the product of the last  $n + 1 - k$  elements, for some positive natural  $k$  with  $k \leq n$ . By the strong inductive hypothesis, each of these two products, no matter how they are parenthesized, must be in  $X$ . Therefore the final product is also in  $X$ , and we have proved our inductive goal.

**Exercise 4.4.10**  $P(k)$  is the statement "If  $m_1, m_2, \dots, m_k$  are any pairwise relatively prime naturals, and  $a_1, a_2, \dots, a_k$  are any naturals, then the family of congruences  $x \equiv a_1 \pmod{m_1}$ ,  $x \equiv a_2 \pmod{m_2}, \dots, x \equiv a_k \pmod{m_k}$  is equivalent to a single congruence  $x \equiv c \pmod{m_1 m_2 \dots m_k}$ ." For the base cases,  $P(1)$  is trivially true, and we are assuming  $P(2)$ . Assume that  $P(k)$  is true, and let our inductive goal be  $P(k + 1)$ . Given a system of  $k + 1$  congruences as described, use  $P(k)$  to convert the first  $k$  of them to a single congruence with modulus  $m_1 m_2 \dots m_k$ . Note that this modulus must be relatively prime to  $m_{k+1}$  because the original moduli were pairwise relatively prime (if  $m_{k+1}$  shared a prime factor with the new modulus, it would share it with at least one of the old moduli). So applying  $P(2)$  again to the new congruence and the last congruence, we get a single congruence which is exactly the one we want, with modulus  $m_1 m_2 \dots m_k m_{k+1}$ .

**Exercise 4.6.1** Let  $x$  be an arbitrary natural. We have that  $\text{times}(x, S(0)) = \text{times}(x, 0) + x = 0 + x = x$ . We need no induction but in the last step we use a rule proved in Subsection 4.6.2.

**Exercise 4.6.2** Let  $P(x)$  be the statement  $\text{times}(1, x) = x$ .  $P(0)$  says that  $\text{times}(1, 0) = 0$ , and this is true by the definition of *times*. Assume that  $P(n)$  is true. Then  $P(n + 1)$  says that  $\text{times}(1, S(n)) = S(n)$ . Using the definitions,  $\text{times}(1, S(n)) = \text{plus}(\text{times}(1, n), 1) = S(\text{times}(1, n))$  which is  $S(n)$  by the inductive hypothesis.

**Exercise 4.6.3** We prove  $P(y)$ , which is  $\forall x : (x + y) - y$  by induction on  $y$ . For the base case, let  $x$  be arbitrary. Then  $(x + 0) - 0 = x + 0 = x$  by the definitions of subtraction and addition. So assume that  $P(y)$  is true, and we set out to prove  $P(S(y))$ , which is  $\forall x : (x + S(y)) - S(y) = x$ . We first need a lemma, that for any  $x$  and  $y$ ,  $(x >$



$y) \leftrightarrow (x - y \neq 0)$ . We prove this lemma by induction on all  $x$  with  $x > y$ . The base case is  $x = S(y)$  where  $x - y = 1 \neq 0$ . Then if  $x - y \neq 0$ , it follows that  $S(x) - y = S(x - y) \neq 0$ .

Given the lemma, we let  $x$  be arbitrary and compute  $(x + S(y)) - S(y)$ . By the definition of subtraction, it is the predecessor of  $(x + S(y)) - y$  unless  $(x + S(y)) - y = 0$ , but the lemma rules out this latter case because  $x + S(y) \geq S(y) > y$ . By a fact we proved about addition,  $(x + S(y)) - y = (S(x) + y) - y$ . Since we are assuming  $P(y)$ , we know that this latter expression equals  $S(x)$ , as desired.

We now turn to  $Q(y)$ , the statement  $\forall x : (x \geq y) \rightarrow (x - y) + y = x$ . For the base  $Q(0)$ , let  $x$  be arbitrary and note that  $(x \geq 0) \rightarrow ((x - 0) + 0 = x)$  follows from trivial proof, given the definitions of subtraction and addition. So assume  $\forall x : (x \geq y) \rightarrow (x - y) + y = x$  and we set out to prove  $\forall x : (x \geq S(y)) \rightarrow (x - S(y)) + S(y) = x$ . Let  $x$  be arbitrary and assume that  $x \geq S(y)$ . By the Lemma, we know that  $x - S(y) = 0$  if and only if  $x = S(y)$ . In the case that  $x = S(y)$ ,  $(x - S(y)) + S(y) = 0 + S(y) = S(y)$  as desired. In the other case, we know that  $x - S(y)$  is not 0, so it is the predecessor of  $x - y$ . We thus need to compute  $\text{pred}(x - y) + S(y) = S(\text{pred}(x - y)) + y = x - y + y = x$ , where the next to last step uses the fact that  $x - y \neq 0$ .

**Exercise 4.6.4** Let  $x$  and  $y$  be arbitrary, and use ordinary induction on  $z$ . For the base case,  $x - (y + 0)$  and  $(x - y) - 0$  are both equal to  $x - y$ . For the inductive case, assume that  $x - (y + z) = (x - y) - z$  and set out to prove  $x - (y + S(z)) = (x - y) - S(z)$ . The left-hand side is  $x - S(y + z)$ , which is the predecessor of  $x - (y + z)$ , or 0 if  $x - (y + z) = 0$ . The right-hand side is the predecessor of  $(x - y) - z$ , or 0 if  $(x - y) - z = 0$ . By the inductive hypothesis, then, the left-hand side and right-hand side are the same.

**Exercise 4.6.5** We first prove a lemma that  $\text{times}(x, \text{pred}(w)) = \text{times}(x, w) - x$  if  $w > 0$ . We use induction on all positive  $w$ . For the base of  $w = 1$ ,  $\text{times}(x, 0) = \text{times}(x, 1) - x$  is true as both sides are 0. For the induction,  $\text{times}(x, \text{pred}(S(w))) = \text{times}(x, w) = x + \text{times}(x, \text{pred}(w))$ . By the inductive hypothesis this is  $x + (\text{times}(x, w) - x) = (\text{times}(x, w)) - x = \text{times}(x, S(w)) - x$ .

Now to the main result. We let  $x$  and  $y$  be arbitrary and use induction on  $z$ . For the base case of  $z = 0$ ,  $\text{times}(x, y - 0) = \text{times}(x, y) - \text{times}(x, 0)$  as both sides equal  $\text{times}(x, y)$ . So assume that  $\text{times}(x, y - z) = \text{times}(x, y) - \text{times}(x, z)$  and set out to prove that  $\text{times}(x, y - S(z)) = \text{times}(x, y) - \text{times}(x, S(z))$ . The left-hand side is  $x$  times a number that is either the predecessor of  $y - z$ , or 0 if  $y - z = 0$ . By the lemma, this is  $\text{times}(x, y - z) - x$ , or 0 if  $y - z = 0$ . The right-hand side is  $\text{times}(x, y) - (\text{times}(x, z) + x)$ . If  $\text{times}(x, y) \geq \text{times}(x, z)$ , this is  $(\text{times}(x, y) - \text{times}(x, z)) - x$ , which by the inductive hypothesis equals the right-hand side. The other case of  $\text{times}(x, y) < \text{times}(x, z)$  implies that  $y < z$ , and in this case both sides of the equation are zero. (We are implicitly using a lemma that if  $x > 0$ ,  $\text{times}(x, y) < \text{times}(x, z)$  if and only if  $y < z$  - this is easy to prove by induction on  $x$ .)

**Exercise 4.6.6** (a) We must show that each operation is commutative, associative, and has an identity, and that the distributive law holds. In the case of modular arithmetic, we know that the class modulo  $m$  of a sum or a product does not depend on the representative of the congruence class we choose as an input. Thus to prove any identity over congruence classes, such as  $a + b = b + a$  or  $a(b + c) = ab + ac$ , it

suffices to observe that the same identity holds over the integers. If, for example, we choose any three integers  $a$ ,  $b$ , and  $c$ , we know that  $a(b + c)$  and  $ab + bc$  are the same integer. If we replace any of those three integers by others that are congruent to them modulo  $m$ , the congruence class of each expression remains the same, so the two classes remain equal. The identity properties of 0 for  $+$  and 1 for  $\times$  over the integers imply the same properties modulo  $m$ .

- (b) Here once again we can think of an operation threshold  $t$  as being the same operation over the naturals, followed by replacing the result by its equivalence class for the relation where all naturals  $t$  or greater are considered equivalent. For each of the properties, the left-hand and right-hand sides yield the same natural result for any particular choice of inputs, so they yield equivalent results if we choose equivalent representatives. The identities remain 0 for  $+$  and 1 for  $\times$ .

Exercise 4.6.7 We know that the minimum and addition operations are commutative and associative for ordinary real numbers, and it is easy to check that the presence of  $\infty$  does not change any of these properties. The minimum of  $x$  and  $y$  is the same as the minimum of  $y$  and  $x$ , and the minimum of  $x$ ,  $y$ , and  $z$  does not depend on any parenthesization. The sum of any two numbers, or of any three numbers, including  $\infty$  is just  $\infty$ , no matter the order or the parentheses. For the distributive law, we must check that  $x$  plus the minimum of  $y$  and  $z$  is the minimum of  $x + y$  and  $x + z$ , which is clearly true by cases. The identity for the new “addition” (minimum) is  $\infty$ , and the identity for the new “multiplication” (addition) is 0.

Exercise 4.6.8 We’ve observed in Chapter 1 that the AND and OR operations are each commutative and associative, and that AND distributes over OR. (OR also distributes over AND, but that is not a semiring property.) The identities are 0 for OR and 1 for AND.

Exercise 4.6.9 A polynomial in  $S[x]$  is a finite sum of terms of the form  $ax^i$ , where  $a$  is an element of  $S$  and  $i$  is a natural. If  $f$  and  $g$  are each such polynomials, we define  $f + g$  to be the sum of terms  $(a + b)x^i$  for every  $i$ , where  $a$  and  $b$  are the  $x^i$  coefficients in  $f$  and  $g$  respectively and the sum uses the addition operation of  $S$ . (We can take any missing coefficients to be 0.) This addition of polynomials is commutative and associative because for each value of  $i$ , the coefficients on each side are computed from the same inputs using the commutative and associative operations of  $S$ . The empty sum 0 is the additive identity.

To find the product of two polynomials  $f$  and  $g$ , we take every pair of terms  $ax^i$  in  $f$  and  $bx^j$  in  $g$  and form a new term  $(ab)x^{i+j}$ , with the multiplication being taken in  $S$ . We then collect all these terms, adding together (using the addition of  $S$ ) any that have the same exponent of  $x$ . If we reverse  $f$  and  $g$ , we get the same terms because  $S$ ’s multiplication is commutative, and the same sum because  $S$ ’s addition is commutative and associative. The polynomial “1”, with the single term  $1x^0$  where 1 is the identity of  $S$ , is the identity for polynomial multiplication.

Why is this multiplication associative? If we compute  $f(gh)$  or  $(fg)h$  for three polynomials  $f$ ,  $g$ , and  $h$ , then either way we will produce exactly one term for each triple of terms  $\langle ax^i, bx^j, cx^k \rangle$  coming from  $f$ ,  $g$ , and  $h$  respectively, and the value of this term will be  $(abc)x^{i+j+k}$ . Because multiplication in  $S$  and addition in  $\mathbf{N}$  are each associative, we get equal terms, and because addition in  $S$  is commutative and associative,

we will get the same result either way for the sum of terms with each exponent.

Similarly, in computing  $f(g + h)$  and  $fg + fh$ , we will get the same terms of the form  $abx^{i+j}$  or  $acx^{i+k}$  either way, and because addition in  $S$  is commutative and associative we will get the same result as the coefficient of either for each possible exponent.

Exercise 4.6.10 We prove  $\forall x : A(x, x)$  by induction on all naturals  $x$ . For the base case, we are given that  $A(0, 0)$  is true. Our inductive hypothesis is  $A(x, x)$ , and our inductive goal is  $A(Sx, Sx)$ . The first general rule tells us that  $A(Sx, y)$  is false, and the second (specialized to  $Sx$  and  $y$ ) tells us that  $A(Sx, Sx)$  is true, as desired.

Exercise 4.7.1 By the first axiom  $\lambda$  is a string, by using the second axiom three times we show that  $a = \text{append}(\lambda, a)$ ,  $ab = \text{append}(a, b)$ , and  $aba = \text{append}(ab, a)$  are all strings.

Exercise 4.7.2

```
public boolean isEqual (string x, string y)
{
    // returns true if x == y
    if (isEmpty (x)) return isEmpty (y);
    if (isEmpty (y)) return false;
    if (last(x) == last(y))
        return isEqual (allButLast(x), allButLast(y));
    else return false;
}
```

Exercise 4.7.3 Define  $oc(\lambda)$  to be  $\lambda$ ,  $oc(w0)$  to be  $oc(w)1$ , and  $oc(w1)$  to be  $oc(w)0$ .

Exercise 4.7.4

```
public string oc (string w)
{
    // returns one's complement of w
    if (isEmpty (w)) return emptyString;
    string ocabl = oc(allButLast(w));
    if (last(w) == '0') return append (ocabl, '1');
    if (last(w) == '1') return append (ocabl, '0');
    throw new Exception ("oc called on non-binary string");
}
```

Exercise 4.7.5

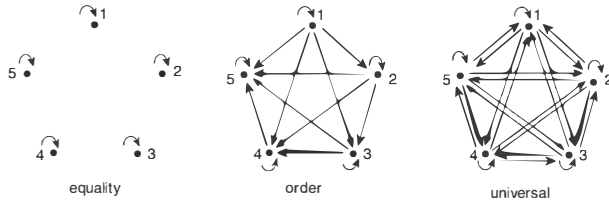
```
public String rev (String w)
{
    // returns reversal of w, computed without recursion
    String out = "";
    for (int i = w.length() - 1, i >= 0, i--)
        out += w.charAt(i);
    return out;
}

public String revRec (String w)
{
    // returns reversal of w, computed recursively
    int n = w.length();
    if (n == 0) return "";
    return revRec(w.substring(1,n)) + w.substring(1);
}
```

Exercise 4.7.6 A string  $u$  is a suffix of  $\lambda$  if and only if it is empty. A string  $ua$  is a suffix of a string  $w$  if and only if (1)  $w = va$  for some string  $v$  and (2)  $u$  is a suffix of  $v$ .

Exercise 4.7.7

```
public static boolean isSuffix (string u, string v) {
    if (isEmpty(u)) return isEmpty(v);
    if (isEmpty(v)) return false;
}
```



©Kendall Hunt Publishing Company

Figure S-10: Graphs of Three Relations for Exercise 4.9.1

```
if (last(u) != last (v)) return false;
return isSuffix(allButLast(u), allButLast(v));}
```

Exercise 4.7.8 We define  $stars(\lambda)$  to be  $\lambda$ , and  $stars(wa)$  to be  $stars(w)*$ .

```
public static string stars (string w) {
    if (isEmpty(w)) return emptystring;
    return append(stars(allButLast(w)), '*');
```

Exercise 4.7.9 The relation is false if  $u$  is empty. Otherwise if  $u = vx$  for some string  $v$  and some character  $x$ ,  $contains(u, a)$  is true if and only either  $contains(v, a)$  or  $x = a$ .

```
public static boolean contains (string u, char a) {
    if (isEmpty(u)) return false;
    if (last(u) == a) return true;
    return contains(allButLast(u), a);}
```

Exercise 4.7.10 The key to the inductive step is that the double letter either occurs in  $allButLast(w)$  or is the last two letters of the string. two l

```
public static boolean hasDouble (string w) {
    if (isEmpty(w)) return false;
    if (isEmpty(allButLast(w))) return false;
    if (last(w) == last(allButLast(w))) return true;
    return hasDouble(allButLast(w));}
```

Exercise 4.9.1 The three graphs are shown in Figure S-10.

Exercise 4.9.2 An undirected graph's edge relation is anti-reflexive (since the graph has no loops) and symmetric (since every edge is bidirectional).

Exercise 4.9.3 The edge relation is reflexive if the graph has a loop at every vertex. It is anti-reflexive if the graph has no loops. It is symmetric if all non-loop edges are bidirectional (each arc has a corresponding arc in the other direction). It is antisymmetric if there is *no* non-loop arc with a corresponding arc in the other direction. It is transitive if every two-step path has a corresponding shortcut, an arc from the start of the first arc to the end of the second. The directed graph of an equivalence relation consists of a set of complete directed graphs, one for each equivalence class. The directed graph of a partial order has a loop at every vertex, no other cycles, and the transitivity property above. To get from this graph to the Hasse diagram, remove the loops and remove all shortcut edges – all edges that have the same start and finish as a path of two or more edges.

Exercise 4.9.4 Case 1: The path  $\alpha$  is empty, and there is nothing to prove. Case 2: The path  $\alpha$  is a path  $\beta$  followed by an edge  $e$ , where  $\beta$  is an empty path. In this case  $e$  is the first edge, since  $\alpha$  is  $e$  followed by an empty path. Case 3: The path  $\alpha$  is  $\beta$  followed by an edge  $e$ , where  $\beta$  is not empty. By the inductive hypothesis,  $\beta$  has a first edge  $c$  and consists of  $c$  followed by some path  $\gamma$ . In this case  $c$  is also the first edge of  $\alpha$ , since  $\alpha$  is  $c$  followed by  $\gamma$  followed by  $e$ , which since paths are transitive is  $c$  followed by the path made by composing  $\gamma$  and  $e$ .

Exercise 4.9.5 On a directed graph the path predicate need not be symmetric. On an undirected graph, the path predicate is reflexive by definition. It is clearly symmetric because any path can be reversed edge by edge (this is proved formally as Problem 4.9.1). It was proved to be transitive in the Theorem of Section 4.9.

To show that  $P(x, y) \wedge P(y, x)$  is an equivalence relation: It is reflexive because  $P(x, x)$  is true and thus  $P(x, x) \wedge P(x, x)$  is given by the definition of  $P$ . It is clearly symmetric in  $x$  and  $y$  (using the commutativity of  $\wedge$ ) and thus is a symmetric relation. It is transitive because if  $P(x, y) \wedge P(y, x)$  and  $P(y, z) \wedge P(z, y)$  are given,  $P(x, z)$  and  $P(z, x)$  follow by separation and the transitivity of  $P$ .

The relation  $P(x, y) \vee P(y, x)$  is reflexive and symmetric, but it need not be transitive on directed graphs. For a counterexample, consider the graph with vertex set  $\{a, b, c\}$  and arcs  $\langle a, b \rangle$  and  $\langle c, b \rangle$ . Here the relation holds for  $a$  and  $b$  (since  $P(a, b)$  is true), and for  $b$  and  $c$  (since  $P(c, b)$  is true) but not for  $a$  and  $c$  (since neither  $P(a, c)$  nor  $P(c, a)$  is true).

Exercise 4.9.6 Any directed graph is isomorphic to itself, via the function that takes each vertex to itself, so isomorphism is a reflexive relation. If  $f$  is an isomorphism from  $G$  to  $H$ , we know that it has an inverse function from  $H$  to  $G$  (because it is a bijection) and this function is an isomorphism because it takes arcs to arcs and non-arcs to non-arcs. This makes isomorphism a symmetric relation. Finally, if  $f$  is an isomorphism from  $G$  to  $H$  and  $k$  is an isomorphism from  $H$  to  $I$ , then the composition  $k \circ f$  is a function from  $G$  to  $I$ . We know that the composition of two bijections is a bijection, and it is easy to see that it also takes arcs to arcs and non-arcs to non-arcs. This shows that isomorphism is a transitive relation, making it an equivalence relation.

Exercise 4.9.7 (a) An isomorphism  $f$  from an undirected graph  $G$  to another undirected graph  $H$  creates a bijection from the edges of  $G$  to the edges of  $H$ , taking edge  $\langle x, y \rangle$  of  $G$  to edge  $\langle f(x), f(y) \rangle$  of  $H$ . Because  $f$  has an inverse  $f^{-1}$ , the mapping on edges also has an inverse and must be a bijection.

- (b) An undirected graph with three nodes could have zero, one, two, or three edges. If two such graphs each have zero edges, or each have three edges, then any bijection of the nodes is an isomorphism. If two such graphs each have one edge, we can choose a bijection of the nodes that takes the endpoints of one edge to the endpoints of the other. And if they have two edges, we choose a bijection that maps one node with two neighbors to the other, and this will also be an isomorphism.
- (c) Let  $G$  have node set  $\{a, b, c, d\}$  and edges  $(a, b)$ ,  $(a, c)$ , and  $(a, d)$ . Let  $H$  have node set  $\{w, x, y, z\}$  and edges  $(w, x)$ ,  $(x, y)$ , and  $(y, z)$ . If  $f$  were an isomorphism from  $G$  to  $H$ , the node  $f(a)$  in  $H$  would have to have edges to each of the other three nodes. But none of the four nodes in  $H$  has edges to each other node.

Exercise 4.9.8 If  $f$  is an isomorphism from  $G$  to  $H$ , and  $x$  and  $y$  are any vertices of  $G$ , then  $f$  maps any path from  $x$  to  $y$  into a path from  $f(x)$  to  $f(y)$ , and the inverse of  $f$  maps any path from  $f(x)$  to  $f(y)$  to a path from  $x$  to  $y$ .

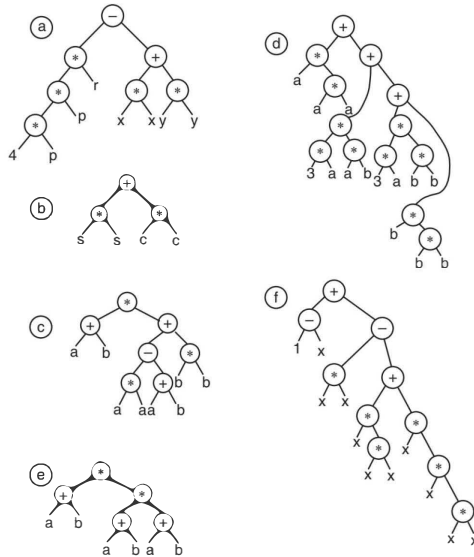
- (a) If there is a path in  $G$  from any  $x$  to any  $y$ , there is a path in  $H$  from  $f(x)$  to  $f(y)$ . Thus if  $G$  is connected, any two vertices in  $H$  have a path from one to the other, since they are  $f(x)$  and  $f(y)$  for some vertices  $x$  and  $y$  in  $G$ . So  $H$  is also connected.
- (b) Similarly any cycle in  $G$  is mapped by  $f$  to a cycle in  $H$ , and vice versa by  $f$ 's inverse. So  $G$  has a cycle if and only if  $H$  does, and thus has no cycle if and only if  $H$  does.
- (c) This follows directly from parts (a) and (b) by the definition of a tree as a connected forest.

Exercise 4.9.9 There is one graph with no arcs, in a class by itself, and one graph with four arcs, in a class by itself. There are four graphs with exactly one arc, and these form two isomorphism classes depending on whether the arc is a loop. Similarly the four graphs with exactly three arcs form two classes, depending on whether the missing arc is a loop. The six nodes with exactly two arcs divide into four equivalence classes (giving us ten equivalence classes in all). To see this, note that the two arcs could include zero, one, or two loops. If there are zero or two, there is only one possible graph, but with one loop the non-loop could be directed either into or out of the node with the loop.

Exercise 4.9.10 Rename the vertices of  $G$  so that the  $n$  edges of the path become  $(v_0, v_1)$ ,  $(v_1, v_2)$ ,  $\dots$ ,  $(v_{n-1}, v_n)$ . This may result in more than one name being given to a particular node. In fact this *must* happen, because there are  $n + 1$  names  $(v_0, v_1, v_2, \dots, v_n)$  and only  $n$  different nodes. This means that at least one node has two different node numbers, that is, it is both  $v_i$  and  $v_j$  where  $i < j$ . The portion of the path from  $v_i$  to  $v_j$ , consisting of  $j - i$  edges, is then a directed cycle.

Exercise 4.10.1 We add a second clause saying that if  $S$  is a tree with root  $s$ , the following is a tree: Nodes are a new node  $x$  plus the nodes of  $S$ , arcs are  $\langle x, s \rangle$  plus the arcs of  $S$ . The root of the new tree is  $x$ .

Exercise 4.10.2 Here is pseudo-Java code for the desired method:



©Kendall Hunt Publishing Company

Figure S-11: Six Trees for Exercise 4.10.4

```
natural numAtoms()
{
  // returns number of atoms in calling object
  if (isAtom) return 1;
  return left.numAtoms() + right.numAtoms();
}
```

- Exercise 4.10.3 (a)  $((4*p)*r)-((x*x)+(y*y))$   
 (b)  $++ss*cc$   
 (c)  $ab+aa*ab*-bb***$   
 (d)  $++a*aa+3*a*ab+3*a*bb*b*bb$   
 (e)  $ab+ab+ab***$   
 (f)  $(1-x)+(x*x)-(x*x*x)+(x*x*x*x)$

Exercise 4.10.4 The six trees are given in Figure S-11.

Exercise 4.10.5 Every arc enters exactly one node and so contributes 1 to the in-degree of exactly one node. Since a tree has one node of in-degree 0 (the root) and  $n - 1$  nodes of degree 1, the sum of the in-degrees is  $n - 1$  and there must be exactly  $n - 1$  arcs.

Exercise 4.10.6 (a) For the base case, the rooted directed tree has one node and no arcs, so the only possible path is the trivial one from the node to itself, which is finite. The depth

of this tree is 0. For the inductive case, we have a rooted directed tree  $T$  with a root and arcs to the roots of one or more rooted directed trees, each of which by the inductive hypothesis has only finite paths. Any path in  $T$  is either a path entirely in one of the subtrees (which must be finite) or an arc from the root of  $T$  followed by a path in a subtree (which is also finite).

- (b) The depth of the one-node tree is 0. If we make a rooted directed tree  $T$  by taking a root with arcs to the roots of rooted directed trees  $S_1, S_2, \dots, S_k$ , then the depth of  $T$  is 1 plus the largest depth of any of the  $S_i$ 's. (This is because the longest path in  $T$  must take an arc to the root of one of the  $S_i$ 's, then take a longest path within  $S_i$ .)

Exercise 4.10.7 

```
public boolean contains (thing target) {
    if (isAtom) return equals (contents, target);
    return car().contains(target) || cdr().contains(target);}
```

Exercise 4.10.8 (a)  $4(2^3) - (2^2 + 2^2) = 32 - 8 = 24$

(b)  $(2 * 2) + (2 * 2) = 8$

(c)  $4(4 - 4 + 4) = 16$

(d)  $2^3 + 3(2^3) + 3(2^3) + 2^3 = 64$

(e)  $(2 + 2)(2 + 2)(2 + 2) = 64$

(f)  $1 - 2 - 2^2 - 2^3 - 2^4 = -29$

Exercise 4.10.9 For depth 0 we can only have 1. For depth 1,  $1 + 1$  has a larger value than  $1 \times 1$  so the answer is 2. For depth 2, we can either add or multiply two maximal expressions of depth 1, and either way we get 4. For higher depth, we want to multiply: depth 3 gives  $4 \times 4 = 16$ , depth 4 gives  $16 \times 16 = 256$ , and depth 5 gives  $256 \times 256 = 65536$ .

Exercise 4.10.10 (a) Could be either, for example  $1 + 1$  and  $1 + 1 + 1$ .

(b) Must be even. All constant expressions are even naturals, and if we have the sum of two even naturals the result is even.

(c) Must be odd. All constant expressions are odd naturals, and if we multiply two odd naturals the result is odd.

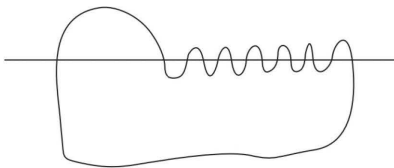
(d) As in (b), since the product of two even naturals is even, the result must be even.

Exercise 4.11.1 If 3 divides  $n$ , the  $2 \times n$  rectangle divides into  $2 \times 3$  rectangles, each of which can be covered by two L-shaped tiles. If 3 does not divide  $n$ , it does not divide  $2n$  either, and the  $2 \times n$  rectangle cannot possibly be covered by L-shaped tiles of size 3 each.

Exercise 4.11.2 For the base case, we have no cuts and one piece. For the induction, we assume that there is a way to make  $(n^2 + n + 2)/2$  pieces and no way to make more. We showed in the section that the  $n+1$ 'st cut could always be chosen to add up to  $n+1$  more pieces, but no more. The new maximum number of pieces is thus  $(n^2 + n + 2)/2 + (n + 1) = (n^2 + 3n + 4)/2 = ((n + 1)^2 + (n + 1) + 2)/2$ , as desired.

Exercise 4.11.3 Figure S-12 shows how one cut of a non-convex piece can yield three pieces. There is no limit to how many pieces could be produced if the boundary of the pizza is very wiggly.





©Kendall Hunt Publishing Company

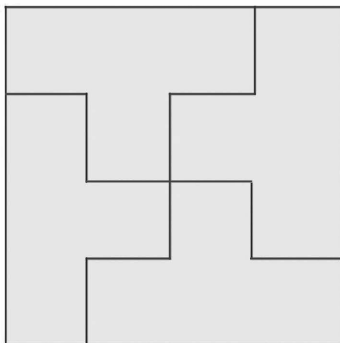
Figure S-12: Cutting a non-convex pizza.

Exercise 4.11.4 Clearly with no lines we have one region, so the base case holds. Each new line intersects each of the old lines exactly once, so it passed through exactly  $n + 1$  old regions (as we discussed in the section) and thus adds exactly  $n + 1$  to the number of regions. By the arithmetic in Exercise 4.11.2, the solution is  $(n^2 + n + 2)/2$  regions for  $n$  lines.

Exercise 4.11.5 For  $n = 0$ , we have that  $1 \neq 0$  as desired. For  $n = 1$ ,  $2 \neq 1$  so the desired statement is false. For  $n = 2$ ,  $3 \neq 1$  as desired. For larger  $n$ , we know that  $F(n) < F(n + 1)$ , so the equation  $F(n + 2) = F(n + 1) + F(n)$  tells us that  $F(n + 2) > F(n + 1)$  as desired. For  $n = 0$  the Euclidean Algorithm takes no steps, for  $n = 1$  it takes one, and for  $n = 2$  it takes one. For  $n = 3$  it takes two, for  $n = 4$  three, and in general for  $n \geq 2$  it takes  $n - 1$ . To prove this, we take  $n = 2$  as our base case and prove the induction by referring to the first half of this exercise: On input  $F(n + 2)$  and  $F(n + 1)$ , we do one division and are left with  $F(n + 1)$  and  $F(n)$ , which by the inductive hypothesis take  $n - 1$  more steps. Thus we have  $n = (n + 1) - 1$  total steps, and the inductive step is complete.

Exercise 4.11.6 Let  $t(n)$  be the number of tilings. The empty tiling makes  $t(0) = 1$ , and clearly  $t(1) = 1$  as well. If we have a  $2 \times n$  rectangle for  $n > 1$ , we can tile it by either (a) placing a domino vertically at the right end, then tiling the remaining  $2 \times (n - 1)$  rectangle in one of  $t(n - 1)$  ways, or (b) placing two dominoes horizontally at the right end, then tiling the remaining  $2 \times (n - 2)$  rectangle in one of  $t(n - 2)$  ways. Hence  $t(n) = t(n - 1) + t(n - 2)$  and we have the Fibonacci recurrence, though with different starting conditions. We can see that  $t(0) = t(1) = 1$  gives us the two base cases for an inductive proof that  $t(n) = F(n + 1)$ , where  $F(n)$  is the Fibonacci function from Excursion 4.5.

Exercise 4.11.7 We can easily define a bijection from the perfect matchings of the grid graph to the tilings of the rectangle by dominoes. Given a matching, we place a domino over each edge of the matching, so each endpoint of the edge is covered by one of the squares of the domino. Since the matching is perfect, each square of the rectangle is covered exactly once, and we have a tiling. Given a tiling of the rectangle, we create a grid graph by placing a node at the center of each of the  $2n$  squares of the rectangle and placing an edge between nodes that are adjacent horizontally or vertically. Then each domino in the tiling corresponds to an edge between the two nodes in the center of the domino's two squares. The edges for the tiling dominoes include each vertex of the grid graph exactly once as an endpoint, since the tiling includes each square exactly once.



©Kendall Hunt Publishing Company

Figure S-13: Tiling a  $4 \times 4$  square with T tetrominos.

So the edges form a perfect matching. Since each tiling corresponds to a different matching and vice versa, we have a bijection and the number of each is the same. Exercise 4.11.6 gives the number of tilings.

- Exercise 4.11.8 (a) Figure S-13 shows a tiling of the  $4 \times 4$  square with four T tetrominos. If 4 divides  $n$ , we can divide a  $4 \times n$  rectangle into  $n/4$  such squares and tile each one separately.
- (b) Clearly any tiling of a  $4 \times n$  rectangle will involve exactly  $n$  tetrominos. If we color the squares of the rectangle as suggested, each T tetromino will have three squares of one color and one of the other. If there are  $k$  tetrominos with three black squares, we will have a total of  $3k + 1(n - k)$  black squares. If  $n$  is odd, this number must be odd, but the rectangle has  $2n$  black and  $2n$  white squares. (We've left open the question of whether the tiling is possible when  $n$  is even)

Exercise 4.11.9 We first prove that  $F(i)$  and  $F(i + 6)$  are always congruent modulo 4 – the stated result follows immediately from this by induction on  $k$ . Using the Fibonacci rule, we can compute over the integers that

$$F(i + 6) = F(i + 5) + F(i + 4) = 2F(i + 4) + F(i + 3) =$$

$$3F(i + 3) + 2F(i + 2) = 5F(i + 2) + 3F(i + 1) = 8F(i + 1) + 5F(i).$$

Modulo 4, this last expression is congruent to  $0F(i + 1) + 1F(i) = F(i)$ .

- Exercise 4.11.10 We'll assume without loss of generality that  $i \leq j$ . If  $i = 0$ ,  $2^i + 1 = 2$  divides itself, but fails to divide  $2^j + 1$  for all positive  $j$  because these numbers are odd. If  $i = 1$ , so that  $2^i + 1 = 3$  we can look at  $2^j + 1$  modulo 3 for all  $j$ . It starts at 2, and each time when we increase  $j$  by 1 we double the number and subtract 1. So 2 becomes  $2(2) - 1 = 3 = 0$ , and 0 becomes  $2(0) - 1 = 2$ , so we can prove by induction that  $(2^j + 1) \% 3$  is 0 for odd  $j$  and 2 for even  $j$ . So  $2^i + 1$  divides  $2^j + 1$  if and only if  $j$  is odd.

For larger  $i$  we again look at the periodic behavior of  $2^j + 1$  as  $j$  increases. We start at 2 for  $j = 0$ , and then run through 3, 5, 9, ... (the values of  $2^x + 1$ ) until we reach  $2^i + 1$  which is congruent to 0 modulo  $2^i + 1$ . We then get  $2(0) - 1$  which is congruent to  $2^i$ , then go through  $-3, -7, -15, \dots$  until we reach  $1 - 2^i$  which is congruent to 2 modulo  $2^i + 1$ , so that the process continues with period  $2i$ . Thus  $2^i + 1$  divides  $2^j + 1$  if and only if  $j = ik$  for some odd natural  $k$ .